Calculus Unraveled

Intuition, Proofs, and Python

Dr. Mike X Cohen

This page contains some important details about the book that basically no one reads but somehow is always in the first page.

0.1 Front matter

© Copyright 2025 Michael X Cohen.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage and retrieval system without the written permission of the author, except where permitted by law.

ISBN: 9798309397228, edition 1.

This book was written and formatted in LaTeX by Mike X Cohen.

0.2 Dedication

If you're reading this, then the book is dedicated to you. I wrote this book for *you*. Now turn the page and start learning calculus!

0.3 Forward

The past is immutable and the present is fleeting. Forward is the only direction.

Contents

0.1 Front matter 2
0.2 Dedication 2
0.3 Forward 2

1 Introduction to this book **13**
1.1 What is calculus and why learn it? 14
1.2 Why learn calculus from this book? 14
1.3 Target audience 15
1.4 Prerequisites 16
1.4.1 Brand new to Python? 17
1.5 Exercises 18
1.6 Using the code with this book 19
1.6.1 Code organization 21
1.6.2 Modifying and reposting my code 22
1.7 Online resources 23
1.8 AI assistance 24

2 Functions **27**
2.1 What is a function? 28
2.1.1 Formal definition 29
2.1.2 What is a variable? 31
2.1.3 Function outputs and accuracy 32
2.1.4 Examples of non-functions 33
2.1.5 Vertical and horizontal line tests 35
2.1.6 Functions in math vs. Python 37
2.1.7 f(x) vs. y 38
2.1.8 Is math notation necessary to define functions? 39
2.1.9 Conventions for writing functions 40
2.1.10 Only numbers? 41
2.2 Number fields 41
2.3 Domain, codomain, and range of a function 42

2.3.1 Restrictions on domains and ranges 45
2.3.2 Notations for domain and range 46
2.3.3 Determining the domain and range 47
2.3.4 Why do you need to know the domain and range? 49
2.4 Injective, surjective, and bijective 49
2.5 How to "understand" functions 52
2.5.1 Working with "weird" functions 56
2.6 Functions in `numpy` and `sympy` 57
2.6.1 From `sympy` to `numpy` 59
2.7 Exercises 61

3 Families of functions **69**
3.1 Families of functions 70
3.2 Linear vs. nonlinear functions 70
3.3 Polynomials 73
3.3.1 What do polynomials look like? 74
3.3.2 Math with polynomials 77
3.4 Rational functions 78
3.5 Natural exponential and log functions 79
3.5.1 Natural exponential 80
3.5.2 Natural log 82
3.6 Trigonometric functions 85
3.6.1 Trig functions as ratios 85
3.6.2 Trig functions as angles 86
3.6.3 Trig functions as wavey lines (time series) . 87
3.6.4 Trig identities 88
3.7 Piecewise functions 90
3.8 Continuity and discontinuities 92
3.8.1 Formal definition of continuity 94
3.8.2 Types of discontinuities 95
3.9 Composite functions 96
3.9.1 Commutativity of composite functions . . . 97
3.9.2 Multiple composite functions 98
3.9.3 Applications of composite functions 99
3.9.4 Composite functions in coding 100
3.10 Inverse functions 101
3.10.1 How to find the inverse of a function 103
3.10.2 Which functions have inverses? 104
3.10.3 Domain and range of inverse functions . . . 104

3.10.4 Inverse trig functions . . . 107
3.11 Function symmetry and parity . . . 108
3.11.1 Determining function parity . . . 110
3.11.2 Why is function symmetry important? . . . 114
3.12 Exercises . . . 115

4 Limits **131**
4.1 Zeno's paradox and limits . . . 132
4.2 Geometric intuition of limits . . . 133
4.3 Limits notation . . . 136
4.4 "Easy" limits by plugging in or factoring . . . 138
4.4.1 Factoring . . . 139
4.4.2 Limits and denominators . . . 140
4.5 One-sided limits and infinities . . . 140
4.6 Properties of limits . . . 142
4.7 Continuity and discontinuities, revisited . . . 144
4.8 Limits of trig functions, part 1 . . . 145
4.9 Squeeze theorem . . . 149
4.9.1 Three examples . . . 150
4.10 Limits of trig functions, part 2 . . . 152
4.11 Limits categories . . . 156
4.12 Exercises . . . 158

5 Fundamentals of differentiation **169**
5.1 Slope of a line . . . 170
5.2 Formal definition of derivative . . . 173
5.2.1 Example with a linear function . . . 175
5.2.2 Example with a quadratic function . . . 177
5.2.3 Proof: Derivative of a constant is 0 . . . 178
5.3 Various notations for the derivative . . . 179
5.3.1 Geometric terms . . . 181
5.4 Interpreting derivatives plots . . . 183
5.4.1 More examples . . . 185
5.5 Differentiation is linear . . . 187
5.5.1 Proof of linearity . . . 188
5.6 Continuity is necessary but not sufficient . . . 190
5.7 Derivatives of polynomials . . . 191
5.7.1 Derivative of square root . . . 194
5.8 Derivatives of cosine and sine . . . 195
5.8.1 Proving the cosine-sin cycle . . . 195

5.9 Derivatives of absolute value and signum 197
5.10 Derivatives of log and exp 199
5.10.1 Logs of various bases, and their derivatives 200
5.10.2 Derivative of e^x 201
5.11 Higher-order derivatives 202
5.12 Exercises 205

6 Critical and inflection points 217
6.1 Taxonomy of critical points 219
6.2 Analytic methods to find critical points 221
6.2.1 Via solving $f'(x) = 0$ 222
6.2.2 Via discontinuities 223
6.2.3 Endpoints of a restricted domain 224
6.2.4 Conclusions about finding critical points . . 225
6.3 Empirical methods to find critical points 225
6.3.1 Via visual inspection 226
6.3.2 Grid search 228
6.3.3 Gradient descent 231
6.4 The "first derivative test" 234
6.4.1 First derivative test 235
6.5 Examples of critical points 235
6.6 Inflection points 240
6.6.1 Identifying inflection points 242
6.6.2 Example of sigmoid function 242
6.7 Evaluating inflection points 244
6.8 Applications of critical points 245
6.8.1 Real-world applications of second derivatives 246
6.9 Exercises 247

7 Differentiation rules 257
7.1 Product rule 258
7.1.1 Intuition of the product rule 261
7.1.2 Proving the product rule 262
7.1.3 More than two functions 263
7.2 Chain rule 263
7.2.1 More than two functions 266
7.3 Quotient rule 266
7.3.1 Proving the quotient rule 268
7.4 Implicit differentiation 269
7.4.1 Implicit vs. explicit functions 270

7.4.2 How to differentiate implicitly 270
7.4.3 Implicit differentiation in `sympy` 277
7.5 Numerically solving implicit differentiation 278
7.5.1 Searching for solutions in `numpy` 279
7.5.2 Implicit plots in `sympy` 283
7.6 Implicit differentiation proofs (log, exp, power) . . . 283
7.6.1 Natural log 283
7.6.2 Natural exponential 284
7.6.3 Power rule 285
7.7 L'Hôpital's rule 286
7.7.1 Conditions for L'Hôpital's rule 290
7.8 Exercises 293

8 Differentiation theorems **301**
8.1 Differentiability implies continuity 302
8.1.1 Proof 302
8.1.2 Does continuity imply differentiability? . . . 304
8.2 Intermediate value theorem 304
8.3 Rolle's theorem 306
8.3.1 Example problem 307
8.3.2 Conditions for Rolle's theorem 309
8.4 Mean value theorem 310
8.4.1 What is the "mean value"? 313
8.5 The Fundamental Theorem of Calculus 313
8.5.1 Fundamental Theorem of Calculus, part 1 (FTC-1) 314
8.5.2 Fundamental theorem of calculus, part 2 (FTC-2) 314
8.6 Exercises 316

9 Applications of differentiation **323**
9.1 Linear approximations 324
9.1.1 Intuition of linear approximations 326
9.1.2 Approximation accuracy 327
9.1.3 A second example 328
9.2 Newton's method for finding roots 330
9.2.1 Newton's method: algorithm 332
9.2.2 Challenges and failures 333
9.3 Optimization 334

9.3.1 How to solve derivative-based optimization problems . . . 336
9.3.2 Example 1: Optimize for surface area . . . 339
9.3.3 Example 2: Optimize for volume . . . 342
9.4 Exercises . . . 345

10 Multivariable differentiation **351**
10.1 Two-variable functions . . . 353
10.1.1 Creating 2D functions in Python . . . 354
10.1.2 Visualizing 2D functions . . . 356
10.2 Limits of multivariable functions . . . 358
10.3 Partial derivatives . . . 362
10.3.1 Visualizing partial derivatives . . . 364
10.3.2 Interpreting partial derivatives . . . 365
10.3.3 Partial derivatives: formal definition . . . 366
10.4 Higher-order partial derivatives . . . 367
10.4.1 Symmetry of higher-order partial derivatives 369
10.4.2 Example . . . 371
10.5 Gradients and gradient fields . . . 372
10.5.1 Gradient fields . . . 373
10.5.2 Gradients and slopes . . . 377
10.5.3 Jacobian matrix for first-order partial derivatives . . . 377
10.5.4 Hessian matrix for second-order partial derivatives . . . 379
10.6 Gradient descent in 2D . . . 381
10.7 Exercises . . . 383

11 Make differential art! **397**
11.1 Elf hat . . . 398
11.2 Elf hat to infinity (and beyond) . . . 399
11.3 Acorn from Ooo . . . 399
11.4 Smooth gray waves . . . 401
11.5 Fun by diff . . . 402
11.6 Dancing petals . . . 403
11.7 Radial curves . . . 404
11.8 Rose curves . . . 405
11.9 Riemann's complex nondifferentiable ice cream . . . 406
11.10 Mandelbrot set . . . 408
11.11 The beginning . . . 409

12 Intuition about integration **411**
12.1 Integration as "inverse differentiation" 412
12.1.1 Cumulative summation 413
12.1.2 Approximate integration 414
12.2 Integration as geometric area 416
12.3 Integral as a function or number? 419
12.3.1 Two perspectives of integration 420
12.4 Terms and notations 421
12.5 Fundamental Theorem of Calculus, part 1 422
12.6 Fundamental Theorem of Calculus, part 2 423
12.7 Reversing the limits of integration 426
12.8 Integration is harder than differentiation 427
12.9 Exercises 429

13 Geometry of integration **435**
13.1 Riemann sums 436
13.1.1 Definition of Riemann sum 437
13.1.2 Riemann summation rules 439
13.1.3 Riemann integral 442
13.1.4 Why always the x-axis? 443
13.2 Net vs. total area 443
13.2.1 Net area 444
13.2.2 Total area 444
13.2.3 Calculating net and total area 445
13.3 Definite integrals in `sympy` 448
13.4 Lebesgue integrals 449
13.4.1 Partition the domain vs. the range 450
13.4.2 Riemann vs. Lebesgue comparisons 451
13.5 Exercises 453

14 Integrating functions **465**
14.1 The power rule for integration 466
14.1.1 Indefinite integrals 467
14.1.2 Definite integrals 469
14.2 The constant of integration 472
14.2.1 Fundamental Theorem of Calculus, with C 473
14.2.2 Geometry of C 474
14.2.3 Indeterminacy of indefinite integrals 475
14.2.4 Initial value problems 476
14.3 Linearity of integration 477

14.3.1 Proving the scalar multiplication property . 479
14.3.2 Proving the function addition property . . . 480
14.3.3 Examples 481
14.3.4 Mistakes to watch out for 482
14.3.5 Geometry of scalars in integration 483
14.4 Integrating transcendental functions 484
14.4.1 Integrating the natural exponential 484
14.4.2 Integrating $ln(x)$ 485
14.4.3 Integrating sin and cos 487
14.5 Product, quotient, and chain rules? 488
14.6 Calculating net and total area 489
14.6.1 Problem type 1: given integrals 490
14.6.2 Problem type 2: Given graph with roots . . 491
14.6.3 Problem type 3 493
14.7 Integrating even and odd functions 495
14.8 Numerical integration in `scipy` 496
14.9 Exercises 499

15 Improper integrals **511**
15.1 What are improper integrals? 512
15.2 One infinite integration bound 513
15.3 Convergence and divergence 516
15.3.1 The rule 517
15.3.2 Important caveat to this rule 518
15.3.3 Check the highest-power term 518
15.4 Two infinite bounds 519
15.5 Improper trig integrals 523
15.5.1 Definite integrals of sine and cosine 523
15.5.2 Gausian-tapered cosine 525
15.6 Functions with discontinuities 527
15.6.1 Jump discontinuity 527
15.6.2 Infinite discontinuity 529
15.6.3 Removable discontinuity 530
15.7 Exercises 532

16 Integration techniques and tricks **543**
16.1 U-substitution 544
16.2 Integration by parts 548
16.3 Partial fractions 553
16.4 Exercises 558

17 Integration applications in geometry **563**
17.1 Area between two curves 564
17.1.1 Area with predefined intervals 566
17.1.2 Area defined by function intersections . . . 569
17.2 Parametric curves 570
17.2.1 Examples of parametric curves 571
17.2.2 Applications of parametric curves 572
17.3 Arc (curve) length 573
17.3.1 Deriving and understanding the formula . . 573
17.4 Numerical arc length approximations 576
17.4.1 Direct implementation in `numpy` 578
17.4.2 Approximations using `scipy` 581
17.5 Volumes of solids of revolution 584
17.5.1 Visualizing volumes 587
17.5.2 The disk method 588
17.5.3 The washer method 593
17.5.4 Numerical approximations 596
17.6 Exercises 600

18 Integration and statistics **613**
18.1 Introduction to probability and statistics 614
18.1.1 What are distributions? 615
18.2 Probability density functions 616
18.2.1 Examples of pdfs 618
18.3 Cumulative distribution functions 623
18.3.1 Examples of cdfs 626
18.4 Application: pdf and cdf in the z-test 629
18.4.1 Probability of IQ scores in a certain range . 631
18.4.2 Statistical inference 633
18.5 Data sample distributions 634
18.6 Statistical moments 636
18.6.1 What are moments? 636
18.6.2 Calculating analytic and empirical moments 637
18.6.3 Moments of pdfs 638
18.6.4 Moments of numerical data 639
18.7 Exercises 641

19 Multivariable integration **649**
19.1 Terms and concepts 650
19.1.1 Terminology 650

19.1.2 Multivariable definite integrals and volume 651
19.2 Multivariable indefinite integrals 651
19.2.1 Constants of integration 653
19.3 Visualizing double partial antiderivatives 656
19.3.1 Using `sympy` 657
19.3.2 Using `scipy` 658
19.4 Multivariable definite integrals 663
19.5 Approximating definite integrals 664
19.5.1 Numerical approximations using `numpy` . . . 664
19.5.2 Numerical approximations using `scipy` . . . 666
19.6 Integration with variable limits 667
19.6.1 Numerical approximations 669
19.7 Exercises 673

Chapter 1

Introduction to this book

1.1 What is calculus and why learn it?

Calculus is the study of *change*. In particular, the idea of calculus is to use simple mathematical techniques (mostly differentiation and integration, which are just fancy ways of expressing subtraction and addition) to understand the complicated behavior of mathematical functions (a function is a set of rules that governs how numbers behave).

I learned, forgot, and re-learned calculus several times in my life. I consider this an advantage in teaching calculus, because each time I re-learned it, I saw calculus from a different perspective and with a new appreciation for its conceptual simplicity, beauty, and occasional surprising conclusions. I am reasonably confident that you will also learn and forget calculus (perhaps you already have). That's just how it goes.

There are many reasons why someone (e.g., *you*) would want to learn calculus. Here's a non-exhaustive list:

1. You want to understand other topics that have calculus as a prerequisite, such as physics, engineering, computational biology, finance, or myriad other technical subjects.
2. You are a practicing or aspiring data scientist and want a deeper understanding of statistical and machine-learning algorithms.
3. You like to challenge yourself and learn math as a hobby (yes, this really is true for lots of people!).
4. You don't actually *want* to learn calculus; it's required for your degree or educational program. There's nothing wrong with this motivation!

1.2 Why learn calculus from this book?

There are many calculus textbooks — a few of which are excellent, while many others are either impenetrable or are irrelevant to modern computer-based applications.

Here's the point: If you want a calculus book like any other calculus book, then buy any other calculus book.

I've tried to organize and write this book to be different and unique, to provide a learning approach and perspective that is rigorous but approachable, intuitive yet practical, blending theory with hands-on coding and problem-solving techniques. You will solve exercises, create high-quality graphics, and code algorithms. I believe this will help you understand modern, real-world applications of calculus without getting bogged down in endless repetitive equations to solve.

Most importantly, I use Python code to help you understand concepts in math, because *you can learn a lot of math with a bit of coding*. Many more people in the world have coding experience than have formal math training. In over 20 years of teaching, I have witnessed uncountable situations where learners can't make sense of equations — until they implement them in code, make graphs, change parameters, and solve code-based exercises.

1.3 Target audience

I envision three types of people who would benefit from this book.

1. Students using this book to help them learn calculus. Perhaps your assigned calculus book is overly stuffy or leaves everything "as an exercise to the reader."

2. Professionals in fields related to the quantitative sciences, including data science, engineering, physics, and computational biology, who want to understand the analyses they are applying, develop new algorithms, or advance in their

career by increasing their math and math-coding knowledge.

3. Math enthusiasts. You might be surprised how many people choose to learn math in their free time, simply as an enjoyable way to pass the time while increasing mental acuity.

1.4 Prerequisites

The obvious You need to *want* to learn calculus. You need an intention and a goal. Maybe that goal is to pass your university calculus exam, maybe that goal is to get a data-science job, maybe that goal is to understand machine-learning algorithms, or maybe your goal is simply to have an intellectually stimulating hobby. It doesn't matter what your goal is (it is, after all, *your* goal).

I hope that having an intention to learn is almost enough for you to succeed in learning calculus from this book. All the points below are minor in comparison to the importance of having a reason to make even a small but consistent effort to learn a new skill.

High-school math You need to be comfortable with arithmetic and basic algebra. Can you solve for x in $4x^2 = 9$? Then you have enough algebra knowledge to continue. Other concepts will be introduced as the need arises.

There are formulas, equations, and algorithms in this book, but I try to explain them in plain English, illustrate them with graphs and diagrams, and provide you with Python code so you can explore the math through simulations and visualizations.

Programming It is no understatement to write that modern applied mathematics relies 100% on programming. There is simply no way to implement math without knowing at least a little bit of coding.

Fortunately, there are well-developed coding libraries that implement the low-level details. This means that you don't need to be a professional programmer to understand all the code in this book. But you do need to be comfortable with coding.

In this book, I use Python, because it is one of the most popular languages for modern applied math (statistics, machine-learning, deep learning, etc). I'm pretty sure it won't *always* be so popular; some other language will be developed that is better, easier, and faster[1]. But the good news is that all programming languages share some similarities, so learning calculus in Python will help you apply math in any other language. In other words, learning coding is time *invested*, not time *wasted*, even if you use a different language in practice. Anyway, ChatGPT (or other advanced language AI) can translate Python code into R, MATLAB, Julia, or other languages with decent accuracy.

To be clear: You do not *need* any coding to work through this book. You can skip all the code and all the coding exercises and focus on the conceptual and interpretational topics. But I designed this book such that a deep understanding of the material comes from working through the code and code exercises.

People learn best by seeing and doing, rather than by letting their eyes bounce over words and equations. This is a guiding philosophy of my writing and my teaching.

1.4.1 Brand new to Python?

I assume some basic Python proficiency in this book. If you're brand-new to Python, then you should probably wait to delve into this book until after taking an intro-Python course or book. There are myriad free and paid resources available for learning Python.

I tried to keep the code fairly simple in this book, while being advanced enough to express the nuanced principles of calculus. The

[1] Julia Programming Language looks promising, for example.

point is to use Python as a tool to help you learn and understand calculus, not to use gratuitously fancy and dense code. And if you want to challenge yourself, then please embellish my code! You should see my code as a starting point for your own explorations and applications, not as immutable text with no room for customization.

1.5 Exercises

I am sure you have heard this before: *Math is not a spectator sport*. If you simply read this book without solving any exercises, then sure, you'll learn something and I hope you find the book useful. But to really understand calculus, you need to solve calculus problems.

I designed the exercises to require some effort and creativity. They can be solved only through coding. These are opportunities for you to explore concepts, visualizations, and parameters in ways that are difficult or impossible to do without code.

I very strongly encourage you to work through the exercises[2]. They are not just busy work; they are ways to solidify, explore, and expand your understanding of calculus in ways that are not possible from only reading the chapters. The exercises also provide a wealth of code that you can use to continue learning and applying calculus concepts.

I provide my code solutions to all exercises, but keep in mind that there are many correct coding solutions; the point is for you to explore and understand calculus using code, not to reproduce my code exactly.

Feel free to look at my code solutions as much as you like. It's not cheating! This is especially true if you are new to Python, and if you understand the calculus concepts but struggle with the coding

[2] I mean, like, really super-duper a lot encourage this.

syntax. The purpose of the exercises is to give you opportunities to learn, explore, and expand your knowledge; they are not meant to be quantitative assessments.

In addition to my coding solutions, I have also created online videos in which I explain the Python code and solutions. You can find those videos on my YouTube channel www.youtube.com/@mikexcohen1, or see the online code on github for a direct link.
`https://github.com/mikexcohen/calculus_book`

1.6 Using the code with this book

You can use any IDE that you find most comfortable. I wrote all the Python code using Google's Colab service, which is free and integrates well with their other products, including Google Drive. Therefore, I recommend using Colab to follow along in this book, especially if you are relatively new to Python. The main libraries I use are `numpy`, `scipy`, `sympy`, and `matplotlib`.

Getting the book code into Google Colab involves three steps: download the code from github, upload the code to Google Drive, open the code files in Google Colab.

Download the code The code for this book is available at https://github.com/mikexcohen/Calculus_book.
If you are comfortable with git, then you can clone this repository to sync the files locally. If you have no idea what that previous sentence means, then don't worry! You can get all the code without knowing anything about git, and without needing to log in, sign up, give an email address, pay, or anything else.

Simply go to that URL, look for the green button that says "Code", click on that button, and look for the link that says "Download zip" (see Figure 1.1A). This will download one zip file that contains all the code material that you will need for this

entire book.

Unpack that zip file on your computer.

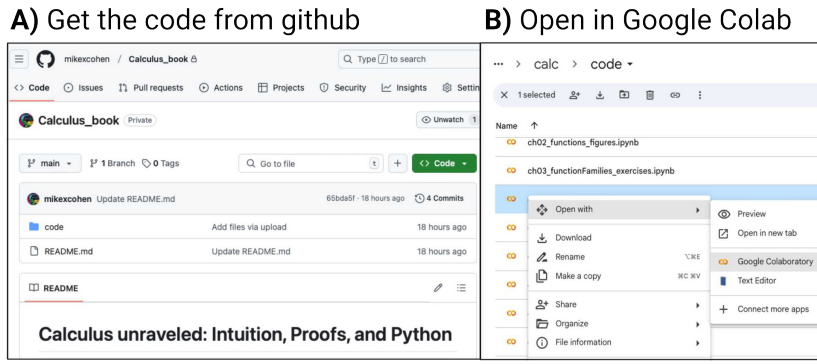


Figure 1.1: Screenshots of getting the book code from my github to your Google-colab. It might look slightly different on your computer.

Upload the code to Google Drive Google Drive is a free cloud-based storage service that Google provides to its users. You do not need to pay to use Drive, but you do need a Google account.

In a browser, go to drive.google.com (log in to your Google account if you're not already logged in). Create a folder for this book. Find the files you downloaded to your local computer, and simply select and drag them into the Drive folder to upload the files. Note that Drive does not unpack zip files, so you'll need to unzip the files on your local computer first.

Now those files are stored in the cloud. That's convenient because you can access them from any internet-connected computer. In fact, you can delete the files from your local computer if they bother you for some reason.

Open the files in Colab Select a Python notebook file (they end with extension ".ipynb"), and either double-click the file or right-click and select Open With, Google Colaboratory (Figure 1.1B). It will open a new tab with the notebook file. If you have absolutely no idea what you are looking at, then you need to learn some Python before continuing with this book. If you are

familiar with Python but are new to Colab, then you can consider watching a YouTube video about Colab to become familiar with the environment. But the Colab interface is easy and won't take long to master.

It is also possible to import the files directly from github onto Colab via the File, Open menu options. But this imports only one file at a time, so I think it's easier to upload them all at once to your Drive.

(A note on paying for Google services: You can pay to upgrade your Drive storage limit and/or to get better access to Colab servers. Paying for these services is absolutely not necessary for this book, but something you might consider if you anticipate heavy usage for other applications.)

I recommend creating a copy of my code to modify. That way, you have the original version of my code, and you can feel comfortable making whatever changes you like to your version. Of course, you can always download a fresh version from my github repository.

Using a different Python IDE Using Google's Colab service will help ensure that you can reproduce all the results and figures in this book. Therefore, I recommend using Colab.

You can use any other IDE, on a cloud or on your local computer. But keep in mind that some things in Python are IDE- and version-dependent. It is possible that you will need to make some modifications to my code, for example for visualizations and LaTeX formatting in `sympy` output. Please understand that I cannot provide detailed support for difficulties or errors encountered when using the code outside of Colab.

1.6.1 Code organization

Each chapter has two Python notebook files: one file to create the figures in the text, and one file that provides full solutions to all

of the exercises. (Some figures appear in the text but are created as part of the exercises; these instances are indicated in the code files with comments.)

The file naming is by chapter number, chapter label, and file contents. For example, here are the two file names for Chapter 4:
`ch04_limits_figures.ipynb`
`ch04_limits_exercises.ipynb`

Within each file, there is a subheading for each figure or exercise. A few figures in the book were made in Inkscape; these do not have corresponding code cells.

1.6.2 Modifying and reposting my code

Modifying my code Yes, please do! I very much hope that you see my code not as immutable text, but instead as a source of inspiration for you to modify, adapt, and explore.

Reposting my code I am occasionally asked whether I allow people to post my code, or modifications to it, on websites, blogs, github, or the like. My answer is *yes!* Absolutely, please feel free to use and share my code as you like. I ask only that you cite the source at the top of the code file. You can include the url to the book on github or the link to the book on amazon.com or wherever you purchased it. It can be something as simple as:

```
This code is modified from Mike X Cohen's book
on calculus; for code and links to the book, see
https://github.com/mikexcohen/calculus_book
```

Using other computer languages I have zero doubt that all exercises and visualizations can be solved using MATLAB, R, Julia, Mathematica, C++, or any other numerical processing language. If you want to use another language, then that's great! But I cannot support questions or difficulties for other languages.

If you would like to translate all of the book code into a different language, please contact me. I would be happy to link to your github page from mine. At the time of this writing, ChatGPT is impressive, but imperfect in translating a large amount of code. Any AI-translated code would need to be thoroughly checked by a human expert.

1.7 Online resources

Online course This book is based on online courses that I created. The book and the courses are similar, but not redundant. You don't need to enroll in the online course to follow along with this book, or the other way around.

Some people prefer to learn from online videos while others prefer to learn from textbooks. I try to cater to both types of learners. Following both may be beneficial, but I want to make it clear that the two resources are independent, and it is not my intention to upsell you.

You can find a list of all my online courses at sincxpress.com.

This online course is separate from the free video explanations of the exercises that accompany this book. Those videos are designed for this book and are available on my YouTube channel or via a direct link from the github repository that contains the code for this book.

Searching for explanations Although I have tried to write this book as a self-contained resource to master calculus, it is naive to think that everyone will find it the perfect resource that I intend it to be. Everyone learns differently, and everyone has a different way of understanding mathematical concepts.

If you struggle to understand something, don't jump to the con-

clusion that you aren't smart enough to understand it; a simpler possibility is that the explanation I find intuitive is not the explanation that you find intuitive. I try to give several explanations of the same concepts, in hopes that you'll find traction with at least one of them. If none of those works for you, don't hesitate to search the Internet or other textbooks if you need different or alternative explanations.

Online code The book itself has only the occasional snippet of code. Instead, *all* of the code — including the code to create the figures in the book and the solutions to the exercises — is available at
`https://github.com/mikexcohen/calculus_book`

The best way to learn from this book is to have the code in front of you while reading the book. You can see how the statistics concepts are implemented and how I created the figures. You can adjust the code to explore the concepts, and you can compare your exercise solutions with mine.

I apologize for stating multiple times that the code is available online and not entirely printed in the book, but I have had quite a few experiences of people complaining that I don't make my code available or that my code is incomplete — even giving my books and courses poor ratings online. I'm not angry about it, because I know that those people were just so excited to learn that they rushed through the introductory material (can you blame them??).

1.8 AI assistance

I wrote this book in 2024, when large language models like ChatGPT were becoming very popular and very hyped. It is impossible to predict how technology will develop and impact life in the future; but deep, complex language models like ChatGPT, Claude,

and Gemini have the potential to have a significant impact on the way we write and learn.

I spent a lot of time exploring ChatGPT (models 3.5, and 4/o), and considering whether it could be useful for my books. ChatGPT is a remarkably good writer, but there were three limitations that prevented me from incorporating it into my writing (this book and my other books and courses): First, several explanations were incorrect, although the writing style was so fluid and authoritative that it would be easy to be fooled. Second, I found ChatGPT4's writing style to be stuffy, overly formal, and sesquipedalian[3], like a gifted teenager who consults his thesaurus too often. Third, ChatGPT often equivocates and hedges, refusing to state that something is correct or incorrect, right or wrong, or appropriate or inappropriate.

There were times that I struggled with phrasing a sentence, and which I asked ChatGPT4 to rewrite. In none of those cases did I take ChatGPT's suggestions verbatim (I have tried and failed repeatedly to get ChatGPT to emulate my writing style), but in all of those cases ChatGPT helped me improve the readability of those sentences.

Most of the AI assistance in creating this book came from me asking ChatGPT to help implement some aspects of coding, including details of visualizations, debugging, and styling LaTeX. Interestingly, ChatGPT often failed to provide a correct solution, but usually gave good tips that helped me find the right solution.

Anyway, the point is that on the cusp of what may be the "era of AI writing," I can assure you that this book is fully human-written, with the occasional assistance from ChatGPT-4 to smooth out clunky sentences and help with coding issues.

[3]Yes, I had to look up this word; I love the meaning but never remember the word itself.

Chapter 2

Functions

2.1 What is a function?

You probably know, or at least have an intuition about, what a mathematical function is. Functions are the foundation of calculus (and, like, everything else in math), and so we need to start with a formal and rigorous definition, even if it's something you learned in elementary school.

I'll provide a formal definition in the next section, but first: have a look at a few enticing examples of functions to get you in the mood. If you're not familiar with some of the notations, then don't worry; you'll learn everything you need to know as you work through this book. The functions are visualized in Figure 2.1.

$$f_1(x) = 2x^2 \tag{2.1}$$

$$f_2(x) = x + 4\sqrt{\frac{|x^3|}{\pi}} \tag{2.2}$$

$$g(t) = \sin(2\pi t) \tag{2.3}$$

$$h(s) = \begin{cases} 0 & s \leq 0 \\ \log(s) & s > 0 \end{cases} \tag{2.4}$$

Functions are usually indicated using one italicized letter like f or g, and then the single-letter variable in parentheses. $f(x)$ can be read out loud as "the function f of x," "f is a function of x," or simply "f of x." Capital letters are allowed, but are generally reserved for integration and frequency-domain functions. For example, $F(x)$ might be the indefinite integral of $f(x)$. You'll learn about integration later in the book; for now, suffice it to say that common practice is to indicate functions using one lowercase italicized letter.

Using more than one letter risks confusing your audience. For example, does $ef(x)$ refer to one function $[ef](x)$ or is it the ir-

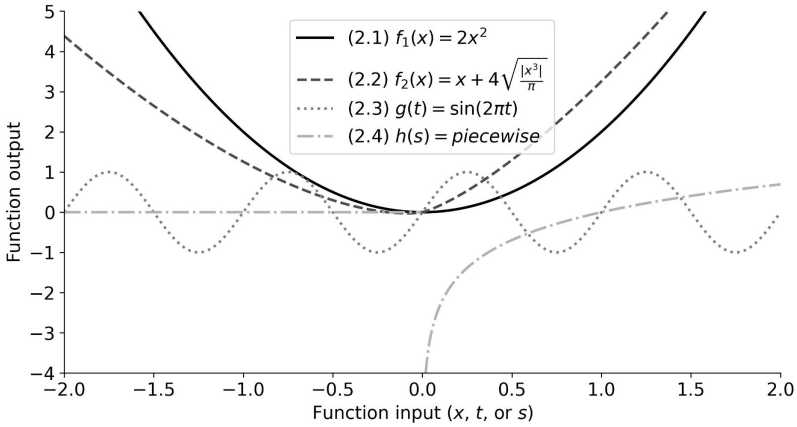


Figure 2.1: Visualization of Equations 2.1-2.4.

rational number e times a function $f(x)$? That said, there are exceptions to the single-letter rubric. For example, an important function in statistics is called the "error function" and is notated $erf(x)$. And of course we have the trig functions like $\sin(x)$ and the transcendental functions like $\ln(x)$.

$e \approx 2.178$. More on this in Chapter 3!

It is also common to use subscripts to differentiate functions, like $f_1(x)$ vs. $f_2(x)$, or perhaps $f_{car}(x)$ vs. $f_{truck}(x)$. Subscripting is useful when the functions are related to each other, for example different functions that use the same variables, or that characterize the same system.

Referring to functions using more than one non-superscripted letter is common in applied math and can facilitate clarity. For example, $\text{RMS}(x)$ indicates the function *root mean square*. In these cases, it's good practice to use all capital letters.

2.1.1 Formal definition

In mathematics, a *function is a relation between inputs and outputs, where each input is related to exactly one output.*

That definition has several terms and concepts; let's discuss each in more detail:

- **Relation**: Philosophers have heated debates about how to define a *relation*. And many people build their entire lives around their familial, friendship, romantic, and work relations. In the context of mathematical functions, *a relation is a set of ordered pairs that follows some rule*. For example, the relation *less than* includes the ordered pairs $(3, 4)$ and $(\pi, 10)$ but does not include the ordered pairs $(4, 3)$ or $(-e, -5e)$. You can think of *relation* as the rules that describe how pairs of numbers relate to each other — that's a bit circular but perhaps you find it useful. Relations can also act as mappings or transformations.

  The typical arithmetic functions are relations. For example, "addition" is the relation between the pair (2,3) and the number 5; "squaring" is the relation between the pair (3,9). "Equality" is the relation between the pair (17.4,17.4).

- **Inputs and outputs**: Imagine that a function is a machine; the inputs are what you put into the machine and the outputs are what the machine produces. In the example functions visualized in Figure 2.1, the inputs are the variables x, t, and s, and the outputs are the result of applying the relations specified on the right-hand-side of the equation definitions.

  Most of this book concerns functions with one input and one output. For example, $y = f(x)$ means there is one input (x) and one output (y). These are called *single-variable functions*. You will learn about *multivariable* functions (e.g., two inputs and one output: $Y = f(x_1, x_2)$) in Chapters 10 and 19.

- **Each input to one output**. This is an important part of the definition of a function. A given input can be transformed into only one output. For example, for some function $f(x)$, if $f(2) = 4$ then $f(2) \neq 5$ or any other possible value except 4. This constraint guarantees uniqueness.

  Why is this constraint necessary? Actually, it isn't strictly *necessary*; mathematicians could have allowed functions to have multiple distinct output values for a given input value. In other words, there is no fundamental law of the universe

that dictates this constraint. But including this constraint facilitates interpreting, working with, and applying functions.

Note that the opposite of this constraint is not true: many inputs can map onto the same output value. For example, the function $f(x) = 7.17$ maps every possible value of x onto the number 7.17. Perhaps you think that's a weird function, but it is a perfectly valid function. Another function is the trigonometric sine function: $\sin(0)$ and $\sin(2\pi)$ produce identical output although the inputs are distinct (in fact, $\sin(2\pi k) = 0$ for every integer value of k).

2.1.2 What is a variable?

What is the x in $f(x)$? There are two ways to think about the input variable:

- **A placeholder for any valid input number**. Here you think of the variable as a stand-in for a large set of numbers. For example, in the function $f(x) = x^2$, setting $x = 2$ gives $f(2) = 4$ and setting $x = -1$ gives $f(-1) = 1$. In this interpretation, the function describes a transformation and the variable is the value to which that transformation applies.
- **A point on a grid to evaluate the function**. Imagine a curve that is drawn on top of a horizontal axis. That curve is described by the function and the horizontal axis is the "grid" over which the function is evaluated. In abstract math, that grid has infinitesimal resolution, while in digitized and applied math, the resolution is determined by the measurements, the computer, or your Python code.

  In Figure 2.2, for example, the thin vertical lines indicate values of input variable x and the height of the boxes correspond to the value of the function for each evaluated x-axis point. You can imagine that in the world of abstract mathematics, the spacing between grid points is infinitesimal.

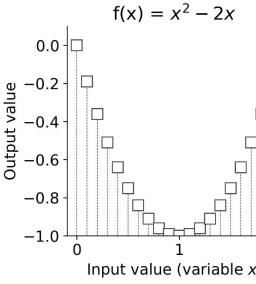


Figure 2.2: Visualizing a variable as the x-axis grid upon which to evaluate a function.

In abstract math, the variable need not have an inherent physical

meaning; it is simply a placeholder for any possible number that is valid for that function. In many functions, that variable can take on any numerical value with any arbitrary precision. Other functions have constraints on the values of the variable. This is called the "domain" of the function and you'll learn about this in Section 2.3.

In applied math, the variable is some physical quantity or dimension upon which the function is evaluated. For example, a function $f(t)$ might have *time* as the variable with *seconds* as units; perhaps the function describes the dynamics of radioactive decay, voltage fluctuations in an electrical signal, or heat loss in a cappuccino at an outdoor cafe in Bucharest in May. The variable might also be space; for example, a function $f(s)$ that describes radiation levels as a function of distance from Marie Curie's grave, where s has units of meters.

Input variables can also be called the *independent variable*, while the output variable can be called the *dependent variable*. That is, in the function $y = f(x)$, x is the independent variable and y is the dependent variable. They are given these names because you can specify the value of x, whereas the value of y is determined by the function. These terms are common in statistics.

2.1.3 Function outputs and accuracy

Consider the following two functions:

$$f(x) = 2x \tag{2.5}$$

$$g(x) = \sqrt{x} \tag{2.6}$$

When setting $x = 4$, $f(4) = 8$ and $g(4) = 2$. But when setting $x = 2$, $f(2) = 4$ and $g(2) = ...$ what exactly is the output of $g(2)$? I could write that it equals $\sqrt{2}$, but is that actually an answer, or is that just rewriting the function definition? Perhaps you're

expecting an answer like 1.41421. But that is not the *exact* correct answer; it is merely an approximation. In other words:

$$g(2) = \sqrt{2}$$

$$g(2) \neq 1.41421$$

$$g(2) \approx 1.41421$$

$\neq$ means "not equal to" and $\approx$ means "approximately equal to."

Perhaps you think this is a trivial point to make, but it is profound. Outputs of functions can be irrational numbers, which means they go on and on forever. You cannot write out the true numerical result of $g(2)$, although you can write out an approximation.

Does that matter? It depends on what the function means and what application, if any, it is used for. In the context of pure math, the result 1.41421 is simply incorrect. In the context of applied math, it might be precise *enough*. For example, if that number corresponds to the average number of kilograms lost in a group of 20 participants in a diet study, then including five numbers after the decimal point is meaningless, because the variability across participants is likely to be so large as to give that value a false sense of precision. On the other hand, if that number corresponds to a tolerance in cm of a screw that holds an airplane door in place, then perhaps five numbers after the decimal point is not enough precision to guarantee safety.

In other words, the accuracy of numerical approximations is not merely academic; understanding the limitations of how pure math is translated into applied math has real — and potentially serious — implications for how math is used in engineering (think of safety guarantees for cars and buildings, GPS localization accuracy, stock prices...).

2.1.4 Examples of non-functions

Functions are great, but they're not the only gifts that mathematics has bestowed upon us; many expressions may look and behave

like functions without being functions. Here are a few examples; they are visually depicted in Figure 2.3.

Plus-minus Consider the expression $y = \pm x$. This is a perfectly valid mathematical expression. But it is not a *function* because it violates the constraint of one output to a unique input. For example, when $x = 4$ then $y = -4$ *and* $y = +4$.

Square root A commonly discussed non-function is the square root operation $y = \sqrt{x}$. This relation seems to violate the unique-output constraint of a function, because, for example, $\sqrt{4} = 2$ *and* $\sqrt{4} = -2$. However, because it is convenient to use the square-root operator in functions, we can add the assumption that $f(x) = \sqrt{x}$ refers to the *principal* square root, that is, the positive square root. When the negative square root is needed, you can define separate functions for the negative and positive square roots. That is:

$$\begin{aligned} f(x) = \pm\sqrt{x} \quad &\rightarrow \quad \text{Not a function} \\ f_p(x) = +\sqrt{x} \quad &\rightarrow \quad \text{Yes a function} \\ f_m(x) = -\sqrt{x} \quad &\rightarrow \quad \text{Yes a function} \end{aligned}$$

"Unit circle" means that the radius is 1.

Circle The formula that describes all points on a unit circle centered at the origin is $1 = x^2 + y^2$. If you solve for y, you get a square root function; the principal square root is the upper-half of the circle and the negative square root is the lower-half of the circle. As with the square root example, you can express a circle as a function by considering the union of two functions.

There is an uncountable number of mathematical expressions that look and act like functions but are not formally functions — or that could be functions for certain values of x but not for others. Non-functions can be called relations, expressions, or equations.

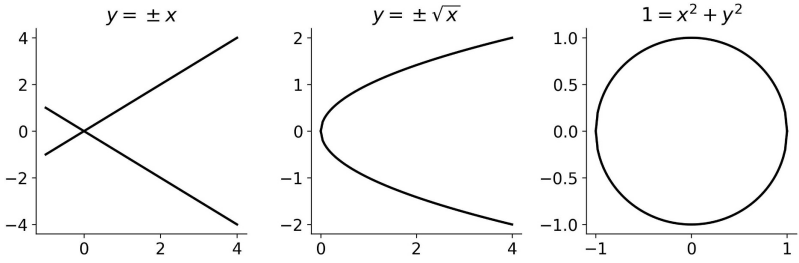


Figure 2.3: Visualization of three non-function expressions.

My advice: Don't get hung up on the technicalities of the terminology; focus on what you can do with mathematical expressions and how you can implement them on computers.

2.1.5 Vertical and horizontal line tests

There are two visual tests that you can use to help you identify functions and properties of functions.

Vertical line test The vertical line test is a graphical rubric for identifying a function. The idea is simple: Imagine a vertical line at any given x-axis value; if that vertical line passes through the function more than once, then the graph is *not* defined by a function.

Have a glance back at Figure 2.3; all three expressions fail the vertical line test, because all expressions have at least one x-axis value where a vertical line would pass through two points on the curve.

Horizontal line test The horizontal line test is used to determine whether a function has multiple mappings between input and output. Imagine a horizontal line on the graph; if that horizontal line passes through more than one point, it is not possible to determine a unique value of x for a given value of y.

Failing the horizontal line test does not mean the graph is not a

function; many valid functions fail the horizontal line test. Instead, the horizontal line test will help you categorize function behaviors. I'll get back to these categorizations later in this chapter.

Examples Consider the graphs shown in Figure 2.4. Panel A depicts an expression that passes the horizontal line test (there is no value of y for which there are multiple values of x) but fails the vertical line test (there are multiple values of y for certain values of x). Panel B passes the vertical line test but fails the horizontal line test. Finally, panel C passes both tests.

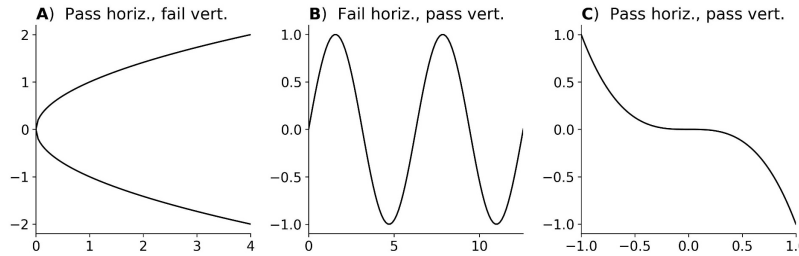


Figure 2.4: Illustrations of passing and failing the horizontal (horiz.) and vertical (vert.) line tests.

From these tests, we can conclude that the curve drawn in panel A does not correspond to a function. Panels B and C do depict functions, and the horizontal line provides important information about the function that I will introduce later in this chapter, using terms like injective and surjective. For now, I'd like you to focus on the idea of using the vertical and horizontal line tests to determine whether the function passes through two or more points vertically and horizontally.

Disclaimer Keep in mind that although the vertical line test can definitively prove that a curve is *not* a function, the converse is not true: You cannot prove that a curve depicts a function simply because it passes the vertical line test; the curve could fail the vertical line test for x-axis values that are not shown in the plot.

2.1.6 Functions in math vs. Python

Inconsistent or confusing terminology is a major pain when learning any new field. This is true in math, and it's also true in science, engineering, politics, marketing, social media apps, TV series, awkward first dates, and basically everything that humans use language for.

In the context of using coding to learn math, there is inconsistent terminology where the same word means different things in computer science vs. math. There is nothing to be done about this inconsistency: you must accept it with zen-like grace and compassion.

I will introduce and discuss terminological confusions between coding and math as they arise throughout this book. The first inconsistency to discuss is the word *function*. I just described the math definition of a function; the coding definition of a function is *a sequence of commands that can be reused*. Coding functions often have inputs and provide outputs, but they don't need to have either, and there is no constraint about the uniqueness of inputs to outputs.

On the other hand, functions in code and functions in math can be the same, for example, if you write a Python function that implements a math function. Consider, for example, the following two lines:

$f(x) = 2(x-3)^4$

```
def fx(x): return 2*(x-3)**4
```

The first is a math function whereas the second is a Python function. Are they the same? If you want to get nerdy and technical, then *no*, they are not the same: The Python function is limited to numbers that digital computers are capable of representing, which is a subset of all possible numbers that can be used in the math function. For this reason, the function output might be incorrect

for values of the function that have higher precision than what modern computers are capable of representing.

But for practical purposes in this book and in many real-world applications, you can consider the math function and the Python function to be identical — or at least, a close enough match that their differences are negligible.

I will discuss and show examples throughout this book where finite computer precision impacts the accuracy with which functions can be implemented on computers — and how you can use the `sympy` and `scipy` libraries to combat this issue. The main points to appreciate for now are that the word *function* means something different in math vs. computer science, and that code functions can be crafted to implement math functions.

An example that I'll discuss later is π vs. `np.pi`.

2.1.7 f(x) vs. y

Consider the following two expressions:

$$f(x) = x + 7 - x\pi \tag{2.7}$$

$$y = x + 7 - x\pi \tag{2.8}$$

Are those two equations the same? The right-hand-side of the equations are identical, so does that mean that $y = f(x)$?

$f(x)$ and y actually indicate different things. $f(x)$ refers to a function — that is, a set of rules for mapping x onto another number as specified by the transformation on the right-hand-side of the equation — whereas y refers to a specific value of the function for a particular (though here unspecified) value of x. So, $f(x)$ defines a relation and y is the result of applying that relation to a particular set of input values.

Figure 2.5 shows a conceptual representation of this: Imagine the gray squares are y whereas the black line is $f(x)$. y is a set of dis-

crete points at which the function is evaluated for corresponding values of x, whereas $f(x)$ is the mathematical relation through which an infinite number of points can be transformed (of course, the number of points in the figure is limited by the resolution of your reading device or paper, but I think you get the idea).

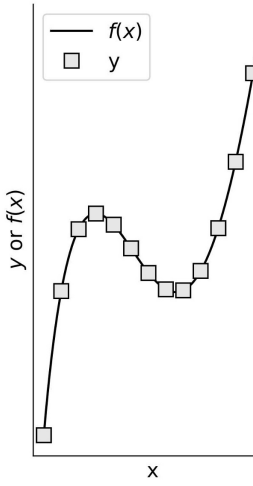


Figure 2.5: Depicting the distinction between $f(x)$ and $y = f(x)$.

You can also think about the distinction between y and $f(x)$ in terms of code. Going back to the Python function:

```
def fx(x):  return 2*(x-3)**4
```

You can consider that code `2*(x-3)**4` corresponds to $f(x)$ whereas the specific numerical outputs for specific numerical inputs correspond to y.

The distinction between $f(x)$ and y is subtle but quite important for implementing math in code: if the resolution of x is too low (that is, too few x values), you may come to incorrect conclusions about the behavior of the function. You'll see a few examples of this in the exercises for this chapter, and in several other places throughout the rest of this book.

That said, in practice, $f(x)$ and y are sometimes used interchangeably, and there are situations where using y instead of $f(x)$ facilitates understanding a particular concept or proof.

2.1.8 Is math notation necessary to define functions?

The answer is *no*. Consider this definition:

> Bidh an gnìomh a' gabhail àireamh dearbhach sam bith mar chur-a-steach, agus a' toirt a-mach co-mheas na h-àireimh sin agus aon ris a' chiùb aige.[1]

Unless you can read Scottish Gaelic, you probably have absolutely no idea what that function does. Now consider this definition:

$$f(x) = \frac{x+1}{x^3} \qquad x \in \mathbb{R}, x > 0 \tag{2.9}$$

[1] Translated using Google.

I'm sure you get the idea: using math notation helps make the function definition rigorous and precise, and it provides a common notation that can be understood by anyone from any spoken language.

That said, providing additional linguistic explanations or graphs can be an excellent way to help your audience understand the function.

It is also possible to represent functions using tables or visual diagrams. Maybe a clever dancer can represent a function using an interpretive dance choreographed to a Nine Inch Nails song. But I will follow common math convention and use math notation, and I recommend you do as well.

2.1.9 Conventions for writing functions

Left- and right-hand-sides are sometimes abbreviated LHS and RHS. I prefer to avoid those abbreviations, but you might encounter them elsewhere.

The common convention for defining a function is to have the function name on the left-hand-side of the equation and the definition on the right-hand-side, like this:

$$y = 5x + 3\sqrt{x + x^3}$$

That convention is used simply because it makes the equation visually clean and organized. But it can also be turned around:

$$5x + 3\sqrt{x + x^3} = y$$

Or some variables might appear on both sides:

$$-5x = -y + 3\sqrt{x + x^3}$$

When in doubt, take a moment to rearrange the expression to conform to the common convention of the output on the left and the definition on the right.

There are other conventions for displaying functions that you'll learn about throughout this book. Another notation, for example, is $f : A \to B$ to indicate that a function f transforms A into B. That notation generally indicates a relation without specifying the nature of the rule or transformation that brings A to B.

Keep in mind that conventions are not iron-clad requirements; they are generally agreed-upon styles that facilitate communication and comprehension. You should apply conventions when possible and break convention when justified.

2.1.10 Only numbers?

All the functions I've discussed so far have numeric inputs and numeric outputs. Functions can also have non-numeric inputs or outputs. For example, imagine a dessert-recommending function that takes the outside temperature as input and provides as output the words "hot chocolate" or "ice cream." Or a function learned by a computer vision model that takes a set of numerical input values corresponding to a picture and outputs a category label such as "cat" or "truck." Or a function that takes a color as input and outputs a food that matches that color.

As long as the inputs, outputs, and transformations conform to the definition of a function, that function can have any type of input and output.

All that said, for the rest of this book, we will only work with functions that take numbers as inputs and provide numbers as outputs.

2.2 Number fields

A *field* in math is a set of numbers that you can do basic arithmetic with. More formally, a field is a set F equipped with the operations "addition" and "multiplication" such that:

1. You can add, subtract, multiply, and divide (except by zero) without leaving F.
2. Arithmetic laws such as commutativity, associativity, dis-

tributivity hold in F.

3. There is an "additive identity" (0) and a "multiplicative identity" (1). Thus, $x + 0 = x$ and $x \times 1 = x$.

There are many fields in mathematics that you are already familiar with, even if you don't know that they are called fields. For example, there is the field of real numbers (indicated as $\mathbb{R}$), the field of complex numbers ($\mathbb{C}$), and the field of rational numbers ($\mathbb{Q}$).

Here is the crux of this discussion: In this book, we will only consider the set of real numbers. Well, mostly. There are a few exercises that use complex numbers, but that's because many complex functions look beautiful when visualized, and beautiful things increase joy in life and motivation to learn math.

In code, you will mostly use numbers that are of data type `float` and `int`. The `float` data type roughly corresponds to the set of real numbers[2]. Using `int` data type is for practical purposes of indexing, and therefore roughly corresponds to natural numbers (0, 1, 2, ..., indicated $\mathbb{N}$). Technically, the natural numbers are not a *field*, but that's a detail I recommend not worrying about.

To indicate a variable corresponding to a real number, you use the notation $x \in \mathbb{R}$. You can read this out loud as "x is an element of the set of real numbers," "x is contained in the real numbers," "x is in the reals," or simply "x in R." This notation is relevant for indicating the domain and range of a function, as you will learn in the next section.

2.3 Domain, codomain, and range of a function

These three terms are important yet simple to understand:

[2] Only "roughly" because finite precision prevents digital computers from representing all real numbers.

Domain

The set of all possible input values x. The domain is indicated using the boundaries for which the input into the function is valid. For example, the domain of the function $f(x) = \sqrt{x}$ has boundaries of zero and ∞.

Codomain

The codomain is the set of all possible outputs of a function, even if the function definition cannot lead to all of those values. Very often in numerical functions, the codomain and the domain are the same — that is the case for any function that takes in a number and outputs a number. But imagine a function that takes a number as input and returns a color label as output. That function has all real-valued numbers as the domain, and all colors as the codomain.

Most of the time, the domain and range are important to identify; I define the codomain here only for completeness.

Range

The set of all possible output values y, which is some subset of the codomain.

A mnemonic: x comes before y in the alphabet, and d before r. So, ***d****omain* refers to x while ***r****ange* refers to y.

The difference between codomain and range can be confusing at first. Here are two examples that I hope will clarify the distinction: (**1**) In the function $f(x) = |x|$, the domain and codomain are all real-valued numbers while the range is all non-negative numbers. (**2**) Imagine a deep-learning model that transforms a picture (represented as a matrix of numbers) into a color label. The codomain is all color labels, but the range is the set of seven colors that the model has been trained to identify.

As an example, consider the function $f(x) = x^2$, illustrated in Figure 2.6. Before reading the next paragraph, determine the domain and range of this function. I will introduce the formal notation options for domain and range later in this section, so for now you can just describe the domain and range using words.

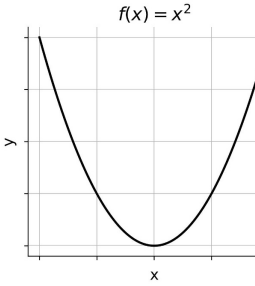


Figure 2.6: Determine the domain and range of this function.

The domain is the set of all x values that are allowed in this function, which is all possible real numbers. So the domain is all real numbers between $-\infty$ and $+\infty$, which we can write as $x \in \mathbb{R}$.

How about the range? Well, any values of x^2 will be non-negative (zero or positive). It is not possible to square a real-valued number and get a negative result, so all negative numbers are excluded from the range[3]. So the range of this function is 0 to $+\infty$.

Figure 2.7 shows another example. The domain and range of the function $f(x) = \sqrt{x}$ are both 0 to ∞. The domain of a function corresponds to all possible values on the x-axis that are valid inputs to the function (horizontal arrow below the x-axis), and the range is all possible output values (vertical arrow left of the y-axis). *Warning:* Do not be confused by the axis boundaries in the plot: Although there are some cases in which it is possible to visualize the entire domain in a plot, never assume that the *plot boundaries* correspond to the *function boundaries* (domain and range).

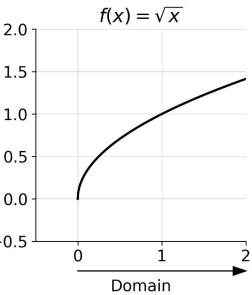


Figure 2.7: Visual indication of the domain and range.

Figure 2.8 shows abstract schematics that describe domains and ranges of four functions. Each of four connected diagrams depicts a function, with inputs in the left squircle[4], the function definition in the right-facing arrow, and outputs in the right squircle. The dashed line indicates all real numbers ($\mathbb{R}$; that is, imagine that the field of all real numbers is contained in that squircle), and the shaded region indicates the portion of the real numbers comprising the domain ($D(f)$) and range ($R(f)$) of the function. The codomain is the dashed squircle on the right; notice that the range is a subset of the codomain.

For the function $f(x) = x^2$ (top-left of the figure), the domain is all real numbers because all real numbers are valid values of x, and the range is a subset of the reals because all possible outputs are non-negative. The function $f(x) = \pi$ (top right) maps all real numbers onto exactly one value (technically the range is one infinitesimal point, but I drew it as a small circle for visibility). The lower-left of Figure 2.8 shows an example of a function for which the domain and the range are subsets of $\mathbb{R}$. Finally, the lower-right of the figure depicts the idea that the domain and range can be non-consecutive subsets of numbers. This could be

[3] Of course, $x^2 < 0$ is possible with complex numbers, but we're considering only real-valued numbers in this book.

[4] A squircle is a square with rounded edges.

the case for nonlinear functions such as piecewise functions.

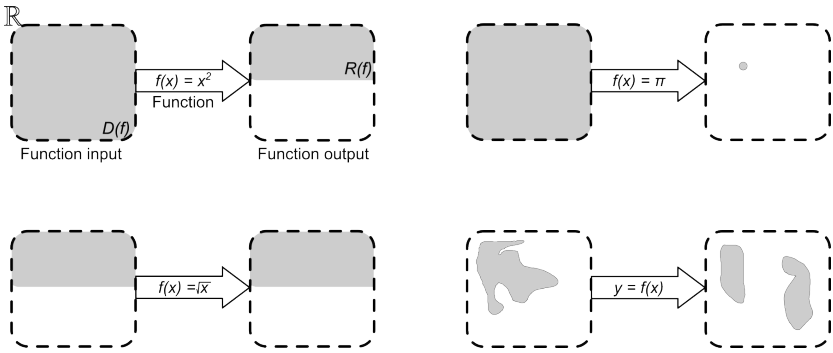


Figure 2.8: Abstract representations of domains and ranges.

2.3.1 Restrictions on domains and ranges

The domain of $f(x) = x^2$ is all of $\mathbb{R}$, but we can impose a *restriction* on the domain to, for example, all numbers between .1 and 2.4. This "subdomain" is called a restricted domain or interval, and the corresponding restricted range would be all numbers between $.1^2$ and 2.4^2.

The boundary values of a restricted domain are called *boundaries*, *endpoints* or *interval endpoints*.

A function with domain restrictions imposed can be called a *restricted function*. Figure 2.9 shows an example of a function (thin gray line) and its restricted version (overlaying thick black line). The thick lines next to the x- and y-axes indicate the restricted domain and range.

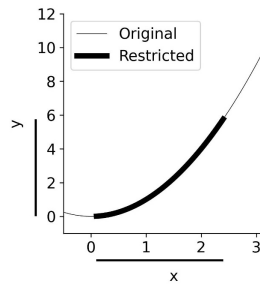


Figure 2.9: A function and its restricted version.

Restrictions are placed on the domain of a function to make the function be invertible (you'll learn about this in Chapter 3), or to make it relevant for real-world applications (e.g., a function that describes weight might be restricted to positive numbers).

2.3.2 Notations for domain and range

There are three notations for describing the domain and range of a function. All three notations indicate the same thing; they are just different visual depictions. The options are shown in Figure 2.10 and described below, using the function $f(x) = x^2$ as an example.

Interval	Inequality	Set-builder
$D = (-\infty, +\infty)$	$-\infty < x < \infty$	$D : \{ x \mid x \in \mathbb{R} \}$
$R = [0, \infty)$	$y \geq 0$	$R : \{ y \mid y \geq 0 \}$

Figure 2.10: Three notations to indicate the domain and range of the function $f(x) = x^2$.

Interval notation

Here you list the domain (D) and range (R) as equations. Notice the two types of brackets: the square brackets [and] are *inclusive* bounds, whereas the parentheses (and) are *exclusive* bounds. What do those terms mean? A bound is *in*clusive when the exact number is part of the domain. For example, in the function $f(x) = \sqrt{x}$, the domain *in*cludes $x = 0$ because $f(0)$ is a valid function value. In contrast, in the function $f(x) = \ln(x)$, the domain *ex*cludes $x = 0$ because the natural log function is defined for any real number arbitrarily close to 0, but not exact 0. Therefore, the domain for the logarithm function is $D = (0, \infty)$.

The domain of $f(x) = x^2$ has exclusive bounds. Why is $\pm\infty$ excluded? Well, this gets a bit into number theory, but ∞ is not considered to be a real "number." Therefore, the upper boundary gets arbitrarily far away from zero — arbitrarily close to infinity — but doesn't actually include infinity.

Note the difference between the symbol $\cup$ and the letter U.

If the domain contains multiple bounds, you can group them together using the "union" symbol from set notation. For example, a function that is undefined between 0 and 2 might have a domain like this:
$D = (-\infty, 0) \cup [2, \infty)$

Inequality notation

This notation might look more familiar to you. Compare how

the interval and inequality notations express inclusive bounds (parenthesis and less-than sign), and exclusive bounds (bracket and greater-than-or-equal-to sign).

Set-builder notation

If this notation is unfamiliar to you, then please take a moment to understand it, because it is used often in advanced mathematics and applications. A "set" is a collection of objects or expressions, and is indicated by the curly brackets. In set-builder notation, you first write the variable, then a vertical line, and then write the collection of all possible values of that variable. The vertical line is the "pipe" key on a keyboard, and in math it means "given that" or "for." The example shown in Figure 2.10 would be read out loud as "the domain is x, for x in all possible real-valued numbers." And the range would be read "the range is y, for all possible values greater than or equal to zero." (or: "for all non-negative values."). In this example, you could also clarify that y includes only real-valued numbers:

$R : \{y \mid y \in \mathbb{R}, y \geq 0\}$

When to use which notation? Because they all express identical concepts, it doesn't really matter. Some people have personal preferences, some publishers have in-house styles, and some disciplines have common conventions. Sometimes there are situations where one format is easier to read or looks nicer on the page. I will use all three notations in this book to help familiarize you, but in your writing and presentations, it's best to pick one format and use it consistently.

2.3.3 Determining the domain and range

There is no simple algorithm or formula that you could blindly apply to any function and accurately obtain the domain and range. Instead, you need to inspect the function — ideally both the equation and its graph, although the graph alone is usually insufficient — and use your math skills.

It's generally best to determine the domain first, and then determine the range.

Some functions are simple enough that you can determine the domain with a quick glance. For example: $f(x) = 2x$ or $f(x) = x^3 - 8x$.

For functions that have more complicated operators or multiple terms, look for terms that might impose restrictions on the domain. Examples: $\sqrt{x}$ is undefined for negative numbers, $\ln(x)$ is undefined for non-positive numbers, and $1/(x-a)$ is undefined for $x = a$ (where a is any number like 0, -4, or 100).

To get some practice, determine the domains of the following functions; answers are in the footnote[5].

$$f(x) = 23 - \pi x + (x-1)^3 \tag{2.10}$$

$$g(x) = x + \sin(\sqrt{x}) \tag{2.11}$$

$$h(x) = \frac{4-x}{x-5} \tag{2.12}$$

Once you've determined the domain, you can determine the range. Again, sometimes it's easy (e.g., $f(x) = x^3$) and sometimes it's trickier. Sometimes there are rubrics you can apply; for example, in the next chapter you will learn that the range of a polynomial function depends on its highest order (preview: x^k has a range of $-\infty < y < \infty$ when k is odd and $0 \leq y < \infty$ when k is even). The trigonometry sine function has a range of $[-1, 1]$ while the domain is $-\infty < x < \infty$. Be mindful of how other terms might modify the range. For example, the range of $f(\theta) = 2 + 2\sin(\theta)$ is $[0, 4]$.

For more practice, determine the domains and ranges of the following functions; answers are in the footnote[6].

[5] $D_f = (-\infty, \infty)$, $D_g = [0, \infty)$, $D_h = (-\infty, 5) \cup (5, \infty)$.

[6] $D_t = (-\infty, \infty)$, $R_t = 4$. $D_u = [14, \infty)$, $R_u = [0, \infty)$. $D_v = (0, \infty)$, $R_v = [-1, \infty)$. $D_w = (-\infty, 0) \cup (0, \infty)$, $R_w = (0, \infty)$.

$$t(x) = 4 \tag{2.13}$$

$$u(x) = \sqrt{14 - x} \tag{2.14}$$

$$v(x) = x + \sin(\ln(x)) \tag{2.15}$$

$$w(x) = \frac{1}{x^2} \tag{2.16}$$

As with many other concepts in modern applied mathematics, it's important to understand how to determine the domain and range of a function by visual inspection and careful thinking, but in practice, its easiest to get the answers using `sympy`. I'll show you how to do this in the exercises in Chapter 3.

2.3.4 Why do you need to know the domain and range?

Knowing the domain and range of a function offers several advantages in both pure and applied math.

Pure math Knowing the restrictions on a function is one way to understand the function's behavior and identify extreme values, and it helps you to find valid solutions to equations.

Applied math in code Knowing the domain and range of a function helps you: (**1**) avoid coding errors; (**2**) find and fix bugs in your (or someone else's) code; (**3**) determine appropriate values for evaluating and plotting function inputs and outputs; (**4**); develop and modify models of physical or biological systems; and (**5**) properly implement machine-learning and data-analytic models. You'll see examples of all of these advantages in the exercises throughout the book.

2.4 Injective, surjective, and bijective

If you think those three terms refer to some kind of uncomfortable medical procedure, then don't worry! There are no health risks associated with this section. These terms instead refer to ways in which a function maps inputs to outputs. Below are descriptions of each type; I will show visualizations of these characterizations thereafter, but try to understand the concepts based on their descriptions before looking at the figure.

Injective (a.k.a. one-to-one)

A function is injective if each output value has at most one input value. That is, in the function $f(x) = y$, a given value y cannot arise from more than one value of x. Formally, function f is injective if $f(x_1) = f(x_2)$ is true only if $x_1 = x_2$ for all x_1, x_2 in the domain of f.

Surjective (a.k.a. onto)

A function is surjective if every value in the codomain has at least one corresponding input value x in the domain. In other words, the function $y = f(x)$ is surjective if for every $y \in \mathbb{R}$, there exists an $x \in \mathbb{R}$ such that $f(x) = y$.

Bijective (a.k.a. invertible)

A function is bijective when it is both injective and surjective. Such a function is called *invertible* because it is possible to reverse the mapping from y to x. More formally, a function f is bijective if for every y in the codomain, there exists a unique x in the domain such that $f(x) = y$. More on function inverses in a later section.

Neither

Functions need not be injective or surjective. The constant function $f(x) = 7$ is an example of a perfectly valid function that is neither injective nor surjective (and therefore also not bijective).

Now for the abstract visual representations. Figure 2.11 shows abstracted representations of x (gray circles) to y (black circles) via $f(x)$ (arrows). Please take a moment to inspect the figure and

understand how the visual representation illustrates each concept before reading my explanation in the following paragraph.

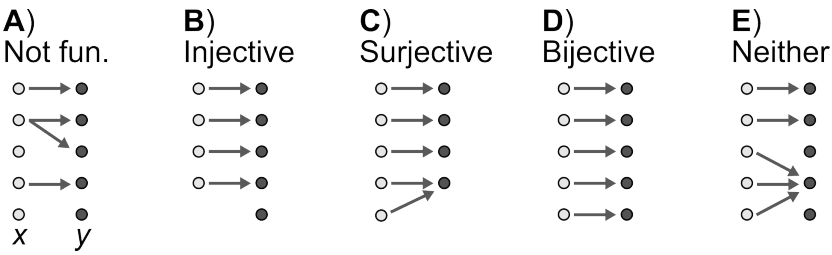


Figure 2.11: Abstracted depictions of injective, surjective, and bijective functions. In each panel, the gray dots represent x, the black dots represent y, and the arrows depict a relation.

Panel A does not depict a function, because there are multiple values of y for one value of x. Panels B through E depict functions. Panel B depicts an injective function, because each value of y comes from at most one value of x (although there can be values of y that cannot be reached from the function; this shows the distinction between the codomain comprising all black dots and the range comprising the dots with arrows). Panel C depicts a surjective function because all possible values of y can be reached from x. Panel D depicts a bijective function because it is both injective and surjective. Finally, panel E depicts a function that is neither injective nor surjective: not all values of y can be reached, and some values of y can be reached from multiple values of x.

You will get more practice labeling functions with these categorizations in Exercise 5.

Restricted domains Many functions can exhibit different properties in a restricted domain. For example, a given function might be neither injective nor surjective across the *entire* domain, but injective for some restricted domain and surjective for a different restricted domain. This is illustrated in Figure 2.12: The sine function is neither injective (multiple x values produce the same y value) nor surjective (the y values are limited to a range of $[-1, +1]$), however, a piece of the sine function is bijective when imposing a restricted domain (indicated by the horizontal gray dashed line, and the vertical dashed line indicates the range).

The ability for a function to become bijective within some subset of points is relevant for determining whether a function is invertible, because some functions do not have a "true" inverse while being "partly invertible" in a restricted domain. More on this point later. Domain-restricted bijectivity is also relevant for linear approximations of nonlinear functions — even the wildest curves can appear straight when you zoom in enough.

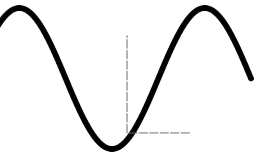


Figure 2.12: Example of a function (thick black line) that is neither injective nor surjective when considering the entire domain, but is bijective within a restricted domain (thin gray dashed lines).

2.5 How to "understand" functions

> "In mathematics, you don't understand things.
> You just get used to them."
>
> – John von Neumann

There is a fascinating philosophical discussion to be had about what it means to "understand" something, to what extent our understanding of nature is limited by — or biased by — our measurement devices and our imaginations, and whether the human nervous system is fundamentally capable of truly understanding nature as opposed to simplified approximations that are sufficient to manipulate materials and build technologies. And there is a related philosophical discussion about whether math really exists independent of humans, or whether it is a language that humans invented, a tiny sliver of which correlates reasonably well with a tiny sliver of the natural world.

But I will not get into those discussions here.

Instead, I will take a more pedestrian view of the topic, and define "understanding functions" as having some intuitive grasp for how the function looks when represented as a graph (e.g., a straight line going up vs. a curve that bends downwards), and having some intuition about how that function behaves around important numerical values, which are often zero, $\pm\infty$, and potentially other specific values based on the function definition. Later in the book,

you will also "understand" functions by having some intuition for their derivative and indefinite integral.

How do you build such intuition? Obviously, working with equations and functions on paper and in Python helps a lot. I will try to help you build intuition using three approaches: math notation, code, and visualization (Figure 2.13).

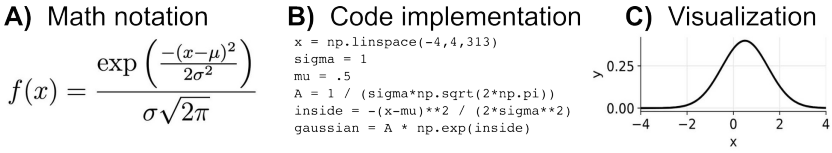


Figure 2.13: Three ways to understand a function: by inspecting its formula, by implementing it in code, and by looking at its graph.

Math notation

This is the most rigorous and universal language for expressing math, for reasons I outlined earlier in this chapter. However, many people find that looking at math notation generates feelings of discomfort, confusion, and insecurity. And equations do not necessarily lend themselves to intuition.

Code implementation

In our modern world, many more people have coding experience than math experience, and therefore many people are more comfortable with code than with math. Code is an amazing way to explore mathematics, because it allows for direct manipulations of the equations and endless visualizations. I believe that this is, by far, the best way to build intuition and really understand math (I acknowledge and respect that some people disagree with that claim).

Visualization

Visualization can be done by hand-sketching functions, which is a great skill to work on. But computer-based visualizations are faster, more accurate, and allow for more varied explorations of functions than what is possible only by hand-sketching. The advantage of visualization is that it provides immediate and deep intuition of the ebbs and flows of a function. This is important for concepts like limits, derivatives, and integrals, and

is also important for applications of calculus including optimization, statistics, machine learning, and control engineering.

Here is the goal: You want to be able to look at an equation and make some predictions about what that equation looks like on a graph and how it behaves for different values of the input variable. This is a skill that you can learn with practice; and the greater your skill, the better you will understand anything that involves applied math (which is nearly everything in modern human technology).

For now, your goal is to be able to answer three questions when looking at an equation (I'll add more questions to this list later in the book):

1. What does the function look like when represented using a line on a graph?
2. What is the function output when the input is close to zero? (That is, what is $y \approx f(0)$?)
3. What happens to the function when the input gets really far away from zero? (That is, what happens when x approaches $-\infty$ and $+\infty$?)

Let's start with a simple example; answer the three "understanding" questions simply by looking at the equation, before reading the text below:

$$f(x) = 7 - 2x \tag{2.17}$$

And here are my answers: (**1**) The graph of the function is a straight line that goes down as x increases (that is, the slope is negative). (**2**) $f(0) = 7$, which means that the line touches the coordinate (0,7) in the plot. We could also set $y = 0$ to determine that a second coordinate on the line is (3.5,0). (**3**) As x goes far to the left of the plot (that is, as x approaches $-\infty$), the function goes up towards $y = \infty$. And as x goes far to the right of the plot, the function goes down towards $-\infty$.

In Chapter 4, I'll formalize the second and third questions using the concept of a *limit*; for now, just focus on qualitative descriptions.

Shall we do a second example? (I can't hear you answer, so I'll assume you gave an enthusiastic *yes*.)

$$f(x) = \frac{x^3}{x-1} \tag{2.18}$$

This equation is a little trickier. If you squint your eyes and ignore the "-1", you will see that Equation 2.18 is kind of x^2. Something horrible happens at $f(1)$ because we get $1/0$, but let's ignore that for a moment to answer the three questions: (**1**) The graph of this function is a quadratic curve that bends upwards (that is, it's shaped like a "U"), similar to x^2, although we can expect the function to look very different as x approaches -1. (**2**) $f(0) = 0$, which means the curve representing this function passes through the origin $(0, 0)$. (**3**) The function quickly increases to $+\infty$ as x goes far to the left and to the right of the graph (Figure 2.14). Note that the numerator x^3 is negative for $x < 0$, but so is the denominator; the signs cancel and so the long-range behavior of the function is positive for $\pm x$.

You should also notice that strange things will happen as x gets really close to 1. In fact, as x approaches 1, the denominator gets really close to zero, which means y will get extremely large (positive or negative, depending on whether x is just below or just above 1). In fact, this function has an *infinite discontinuity* and limits are the tool to understand this behavior. Limits are so important in calculus that they get their own chapter. What you should focus on now is the skill of looking at an equation and noticing that there are values of x where the function has interesting behaviors.

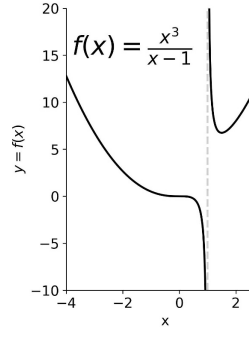


Figure 2.14: A function, visualized. Vertical dashed line indicates $x = 1$.

I hope you enjoyed building the skill of understanding functions. This is a skill that you will continue to develop throughout this book — and every encounter with equations for the rest of your life.

2.5.1 Working with "weird" functions

Have another look at Equation 2.18. What does that function *mean*? Does it correspond to something in nature, like the height of a mulberry bush over time or the distance between a comet and the Earth? I admit that I do not know for sure, because I do not have access to everything in nature. But I'm pretty sure that Equation 2.18 has absolutely no direct connection to any measurable physical quantity in our lovely universe.

If you are interested in pure mathematics, then the previous paragraph probably doesn't bother you. Math has a set of rules — a syntax — and Equation 2.18 fits perfectly within those rules, so who cares if it relates to any physical feature that we can observe in the universe?

On the other hand, if you are interested in applied math, that is, using math as a means to build cars or machine-learning algorithms, then you might experience some combination of confusion, boredom, and discomfort with the idea of spending time on functions that have no identifiable reference to the real world. In fact, it's possible that the abstractness of pure math is the reason why you didn't pursue math when you were younger, and the reason why you are reading this book right now[7].

The point is that many *many* ***many*** functions in math books have no direct application to the physical world. I half-jokingly call these "weird" functions.

Why work with "weird" functions? The answer is that "weird" functions help you build crucial foundational mathematical knowledge. The more equations, and variety of equations, you work with, the easier it will be to understand the equations that you really do care about.

Think of the gym as an analogy: You go to the gym to increase your fitness, build muscle, boost your cardio capacity, and so on.

[7] I write this from personal experience: I only started making real effort to learn math in order to understand statistics during my PhD.

But most things you do at the gym are really weird: When in your life do you need to lift a long, thin metal pole with heavy circular disks at both ends?? But that's not the point; the point is that doing these "weird" exercises in the gym helps you do everything else in day-to-day life. Same idea with math: You work with "weird" functions in order to build knowledge and intuition, and outside of this book, you use those skills to help you understand, implement, and develop applications in applied math with meaningful equations.

2.6 Functions in `numpy` and `sympy`

You will use `numpy` and `sympy` a lot in this book — and in your future adventures with applied math. Both libraries are amazing collections of functions, and both are unique. The key difference is that `numpy` is for *numbers* while `sympy` is for *symbols*. What does that difference mean?

When you write down math on a piece of paper or chalkboard, you use symbols like x, π, $\sqrt{2}$, and so on. But `numpy` cannot represent such symbols; you have to define *specific numerical values* that those symbols take on. Let me explain using an example. Consider the following code:

```
# symbolic pi
pi = sym.symbols('pi')
print('From sympy:')
display(pi)
display(pi + 3)
```

`display` is a functio that can display non-ASCII characte in an IPython notebook.

```
From sympy:
π
π + 3
From numpy:
3.141592653589793
6.141592653589793
```

Figure 2.15: Screenshot of Python output. Note the difference in font face between the outputs of `display` and `print`.

That code creates a symbolic variable π and then displays it. `sympy` (abbreviated to `sym`) represents `pi` as the *symbol* π, not the *number* $\pi \approx 3.1415...$ Therefore, the mathematical expression $\pi + 3$ prints out as, well, it prints out as $\pi + 3$ (Figure 2.15).

Were you expecting the result to be 6.14...? Well, 6.14... is *not* $\pi+3$. Instead, it is an approximation of $\pi+3$ that is probably accurate

enough for most real-world applications, but it is incorrect in a pure mathematical sense.

Now let's see what happens when we use `numpy` (code is below, output is shown in Figure 2.15:

```
print('From numpy:')
display(np.pi)
display(np.pi + 3)
```

I guess that this is the output you were expecting. But as I wrote above, it is formally incorrect. The point here is that `numpy` cannot actually represent π; instead, it calculates an approximation of π to machine precision.

Does this actually matter? In some applications, it does not. But the difference between these libraries becomes more apparent when creating expressions with variables.

Substituting numbers for variables Symbolic calculations are great, but there are many situations where you need numeric results. For example, for some function $f(x)$, you might need to know the numerical value of $f(5)$, even if it's an approximation. For a symbolic expression in `sympy`, this is done using the `.subs()` method. To demonstrate, I'll first create an expression, which is automatically created as a `sympy` object because it contains a symbolic variable:

```
from sympy.abc import x
expr = 2*x + 3*x**2
```

You can evaluate that expression for a specific value of x using the following code:

```
expr.subs('x',1)
expr.subs({x:1})
```

The output of both of those lines of code is the function evaluated for $x = 1$ (the result is 5). You can provide either two inputs for the variable and the number, or you can input a dictionary with key-value pairs. Both lines of code produce identical outputs; the dictionary input is for expressions with multiple variables to substitute (e.g., `expr.subs({'x':1,'y':3)}`).

2.6.1 From `sympy` to `numpy`

There are several reasons why you might need to transform a `sympy` object into a `numpy` variable; the two main reasons are (**1**) you need `numpy` variables for other parts of the code, and (**2**) you want to visualize the result. `sympy` does have a plotting engine (`sym.plot()`) but it is awkward to use and limited in possibilities. It's much better to convert your `sympy` result into `numpy` and then visualize using `matplotlib`.

There are two ways to convert from `sympy` to `numpy`.

Lambdafication To "lambdify" a `sympy` object means to convert it into a Python function that accepts and returns `numpy` data types[8]. Consider the following example:

```
fx = sym.lambdify(x,expr)
fx(1)
>> 5

print( fx( np.array([0,1,2]) ) )
>> array([ 0,  5, 16])
```

Thus, to use `sym.lambdify`, input the symbolic variable and then the symbolic expression. The output is a function that you can use the same way you'd use a regular Python function.

[8] Given that the code object is called a "lambda expression," transforming a function into a lambda expression should be spelled lambd*a*fication. But the procedure is also spelled lambd*i*fication in some texts and function names.

List comprehension The second method to evaluate a `sympy` expression for particular numbers is to use variable substitution in combination with list comprehension. Here's an example:

```
xvals = np.array([0,1,2])
[ expr.subs('x',xi) for xi in xvals ]
>> [ 0, 5, 16 ]
```

The reason why list comprehension is necessary is that the `.subs()` method is limited to individual numbers; it is not possible to substitute with a vector.

The advantage of this method over lambdafication is that it requires less code. The two disadvantages are that it is not reusable the way a function is, and that it is relatively slow.

The conclusion is that if you will convert one fairly small `sympy` object into a numeric vector (either a list or `numpy` array) that you need only once, then use the list comprehension method; and if you need to access the `sympy` object multiple times or if you have many numbers to evaluate, then use the lambdafication method.

2.7 Exercises

A quick reminder of what I wrote in Chapter 1: These exercises are your opportunity to explore concepts in calculus using code. I hope you see them not as tedious work to labor through, but as inspiration to continue coding and experimenting on your own. The solutions are not printed in the book, but they are all available at

`github.com/mikexcohen/Calculus_book`

Video explanations and discussions of each exercise are on my YouTube channel: `https://www.youtube.com/@mikexcohen1` (a direct link to the playlist is in the github repository linked above).

2.1. Recall that the formula for a circle is $r^2 = x^2 + y^2$. Derive math functions $f_p(x)$ and $f_n(x)$ for the positive (upper-half) and negative (lower-half) halves of the circle. Plot the two functions as in Figure 2.16. (I used $r = 2$ but you can choose any value of r, and any resolution, that you like.)

Notice that the plot looks like a circle only if the x- and y-axes have equal spacing. You can impose this by setting the limits of both axes to be, e.g., $[-r-.1, r+.1]$. Try changing the bounds of the axes, e.g., setting the y-axis limits to be $[-2r, 2r]$ without changing the x-axis limits. The graph no longer looks like a circle, although the numerical values of the coordinates have not changed. The impact of axis scaling on visual interpretation is an important part of data visualization.

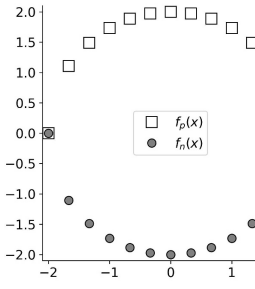


Figure 2.16: Plot f
Exercise 1.

2.2. This exercise will illustrate the importance of x-axis resolution when visualizing a function. Start by writing a Python `lambda` expression that calculates the function $(x+1)/x^3$. Then, compute and visualize that function using x-axis bounds $[-.1, .1]$ with resolutions determined by the number of points in that interval given by $N = \lfloor \sqrt{2} \times 10^k \rfloor$ for $k \in \{1, 2, 3\}$. The $\lfloor . \rfloor$ indicates the *floor* operator, which

means to round down to the nearest integer.

Plot all three lines as in Figure 2.17. I added y-axis offsets to each of the lines to improve visibility.

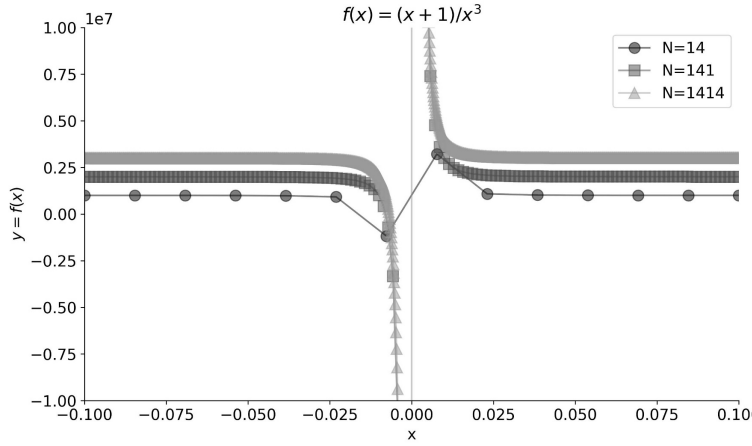


Figure 2.17: Plot for Exercise 2. The functions were shifted on the y-axis to facilitate comparison.

When you run the code in Python, you will get a warning message about `divide by zero`. That warning appears only once, because the warning happened only for one of the resolutions. Which resolution caused the warning and why? The answer to this question is printed in the online code.

One of the take-home messages from this exercise is an illustration of why it is incorrect to draw lines between successive markers: The N=14 line implies *incorrect* function values! Markers without connecting lines are more accurate. That said, when the resolution is sufficiently high compared to the changes in the function — and if multiple functions are drawn on the same plot — then using lines instead of individual markers can be acceptable if it facilitates interpretation.

2.3. $\sqrt{2}$ can be approximated using the "Babylon method" of estimating roots. It is a simple algorithm and works surprisingly well. Implement the following algorithm in Python:

$$a_n = \frac{1}{2}\left(a_{n-1} + \frac{2}{a_{n-1}}\right) \tag{2.19}$$

where a_n is the n^{th} estimate of $\sqrt{2}$, and the starting value a_0 is any number that you think might be a reasonable starting guess.

Run this algorithm in a for-loop for five iterations, starting with $a_0 = 1$. Print a table of the iteration number in the first column, the estimate in the second column, and the difference from `np.sqrt(2)` in the second column, like below.

```
 n  ::   Approx.  ::    diff
..............................
 1  ::  1.500000  ::   0.085786
 2  ::  1.416667  ::   0.002453
 3  ::  1.414216  ::   0.000002
 4  ::  1.414214  ::   0.000000
 5  ::  1.414214  ::  -0.000000
```

I find it remarkable that this simple algorithm — based only on algebra — works so well and converges so fast.

Try some other starting values. What happens when you set $a_0 = 10$? How about a negative number like $a_0 = -1$?

For the next part of this exercise, take the code you just wrote and put it in a second for-loop over logarithmically spaced starting values between .5 and 2. Visualize the results as in Figure 2.18.

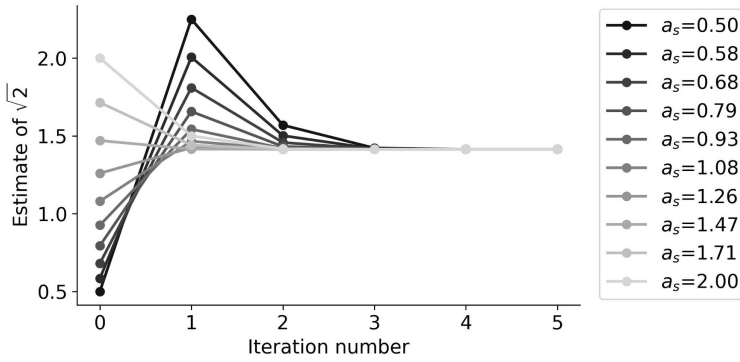


Figure 2.18: Plot for Exercise 3.

Finally, explore the Babylon algorithm by modifying the code: What happens if the starting values range from negative to positive numbers? How about if they include $a_0 = 0$? Why do starting values between 0 and 1 have that strange overestimation for the first few iterations?

2.4. Consider the graph of $y = \pm x$ (Figure 2.19). This expression is not formally a function (e.g., it fails the vertical line test); however, there are at least two pairs of functions you can derive that will produce exactly the same graph. How many can you come up with? I show my two solutions in the online code.

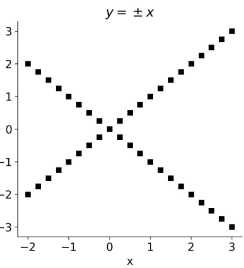


Figure 2.19: Visualization for Exercise 4.

2.5. The purpose of this exercise is to explore the importance of data type on arithmetic calculations, and as inputs into `numpy` functions.

Write a Python function that evaluates the math function $5 \times 4/3$. Force the data type of the three numbers to be `int` or `float` (e.g., `int(5)` vs. `float(5)`) as specified in an input to the function. Then force the result of that calculation to be `int` or `float` as specified by the second input to the function. In other words, a function call might look something like this: `fun('int','float')` to do the calculations on integers with a floating-point output. Print

the results as follows, and ponder the implications.

```
Inputs are floats, output is float, result is 6.6666667
Inputs are floats, output is int, result is 6
Inputs are ints, output is float, result is 6.6666667
Inputs are ints, output is int, result is 6
```

A key outcome of this exploration is that Python converted the two integers into a floating-point number. On the other hand, the floating-point result was clipped when converted into an integer.

In the second part of this exercise, use `np.linspace` to create linearly spaced numbers between 0 and 10, with the number of numbers defined by `N=10/2`. Of course, that just means five numbers, but the question at hand is whether the third input into `np.linspace` can have a data type of floating-point number or integer. Try it both ways![9]

The overall conclusion of this exercise is that sometimes Python is flexible enough to convert data types when necessary, while other times you need to convert data types on your own. This is something to be mindful of, and something to check when getting error messages that mention the keyword `TypeError`.

2.6. Implement and visualize the following functions in Python, using `lambda` expressions and `numpy`. Plot using an x-axis domain of -2 to 4, but note the domain restrictions in functions b and k. Label each function as being injective (but not surjective), surjective (but not injective), bijective, or neither.

a) $\max(0, x)$

b) $x^{1/2}, \quad x > \pi$

c) $2x^2 + 3x^2$

d) e^x

e) $2x(3x - 1)(4x + 1)$

f) $x + \sin(x)$

[9] Spoiler alert: You'll get an error with one of the data types, but pay attention to the error message.

g) x

h) $2x + 1$

i) $\cos(x)$

j) $\tanh(x)$

k) $\ln(x) - 4e^{-(x-1)^2}, \quad x > 0$

l) 1

The results are shown in Figure 2.20. I've blanked out the function types to preserve the mystery for you to discover, but the answers are shown in the online code.

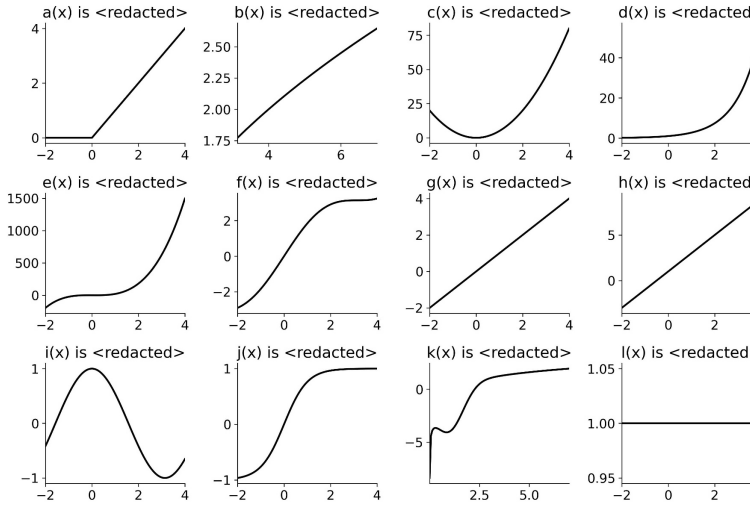


Figure 2.20: Plot for Exercise 6.

Note: You probably won't encounter these *-ective* terms in applied math, but understanding how certain functions map inputs to outputs is a handy skill that will boost your ability to work with, apply, and develop math-based algorithms in any application. Additionally, this exercise helped you gain more experience (**1**) translating formulas into code, and (**2**) understanding functions by careful inspection.

2.7. Pick a few of the functions in the previous exercise to implement as `sympy` expressions. Then obtain `numpy` format output using
`sym.lambdify` and list-comprehension. Make sure the output is a `numpy` array. I recommend starting with function k so you can replicate my results below, but I also rec-

ommend solving this exercise several times from scratch: being able to output a `numpy` array from a `sympy` expression is a really useful skill to have at your fingertips.

I used symbolic variable τ instead of x. Here's how the function looks when printed using LaTeX in `sympy`:
$k(\tau) = \log(\tau) - 4e^{-(\tau-1)^2}$

Evaluating that function for $\tau \in \{1, 2, 3\}$ produces the following result:
`array([-4., -0.77837058, 1.02534973])`

2.8. I mentioned earlier that the `sympy` plotting engine is fine for quick visualization of `sympy` expressions, but if you want to customize the graph, it's better to use `matplotlib`. On the other hand, `matplotlib` cannot plot `sympy` expressions. So you need to convert to `numpy` and then use `matplotlib`.

The goal of this exercise is to pick one of the functions from Exercise 6, and visualize it using `sym.plot` and `matplotlib`. Don't worry about customizing the `sympy` plot in terms of axis labels or locations, line color and thickness, title and legend, etc. But you should set the x-axis limits to [0,3] and the y-axis limits to $[-6, 1]$.

I don't show the plots here because they are redundant with Figure 2.20, but the online solution code shows a `sympy` and `matplotlib` visualization of function $k(x)$.

Chapter 3

Families of functions

3.1 Families of functions

There is an infinite number of functions, because there is an infinite number of ways of combining terms and operations into a function. Here is an absurdly tiny subset of the uncountable myriad of possible functions:

$$f_1(x) = 2x + 1 \tag{3.1}$$

$$f_2(x) = 2x + 1.001 \tag{3.2}$$

$$f_3(x) = 2x + 1.002 \tag{3.3}$$

$$f_4(x) = 2.01x + 1.002 \tag{3.4}$$

I'm sure you've figured out that the equations above are really similar to each other, even though they are technically distinct. The purpose of this chapter is to introduce you to *families* of functions that are grouped together because they share similar characteristics. This knowledge is important because understanding the groupings of functions will help you understand and apply functions in real-world applications.

3.2 Linear vs. nonlinear functions

What do you think is the difference between ***line***ar and *non****line***ar functions? I'm sure you have an intuition for linear (represented as straight lines) and nonlinear (represented as bending curves) functions; in this section I will provide some definitions that will be helpful throughout the rest of the book, and will set the stage for understanding derivatives.

Here is one way to formalize the distinction between linear and nonlinear functions:

$$\textbf{Linear:}\ \text{Can be simplified to } y = mx + b \tag{3.5}$$

$$\textbf{Nonlinear:}\ \text{Cannot be simplified to } y = mx + b \tag{3.6}$$

where m is called the "slope" and b is called the "intercept." You may need to do some algebraic rearrangement to get the equation of a linear function into the form $y = mx + b$.

Below are three functions; take a moment to determine whether each equation describes a linear or nonlinear function:

$$y = 2\sqrt{\cos(x)} + 6 \tag{3.7}$$

$$x^2 = \frac{y}{3} + 4/3 \tag{3.8}$$

$$\cos(6) = \sqrt{2}x + y \tag{3.9}$$

Equation 3.9 is the only linear function. Don't be confused by the presence of the nonlinear operations (cos and $\sqrt{}$) — those operations are applied to m and b; no nonlinear operations are acting on variables x and y. The other two functions are nonlinear because the nonlinear operation is applied to x or y. In other words, nonlinear operators can appear in linear functions, as long as those nonlinear transformations are not applied to the variables x or y.

These three functions are graphed in Figure 3.1.

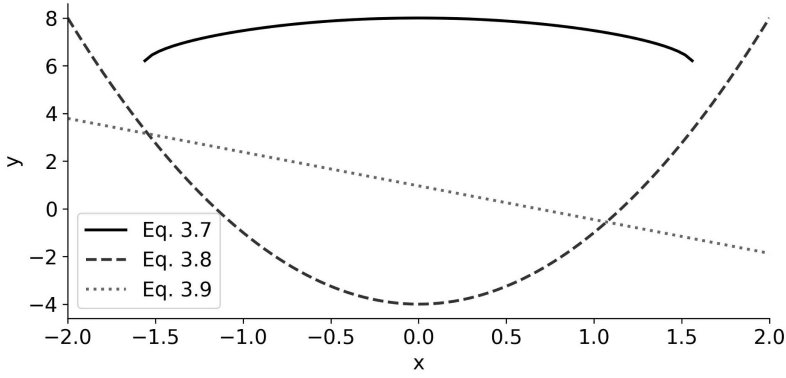


Figure 3.1: Visualization of the three equations. Visual inspection can be used to identify linear vs. nonlinear functions.

Please do not rely exclusively on a visual representation of a function to determine linearity; nonlinear functions may appear linear while being subtly nonlinear, or may be linear in the visualized subdomain of the function while being nonlinear in some other subdomain.

Constant changes Now I will present an alternative definition of linearity. This definition concerns the way a function changes on the y-axis for different values of x. Before presenting this definition, notice in Figure 3.1 that the nonlinear functions change at different rates for different values of x. For example, Equation 3.8 has small changes in y when x is close to zero, and large changes in y as x gets further from zero. On the other hand, for the line corresponding to Equation 3.9, the changes in the y-axis look the same regardless of the x-axis value. To be clear, the values of the function change throughout the graph, but the *way* that the function changes is the same at all x-axis values only for the linear function.

Here is the formulation:

$$\textbf{Linear: } \frac{\Delta y}{\Delta x} = c \tag{3.10}$$

$$\textbf{Nonlinear: } \frac{\Delta y}{\Delta x} \neq c \tag{3.11}$$

where c is some constant (a number like 9 or 17.4 or $-\pi^2$), and Δ (Greek capital letter "delta") is a math symbol that indicates a change (e.g., $\Delta = 3 - 1 = 2$). Equation 3.10 says that the ratio of the change in the function output to the function input is always the same. So, however the function behaves in response to its input, that behavior is the same for all values of x in the domain. The nonlinear functions, on the other hand, do not have that constraint: perhaps they increase for some values of x, decrease for other values of x, and are flat for yet other values of x.

I hope you can see the link between Equations 3.5 and 3.10: imagine that $b = 0$, ignore the Δ, and set $m = c$.

To make this Δ definition more concrete, consider the two functions $f(x) = 2x$ and $g(x) = x^2$. Figure 3.2 shows a table with several x, y pairs and their difference ratio. Each row has unique values of x and y, but that's not what Equation 3.10 is about. Instead, the *change* in y divided by the *change* in x is constant

for $f(x)$ while varying for $g(x)$. Thus, $f(x)$ is a linear function while $g(x)$ is a nonlinear function.

I hope that makes sense. The concept of considering changes in y by changes in x is the basis of *differentiation*, which is one of the central concepts in calculus.

f(x) = 2x			g(x) = x		
x	**y**	**Δy/Δx**	**x**	**y**	**Δy/**
0	0		0	0	
2	4	4/2	2	4	4
4	8	4/2	4	16	12
6	12	4/2	6	36	20

Figure 3.2: Tables of two functions, values of x and y, and the change in each row from the previous.

Affine One final point for this section: In linear algebra, a distinction is made between linear and *affine*. In that context, "linear" has the additional constraint that the line must pass through the origin of the graph ($f(0) = 0$). A straight line that does not pass through the origin is termed *affine*. I won't use that term for the rest of this book, and mention it here only to preemptively avoid confusion for those with a linear algebra background.

3.3 Polynomials

Polynomial functions are quite simple: take the variable x and raise it to higher powers, multiply by some coefficient, and sum the terms together. The general form of a polynomial looks like this:

Gentle reminder that $x^0 = 1$ and $x^1 = x$.

$$f(x) = a_0x^0 + a_1x^1 + a_2x^2 + ... + a_kx^k \tag{3.12}$$

The a terms are called the *coefficients* and are real-valued numbers like 8 or -9.3 or 0. The a_0 term is sometimes called the *intercept* of the equation, because it's where the function intercepts (crosses) the y-axis; notice that $y = a_0$ when $x = 0$.

Equation 3.12 can be expressed more compactly using summation notation:

$$f(x) = \sum_{i=0}^{k} a_ix^k \tag{3.13}$$

k is an integer and is called the *order* of the polynomial. For example, if $k = 3$, then the highest power in the equation is x^3, and the function would called a "third-order polynomial." Note that a k^{th} order polynomial has $k + 1$ coefficients, because of the $a_0 x^0$ term (the intercept). Below are a few examples of polynomials; please determine the order of each function.

$$f_1(x) = 4 + x - 3x^2 + \sqrt{e^\pi} x^3 \tag{3.14}$$

$$f_2(x) = -5x^5 - 4x^2 \tag{3.15}$$

$$f_3(x) = 1 + 2x + 3x^2 + 4x^{-3} + 5x^{1/2} \tag{3.16}$$

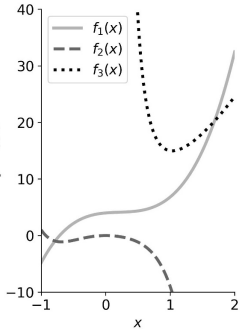


Figure 3.3: Equations 3.14-3.16 visualized.

$f_1(x)$ is a 3^{rd}-order polynomial. $f_2(x)$ is a 5^{th} order polynomial. Don't be confused by "missing" terms like x and x^4; the order of a polynomial is defined solely by the largest-power term. You can imagine the other x terms being present but multiplied by a coefficient of zero. Finally, Equation 3.16 is a... what order polynomial is this??

Actually, that one's a trick that I tossed in just to confuse you. Polynomials can only have x terms raised to non-negative integers. Therefore, $f_3(x)$ is *not* a polynomial. To be sure: it is a perfectly valid function, but it does not belong to the polynomial family.

Why are polynomials restricted to non-negative integer powers? This is not some fundamental physical law that was written into existence in the Big Bang. Instead, mathematicians all agree to impose this restriction because it helps make polynomial functions more homogeneous in terms of their properties and applications.

3.3.1 What do polynomials look like?

There is a special place in every mathematician's heart for polynomials — and I'm sure that by the end of this book, you will share this inexhaustible fondness for polynomials. Polynomials have lots

of great properties: they have easily calculable derivatives and integrals, are used to model biological and physical systems, are used in statistics and predictive analytics, simulations and video games, deep learning models, and myriad other applications.

One of the many great features of polynomials is that their visual features are easy to recognize. Figure 3.4 shows plots of polynomials of different orders. Please take a moment to inspect the figure and draw some general conclusions about the behavior of polynomials.

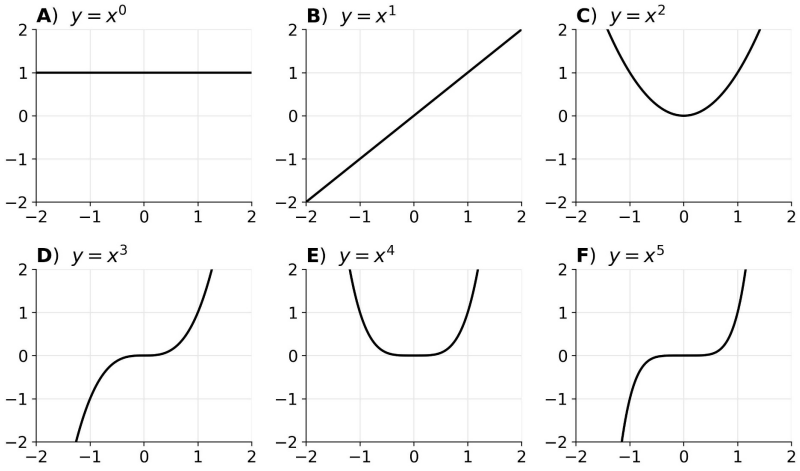


Figure 3.4: Polynomials of various orders.

Some comments: A zero-order polynomial (panel A) is a flat line, because $x^0 = 1$ for all values of x. A first-order polynomial (panel B) is a straight line — a linear function. The slope of the line depends on the coefficient (that is, the value of a in $y = ax$), which here is implicitly $a = 1$.

For higher powers of x, the plots have alternating appearances: All the even powers are parabola-shaped whereas all the odd powers go down to the left and up to the right. The symmetric/anti-symmetric property is related to the *parity* of a function ("even" or "odd" functions), and you'll learn more about that later in this chapter. You can also see from the figure that the larger the power, the faster the polynomial climbs to $\pm\infty$.

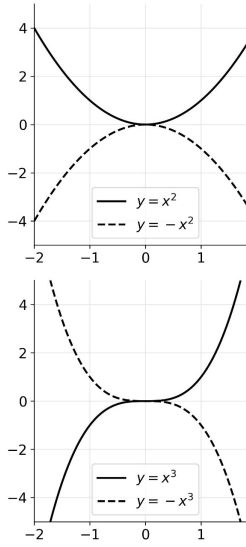


Figure 3.5: Impact of the sign of the coefficient on the polynomial.

The sign of the coefficient of the polynomial determines whether

the parabola opens up or down (for even-ordered polynomials) or whether the line goes negative-to-positive or positive-to-negative (for odd-ordered polynomials) (Figure 3.5).

Polynomial power domination The behavior of a polynomial is dominated by the highest power for larger values of x, regardless of coefficients and the lower-power terms. Consider the following polynomial:

$$y = 10 - x - 10000x^2 + .001x^3 \tag{3.17}$$

You might think that the function would behave like $y = -x^2$ considering the sizes of the coefficients. That actually is true for values of x relatively close to zero. But the larger $|x|$ becomes (that is, the further from zero in either direction), the more the function behaves like $y = x^3$ (see Figure 3.6).

This property of domination by the largest-power terms is relevant for understanding limits, which you'll learn about in Chapter 4.

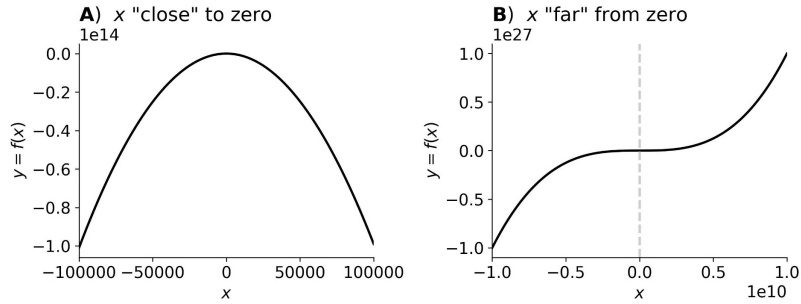


Figure 3.6: Every polynomial function (in this example, Equation 3.17) eventually behaves like the highest-power term when x gets "sufficiently large." Note the different x- and y-axis limits. Vertical dashed gray lines in panel B indicate the x-axis boundaries in panel A (the two lines are so close together that they appear as one).

3.3.2 Math with polynomials

You need to know how to do arithmetic with polynomials, but I'm pretty sure that the material in this section will be a review of what you learned (and possibly forgot) in school.

Addition and subtraction To add polynomials, you add the coefficients with terms of the same power ("like terms"). For example:

$$f(x) = 3x + 5x^2 - 6x^3 + x^6 \tag{3.18}$$

$$g(x) = 2x - 2x^2 + 3x^3 - 2x^5 \tag{3.19}$$

$$f(x) + g(x) = (2+3)x + (5-2)x^2 + (-6+3)x^3 - 2x^5 + x^6 \tag{3.20}$$

$$= 5x + 3x^2 - 3x^3 - 2x^5 + x^6 \tag{3.21}$$

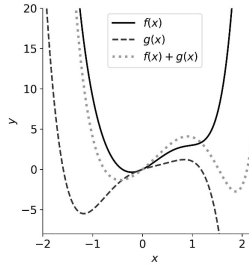


Figure 3.7: Curves corresponding to Equations 3.18-3.21.

Subtraction doesn't require a separate discussion, because you can consider $a - b$ to be $a + (-b)$.

Multiplication Multiplying polynomials is not difficult per se, but it does require some concentration to avoid careless mistakes. Before explaining polynomial multiplication, let me remind you of the Law of Exponents:

$$y^a + y^b \cdot x^c \cdot x^d = y^a + y^b \cdot x^{c+d} \tag{3.22}$$

Not directly relevant here but just as a reminder: $(x^a)^b = x^{ab}$

In words, the Law of Exponents states that exponents sum when the bases are the same (x in this example), but only for multiplication (that is, $y^a + y^b \neq y^{a+b}$).

To multiply two polynomials, expand the multiplication and sum exponents for terms with the same base. The example below shows polynomials of two variables. Please make sure you can reproduce my results (also to check my work — math teachers are not infallible!).

$$f(x) = x^5 + 2xy + y^2 \tag{3.23}$$

$$g(x) = x - 3xy \tag{3.24}$$

$$f(x) \times g(x) = (x^5 + 2xy + y^2)(x - 3xy) \tag{3.25}$$

$$= x^6 - 3x^6y + 2x^2y - 6x^2y^2 + xy^2 - 3xy^3 \tag{3.26}$$

I will discuss division with polynomials in the next section.

3.4 Rational functions

You might be thinking that a *rational function* is a term in economics that describes the best course of action after considering all potential positive and negative outcomes. That would be a description of a rational agent.

A rational function in math actually comes from the word ***ratio***.

A rational function is a function expressable as the ratio of two polynomials. In other words, a rational function is:

$$f(x) = \frac{p(x)}{q(x)} \tag{3.27}$$

where $p(x)$ and $q(x)$ are polynomials. All polynomials are rational functions; are all rational functions polynomials? The answer is

in the next paragraph, but I encourage you to think of an answer before reading further.

Consider that any polynomial function can be divided by $q(x) = x^0$, therefore, all polynomial functions are rational functions. The converse, however, is not true: not all rational functions are polynomials. The reason is that polynomials are defined only for positive integer powers. Here's an example of a rational function that comprises polynomials but is itself not a polynomial function:

$$p(x) = 7x^2 \tag{3.28}$$

$$q(x) = 3x^5 \tag{3.29}$$

$$r(x) = \frac{p(x)}{q(x)} = \frac{7x^2}{3x^5} \tag{3.30}$$

$$= 7x^2 \cdot 3^{-1}x^{-5} \tag{3.31}$$

$$= \frac{7}{3}x^{-3} \tag{3.32}$$

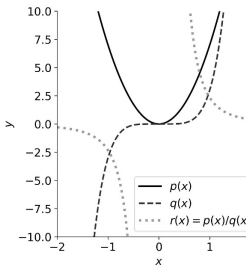


Figure 3.8: Curves corresponding to Equations 3.28-3.30.

A slight tangent: Are the following two functions equal to each other?

$$f(x) = x^3, \qquad g(x) = \frac{x^4}{x}$$

You might think they are, because $g(x)$ can be algebraically simplified to x^3. And it certainly is true that $f(a) = g(a)$ for most values of a. But consider the domains of each function.

The domain of $f(x)$ is all real numbers whereas the domain of $g(x)$ is all real numbers *except* $x = 0$. In math notation, you can write the domain of $g(x)$ as $(-\infty, 0) \cup (0, \infty)$, as $\{x \in \mathbb{R} : x \neq 0\}$, or as $\mathbb{R} - \{0\}$.

Therefore, $f(x)$ and $g(x)$ are distinct functions, although they do behave in the same way for most values of x.

3.5 Natural exponential and log functions

I'd like to start this section by introducing the term "transcendental function." Transcendental functions are those that cannot be expressed as the combination of a finite number of algebraic terms. They "transcend" algebra.

That sounds esoteric and magical, but lots of commonly used functions are transcendental. For example, c^x (for some constant c and variable x) and x^π are transcendental. In this book we will mostly work with three transcendental functions: $\sin(x)$, $\ln(x)$, and e^x. (Other trig functions like $\cos(x)$ and $\tan(x)$ are basically the same as $\sin(x)$, as you'll learn later.)

3.5.1 Natural exponential

The natural exponential function is one of the most important functions in all of mathematics. It shows up everywhere — in calculus, differential equations, trigonometry, biology, finance, navigation, physics, signal processing, population modeling, statistics... where you see applied math, you'll probably see the natural exponential function.

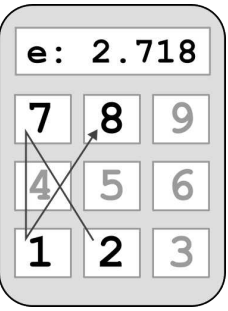


Figure 3.9: To remember the approximation of e, draw a tall "x" pattern on a number keypad.

What is the natural exponential function? It is simply the constant e raised to the power x: $f(x) = e^x$.

OK, simple enough. But what is the number e? e is an irrational number, meaning it cannot be represented as the ratio of two integers like 22/7. Instead, the numbers after the decimal point go on forever without a repeating pattern. I couldn't write out all of e here, because it would take all of the mass and energy of the universe to write out an irrational number — and even then, we wouldn't have the complete number; we'd just exhaust the universe's supply of matter. All that said, it is good to commit the first few digits of e to memory:

$$e \approx 2.718 \tag{3.33}$$

I remember these four numbers because they make a nice spatial pattern on a number keypad (Figure 3.9).

e^x is sometimes written out $\exp(x)$. That notation facilitates interpretation when the exponent has multiple terms. For example, many people find the expression $\exp(-(x-3)^2/\sigma^2)x^2$ easier to read than $e^{-(x-3)^2/\sigma^2}x^2$.

The `numpy` function to approximate e is `np.exp()`.

e^x has many amazing properties that you will learn about throughout this book and in your future adventures through mathland. Figure 3.10 shows a plot of e^x. I'll make a few remarks about this function in the next paragraph, but before reading further, please make a list of properties of e^x based on what you see in the graph.

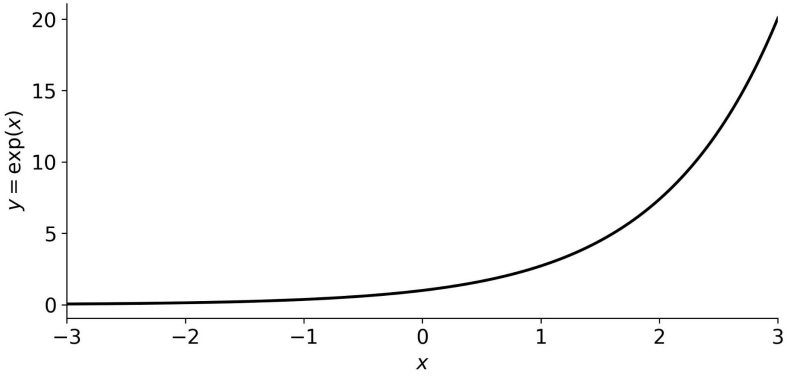


Figure 3.10: Graph of the function e^x.

Here are some interesting properties of e^x: (**1**) The domain of $f(x) = e^x$ is all real numbers, whereas the range is $(0, \infty)$. In other words, e^x can never be negative — nor can it equal exact zero, although it does get increasingly close to zero as x goes towards $-\infty$. Thus, e^x is an injective function.

(**2**) $e^0 = 1$ (of course). $e^x > 1$ for any $x > 0$, and $e^x < 1$ for any $x < 0$. In words: Negative values of x lead to small positive values

of e^x whereas positive values of x lead to large positive values of e^x.

(**3**) As x gets larger, e^x goes up to ∞, and that rate of increase is not constant: The larger x gets, the faster e^x grows to ∞. In fact, the derivative of e^x is e^x; it's the only function that is its own derivative. You'll learn more about that in later chapters.

Where does *e* come from? Another remarkable feature of e is that it has multiple definitions. You can define e based on having a desired geometric property, having a desired derivative property, using trigonometry, or using the sum of an infinite series. All definitions lead to the same irrational number. Here is one definition of e:

e is also called "Euler's number."

$$e = \lim_{n \to \infty} \left(1 + \frac{1}{n}\right)^n \tag{3.34}$$

If you're unfamiliar with the "lim" notation, then don't worry! You'll learn about that in the next chapter. For now, just imagine it meaning that n is really really *really* big, up to ∞. For any finite value of n, Equation 3.34 is just an approximation. You can quickly calculate that when $n = 1$, $e \approx 2$. When $n = 10$, $e \approx 2.59374$. In Exercise 6, you'll implement Equation 3.34 to observe how the equation approaches e as n increases.

3.5.2 Natural log

It is no random coincidence that I'm writing about the natural exponential and the natural log in the same section — or that they both have the word "natural." In fact, these two functions are *inverses* of each other. I'll define inverse functions in Section 3.10, but it basically means that one function "cancels" the other.

The natural log is written as $\ln(x)$. Wait a minute: isn't that backwards? Shouldn't the ***n***atural ***l***og be written $\text{nl}(x)$? That

would indeed make more sense in English. But the abbreviation is from the Latin ***l**ogarithmus **n**aturalis*.

Figure 3.11 shows a plot of $f(x) = \ln(x)$. Please take a moment to make some statements about the behavior of $\ln(x)$ before reading my remarks below.

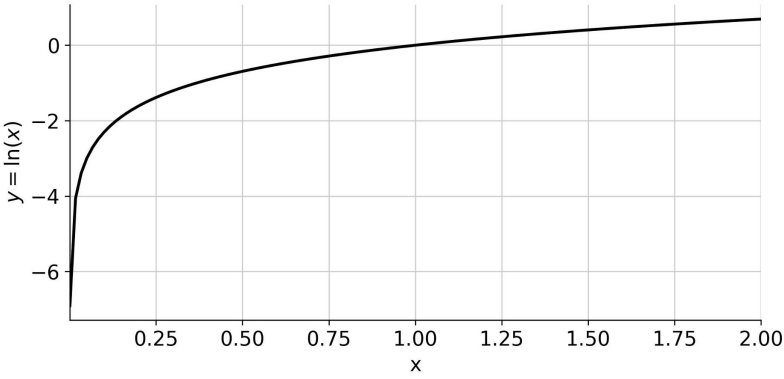


Figure 3.11: Graph of the function $ln(x)$.

There are several important characteristics of the natural log to know about, although not all of these characteristics are obvious from the graph. I will describe these characteristics below, and then write about where the log function comes from and how to interpret it below that.

(**1**) The domain of the natural log function is $x > 0$. The natural log of a negative number is a complex-valued result, and is therefore not considered in this book (except in Exercise 24, which I'm sure you're going to enjoy!), or in many applications. Furthermore, $\ln(0)$ is undefined — not complex-valued, but undefined in the way that $0/0$ is undefined.

(**2**) The range of $\ln(x)$ is $(-\infty, \infty)$. That is, the natural log function dives down to $-\infty$ as x gets arbitrarily close to zero, and it goes up to ∞ as x increases. The rate at which the function increases slows down, but in theory it will eventually reach ∞.

(**3**) $\ln(1) = 0$. The natural log of any number between 0 and 1 is negative, and the natural log of any number greater than 1 is positive. If you are an aspiring data scientist, statistician, or

machine-learning engineer, then this fact is worth committing to memory.

Where does the log function come from? As I mentioned at the outset of this section, the natural log is the inverse function of the natural exponential. Logarithms are defined as being the opposite of powers. Consider the following two equations:

$$x = b^y \tag{3.35}$$

$$y = \log_b(x) \tag{3.36}$$

In other words, the value of y that you put in the exponent of b to get x, is the log of x. b is referred to as the "base" of the log. For example, when $b = 2$, then $\log_2(x)$ is pronounced "log base two of x." b can be any positive number, although there are three bases that are most commonly used: $\log_2$ (used in applications such as entropy), $\log_{10}$ (used in measurements and growth/decay functions), and $\log_e$.

$\log_e$ is so important and widely used that we simply shorten it to ln. The `numpy` function to estimate the natural log of a number is `np.log()` (log-base-2 is `np.log2()` and log-base-10 is `np.log10()`). I will have more to say about the relationship between $\ln(x)$ and e^x when I introduce inverses (Section 3.10).

Equation 3.35 also shows why the log of zero does not exist: nothing raised to a power can be zero, hence there is no solution to Equation 3.35 for $x = 0$.

Notation Final note about the natural log: Some people omit the parentheses, and write $\ln x$. I find the omission potentially confusing, because it is difficult to know whether "$\ln x + 1$" means "$\ln(x) + 1$" or "$\ln(x + 1)$". Thus, I always use parentheses and I recommend you do as well, but be aware of this notational variability. Later in the book, you'll see the natural log of absolute

values, and it is common to use notation like $\ln|x+1|$ instead of $\ln(|x+1|)$.

3.6 Trigonometric functions

Trigonometry is the study of angles. This section is an introduction to the trigonometry that you need to know for the rest of this book. In particular, I will focus on sine, cosine, and tangent: how they are defined, what they mean, and why they are important. You'll learn more about trig functions later in the book, when studying derivatives and integrals. I hope that some or all of the material in this section is familiar to you from high school, and I hope that you find this refresher refreshing.

Sine, cosine, and tangent are often abbreviated sin, cos, and tan.

The word "trigonometry" comes from *trigon* and *metry*; thus, the measurement of a trigon. Trigon is an old-school name for a triangle ("tri" for three, "gon" for shape).

There are several ways to define, conceptualize, and use trig functions. I will introduce three approaches: as ratios, as angles, and as time series. Those are not actually *distinct* approaches; they are different perspectives of the same mathematical constructs.

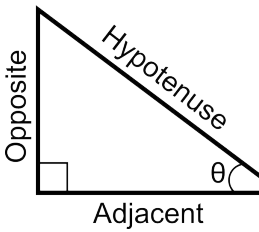


Figure 3.12: Trig functions defined by ratios of sides of a right triangle.

3.6.1 Trig functions as ratios

Trig functions can be defined as the ratios of the sides of a right triangle.

Figure 3.12 shows a right triangle (remember that a "right triangle" has one 90° angle). The side opposite the right angle is called the *hypotenuse*. One of the other angles in the triangle is labeled θ, and the two other sides of the triangle are labeled *adjacent* and *opposite* to that angle.

The units of θ are either radians or degrees, with $rad. = 180°/\pi$.

Cosine, sine, and tangent are all defined as ratios of lengths of

sides (o is opposite, h is hypotenuse, a is adjacent):

$$\sin(\theta) = \frac{o}{h} \tag{3.37}$$

$$\cos(\theta) = \frac{a}{h} \tag{3.38}$$

$$\tan(\theta) = \frac{o}{a} = \frac{\sin(\theta)}{\cos(\theta)} \tag{3.39}$$

One nice property of ratios is that they are scale-less: it doesn't matter if the lengths of the triangles are measured in feet or mm or light-years; the units will cancel and you end up with the same ratios.

When I learned these ratios in school, we learned the mnemonic "soh-cah-toa": soh (sine = opposite/hypotenuse), cah (cosine = adjacent/hypotenuse), and toa (tangent = opposite/adjacent). Perhaps you know that one, or perhaps you learned a different mnemonic.

3.6.2 Trig functions as angles

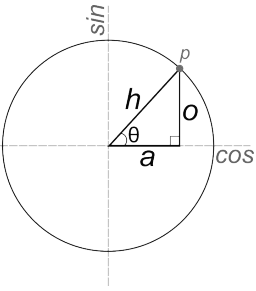


Figure 3.13: Right triangle embedded in a unit circle with the x-axis corresponding to cosine and the y-axis corresponding to sine. a = adjacent, o = opposite, h = hypotenuse, p = the point where the triangle touches the circle. A unit circle means $h = 1$.

Imagine taking a right triangle, scaling it (without changing the angles) such that the length of the hypotenuse is 1 unit, and placing it inside a circle with the θ corner at the origin. Then label the x-axis "cosine" and the y-axis "sine." Each point on the circle is exactly one unit from the origin, so we call this a "unit circle" (Figure 3.13).

When the triangle is normalized to have $h = 1$, then the lengths of a and o are bound between -1 and $+1$, and correspond to the cosine and sine of θ. In other words, $\cos(\theta) = a$ and $\sin(\theta) = o$.

This way of thinking about trig functions is no different from what I presented in the previous subsection. The only change is to normalize the size of the triangle and put it in a circle. Notice that when $h = 1$, the equations I wrote earlier simply such that, for example, $\sin(\theta) = o$, which you can think of geometrically as the length of the vertical line from point p down to the *cos* axis.

3.6.3 Trig functions as wavey lines (time series)

When you think of sine and cosine — especially if you have some exposure to time series analysis, signal processing, or biological or physical systems that change over time — you probably think about those wavey lines going up and down over time. Where do those wavey lines come from and how are they related to the triangles and circles I described above?

Consider point p in Figure 3.13. Imagine that point careering around the unit circle in counter-clockwise motion. That is, Figure 3.13 shows one snapshot in time, and then at the next time point, p moves a little bit north-by-northwest towards the *sin* axis, without leaving the circle. This means that point p is always exactly 1 unit away from the origin. The triangle changes shape to stay with point p, although it remains a right triangle. As p moves towards the *sin* axis, a gets smaller while o gets bigger. After p crosses the *sin* axis, it continues its path to the left, now starting to move down towards the *cos* axis. During this part of its trajectory, a gets increasingly negative while o remains positive but shrinking.

Keep going with this animation in your imagination. You should be able to visualize that as point p traverses the circle, a bounces smoothly from $+1$ to 0 to -1 to 0 back to $+1$, and so does o. We can plot these values on the y-axis of a graph where the x axis is "time" in the journey of p. That's shown in Figure 3.14.

I really hope you made the mental effort to imagine this animation. It's challenging to see it all in your head, but making the effort to visualize mathematics in your mind is a really effective tool for understanding. I now encourage you to find an animation of this online[1]. Then come back to the static world of textbooks and try to imagine the animation again.

Tangent function Tangent is a ratio of sine to cosine. Although the sine and cosine functions are individually bounded by -1 and

[1] e.g., https://en.wikipedia.org/wiki/Sine_and_cosine

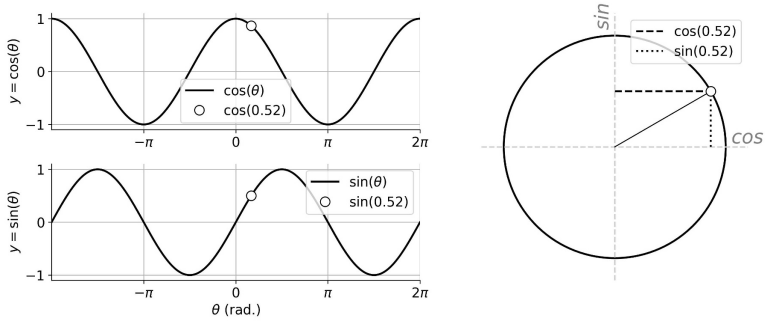


Figure 3.14: The ratio of opposite to adjacent sides for a particle on a unit circle (small open circle in the unit circle) projected onto the cosine and sine axes (dashed and dotted lines) can be drawn on a "wavey line plot" (line plots on the left), as the particle moves counter-clockwise around the unit circle.

+1, the ratio between them is unbounded. Indeed, as cosine approaches zero (twice each cycle), the tangent function blows up to ∞ or down to $-\infty$.

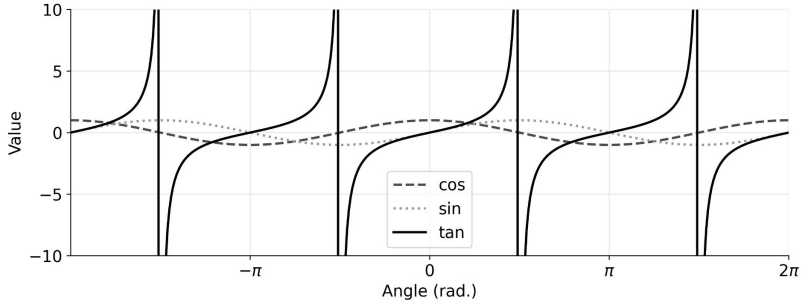


Figure 3.15: Tangent (solid black line) is the ratio of sine to cosine.

3.6.4 Trig identities

There are many interesting properties of trig functions, some of which are called "identities" because they involve equals signs. There are a few dozen trig identities; many are either obscure and you'll never use them, or may be useful very occasionally and you can look them up online as needed.

Arguably the most important trig identity is the "Pythagorean

identity," which describes how the two sides relate to the hypotenuse of a triangle on a unit circle:

$$\sin^2(\theta) + \cos^2(\theta) = 1 \tag{3.40}$$

In the context of Figure 3.13, Equation 3.40 says that $a^2+o^2 = h^2$. Of course you recognize this as the Pythagorean theorem. When embedded in the unit circle, $h = 1$. We can also show this by writing out Equation 3.40 using the ratio definitions of cosine and sine:

$$\cos^2(\theta) + \sin^2(\theta) = 1 \tag{3.41}$$

$$\left(\frac{a}{h}\right)^2 + \left(\frac{o}{h}\right)^2 = 1 \tag{3.42}$$

$$\frac{a^2}{h^2} + \frac{o^2}{h^2} = 1 \tag{3.43}$$

$$\frac{a^2 + o^2}{h^2} = 1 \tag{3.44}$$

$$a^2 + o^2 = h^2 \tag{3.45}$$

Other trig identities you'll need for this book include the angle sum and angle difference identities, but I'll introduce those when they are necessary to prove some differentiation rules. Many trig identities are not intuitive and are difficult to memorize. Here's one, for example:

$$\sin(\theta)\cos(\phi) = \frac{\sin(\theta+\phi) + \sin(\theta-\phi)}{2}$$

Please don't try to memorize, or derive on your own, that particular identity. It's just an example of what many trig identities look like.

Notation Final note for this section. To indicate the product of the cosine function with itself, write $\cos^2(\theta)$, not $\cos(\theta)^2$. $\cos(\theta^2)$ means to multiply θ by itself, before the cosine function is applied. Same notation for the other trig functions.

Also, sometimes people drop the parentheses and write $\cos\theta$, just like with the natural log. As I wrote in that discussion, I dislike

this notation because it risks confusion. I always use parentheses and I recommend that you do as well, but be aware that not everyone agrees with or follows that convention. You know, some things in life are worth stressing about and fighting for, and other times you just gotta ¯_(ツ)_/¯

3.7 Piecewise functions

A piecewise function has multiple pieces. Quite straightforward. The idea is to segment the domain into two or more restricted domains, and define a separate function for each subdomain. Piecewise functions are described using the "cases" notation. Here's an example:

$$f(x) = \begin{cases} x & , \quad x < 0 \\ -x^2 + 3 & , \quad x \geq 0 \end{cases} \tag{3.46}$$

In the cases notation, each row corresponds to a piece of the function; the expression in the first column (before the comma) is the definition of that piece, and the expression in the second column (after the comma) indicates the subdomain for that piece. There are several different ways to indicate piecewise functions, although they're all quite interpretable. For example, Equation 3.46 could also be written as:

This example has two pieces, but piecewise functions can have any number of pieces.

$$f(x) = \begin{cases} x & \text{for } x < 0 \\ -x^2 + 3 & \text{otherwise} \end{cases} \tag{3.47}$$

There are two things to be mindful of when working with piecewise functions: (**1**) function characteristics, such as the domain, range, linearity, derivative, and integral, need to be considered separately

for each piece. (**2**) Piecewise functions can have gaps in the domain where the function is undefined. For example, imagine if the subdomain of the second piece in Equation 3.46 was $x > 0$ (making the function undefined for $x = 0$) or $x \geq \sqrt{3}$ (making the function undefined between 0 and $\sqrt{3}$). In the next section, you will learn that these gaps are called "discontinuities" and are important for understanding and differentiating functions.

Here's another example of a piecewise function. See if you recognize and can name this function before reading the paragraph below.

$$g(x) = \begin{cases} x & \text{when} \quad x \geq 0 \\ -x & \text{when} \quad x < 0 \end{cases} \tag{3.48}$$

Have you figured it out? The function returns x when x is positive, and $-x$ when x is negative. It's commonly known as the absolute value function[2]: $g(x) = |x|$.

One more example: Function $h(x)$ in Equation 3.49 is called the "ReLU" (rectified linear unit) function, and is used in artificial neural networks. In particular, this function inserts a nonlinearity into deep learning models that allow the models to learn complex mappings such as a set of pixel intensity values in a picture to a label "cat."

$$h(x) = \begin{cases} 0 & , \quad x < 0 \\ x & , \quad x \geq 0 \end{cases} \tag{3.49}$$

Figure 3.16 shows graphs of the three piecewise equations above.

Piecewise functions can be tricky to implement and visualize correctly. In particular, notice that the black line for function $f(x)$

[2]A more general definition of the absolute value function is the L2 (Euclidean) distance from the origin of $\mathbb{R}$ or $\mathbb{C}$; this definition generalizes to complex-valued numbers.

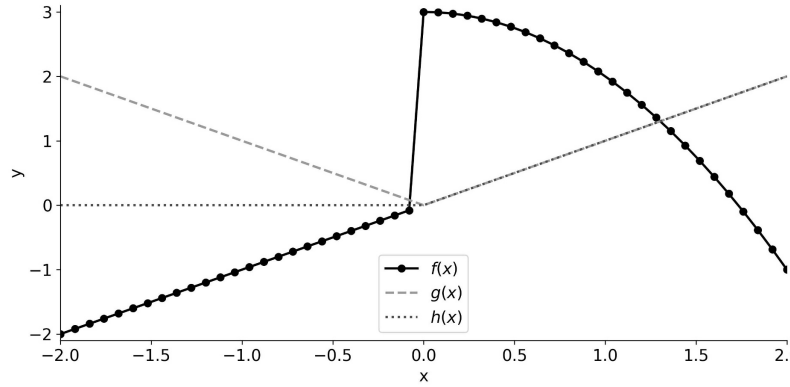


Figure 3.16: Visualization of the three example piecewise functions in this section.

appears to increase linearly between the two pieces around $x = 0$. This is *misleading* — it's an artifact of using lines to connect discrete function points with low resolution in combination with the break between function pieces. In fact, the function goes from $y \approx 0$ to $y = 3$ with no intermediate values.

Fortunately, it is possible to draw piecewise functions accurately, even with limited x-axis resolution. You'll learn how to code and draw piecewise functions using `numpy` and `scipy` in Exercises 9-14.

3.8 Continuity and discontinuities

Points on functions can be characterized as "continuous" or "discontinuous." I'm sure you already have an intuition for what those terms mean — and your guess is probably correct. In this section you will learn the formal definition of function continuity, and you'll learn four types of discontinuities. These are important because certain mathematical operations and applications are valid only for continuous functions, and other operations can be adapted to functions with discontinuities but those adaptations depend on the type of discontinuity.

Intuitively, a function is continuous if its graph passes the "finger tracing" test. The finger tracing test means you run your finger along the line of the function; if you can trace the function without lifting your finger off the page, then the function is continuous within the restricted domain where you started and stopped the test.

Figure 3.17 illustrates the concept. The figure shows different *types* of discontinuities that I'll introduce later; for now, determine which functions pass the "finger tracing" test and are therefore continuous.

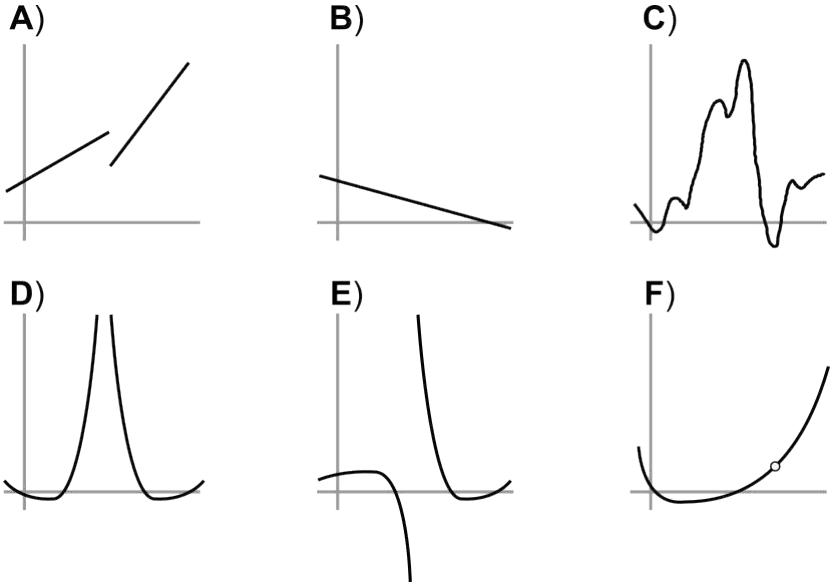


Figure 3.17: Examples illustrating continuous (B and C) functions and functions with discontinuities. Panel A shows a *jump discontinuity*, panels D and E show *infinite discontinuities*, and panel F shows a *removable discontinuity* (the small white circle represents a missing point on the line).

The finger tracing test is a good elementary-school-level intuition, but it's not formal enough, in part because discontinuities might exist outside the plotting area, and in part because some discontinuities may be undetectable from visual inspection.

3.8.1 Formal definition of continuity

The formal definition of continuity relies on the concept of *limits*. I briefly introduced limits in the previous chapter, and you'll learn a lot more about them in the next chapter. For now, try to focus on the concepts as indicated by the equations and my explanation in the following paragraph.

A function $f(x)$ is continuous at $x = a$ if:

1. $f(a)$ is defined.
2. $\lim_{x \to a} f(x)$ exists.
3. $\lim_{x \to a} f(x) = f(a)$.

What do these three conditions mean? First of all, notice that this definition concerns continuity *at a specific point*; the definition is not about the entire function. Condition 1 states that the value of the function at point $x = a$ must be defined. As an example, the function $f(x) = 1/x$ is defined at $x = 1$ but not at $x = 0$. So $f(x)$ is not continuous at $x = 0$.

Condition 2 states that the limit as x approaches a must exist. Again consider $f(x) = 1/x$. When $a = 1$, then as x gets really really close to — but not exactly equal to — 1, then the function approaches the same value when x is a tiny bit less than one, and when it is a tiny bit greater than one. There's more to say about when limits do and do not exist, and you'll learn the details in the next chapter. But for some intuition, have a look at Figure 3.17E: the function takes on different values as it approaches the discontinuity from the left vs. from the right.

Finally, condition 3 states that the limit of the function as x approaches a is the same as the function value at $x = a$. Consider that when x is extremely close to 1, then that function value is the same as $f(1)$. Figure 3.17A shows an example of a function that violates this condition. Imagine point a being the breakpoint between the two pieces; notice that the value of the function is different when approaching the breakpoint from the left vs. from

the right.

Continuity is defined for a specific value of x. This means that functions can be continuous within certain restricted domains while having discontinuities within other restricted domains. That said, it is common to refer to *continuous functions* as those that are continuous for all points $a \in x$, and *discontinuous functions* as those that have at least one discontinuity.

3.8.2 Types of discontinuities

There are several types of discontinuities. The descriptions below introduce four types.

Jump discontinuity (panel 3.17A)
A jump discontinuity occurs when a function suddenly transitions from one value to another, without any y-axis values in between. Many piecewise functions have jump discontinuities.

Infinite discontinuity (panels 3.17D-E)
Here the function blows up to $+\infty$ or $-\infty$. This type of discontinuity is observed in functions with x in the denominator, such as $1/x$.

Removable discontinuity (panel 3.17F)
These discontinuities are tricky, because they are often invisible on a graph (textbook authors may draw a circle on the plot as I've done, but that's done out of kindness, not because it's what the function really looks like). It may seem strange that a function can look so smooth and yet be missing one point. This can arise from a piecewise function that specifies an exception for one particular value of x, or this can arise from non-piecewise functions like this one:

$$f(x) = \frac{x^2 - 2x}{x^2 - 4}$$

Notice that $f(2) = 0/0$, but the function doesn't approach ∞ as x gets close to 2; there is simply a point missing in the function at one value of x.

Oscillating discontinuity

This type of discontinuity is not illustrated in the figure, but you'll learn about it in Exercise 3.16. Oscillating discontinuities involve functions that cycle up-and-down faster and faster as they approach a point, without settling onto any specific value.

3.9 Composite functions

Composite functions are a way of combining multiple functions into one. The idea is to embed one function inside another function, and that resulting new function is a *composite* of the two functions. You can compose any number of functions, but I'll start this section with two functions to introduce the concept.

What does it mean to compose two functions? Remember that a function is a set of rules that maps one variable to another: The function $f(x)$ transforms x into y. With composite functions, we take the output of $f(x)$ and push that through another function $g(x)$. Figure 3.18 depicts the general idea of transforming x into y and then into z.

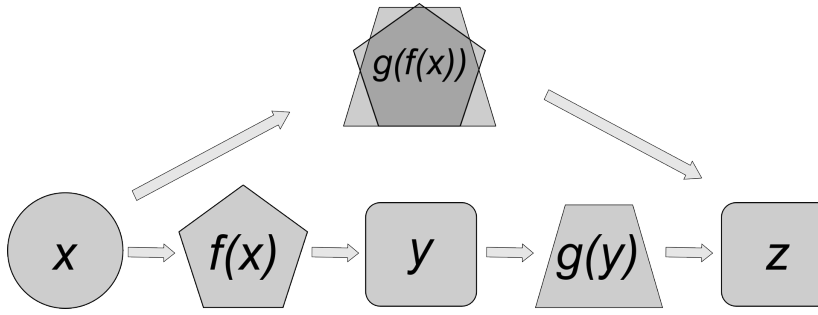


Figure 3.18: Concept of composing two functions.

Here's an example:

$$f(x) = 2x + \cos(x) \tag{3.50}$$

$$g(x) = x - 2 + \sqrt{|x|} \tag{3.51}$$

$$h(x) = f(g(x)) \tag{3.52}$$

$$= 2\left(x - 2 + \sqrt{|x|}\right) + \cos\left(x - 2 + \sqrt{|x|}\right) \tag{3.53}$$

Notice that the entirety of $g(x)$ is swapped in for x in $f(x)$, in each place where x appears.

The notation of a composite function is either $f(g(x))$ or $(f \circ g)(x)$. I prefer the former notation because it makes the sequence of embeddings obvious.

3.9.1 Commutativity of composite functions

I defined function $h(x)$ in Equation 3.53 as $f(g(x))$; would the definition of $h(x)$ be the same if I had defined it as $g(f(x))$?

The answer is no; it would have been a different function. More generally, $f(g(x)) \neq g(f(x))$. To be sure, there are special cases of function pairs where the order doesn't matter — and you'll learn about one of those exceptions in the next section — but those are exceptions.

This order relates to a property called *commutativity* of composite functions. As a reminder: Commutativity in math means that the order doesn't matter. The commutativity property does hold for many operations in mathematics, including addition ($2+3 = 3+2$) and multiplication ($2 \times 3 = 3 \times 2$), although it does not hold for other operations such as subtraction ($2 - 3 \neq 3 - 2$) and division ($2/3 \neq 3/2$). And it does not hold for composite functions ($f(g(x)) \neq g(f(x))$).

Figure 3.19 visualizes the two functions listed above (Equations 3.50 and 3.51), and the two ways of ordering their composition. If these two functions were to commute, then the two composition orders would overlap perfectly and you'd see only one line in panel B.

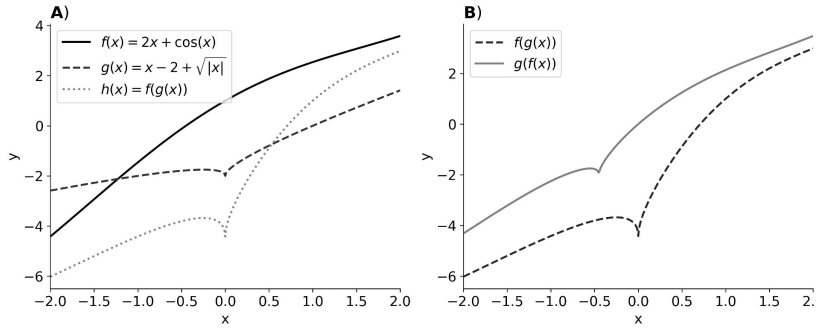


Figure 3.19: Example of two functions, their compositions, and their lack of commutativity.

3.9.2 Multiple composite functions

Function composition is not limited to two; you can compose any number of functions. Here is an example of three functions being composed.

$$f(x) = \sin(x) \tag{3.54}$$

$$g(x) = \sqrt{x} \tag{3.55}$$

$$h(x) = 2x^2 + 5 \tag{3.56}$$

$$f(g(h(x))) = \sin\left(\sqrt{2x^2 + 5}\right) \tag{3.57}$$

It's good practice to think about the domains of these functions, and how some functions may restrict the domains of functions into which they are embedded. In the equations above, please determine the domains of each individual function, and the domain of the composed function. Please do this before reading the next paragraph.

The answers are that functions f and h have no restrictions on the domain, so their domains are $-\infty < x < \infty$. Function g is limited to non-negative numbers, so $0 \leq x < \infty$. But does the domain restriction on g have consequences for the domain of the composed function? Actually, the answer is *No*, because

even when x is negative, composing h into g guarantees function validity even for $x < 0$.

3.9.3 Applications of composite functions

It may seem strange to have composite functions: Why not just define $h(x)$ right from the start and avoid wasting space and ink by separately defining the individual composite functions?

The main purpose of composite functions is to make our lives easier. In the toy examples I showed above, it is indeed unnecessary to define $f(x)$ and $g(x)$ separately. But real-world applications may have a plethora of functions that need to be embedded with different orders depending on different circumstances. Here is one example: A time series signal $s(t)$ may pass through multiple filters before it can be interpreted. The final signal can be expressed as $y(t) = f_3(f_2(f_1(s(t))))$. And perhaps a different signal requires different processing and is defined as $y_2(t) = f_1(f_3(s_2(t)))$.

Relatedly, some functions are really long and confusing when written out entirely, and so breaking up the function into the composite of multiple functions helps the audience understand and interpret the math. Consider the following example (this is Welch's test from statistics; don't worry about what the variables mean):

$$t = \frac{\overline{X} - \overline{Y}}{\Theta}$$

$$\Theta = \sqrt{\frac{s_x^2}{n_x} + \frac{s_y^2}{n_y}}$$

$$s_x = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Imagine writing out the complete definition of t in one equation! That single equation would be really difficult to interpret, whereas

expressing the function as the composite of simpler functions helps express both the conceptual and mechanical details.

3.9.4 Composite functions in coding

Consider the following Python code:

```
x = [1,1,2,3,5,8,13]
y = np.linspace(0,1,len(x))
z = x + np.sqrt(y)
```

Notice the parallels to composite functions in math: The output of one function (`len()`) is used as the input to another function (`np.linspace()`). I could have created variable `z` without intermediate variables:

```
z = [1,1,2,3,5,8,13] + np.sqrt(np.linspace(0,1,7))
```

But decomposing the definition of `z` into multiple lines has several advantages, including ease of interpretation and the possibility of using variables `x` and `y` later in the code. The point is that composite functions in mathematics and composite functions in coding are conceptually similar.

Coding also helps explain why the commutivity principle does not hold for composite functions. Consider the following code:

```
def f(x): return x+x**2
def g(x): return np.log(abs(x+1))

print(f(g(3))) # result: 3.31
print(g(f(3))) # result: 2.56
```

The nonlinear transformations in the functions ensure that `f(g(x))` and `g(f(x))` are different.

A general point here is that, although pure math and mathematical coding are not the same, many concepts overlap sufficiently that understanding one helps to learn the other.

3.10 Inverse functions

Inverse function: the mathematical Ctrl-z.

An *inverse function* is one that negates, or "undoes," another function. Before explaining inverse functions, I will begin by introducing inverse numbers[3]. For example, imagine we want to isolate y on the left-hand-side of the equation:

$$4y + 1 = 3x^3 \tag{3.58}$$

$$4y = 3x^3 - 1 \tag{3.59}$$

$$\frac{1}{4}4y = \frac{1}{4}(3x^3 - 1) \tag{3.60}$$

$$y = \frac{3x^3 - 1}{4} \tag{3.61}$$

In this case, -1 is the additive inverse of 1 and $1/4$ is the multiplicative inverse of 4. The key points are (**1**) we can derive a number that "cancels" another number, (**2**) there are different inverses depending on the context (e.g., additive vs. multiplicative inverses), (**3**) not all numbers have an inverse (consider that the multiplicative inverse of 0 cannot be $1/0$).

Back to functions: function $g(x)$ is called an "inverse" of function $f(x)$ if $g(f(x)) = x$. In other words, an inverse function "cancels" the transformation of x. Let's see an example.

$$f(x) = 2x + 3 \tag{3.62}$$

[3] You actually can consider this example to be about functions, for example by defining the number 4 to be the function $f(x) = 4$.

$$g(x) = \frac{x-3}{2} \tag{3.63}$$

To confirm that g is an inverse of f, we can test the composition:

$$g(f(x)) \;=\; \frac{(2x+3)-3}{2} \;=\; \frac{2x}{2} \;=\; x \tag{3.64}$$

What would happen if we swapped the order of f and g? Let's find out:

$$f(g(x)) \;=\; 2\left(\frac{x-3}{2}\right)+3 \;=\; x-3+3 \;=\; x \tag{3.65}$$

Interestingly, $f(g(x)) = g(f(x))$, i.e., these two functions commute. That's not just a quirk of this one example; it is *always* that case that if g is the inverse of f, then f is also the inverse of g. Thus, the commutativity property of function composition holds if the functions are inverses of each other. (There may be domain restrictions on the commutativity; I'll get back to this later.)

Notation It is tedious to keep writing out "this function is the inverse of this other function." Therefore, mathematicians agreed to use a common notation for inverse functions: $f^{-1}(x)$. Thus, an inverse function is one for which the following statement is true:

$$f^{-1}(f(x)) \;=\; x \;=\; f(f^{-1}(x)) \tag{3.66}$$

There is a potential point of confusion with this notation: The inverse of a function is generally not the reciprocal of that function. In particular:

$$(f(x))^{-1} = \frac{1}{f(x)} \quad \Rightarrow \quad \text{Not the inverse} \tag{3.67}$$

$$f^{-1}(x) \quad \Rightarrow \quad \text{Yes the inverse} \tag{3.68}$$

Why do we use this notation? Partly because it helps link to the concept of the multiplicative inverse of a number: 3^{-1} is the inverse of 3, because $1/3 \times 3 = 1$. For numbers, the multiplicative inverse actually is the same thing as the reciprocal (Equation 3.67), but that's generally not the case for functions.

Secondly, you can think about the $^{-1}$ as being a multiplicative iteration on the function. For example, if we consider that $f^2(x)$ means $f(x)f(x)$, then $f^{-1}(x)f^2(x) = f^1(x)$, and therefore, you can think of $f^{-1}(x)f^1(x) = f^0(x) = x$. That's conceptual but I hope it helps. Perhaps an abstract diagram will also help (Figure 3.20).

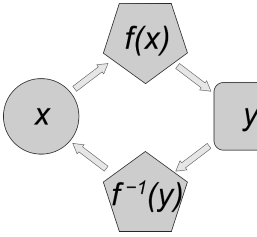


Figure 3.20: Abstraction of x being transformed into y via $f(x)$, only to be transformed back into x via $f^{-1}(y)$.

3.10.1 How to find the inverse of a function

Conceptually, computing the inverse function is easy: You just write out the function as $y = f(x)$, solve for x, and then swap variables x and y. Here's an example.

$$f(x) = 2x + 3 \tag{3.69}$$

$$y = 2x + 3 \tag{3.70}$$

$$y - 3 = 2x \tag{3.71}$$

$$\frac{y-3}{2} = x \tag{3.72}$$

$$\frac{x-3}{2} = y \; = \; f^{-1}(x) \tag{3.73}$$

That's the idea of finding a function inverse. You'll go through some more examples in the exercises, but here's the thing: in modern applications, you will never actually need to hand-calculate a function inverse. So please commit to memory the *idea* of the function inverse as "solve for x, swap y and x", and in practice you use libraries like `sympy` to find the inverse for you.

3.10.2 Which functions have inverses?

Not all functions have an inverse. For a function to be invertible, it must be bijective. To understand why, look at Figure 3.21 (this is Figure 2.11 redrawn); inverting a function means swapping x and y, which here can be conceptualized as reversing the arrowheads. The functions depicted in panels B, C, and E cannot be inverted because that would violate the "each input to one output" part of the definition of a function. Only the function depicted in panel D can be inverted.

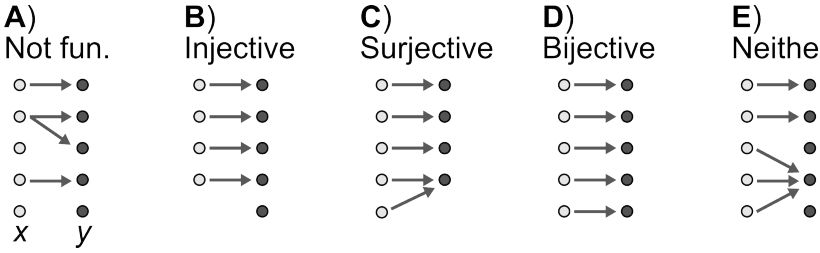


Figure 3.21: Same as Figure 2.11, redrawn here for your convenience.

Thus, a function can be inverted only if it is bijective. The visual intuition is that a function can be inverted only if it passes both the horizontal and the vertical line tests.

That said, many functions can be inverted on a restricted domain where bijectivity exists. One example is the function $f(x) = x^2$; that function is not bijective and therefore has no "true" inverse. However, if we consider the restricted domain of $x \geq 0$, then we can define $f^{-1} = \sqrt{x}$. You can think of this as a "partial inverse" or a "restricted inverse" — those are not formal math terms but might help you understand the concept.

3.10.3 Domain and range of inverse functions

Because computing the inverse involves swapping x and y, the domain and range of the inverse function also swaps. That is, the domain of $f^{-1}(x)$ is the range of $f(x)$, and the range of $f^{-1}(x)$ is the domain of $f(x)$.

However, it is not always as simple as swapping the domain and range; as I mentioned above, some function inverses are defined only within a subdomain of the function's range. Here's another example:

$$f(x) = 3x^2 + 4 \tag{3.74}$$

$$y - 4 = 3x^2 \tag{3.75}$$

$$\frac{y-4}{3} = x^2 \tag{3.76}$$

$$\sqrt{\frac{y-4}{3}} = x \tag{3.77}$$

$$\sqrt{\frac{x-4}{3}} = f^{-1}(x) \tag{3.78}$$

The question at hand is whether the domain and range of Equation 3.74 respectively equal the range and domain of Equation 3.78. Here are the domains and ranges:

$$D_f = (-\infty, \infty)$$

$$R_f = [4, \infty)$$

$$D_{f^{-1}} = [4, \infty)$$

$$R_{f^{-1}} = [0, \infty)$$

The range of f does match the domain of f^{-1}, but the range of f^{-1} is merely a subset of the domain of f. Why is that the case? The original function had a squared term so its inverse contains a square root. Square roots are not defined for negative numbers, so f^{-1} cannot be defined for $x < 4$. Of course, you could create a complementary inverse function with $-\sqrt{...}$ that would have a range of $(-\infty, 0]$.

The key take-home is that nonlinear operations can limit the domain or range of the function inverse. The function and its inverse shown above are illustrated in Figure 3.22. Please inspect the figure and consider the discussion above. I will make several remarks below the figure.

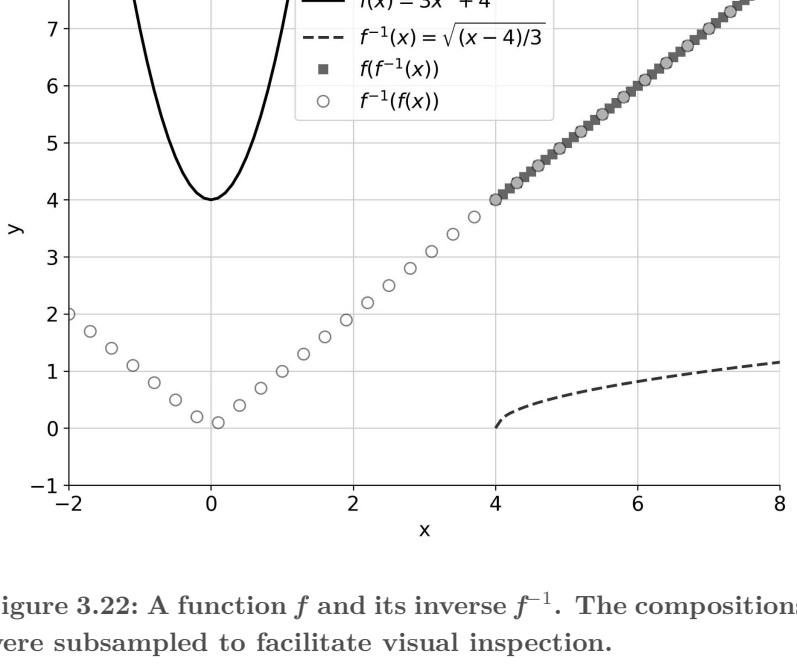


Figure 3.22: A function f and its inverse f^{-1}. The compositions were subsampled to facilitate visual inspection.

Remarks. You can see that the domain (x-axis) of f^{-1} equals the range (y-axis) of f, but that the range of f^{-1} is strictly positive whereas the domain of f allows $x < 0$. Another interesting observation is the differences between their compositions: $f(f^{-1}(x)) = f^{-1}(f(x))$ for $x \geq 4$ but $f(f^{-1}(x))$ is not defined for $x < 4$. This means that f^{-1} is an inverse function of f only in the domain of f^{-1}. Using my non-technical terminology from earlier: f^{-1} is a "restricted inverse" because the function is invertible only in a restricted domain. You could capture the "full" inverse by creating a complimentary inverse function that contains the negative square root for $x < 4$.

When you run the code that creates this figure, you'll get the following warning message:

```
<ipython-input-79-ef5dd179f371>:3: RuntimeWarning: inva
value encountered in sqrt def f1(x): return np.sqrt((x-4
```

The coding interpretation of this warning message is that there are errors when `x` is less than 4, because the code is not designed to handle complex-valued numbers. The math interpretation of this warning message is that there are input values outside the domain of the function.

The broader take-home message of this example is that compositions, domains, and ranges can be tricky. These are important features of functions to understand, and don't rush through determining them.

Because inverse functions allow you to "undo" a mathematical transformation, they are used in applications like control engineering, optimization, statistics, and machine learning.

3.10.4 Inverse trig functions

Here's a question: Can you solve for x in $\cos(x) = y$?

The answer is *yes*, it is possible to solve for x. But the solution is not $y/\cos$. Instead, you need to invert the trig function.

The inverse of the sine function is called the arcsine function, and is indicated as arcsin, asin, or $\sin^{-1}$. Thus:

$$x = \sin(y) \tag{3.79}$$

$$\sin^{-1}(x) = y \tag{3.80}$$

$$x = \cos(y) \tag{3.81}$$

$$\cos^{-1}(x) = y \tag{3.82}$$

An example sine wave and arcsine are shown in Figure 3.23. What

do you notice about the graphs?

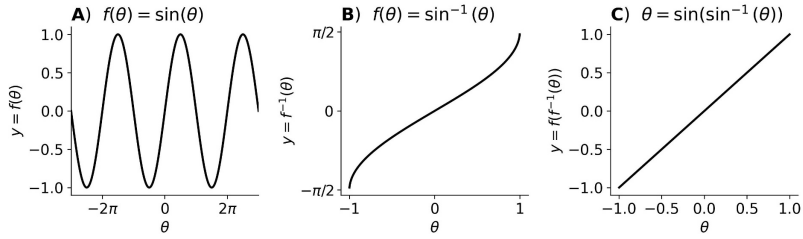


Figure 3.23: A sine, its inverse, and their composition.

The straight diagonal line produced by their composition in panel C is visual verification that the two functions are inverses of each other.

But much more striking is the restricted domain of the arcsine function (panel B). Well, it might be striking, but is it surprising? It shouldn't be, when keeping in mind that domains and ranges swap in function inverses. The range of the sine function is $[-1, 1]$, and that's the domain of the arcsine function.

On the other hand, the range of arcsine is $[-\pi/2, \pi/2]$, which is not the domain of sine. Why is the range also restricted? I'll answer the question in the next paragraph, so please think of your own answer before then.

The reason why the range of arcsine is restricted is that the sine function is not bijective, and therefore has no "full inverse." However, the function is bijective in a restricted domain, and that restricted domain is the range of arcsine.

Figure 3.24 shows the sine and arcsine functions in the domain of $[-1, 1]$ (panel A), and the cosine and arccosine in the same domain (panel B).

The lines in panel C illustrate the two inverse trig functions on the x-axis with θ on the y-axis. There are several interesting properties of inverse trig functions, some of which you will encounter in later chapters and some are relevant for particular applications but are not further discussed here. One property I find interesting is that $\sin^{-1}(\theta) + \cos^{-1}(\theta) = \pi/2$ for any θ in the domain.

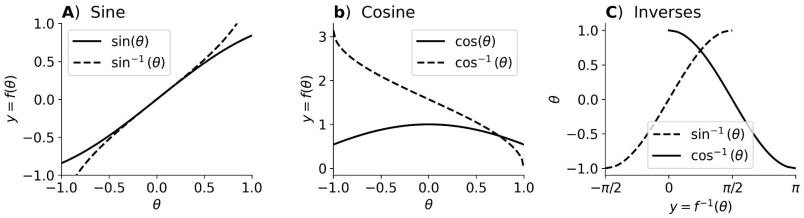


Figure 3.24: Sine and arcsine.

3.11 Function symmetry and parity

I introduced the term "symmetry" in the context of polynomial functions. Now it's time to learn the formal definition of function symmetry, and what it means for a function to be even, odd, or neither.

"Parity" means odd or even. *Function* parity is different from *number* parity (1 and 3 vs. 2 and 4).

Let's start with some visual intuition. Take a moment to inspect Figure 3.25, and see if you can understand why panels A and B show even functions, C and D show odd functions, and E and F show functions that are non-symmetric (neither even nor odd). After developing an hypothesis about what these characterizations mean, read my descriptions and definitions below.

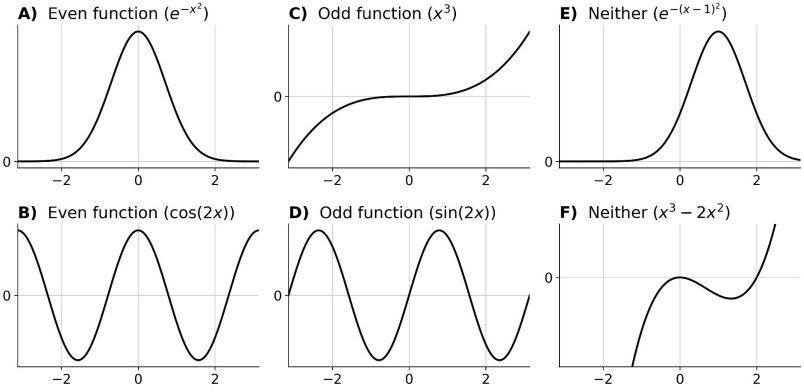


Figure 3.25: Examples of functions that are even, odd, and neither.

Even functions ($f(-x) = f(x)$)

Geometrically, a function is even if it is mirrored across the $x = 0$ line. That is, imagine printing out the graph on a piece

of paper and folding the paper along the vertical $x = 0$ line. Is the function exactly the same for $-x$ as $+x$? If so, then the function is even.

Formally, a function is even if $f(-x) = f(x)$. That is, input a negative number and the output is the same as when you input the positive of that number. That's clearly the case for $f(x) = x^2$: $f(-4) = 16 = f(4)$.

Odd functions ($f(-x) = -f(x)$)

Geometrically, a function is odd if its graph is the same when flipping the axes twice: first flip across the vertical $x = 0$ line, then again across the $y = 0$ line. Have a look at Figure 3.25C: the function value at $x = 3$ is not the same as at $x = -3$, but it is the same when looking across the y-axis.

Formally, a function is odd if $f(-x) = -f(x)$. In other words, the absolute value of the output is the same, but the sign flips. Consider the function x^3: $f(2) = 8$ while $f(-2) = -8$. Same magnitude but opposite sign. The two negative signs (one on x and one on f) is why you need to flip both axes to get the mirror effect.

Neither ($|f(x)| \neq |f(-x)|$)

Functions that are even or odd are called "symmetric" (thus, even-symmetric or odd-symmetric). But not all functions are symmetric; if a function is neither even nor odd, then it is nonsymmetric (Figure 3.26). Any function for which $|f(x)| \neq |f(-x)|$ is neither odd nor even.

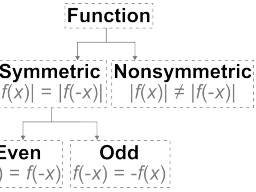


Figure 3.26: Taxonomy of function parity.

Now that you've read those descriptions, please inspect Figure 3.27, which shows more examples plus arrows to help you conceptualize "flipping" the axes.

3.11.1 Determining function parity

How do you determine if a function is even, odd, or neither?

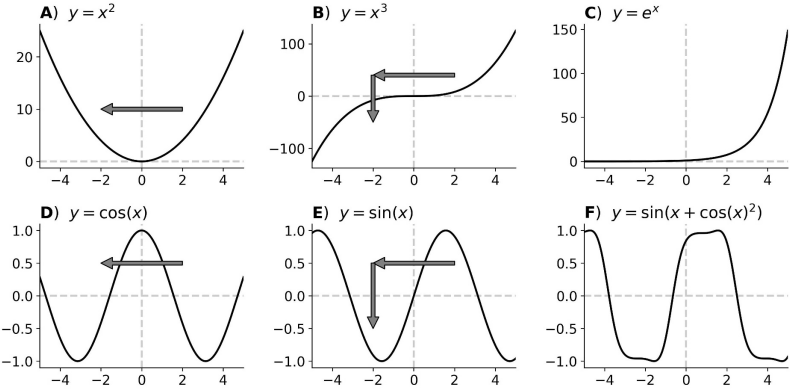


Figure 3.27: More examples of functions that are even, odd, and neither. The arrows indicate how to flip the axes to mirror the symmetry.

Memorization The parity of some functions is easy to memorize. For example, any even-order polynomial (that is, x^n for any even integer n) is an even function, while any odd-order polynomial (x^m for any odd integer m) is an odd function. The cosine function is even while the sine function is odd.

Be mindful that shifts in the function can break symmetry. For example, $f(x) = (x-1)^2$ is not an even function, because $f(2) = 1$ but $f(-2) = 9$.

Visual inspection This will sound familiar to you by this point in the book: Visualizing math functions is one of the best ways to build lasting and deep intuition about math, but you should never rely exclusively on a plot to make inferences about a function. A certain function may *look* even on a graph when in fact it is not, due to small shifts in the function or due to misrepresentation in the plot from a restricted plotting subdomain or low x-axis resolution.

That said, looking at a graph of a function can give you quick and low-effort guess about whether that function is even, odd, or neither, and then you can confirm your hypothesis by looking at the equation.

Test with arithmetic This approach involves picking 2-3 numbers that seem easy to calculate based on the function definition, and then evaluating the function for the positive and negative of those functions. That is, calculate $f(a)$ and $f(-a)$ for a few different values of a.

For example, given the functions $f(x) = 4|x|$ and $g(x) = 4x$, you can determine that $f(3) = 12 = f(-3)$, while $g(3) = 12 = -g(-3)$. On the other hand, for function $h(x) = 4|x + 1|$, $h(3) = 16 \neq |h(-3)|$. Thus, h is neither even nor odd.

This approach is fine for practicing your mental arithmetic and can be useful in some cases, but it is risky to rely on, because you might pick numbers that give a false impression. For example, $g(0) = g(-0)$, although that function is not even. This is why it's important to test several different values.

Test with algebra Instead of using specific numbers, you can input $-x$, simplify, and compare the result against the definition of $f(x)$. Here are two examples of even functions:

$$f(x) = x^2 \tag{3.83}$$
$$f(-x) = (-x)^2 = x^2 \tag{3.84}$$

$$g(x) = x^4 + |x|\cos(x) \tag{3.85}$$
$$g(-x) = (-x)^4 + |-x|\cos(-x) \tag{3.86}$$
$$= x^4 + |x|\cos(x) \tag{3.87}$$

In both cases, the function was identical regardless of whether x or $-x$ was used. Now consider the following two examples:

$$h(x) = x^3 \tag{3.88}$$
$$h(-x) = (-x)^3 = -x^3 \tag{3.89}$$

$$k(x) = x^5 + \sin(x) \tag{3.90}$$
$$k(-x) = (-x)^5 + \sin(-x) \tag{3.91}$$
$$= -\left(x^5 + \sin(x)\right) \tag{3.92}$$

For functions h and k, inputting $-x$ led to $-h$ and $-k$. Same numerical value, opposite sign; hence, odd symmetric functions.

And finally, two examples of nonsymmetric functions:

$$l(x) = (x-1)^2 \tag{3.93}$$
$$l(-x) = (-x-1)^2 \tag{3.94}$$

$$m(x) = x^3 - 4x^2 \tag{3.95}$$
$$m(-x) = (-x)^3 - 4(-x)^2 \tag{3.96}$$
$$= -x^3 - 4x^2 \tag{3.97}$$

For both l and m, inputting $-x$ leads to a result that is distinct from $+x$, but does not equal the negative of the function (e.g., $m(-x) \neq -m(x)$).

Testing with `sympy` In practice, evaluating function symmetry by hand can become infeasible due to the complexity of the functions. In these cases, let Python do the hard work. Perhaps you already have an idea of how to use `sympy` to determine function symmetry. I'll walk you through the procedure in Exercise 22.

Are most functions even, odd, or neither? "Most" is a difficult word to use in math, because of infinities: There is an infinite number of even, odd, and nonsymmetric functions.

On the other hand, it would make sense to ask whether most of the functions you'll work with in particular applications are even,

odd, or nonsymmetric; but the answer to that question depends on the particular application.

3.11.2 Why is function symmetry important?

There are three reasons to know about the concept of function symmetry.

First, symmetry endows functions with interesting properties that are useful in calculus, for example in integration. In fact, knowing that a function is symmetric can save you time and effort when integrating functions by hand.

Second, symmetry is an interesting feature of the mathematical universe to know about. Many physical and biological systems exhibit symmetry (e.g., snowflakes, human faces, butterflies), and symmetry is a central foundational principle for several theories in biology and physics. For example, the Large Hadron Collider cost nearly $5 billion USD, and one of its primary goals was to determine whether symmetry exists at the tiniest scale of nature.

Third, there is an interesting property of mathematical functions: Any function — even nonsymmetric functions — can be decomposed into the average of an even and an odd function.

The "even-odd decomposition" of a function.

$$f(x) = \frac{f_e(x) + f_e(-x)}{2} + \frac{f_o(x) - f_o(-x)}{2} \tag{3.98}$$

where f_e is some even function and f_o is some odd function. $f(x)$ can be any symmetric or nonsymmetric function. This little factoid is not relevant for this book, but I find it fascinating that hidden inside every nonsymmetric function is the combination of two symmetric functions.

All that said, function symmetry is not relevant in many applications. You might have a decades-long career in machine-learning or computational sciences and never once need to incorporate function symmetry in your code or conclusions.

3.12 Exercises

3.1. The goal of this exercise is to explore how linear vs. nonlinear functions change over the x-axis. In Section 3.2 I explained that $\Delta y/\Delta x$ is constant for linear functions and changes for nonlinear functions (e.g., Figure 3.2).

Start by implementing two functions: $f_1(x) = 2x$ and $f_2(x) = x^2$, using 101 x-axis grid values between -2 and $+2$. Plot the functions as in Figure 3.28A. Implement the differences calculations twice: For f_1, use a for-loop in which each value in the function is subtracted from the previous value. For f_2, use vectorized code instead of a for-loop. Plot both results as in Figure 3.28B.

In practice, it's better to vectorize code instead of using for-loops, but for-loops often help build intuition for algorithms and operations.

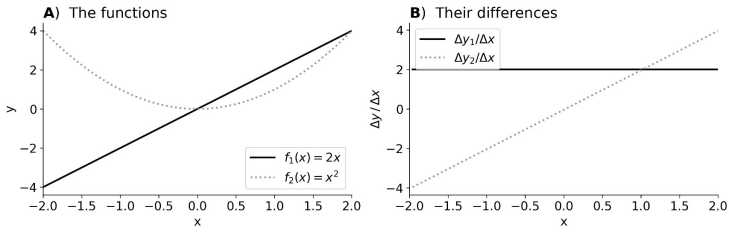


Figure 3.28: Plot for Exercise 1.

Two follow-up questions: (**1**) Do the differences values depend on the resolution of the x-axis? Try recreating the plot using various discretization numbers, e.g., 20 or 3000. (**2**) Δx never changes within a simulation, because the distance between x-axis values is constant. Is the Δx term even necessary? Try running through the code again without the Δx term. What changes and what doesn't?

3.2. Generate and plot a third-order polynomial. Sample four numbers at random to use as the coefficients, build a poynomial using those random coefficients, plot the function, and display the equation in the title. Your plot will not look *exactly* like Figure 3.29 because of random numbers, but it should look comparable.

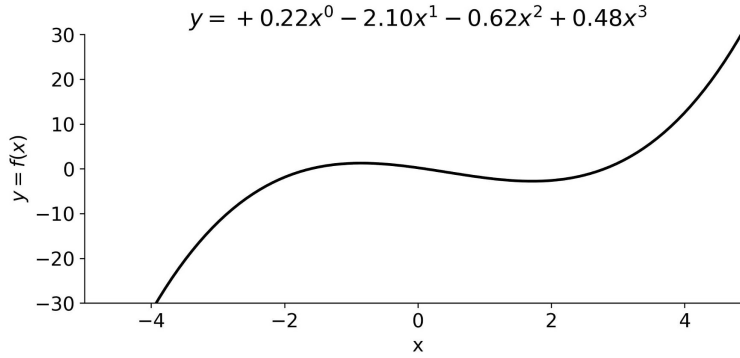


Figure 3.29: Plot for Exercise 2.

3.3. Polynomials are so common in applied mathematics because they can be used to approximate nearly any function. In this exercise, you will demonstrate this by approximating a sine wave using polynomials. Sounds crazy, right? How can a set of polynomials look like a sine wave?

Implement the "Maclaurin series" following Equation 3.99. Then generate a plot that shows all individual polynomials, their sum (thick black line in Figure 3.30), and a sine wave generated by `np.sin()` (blue dots). *Note*: It's not feasible to simulate an infinite number of polynomials; use $n = 10$.

$$\sin(x) \;=\; \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \tag{3.99}$$

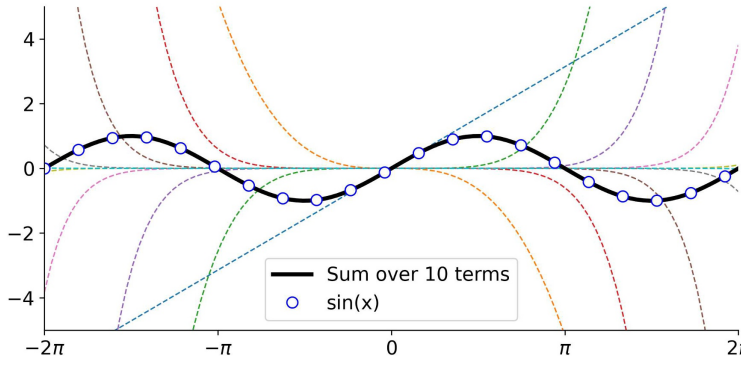


Figure 3.30: Plot for Exercise 3.

After writing the code, explore different values of the n pa-

rameter, and also explore different x domains (e.g., $\pm 4\pi$). What inferences can you make about the relationship between the number of polynomials and the span of the reconstructed sine wave? Do you understand why the Maclaurin series includes ∞ terms?

3.4. The goal of this exercise is to reproduce Figure 3.31. I generated a series of one-term polynomials of 1^{st} order (left panel), 2^{nd} order (middle), and 3^{rd} order (right), using coefficients ranging from .1 to 2, and coloring the lines according to the coefficient magnitude. I think the figure looks neat, and I hope you agree :) ... but if you disagree, then I challenge you to make the plots look more interesting by using different polynomials, coefficients, and colors.

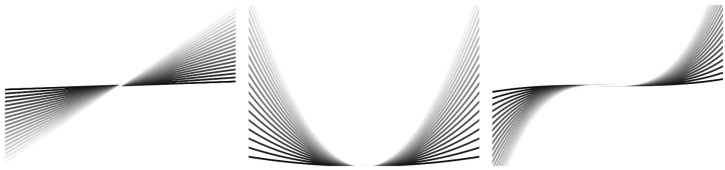


Figure 3.31: Plot for Exercise 4.

3.5. Visualizing math and data is generally easier and better using
`matplotlib` than `sym.plot`, but being able to quickly plot a function using `sym.plot` is a good skill to have. Your goal here is to create Figure 3.32 using `sympy`. In case the function equation is insufficiently legible in the title of the figure, here it is:

$$f(x) = \frac{x^2 - 2x}{x^2 - 4} \tag{3.100}$$

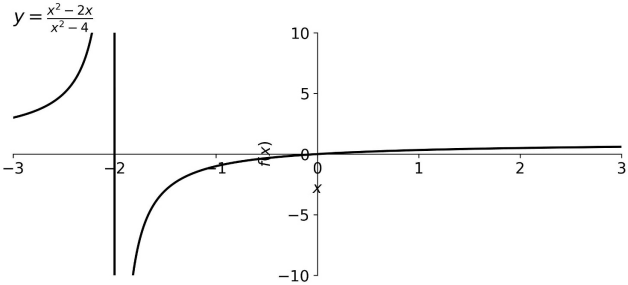


Figure 3.32: Plot for Exercise 5.

3.6. This two-part exercise will show you how quickly an approximation to e converges. Translate Equation 3.34 (page 82) into Python code, use `np.exp(1)` to get the "true"[4] value of e, and print a table like the following:

```
n:  1,  est.e: 2.00000,  diff to e: 0.7182818
n:  2,  est.e: 2.25000,  diff to e: 0.4682818
n:  5,  est.e: 2.48832,  diff to e: 0.2299618
n: 10,  est.e: 2.59374,  diff to e: 0.1245394
```

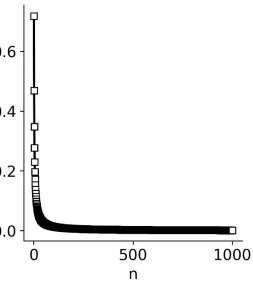


Figure 3.33: Visualization for Exercise 6.

Next, implement the approximation over a range of values of n from 1 to 1000. Plot the differences as a function of n as in Figure 3.33. It's interesting to see that the errors quickly collapse. You can also use a logarithmic y-axis scaling to better observe the convergence.

3.7. In this exercise, you will explore the difference between e in `numpy` vs. `sympy`. Implement the following function as a `sympy` object:

$$f(\beta) = e^{\beta} - \ln(\beta) - e \tag{3.101}$$

Use `sym.exp` to implement the first e and `np.exp` to implement the final e. Plot the resulting graph like in Figure 3.34. Notice in the title how the first e is a symbolic variable whereas the final e is a number. Finally, rerun the code but use `sym.exp()` for both implementations of e.

[4] As I wrote earlier in this chapter, `np.exp(1)` is *not* the true value of e, but it is a sufficiently accurate approximation that we can use it to evaluate other approximations.

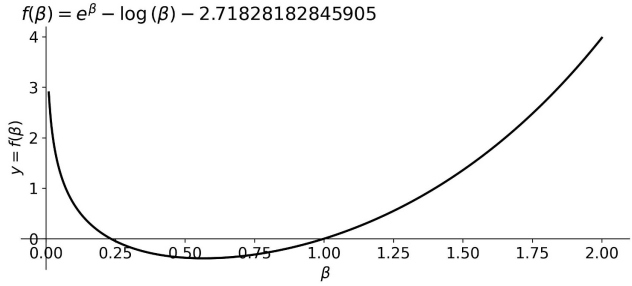


Figure 3.34: Plot for Exercise 7.

3.8. I call this exercise "random points on logs." Generate 80 numbers randomly sampled between .01 and 5, and plot those values by the log of those values. Do this for $\ln(x)$, $\log_2(x)$, and $\log_{10}(x)$. Figure 3.35 shows an example — your plot will look kind of similar but of course the locations of the dots will be different each time you run the code.

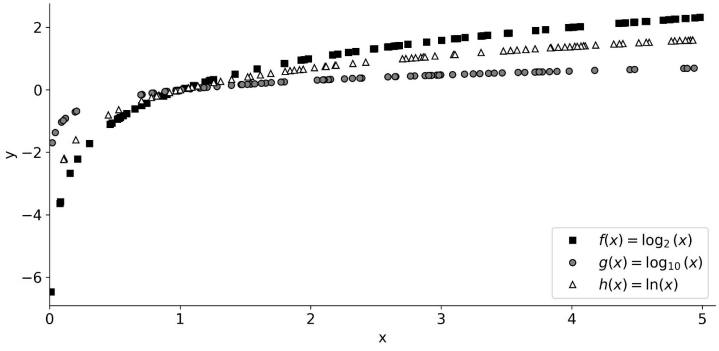


Figure 3.35: Plot for Exercise 8.

3.9. Piecewise functions are tricky to plot in Python, for the same reason that line plots can be misleading when the x-axis resolution is too coarse: `matplotlib` draws lines connecting coordinates, even if there are no function values between those coordinates. The purpose of this and the next several exercises is to explore coding and visualizing piecewise functions.

Start by implementing Equation 3.102 in `numpy`. Implement the function twice: Once creating a single `numpy` ar-

ray that contains all three pieces (for x from -3 to $+5$), and once by plotting three individual arrays separately from each other (one array for each function piece). Plot the results as in Figure 3.36.

Before reading my text below the figure, please take a moment to consider the advantages and disadvantages of each approach, in terms of visual accuracy and ease of implementation.

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ -2x & \text{if } 0 \leq x < 3 \\ x^3/10 & \text{if } x \geq 3 \end{cases} \tag{3.102}$$

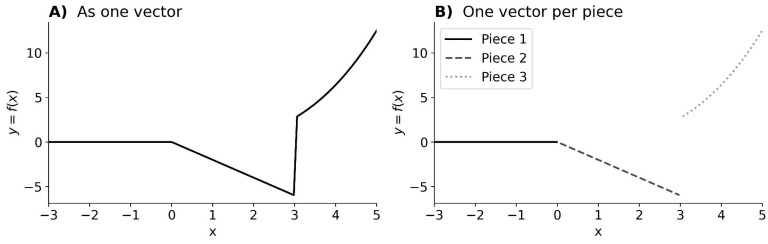


Figure 3.36: Plot for Exercise 9.

Panel A gives the misleading impression that the function goes smoothly but steeply from $y = -6$ to $y = 9/10$, when in fact the function jumps with no intermediate values (as depicted in panel B). On the other hand, the transition between the first and second pieces is accurately depicted in panel A. The conclusion from these observations is that encoding the piecewise function using separate vectors is more accurate when plotted.

This misrepresentation is purely visual, though: The numerical data stored in the `numpy` variable are accurate; there are no interpolated intermediate values. Therefore, the single-vector encoding of the piecewise function might be more convenient for applications.

3.10. Perhaps `sympy` knows how to plot piecewise functions correctly? Re-implement Equation 3.102 using `sym.Piecewise`, and then visualize using `sym.plot()`.

Spoiler alert: `sym.plot()` draws lines just like `matplotlib` (Figure 3.37), although `sympy` does accurately represent the function symbolically.

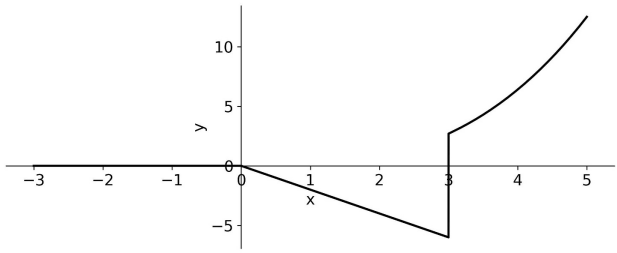


Figure 3.37: Plot for Exercise 10.

3.11. Now for some exercises on discontinuities. Implement the function described below. You can implement it as separate arrays for each piece or as one array, but use `numpy`; you'll use `sympy` in the following exercise. Try to make your plot match Figure 3.38.

$$f(x) = \begin{cases} \sin(x\pi) & \text{if} \quad x < 0 \\ 1.5 & \text{if} \quad x = 0 \\ -(x-2)^2 & \text{if} \quad x > 0 \end{cases} \tag{3.103}$$

$$D : -1 \leq x \leq 2$$

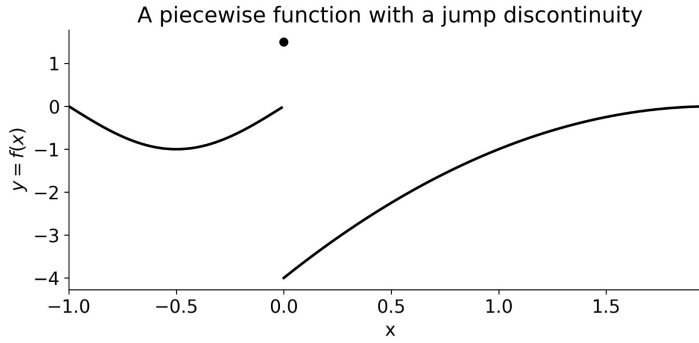


Figure 3.38: Plot for Exercise 11.

3.12. Repeat the previous exercise but creating the function in `sympy`. This is quite similar to what you did for Exercise 3.10, and the conclusion will be the same. Still, it's good to gain some more familiarity with creating piecewise functions in the `sympy` library.

3.13. Next up is a removable discontinuity. This function is continuous for all x except for one point. The equation is below, and my visualization is in Figure 3.39. Create the function as a `numpy` array and visualize it using `matplotlib`.

$$f(x) = \begin{cases} \sin(x\pi) + x^2 & \text{for} \quad x \neq 0 \\ \pi & \text{for} \quad x = 0 \end{cases} \tag{3.104}$$

3.14. Now for infinite discontinuities. Implement the following function in `numpy` using 1001 points linearly spaced in the subdomain $x = \pm 2$. Write code to reproduce Figure 3.40.

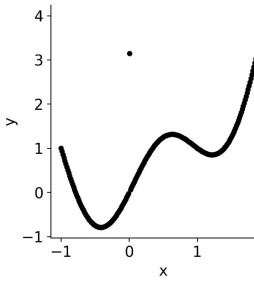


Figure 3.39: Visualization for Exercise 13.

$$f(x) = \frac{3}{1 - x^2} \tag{3.105}$$

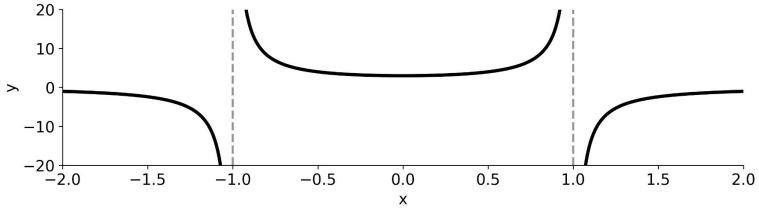


Figure 3.40: Plot for Exercise 14.

Observation: You'll get a warning message in Python about `divide by zero`. Why do you get that warning, and which value of the function produces it? What happens to that warning message and the plot when you use 1000 x-axis points?

3.15. One of the many useful features of `sympy` is that it can identify domains, domain restrictions, and singularities (a singularity in this context is a point with an infinite discontinuity). Implement Equation 3.105 in `sympy`, and use the function `continuous_domain` in the `sym.calculus.util` module to find the domain of this function. You should get a result of $[-2, -1) \cup (-1, 1) \cup (1, 2]$

Next, identify the x values at which the function has infinite discontinuities. I won't tell you which function to use, but there's a hint embedded in the previous paragraph ;)

3.16. An oscillating discontinuity is where an oscillating function, such as a sine wave, increases frequency as it approaches some x-axis value, and therefore never settles onto a particular function value. Implement the function below in `numpy` and create Figure 3.41 using `matplotlib`.

$$o(\theta) = \sin\left(\frac{1}{\theta - 1}\right) \tag{3.106}$$

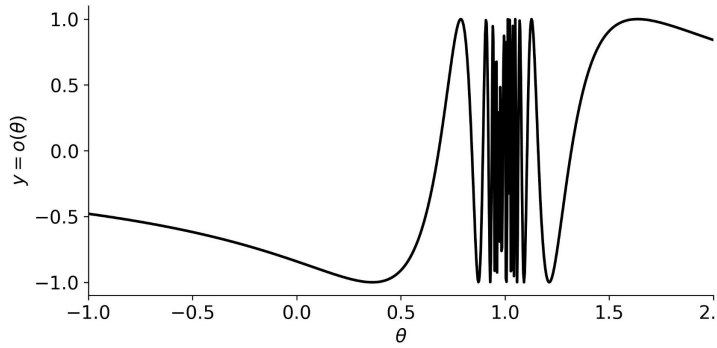


Figure 3.41: Plot for Exercise 16.

As the function approaches $\theta = 1$, the sine wave cycles faster and faster — it appears to get choppy and have smaller values, but that's an artifact of the discretization. Redraw it using $\theta \in [.9, 1.1]$, or even closer around 1. The more you zoom in, the faster the oscillations get. The function does not converge onto a single value. In the next chapter, you'll learn how to describe this behavior by stating that $\lim_{\theta \to 1}$ does not exist.

3.17. This and the next several exercises focus on implementing and exploring composite functions. Begin by translating the following two math functions into Python functions.

$$f(x) = 2x^2 - 4 \tag{3.107}$$

$$g(x) = 7|x| + 3 \tag{3.108}$$

Next, plot both functions, and their compositions, as in

Figure 3.42. Clearly, these two functions do not commute.

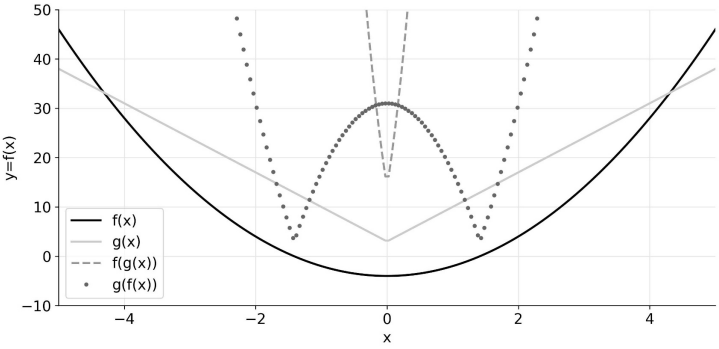


Figure 3.42: Plot for Exercise 17.

3.18. Now for a *menage a trois* (of functions). Translate the following math functions into Python code using `lambda` expressions[5]. Visualize the two compositions shown in Figure 3.43.

$$f(x) = \sin(x) \tag{3.109}$$

$$g(x) = \ln(x) \tag{3.110}$$

$$h(x) = 2x^2 + 5 \tag{3.111}$$

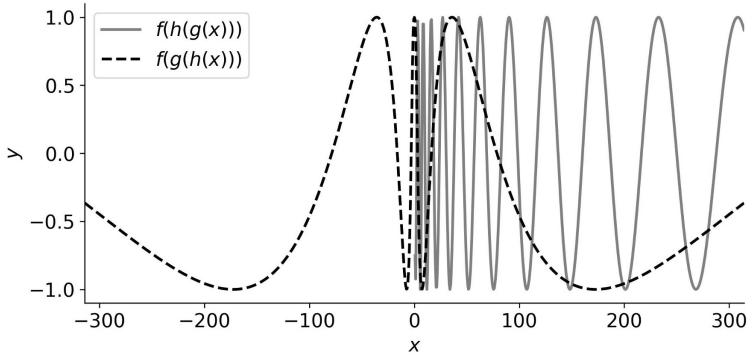


Figure 3.43: Plot for Exercise 18. Why does $f(h(g(x)))$ have no values for $x < 0$? Are there other ways of composing these functions where the domain is $x \in \mathbb{R}$?

[5]The reason why I sometimes ask for Python regular functions and sometimes `lambda` expressions is to give you practice with both.

3.19. Implement the following two functions using regular Python functions, and plot their compositions (Figure 3.44). Use x-axis values from .1 to 5.

$$p(x) = \ln(2x) \tag{3.112}$$

$$q(x) = e^x/2 \tag{3.113}$$

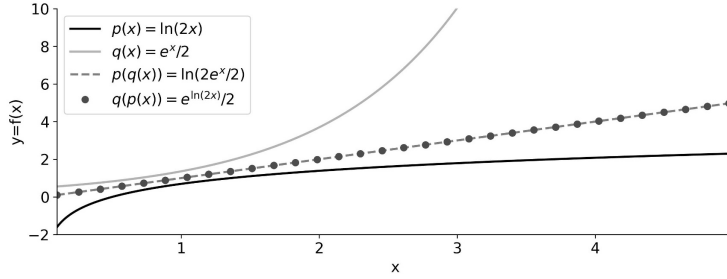


Figure 3.44: Plot for Exercise 19. I downsampled $q(p(x))$ to facilitate visualization.

It looks like these two functions commute. Although it is risky to make strong mathematical claims based on a plot, these two functions really do commute: $p(q(x)) = q(p(x))$ — at least for $x > 0$ (confirm in the code by redefining x to contain negative values). This is a visual confirmation that the natural log and natural exponential are inverses of each other (the factor of 2 might be confusing, but you'll understand why the "2" is divided vs. multiplied when you learn about differentiation).

3.20. In the previous chapter, I introduced you to the `.subs()` method on `sympy` objects as a way to substitute a number for a variable. In this exercise, you will learn how to substitute a *function* for a variable. This is how to create composite functions in `sympy`. Implement the following three equations in `sympy` and then compose them as shown in Figure 3.45.

$$f(x) = \sin(x) \tag{3.114}$$

$$g(x) = \ln(x) \tag{3.115}$$

$$h(x) = 2x^2 + 5 \tag{3.116}$$

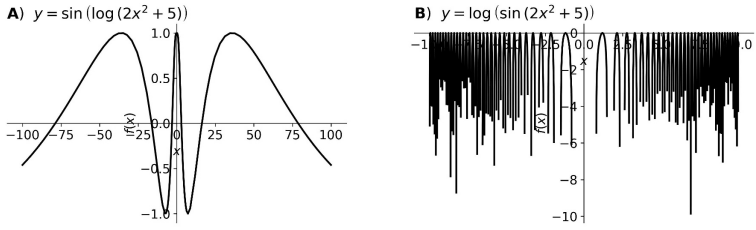


Figure 3.45: Plot for Exercise 20.

3.21. In Section 3.10, I explained how to calculate the inverse of a function by swapping x and y and solving for y. And I explained that except for illustrative examples in textbooks, it's better to use `sympy` to invert functions. Now it is time to learn how to do that.

To solve an equation in `sympy`, rewrite the equation to equal 0 (e.g., the function $4x = 2$ is expressed as $4x-2 = 0$) and then use `sym.solve`.

With that in mind, find the inverse function of $f(x) = 2x+3$. Next, use the `.subs` method to show that the composition of the function and its inverse gives the original x. In other words, demonstrate in `sympy` that $f(f^{-1}(x)) = x$ and that $f^{-1}(f(x)) = x$.

Finally, repeat this exercise using the function $g(x) = 2x + \sin(x)$ (make sure to use `sym.sin` and not `np.sin`). What happens and what does it mean?

3.22. The purpose of this exercise is to demonstrate function symmetry using `numpy` (part 1) and `sympy` (part 2).

Part 1: Craft three functions: one even, one odd, and one

neither. Then, evaluate and print out $f(x)$ and $f(-x)$ for three values of x. I used $x = [1, 2, \pi]$. My results are below. You can sneak a peak at the functions I used, although I encourage you to craft your own functions based on what you learned in Section 3.11.

```
Even   : f(1.00) =  0.00;  f(-1.00) =  0.00
Odd    : g(1.00) =  2.83;  g(-1.00) = -2.83
Neither: h(1.00) =  1.00;  h(-1.00) =  0.02

Even   : f(2.00) =  4.00;  f(-2.00) =  4.00
Odd    : g(2.00) = -1.47;  g(-2.00) =  1.47
Neither: h(2.00) = -0.33;  h(-2.00) = -0.69

Even   : f(3.14) = 21.14;  f(-3.14) = 21.14
Odd    : g(3.14) =  3.10;  g(-3.14) = -3.10
Neither: h(3.14) = -1.13;  h(-3.14) = -1.14
```

en function:
$x) = -x^2 + |x^3|$
$-x) = -x^2 + |x^3|$
$(x) = x^2 - |x^3|$

d function:
$x) = \frac{x^3}{10} - 3\sin(2x)$
$-x) = -\frac{x^3}{10} + 3\sin(2x)$
$(x) = -\frac{x^3}{10} + 3\sin(2x)$

nsymmetric function:
$x) = -\log(|x|) + e^{-(x-1)^2}$
$-x) = -\log(|x|) + e^{-(-x-1)^2}$
$(x) = \log(|x|) - e^{-(x-1)^2}$

Figure 3.46: Python output for Exercise 22.

Part 2: Rewrite those functions in `sympy`, substitute $-x$ for x, and confirm the rules of even, odd, and nonsymmetric functions. My results are shown in Figure 3.46.

3.23. The final two exercises in this chapter are just some neat visualizations. This exercise I call "I saw the sine." Translate into python and plot the following sine-endowed functions (I used `numpy` and `matplotlib`, but you can use `sympy` and `sym.plot` if you prefer):

a) $\sin(x)$ **b)** $\sin^2(x)$ **c)** $\sin(x^2)$

d) $\sin(\cos(x))$ **e)** $\cos(\sin(x))$ **f)** $\cos(x)$

I'm not showing the figure here but you can reconstruct it in the online code.

3.24. I call this exercise "the vortex of the logs." When I introduced the log function family, I wrote that the log of a

negative number is complex-valued[6].

Figure 3.47 looks pretty neat, doesn't it? That's what happens when you apply the logarithm to a number repeatedly. That is, pick a number, say, 2. Then take its log: $y = \ln(2)$. Then take the log of that: $z = \ln(y)$ and keep going (in other words, take $\ln(\ln(\ln(2))))$. Pretty quickly the result will be negative (recall that the log of a number between 0 and 1 is negative), which will lead to complex-valued numbers. This procedure is known as "iterative logarithms."

How did I make Figure 3.47? I picked a number at random between .1 and 3, applied the log operator 20 successive times storing each result, and plotted a line of the real parts by the imaginary parts of those 20 iterative logs. That procedure generates one of the grayscale lines in the figure; I then repeated this procedure 100 times for 100 random initial numbers.

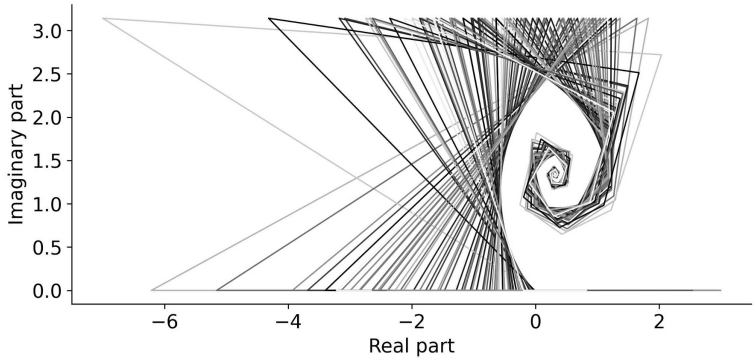


Figure 3.47: Plot for Exercise 24.

The reason for the convergence is related to *fixed points* of complex functions, which, in this case, is when $\ln(z) = z$. Complex analysis is a fascinating and beautiful topic in mathematics! It's beyond the scope of this book, but I

[6] A complex number is one that contains two parts: a real part and an imaginary part, where the imaginary part is a number times $i = \sqrt{-1}$. For example, $4 + 2i$ is a complex number. You can visualize complex numbers in a Cartesian axis with the real part for x and the imaginary part for y.

will show a few more examples of the beauty of complex numbers in later chapters.

Chapter 4

Limits

I hope you enjoyed learning about functions and characteristics of functions. Functions are foundational for all of mathematics, not only calculus.

Limits, on the other hand, are foundational to calculus. Indeed, one could argue that calculus is nothing more than the basic arithmetic operations (addition, subtraction, multiplication, and division) taken to their limits. Fortunately, though, limits are fairly simple conceptually, and, in many cases, limits of functions are easy to calculate.

4.1 Zeno's paradox and limits

Zeno's paradox is an ancient philosophical conundrum that remains discussed in modern times, and I think it provides an interesting way to introduce limits. There are several distinct but related paradoxes that Zeno has gifted upon humanity; the one I'll discuss here is called the "dichotomy paradox."[1]

Zeno's dichotomy paradox says that you cannot walk from the couch to the refrigerator to treat yourself to an ice cream after a long day studying calculus.

Figure 4.1: Zeno's ice cream paradox: To get from the couch (left) to the fridge (right), you need to walk halfway there (black vertical line in the middle), then half of the remaining distance, and so on. From this perspective, you'll get ever-closer to the fridge without ever reaching it.

Why not? Well, to get from the couch to the fridge, you need to halve the couch-fridge distance. And then you need to halve *that* distance. And so on. The process of approaching the fridge for your well-deserved ice cream involves repeatedly passing through half the distance from the previous halfway point (Figure 4.1). This must continue ad-infinitum, which means you'll spend eternity approaching the fridge in tinier steps — but never actually reaching your destination.

What's the point here? How does this relate to limits? The idea of limits is to understand how a function behaves when it gets

[1] If you enjoy this paradox, then some Internet searching will reveal Zeno's other paradoxes.

really really *really* close to a certain value — without actually equaling that value. The value of a function at a certain point $x = a$ is like the fridge, and the limit is like you halving your distance to the fridge without actually reaching it.

Let's not get into the physical and philosophical ramifications of Zeno's paradox (there are several proposed solutions, and anyway it breaks down at Planck scale, where concepts of measuring space and physical distance no longer make sense); the relevant take-home message is that you keep approaching a target but you never actually reach that target. This is the idea of a limit: Get closer and closer to a number — infinitesimally close — without actually reaching that number.

I hope you enjoyed that story (it's meant as a fun introduction to limits, not a rigorous mathematical definition). In Exercise 1 you will implement an algorithm based on this paradox to approximate limits. But for now I will transition to a more formal introduction to limits, and then explain how to compute limits of several different function families.

4.2 Geometric intuition of limits

The concept of a limit is so simple that you might think you're misunderstanding it, because how can such an obvious idea be the basis for all of calculus?

Figure 4.2 shows some function and some point a. It doesn't matter what this function is or what the underlying equation might be; the question at hand is: What happens as $f(x)$ approaches point $x = a$? You might think that the answer is simply the value of the function at $f(a)$. Some limits actually can be solved just by plugging in a. But the question here is not the *value* of exactly $f(a)$; the question is, What is the value of the function *as we get closer and closer* to $f(a)$? That's the limit of the function as x approaches a.

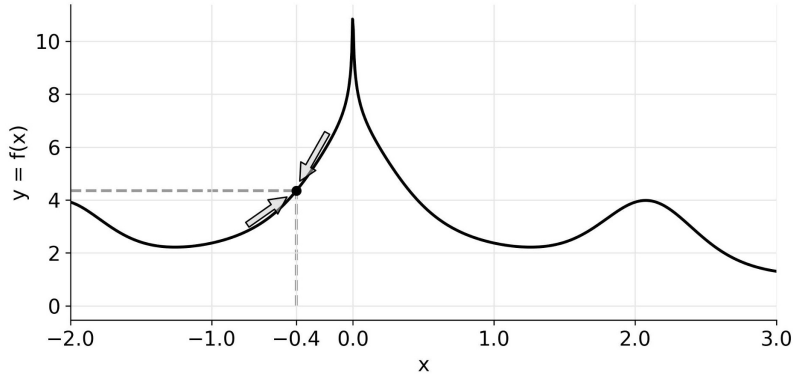


Figure 4.2: Illustration of the concept of a limit. The goal is to find the value of the function as x approaches $a = -.4$ — without actually exactly equaling a.

In the example illustrated here, the limit of the function as it approaches a is actually the same as $f(a)$. That happens because this function is continuous at a. Continuity and limits are closely related, which you saw in the previous chapter and which I will further elucidate in this chapter.

Limits are not always so easy to compute. Consider Figure 4.3. What's the limit of the function in panel A as x approaches 2? Perhaps it's ∞? Or maybe some finite but large number? How about the function in panel B? It looks like the function goes to $-\infty$ on the left of 2 and $+\infty$ on the right of 2. Does this function have two limits? Or an undefined limit? Panel C is also confusing: There is a removable discontinuity at $x = 2$. Is there no limit there, or is the limit the value of $f(2)$ that we could expect given what the function looks like close to 2? I promise you will know how to answer these questions by the end of this chapter; for now I'd like you to appreciate that determining the limit of a function at a particular point is not always trivial.

Limits at infinities The discussion so far has concerned limits of a function as x approaches some finite number. Calculus is also interested in the limit of a function as x approaches $-\infty$ or $+\infty$. Here the question is about the behavior of a function as x gets further and further away from zero. Many functions either blow

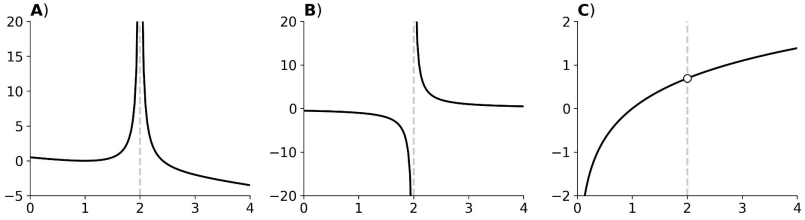


Figure 4.3: Three examples of when limits are less trivial than in the previous illustration.

up to $y = \pm\infty$ or asymptote to a specific numerical value.

But limits at infinities can be tricky to determine. Consider Figure 4.4A. The function continues oscillating as x gets infinitely far from zero — an oscillating discontinuity. What is the limit? How about panel B: Does that line keep going up to ∞ as x increases, or does it asymptote to some value of y that it never exceeds? Finally, panel C: It looks like the limit might be zero as x goes to ∞, but perhaps the function does become negative or asymptotes to some non-zero value like 1, but the y-axis scaling is too coarse to see this? Again, you'll learn answers to all these questions throughout this chapter, but I want to seed your imagination with observations and questions.

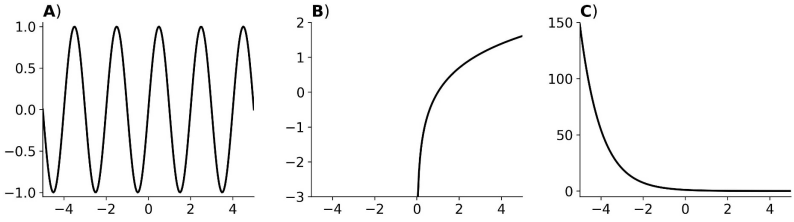


Figure 4.4: Three examples of determining limits as x tends to $\pm\infty$.

By the way, can you guess the three functions depicted in Figure 4.4? Being able to recognize a function based on its graph is an important skill. Please take a moment to guess the three functions. My answers are in the footnote[2].

Final comment for this section: Limits concern a function's be-

[2] A: $\sin(\pi x)$. B: $\ln(x)$. C: e^{-x}.

havior at one point. We do not speak of the limit *of a function*; we speak of the limit of a function *at one specific point*. That point may be a finite value or infinity, but it's still the limit as the function approaches one point. If you want to know the limit at two points x_1 and x_2 of the same function, then you need to calculate two separate limits.

4.3 Limits notation

You've already seen the notation for limits in the previous chapter, and perhaps in other mathematical texts or presentations. In this section I will describe all aspects of limits notation, and link it to the geometric intuition.

$$\lim_{x \to a} f(x) = L \tag{4.1}$$

You would read this out loud as "the limit as x approaches a of the function f of x, is L." You could also shorten it to say "L is the limit at a." Just make sure not to say "the function equals L when $x = a$." That phrasing is true for some limits, but it is not generally true, and it's not what a limit demonstrates.

When written in-line with text, limits are often indicated using subscripts: $\lim_{x \to a} f(x) = L$.

Equation 4.1 is called a "two-sided limit" and depicts the value as x approaches a from both left and right of a. There are also two one-sided limits, as x approaches a from either side individually. These are indicated using the equations below.

$$\lim_{x \to a^-} f(x) = L^- \tag{4.2}$$

$$\lim_{x \to a^+} f(x) = L^+ \tag{4.3}$$

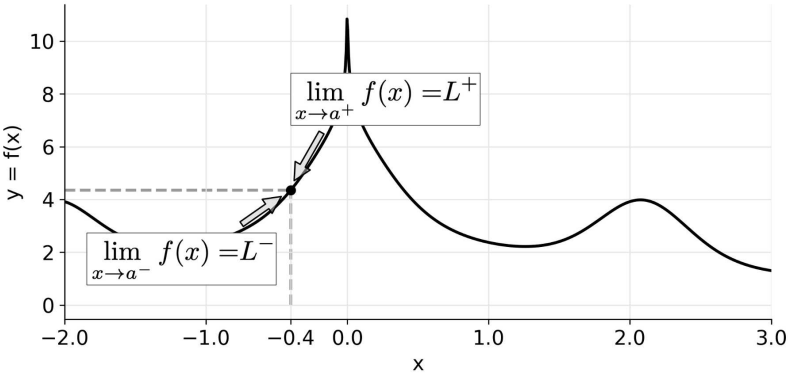


Figure 4.5: Modification of Figure 4.2 to show the left (−) and right (+) one-sided limits.

Note the − and + signs. The minus sign means approaching a from the left (where $x < a$) while the plus sign means approaching a from the right (where $x > a$). These are spoken out loud as "the one-sided limit as x approaches a from the left of the function f of x, is L."

You'll learn later that these two one-sided limits sometimes equal each other, and sometimes differ. They can differ at infinite or jump discontinuities, like in Figure 4.3B.

It is also common to write out the expression instead of "$f(x)$", as in this example:

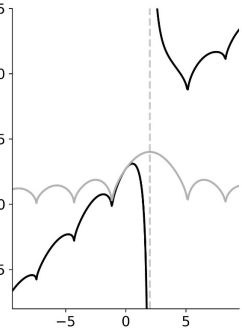
$$\lim_{x\to 2}\left(\frac{x^2+3x-10}{x-2}\right) \;=\; 7 \tag{4.4}$$

I guess you're not struggling with the notation, but how about the answer?! If you plug in $x = 2$ then you get $0/0$... how can the limit possibly be 7 when $f(2) = 0/0$? I'll let this seemingly inexplicable result simmer for a bit, as a segue into the mechanisms of calculating limits.

Parentheses One final note about notation: You know by this point in the book that I am a fan of unnecessary parentheses for

visual grouping, and therefore you won't be surprised to read that I recommend using parentheses to indicate limits.

In some cases, the expression is clearly legible without the parentheses, while in other cases the parentheses can resolve ambiguity. In subsequent chapters, when working through derivatives and integrals, that limits can contain multiple terms, appear inside other expressions or next to other limits, and therefore can get visually confusing. Consider, for example, the first limit expression below; which of the two subsequent expressions is the intended meaning?



Limits on clouds.

$$\lim_{x\to 2} \frac{x^2}{x-2} + 4\sqrt{\left|\frac{\sin(x-2)}{x-2}\right|}$$

$$\lim_{x\to 2} \left(\frac{x^2}{x-2}\right) + 4\sqrt{\left|\frac{\sin(x-2)}{x-2}\right|} \qquad \textit{or} \qquad \lim_{x\to 2} \left(\frac{x^2}{x-2} + 4\sqrt{\left|\frac{\sin(x-2)}{x-2}\right|}\right.$$

Conclusion: parentheses can improve legibility and resolve ambiguity.

4.4 "Easy" limits by plugging in or factoring

If a function is continuous at $x = a$, then use the "plug in" method, which, as you've probably guessed, simply involves plugging in the value a for x in the function. Here's an example:

$$\lim_{x\to 4} 2x \tag{4.5}$$

This limit problem is so easy to calculate it's almost patronizing: As x approaches 4, then the function approaches 7.[3]

[3] That's intentionally inaccurate; I'm just making sure you're paying attention!

How do you know when it's safe to use the plug in method? The formal answer is *when the function is continuous at* $x = a$. The informal answer is you try to plug in, see what happens, and look at the function to see if anything is amiss such as a zero in the denominator or an answer of $\pm\infty/\infty$.

Keep in mind that the plug-in method is a bit of a cheat: The limit is *not* the value of the function at some point, but instead is the value of the function as it gets really really close to that point. But for continuous points, the limit is the same as the value.

4.4.1 Factoring

The plug-in method is great, and you should use it whenever possible. If it's not immediately possible, then your next strategy is to factor the equation and simplify — and then try again to plug in the limit value. Let's return to the perplexing example in the previous section; notice how factoring the numerator gets rid of the fraction, thereby revealing the limit.

Keep in mind that the domain of this function still excludes $x = 2$ due to the original specification.

$$\lim_{x\to 2} \frac{x^2 + 3x - 10}{x - 2} = ? \tag{4.6}$$

$$\lim_{x\to 2} \frac{(x + 5)(x - 2)}{x - 2} = ? \tag{4.7}$$

$$\lim_{x\to 2} (x + 5) = 7 \tag{4.8}$$

Is it really that simple?

Let's take a moment to think about this. Consider that plugging in $x = 2$ in Equation 4.6 gives $0/0$ (actually, you don't even need to calculate the numerator; you can sense trouble as soon as you see that the plug-in method gives a zero in the denominator).

But notice that when x *approaches but is not exactly equal* to 2, then the denominator is *close to but not exactly equal* to 0. It's a tiny number, but it's not actually zero. So you might think that

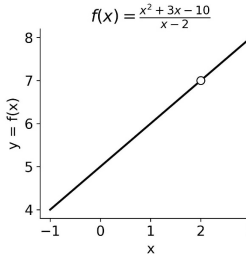


Figure 4.6: Some functions look boring but close inspection reveals some flair.

this equation blows up to ∞ as x approaches 2. Now let's think about the numerator: When x is a tiny little bit bigger than 2, then the numerator is also a tiny-but-nonzero number. The point is that as x approaches 2, this expression ends up being a tiny number divided by another tiny number — not $0/0$. See Figure 4.6.

The upshot of this example is this: When the plug-in method fails, try some algebraic manipulations in attempt cancel or modify the denominator so that it no longer equals zero at $x = a$, at which point you can use the plug-in method. In other words, the solution is still to use the plug-in technique, but you might need to massage the expression before that's possible.

4.4.2 Limits and denominators

Consider the following three expressions:

$$\frac{0}{0}, \qquad \lim_{x\to 0} \frac{0}{x}, \qquad \lim_{x\to 0} \frac{1}{x}$$

The first expression is undefined: You cannot divide by zero. The reason is that $a = b/c$ means $b = ac$, and if $c = 0$ then there is no possible value of b that can produce a non-zero a.

The second expression is simply 0. You might be tempted to use the plug-in method and conclude that the limit is $0/0$, but this is not the case: The limit involves considering what happens as x *approaches* 0. Thus, the second expression is $0/\epsilon$ where ϵ is some vanishingly tiny number — though not exactly equal to zero.

The third expression is where we run into trouble with limits. Again, the answer is not $1/0$; the denominator is so small that the fraction is effectively $\pm\infty$ (the sign will depend on whether you approach 0 from positive or negative numbers). That is not helpful for limits calculations, and so the plug-in method will fail to produce sensible results.

4.5 One-sided limits and infinities

Can you solve this limit problem by plugging in or factoring?

$$\lim_{x \to 2} \frac{|x-2|}{x-2} = ? \tag{4.9}$$

Plugging in produces 0/0, and there is nothing to factor. Let's consider the graph of this function (Figure 4.7).

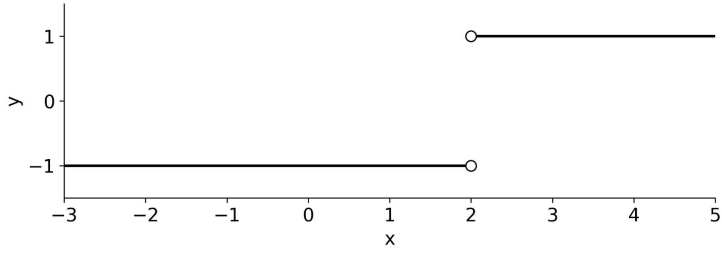


Figure 4.7: Graph of Equation 4.9. Open circles indicate a jump discontinuity with no function value at $x = 2$.

The plot is insightful: It makes no sense to talk about *the* limit. In fact, there are *two* limits: as we approach 2 from the left (that is, numbers a tiny bit smaller than 2), and as we approach from the right (numbers a tiny bit larger than 2). So, this function has two one-sided limits, both of which are perfectly valid — and yet distinct from each other.

The limit from the left is -1 and the limit from the right is $+1$. The two-sided limit as $x \to 2$ is not defined, and the function is also undefined at $x = 2$.

$$\lim_{x \to 2^-} \frac{|x-2|}{x-2} = -1 \tag{4.10}$$

$$\lim_{x \to 2^+} \frac{|x-2|}{x-2} = 1 \tag{4.11}$$

$$\lim_{x \to 2} \frac{|x-2|}{x-2} = \text{d.n.e.} \tag{4.12}$$

d.n.e. stands for

Having distinct one-sided limits does not necessarily mean the function is undefined at that point. For example, the equation below shows the Heaviside function, which is visualized in Figure 4.8 and is used in engineering and signal processing.

$$H(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases} \tag{4.13}$$

The Heaviside function[4] has two distinct one-sided limits as $x \to 0$ and is defined at $x = 0$.

The Heaviside function

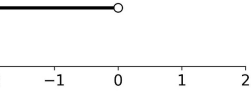


Figure 4.8: The Heaviside step response is an example of a piecewise function that has no two-sided limit at $x = 0$ but is defined at that point.

One- vs. two-sided conventions When someone says or writes "the limit exists," or "the limit is L," they are referring to the two-sided limit, which is the same thing as saying that the two one-sided limits equal each other. If they say "the limit does not exist," it can mean that the two one-sided limits exist but differ from each other.

4.6 Properties of limits

There are four important properties of limits that allow you to prove theorems and solve problems in calculus. They are listed below, and I will explain each thereafter. In these equations, γ is a constant while f and g are functions of x (I'm omitting the "(x)" for readability).

$$\text{Factorable:} \quad \lim(\gamma f) = \gamma \lim(f) \tag{4.14}$$

[4] I used to think that this function was so named because it is "heavier" on one side. Alas, it's named after Oliver Heaviside, the brilliant self-taught mathematician. It doesn't take a genius to invent a step function, but the naming honors his myriad contributions to differential equations among other topics.

$$\text{Additive:} \quad \lim(f+g) = \lim(f) + \lim(g) \tag{4.15}$$

$$\text{Multiplicative:} \quad \lim(f \times g) = \lim(f) \times \lim(g) \tag{4.16}$$

$$\text{Divisive:} \quad \lim(f/g) = \lim(f)/\lim(g) \tag{4.17}$$

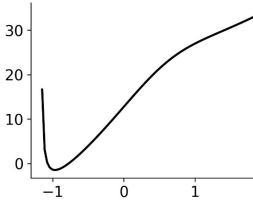
These properties are important to know and easy to memorize, because they reflect the normal arithmetic operations that you're familiar with. These properties are characteristics of linear operators, and the limit is a linear operator.

By the way, notice that I have omitted the "$x \to a$" under the "lim" text. Notations in mathematics are often shortened when details are unnecessary.

Limits are factorable

Any constant can be pulled out of a limit. This can simplify the math of a limits problem, and is used in some proofs that you'll encounter later in the book. Here's an example:

$$\lim\left(\frac{16+8x}{\sqrt{x^3+\pi}} + 12x\right) \;=\; 4\lim\left(\frac{4+2x}{\sqrt{x^3+\pi}} + 3x\right) \tag{4.18}$$



Limits are factorable: Check! (Equation 4.18)

Why are limits factorable? There are two ways to think about this. First, when you multiply a function by a constant, you are merely scaling its values. Imagine stretching or compressing the graph of $f(x)$ by a factor of γ; the new limit is stretched or compressed by that same factor. A second way to think about the factorability of limits is that a limit is simply a *number*. Limits are taken of *functions*, but the result of taking the limit of a function at a point is one numerical value. Therefore, properties of scalar arithmetic apply to limits arithmetic.

Limits are additive

The limit of sums equals the sum of limits. An example:

$$\lim_{x\to\pi}\left(x^2 + \ln(x) - \frac{\sin(x)}{x-\pi}\right) = \lim_{x\to\pi}\left(x^2\right) + \lim_{x\to\pi}\left[\ln(x)\right] - \lim_{x\to\pi}\left(\frac{\sin(x)}{x-\pi}\right) \tag{4.19}$$

This property is particularly useful when some limits are easy to solve (like the first two terms above) while others are trickier (like the third term).

Limits are multiplicative

The limit of the product of terms equals the product of the limits of the individual terms. An example:

Use square brackets [] to disambiguate groupings when parentheses ( ) are already used.

$$\lim \left[e^{x-3} \ln(x-\pi) \right] \;=\; \lim \left(e^{x-3} \right) \times \lim \left[\ln(x-\pi) \right] \quad (4.20)$$

Again, this property is useful for solving tricky limits problems and proving some theorems that you'll encounter later in the book.

Limits are divisive

Last but not least: The limit of the ratio of functions equals the ratio of the limits of the individual functions.

$$\lim \left[\frac{(x-3)^3}{(\sqrt{\pi}-x)^2} \right] = \frac{\lim (x-3)^3}{\lim (\sqrt{\pi}-x)^2} \quad (4.21)$$

Unlike the previous three properties, the divisive property is not guaranteed to hold: you need to watch out for cases where the limit in the denominator is zero.

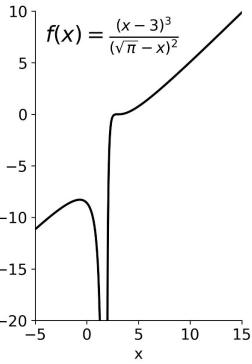


Figure 4.9: Visualization of the expression used in Equation 4.21.

Only when the limit exists The properties described above are valid only when the limit exists. If the limit does not exist at a given point, then you should not rely on these properties. Be particularly mindful when dealing with limits at discontinuities where the function is undefined. You'll see examples of how limits properties break down in the exercises.

4.7 Continuity and discontinuities, revisited

In the previous chapter, I presented the mathematical definitions of continuity and discontinuities using limits — and told you not to worry about those definitions until you learn about limits in this chapter.

And here we are.

Below is the definition of continuity, copied from the previous chapter. I hope you feel that this definition is more comprehensible now.

A function $f(x)$ is continuous at $x = a$ if:

1. $f(a)$ is defined.
2. $\lim_{x \to a} f(x)$ exists.
3. $\lim_{x \to a} f(x) = f(a)$.

Condition 2 means that the *two-sided* limit exists. If the function has two distinct one-sided limits as $x \to a$, then condition 2 is violated and the function is not continuous at point a.

Condition 3 means that the limit is the value of the function when x gets really close to — but not exactly equal to — a. A removable discontinuity is an example of a violation of condition 3, because the two-sided limit exists but does not equal the function value at that point.

And with that in mind, you can now better understand the different types of discontinuities (Figure 4.10).

4.8 Limits of trig functions, part 1

There are several important trig limits to know in any study of calculus. In this section I will introduce you to two of them, and then I'll present a few more in a later section. The limits and

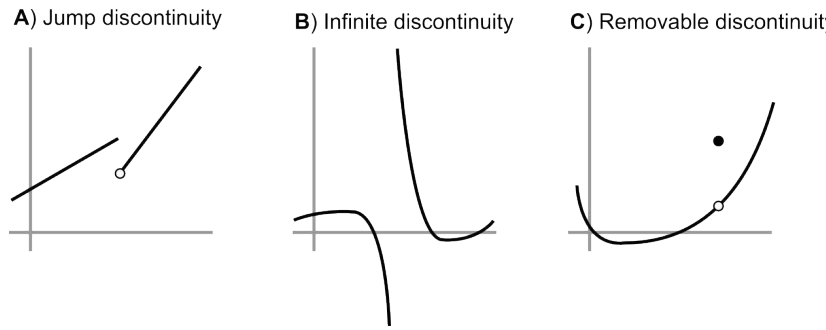


Figure 4.10: Examples of discontinuities. In panel A, condition 2 is violated. In panel B, all three conditions are violated. In panel C, conditions 1 and 2 are met while condition 3 is violated.

their geometric pictures are below, and then I'll prove those limits thereafter.

$$\lim_{\phi \to 0} \left(\frac{\sin(\phi)}{\phi} \right) = 1 \tag{4.22}$$

$$\lim_{\phi \to 0} \left(\frac{\cos(\phi) - 1}{\phi} \right) = 0 \tag{4.23}$$

Those limits are not obvious when looking at the equations. Indeed, if you try plugging in $\phi = 0$, you'll get $0/0$.

On the other hand, these two limits appear quite sensible when looking at the graphs in Figure 4.11.

Proving the sine limit There are several ways to prove Equation 4.22, some more difficult and some easier. Here I will present a simple geometric intuition, and then I'll show a formal proof in Chapter 7 when I introduce L'Hopital's rule.

Inspect the left panel of Figure 4.12. You can see a unit circle with a point on that circle corresponding to $\phi = \pi/6$ and $r = 1$. Because it is a unit circle, the distance from that point to the horizontal axis (black dashed line) equals $\sin(\phi)$, and the length of the arc subtended from that point to the horizontal axis (black

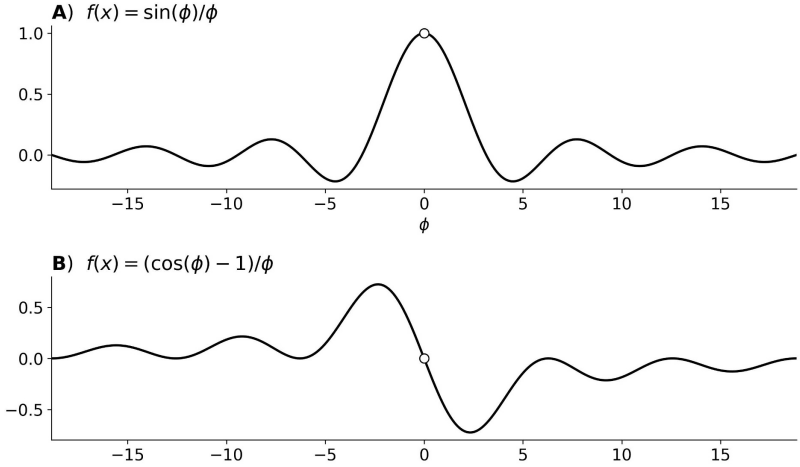


Figure 4.11: Two trig functions with removable discontinuities.

solid curved line) equals ϕ. It is obvious that the length of the line is less than the length of the arc, hence their ratio is less than 1.

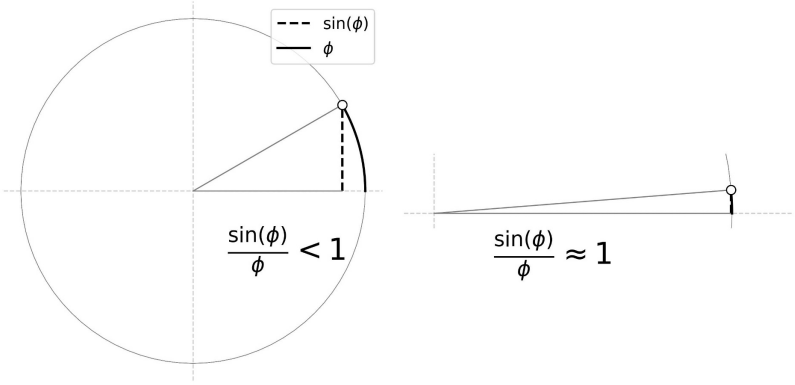


Figure 4.12: Geometric intuition for why $\lim_{\phi \to 0} \sin(\phi)/\phi = 1$.

Now consider what happens as ϕ gets smaller, close to zero (right-side panel): Both the line and the arc get smaller — eventually, the arc becomes such a tiny sliver that it has no effective curvature, and thus the length of the line is nearly the same as the length of the arc. At the limit as $\phi \to 0$, the arc and line have equal length. Hence:

$$\lim_{\phi \to 0} \frac{\sin(\phi)}{\phi} = 1 \tag{4.24}$$

Proving the cosine limit Try to follow each of the steps below before reading the postceding text. Notice how this proof relies on the limits properties I introduced earlier.

$$\lim_{\phi \to 0} \frac{\cos(\phi) - 1}{\phi} = \lim_{\phi \to 0} \frac{\cos(\phi) - 1}{\phi} \times \frac{\cos(\phi) + 1}{\cos(\phi) + 1} \tag{4.25}$$

$$= \lim_{\phi \to 0} \frac{\cos^2(\phi) - 1}{\phi(\cos(\phi) + 1)} \tag{4.26}$$

$$= \lim_{\phi \to 0} \frac{-\sin^2(\phi)}{\phi(\cos(\phi) + 1)} \tag{4.27}$$

$$= \lim_{\phi \to 0} \left[\frac{\sin(\phi)}{\phi} \times \frac{-\sin(\phi)}{\cos(\phi) + 1} \right] \tag{4.28}$$

$$= 1 \times \lim_{\phi \to 0} \left[\frac{-\sin(\phi)}{\cos(\phi) + 1} \right] \tag{4.29}$$

$$= \frac{-\sin(0)}{\cos(0) + 1} = \frac{0}{1 + 1} = 0 \tag{4.30}$$

We begin in Equation 4.25 with multiplying the expression by $\cos(\phi) + 1$ divided by itself. You might be tempted to expand the multiplication in the denominator of Equation 4.26, but keeping the terms factored is the key to solving this problem by leveraging the multiplicative property of limits.

Did you understand the transition from Equation 4.26 to 4.27? That comes from rearranging the trig identity $\cos^2(\phi) + \sin^2(\phi) = 1$.

Then, Equation 4.28 separates the fraction into the product of two fractions, one of which is the sine limit that I proved earlier. Finally, we get to Equation 4.30, where we have algebraically manipulated the original equation to a point where the plug-in method works.

If you struggled to understand the sequence of equations the first time you went through them, please go back through them now after having read this explanation. Better still: Put this book aside and see if you can re-derive this proof with paper and pen.

Other proofs It turns out that both of these limits are easier and more intuitive to prove using *L'Hopital's rule*. But you need to understand differentiation before you can understand L'Hopital's rule. We will revisit these proofs later in the book. Perhaps there is a life lesson in this: Lots of things in life seem difficult when you first get started, but the more you learn, the easier they get.

4.9 Squeeze theorem

The *squeeze theorem*, also called the "sandwich theorem" or "pinch theorem," is a way to find the limit of a function at a point when it's otherwise impossible to calculate the limit. The idea is to "squeeze" the function between two other functions that have easily calculable limits. In this section, I will introduce the squeeze theorem and show some simple examples, and then in the next section I'll show how we use the squeeze theorem to find the limits of some trig functions that are tricky or impossible to calculate without the squeeze.

The algebra of the squeeze theorem is stated below. I'll provide a written explanation of what those equations mean, but first I would like you to try to understand what the squeeze theorem means by looking at the equations.

$$g(a) \leq f(a) \leq h(a) \tag{4.31}$$

$$\lim_{x \to a} g(x) \leq \lim_{x \to a} f(x) \leq \lim_{x \to a} h(x) \tag{4.32}$$

$$L \leq f(a) \leq L \tag{4.33}$$

Do those equations make sense? Here's the idea: We're interested in the limit of $f(x)$ as $x \to a$, but perhaps we don't know the full formula for that function, or perhaps the limit is difficult or impossible to calculate at a. Now imagine we have two other well-characterized functions, $h(x)$ and $g(x)$, such that

$h(a) \leq f(a) \leq g(a)$. That's the squeeze: $f(a)$ is "sandwiched in between" $h(a)$ and $g(a)$. If those assumptions hold, then it must be true that the limit at $f(a)$ is in between $\lim_{x \to a} g(x)$ and $\lim_{x \to a} h(x)$. Finally, if we determine that the limits of g and h both equal L, then the limit of $f(x)$ as $x \to a$ must also be L, because the only value between L and L is... well, it's L.

Figure 4.13 highlights the idea of the squeeze theorem. We want to know $\lim_{x \to a} f(x)$. But $f(x)$ is some weird function and it's impossible to compute the limit by plugging in or factoring. So we take two functions for which we can calculate the limits, and then squeeze these three functions together.[5]

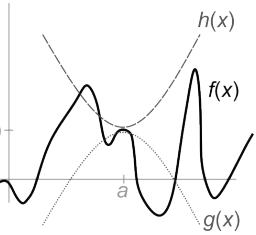


Figure 4.13: Visual depiction of using two simpler functions to determine the limit of a complicated function at some point.

4.9.1 Three examples

The examples in this section are fairly easy, and some of these limits problems could be solved without the squeeze theorem, but I hope you agree that they help elucidate the squeeze process.

Example 1: Limit without a function definition Figure 4.14 shows some wacky $f(x)$ that is not described by any mathematical expression; instead, I scribbled it in Inkscape. But that doesn't matter; you already have enough information to determine $\lim_{x \to 2} f(x)$. Determine that limit before reading the next paragraph.

You can infer the limit using the graph and the equations of $g(x)$ and $h(x)$. Those two "squeezing" functions pinch $f(x)$ at $x = 2$, and you can use the plug-in method to determine that the limits of those two functions as x approaches 2 is 5.

Example 2: Real functions Determine the limit of $f(x)$ given the information provided below, and the graphs of the functions

[5] The inventor of the squeeze theorem is not exactly known, although it is attributed both to Archimedes and Gauss. Solutions to these kinds of problems become part of the corpus of calculus, and their true origins are sometimes lost to the impermanence of parchment.

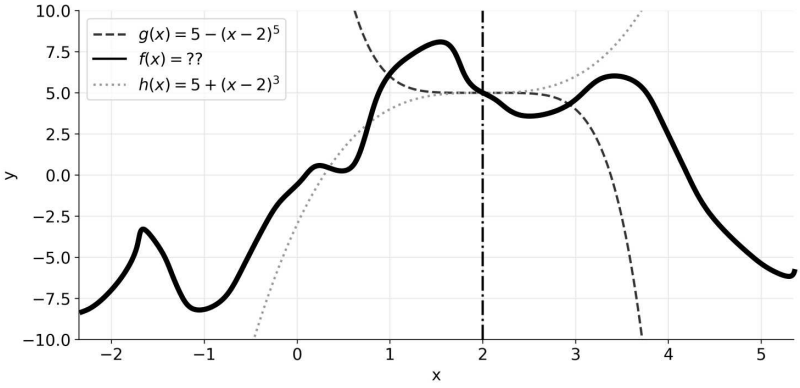


Figure 4.14: Example of using the squeeze theorem to find the limit of an unknown function.

in Figure 4.15. You can first quickly confirm that plug-in and factoring won't be much help.

$$g(x) = .5 + (x-2)^2 \tag{4.34}$$

$$f(x) = \frac{x^2 - 2x}{x^2 - 4} \tag{4.35}$$

$$h(x) = .5 - (x-2)^3 \tag{4.36}$$

$$\lim_{x \to 2} f(x) = ?? \tag{4.37}$$

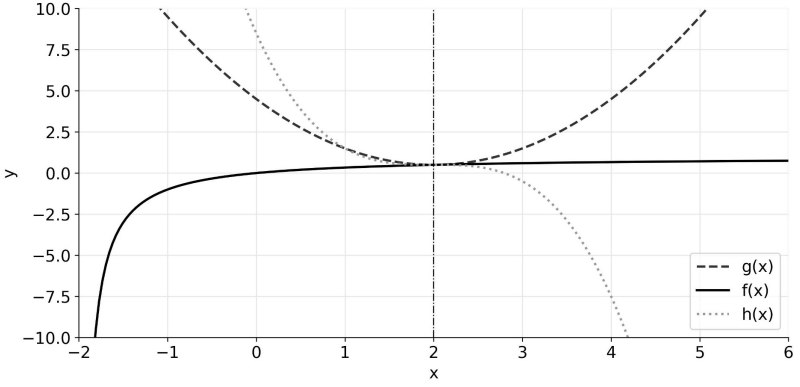


Figure 4.15: Example 2 of using the squeeze function.

$f(x)$ appears continuous at $x = 2$ in the graph, but in fact, there is a removable discontinuity because $f(2) = 0/0$. On the other hand,

the two squeezing functions cleanly evaluate to $g(2) = h(2) = .5$. More formally, we can write:

$$\lim_{x\to 2} g(x) \leq \lim_{x\to 2} f(x) \leq \lim_{x\to 2} h(x) \tag{4.38}$$

$$.5 \leq \lim_{x\to 2} f(x) \leq .5 \tag{4.39}$$

$$\lim_{x\to 2} f(x) = .5 \tag{4.40}$$

Example 3: Finding the squeezing functions This example is simplistic in that the limit can be determined without the squeeze theorem. But the goal of this example is to show one way of crafting appropriate squeezing functions.

The goal is to find the limit of the function $f(\theta) = \theta\cos(\theta)$, as $\theta \to 0$. Its graph is shown in Figure 4.16.

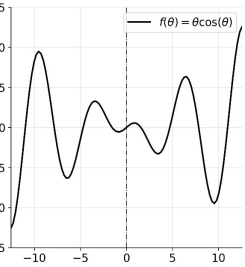


Figure 4.16: What do you think is the limit as θ approaches zero?

What squeezing functions should we use? There's a trick that sometimes works with trig functions, which is write down an inequality based on the range of the trig function, and then do some algebraic manipulations to get the middle expression into the form of $f(x)$. Observe:

$$-1 \leq \cos(\theta) \leq 1 \tag{4.41}$$

$$-\theta \leq \theta\cos(\theta) \leq \theta \tag{4.42}$$

The cosine of θ is bound by -1 and $+1$ (Equation 4.41). Then, multiplying through by θ gives the desired function in the middle, and simultaneously provides $g(\theta)$ and $h(\theta)$ (Equation 4.42). That's the trick. You'll get to use this trick in the next section to solve a more difficult trig limit.

Figure 4.17 shows the three functions. You can easily calculate, and visually confirm, that $\lim_{\theta\to 0} [\theta\cos(\theta)] = 0$.

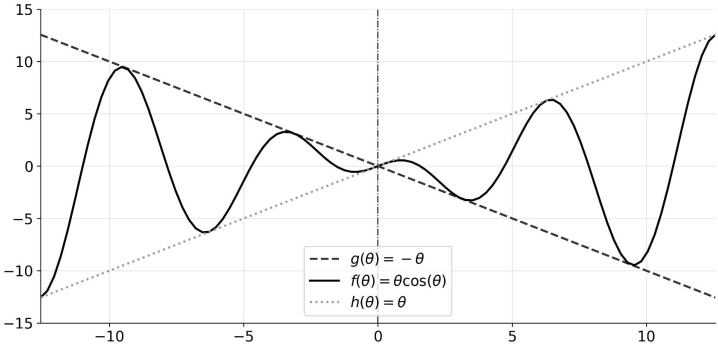


Figure 4.17: The three functions in example 3.

4.10 Limits of trig functions, part 2

The goal of this section is to solve three trig limits problems. The first two are solved with the squeeze theorem, and the third is solved using the two trig limits introduced in "part 1" (Section 4.8). The three functions are printed below and illustrated in Figure 4.18.

$$\lim_{\theta\to 0}\left[\theta^2 \sin\left(\frac{1}{\theta}\right)\right], \qquad \lim_{\theta\to\infty}\left[\frac{\sin(\theta)}{\theta}\right], \qquad \lim_{\theta\to 0}\left[\frac{\tan(\theta)}{\theta}\right]$$

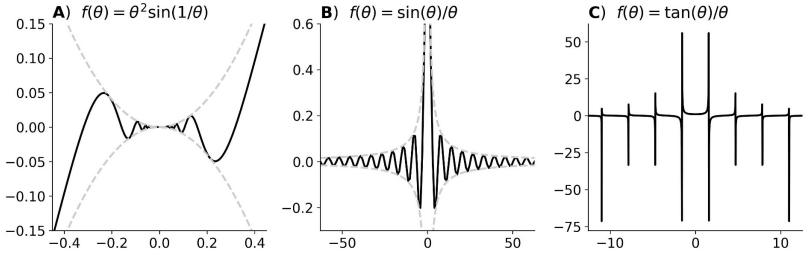


Figure 4.18: More fun with trig limits :)

Figure 4.18A Let's begin with the function depicted in panel A of Figure 4.18. Visually, it appears that the limit as $\theta \to 0$ is zero. But algebraically, we cannot plug in $f(0)$, because that gives $1/0$ inside the sin function.

Enter the squeeze.

The two squeezing functions in dashed gray lines appear to be quadratics (something of the form θ^2). But what are those functions exactly, and how did I pick them? Let's use the trick I explained earlier: write down the range of part of the function as inequalities, then multiply through to obtain the original function:

$$-1 \leq \sin\left(\frac{1}{\theta}\right) \leq 1 \tag{4.43}$$

$$-\theta^2 \leq \theta^2 \sin\left(\frac{1}{\theta}\right) \leq \theta^2 \tag{4.44}$$

Equation 4.43 states that the range of the sine function is bound between -1 and $+1$. The function in the middle is not the one we are trying to solve, but multiplying the entire inequality by θ^2 gives us our desired squeezing functions ($\pm\theta^2$), which are drawn as dashed gray lines in 4.18A.

The next step is to take the limits of each term:

$$\lim_{\theta\to 0} -\theta^2 \leq \lim_{\theta\to 0}\left[\theta^2 \sin\left(\frac{1}{\theta}\right)\right] \leq \lim_{\theta\to 0}\theta^2 \tag{4.45}$$

$$0 \leq \lim_{\theta\to 0}\left[\theta^2 \sin\left(\frac{1}{\theta}\right)\right] \leq 0 \tag{4.46}$$

I haven't solved the limit of the actual function; we don't need to. We know that the limit is $0 \leq L \leq 0$, so we can conclude the $L = 0$. I hope that also makes intuitive sense: $\sin(1/\theta)$ is an oscillating discontinuity as $\theta \to 0$, but it gets multiplied by θ^2, which approaches zero as the limit goes to zero.

Figure 4.18B Now onto the next problem. This problem is solved using the same setup as the previous problem. Therefore, I

encourage you to try to solve that problem before reading my answer below. And then check whether your answer seems visually consistent with the graph of the function in Figure 4.18B.

The key insight here is to separate the function into the product of two functions: $\sin(\theta)$ and $1/\theta$. That leads to the following sequence of inequalities:

$$-1 \leq \sin(\theta) \leq 1 \tag{4.47}$$

$$-\frac{1}{\theta} \leq \frac{1}{\theta}\sin(\theta) \leq \frac{1}{\theta} \tag{4.48}$$

From here, we apply the limits as in the first problem:

$$\lim_{\theta\to\infty}\left(-\frac{1}{\theta}\right) \leq \lim_{\theta\to\infty}\left[\frac{\sin(\theta)}{\theta}\right] \leq \lim_{\theta\to\infty}\left(\frac{1}{\theta}\right) \tag{4.49}$$

$$0 \leq \lim_{\theta\to\infty}\left[\frac{\sin(\theta)}{\theta}\right] \leq 0 \tag{4.50}$$

Again, it's not necessary to try to calculate the limit in the middle, because the only number in between 0 and 0 is 0. This result should also make sense intuitively: $\sin(\theta)$ has no finite limit because it continues oscillating forever, but when divided by θ, the ratio becomes a set of numbers bounded between -1 and +1 divided by a number that is so fantastically galactically large (approaching ∞) that it is effectively zero.

And of course, this result is also consistent with the graph, which shows that the function tapers towards zero as the function gets further from $\theta = 0$.

Figure 4.18C Now for the third problem. This problem does not rely on the squeeze theorem, but does rely on the two trig limits we solved in Section 4.8. Try to solve this on your own before reading my solution below.

Solving this problem involves re-expressing the tangent function as the ratio of sine to cosine, and applying a bit of algebra to arrive at the trig limits problems from earlier:

$$\frac{\tan(\theta)}{\theta} = \frac{\sin(\theta)/\cos(\theta)}{\theta} \tag{4.51}$$

$$= \frac{\sin(\theta)}{\cos(\theta)} \times \frac{1}{\theta} \tag{4.52}$$

$$= \frac{\sin(\theta)}{\theta} \times \frac{1}{\cos(\theta)} \tag{4.53}$$

After this algebraic manipulation, the limit is easy to calculate:

$$\lim_{\theta \to 0} \left(\frac{\tan(\theta)}{\theta} \right) = \lim_{\theta \to 0} \left(\frac{\sin(\theta)}{\theta} \right) \times \lim_{\theta \to 0} \left(\frac{1}{\cos(\theta)} \right) \tag{4.54}$$

$$= 1 \times 1 \tag{4.55}$$

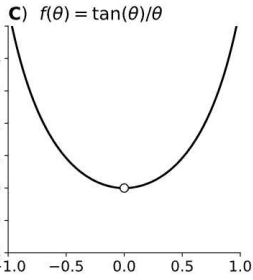


Figure 4.19: A zoom-in of panel 4.18C.

The result is not obvious from Figure 4.18C because of the y-axis scale. But the limit is visually consistent when zooming in (Figure 4.19).

4.11 Limits categories

Limits problems can be grouped into a few categories. Learning these categories can help you determine the appropriate strategy for solving difficult limits problems. Some of the strategies rely on L'Hopital's rule, which I mentioned briefly earlier in this chapter and which you'll learn about in Chapter 7.

I've compiled the categories into two figures according to whether the limit is to a finite number ($\lim_{x \to a}$) or to infinity ($\lim_{x \to \infty}$). Table 4.1 shows the categories for limits to finite numbers.

It's always a good idea to look at a visualization of a function to confirm what the math indicates, and that is especially true for

Label	Case	Outcome	Example
"Easy"	$\lim_{x\to a} f(x) = c$	c	$\lim_{x\to 2} \frac{x+3}{x-1} = 5$
Nonzero over zero	$\lim_{x\to a} f(x) = \frac{c}{0}$	$-\infty, \infty$, d.n.e.	$\lim_{x\to 2} \frac{x+2}{x-2} = \infty$
Indeterminant	$\lim_{x\to a} f(x) = \frac{0}{0}$	Try algebra or L'Hopital's rule	$\lim_{x\to 0} \frac{\sin(x)}{x} = 1$

Table 4.1: Categories of limits outcomes when calculating limits at a finite number.

the "nonzero over zero" category. It's easy to make arithmetic or algebraic mistakes, but limits of $\pm\infty$ are unmistakeable in a graph. Of course, ∞ is a perfectly reasonable limit; my advice here is to visually confirm that it's the correct answer and not a silly little math mistake. "Indeterminant" limits problems are often best solved using L'Hopital's rule, but you'll have to rely on other methods until you learn that trick ;)

Label	Case	Outcome	Example
Infinite over finite	$\lim_{x\to\infty} f(x) = \frac{\infty}{c}$	$-\infty, \infty$, undefined	$\lim_{x\to\infty} \frac{\sqrt{x}}{2} = \infty$
Finite over infinite	$\lim_{x\to\infty} f(x) = \frac{c}{\infty}$	0	$\lim_{x\to\infty} \frac{\cos x}{x} = 0$
Infinite over infinite	$\lim_{x\to\infty} f(x) = \frac{\infty}{\infty}$	Try algebra or L'Hopital's rule	$\lim_{x\to\infty} \frac{x^x}{x^3} = \infty$
Finite over finite	$\lim_{x\to\infty} f(x) = \frac{c}{d}$	Balanced race to infinity	$\lim_{x\to\infty} \frac{x^2}{2x^2} = \frac{1}{2}$

Table 4.2: Categories of limits outcomes when calculating limits at infinity.

I realize that there is a lot of information presented in these two tables, including terms and concepts that I haven't yet introduced. You will learn more about limits in later chapters, and see examples of each category in these tables. Indeed, proofs of several differentiation rules are based on these categories. You will refer back to these tables in later chapters.

4.12 Exercises

4.1. The goal of this exercise is to estimate one-sided limits empirically using Zeno's paradox[6]. The goal is to compute the following limit:

$$\lim_{x \to 1} \cos^2(x^2) + \pi \tag{4.56}$$

Implement that equation in `numpy`, and produce Figure 4.20.

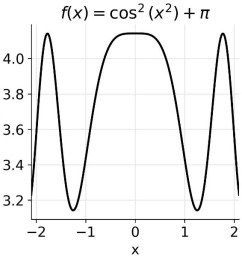


Figure 4.20: Function used in Exercise 1.

Now for the algorithm to estimate the limit. Start at $x_0 = 0$ and compute the function value. Then halve the distance to $a = 1$ and compute the function value. Halve the distance again, and again, for 10 iterations in total. Notice that each iteration involves getting closer to $x = a$ without exactly equaling a. This approximates the left limit; repeat for the right limit starting from $x = 2$. Report the results in a table like the following (first column is x, second column is $f(x)$):

```
Limit from the left:
[[0.          4.14159265]
 [0.5         4.08038393]
 [0.75        3.85718091]
 [0.875       3.66136066]
 [0.9375      3.54862869]
 [0.96875     3.49089446]
 [0.984375    3.46209466]
 [0.9921875   3.44777002]
 [0.99609375  3.44063426]
 [0.99804688  3.43707401]]

Limit from the right:
[[2.          3.56884264]
 [1.5         3.53619475]
 [1.25        3.14166148]
```

[6] This exercise is not exactly implementing Zeno's paradox; let's call it the Zeno-inspired limit-estimating algorithm: ZILEA.

```
[1.125       3.23186682]
[1.0625      3.32447611]
[1.03125     3.3776296 ]
[1.015625    3.40531326]
[1.0078125   3.41936069]
[1.00390625  3.42642726]
[1.00195312  3.42997022]]
```

You can see that the algorithm gets closer and closer to $x = 1$ from either side, and that the function values approach 3.43. Because this function is continuous at $f(1)$, you can use the plug-in method to calculate the function value:

```
Function value at x=1:
3.433519235316222
```

Finally, draw the function again with dots correspond to each sampled point on the function, as in Figure 4.21.

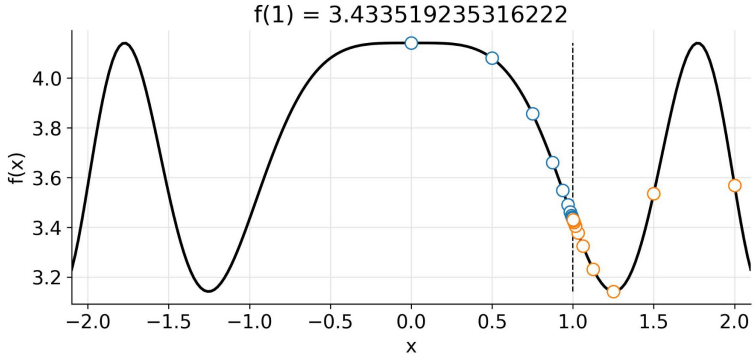


Figure 4.21: Plot for Exercise 1.

Note: This is not a rigorous approach to determining limits in practice; the goals here are to (**1**) solidify your understanding of what limits mean, and (**2**) to gain more experience with implementing mathematical algorithms in Python.

4.2. Now you will compute the limit in Equation 4.56 analytically using `sympy` with symbolic variables. The `sympy` function to obtain limits is `sym.limits`, and you will need to

read the docstring or look online to learn how to calculate one- vs. two-sided limits. Use the `display(Math())` function pair to print out the function definition and its value at $x = 1$. Your Python notebook output should look like this:

$$f(x) = \cos^2\left(x^2\right) + \pi$$

$$f(1) = \cos^2(1) + \pi$$

Notice that cosine function is not actually evaluated numerically; instead, `sympy` prints out the symbolic number.

Next, compute the two one-sided limits and the two-sided limit using `sympy`, and the value of the function at $x = 1$ using `numpy` and `sympy`. Display all results as shown in Figure 4.22. I printed the one-sided limit from the left twice: once as a symbolic result and once as a numerical approximation.

```
Limit as x approaches 1 from the left:
cos²(1) + π
3.43351923531622

Limit as x approaches 1 from the right:
3.43351923531622

Two-sided limit as x approaches 1:
3.43351923531622

Function value at limit (numpy):
3.433519235316222
```

Figure 4.22: Output of Exercise 2.

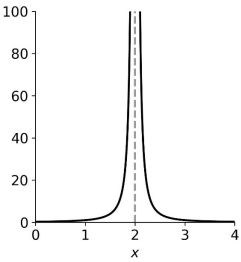


Figure 4.23: Function used in Exercise 3.

4.3. How does `sympy` deal with limits approaching infinite discontinuities? Let's find out! Implement the following function as a `sympy` object:

$$f(x) = \frac{1}{(x-2)^2} \tag{4.57}$$

Convert this `sympy` object into a regular Python function using

`sym.lambdify`, and use `matplotlib` to generate a plot like Figure 4.23.

Then compute three limits of this function (two one-sided and one two-sided) as $x \to 2$ in `sympy` and print out the results like this:

```
Limit as x->2 from the left:  oo
Limit as x->2 from the right: oo
Limit as x->2 (two-sided):    oo
```

`sympy` displays infinity (∞) as `oo` in ascii text.

Although graphing the function is not part of this exercise, it's always a good idea to make a quick plot to understand the function's behavior.

4.4. Let's continue exploring `sympy` and limits. Compute and print the following limits (these are Equations 4.10-4.12 reprinted here):

$$\lim_{x \to 2^-} \frac{|x-2|}{x-2} = ? \tag{4.10}$$

$$\lim_{x \to 2^+} \frac{|x-2|}{x-2} = ? \tag{4.11}$$

$$\lim_{x \to 2} \frac{|x-2|}{x-2} = ? \tag{4.12}$$

I won't print out the results here because I want you to discover for yourself what happens in `sympy` when two one-sided limits exist but the two-sided limit does not exist.

4.5. This and the next three exercises will allow you to demonstrate the four properties of limits that I explained in Section 4.6. These four exercises are closely linked.

We start here with the scalar-factorization property. Implement the following function in `sympy` and graph it as in Figure 4.24.

$$f(x) = \frac{x^3}{3} + 100\sqrt{|x|} \tag{4.58}$$

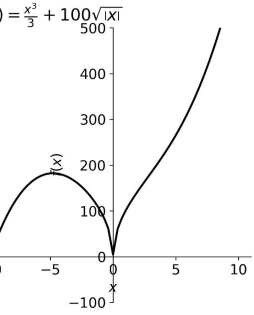


Figure 4.24: Function used in Exercise 5.

Next, generate any random constant γ (I used a random number generator, so the value will be different each time), and demonstrate that $\gamma \lim_{x\to 5}(f) = \lim_{x\to 5}(\gamma f)$. Print the results like this:

```
   lim(c*fx):
-534.052811185022

   c*lim(fx):
-534.052811185021
```

Note 1: Not only will your numerical results differ from mine, your results may differ each time you run the code if you generate a random γ. The point is that the two printed numbers are the same (within some reasonable tolerance for computer rounding errors — notice that my two results above differ on the final digit).

Note 2: A friendly reminder that *demonstrations* using specific examples in code are not the same thing as rigorous proofs. Demonstrations and visualizations are amazing tools for building flexible and lasting intuition in mathematics.

4.6. Next up is the additive property of limits. The goal is to demonstrate that $\lim(f+g) = \lim(f) + \lim(g)$. Use the following two functions (in `sympy`).

$$f(x) = \ln(x) + x^2 \tag{4.59}$$

$$g(x) = e^{-x} + x^3 \tag{4.60}$$

Choose a value at which to evaluate these limits (I used $x \to \pi$ to obtain the values below, but you can use any limit value you like), and print the limits as follows:

```
     lim(f+g) = 42.0638248855023
lim(f)+lim(g) = 42.0638248855023
```

4.7. Now for the multiplicative property. Use the same two functions as in the previous exercise, but compute and print $\lim(f \times g)$ and $\lim(g) \times \lim(g)$.

```
       lim(f*g) = 341.989468891742
lim(f) * lim(g) = 341.989468891742
```

And again: Your numbers will differ from mine if you use a different limit value; the important thing is that you get the same results for both expressions.

As a final test here, compare $\lim(f^3)$ with $\lim(f) \times \lim(f) \times \lim(f)$.

4.8. Final exercise in this set: the divisive property. Same procedure as above: Use $f(x)$ and $g(x)$ from Exercise 6 and show that $\lim(f/g) = \lim(f)/\lim(g)$.

Finally, create function $h(x) = x^3 + x^2 + x$, and print $\lim_{x \to 0} g(x)$, $\lim_{x \to 0} h(x)$, and $\lim_{x \to 0} g(x)/h(x)$.

4.9. `sympy` struggles with some limits problems. Let's explore an example. Implement the following piecewise expression using the `Piecewise` function in `sympy`, and visualize the function using `sym.plot`.

$$f(x) = \begin{cases} \sin(\pi x) & \text{for } x < 0 \\ 1.5 & \text{for } x = 0 \\ -(x-2)^2 & \text{for } x > 0 \end{cases} \tag{4.61}$$

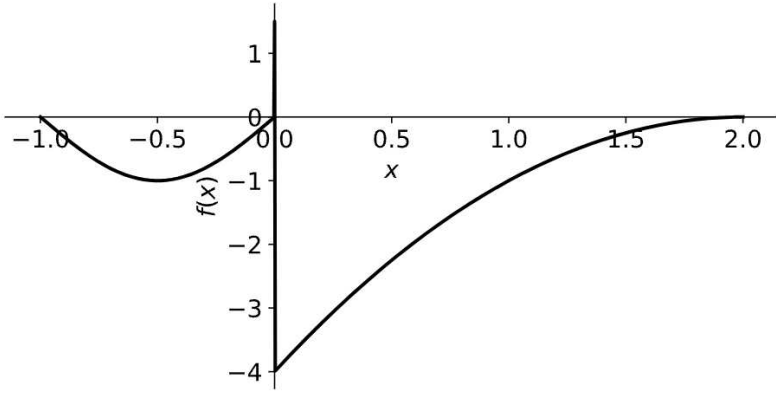


Figure 4.25: Output of Exercise 9.

Notice that `sympy` draws lines between the discrete pieces, which you saw in Chapter 3.

Now for the test. Compute and print the two one-sided limits and two-sided limit as x approaches zero.

```
Limit as x approaches 0 from the left:
1.50000000000000

Limit as x approaches 0 from the right:
3/2

Two-sided limit as x approaches 0:
3/2
```

Finally, use `np.finfo(float).eps`[7] to calculate the value of the function just below, and just above, the limit value. That is, compute and print $f(a - \epsilon)$ and $f(a + \epsilon)$:

```
Function value just left of limit:
-sin(pi/4503599627370496)

Function value at limit:
1.50000000000000
```

[7] "eps" is machine epsilon, and corresponds to the numerical precision of the computer. It's somewhere around 10^{-16}

```
Function value just right of limit:
-4.00000000000000
```

Putting it together, `sympy` is able to understand and evaluate the function values at and surrounding the discontinuities, but it does not accurately state the one-sided limits. The conclusion here is that `sympy` is a powerful library for symbolic calculations, but you should always verify its output through a combination of additional testing, visualization, and common sense. As the famous adage goes: Trust but verify.

4.10. Is `sympy` more accurate when working with oscillating discontinuities? Discover the answer by implementing the function $f(x) = \sin(1/x)$. Generate a plot like Figure 4.26, and then compute and print the limits to match my output below.

```
Limit as x approaches 0 from the left:
AccumBounds(-1, 1)

Limit as x approaches 0 from the right:
AccumBounds(-1, 1)

Two-sided limit as x approaches 0:
AccumBounds(-1, 1)
```

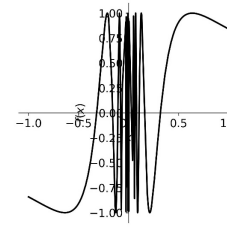


Figure 4.26: Function used in Exercise 10.

The output `AccumBounds(-1,1)` indicates that the function does not approach a single finite value, but instead oscillates between -1 and 1. That is, the answer to the limit is the accumulation of values between -1 and $+1$.

4.11. Equation 4.62 is interesting; implement and visualize it in `sympy` to produce Figure 4.27.

$$f(\theta) = \theta^2 e^{-\theta^2} \ln\left(\theta^2\right) \sin(\theta) \tag{4.62}$$

The function is undefined at $\theta = 0$ because of the ln term (and you should confirm this in `sympy`). It appears, based on visual inspection, that the limit as $\theta \to 0$ is 0. Confirm this result using `sympy`.

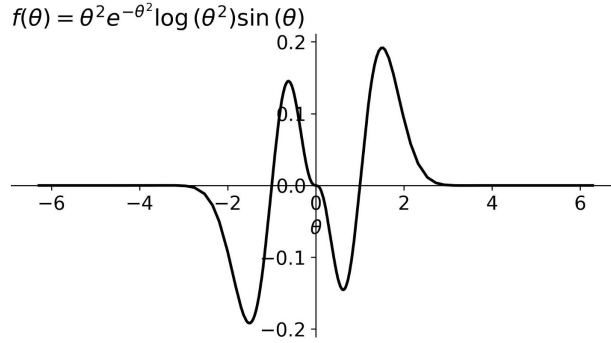


Figure 4.27: Visualization for Exercise 11.

Now let's think about the domain of this function. The natural log is defined only for positive inputs, but the variable in the log operator is θ^2. Thus, the function is defined for all real-valued θ except for $\theta = 0$. Use `sympy` to confirm the domain of this function (consult Exercise 3.15 for a reminder about the relevant `sympy` function). You should get $(-\infty, 0) \cup (0, \infty)$

$\lim_{\theta \to 0} \theta^2 = 0$

$\lim_{\theta \to 0} e^{-\theta^2} = 1$

$\lim_{\theta \to 0} \log(\theta^2) = -\infty$

$\lim_{\theta \to 0} \sin(\theta) = 0$

oduct of all limits is NaN

Figure 4.28: `sympy` output from Exercise 11.

Final part of this exercise: The multiplicative property of limits states that the limit of the product of functions equals the product of the limits of those functions. The function at hand in this exercise is the product of four terms. Compute the limit of each term, the product of all terms, and report the results as in Figure 4.28. Finally, try this again for another value where you expect a finite limit, e.g., $\theta = 1$.

4.12. The purpose of this exercise is simple: Use `sympy` to double-check my work in the expressions below. Perhaps I made a mistake ;)

I encourage you to write the code for this exercise *de novo* — without looking back at any other code from this book — as a way to test your Python-calculus coding skills.

$$\lim_{\phi \to 0} \left(\frac{\cos(\phi) - 1}{\phi} \right) = 0, \qquad \lim_{\phi \to 0} \left(\frac{\sin(\phi)}{\phi} \right) = 0$$

$$\lim_{\theta \to 0} \left[\theta^2 \sin\left(\frac{1}{\theta} \right) \right] = 0, \qquad \lim_{\theta \to \infty} \left(\frac{\sin(\theta)}{\theta} \right) = 0, \qquad \lim_{\theta \to 0} \left(\frac{\tan(\theta)}{\theta} \right) = 1$$

Chapter 5

Fundamentals of differentiation

This might sound strange or difficult to believe, but there isn't anything new to learn in the rest of this book. Functions and limits are the basis for everything in calculus. For the rest of this book, everything "new" that you will learn is just different or unintuitive ways of combining limits and algebra.

In this chapter, you will learn how to differentiate individual functions, and then in the next few chapters, you'll learn the rules for differentiating combinations of functions, applications of differentiation, then on to integration.

One small point about grammar: You will come across the terms *derivatives* and *differentiation*. Derivatives are the slopes of a function at each value of x. A derivative is a noun. Differentiation is the process of obtaining the derivative. It's an algorithm you apply. A verb. So, the goal of differential calculus is to obtain the derivative of a function, and the way you get that derivative is through a technique called differentiation.

5.1 Slope of a line

You may be wondering why there is a section on slopes in a chapter on derivatives. Here's why: The derivative is the slope of a line. More specifically, the slope of a tiny line segment. If you are already comfortable with the math and interpretation of a slope, then that's great! You have a head start on learning about derivatives and how to calculate them. By the end of this section, you will have a deep intuition for how slopes of lines lead to derivatives, and then in the next section you'll learn the formal definition of a derivative (hint: it's what happens when you combine slopes of lines with limits to zero).

Calculating slope What is the slope of the line in Figure 5.1? See if you can calculate it just based on the information provided in the graph.

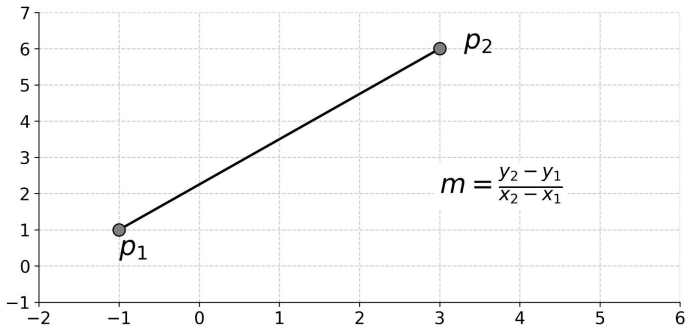


Figure 5.1: What is the slope of this line?

I hope you were able to determine that the slope is 5/4 (1.25 is an equally acceptable answer). How did you get that number? I guess you quantified the change in the y-axis by the change in the x-axis. The function traversed 5 units on the y-axis ($y = 1$ to $y = 6$) while traversing 4 units on the x-axis ($x = -1$ to $x = 3$). In fact, you calculated the following formula:

$$m = \frac{\Delta y}{\Delta x} \tag{5.1}$$

m is the common variable name to indicate the slope. It comes from the equation for a line, which you've seen in Chapter 3 when I introduced the difference between linear and nonlinear equations.

$$y = mx + b \tag{5.2}$$

As a reminder, m is the slope and b is the intercept, which is the value of the function when $x = 0$; in other words, where the function line crosses the y-axis. Importantly, m and b are separate parameters: you can have an infinite number of functions with the same slope but different intercepts (Figure 5.2). This also means that if I give you a value m and ask you to reconstruct the line, you wouldn't know the exact line to reconstruct; you would only know how to set the angle of that line. This is an extremely

important concept in calculus, and in particular, when integrating functions.

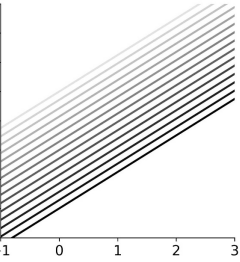


Figure 5.2: 15 lines that have identical slope ($m = 5/4$) and different intercepts.

One final point before moving on: The slope of a perfectly vertical line is undefined. That's clear from Equation 5.1: If the line moves straight up, then it doesn't change on the x-axis, which means the denominator of the slope equation is zero.

Two slopes Now consider Figure 5.3, which shows a piecewise function comprising two linear segments. What's the slope of the line depicting this piecewise function?

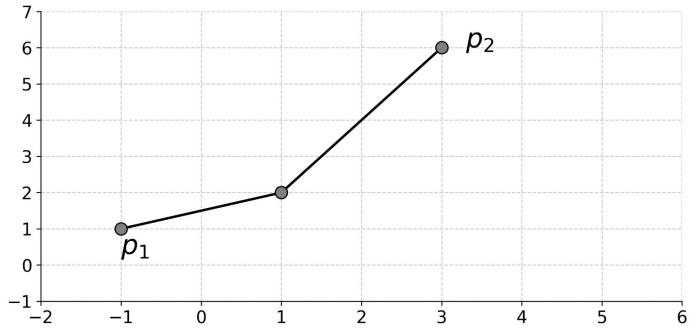


Figure 5.3: A piecewise function.

I hope you have a feeling of unease and confusion when looking at that line with the goal of calculating its slope. Sure, you could calculate the slope of a line that directly connects points p_1 and p_2 (the slope of that line would be 5/4), but that line is not actually drawn. The line that is actually drawn doesn't have *one* slope. It is a line comprising two segments, each with a different slope. Measuring one slope between points p_1 and p_2 isn't really wrong per se, but it misses important information about the function.

I am sure you agree that the most accurate approach here is to label that third point p_3 and then calculate two slopes for that line: the slope $m_{1\to3}$ and the slope $m_{3\to2}$. (Those slopes are, respectively, 1/2 and 2.)

Many slopes I am also sure that you see where this is going: The more segments a line has, the more slopes we can use to

characterize that line. How many slopes would you need for the line in Figure 5.4?

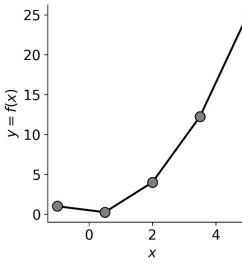


Figure 5.4: Five points describe a piecewise function comprising four segments.

The answer is four. That function has five points, and the slope is defined as the difference between successive points, and so that leads to four slopes. That's a general feature of computing discrete differences: A line defined by N points has $N-1$ segments with $N-1$ slopes, and could be defined as a piecewise function with $N-1$ pieces. This will be relevant for computing *discrete differences*, which is basically the digitized version of a derivative. It's also how you calculate the derivative of an empirical function that is not defined by a mathematical formula (for example, empirical time series data).

The examples I've shown so far are all straight line segments. But lots of functions are not represented by straight lines; they are represented by curves. How can we adapt this idea of multiple slopes for multiple line segments to characterize a curve like x^2? The answer is in the next section, but I encourage you to think of an answer based on the discussion above and on what you learned in the previous chapter.

5.2 Formal definition of derivative

The derivative is conceptually straightforward: calculate the slopes of small line segments. How small are those line segments? They are really *really* small. So tiny, in fact, that they are almost zero-length long. In fact, a derivative is the sequence of slopes of the line segments as the x-axis distance goes to zero:

$$\lim_{\Delta x \to 0} \frac{\Delta y}{\Delta x} \tag{5.3}$$

I hope that equation makes sense and feels intuitive after reading the previous section. However, that's not the definition of

a derivative that you normally see. The formal definition is as follows; before reading the text below, please take a moment to understand how the following equation expresses the same concept as in Equation 5.3.

$$\frac{dy}{dx} = \lim_{h \to 0} \left[\frac{f(x+h) - f(x)}{h} \right] \tag{5.4}$$

This definition is called the "limits-based definition" or the "incremental form" of the derivative. It is one of the most important equations in all of calculus, so I will explain each part of it in detail.

dy and *dx*

dx is the same as Δx but for really tiny steps. We use Δ for "big" intervals and d for "tiny" intervals; hence, Δx indicates slopes and dx indicates derivatives. What's the cut-off for "big" vs. "tiny"? Basically, in terms of calculus, any number that you can think of is "big", and any number that is so incredibly tiny that it's not really distinguishable from zero (though it is technically bigger than zero) is "tiny." In terms of Python implementation, `sympy` is capable of representing dx whereas anything done in `numpy` can be considered Δx.

You might be tempted to use the term "infinitesimal" (infinitesimal is $1/\infty$) — that word was historically used in calculus but it fell out of fashion and modern calculus is built on the idea of limits to zero instead of infinitesimals. You can think of dx as what happens when the step size of Δx goes to zero. Of course, dy is the same concept but reflects how much the function changes on the y-axis for each tiny line segment defined by dx. dy therefore depends on the definition of the function, whereas dx involves cutting up the x-axis grid to vanishingly small intervals.

h

This is a short-hand for Δx. h is easier and faster to write. Think of h as the vanishingly tiny discretization of the x-axis.

You can immediately see from Equation 5.4 that the plug-in

method won't solve this limit problem. That's the difficult part of differentiation: finding the tricks that will solve this limit problem for each type of function. Once you solve the limit problem, you have the derivative.

f(x)

This is simply the function that we're trying to differentiate.

Δy refers specifically to two values on the y-axis. Replacing Δy with $f(x+\Delta x)-f(x)$ makes the equation more general (recall the discussion in Chapter 2 about the difference between y and $f(x)$). Therefore, as Δx shrinks down to dx, Δy shrinks down to dy.

How to pronounce *dy/dx*

There are several different notations for the derivative (Leibiz, Newton, Lagrange, Euler) that I will explain in Section 5.3. dy/dx is one of the most common notations, and is pronounced as "dee why by dee ex," "dee why dee ex," "dee by dee ex," or "the derivative of why with respect to ex." You will also see df/dx for function $f(x)$, or sometimes d/dx.

x is a commonly used variable name, but you will see other letters. For example, dy/dt might indicate the derivative over time, dy/ds might indicate the derivative over space, $dy/d\theta$ might indicate the derivative over angles, etc. Similarly, the derivative of function $r(\tau)$ would be written $dr/d\tau$.

5.2.1 Example with a linear function

The first example will involve using the derivative definition to confirm the slope of the line in Figure 5.1. The main goal is to warm you up to working with Equation 5.4.

The function that defines that line is $f(x) = \frac{5}{4}x + \frac{9}{4}$. Let's set $h = 1$ and we will pick a specific value of $x = 0$.[1] Here's how that looks:

[1] $x = 0$ is a convenient choice because it simplifies the arithmetic. You should try it again using $x = 1$ or $x = \pi^{\sqrt{e^{17}}}$ or any other number. It won't affect the final result in this example.

When h=1, the plug-in method solves this limit.

$$m = \lim_{h\to 1}\left[\frac{f(x+h)-f(x)}{h}\right] \tag{5.5}$$

$$= \left[\frac{\left(\frac{5}{4}(0+1)+\frac{9}{4}\right)-\left(\frac{5}{4}0+\frac{9}{4}\right)}{1}\right] \tag{5.6}$$

$$= \left(\frac{5}{4}+\frac{9}{4}\right)-\frac{9}{4} \tag{5.7}$$

$$= \frac{5}{4} \tag{5.8}$$

Notice that the intercept canceled. That's not a quirk of this particular example — the intercept will *always* cancel in the derivative formula because the intercept does not multiply an x, and therefore is unaffected by $x + h$. And that is why an infinite number of functions with different intercepts can have identical derivatives. Pin that concept in the back of your mind; it becomes important later in the book when learning about integrals and antiderivatives.

Perhaps you think this wasn't a good illustration of the derivative formula, because I set the limit as $h \to 1$ and picked one specific value for x. Fair point; I accept your criticism.

Let's try it again, with the limit to zero and using the variable x instead of a specific value of x:

$$\frac{dy}{dx} = \lim_{h\to 0}\left[\frac{f(x+h)-f(x)}{h}\right] \tag{5.9}$$

$$= \lim_{h\to 0}\left[\frac{\left(\frac{5}{4}(x+h)+\frac{9}{4}\right)-\left(\frac{5}{4}x+\frac{9}{4}\right)}{h}\right] \tag{5.10}$$

$$= \lim_{h\to 0}\left[\frac{\frac{5}{4}x+\frac{5}{4}h+\frac{9}{4}-\frac{5}{4}x-\frac{9}{4}}{h}\right] \tag{5.11}$$

$$= \lim_{h\to 0}\left[\frac{\frac{5}{4}h}{h}\right] \tag{5.12}$$

$$= \frac{5}{4} \lim_{h \to 0} \left[\frac{h}{h} \right] = \frac{5}{4} \tag{5.13}$$

A few remarks:
(**1**) Notice that x is replaced by $x + h$ in the function definition (Equation 5.10). It's not so confusing in this example because there is only one x in the function, but this is a source of mistakes when differentiating, so be mindful of replacing x with $x + h$ in the first function term in the numerator.
(**2**) Even though we used the variable x instead of a specific value of x, it still canceled to give a final derivative of the function that equals the slope of the line. That does not happen for all functions; it is a unique property of straight lines.
(**3**) Equation 5.13 relied on the factorable property of limits.
(**4**) The plug-in method seems like it would have failed, but we ended up with h/h. That is *not* $0/0$ — remember we are taking the limit *as h approaches but does not exactly equal zero.* In other words, h is an exceedingly small number, and when divided by itself, equals 1.

5.2.2 Example with a quadratic function

A quadratic function is any 2^{nd} order polynomial, thus a function of the form $f(x) = ax^2 + bx + c$ where $a \neq 0$.

Now let's differentiate the function $f(x) = x^2$. The full solution is shown below, but I encourage you to try it on your own first: Start from Equation 5.4 and keep working with the equations until you can solve the limit and obtain a dy/dx for $f(x) = x^2$.

$$\frac{dy}{dx} = \lim_{h \to 0} \left[\frac{(x+h)^2 - x^2}{h} \right] \tag{5.14}$$

$$= \lim_{h \to 0} \left[\frac{x^2 + 2xh + h^2 - x^2}{h} \right] \tag{5.15}$$

$$= \lim_{h \to 0} \left[\frac{h(2x + h)}{h} \right] \tag{5.16}$$

$$= \lim_{h \to 0} [2x + h] \tag{5.17}$$

$$\frac{dy}{dx} = 2x \tag{5.18}$$

I hope you were able to derive $2x$ on your own. If you struggled with your work and looked at the answer above, then I now encourage you to read through my remarks below, then try it again on your own.

Remarks: (**1**) Again, it seems like the plug-in method would fail, but some algebraic expansions and simplifications allows us to cancel the h from the denominator.
(**2**) There are no problems when plugging in $h = 0$ outside the denominator (Equation 5.17).
(**3**) Even for this simple function, the algebra can get complicated with the substitutions and cancellations. Never rush through a differentiation problem, and never hesitate to check your work.
(**4**) Notice that the derivative here has an x, unlike the derivative for a straight line in the previous example.

5.2.3 Proof: Derivative of a constant is 0

Here is a fact:

$$f(x) = c \tag{5.19}$$

$$\frac{d}{dx} f(x) = 0 \tag{5.20}$$

In other words, the derivative of any constant is zero. I'm sure you agree that that is an intuitive fact: A function that is flat along the x-axis has a slope of zero. The proof of that claim is simple, and involves applying the derivative formula (Equation 5.4) to the function $f(x) = c$ and solving the limit. I'm going to show the proof below, but I encourage you to work through it on your own first.

$$f(x) = c \tag{5.21}$$

$$\frac{df}{dx}f(x) = \lim_{h\to 0}\frac{f(x+h)-f(x)}{h} \tag{5.22}$$

$$= \lim_{h\to 0}\frac{c-c}{h} \tag{5.23}$$

$$= \lim_{h\to 0}\frac{0}{h} \;=\; 0 \tag{5.24}$$

Because the function does not include an x term, there is nowhere to substitute h. Thus, the numerator cancels to zero. You might think that this produces a fraction of $0/0$ as $h \to 0$, but remember that h is not exactly zero; it's a number that is fantastically close to zero. In the expression $0/h$, we don't need to plug in $h = 0$ to see that the fraction equals zero for any value of h that is not exactly equal to zero.

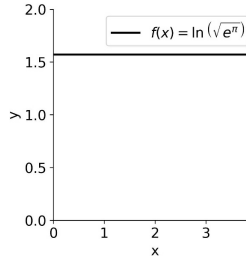


Figure 5.5: A constant function doesn't change on the y-axis, and therefore has a derivative of zero.

Now you've seen a few examples of applying the differentiation formula. What do the results mean and how can you interpret them? That's the topic for the rest of this chapter — and the next five chapters.

But first, I have an assignment for you: Write down Equation 5.4 on a piece of paper many times. Maybe three times. Maybe seven times. That equation is one of the most important in calculus, and is worth committing to memory. And not just memorizing, but understanding the simple but powerful concept that the derivative of a function is how much the output of a function (y) changes when its input (x) changes by a tiny amount.

5.3 Various notations for the derivative

There are several notations for the derivative. Four of which are listed below, organized by their namesake (left) and the derivative notation (right). The Leibniz and Lagrange notations are most commonly used; Newton's notation is common in physics when

studying temporal derivatives. Euler's notation is not commonly used for single-variable differentiation but is used in multivariable functions[2].

Leibniz	$\frac{dy}{dx}$
Lagrange	f'
Newton	$\dot{y}$
Euler	$\mathbf{D}f$

You may find it confusing or annoying that there are different notations for the same concept. To some extent, that is a fair reaction. But each notation has both conceptual and typographical advantages, and there are reasons for learning and using each format. I will write brief descriptions below, and you'll learn more about these notations as you progress through the book.

Leibniz (*dy/dx*)

This notation is conceptually the most enlightening because it matches the equation for the slope of a line. As I wrote earlier, you'll also see variations like df/dx and d/dx. Leibniz notation is also illuminating when learning about the chain rule $\left(\frac{dy}{du} = \frac{dy}{dx} \cdot \frac{dx}{du}\right)$ and differential equations $\left(\frac{dy}{dx} = k \Rightarrow dy = dx\, k\right)$.

A disadvantage of Leibniz notation is that it is cumbersome to write out. That may seem like an excuse for laziness, but when equations become dense and copious, compact notations can be the difference between comprehension and a massive headache.

Lagrange

This is a popular notation because it is compact and elegant. f' (read out loud as "eff prime") is the derivative of f. You can also write $f'(x)$ or f'_x to indicate the derivative with respect to x (c.f. f'_θ or f'_t). This notation is also convenient because it can be written without any LaTeX or other fancy formatting.

A disadvantage of the Lagrange notation is that it is not as conceptually illuminative as the Leibniz notation.

[2] According to Wikipedia, this notation was actually developed by Louis Arbogast although it is refered to as Euler's notation.

Newton

Newton's notation ($\dot{y}$) is compact, but also requires special formatting tools on a keyboard. The dot on top can also easily be missed if the font size is small, the pen is leaking, the printer has ink smudges, or the laptop screen is dirty. If you study changes over time in a physics course, you may use Newton's notation. I won't use it in this book, and mention it here only so that you can recognize it.

Euler's notation

Euler's notation for single-variable derivatives is not commonly used. I suspect that's because it has no real advantages over the Lagrange notation, and is easily confused with the notation to indicate the domain of a function. Euler's notation is more commonly used for partial derivatives, and I will introduce that flavor of the notation later in the book.

One more bit of notation: Instead of explicitly defining a function $f(x) = ...$ and then notating its derivative using $f'(x)$, you can indicate the derivative of an expression by applying one of the derivative operators directly to an expression, as in one of these examples:

$$\frac{d}{dx}\left[\ln(4x) - e^{x-x^2} + \cos(\pi x)\right] \tag{5.25}$$

$$\Big(\ln(4x) - e^{x-x^2} + \cos(\pi x)\Big)' \tag{5.26}$$

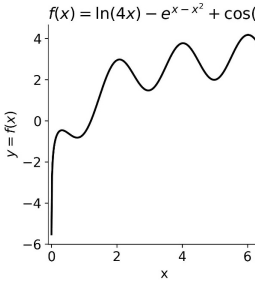


Figure 5.6: Equation 5.25 visualized.

5.3.1 Geometric terms

There are several geometric terms that are relevant for differential calculus, and in this section you will learn two: *tangent* and *secant*.

I'm going to start by providing definitions that are over-simplified and therefore not entirely correct. But I believe that introducing the concepts this way — and then correcting them — will help build your critical thinking skills. While reading the definitions

below, please inspect Figure 5.7 and think about why the definitions are imprecise, possible failure scenarios, and how the terms could be more precisely defined.

Tangent line (dotted line in Figure 5.7)

A tangent line passes through a function at one point.[3]

Secant line (dashed line in Figure 5.7)

A secant line passes through a function at two points.

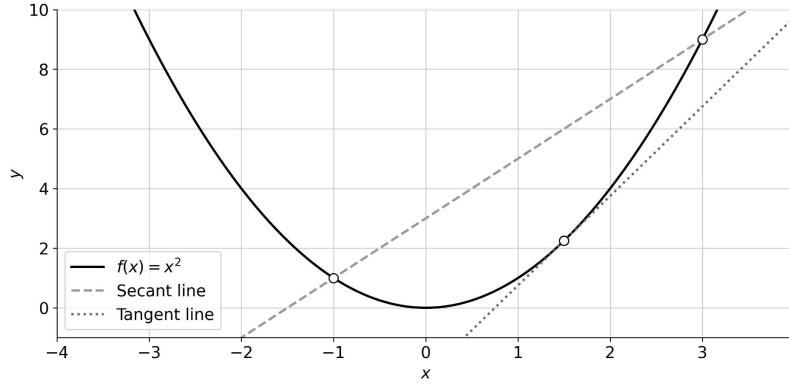


Figure 5.7: Tangent and secant lines on a function. You will write code to generate this figure in Exercise 6.

Do you have any thoughts about how those definitions might go awry? In fact, the graphs in Figure 5.7 fit perfectly with the the definitions above.

The problem with those definitions is that they aren't specific enough. Consider Figure 5.8. Panel A fits the tangent definition above, but it is not a tangent line. Panel B shows a tangent line, but it passes through the function at more than one point.

Here are more precise definitions:

Tangent line

A tangent line is a straight line with a slope equal to the derivative of a function at the point at which the tangent line inter-

[3] You might be familiar with a "tangential conversation," like when you're talking to your mother about dinner plans and she suddenly reminds you of something inappropriate you said when you were 5 years old.

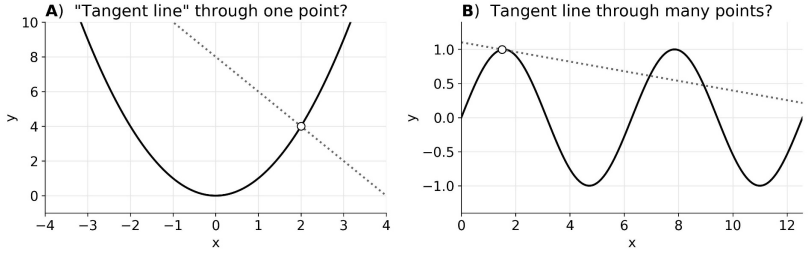


Figure 5.8: Panel A shows a line that touches the function once but is not a tangent line. Panel B shows a true tangent line that touches the function at multiple points.

sects the function. (The dotted line in Figure 5.8A is not a tangent line.)

Secant line

A secant line intersects a function at two or more points, and the slope of the secant line is defined from two intersection points. (The dotted line in Figure 5.8B is both a tangent and a secant.)

There are simple algorithms to find the equation for tangent and secant lines. You'll learn and implement those algorithms in Exercise 6.

5.4 Interpreting derivatives plots

The derivative is a *function*. It is a function of x just like the function it was computed from. It may seem like the derivative of the line is not a function, but it is: $f(x) = m$. There is no x term in the function definition, meaning this function does not vary — it's a flat line from $-\infty$ to $+\infty$.

Figure 5.9 shows two functions we worked with earlier (panels A and C on the top row) and their derivatives (panels B and D on the bottom row).

If you are confused at how the derivative relates to the function,

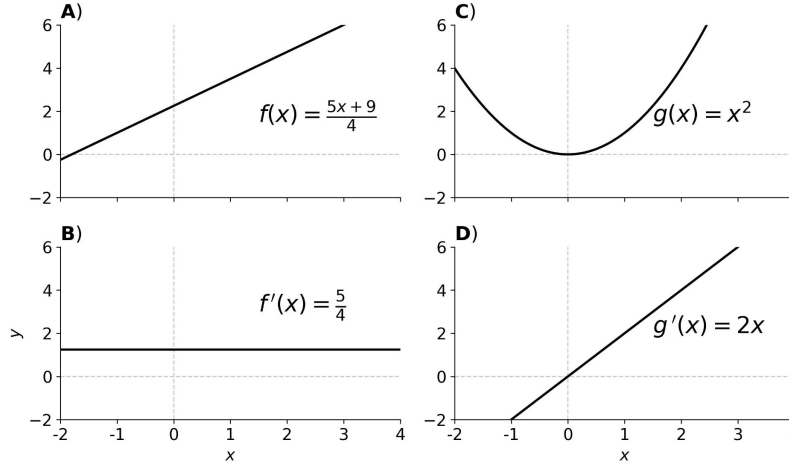


Figure 5.9: Two functions (top row) and their derivatives (bottom row).

then that is perfectly normal. Everyone gets confused when they are new to looking at plots of derivatives, because they expect them to be closely visually related to the original functions.

The linear function (panel A) has a constant slope for all values of x. The derivative of that function (panel B) is a flat line, because the slope doesn't change for different values of x. $f(x)$ goes up at exactly the same rate, regardless of whether x is negative, positive, large, small, etc.

On the other hand, the quadratic function (panel C) does not have a constant slope; the slope changes depending on the value of x. x^2 curves down for $x < 0$, is momentarily flat when $x = 0$, and then curves up for $x > 0$.

The first thing to look for in a derivative plot is the sign of the derivative (ignoring the actual values).

The graph of the derivative is a straight line with a positive slope that touches $x = y = 0$. First, just think about the sign of the derivative, ignoring its value. When x^2 curves downward, its derivative is negative. And when x^2 curves upwards, its derivative is positive. So, the sign of the derivative isn't telling us about the *value* of the function; it's telling us about the *direction* of the function as it moves to the right on the x-axis.

Now let's consider the numerical values of the derivative (not just

the sign). The values of the derivative encode how fast the function changes. When the derivative is closer to zero on the y-axis, the function changes relatively slowly (gentle slope), and when the derivative is further from zero on the y-axis, the function changes faster. Now looking back to the original function in panel C, x^2 changes faster and faster (that is, greater slopes) the further away it gets from $x = 0$, while the closer it gets to $x = 0$, the more slowly it changes on the y-axis.

I hope that makes sense. Please take a while to stare at Figure 5.9 until it does make sense. The more time you invest in understanding derivatives plots now, the easier you'll find the rest of calculus.

5.4.1 More examples

Please take a few moments to study Figure 5.10. Try to see how each of the three functions in the top row relates to their derivatives in the bottom row. Don't worry about how I obtained the derivatives; you'll learn that over the next several sections. For now, just focus on trying to match the *changes* in the function to the *values* of the derivative.

In particular, answer the following two questions for each column: (**1**) What is the *sign* of the derivative when the function goes up vs. down; (**2**) What is the *value* of the derivative when the function *changes direction*?

Panels A and B

$f(x)$ mostly goes upwards as x increases, but there is a piece in the middle where $f(x)$ decreases. Where $f(x)$ increases, $f'(x)$ is positive, and where $f(x)$ decreases, $f'(x)$ is negative. Also notice that the x-axis values at which $f(x)$ transitions from going up to going down correspond to zero-crossings in $f'(x)$. In other words, $f' = 0$ when f changes direction. These are called "critical points" and I'll discuss them in more detail in the next chapter.

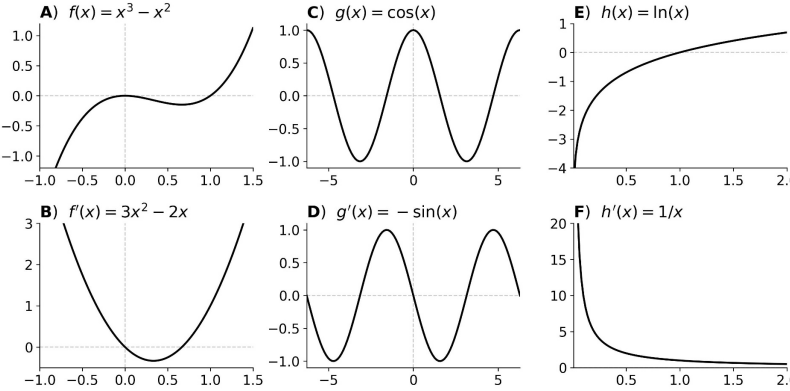


Figure 5.10: Three more functions (top row) and their derivatives (bottom row).

Panels C and D

Let's start at $x = 0$. $g(x)$ starts out momentarily flat; its slope at $x = 0$ is $g' = 0$. Then g goes down — first steeply then more gently — until it reaches $y = -1$, and which point we have another moment of a horizontal tangent, then it bends upwards until $y = 1$ and the cycle repeats. *Ignore your first instinct to think that g' is a shifted version of g.* (In fact, it is a shifted version, but that's not how you want to look at it.) Instead, focus on how the *values* of the derivative reflect the *changes* in the function. The steeper g decreases, the more negative g'; when g changes from going downwards to going upwards, $g' = 0$.

Panels E and F

Ignore your instinct to think that $1/x$ is the inverse of $\ln(x)$; e^x is the inverse of $\ln(x)$. Inverses are not derivatives.

The natural log function starts negative and goes upwards. It never stops going up. The *rate* at which $h(x)$ increases, decreases. That is, the larger the value of x, the slower $h(x)$ increases, but it's never flat and never bends downwards. Notice how this behavior manifests in the derivative: $h'(x)$ is always positive; is very large close to $x = 0$, and gets smaller and smaller as x increases but never touches zero or becomes negative.

It takes some time and practice to develop the skill of interpreting plots of functions and their derivatives. It's a skill that you will continue to improve throughout the rest of this book.

Three final messages for this section:

1. The most important thing to remember about the plot of a derivative is that it shows the slopes of the function at each point. Not the *values* of the function, but the *changes* in the function at each x-axis value.

2. The plot of a derivative in general looks nothing like the plot of its function, with a few noteworthy examples that I'll introduce later, such as sine and e^x.

3. Look for different features in plots of functions vs. derivatives. When inspecting a function, look at its values, behavior, striking or unusual parts, etc. When inspecting a derivative, focus on the sign and zero-crossings, and look at the value of the derivative only thereafter.

5.5 Differentiation is linear

What does it mean for differentiation to be a linear operator? Mathematically, it is described by Equation 5.27; try to understand what this equation means before reading the text below (α and β are two constants).

$$\frac{d}{dx}\big[\,\alpha f(x) + \beta g(x)\,\big] \;=\; \alpha f'(x) + \beta g'(x) \tag{5.27}$$

There are two operations described by Equation 5.27, one about scalar multiplication and one about addition. Let's discuss the multiplication part first.

Scalar multiples can be factored out of derivatives, just like how they were factored out of limits.

$$\frac{d}{dx}\big[\,\alpha f(x)\,\big] \;=\; \alpha \frac{d}{dx}\big[\,f(x)\,\big] \tag{5.28}$$

In other words, the derivative of a constant times a function equals that constant times the derivative.

The next part of linearity is that the derivative of the sum of two functions equals the sum of the derivatives of those functions:

$$\frac{d}{dx}\big[\, f(x) + g(x) \,\big] \;=\; \frac{d}{dx}\big[\, f(x) \,\big] \;+\; \frac{d}{dx}\big[\, g(x) \,\big] \tag{5.29}$$

Equation 5.27 combines those two concepts into one line.

The linearity of differentiation is useful because it allows you to break down complicated multiterm functions into a collection of smaller, more manageable parts.

Figure 5.11 shows a demonstration of linearity. The plot shows the following two derivatives:

$$v_1(x) = \left[\, 3x^3 - \frac{2}{\pi}\sqrt{\cos(2\pi x) + 1} \,\right]' \tag{5.30}$$

$$v_2(x) = 3\left[\, x^3 \,\right]' + \frac{2}{\pi}\left[\, -\sqrt{\cos(2\pi x) + 1} \,\right]' \tag{5.31}$$

If differentiation is linear according to Equation 5.27, then the two lines should overlap perfectly in the plot. And they do! On the other hand, this is merely a demonstration with two specific functions and two specific constants. It is necessary to prove that the linearity claim is valid for any function and any constant.

5.5.1 Proof of linearity

Intuitively, you can understand that derivatives are linear because they are merely an application of limits, and limits are linear.

That said, the linearity of differentiation is straightforward to prove, and involves expanding Equation 5.27 using the limits def-

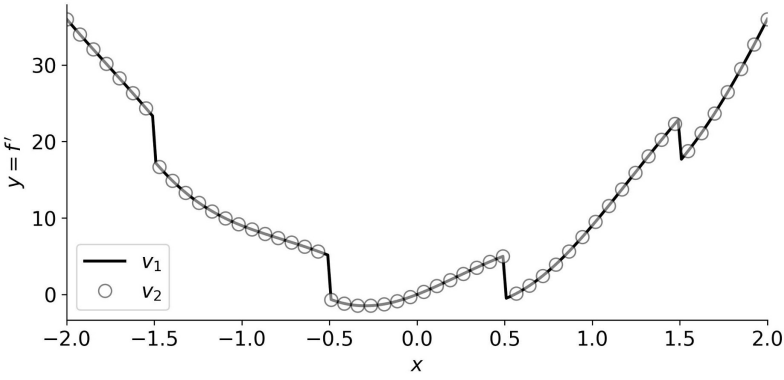


Figure 5.11: Demonstration of the linearity of differentiation. $\mathbf{v}_2$ was down-sampled by a factor of four to facilitate visualization.

inition of differentiation. The sequence of equations is below. As with other proofs in this chapter, this one may look intimidating at first, but I think you'll find it manageable when comparing each equation to the previous.

$$k(x) = \alpha f(x) + \beta g(x) \tag{5.32}$$

$$k'(x) = \lim_{h \to 0} \frac{k(x+h) - k(x)}{h} \tag{5.33}$$

$$= \lim_{h \to 0} \frac{[\alpha f(x+h) + \beta g(x+h)] - [\alpha f(x) + \beta g(x)]}{h} \tag{5.34}$$

$$= \lim_{h \to 0} \frac{\alpha f(x+h) - \alpha f(x) + \beta g(x+h) - \beta g(x)}{h} \tag{5.35}$$

$$= \lim_{h \to 0} \frac{\alpha \left[f(x+h) - f(x)\right] + \beta \left[g(x+h) - g(x)\right]}{h} \tag{5.36}$$

$$= \lim_{h \to 0} \alpha \left[\frac{f(x+h) - f(x)}{h}\right] + \lim_{h \to 0} \beta \left[\frac{g(x+h) - g(x)}{h}\right] \tag{5.37}$$

$$= \alpha \lim_{h \to 0} \left[\frac{f(x+h) - f(x)}{h}\right] + \beta \lim_{h \to 0} \left[\frac{g(x+h) - g(x)}{h}\right] \tag{5.38}$$

$$= \alpha f'(x) + \beta g'(x) \tag{5.39}$$

The only "trick" to this proof is re-ordering the terms to group together the $f(x)$ terms separately from the $g(x)$ terms, and applying the properties of limits you learned in the previous chapter.

5.6 Continuity is necessary but not sufficient

A derivative is defined at some point $x = a$ only if the function is defined at that point. Consider the following function:

$$f(x) = \frac{|x-1|}{x-1} \tag{5.40}$$

What is the derivative of that function at $x = 1$? Well, the function doesn't exist at that point ($f(1) = 0/0$), so it doesn't make sense to think about its derivative at that point. No function value, no slope, no two-sided limit, no derivative.

Another example is the logarithm: what is the derivative of the logarithm function at $x = -4$? The function is not defined for negative numbers, so it doesn't make sense to talk about a derivative for $x \leq 0$.

Is continuity sufficient for differentiation? You might have guessed that the answer is *no*, considering the bolded question. But the relevant question is *why* continuity is not sufficient for a derivative to exist.

Consider the function in Figure 5.12A. It is continuous everywhere, but how can you draw the tangent line at $x = 0$? The answer is you cannot: there is no unique slope at that cusp point. Instead, an infinite number of straight lines can pass through that point.

A function that is differentiable for all values of x is called a "differentiable function" (alternatively, that the function has "total differentiability"). If there are some points in the function where

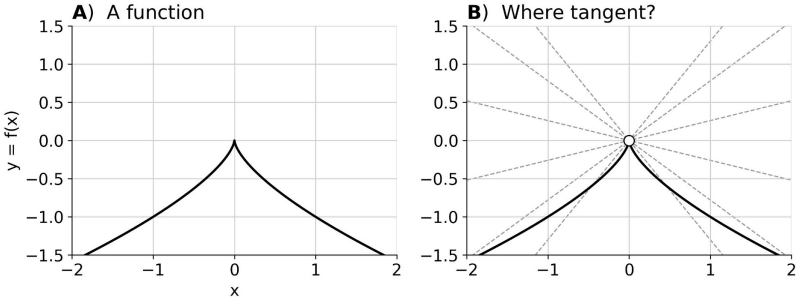


Figure 5.12: A function that is continuous at all points but has no derivative at $x = 0$. Panel B illustrates six of the unlimited number of possible straight lines that pass through the function at that point.

the derivative exists and other points where the derivative does not exist, then it's nondifferentiable.

Many nondifferentiable functions can be differentiable within restricted domains. For example, you could describe a function as being "differentiable everywhere except $x = 0$" or you could write that the function is "differentiable only in the intervals $(-\infty, 0) \cup (0, \infty)$".

5.7 Derivatives of polynomials

Differentiate polynomials with this one simple trick!

I will start this section by showing the rule and then a few examples.

$$f(x) = cx^r \tag{5.41}$$

$$f'(x) = rcx^{(r-1)} \tag{5.42}$$

This is called the "power rule" of differentiation.

where c is any constant and r is any constant in the exponent. In words: to differentiate a polynomial, bring the power down as a

multiplying coefficient and reduce the power by one. Please apply this rule to differentiate the following functions, and then check your work in the footnote.[4]

a) x **b)** x^2 **c)** $5x^5$

d) $4 + 5x + 6x^{20}$ **e)** $\frac{x^2}{2}$ **f)** $x^5 - 3x^2 + .25x^4$

The exception to this rule is x^0: Any number raised to the power of zero is 1, and thus the derivative of x^0 is 0.

Can you differentiate the following function? Try it yourself before reading the next paragraph.

$$f(x) = e^{\pi}x^2 - \pi^e x^4 \tag{5.43}$$

The answer is *yes*, of course you can differentiate that function! The coefficients may look strange, but those are not functions of x; they are just constants. Thus, the derivative of that function is $f' = 2e^{\pi}x - 4\pi^e x^3$

How about the following two functions; can you differentiate them?

$$f(x) = \sin^3(x^2) \tag{5.44}$$

$$g(x) = x^3 e^x \tag{5.45}$$

Here the answer is *no*, you cannot differentiate them. Well, to clarify: Both of these functions are perfectly differentiable, and can be differentiated with little time and effort. But at this point in the book, you haven't learned the rules to differentiate these two functions. Equation 5.44 requires the *chain rule* because of the composite function, and Equation 5.45 requires the *product rule* because two functions of x are multiplied. You will learn both of these rules in the next chapter. Until then, you can only differentiate individual polynomial terms.

[4] **a)** $1x^0 = 1$ **b)** $2x$ **c)** $25x^4$ **d)** $5 + 120x^{19}$ **e)** x **f)** $5x^4 - 6x + x^3$

Alternating power parity An interesting result of the power rule is that the derivative of an even polynomial is an odd polynomial, and the derivative of an odd polynomial is an even polynomial. Observe:

$$f = x^2, \quad f' = 2x^1$$

$$f = x^3, \quad f' = 3x^2$$

$$f = x^4, \quad f' = 4x^3$$

$$f = x^5, \quad f' = 5x^4$$

The power rule describes why this happens algebraically, and Exercise 8 will guide you to exploring why this happens geometrically.

Proving the power rule There are several proofs of the power rule. A few of the proofs work only for positive integer powers, although the power rule works for any number in the exponent (except zero). A general proof that works for all powers — and is compact and elegant — relies on a procedure called *implicit differentiation*. That's a topic for Chapter 7. So for now, please just trust me that the power rule really is correct, and rigorously proving that it is correct is a good motivation to continue working through this book ;)

That said, I would like you to demonstrate the power rule for one specific function, using the limits-based definition of the derivative (Equation 5.4 — that's the formula I told you write down many times in order to memorize).

Use the function $f(x) = 2x^3$, apply Equation 5.4, and work with the function until you can solve the limit problem. My answers are below, but I hope you take this opportunity to work through the problem yourself before checking my work.

$$\frac{dy}{dx} = \lim_{h \to 0} \left[\frac{2(x+h)^3 - 2x^3}{h} \right] \tag{5.46}$$

$$= \lim_{h \to 0} \left[\frac{2(x+h)(x+h)(x+h) - 2x^3}{h} \right] \tag{5.47}$$

$$= \lim_{h \to 0} \left[\frac{2(x^2+h^2+2xh)(x+h) - 2x^3}{h} \right] \tag{5.48}$$

$$= \lim_{h \to 0} \left[\frac{2(x^3+x^2h+h^2x+h^3+2x^2h+2xh^2) - 2x^3}{h} \right] \tag{5.49}$$

$$= \lim_{h \to 0} \left[\frac{2x^3+2x^2h+2xh^2+2h^3+4x^2h+4xh^2 - 2x^3}{h} \right] \tag{5.50}$$

$$= \lim_{h \to 0} \left[\frac{h(2x^2+2xh+2h^2+4x^2+4xh)}{h} \right] \tag{5.51}$$

$$= \lim_{h \to 0} 6x^2 + h(6x+2h) \tag{5.52}$$

$$= 6x^2 \tag{5.53}$$

As I wrote earlier in this chapter, differentiating via Equation 5.4 can be time-consuming and sometimes tricky. But I hope you agree that the insights gained are worth the effort.

5.7.1 Derivative of square root

The square root function $\sqrt{x}$ is not a polynomial, because polynomials are limited to positive-integer powers.

However, the square root function obeys the power rule of differentiation:

$$\sqrt{x} \; = \; x^{1/2} \tag{5.54}$$

$$[\sqrt{x}]' = \frac{1}{2}x^{-1/2} = \frac{1}{2\sqrt{x}} \tag{5.55}$$

Thus, you can differentiate the square root by rewriting it using fractional powers as in Equation 5.54.

5.8 Derivatives of cosine and sine

Cosine and sine have a neat cyclic pattern of derivatives. Behold:

$$\begin{aligned}
[\cos(x)]' &= -\sin(x) \\
[-\sin(x)]' &= -\cos(x) \\
[-\cos(x)]' &= \sin(x) \\
[\sin(x)]' &= \cos(x)
\end{aligned}$$

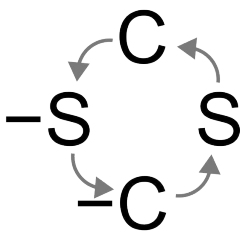


Figure 5.13: The cyclic nature of sine and cosine derivatives.

I previously discussed interpreting a plot of cosine and its derivative in Figure 5.10C-D; those panels are redrawn in Figure 5.14 so you don't have to flip back a few pages. Please take a moment to review why the negative sine is the derivative of cosine: Trace the cosine function with your finger and notice how its behaviors (upwards, downwards, transitions between them) are exactly captured by the minus sine function.

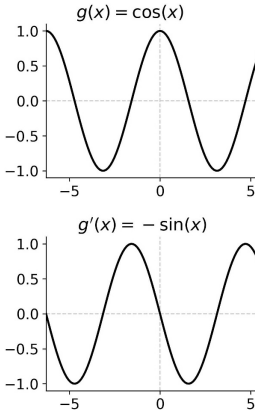


Figure 5.14: Minus the sine is the derivative of cosine. And on and on the cycle goes...

5.8.1 Proving the cosine-sin cycle

In this subsection, I will show the proofs that the derivative of cosine is minus sine, and that the derivative of sine is cosine. If you are new to calculus and mathematical proofs, you may find all the equations intimidating. But it's really not so bad. Just take a deep breath and pay attention to the differences between each equation and the previous.

These proofs rely on two trig identities:

These are called the "angle addition identities."

$$\cos(\theta + \phi) = \cos(\theta)\cos(\phi) - \sin(\theta)\sin(\phi) \tag{5.56}$$

$$\sin(\theta + \phi) = \sin(\theta)\cos(\phi) + \cos(\theta)\sin(\phi) \tag{5.57}$$

$[\cos(x)]' = -\sin(x)$ I suggest to write down each of these steps and try to understand how each line follows from the previous.

$$\frac{dy}{dx} = \lim_{h\to 0}\left[\frac{\cos(x+h) - \cos(x)}{h}\right] \tag{5.58}$$

$$= \lim_{h\to 0}\left[\frac{\cos(x)\cos(h) - \sin(x)\sin(h) - \cos(x)}{h}\right] \tag{5.59}$$

$$= \lim_{h\to 0}\left[\frac{\cos(x)\cos(h) - \cos(x) - \sin(x)\sin(h)}{h}\right] \tag{5.60}$$

$$= \lim_{h\to 0}\left[\frac{\cos(x)(\cos(h) - 1) - \sin(x)\sin(h)}{h}\right] \tag{5.61}$$

$$= \lim_{h\to 0}\left[\frac{\cos(x)(\cos(h) - 1)}{h}\right] - \lim_{h\to 0}\left[\frac{\sin(x)\sin(h)}{h}\right] \tag{5.62}$$

$$= \cos(x)\lim_{h\to 0}\left[\frac{\cos(h) - 1}{h}\right] - \sin(x)\lim_{h\to 0}\left[\frac{\sin(h)}{h}\right] \tag{5.63}$$

$$= \cos(x) \times 0 - \sin(x) \times 1 \tag{5.64}$$

$$= -\sin(x) \tag{5.65}$$

Once you apply the angle addition identity to separate $\cos(x+h)$ into separate terms, the rest of the proof just involves grouping matching terms together and applying the properties of limits, and the trig limits, that you learned in the previous chapter.

Perhaps you were confused with pulling out the $\cos(x)$ and $\sin(x)$ terms in Equation 5.63. These are both functions of x and I explained in the previous chapter that only *constants* can be factored out of a limit. But, here we are taking the limit of h, not

of x. h is related to x in that it is a short-hand for Δx, but from the perspective of h, a function purely of x can be considered a constant. That's why any function of x that does not also include h can be factored out. You will encounter this concept again in multivariable differentiation.

[sin(x)]' = cos(x) The proof that the derivative of sine is cosine follows the same procedure as above: apply the angle addition identity, group terms together, and apply properties of limits and the trig limits from the previous chapter. After working through the previous proof, I'm sure you will agree that this one is easier to follow.

$$\frac{dy}{dx} = \lim_{h\to 0}\left[\frac{\sin(x+h)-\sin(x)}{h}\right] \tag{5.66}$$

$$= \lim_{h\to 0}\left[\frac{\sin(x)\cos(h)+\cos(x)\sin(h)-\sin(x)}{h}\right] \tag{5.67}$$

$$= \lim_{h\to 0}\left[\frac{\sin(x)\cos(h)-\sin(x)+\cos(x)\sin(h)}{h}\right] \tag{5.68}$$

$$= \lim_{h\to 0}\left[\frac{\sin(x)(\cos(h)-1)+\cos(x)\sin(h)}{h}\right] \tag{5.69}$$

$$= \sin(x)\lim_{h\to 0}\left[\frac{\cos(h)-1}{h}\right] + \cos(x)\lim_{h\to 0}\left[\frac{\sin(h)}{h}\right] \tag{5.70}$$

$$= \sin(x)\times 0 + \cos(x)\times 1 \tag{5.71}$$

$$= \cos(x) \tag{5.72}$$

Other trig functions? Cosine and sine are arguably the two most important trig functions, but they're not the only ones. The derivative of the tangent function requires the quotient rule, and so I'll discuss that in the next chapter. Derivatives of other trig functions are used less often, and I won't go through all of them. But I do recommend committing the information presented in Figure 5.13 to memory.

5.9 Derivatives of absolute value and signum

Absolute value ($f(x) = |x|$) and signum ($f(x) = \text{sgn}(x)$; "signum" is the sign function, i.e., $y = -1$ for negative x and $y = 1$ for positive x) are nonlinear functions, and their derivatives are also nonlinear functions.

Before looking at the equations below, I would like you to inspect Figure 5.15 and try to infer the derivatives of these two functions.

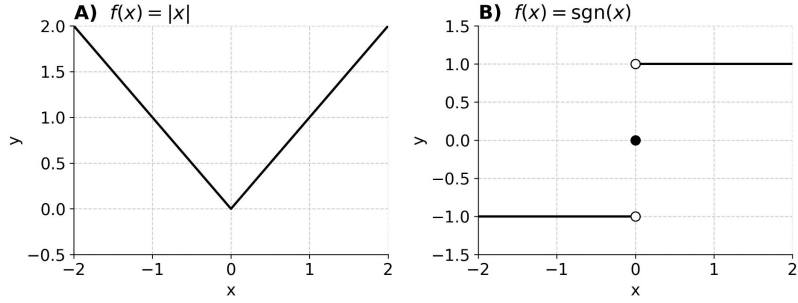


Figure 5.15: Please infer the derivatives of these two functions before reading further.

Let's start with the absolute value function. This function has two pieces: a straight line with a slope of $m = -1$ for any negative x, and a straight line with a slope of $m = +1$ for any positive x. What about when $x = 0$? Zero doesn't have a negative (there is no -0), so $x = 0$ can be included in the positive piece.

The derivative of $f(x) = |x|$ simply involves writing out the piecewise function corresponding to what I wrote above. It turns out that the piecewise definition can be elegantly described by one fraction.

$$|x|' = \left\{ \begin{matrix} -1 & \text{if} & x < 0 \\ 1 & \text{if} & x > 0 \\ \text{d.n.e.} & \text{if} & x = 0 \end{matrix} \right\} = \frac{|x|}{x},\ x \neq 0 \tag{5.73}$$

Now for the signum function (also called the *sign function*). Before discussing its derivative, I would like to write out the formula for this function. This function is -1 for any negative x and $+1$ for any positive x. What is its value when $x = 0$? Zero doesn't have a sign, so the signum function is either undefined (d.n.e.) or given a value of zero for $x = 0$. Taken together, signum can be defined using a three-segment piecewise function.

$$\text{sgn}(x) = \begin{cases} -1 & \text{if} \quad x < 0 \\ 1 & \text{if} \quad x > 0 \\ 0 & \text{if} \quad x = 0 \end{cases} \tag{5.74}$$

You might see the signum function defined more compactly as:

$$\text{sgn}(x) = \frac{|x|}{x}, \; x \neq 0 \tag{5.75}$$

That's pretty neat, because it means that the derivative of the absolute value function is the signum function. Even better: In many applications, a function value *exactly* equal to 0 can be rare due to computer rounding and precision errors. Therefore, in practice, `np.sign(x)` can be used as the derivative of `abs(x)`.

What do you think is the derivative of the signum function? You already know that the derivative of any constant is zero, but this function has three distinct constants in different restricted domains. Don't be confused by that — the derivative is still zero for all values of x except for the one point of transition.

$$[\, \text{sgn}(x) \,]' = \begin{cases} 0 & \text{for} \quad x \neq 0 \\ \text{d.n.e.} & \text{for} \quad x = 0 \end{cases} \tag{5.76}$$

5.10 Derivatives of log and exp

In this section, you will learn the derivatives of the natural log and natural exponential functions. As with a few other derivatives in this chapter, the formal proof of the claims I'll make below rely on a technique called implicit differentiation. So, for now please just trust that these claims are true, and then you'll be able to prove them yourself in the next chapter.

5.10.1 Logs of various bases, and their derivatives

Here is the main take-home message:

$$f(x) = \log_B(x) \tag{5.77}$$

$$f'(x) = \frac{1}{x \ln(B)} \tag{5.78}$$

Recall from Chapter 3 that the natural log and natural exponential are inverses of each other. Therefore, the natural log, defined as $B = e$, simplifies to $f' = 1/x$ because $\ln(e) = 1$.

Logs can be taken to any non-negative base except $B = 1$, but 2, e, and 10 are the most common bases.

I showed the graph of the natural log and its derivative in Figure 5.10 (page 186), and Figure 5.16 below shows the graph again with more log bases. Please take a moment to notice the following features: (**1**) All log bases, and therefore all of their derivatives, have the same basic shape, differing only in the steepness of the increases. (**2**) All log functions increase to ∞, and the bases define how quickly they increase; therefore, all derivatives of log functions are non-negative. (**3**) The log functions are defined only for $x > 0$, and therefore, so are their derivatives. Thus, the derivative of logarithms excludes $x \leq 0$, even though the function $1/x$ is defined for negative x.

If the log function is scaled, that scalar scales the derivative. In

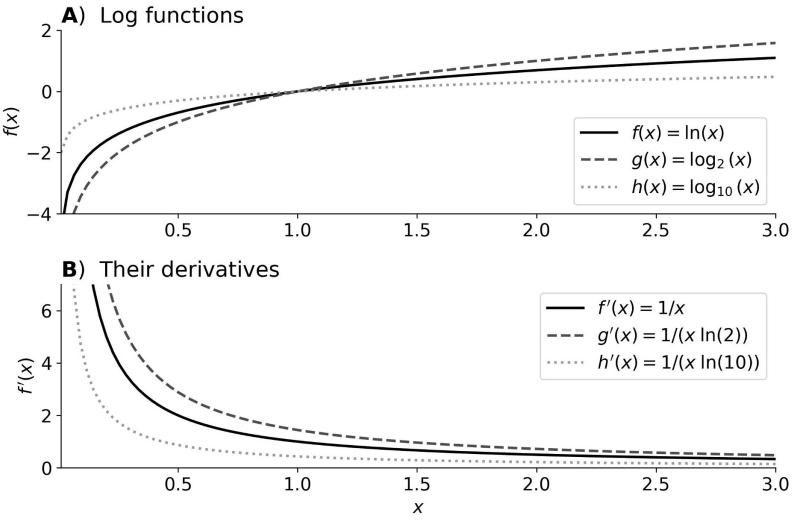


Figure 5.16: Plots of log functions and their derivatives.

particular, for numbers a and b:

$$\Big[\, a \ln(x+b) \Big]' \;=\; \frac{a}{x+b} \tag{5.79}$$

Assuming $x+b > 0$.

5.10.2 Derivative of e^x

The irrational number e is one of the most remarkable and important numbers in all of human-invented mathematics[5], and therefore, the natural exponential function e^x (Figure 5.17) is also one of the most remarkable and important functions.

The remarkable thing about e^x in the context of calculus is that its derivative is also e^x.

$$f(x) = e^x \tag{5.80}$$

$$f'(x) = e^x \tag{5.81}$$

[5]Whether this number is also important in alien mathematics is unknown and can be confirmed only if we meet our mathematically savvy galactic neighbors.

This is the only function that has this property. (The derivative of the constant function $f(x) = 0$ is also zero, but that function can be expressed as $f(x) = 0e^x$.) It means that as x gets larger, e^x increases to ∞. And how fast does that function increase? Well, it increases at the rate of e^x. Not only does it blow up to ∞, the rate at which it increases, increases. At each value of x, the slope of the tangent line going through the function is $m = e^x$.

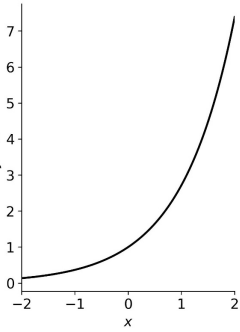


Figure 5.17: e^x

Proving that $[\, e^x \,]' = e^x$, and that that is the only function that equals its own derivative, requires knowledge of integration. We'll get back to this point later in the book!

Constants in the exponents To differentiate $e^{\alpha x}$ for any constant α, bring down that constant as a multiplicative coefficient — but leave it in the exponent. Additive terms in the exponent are ignored. Some examples:

$$\left(e^{2x} \right)' = 2e^{2x} \tag{5.82}$$

$$\left(e^{\pi x+2} \right)' = \pi e^{\pi x+2} \tag{5.83}$$

$$\left(e^{x/2} \right)' = \frac{e^{x/2}}{2} \tag{5.84}$$

For now, commit this rule to memory. When you learn about implicit differentiation in Chapter 7, you will know how to prove this rule.

Why are additive constants ignored? That's because of the law of exponents:

$$e^{2+\pi x} = e^2 \times e^{\pi x} \tag{5.85}$$

$$\left(e^{2+\pi x} \right)' = \pi e^2 e^{\pi x} \tag{5.86}$$

$$= \pi e^{2+\pi x} \tag{5.87}$$

In other words, the additive term in the exponent (in this example, e^2) is actually a multiplicative constant that is placed in the exponent simply because it makes the equation more compact. It's not a function of x.

5.11 Higher-order derivatives

The derivative of a function is itself a function. And that means that you can take the derivative of a derivative. And the derivative of the derivative is also a function, so you can take the derivative of the derivative of the derivative... and so on.

Some functions will eventually yield a constant derivative (e.g., x^n for any non-negative integer n), while other functions are infinitely differentiable (e.g., e^x and $\cos(x)$).

There are no new tricks or techniques to obtain higher-order derivatives. You simply differentiate again. For example:

$$f(x) = \cos(x) + 4x^2 + e^x \tag{5.88}$$

$$f'(x) = -\sin(x) + 8x + e^x \tag{5.89}$$

$$f''(x) = -\cos(x) + 8 + e^x \tag{5.90}$$

$$f^{(3)}(x) = \sin(x) + e^x \tag{5.91}$$

Notation The table below extends the notation table I showed earlier in this chapter; the new column shows how to indicate the second derivative.

Leibniz	$\frac{dy}{dx}$	$\frac{d^2y}{dx^2}$
Lagrange	f'	f''
Newton	$\dot{y}$	$\ddot{y}$
Euler	$\mathbf{D}f$	$\mathbf{D}^2f$

A few notes: (**1**) It may seem strange that the Leibniz notation squares the d and the x but not the y. However, this is short-hand notation. Think about it like this:

$$\left(\frac{d}{dx}\right)^2 y \quad \Rightarrow \quad \frac{d^2}{(dx)^2} y \quad \Rightarrow \quad \frac{d^2y}{dx^2}$$

In other words, (d/dx) is the differentiation operator being applied to y (or f when written df/dx), and that operator is squared.
(**2**) The Lagrange notation has one apostrophe for each derivative, but parenthesized numbers are used after the second derivative. That is, use $f^{(3)}$ instead of f''' and $f^{(10)}$ instead of f''''''''''. Parentheses are used to avoid confusion with powers. Thus: $f^3 = f \times f \times f$ while $f^{(3)}$ is the third derivative of f.

Applications of higher derivatives The first derivative signifies major changes in the function's behavior. And the second derivative signifies changes in the derivative, which means the change in the way that the function is changing.

Higher-order derivatives are used for identifying and interpreting critical and inflection points (Chapter 6), and for differentiation tricks such as L'Hopital's rule (Chapter 7).

Higher-order derivatives have many real-world applications in finance, physics, and engineering. For example, imagine being in a moving car and defining a function of the position of the car. The first derivative is velocity, the second derivative is acceleration, and the third derivative is jerk (in this context, "jerk" is not a rude driver; it is a sudden change in acceleration that causes your head to whiplash). Higher-order derivatives are also used in engineering and materials design, for example computing beam stresses as the fourth derivative, heat transfer, and so on. These calculations help ensure the building you are sitting in doesn't collapse on top of you.

5.12 Exercises

5.1. The purpose of this exercise is to help you understand the link between "local slopes," x-axis grid resolution, and the derivative.

Write `numpy` code to calculate the function $f(x) = x^2$ using $N = 5$ x-axis points linearly spaced between -1 and 5. Connecting these points creates a series of four straight line segments, which you can visualize as in Figure 5.18A. Make sure the segment-calculating code is implemented in a for-loop so that you can change the N parameter.

Next, write code to calculate the slope of each line segment. Visualize the four line segments using horizontal lines, where the x-axis points for each "mini-slope" are defined by the boundary points for each segment, and the y-axis value is the slope of that segment. Visualize this as in Figure 5.18B.

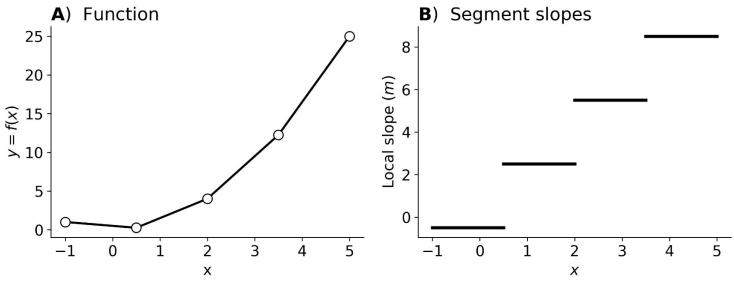


Figure 5.18: Visualization for Exercise 1.

Now that you have the code to create the figure, experiment with the N parameter. **First**, try changing N to larger numbers, e.g., 50 or 500. Pay close attention to the y-axis values in panel B: Does the range of the axis change when you change the x-axis resolution? **Second**, modify the code to remove the denominator from m calculation (that is, calculate the slope of each line segment as Δy without dividing by Δx). Again, vary the N parameter

and observe what happens to the y-axis values in panel B. What conclusions can you draw from these experiments about the importance of scaling the slope by Δx?

Conceptual note: The derivative is what happens when you set the resolution of the x-axis grid to be infinitely fine. That is, as $N \to \infty$, $\Delta x \to dx$ and the y-axis in panel B becomes the derivative.

5.2. `numpy` cannot calculate the analytic derivative of a function, because `numpy` is designed to process numbers, not symbols. However, you can use `numpy` to compute an "empirical derivative," which is a numerical approximation of a derivative. This is also called "numerical differentiation" or "numerical differences." In fact, the previous exercise involved numerically differentiating x^2.

The goal of this exercise is to explore the relation between numerical and analytic differentiation, and to understand how to plot empirical derivatives.

Implement the function $f(x) = x^3 - x^2$ in `numpy` using $N = 10$ x-axis points linearly spaced in the domain $[-1, 1]$. Then, compute two versions of the derivative of this function:

Analytic Write code that directly implements $f'(x)$ (that is, you differentiate the function on paper and then write Python code that implements the derivative function) and evaluates that analytic derivative at the same x-axis points used for the function.

Empirical Calculate $\Delta y / \Delta x$ empirically using the `np.diff` function.

Plot all three functions as in Figure 5.19. You might get an error message when plotting the empirical derivative;

you'll need to figure out why that error happens and what to do about it!

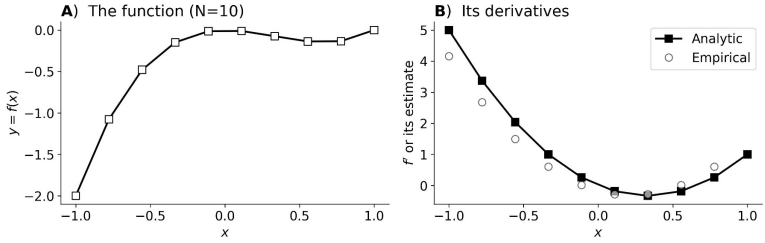


Figure 5.19: Visualization for Exercise 2.

What do you think of the two derivatives? The shapes appear largely similar, but the values don't seem to line up. Before reading the next paragraph, come up with a reason why this might be, and a possible solution.

The reason why the two derivative functions don't line up is the same underlying cause as the error message when plotting: The number of x-axis coordinate values is N whereas the number of points in the empirical derivative is $N-1$. I drew the figure using `x[:-1]`, that is, the first $N-1$ x-axis grid points. Is that the correct way to do it? You could also try `x[1:]`, which skips the first point.

The heart of the issue is this: Given a difference of $y_2 - y_1$, is the corresponding x value x_2 or x_1? The answer is that both are misaligned; they are equally correct and incorrect.

Perhaps you think that the correct solution is to use their average: $(x_1 + x_2)/2$. That average actually does *look* the best on the plot, in that it aligns well with the analytic derivative. But it is also incorrect because it means you are *linearly interpolating* the function; in other words, you are making assumptions about how the function behaves between two evaluated points, although you did not actually evaluate the function at those interpolated points. On the one hand, if the function is smooth and the x-axis resolution is high, then a linear interpolation is likely to be reasonably accurate. Still, the fact is that plotting the

derivative at the boundary averages is based on assumptions. (The reason why averaging the boundaries looks most accurate on this plot is that the black lines for the analytic function are also linear interpolations; thus, the apparent close match simply reflects the fact that these are two linear interpolations of the same function.)

There are alternative formulations for numerical differentiation. Here's one, for example:

$$f' = \frac{f(x+h) - f(x-h)}{2h} \tag{5.92}$$

The idea here is to take the symmetric difference left and right of each center point x_0.

The important point is this: As h gets smaller (finer resolution of the x-axis grid), the various numerical approximation formulations converge to the analytic derivative.

All this discussion aside, you can now confirm that increasing the N value increases the match between the empirical and analytic functions. Spend a minute exploring different values of N on the resulting figure.

5.3. Did you take my advice to explore the qualitative impact of N on the visual match between the empirical and analytic functions? In this exercise, you will explore this relationship quantitatively.

RMS is a common way to measure divergences in data science and engineering.

Calculate the difference between the numerical approximation and the analytic derivatives using the root-mean-square error (RMS):

$$\text{RMS}_N = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(f'_{e_i} - f'_{a_i})^2} \tag{5.93}$$

N is the number of x-axis grid points, f'_{e_i} is the i^{th} value of the numerical difference (the empirical derivative), and f'_{a_i} is the analytic derivative.

The procedure described above provides an RMS for one value of N. Put the code for this procedure inside a for-loop and vary the N parameter from 10 to 300. Visualize the RMS series as a function of the x-axis points as in Figure 5.20.

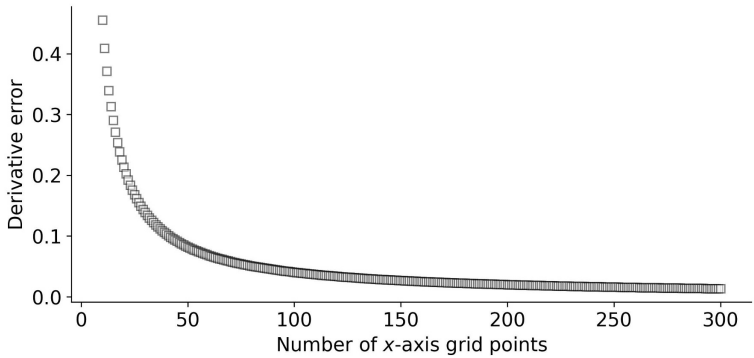


Figure 5.20: Visualization for Exercise 3.

Overall, this plot shows that the numerical approximation of the true derivative is relatively inaccurate when there are few x-axis grid points (which you saw qualitatively in the previous exercise), and becomes increasingly accurate as the number of grid points increases.

The error gets even smaller for larger N. I don't recommend running through this for-loop for really large N simply because of computation time, but you can either modify the code to skip values (e.g., N=10 to 1,000,000 in steps of 10000), or you can copy/paste the inside of the for-loop to evaluate the error for one large value of N. I implemented this latter suggestion in the online code, and found that when $N = 100000$, the RMS error is .00004 (*cf* error of .01 when $N = 300$).

The exact numerical values of this error depend on the behavior of the function within the simulated subdomain: The less the function varies, the smaller the error. You can try this by using different functions that don't change as much as the one in this exercise. Nonetheless, the overall shape of the error function — decreasing with increasing

resolution with the errors tapering down quickly for large resolution — is a general expectation about the match between empirical and analytic functions.

5.4. `numpy` cannot calculate analytic (symbolic) derivatives, but `sympy` is specifically designed for these kinds of mathematical operations. The `sympy` function for differentiation is `sym.diff`. Your goal in this exercise is to figure out the correct way to use this function, and then obtain and print the derivatives of the functions shown in Figure 5.21. Make sure that the fractions 5/4 and 9/4 are represented as symbolic rational numbers, not as the floating-point numbers 1.25 and 2.25.

$x) = \frac{5x}{4} + \frac{9}{4}$

$(x) = \frac{5}{4}$

$x) = x^3 - x^2$

$(x) = 3x^2 - 2x$

$x) = \sqrt{x} + \cos(x)$

$(x) = -\sin(x) + \frac{1}{2\sqrt{x}}$

$x) = e^x$

$(x) = e^x$

Figure 5.21: Python output from Exercise 4.

Finally, use the `.subs()` method to determine both the symbolic and numerical approximations to the derivative of e^x for three values of x, and report the results as below.

$$f'(-1) = e^{-1} \approx 0.367879$$
$$f'(0) = 1 \approx 1$$
$$f'(2) = e^2 \approx 7.38906$$

Given that `sympy` can return the *true* derivative of a function, why would you ever use `numpy` to *estimate* a derivative? There are two situations in which numerical differentiation is necessary: (**1**) Not all functions can be differentiated analytically, and some theoretically differentiable functions cannot be analytically solved by `sympy` due to their complexity. (**2**) Data measured from the real world may not conform to a specific mathematical function, and thus empirical differentiation is the only option. Examples include EKG (heart rate), heat distributions, and Earthquake spread across a landscape.

5.5. The purpose of this exercise is to confirm the linearity of

differentiation in `sympy` and to improve your general `sympy` skills. Here's what you should do: Create symbolic constants α and β, and differentiate the terms combined and separately as shown in the `sympy` output below.

$$\frac{d}{dx}\left[\,\alpha x^2 + \beta\cos(x)\,\right] \;=\; 2\alpha x - \beta\sin(x)$$

$$\alpha\frac{d}{dx}(x^2) + \beta\frac{d}{dx}(\cos(x)) \;=\; 2\alpha x - \beta\sin(x)$$

Figure 5.22: Screenshot of `sympy` output for Exercise 5.

The point is to see that the result is the same regardless of how the differentiation was set up.

The next part of this exercise is more about improving your `sympy` coding skills than reinforcing calculus concepts. Create a symbolic expression corresponding to the function:

$$f(x) = \alpha x^2 + \beta x^3 + \gamma e^{2x} \tag{5.94}$$

Use `sym.diff()` to calculate the derivative of the entire function and print that out. Then, figure out how to separate that expression into separate `sympy` variables corresponding to each term of that function (e.g., one variable for αx^2).

In a for-loop over each term, differentiate each term separately and print the result. Inside the for-loop, sum each piece into one expression and print out the final summed result. Your Python output should match my results in the screenshot below. If you need a hint for separating the parts of the function, see the footnote[6].

Derivative of the entire function: $2\alpha x + 3\beta x^2 + 2\gamma e^{2x}$

Derivative of αx^2 is $2\alpha x$
Derivative of βx^3 is $3\beta x^2$
Derivative of γe^{2x} is $2\gamma e^{2x}$

Combination of individual components: $2\alpha x + 3\beta x^2 + 2\gamma e^{2x}$

[6] `sympy` calls the terms of an expression "arguments," abbreviated as `args`.

Figure 5.23: Screenshot of `sympy` output for the rest of Exercise 5.

5.6. In this exercise, I will describe an algorithm to compute tangent and secant lines. Your goal will be to translate that description into Python code. You can apply this algorithm to any function and x-axis values you want, but I recommend choosing values that will match Figure 5.7.

1. **Define the function and its derivative**. These need to be regular Python functions or `lambda` expressions.
2. **Define x-axis intersection points.** Pick three values: one for the tangent line (x_t) and two for the secant line (x_{s_1} and x_{s_2}).
3. **Calculate the corresponding function values.** The result of the previous and this steps will be three (x, y) pairs (e.g., $(x_t, f(x_t))$).
4. **Calculate two slopes.** The slope of the secant line is $m_s = \Delta y/\Delta x$ for the two function intersection points, and the slope of the tangent line is $m_t = f'(x_t)$.
5. **Write a function to calculate the line values.** These should be Python functions or `lambda` expressions. The functions should return $y_s = m_s(x - x_{s_1}) + y_{s_1}$ for the secant line, and $y_t = m_t(x - x_t) + y_t$ for the tangent line.
6. **Plot the results!** Then print out the figure and send it to your Grandmother so she can be proud of you :D

$) = x^{10},\ f'(x) = 10x^9$
$) = 10x^9,\ f'(x) = 90x^8$
$) = 90x^8,\ f'(x) = 720x^7$
$) = 720x^7,\ f'(x) = 5040x^6$
$) = 5040x^6,\ f'(x) = 30240x^5$
$) = 30240x^5,\ f'(x) = 151200x^4$
$) = 151200x^4,\ f'(x) = 604800x^3$
$) = 604800x^3,\ f'(x) = 1814400x^2$
$) = 1814400x^2,\ f'(x) = 3628800x$
$) = 3628800x,\ f'(x) = 3628800$

Figure 5.24: Screenshot of Python output from Exercise 7.

5.7. Start with $f(x) = x^{10}$ and successively differentiate the function and set the function to be its derivative. Use `sympy`. Print the results as in Figure 5.24.

5.8. This exercise is focused on approximating derivatives us-

ing numerical differences in `numpy`. Start from the function $f(x) = x^4$, calculate the empirical derivative using `np.diff`, then the derivative of that (thus using `np.diff` again) and again until you reach the third derivative. Show the original function and three numerical derivatives as in Figure 5.25. If your result doesn't match mine, then make sure you're normalizing by Δx correctly.

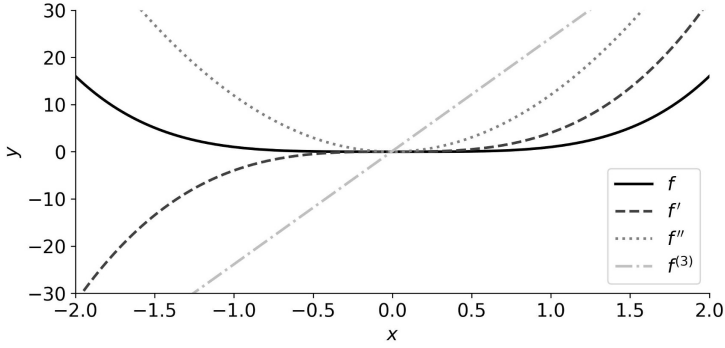


Figure 5.25: Visualization for Exercise 8.

The `np.diff` function takes an optional second input, which is the number of times to differentiate. So, `np.diff(fx,1)` is the first-order derivative (same as the default setting) and `np.diff(fx,2)` is the second-order derivative. Copy the code you wrote above into a new code block, and adjust the differentiation lines so that you compute all derivatives from the original function, not from the successive derivatives. Generate the plot again and make sure it matches perfectly. If it doesn't, think about the Δx scaling!

5.9. The goal of this exercise is to reproduce Figure 5.26. This will reinforce your understanding of trig derivatives, and will help improve your `matplotlib` skills.

The ϕ-axis grid spacing is defined as $\Delta\phi = .01$ in the interval $[-1.5\pi, 1.5\pi]$. The gray line is the derivative of cosine (approximated numerically in `numpy`) whereas the light gray circles are the minus sine function (down-sampled for visibility).

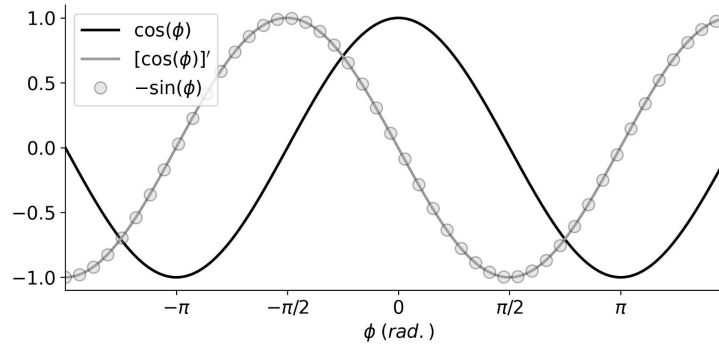


Figure 5.26: Visualization for Exercise 9.

5.10. This one's easy: reproduce the left-hand side of Figure 5.13 (page 195) in `sympy`: Differentiate $\cos(x)$ to get $-\sin(x)$, then differentiate that, and again and again until you get back to $\cos(x)$.

You can copy and adjust the code from Exercise 7, although I recommend challenging yourself and writing the code from scratch.

5.11. This exercise is just a bit of fun with plotting in `matplotlib`. Implement the following two equations in `numpy`:

$$f(x) = a \ln |x| \tag{5.95}$$

$$g(x) = \ln |x + b| \tag{5.96}$$

Then in a for-loop, vary a and b in 31 steps between -1 and 3, and draw each line as you can see in panels A and B of Figure 5.27. Finally, draw $f(x)$ on the x-axis and $g(x)$ on the y-axis as in panel C.

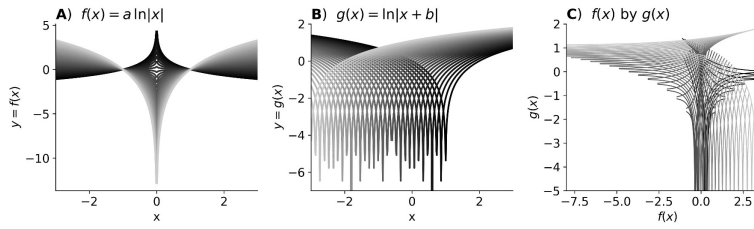


Figure 5.27: Visualization for Exercise 11.

I hope you enjoyed this exercise! Feel free to make other changes to the function definition, parameter ranges, colors, etc.

Chapter 6

Critical and inflection points

Critical points are special points on a function, where the function changes in some way, or where the function is degenerate at a particular point. Critical points are used in many calculus applications, with optimization being one of the most important examples.

The organization of this chapter is as follows: First I will explain the concepts and types of critical points; then I'll show you how to identify them. Thereafter I'll explain the ways to evaluate critical points to determine exactly what they reveal about the function. Finally, I'll walk through some example problems and applications.

But first, I would like you to inspect Figure 6.1. Based on the figure, try to guess what critical points are and how you might identify them.

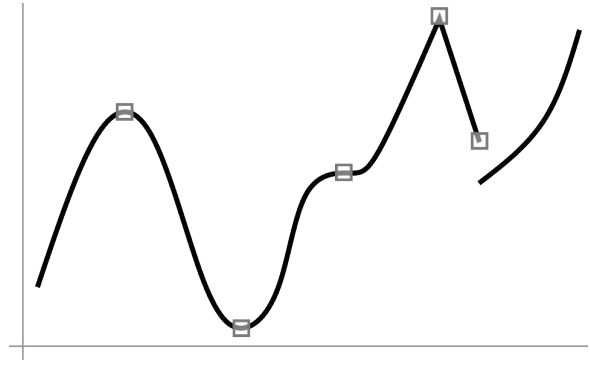


Figure 6.1: Examples of critical points (squares) on a function (black line).

What have you noticed about critical points? They indicate "important" events in the function, such as changes in the direction of the function or sudden jumps between values. Here is the definition of a critical point:

> A *critical point* is a point in a function where the derivative equals zero or is undefined, and where the function value is defined.

The second part of the definition ($f(x)$ is defined) is important: We do not consider a point to be a "critical point" if the function

is undefined, for example at an infinite discontinuity or outside the domain of the function (such as $x < 0$ in $\sqrt{x}$).

Also note the distinction between:

Critical point: The x-axis coordinate.

Critical value: The function value at the critical point.

6.1 Taxonomy of critical points

Figure 6.2 expands on the previous figure to label each critical point according to its "type," meaning what each critical point indicates about the change in the behavior of the function.

The definitions below provide general descriptions of each type of critical point; in the rest of this chapter, you will learn more subtleties and how to identify each of these types. The take-home message is that a critical point can indicate one of several distinct behaviors of a function. That is, simply knowing that "$x = 4$ is a critical point" is not very indicative; you would know that *something* happens at $x = 4$ but you wouldn't know *what* happens. Accurate interpretation of critical points requires deeper investigation and ideally also visualization.

In the context of derivatives, "critical point" can be abbreviated "c.p."

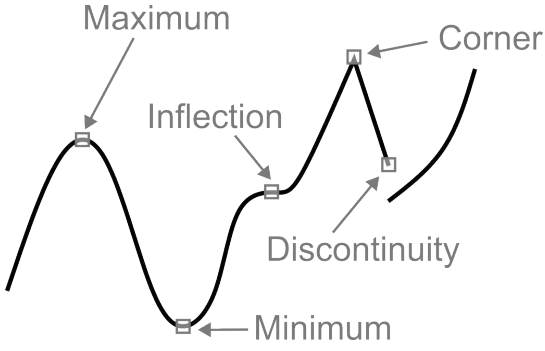


Figure 6.2: Types of critical points.

Maximum

The function increases to the left, and decreases to the right, of

the critical point. Alternatively and equivalently: The derivative is positive prior to this point, and negative after this point. Therefore, this critical point indicates a local maximum of the function.

Be mindful that critical points do not indicate whether the maximum is a *local* or a *global* maximum. In the example shown in Figure 6.2, the critical point labeled "Maximum" is a local maximum.

Minimum

The function decreases to the left (negative derivative), and increases to the right (positive derivative). In between these two behaviors, there is at least one value of x where the function is flat, and hence, where $f'(x) = 0$. So, this critical point indicates a minimum of the function.

As with the maximum critical point, we would not know whether the minimum is a local or global minimum solely from knowing this critical point; visualization or additional tests are required. The labeled minimum in Figure 6.2 is a global minimum in the interval shown.

Inflection

An inflection point is where a function continues to increase (or decrease), but the rate at which it increases changes. The example inflection point in Figure 6.2 shows a momentary flattening of the function in between two regions of increasing values.

Important: Formally, an inflection point is not a type of critical point. As I will explain later, "critical points" are based on the first derivative whereas "inflection points" are based on the second derivative. But these concepts are so closely related that it is sensible to include them in the same discussions.

Discontinuity

Critical points occur at any function discontinuity where the function is defined at that point (remember that the derivative is undefined at jump discontinuities). If the function were undefined at this transition value, it would not be considered a critical point.

Corner or cusp

A corner is similar to a maximum or minimum, but the derivative is undefined at this point, because the tangent line at this point has no unique slope (that is, there is more than one line that could touch the function at that point). The difference between a corner and a cusp is that a corner has straight lines approaching the critical point (illustrated in Figure 6.2) while a cusp has curved lines (as was illustrated in Figure 5.12).

Here's the important thing: When you identify a critical point algebraically (that is, finding the values of x that satisfy $f'(x) = 0$ or where $f'(x)$ is undefined), you don't know what *type* of critical point it is. As you'll learn in the next section, the procedure is to find the critical points first, then evaluate them to determine what they indicate about the function.

Now that you are familiar with critical points, you must be wondering how to find such fantastic beasts? Keep reading!

6.2 Analytic methods to find critical points

Broadly speaking, there are two approaches to identify critical points: analytic methods (differentiating functions and solving equations) and empirical methods (also called numerical methods; this involves simulating equations or gathering data, and finding approximate solutions).

Analytic methods are used when the function is mathematically defined and differentiable, and empirical methods are used when the function is either not mathematically defined (e.g., it comes from data) or its derivative is difficult or intractable to solve (this is often the case in multivariable functions, and is the basis of the famous *gradient descent* algorithm that powers deep learning).

In the rest of this section, I will detail three analytic methods to identify critical points. These are not *distinct* methods and you

get to choose which to apply; different types of critical points are identified in different ways and so you need to use all of these methods to find all of the critical points.

6.2.1 Via solving $f'(x) = 0$

Critical points like minima and maxima are defined as a transition between the function going up and the function going down (or down to up). This means that there is a point in between where the function has a horizontal tangent, thus a point where $f'(x) = 0$. And therefore, you can find critical points by setting the derivative to zero and solving for x. In other words, the solutions to $f'(x) = 0$ are critical points.

One caveat: Constant functions like $f(x) = \pi$ have $f' = 0$ for all x, although those points do not signify a change in the function's behavior. Thus, finding the values of x that satisfy $f'(x) = 0$ identifies *putative* critical points, and you will need to evaluate each one to confirm their nature and interpretation.

Let's go through an example. Find the critical points of the function $f(x) = x^3 - x^2 + 3$. My solutions are listed below, but I encourage you to solve this yourself first.

$$f(x) = x^3 - x^2 + 3 \tag{6.1}$$

$$f'(x) = 3x^2 - 2x \tag{6.2}$$

$$0 = 3x^2 - 2x \tag{6.3}$$

$$0 = (3x - 2)x \tag{6.4}$$

$$x = \{0, 2/3\} \tag{6.5}$$

The critical points of $f(x)$ are $x = 0$ and $x = 2/3$, and the corresponding critical values are $y = 3$ and $y = 77/27$. What do these point/value pairs mean? Let's inspect the graph to find out (Figure 6.3).

Notice that the function *changes behavior* at these critical points — switching from bending upwards to bending downwards. There are no other critical points in this function because its behavior never changes for $x < 0$ or $x > 2/3$. The *values* of the function change, but the *behaviors* of the function do not.

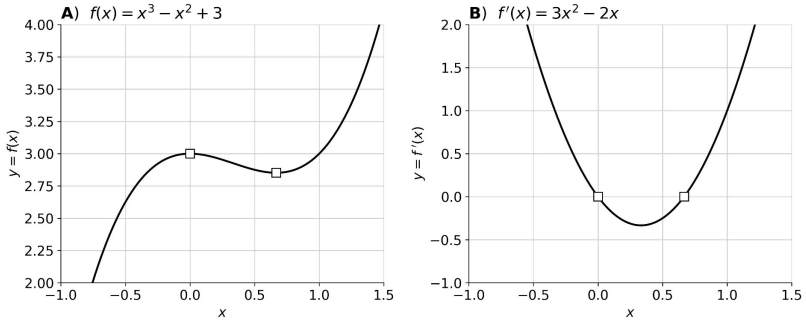


Figure 6.3: A function (panel A) and its derivative (panel B), with critical points/values indicated with squares. Notice that critical points correspond to $f'(x) = 0$.

6.2.2 Via discontinuities

Discontinuities in the function are included in critical points, as long as the function is defined at those points. This definition excludes infinite and removable discontinuities but includes jump discontinuities and corners.

"Stationary points" are critical points where the derivative is defined. Therefore, discontinuities in the derivative where the function is defined are critical points but not stationary points.

An example is the absolute value function: $f(x) = |x|$. This function is defined for all x (including $x = 0$). Its derivative is the signum function and is undefined at $x = 0$. Therefore, $x = 0$ is a critical point of the absolute value function, although it would not be identified by solving $f'(x) = 0$.

As a counter-example, consider the following function and its derivative, shown in Figure 6.4 (differentiating this function relies on the quotient rule, which you'll learn in the next chapter).

$$f(x) = \frac{x^2 - x - 2}{x - 2} \tag{6.6}$$

$$f'(x) = \frac{x^2 - 4x + 4}{(x-2)^2} \tag{6.7}$$

You don't need any fancy algebra to see that the derivative is undefined at $x = 2$. However, the function itself is also undefined at $f(2)$, therefore, $x = 2$ is not a critical point.

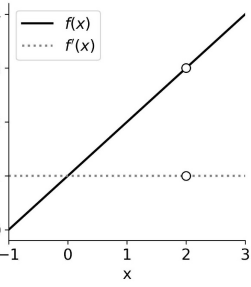


Figure 6.4: A function and its derivative. White circles indicate discontinuities with plotted values coming from the limits.

Visual inspection of the function will also confirm that $x = 2$ is not a critical point: There are no observable changes in the function's behavior at $x = 2$. (Don't over-generalize this point, though: the function $\sin(x)/x$ and its derivative are undefined at $x = 0$ but that *does* correspond to a change in function behavior.)

6.2.3 Endpoints of a restricted domain

If a function has a restricted domain, should the endpoints of the domain be considered critical points?

Consider, for example, the function $f(x) = x^2$, $\{x \in \mathbb{R} \mid 1 \leq x \leq 3\}$ (Figure 6.5). How many critical points are in this function in this subdomain?

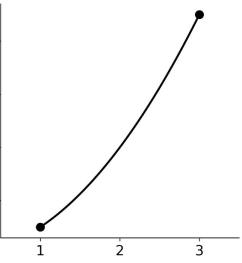


Figure 6.5: The function x^2 defined on the closed interval between 1 and 3.

According to the typical definition of critical points, there are none: the derivative is $f' = 2x$ and never equals zero in the interval $[1, 3]$. Furthermore, the derivative is technically undefined at the boundary points, because there is no limit left of $x = 1$ or right of $x = 3$.

On the other hand, the values $x = 1$ and $x = 3$ are clearly *important* for the behavior of the function, because that's where the function starts and stops. Different mathematicians, and different textbooks, have differing opinions about this issue. In my opinion, this is just a matter of semantics and terminology. However, if you will take a calculus exam then you do need to know whether your instructor considers endpoints to be critical points.

The conclusion here is that endpoints of a restricted domain are clearly significant points that provide information about the na-

ture of the function. Whether they are put into the bucket labeled "critical points" is a matter of personal preference.

6.2.4 Conclusions about finding critical points

Here's the procedure so far:

1. Compute the derivative.
2. Set the derivative to zero and solve for x.
3. Identify discontinuities where the function is defined.

   The set comprising unique values from steps 2 and 3 are the critical points (possibly including endpoints if the function is defined on a restricted interval).
4. Determine the type of each critical point by inspecting a graph of the function and/or applying critical points evaluations.

I haven't yet introduced you to formal methods for evaluating and classifying critical points (step 4 above). First I want to explain how to find critical points using numerical methods.

6.3 Empirical methods to find critical points

Empirical methods of finding critical points are conceptually comparable to the procedure described in the previous section, except that they rely on numerical differences instead of analytic differentiation.

Why would anyone want to approximate critical points empirically instead of getting exact answers using calculus? The answer is no one *wants* to find critical points empirically; they *need* to discover them empirically, because the function comes from empirical

data and thus there is no analytic derivative, because the derivative cannot be analytically calculated, or because the analytic derivative cannot be exactly solved for $f(x) = 0$. Non-solvable derivatives often occur in applications, where the functions may contain hundreds, thousands, or millions of variables, and computing all the derivatives and partial derivatives is either computationally infeasible or completely beyond the scope of human technology[1].

There are a few popular modern methods for finding critical points via numerical simulations. In this section I will introduce two: grid search and gradient descent[2]. But first, I want to discuss the easiest and most ancient method of finding critical points: Using your eyeballs.

6.3.1 Via visual inspection

Geometrically, you can find critical points by inspecting the graph of a function. Visually identifying critical points helps you gain immediate insights into the behavior of the function.

To identify and characterize critical points, you should look at both the function and its derivative. In the previous chapter, I stressed that you look for different features in the graph of the function vs. the graph of the derivative. Here's what to look for when identifying critical points:

What to look for in the function

Maxima and minima are easy to spot — the function changes from going up to down, or from down to up. Discontinuities are sometimes easy to spot, but be mindful that spurious jump-connecting lines can appear in the plots (as you learned in Chapter 3), and that critical points require the function to be defined at that point even if the derivative is undefined.

[1] For example, ChatGPT3.5 is a function of 175 *billion* variables. Functions with a mere 10 variables can be analytically computationally intractable.

[2] In Chapter 9 you will learn another method of finding roots that was developed by Newton.

What to look for in the derivative

The most important feature to look for is where the derivative crosses the $y = 0$ line. That's an important difference from how you look at a function. It may be helpful to draw a thick horizontal line at $y = 0$ to draw your attention to solutions to $f' = 0$.

Also look for discontinuities in the derivative — but only if those points of discontinuity are defined in the original function.

Figure 6.6 shows an example of a function and its derivative. To be sure, some of the critical points can be identified, but others are more ambiguous or difficult to be precise about.

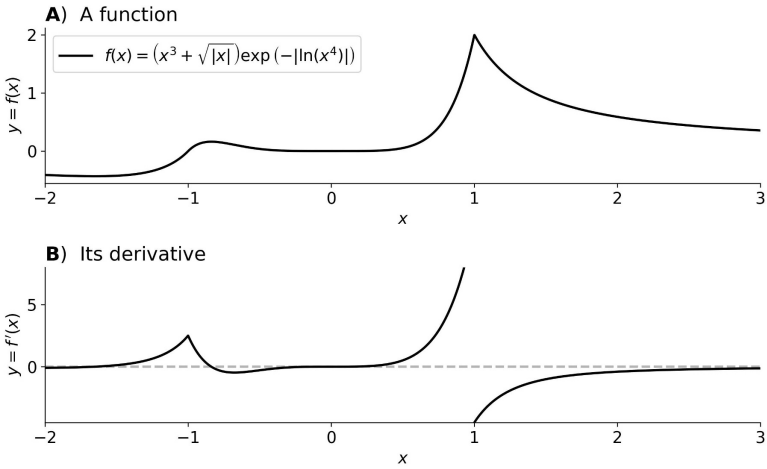


Figure 6.6: A function and its derivative. Can you identify the critical points by visual inspection?

Indeed, as you've been reading this subsection, I hope you've been thinking that visual inspection is not very precise, and may miss critical points that are subtle or are excluded from the plot view. Furthermore, modern applications of calculus involve finding critical points of multivariable functions, which are not visualizable for more than two variables.

This is very true: finding critical points by visual inspection is neither precise nor scalable. But it is, in my humble opinion, an excellent way to build intuition for what critical points look like,

what they mean, and why they are used in applications.

6.3.2 Grid search

The idea of the grid search method is to evaluate the function and its derivative on a *grid* (that is, a set of x-axis points; this is how we've been discretizing functions when using `numpy`), and finding values that equal, or are very close to, zero.

To demonstrate the grid search approach, let's find the critical points of the function $f(x) = x^3 + x^2$. Of course, the critical points can be calculated analytically, so exploring a numerical method will allow us to assess the accuracy of this method.

Figure 6.7 shows the goal: Identify the critical points based on where the numerical difference equals zero.

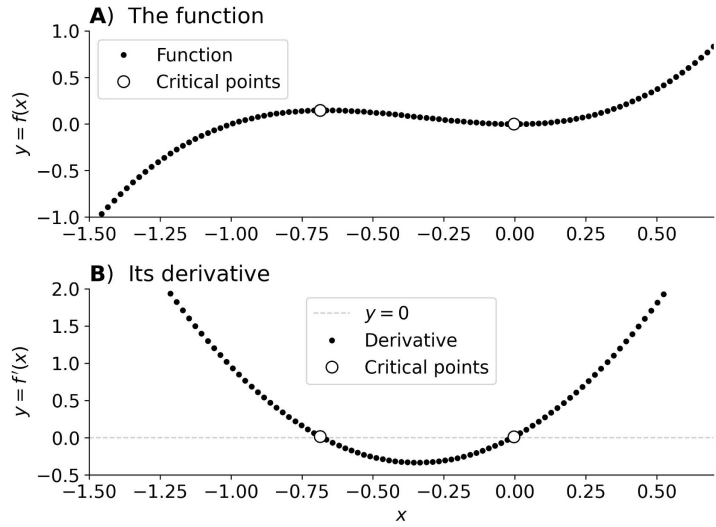


Figure 6.7: Finding critical points numerically.

We start by implementing the function in `numpy`, as shown in the code below. Variable `x` is our "grid" and the difference between successive grid points — which I have previously been calling Δx — is the grid resolution. This empirical approach requires us to compute the numerical difference vector, which, as you know from previous chapters, can be a good estimate of the true derivative

if the x-axis resolution is high compared to the changes in the function.

```
x = np.linspace(-1.5,.7,101) # x-axis grid
fx = x**2 + x**3 # function ("data")
df = np.diff(fx) / (x[1]-x[0]) # numerical difference
```

Now our task should be simple, right? Just search for the points where `df` equals zero. Let's see what happens in the code:

```
exactZero = np.where( df==0 )[0]
```

The variable `exactZero` is empty. There are no zeros in variable `df`. That is obviously inconsistent with Figure 6.7B. The issue here is that because of the finite x-axis grid resolution as well as rounding and precision errors, there are values in `df` that are close to — but not exactly equal to — zero.

This means that we need to adjust our search from finding *exact zero* to *closest to zero*. A way to accomplish this is by searching for the local minima in the absolute value of the derivative estimate (Figure 6.8).

```
from scipy.signal import find_peaks
dfLocalMin = find_peaks(-np.abs(df))[0]

# print the results
for xx,yy in zip(x[dfLocalMin],fx[dfLocalMin]):
  print(f'Critical point/value: ({xx:.3f}, {yy:.3f})')

> Critical point/value: (-0.686, 0.148)
> Critical point/value: (-0.004, 0.000)
```

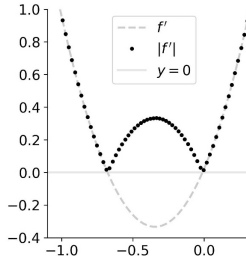


Figure 6.8: Finding points closest to zero.

The resulting empirical critical points are close to the true results that can be calculated analytically: $(-2/3, 4/27)$ and $(0, 0)$.

Zero-crossings vs. minima One potential problem with the method described above is that it cannot distinguish between a root (that is, where the derivative passes through $f' = 0$) and a local minimum. Consider Figure 6.9: The derivative has a local minimum at $x = 0$, which is not a root of the derivative but which is identified by the `find_peaks` function.

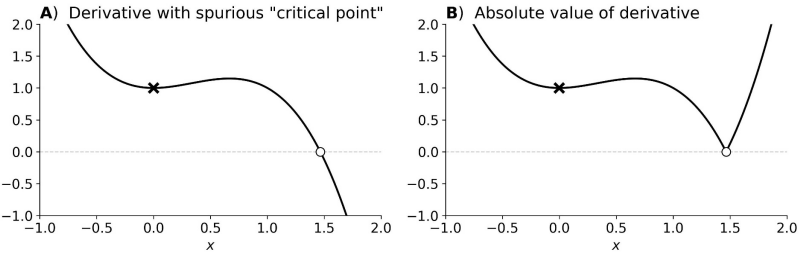


Figure 6.9: Local minima must be dissociated from zero-crossings.

Therefore, it is necessary to use `find_peaks` to identify *putative* critical points, and then confirm or reject each point depending on whether the derivative changes sign before vs. after each minimum. This can be achieved using the code below:

```
critMinima = find_peaks(-np.abs(dy))[0]
critPnts = []
for i in critMinima:
  if np.sign(dy[i-1]) + np.sign(dy[i+1]) == 0:
    critPnts.append(i)
```

The logic of adding the signs is that if the signs before vs. after the minimum are the same, then it indicates a local minimum not a zero-crossing, and the sum of the signs will be ± 2. On the other hand, if the signs before vs. after the local minimum differ, then it reflects a true zero-crossing and their signs will cancel[3].

[3] It is also possible to identify critical points as zero-crossings in the sign of the derivative, although this method can be less accurate due to the binarization of the derivative to its signs. I discuss this, with an example, in the online code

Accuracy and resolution Needless to say, the higher the resolution, the more accurate the results will be. Indeed, as the resolution of the grid approaches $\Delta x \to 0$, then we're back to the analytic derivative.

In small example problems with single-variable functions, it's easy to have a super-high resolution. But in practice, you might have a complicated function of multiple variables, and a high-resolution grid might require trillions of calculations perhaps costing weeks of computation time. There are strategies to deal with situations like this, for example starting with a coarse grid and then refining the search, having randomly placed grid points, parallelizing the calculations, and having multiple distinct starting locations. Such implementational optimization strategies are beyond the scope of this text, but you now understand the idea and basic implementation of a grid search to find critical points without actually differentiating functions.

General importance of "grids" in empirical math The grid search approach that I presented here is used in myriad applications in math, engineering, and physics. In this book, for example, you'll encounter grid-search methods to solve implicit functions and multivariable functions.

Preprocessing data Note that if you are numerically differentiating empirical data, the data might contain noise or other irrelevant fluctuations. It may be necessary to smooth or filter the data before identifying critical points.

6.3.3 Gradient descent

You will learn in Chapter 10 that a "gradient" is a collection of partial derivatives of a multivariable function. Thus, if the function has only one variable, "gradient" is the same thing as derivative. Therefore, you can think of this method as "derivative decent."

The goal of gradient descent is to find the minimum of a function by stepping towards smaller values of the function. It's used on functions that have an analytically solvable derivative, but that derivative function is so complicated or multivariable as to be computationally intractable to solve $f' = 0$ exactly. I'll explain this point in more detail later, but first I will explain the algorithm.

Here's the idea: Start at a random point on a function with a known (analytically expressable) derivative. Let's call that random start location x_0. Then, evaluate the derivative to obtain $f'(x_0)$. Finally, update x_0 by subtracting the derivative at that point.

$$x_0 \leftarrow x_0 - \eta f'(x_0) \tag{6.8}$$

η is called the "learning rate" and is some small number, for example .01. Scaling down the update helps prevent overshooting the minimum.

The updated x_0 will lead to a slightly smaller function value. One single update is unlikely to be sufficient, and therefore Equation 6.8 is repeated until some criterion is reached — either after a specified number of times or when the derivative is below some threshold value.

Figure 6.10 shows an example: The goal is to find the minimum of the function by moving towards $f' = 0$. By random chance, the first guess at the "minimum" was around $x_0 = -.6$. The derivative was negative, so the subsequent guess increased the value of x_0. I used 100 iterations to get to the final estimate; in the exercises you will modify the code to illustrate the progression of learning.

From this figure, you can see why the derivative is *subtracted*: If the current estimate is left of the minimum, then the derivative will be negative and the next guess should be larger. On the other hand, if the estimate is right of the minimum, then the

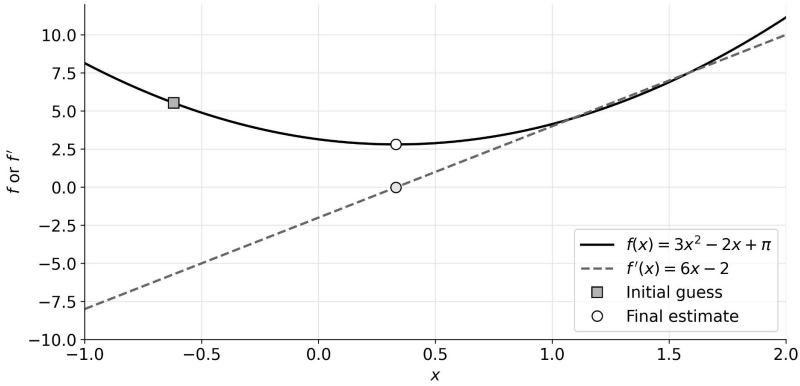


Figure 6.10: A simple example of using gradient descent to find the minimum of a function.

derivative will be positive and the next guess should be smaller. In other words, updating the guess according to the negative of the derivative ensures that each step moves towards the nearest x for which $f(x)' = 0$.

The empirical critical point obtained via gradient descent is rarely *exactly* correct, although it can be quite close, given certain assumptions. Let's see how it looks in this example:

```
Initial guess:  (x,y) = ( -0.619,5.529 )
Final estimate: (x,y) = (  0.331,2.808 )
True minimum:   (x,y) = (  0.333,2.808 )
```

The exact minimum of this function is $(x, y) = (1/3, \pi - 1/3)$, which is very close to the final estimate of gradient descent.

In other words, gradient descent found a critical point (the function minimum) by applying an iterative algorithm that involves moving closer and closer to the minimum by going in the opposite direction as the derivative.

And that's how gradient descent works. In real-world applications in deep learning models such as ChatGPT, there are additional tricks to ensure optimal performance of the algorithm, including

allowing η to vary according to learning performance. But the fundamental mechanism is the same as what you just learned.

Gradient descent is typically used only to find minima of functions, not all critical points. However, it is straightforward to modify the objective of gradient descent to find other critical points.

Why use gradient descent? Why would someone estimate a function minimum using an imprecise and iterative algorithm, when they could simply differentiate the function and immediately find the exact minimum? I wrote the answer to this question earlier in this chapter, but it is asked so often by so many people that it's worth reiterating.

Here's the answer: If you have a simple function with one or a few variables, then it is almost certainly better to solve the problem analytically. But real-world applications have dozens, thousands, millions, or possibly billions of variables. It is somewhere between grossly inefficient and well beyond the sum total of humanity's computational power to solve these problems analytically, and gradient descent — while it may be imperfect — is effective and efficient, and, with some clever tricks, extremely accurate.

6.4 The "first derivative test"

Regardless of how you find the critical points, you cannot know what they indicate about a function's change in behavior until you investigate each critical point.

In some cases, that investigation is simple. For example, if you know that the function has one squared term with a positive coefficient (that is, ax^2 with $a > 0$), then you know for sure that there will be one critical point corresponding to the function minimum. But many other functions will have multiple critical points that

must be investigated.

Visualize when possible I'm sure you're not surprised to read this: Whenever possible, visualize the function and its derivative, with critical points indicated, to understand each critical point.

6.4.1 First derivative test

The "first derivative test" is the formalization of what is obvious from looking at the graph of a function: If the function goes up before a critical point and then down after it, then that critical point is a local maximum; and if the function goes down before a critical point and then up after it, then that critical point is a local minimum. The first derivative test simply involves checking the sign of the derivative around each critical point.

In this context, "before" means to the left while "after" means to the right.

In particular, for critical point c and function $f(x)$:

1. If $f'(x < c) < 0$ and $f'(x > c) > 0$ then $f(c)$ is a minimum.
2. If $f'(x < c) > 0$ and $f'(x > c) < 0$ then $f(c)$ is a maximum.
3. If $f'(x)$ has the same sign both left and right of c, then $f(c)$ is not an extreme.

In this context, $x < c$ means values of x just a bit left of c; not necessarily all the way down to $-\infty$. Same story for $x > c$.

A note about point 3 above: The first derivative test can be uninformative when $f(x)$ is a constant around c, has a jump discontinuity at c, or has a perfectly vertical slope at c.

Figure 6.11 shows an example to illustrate this test. Notice the sign of the derivative around the critical point $c = 1$. (Differentiating this function relies on the "chain rule," which you'll learn in the next chapter.) This example also shows that the 1^{st} derivative test concerns the derivative sign *around* the critical point but not *exactly* at the critical point; indeed, $f'(c)$ is undefined.

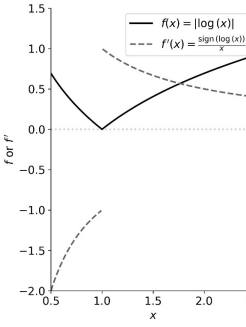


Figure 6.11: Example of the 1st derivative test.

6.5 Examples of critical points

I'm going to walk you through five examples of finding critical points in functions.

Example 1 The first example is the function $f(x) = 3x + 6$. Its graph, and that of its derivative, are shown in Figure 6.12. The equations below show the function, its derivative, and the critical points revealed via the solutions to $f' = 0$.

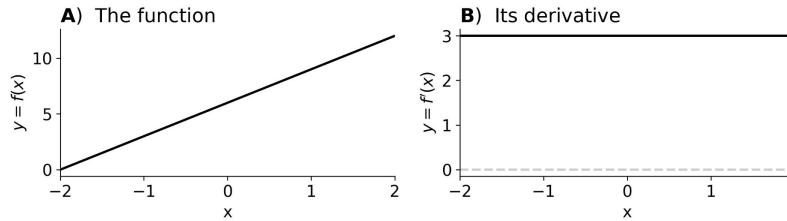


Figure 6.12: Finding and interpreting critical points: example 1.

$$f(x) = 3x + 6 \tag{6.9}$$

$$f'(x) = 3 \tag{6.10}$$

$$0 = 3 \tag{6.11}$$

Setting the derivative to zero and solving for x failed: we ended up with an inconsistent equality. This means that there are no critical points in this function (in principle, we should also check for discontinuities in the derivative, but in this case there are none). Graphically (Figure 6.12B), the derivative is a flat line at $y = 3$; it never touches zero. The absence of critical points should also make sense intuitively: The function is a straight line with a positive slope; the function *values* change over the x-axis, but the function *behavior* never changes.

Example 2 The second example is the function $f(x) = 3x^2 + 2$. Its graph is shown in Figure 6.13 and the equations are below.

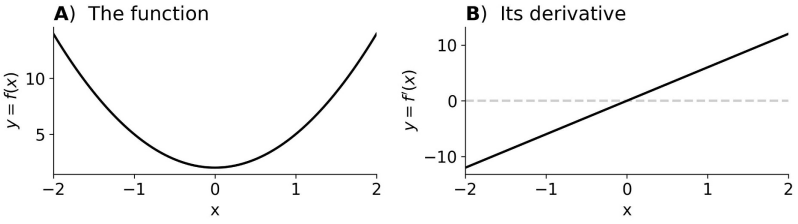


Figure 6.13: Finding and interpreting critical points: example 2.

$$f(x) = 3x^2 + 2 \tag{6.12}$$

$$f'(x) = 6x \tag{6.13}$$

$$0 = 6x \tag{6.14}$$

$$x = 0 \tag{6.15}$$

There is one solution to the equation ($x = 0$), which means there is one critical point. Again, that is consistent with the graph: The function changes behavior exactly once, and the derivative passes through $y = 0$ at exactly that point.

Example 3 Here we have the trig function $f(x) = \cos(2\pi x)$. Its graph is shown in Figure 6.14 and the equations are below.

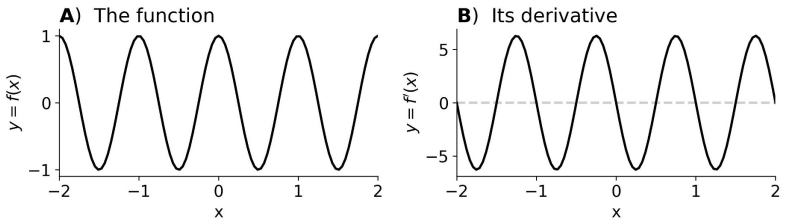


Figure 6.14: Finding and interpreting critical points: example 3.

The "extra" 2π comes from the chain rule, which you'll learn about in the next chapter.

$$f(x) = \cos(2\pi x) \tag{6.16}$$

$$f'(x) = -2\pi \sin(2\pi x) \tag{6.17}$$

$$0 = \sin(2\pi x) \tag{6.18}$$

$$x = \frac{k}{2}, \quad k \in \mathbb{Z} \tag{6.19}$$

This function has an infinite number of critical points, because there is an infinite number of values that satisfy $\sin(2\pi x) = 0$. The solution comes from the fact that $\sin(\theta) = 0$ for any $\theta = k\pi$ where k is some integer ($\mathbb{Z}$ is the set of integers). Thus, the solution is $2\pi x = k\pi$.

It is tempting to look at the derivative of sine simply as the cosine function. That is, of course, true, but please take a moment to look back at Figure 6.14 not as a plot of sine and cosine, but as a plot of sine and its derivative — notice how the sign and zero-crossings in the derivative map onto the behavior of the function.

Example 4 Next up is the elegant $f(x) = e^x$. See Figure 6.14 and equations below.

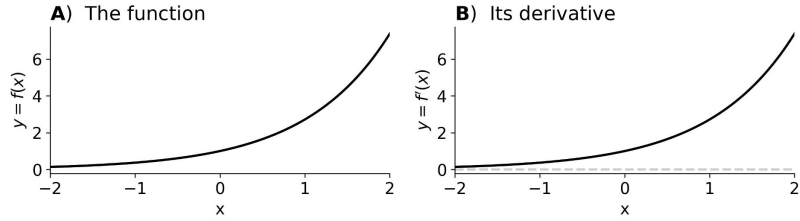


Figure 6.15: Finding and interpreting critical points: example 4.

$$f(x) = e^x \tag{6.20}$$

$$f'(x) = e^x \tag{6.21}$$

$$0 = e^x \tag{6.22}$$

This function also fails the derivative solution test: e^x is a strictly positive function. It can *never* equal zero, and therefore, e^x has no critical points. It goes up forever and never comes back down.[4]

Example 5 This one is a piecewise function. The definition of the function and its derivative are below, and are visualized in Figure 6.16.

$$f(x) = \begin{cases} x/2 & \text{if} \quad x < 0 \\ x^2 + 1 & \text{if} \quad 0 \leq x < 3 \\ -\ln(x) & \text{otherwise} \end{cases} \tag{6.23}$$

$$f'(x) = \begin{cases} 1/2 & \text{if} \quad x < 0 \\ 2x & \text{if} \quad 0 \leq x < 3 \\ -1/x & \text{otherwise} \end{cases} \tag{6.24}$$

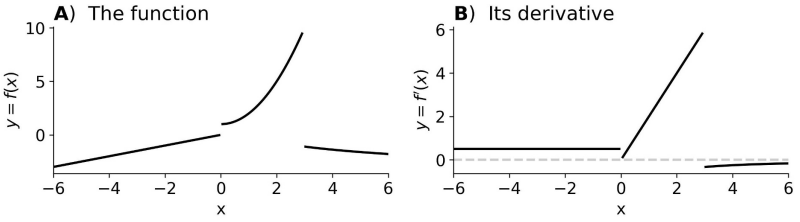


Figure 6.16: Finding and interpreting critical points: example 5.

Each piece of this function needs to be differentiated separately, which means that we have to search through each piece for critical points.

$$1/2 = 0 \ \rightarrow \text{no c.p.} \tag{6.25}$$

[4]Like many concepts in math that involve ∞, the physical interpretation defies the imagination, but the pure mathematics are internally consistent.

$$2x = 0 \ \rightarrow \text{c.p. at } x = 0 \tag{6.26}$$

$$-1/x = 0 \ \rightarrow \text{no c.p.} \tag{6.27}$$

From this we can conclude that this function has exactly one critical point at $x = 0$... or does it have more?!

Remember that critical points occur when the derivative equals zero — *and at discontinuities where the function is defined.* There are two jump discontinuities in this function: at $x = 0$ and at $x = 3$. The function is defined at those two points, and so those two points are included in the list of critical points. This brings the list of critical points to [0,0,3]. Zero is listed twice because it independently satisfies two conditions for being a critical point. But of course, it's only one critical point, so it gets counted only once.

Thus, the conclusion from this example is that this function has two critical points: $x = 0$ and $x = 3$. The corresponding critical values are $y = 1$ and $y = -\ln(3)$.

6.6 Inflection points

As I wrote in the first subsection of this chapter, inflection points are conceptually similar to critical points: They identify x values at which a function's behavior changes. However, the changes in the function behavior indicated by inflection points is more subtle compared to the changes indicated by critical points.

Have a look at Figure 6.17. Panel A shows a polynomial function. It's an odd-ordered polynomial with a positive coefficient, so it goes down to $-\infty$ to the left and up to $+\infty$ to the right. Close to zero, the function has a twist and is briefly downwards-going. The first derivative (panel B) is a 4^{th} order polynomial that dips below zero (due to the -10 term) and crosses the $y' = 0$ line at $\tau = \pm\sqrt[4]{2}$. These are the critical points, and indicate qualitative

changes in the function direction.

On the other hand, the inflection point in panel A is clearly not a critical point: the function does not change direction or have a discontinuity, nor does the first derivative equal zero at this point. Instead, the function (subtly) transitions from bending downwards steeply to bending downwards gradually. The plot of the second derivative crosses zero at $\tau = 0$ (that's the inflection point), whereas the critical points do not cross $y = 0$ in the second derivative.

I'm using τ instead of x here to avoid x-exhaustion.

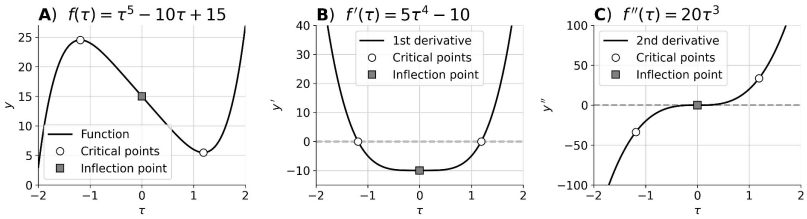


Figure 6.17: Dissociation of critical points and inflection points.

The conclusion is that critical points and inflection points are similar in that they reveal important points in the function. But they are distinct in two ways:

Math: Critical points are identified from the first derivative whereas inflection points are identified from the second derivative.

Interpretation: An inflection point is where the function continues to increase, but the function changes from increasing faster to increasing slower. (Same goes for function decreases.) The technical term for this is *concavity*.

Concavity Think of a *cave* in the mountains: It dimples inwards. In terms of mathematical functions, the second derivative is used to evaluate whether a curve is "concave up" or "concave down" at some point. Concave-down means the function has a local maximum and bends downwards, whereas concave-up means the

function has a local minimum and bends upwards[5].

This is illustrated in Figure 6.18. I generated a function, calculated its second derivative, and marked all the points as black where the second derivative is positive, and gray where the second derivative is negative. In other words, the sign of the second derivative indicates where the function bends upwards vs. downwards.

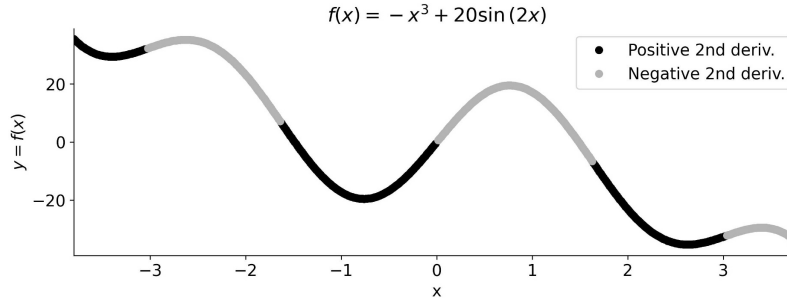


Figure 6.18: Identifying concavities ("concave down") and convexities ("concave up") using the second derivative.

6.6.1 Identifying inflection points

The procedures for identifying inflection points are nearly identical to those for critical points; the difference is that you apply the methods described above to the second derivative instead of to the first derivative. The rest of this section, therefore, is focused on interpreting and using inflection points.

6.6.2 Example of sigmoid function

Another function that illustrates an inflection point is the sigmoid function, which is also called the "s-curve" (Figure 6.19A).

Notice that the function increases through the entire plot area — in fact, this function always increases; it never decreases. That is evidenced by its positive-everywhere derivative (panel B). More

[5] "Concave up" is also called *convex*.

importantly for this discussion, the derivative is never zero, so this function has no critical points.

But it does have an inflection point at $t = 0$.

In other words, the function always increases, but it does not always increase at the same rate. Instead, the function climbs steeply from $t = -5$ to 0, and then increases more slowly as t increases. That's also evidenced by its derivative, which gets larger as $t \to 0$ from the left, and then gets smaller as $t \to \infty$.

Visual inspection of Figure 6.19B reveals that the derivative has a peak at $t = 0$; that's the moment at which the function transitions from increasing *faster* to increasing *slower*.

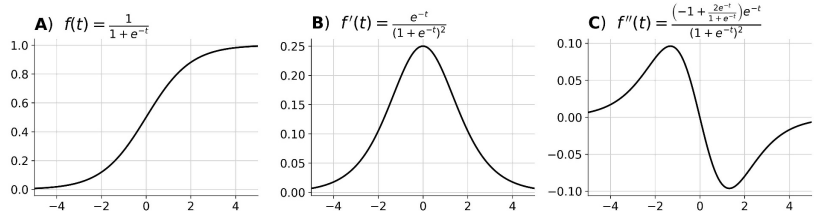


Figure 6.19: A sigmoid function and its first two derivatives.

And this brings us to the second derivative. The second derivative (Figure 6.19C) clearly crosses zero, and is negative for $x > 0$. That zero-crossing indicates the inflection point, and the negative second derivative means that the rate at which the function increases, is decreasing; it's rising more slowly.

The equation of the second derivative would be quite tedious to solve for $f'' = 0$ by hand, but `sympy` confirms that it has a solution at $t = 0$ (shown in the online code). Once you know the solution, you can easily confirm in the equation that $f''(0) = 0$.

Function increases vs. decreases The sigmoid function increases, and so its derivative is positive and the second derivative is positive for $x < 0$ and negative for $x > 0$. A decreasing function would show the opposite pattern: negative second derivative indicating *faster decreases* in the function and positive second

derivative indicating *slower decreases* in the function. This can be demonstrated simply by flipping the sign of the sigmoid (Figure 6.20).

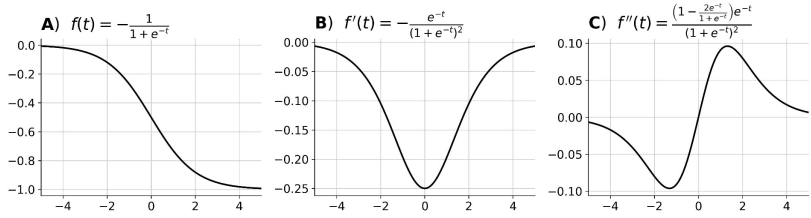


Figure 6.20: A negative sigmoid. Contrast the signs of the first and second derivatives here vs. with those of the positive sigmoid.

6.7 Evaluating inflection points

The concavity test

A word of caution: simply having a second derivative equal zero is not sufficient to prove that that point is an inflection. One really simple example is the function $f(x) = x$. That function has a second derivative of $f''(x) = 0$ for all values of x, and yet does not inflect.

You can combine the second derivative test with a concavity test, in which you examine the first derivative, and the function behavior, just before vs. just after $x = x_0$, where x_0 is the point at which $f''(x_0) = 0$.

Figure 6.21 shows an example function with two critical points shown as black circles and one inflection point shown as a gray circle.

One graph, many visualizations Rather than plotting the function and each derivative in a separate panel, it's possible to include

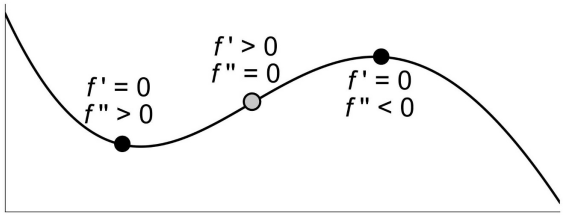


Figure 6.21: Critical points (black circles) and inflection points (light gray circle) on a function, with signs indicated.

all three in the same graph (Figure 6.22). This can be useful because (**1**) it takes up less space and (**2**) it helps visually align the insights gained from each feature.

As long as you are mindful of interpreting the results properly, combining multiple lines into one plot can be advantageous. In this case, the appropriate interpretation is to inspect the graph for the alignment of qualitative features, not the numerical values on the y-axis.

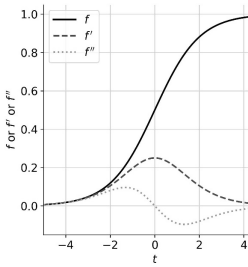


Figure 6.22: All lines from Figure 6.19 in one graph. Note the different y-axis units.

6.8 Applications of critical points

What are critical points good for? Critical points have many applications, two of which I will highlight here.

One is to better understand the characteristics of a function. As you learned throughout this chapter, critical points indicate qualitative changes in the behavior of a function.

A second application of critical points is in optimization. This is the main use of critical points in machine learning, control engineering, deep learning, finance, and myriad other real-world applications. The idea is to represent a real-world objective using a mathematical function (e.g., revenue of an online store, time spent playing a video game, subjective quality ratings of wine), and use critical points to identify relevant extrema in that function such as minima or maxima. The critical points, for example

that maximize a factory revenue function, will reveal how to balance resources between investments, ad revenue, personnel costs, etc. I'll show examples of using critical points in optimization problems in Chapter 9.

6.8.1 Real-world applications of second derivatives

Here are a few examples of how experts use the second derivative to evaluate technological and society dynamics, and to make predictions about the future that guide policy and R&D development.

The human population of planet Earth is increasing, which concerns many environmentalists and economists. The sign of the second derivative is used to make predictions about whether the population may continue increasing over the next century, or may reach a peak and then decline.

A second example: the CO_2 concentration in the atmosphere has been rising since the industrial revolution, causing many people to be concerned about dangerous or even catastrophic effects of climate change. If the second derivative of atmospheric CO_2 is negative, it means that the rate of pollution is decreasing; but if the second derivative is positive, then CO_2 emissions are increasing by an increasing amount. Appropriate policy actions, therefore, depend on accurate estimates of the second derivative of environmental pollution.

A third example: AI has become increasingly capable of emulating human intellectual and creative capabilities (I'm writing this around 1.5 years after the public release of ChatGPT). AI development continues to increase, and the second derivative of that increase is used to guess whether future AI capabilities will be *much much* better than current capabilities, or only a little bit better. You are reading this in what is, for me right now, the future, but the point is that the second derivative of empirical functions is vital to modeling technological developments.

6.9 Exercises

6.1. Find the code that generates Figure 6.7. Spend a few moments exploring the impact of the x-axis grid resolution on the accuracy of the critical points. No need to copy the code into a separate cell yet; just try a few different resolutions by changing the third input into `np.linspace`. Have you noticed any relationship between accuracy of the empirically estimated critical points and the analytic values?

Now to investigate the accuracy systematically. In a new code cell, run an experiment in which you systematically vary the number of points between $x = -1.5$ and $x = .7$ from 15 to 200. Within each iteration of the for-loop over x-axis resolutions, generate the new x-axis grid, recalculate the function and its derivative (using `numpy`), find the critical points, and quantify the accuracy of the critical points estimates as the "sum of squared errors" from the analytic critical points that you can solve on paper.

SSE is a typical measure of discrepancies in statistics and machine learning!

$$\text{SSE}_r = \ln\left(\sum_{i=1}^{k}(c_i - a_i)^2\right) \tag{6.28}$$

where r is the resolution, c is a vector of empirically estimated critical points, a is a vector of analytically solved critical points in the same order as c, and k is the total number of critical points. I've included the log here because it helps to visually compare the errors across the range of resolutions.

Finally, plot the results as in Figure 6.23.

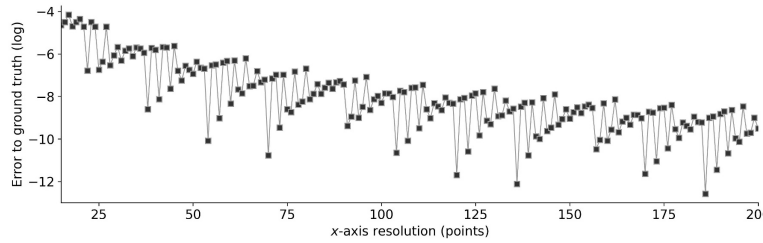


Figure 6.23: Visualization for Exercise 1.

Why are the errors "choppy" instead of smoothly decreasing with increasing resolution? Think of a reason; I'll write the answer after the following two notes.

Note 1: It is interesting to compare this plot to the same results but setting the x-axis to be Δx instead of the number of points (not shown here but it's in the online code).

Note 2: The lines connecting the dots are technically incorrect, because they incorrectly imply known values between the dots. On the other hand, I think the lines help visualize the nature of the errors. Data visualization rules can be broken when the gain in communication outweighs the severity of the violation.

The reason for the non-smooth changes in the estimation errors is that the resolution determines the values of x, which means that the values of x happen to get closer or further away from the analytic solutions.

Overall, the conclusion of this exercise is that empirical search methods are not guaranteed to uncover *exactly* correct answers; instead, they reveal *approximations* that generally become more accurate (though not necessarily monotonically so) as the search-grid resolution increases.

6.2. The purpose of this exercise is to illustrate that inflection points are not necessarily critical points. Start from the function $f(x) = x^3 - 3x$. Using paper and pen (that is, don't use Python yet!) find the critical points and inflection points, and determine the type of each point.

Then, implement the function in `sympy`, confirm that you correctly calculated the first and second derivatives, critical points, and inflection points. Print them out as follows:

$$f(x) = x^3 - 3x$$

$$\frac{df}{dx} = 3x^2 - 3$$

$$\frac{d^2f}{dx^2} = 6x$$

```
Critical points (x,y):
  ( -1.0, 2 )
  ( 1.0, -2 )

Inflection points (x,y):
  ( 0.0, 0 )
```

Finally, use `matplotlib` to visualize the function, its derivatives, and the critical and inflection points as in 6.24. I used substitution with list-comprehension to get numerical results from the symbolic expressions, but you can use lambdafication if you prefer.

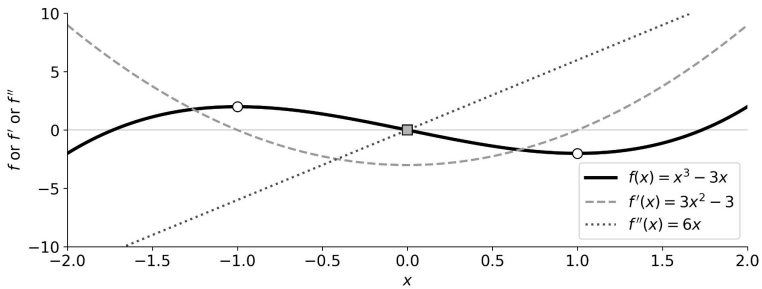


Figure 6.24: Visualization for Exercise 2. White circles indicate critical points, gray square indicates the inflection point.

6.3. This exercise will help you learn how to work with cube root functions in `sympy`. The goal is deceptively simple: Differentiate and plot the function $f(x) = \sqrt[3]{x}$ (recall that $\sqrt[3]{x} = x^{1/3}$). This function is defined for negative numbers, for example, $\sqrt[3]{-8} = -2$ because $(-2)^3 = -8$.

To start, run the following code:

```
x = sym.symbols('x')
fx = x**(sym.sympify(1)/3)
```

If you're unsure what `sym.sympify` does, then observe the difference between `sym.sympify(1)/3` and `1/3`. Try plotting the `sympy` expression `fx`. Does it show results for $x < 0$? Next, try evaluating the code `fx.subs(x,-8)`.

Now rerun the code you just ran but with the following replacements:

```
x = sym.symbols('x',real=True)
fx = sym.real_root(x,3)
```

The first line constrains variable `x` to be real-valued (without that explicit constraint, it could take on complex-valued numbers), and the second line is a `sympy` function that computes only real-valued roots of a polynomial.

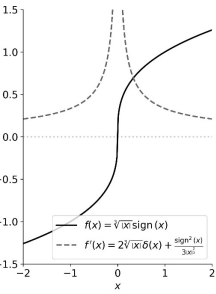


Figure 6.25: Visualization of Exercise 3.

Armed with this new knowledge, I believe you can now create Figure 6.25.[6] But notice the strange formatting of the function and its derivative. `sympy` doesn't like putting negative numbers in radicals.

6.4. This exercise is basically the same as the previous, but do everything in `numpy` instead of `sympy`. You will also encounter warning messages if you try to implement `x**(1/3)`; instead, figure out how to use the function `np.cbrt`.

I didn't print the figure here in the book, but it's in the online code. It looks nearly identical to Figure 6.25, although the equations in the legend look simpler.

[6] You are likely to encounter issues with `lambdify`ing the derivative... part of this exercise is to figure out why and how to resolve the issues.

6.5. The purpose of this exercise is to write code to produce Figure 6.26. This will help you improve your calculus-coding and visualization skills, and will be a gentle introduction to multivariable calculus.

Each line in the plot corresponds to the following equation:

$$f(x, \alpha) = \frac{2(x - \alpha)^2 + 3}{(x - 1)^2 + 2} \tag{6.29}$$

This is a function of *two* variables. However, α is a constant while x is the independent variable. Thus, each line in the plot corresponds to a unique value for α, ranging from -1 (darkest line) to 3 (lightest line) in 22 steps.

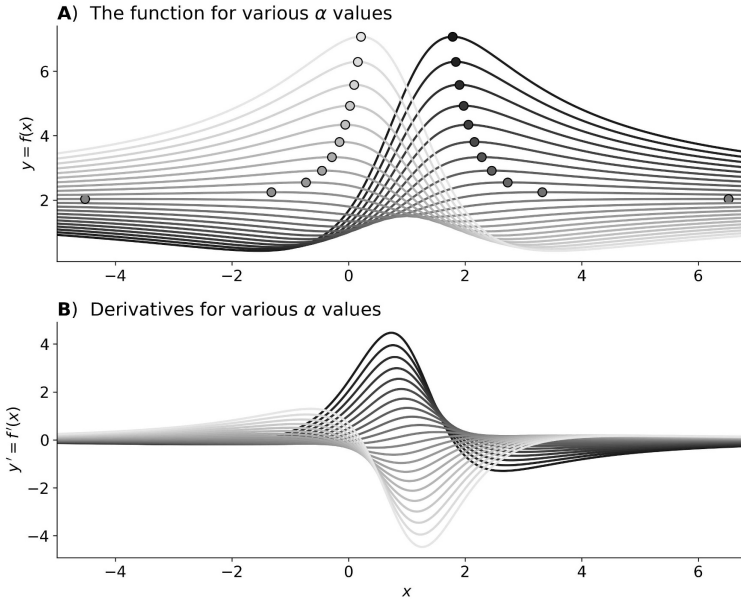


Figure 6.26: Visualization for Exercise 5.

Implement this function and its derivative in `sympy`, then `lambdify` both to be able to evaluate the functions for each line in the plot.

The dots correspond to the critical points with the maximal value of $f(x)$. To calculate those points, use `sympy` to find the critical points, and select only the critical point with the maximal y value (there will be multiple critical points for each function).

I recommend trying to get your plot to match mine as close as possible; and when you're confident with your code, explore and experiment! Try plotting with different colors or line thicknesses, try changing the function definition, try changing which number is α (for example, fix the current α to 2 and replace "-1" in the denominator with α). If you get a really beautiful plot, then you can print it on a t-shirt that your dad can wear to the next family picnic :)

Note: This exercise is tricky, especially figuring out how to get `sympy` to give the desired output. See the footnote for

some hints[7].

Final (optional) part of this exercise: I have previously mentioned that using lists and the `.subs` method to convert `sympy` to `numpy` is convenient but slow, whereas `lambdify` requires additional code but is faster. This exercise is an interesting opportunity to explore this.

6.6. When introducing gradient descent in Figure 6.10, I showed only the initial starting location and the final estimate. The goal here is to visualize the descent itself by plotting intermediate values of x_0.

Copy and paste the code that created Figure 6.10 (or, for an additional challenge, write the code from scratch), and modify it to include plotting the updated minimum inside the for-loop.

There are 100 iterations in the optimization, but 100 dots makes the plot look crowded. Therefore, you should plot only every 5^{th} update. I made a few other modifications, including fixing the starting value to $x_0 = -1$, adding a vertical line corresponding to the analytic solution, and making the dots lighter and smaller as they approach the solution. My results are shown in Figure 6.27.

Now that you have the code, you can explore the gradient descent algorithm, for example by changing η or the number of iterations.

[7] (**1**) `lambdify` the function and derivative before the for-loop using both variables. (**2**) Solve for $f' = 0$ using `sym.solveset` instead of `sym.solve`. (**3**) Restrict the solutions to `sym.S.Reals`. (**4**) Convert the set of critical points to a `numpy` array.

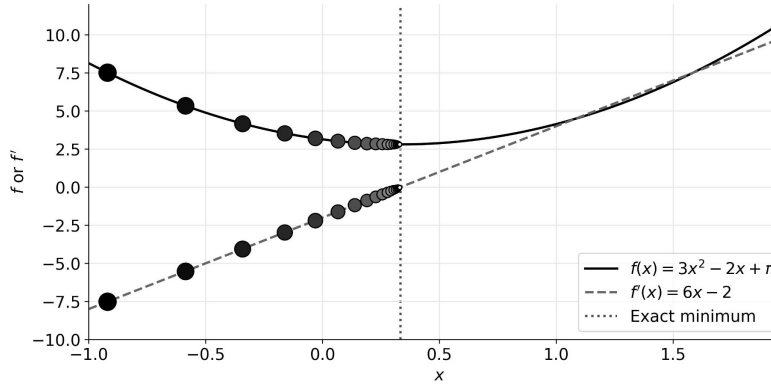


Figure 6.27: Visualization for Exercise 6.

6.7. Here's a wacky function:

$$f(x) = \begin{cases} e^{2\cos(2x)}, & -1.5 \leq x \leq 0 \\ e^{2\cos(2x)} - \log(x^2), & 0 < x \leq 4.9 \end{cases} \tag{6.30}$$

Implement this function in `numpy`. (For an extra challenge, see if you can implement the function using one line of code.) Use the grid-search method to identify the critical and inflection points, and draw them on the graph of the function as in Figure 6.28.

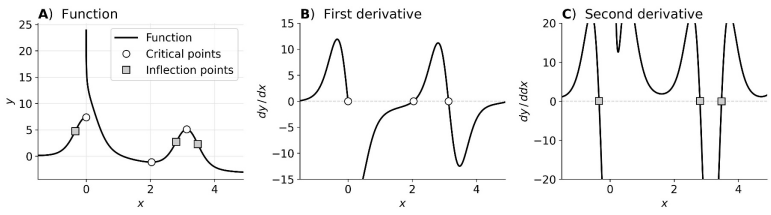


Figure 6.28: Visualization for Exercise 7.

6.8. The goal of this exercise is to continue exploring how to combine analytic methods in `sympy` with numerical approximation methods in `numpy` to identify critical and inflection points in functions that are difficult to solve exactly.

Start from the function $f(x) = \ln|x| + \cos(x)$. Use `sympy` to obtain the first and second derivatives. Print them out as in Figure 6.29.

$$f(x) = \log(|x|) + \cos(x)$$

$$\frac{df}{dx} = -\sin(x) + \frac{\text{sign}(x)}{|x|}$$

$$\frac{d^2f}{dx^2} = -\cos(x) + \frac{2\delta(x)}{|x|} - \frac{\text{sign}^2}{x^2}$$

Figure 6.29: Screenshot of first part of Exercise 8.

Next, find all critical and inflection points in the restricted domain of $-3\pi \leq x \leq 3\pi$, and print out the (x, y) pairs as below:

```
Critical points (x,y):
  ( -9.31, 1.24 )
  ( -6.44, 2.85 )
  ( -2.76, 0.09 )
  ( -1.12, 0.55 )
  (  1.12, 0.55 )
  (  2.76, 0.09 )
  (  6.44, 2.85 )
  (  9.31, 1.24 )

Inflection points (x,y):
  ( -7.88, 2.04 )
  ( -4.67, 1.50 )
  ( -1.86, 0.34 )
  (  1.86, 0.34 )
  (  4.67, 1.50 )
  (  7.88, 2.04 )
```

Finally, visualize the function and its derivatives with critical and inflection points indicated, as in Figure 6.30. *Note:* There are several tricky aspects to this exercise; I have some hints in the footnote[8].

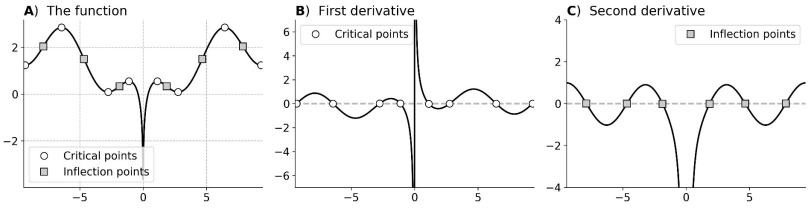


Figure 6.30: Visualization for final part of Exercise 8.

[8] (**1**) `sympy` cannot solve $f' = 0$ or $f'' = 0$ analytically, so you will need to use numerical methods. (**2**) `sym.lambdify` cannot evaluate the second derivative, so you'll need to use `.subs` in combination with converting from `sympy` to `float` to `numpy`.

Chapter 7

Differentiation rules

You now know how to differentiate individual terms, like x^3 or $\ln(x)$. That's a great start and you should be proud of your achievements!

But differentiating individual terms won't take you very far; you need to know how to differentiate functions with terms that multiply, compose, and divide. The goal of this chapter is to introduce you to rules and theorems in differential calculus that will bring your differentiation skills to the next level. By the end of this chapter, you'll be as good as Karl Gauss at differentiating functions[1].

7.1 Product rule

The *product rule* is a formula that you use to differentiate the product of multiple terms, like $x^2 \ln(x)$, where each term can be defined as a one-term function (e.g., $f(x) = x^2$ and $g(x) = \ln(x)$). Most of this section focuses on the product of two terms, but I'll show how to extend the product rule to any number of multiplying functions after explaining the rule and showing a few examples.

Here is the product rule (the second line is simply the first line rewritten without the "(x)" for readability):

$$[\, f(x) \cdot g(x) \,]' = f'(x) \cdot g(x) + f(x) \cdot g'(x) \tag{7.1}$$

$$[\, fg \,]' = f'g + fg' \tag{7.2}$$

In words, the product rule states that the derivative of the product of two functions is the derivative of the first times the second, plus the first times the derivative of the second.

[1] Well, OK, maybe not *that* good, but the point is that differential calculus isn't so difficult to master.

I will show you the proof of that rule in a moment, but first I want to show three examples.

Example 1 We will differentiate the function $h(x) = x^2 \cos(x)$. This is expressed as *one* function, but you can think of it as the product of *two* functions. That is:

$$h(x) = x^2 \cos(x) \tag{7.3}$$

$$f(x) = x^2 \tag{7.4}$$
$$g(x) = \cos(x) \tag{7.5}$$
$$h(x) = f(x)g(x) \tag{7.6}$$

And now we can apply the product rule $(fg)' = f'g + fg'$:

$$f'(x) = 2x \tag{7.7}$$

$$g'(x) = -\sin(x) \tag{7.8}$$

$$h'(x) = 2x\cos(x) - x^2 \sin(x) \tag{7.9}$$

Figure 7.1A shows the three functions, and panel B shows the derivative of $h(x)$.

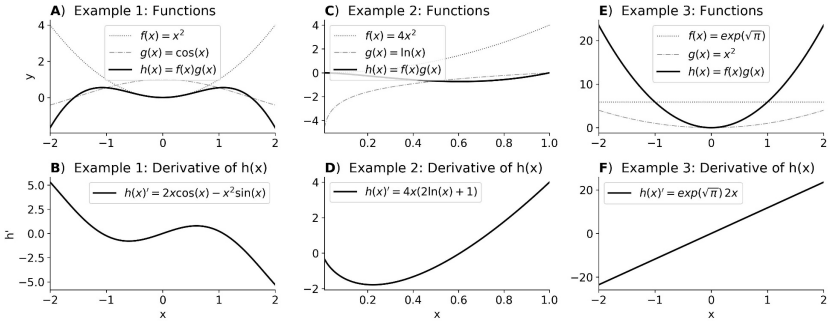


Figure 7.1: Examples of the product rule.

Important: The derivative of the product is *not* the product of the derivatives. That is, $(fg)' \neq f'g'$. I don't show that directly in

the figure, because you will demonstrate this yourself in Exercise 1. You will also understand why that is the case when you see the proof of the product rule in a few pages.

Example 2 Please try to differentiate the function $h(x) = 4x^2 \ln(x)$ before reading my solution below.

Again, start by seeing that this *one* function can be expressed as the product of *two* functions:

$$h(x) = 4x^2 \ln(x) \tag{7.10}$$

$$f(x) = 4x^2 \tag{7.11}$$
$$g(x) = \ln(x) \tag{7.12}$$
$$h(x) = f(x)g(x) \tag{7.13}$$

And now to differentiate:

$$f'(x) = 8x \tag{7.14}$$

$$g'(x) = \frac{1}{x} \tag{7.15}$$

$$h'(x) = 8x \ln(x) + \frac{4x^2}{x} \tag{7.16}$$

$$= 4x(2 \ln(x) + 1) \tag{7.17}$$

What do you think of the final simplification? On the one hand, $4x^2/x$ is actually *not identical to* $4x$ because the latter implies that x may equal zero while the former forbids it. So that algebraic cancellation would require an additional $x \neq 0$ restriction. But in this example, we already exclude any non-negative numbers (which includes zero) because of the natural log function.

These functions are shown in Figure 7.1C-D.

Example 3 One more example, which will additionally provide another proof that $g(cx)' = cg(x)'$ for constant term c (remember this was part of the linearity of differentiation).

$$h(x) = e^{\sqrt{\pi}}x^2 \tag{7.18}$$

$$f(x) = e^{\sqrt{\pi}} \tag{7.19}$$

$$g(x) = x^2 \tag{7.20}$$

$$h'(x) = 0x^2 + e^{\sqrt{\pi}}2x \tag{7.21}$$

$$= e^{\sqrt{\pi}}2x \tag{7.22}$$

Friendly reminder: $(fg)' = f'g + fg'$

$e^{\sqrt{\pi}}$ is a constant, and therefore its derivative is zero. These functions are shown in Figure 7.1E-F.

7.1.1 Intuition of the product rule

The product rule might feel unintuitive. Indeed, if we define $h = fg$ then why isn't $h' = f'g'$? This formulation does actually work for addition: If $h = f + g$ then $h' = f' + g'$.

The key insight is that $h(x)$ is an *interaction* between $f(x)$ and $g(x)$. That is, small changes in h come from two sources: (**1**) How f changes while g is held constant, and (**2**) how g changes while f is held constant.

Thus: $f'g$ describes how f changes, multiplied by the current value of g, while fg' describes how g changes, multiplied by the current value of f.

Here's an analogy: Imagine two friends named $f$rank and $g$eorge. If they are talking to themselves in separate rooms, we can simply listen to each person independently. But when they interact with each other, then we can understand $f$rank only by also listening to $g$eorge, and we can understand $g$eorge only by also listening to $f$rank.

The idea of quantifying how changes in one function depend on changes in another function is also key to partial derivatives and gradients. More on this in Chapter 10!

7.1.2 Proving the product rule

The sequence of equations below shows the proof of the product rule. Much of it simply involves algebraic expansions, groupings, and simplifications (and concentration to avoid sign mistakes!), but there is one trick to getting this proof to work: adding and subtracting $f(x)g(x+h)$ (Equation 7.24). That trick allows you to group and factor the f and g terms, which in turn allows you to separate the derivatives for each function.

My advice is to read through the sequence of equations below and make sure you understand how each one follows from the previous. Then, put the book aside, take out some paper and a pencil, and reconstruct the proof starting from Equation 7.23.

$$[\,f(x)g(x)\,]' = \lim_{h\to 0} \frac{f(x+h)g(x+h) - f(x)g(x)}{h} \tag{7.23}$$

$$= \lim_{h\to 0} \frac{f(x+h)g(x+h) - f(x)g(x+h) + f(x)g(x+h) - f(x)g(x)}{h} \tag{7.24}$$

$$= \lim_{h\to 0} \frac{\big[f(x+h) - f(x)\big]g(x+h) + f(x)\big[g(x+h) - g(x)\big]}{h} \tag{7.25}$$

$$= \lim_{h\to 0} \frac{f(x+h) - f(x)}{h} \lim_{h\to 0} g(x+h) + \lim_{h\to 0} f(x) \lim_{h\to 0} \frac{g(x+h) - g(x)}{h} \tag{7.26}$$

$$= f'g + fg' \tag{7.27}$$

Three notes: (**1**) Remember the multiplicative property of limits ($\lim[f \times g] = \lim[f] \times \lim[g]$). (**2**) In the expression $\lim_{h\to 0} f(x)$, there is no h term. Hence, that limit is simply $f(x)$. (**3**) Equating $\lim_{h\to 0} g(x+h)$ with $g(x)$ is valid only if $g(x)$ is continuous. On

the other hand, the derivative can exist only at continuous points in a function (this claim was stated in a previous chapter and will be proven in the next chapter), so this constraint is a more general reminder about limits, not about this particular proof.

I do not recommend memorizing the proof, but I do recommend memorizing the product rule. You'll use it very often in calculus.

7.1.3 More than two functions

The principle of the product rule applies to any number of functions. For three functions of x, it looks like this:

$$[\,fgh\,]' = f'gh + fg'h + fgh' \tag{7.28}$$

You can see the principle: When multiplying N functions of x, differentiate each individual function and multiply by all other functions, follow this pattern for all other functions, and then sum them together.

Another way to think about this is to group functions:

$$k = fg \tag{7.29}$$

$$k' = f'g + fg' \tag{7.30}$$

$$[\,kh\,]' = k'h + kh' \tag{7.31}$$

$$[\,(fg)h\,]' = (f'g + fg')h + (fg)h' \tag{7.32}$$

7.2 Chain rule

All rules in calculus are important, but the chain rule stands out as being super-duper really a lot important. The chain rule allows you differentiate composite functions $f(g(x))$.

Here's the chain rule; the second line is just a simplified re-write of the first:

$$f(g(x))' = f'(g(x))\, g(x)' \tag{7.33}$$

$$f(g)' = f'(g)\, g' \tag{7.34}$$

In words: to differentiate the composite of one function embedded inside another, differentiate the *outer* function and multiply by the derivative of the *inner* function. Here is an example:

$$f(x) = x^2 \tag{7.35}$$

$$g(x) = \cos(x) \tag{7.36}$$

$$f(g(x)) = [\cos(x)]^2 = \cos^2(x) \tag{7.37}$$

$$f(g(x))' = -2[\cos(x)]\sin(x) \tag{7.38}$$

So, the derivative of $\cos^2$ is $2\cos$, and then multiply by the derivative of cos, which is –sin.

Figure 7.2 shows the functions and derivative. A few notes: (**1**) Squaring the cosine function doubles its frequency because the negative "bounces" become positive — compare the solid black line and dashed gray lines; (**2**) The product of the sine and cosine in the derivative also doubles the frequency; (**3**) The range of $\cos(x)$ is ± 1, but the range of $f(g(x))$ is $[0, 1]$ because of the squaring.

Here's another example:

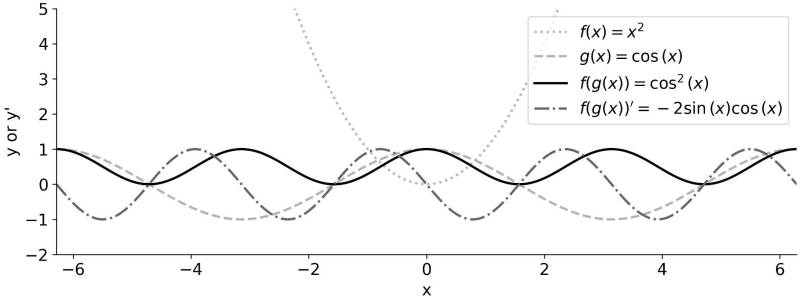


Figure 7.2: Two functions, their composition, and its derivative, all in one plot.

$$f(x) = x^2 + \pi \tag{7.39}$$

$$g(x) = \cos(x) \tag{7.40}$$

$$g(f(x)) = \cos(x^2 + \pi) \tag{7.41}$$

$$g(f(x))' = -\sin(x^2 + \pi)2x \tag{7.42}$$

The derivative of the cosine of *something* is minus sine of *something*, and then multiply by the derivative of that *something* (in this example, *something* is $x^2 + \pi$).

A third example:

$$f(x) = 4x^2 \tag{7.43}$$

$$g(x) = \ln(x) \tag{7.44}$$

$$f(g(x)) = 4(\ln(x))^2 \tag{7.45}$$

$$f(g(x))' = 8\ln(x)\frac{1}{x} \tag{7.46}$$

One final example, using the same two functions defined above:

$$g(f(x)) = \ln(4x^2) \tag{7.47}$$

$$g(f(x)) = \frac{1}{4x^2}8x = \frac{2}{x} \tag{7.48}$$

These last two examples are interesting, because the two base functions were identical but I reversed their composition. And the resulting derivatives are completely different.

7.2.1 More than two functions

The principle of the chain rule applies to any number of functions; you just need to keep unpacking each "inner" function. For example:

$$[\,f(g(h))\,]' = f(g(h))' \times g(h)' \times h' \tag{7.49}$$

7.3 Quotient rule

The quotient rule is another important rule to memorize in differential calculus. Unfortunately, it's a bit longer and less intuitive than the product or chain rules. But there is a limerick that will help you remember it.

Here's the quotient rule (f and g are both functions of x):

Be mindful of the numerator: it initially looks like the product rule, but it isn't.

$$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2} \tag{7.50}$$

I know: it looks weird, unintuitive, and hard to remember. Here's a mnemonic limerick that people use to help remember the rule: *low d-high minus high d-low, square the bottom and away we go!*

First a few examples to give you some practice with the mechanical aspects of implementing the quotient rule, then I'll prove the rule.

In all of these examples, I encourage you to differentiate (f/g) on your own to confirm the answers.

Here's the first example:

$$f(x) = x^2 \tag{7.51}$$

$$g(x) = \cos(x) \tag{7.52}$$

$$\frac{f(x)}{g(x)} = \frac{x^2}{\cos(x)} \tag{7.53}$$

$$\left(\frac{f}{g}\right)' = \frac{2x\cos(x) + x^2\sin(x)}{\cos^2(x)} \tag{7.54}$$

These functions are shown in Figure 7.3. Note the difference in y-axis scaling between this and Figure 7.2, which had the same two base functions. Functions with variables in the denominator can become unstable (blowing up to $\pm\infty$); this poses significant headaches in applied math including machine learning. Indeed, if you study machine learning algorithms, you will encounter "regularization" or "stabilization" terms added to the denominator to reduce such numerical instability risks.

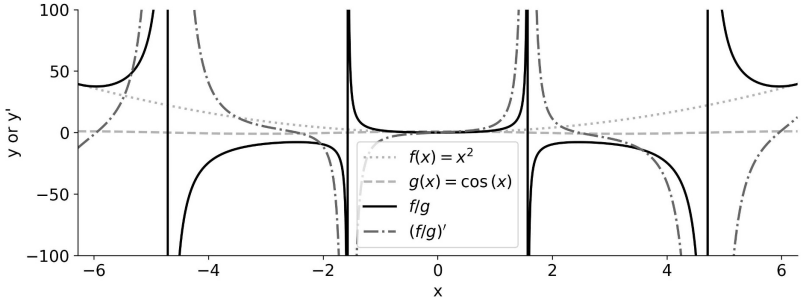


Figure 7.3: Two functions, their composition, and its derivative. $g(x)$ appears to be a flat line because of the y-axis scaling.

The next example:

$$f(x) = \ln(x) \tag{7.55}$$

$$g(x) = 4x^2 \tag{7.56}$$

$$\left(\frac{f}{g}\right)' = \frac{4x^2/x - \ln(x)8x}{16x^4} \tag{7.57}$$

$$= \frac{4x(x/x - \ln(x)2)}{16x^4} \tag{7.58}$$

$$= \frac{1 - 2\ln(x)}{4x^3} \tag{7.59}$$

And the last example (c is a constant):

$$f(x) = x^2 \tag{7.60}$$

$$g(x) = c \tag{7.61}$$

$$\left(\frac{f}{c}\right)' = \frac{2xc - x^2\ 0}{c^2} \tag{7.62}$$

$$= \frac{2x}{c} \ = \ \frac{1}{c} \cdot 2x \tag{7.63}$$

$$= \frac{1}{c} f' \tag{7.64}$$

This final example provides yet another demonstration of the linearity of differentiation.

7.3.1 Proving the quotient rule

There are several ways to prove the quotient rule. You can use the limits definition of the derivative, as I have done to prove several other rules.

But a neat thing about mathematical proofs is that once you prove some rules, you can use those to prove other rules. A simple proof of the quotient rule is to represent the *ratio* of functions as the *product* of functions, and then apply the product rule. Observe:

$$\frac{f}{g} = f \cdot \frac{1}{g} \tag{7.65}$$

$$\left(\frac{f}{g}\right)' = \left(f \cdot \frac{1}{g}\right)' \tag{7.66}$$

$$\left(f \cdot \frac{1}{g}\right)' = f \cdot \left(\frac{1}{g}\right)' + f' \cdot \left(\frac{1}{g}\right) \tag{7.67}$$

$$= \frac{-fg'}{g^2} + \frac{f'}{g} \tag{7.68}$$

$$= \frac{f'g}{g^2} - \frac{fg'}{g^2} \tag{7.69}$$

$$= \frac{f'g - fg'}{g^2} \tag{7.70}$$

The transition from the first term in Equation 7.67 to 7.68 might be confusing. You already know the power rule (knock down the exponent by 1 and bring the original exponent as a multiplicative scalar), which explains how $(1/g)' = (g^{-1})' = g^{-2}$. But why is there still a g' in the numerator?

The answer is that we need to apply the chain rule. g is not a variable, but a function of x. The chain rule tells us to differentiate the "outer" function ($[g(x)]'$) and then multiply by the derivative of the "inner" function ($g(x)$). But what is the derivative of $g(x)$? Well, we don't know what it is here: g is a generic placeholder for some function. So the derivative of g is g'. That's how we get g' in the numerator.

7.4 Implicit differentiation

Can you differentiate Equation 7.71 to obtain 7.72? I will explain how to derive that answer below, but I encourage you to see if you can arrive at that derivative on your own.

$$0 = x^2 + 3xy + y^2 \tag{7.71}$$

$$\frac{dy}{dx} = -\frac{2x+3y}{3x+2y} \tag{7.72}$$

This problem cannot be solved using any rules you've learned so far. Instead, you need to solve this using a technique called *implicit differentiation*. Implicit differentiation is key to advancing in calculus, including solving otherwise-impossible differentiation problems, proving certain rules and theorems, and solving differential equations.

But before learning how to differentiate implicitly, you need to understand the difference between *implicit* and *explicit* functions.

7.4.1 Implicit vs. explicit functions

Functions can be expressed in *implicit* or *explicit* form. Explicit is the style you are used to seeing: y or $f(x)$ on the left-hand-side, and all the other terms on the right-hand-side. Implicit, on the other hand, means that the x and y terms are mixed together.

Some functions can be algebraically transformed from implicit to explicit, for example:

$$\underbrace{xy = 1}_{\text{Implicit}} \quad \Rightarrow \quad \underbrace{y = \frac{1}{x}}_{\text{Explicit}}$$

Other functions, however, cannot be expressed explicitly, such as Equation 7.71. Implicit differentiation is useful in many situations, even when the function can be explicitly expressed, and is necessary when the function can only be expressed implicitly.

7.4.2 How to differentiate implicitly

Here's the procedure:

1. Differentiate both sides of the equation.
2. Define the derivative of y as y'.

3. Solve for y'.
4. Solve for y in the original equation to remove any y terms from the derivative (optional and might be impossible).

Let's start with a simple equation:

$$xy = 1 \tag{7.73}$$

It actually is possible to transform this expression into $y = x^{-1}$ and apply the power rule (adding the assumption $x \neq 0$). But this simple example is a great introduction to implicit differentiation. Observe:

$$xy = 1 \tag{7.74}$$

$$\frac{d}{dx}\Big[xy\Big] = \frac{d}{dx}\Big[1\Big] \tag{7.75}$$

y is a function of x, so y is short-hand for $y(x)$.

$$xy' + 1y = 0 \tag{7.76}$$

$$y' = \frac{-y}{x} \tag{7.77}$$

$$y' = \frac{-1}{x^2} \tag{7.78}$$

Equation 7.75 implements Step 1 (differentiate both sides of the equation). Notice the y' in Equation 7.76: We know that y is a function of x, but we don't know what that function is. Therefore, we need to apply the product rule to xy and use y' as the derivative of y (Step 2).

Getting from Equation 7.75 to 7.76 is the calculus part of implicit differentiation; the rest is algebra. Equation 7.77 is Step 3 (solve for y'). In many implicit differentiation problems, this will be the end and the final solution will have y' on the left-hand side and y-terms on the right-hand side. However, in this case, we can see from Equation 7.74 that $y = 1/x$, and so the y term can be replaced, giving us only x terms on the right.

Example 2 Please try to differentiate the following expression using the 4-step procedure I wrote above, and see if you can confirm my answer. As with the previous example, this one technically could be solved by first transforming it into an explicit function of x, but use the implicit method (remember to use the chain rule when differentiating y^2!).

$$x^3y^2 = 5x^4 \tag{7.79}$$

$$y' = \frac{5\sqrt{5x}}{10x} \tag{7.80}$$

I hope you were able to progress on that problem. Even if you didn't manage to finish it, the important thing is to put in effort; as you learn more and practice more, each problem you solve — even if unsuccessful — contributes to learning.

The sequence of equations below shows the solution. I'll make some remarks below that.

$$x^3y^2 = 5x^4 \tag{7.81}$$

$$(x^3y^2)' = (5x^4)' \tag{7.82}$$

$$3x^2y^2 + 2x^3yy' = 20x^3 \tag{7.83}$$

$$2x^3yy' = 20x^3 - 3x^2y^2 \tag{7.84}$$

$$y' = \frac{20x^3 - 3x^2y^2}{2x^3y} \tag{7.85}$$

$$= \frac{x^2(20x - 3y^2)}{x^2(2xy)} \tag{7.86}$$

Assuming the principal square root to make y a valid function.

$$y = \sqrt{5x} \tag{7.87}$$

$$y' = \frac{20x - 15x}{2x\sqrt{5x}} \tag{7.88}$$

$$= \frac{5}{2\sqrt{5x}} \tag{7.89}$$

$$= \frac{5\sqrt{5x}}{10x},\ x \neq 0 \tag{7.90}$$

The tricky part of this problem is using both the chain rule and the product rule. You need the product rule to differentiate x^3y^2, and then you need the chain rule to differentiate y^2 into $2yy'$.

The calculus part of implicit differentiation is complete at Equation 7.85; thereafter it's just algebraic simplifications. Technically, Equation 7.85 is an acceptable final solution. However, because we could transform the original expression (Equation 7.81) into an explicit function of y, the problem can be further simplified to have only x on the right-hand side.

In the final equation, I removed the radical in the denominator. I don't know why, but many math teachers and textbook authors find radicals in the denominator distasteful.

You'll get to visualize this function and its derivative in Exercise 5.

Finally, notice that the domain of the derivative excludes 0, although that value is permissible in the original function. Thus, $x = 0$ is a critical point that you can spot without any further algebra.

Example 3 As with the previous example, I encourage you to try to solve this one on your own before inspecting my solution below.

$$\cos(xy) = \sin(x)$$

$$y' = \frac{\cos(x) + y\sin(xy)}{-x\sin(xy)}$$

Notice that this problem does not allow for solving for y and substituting, so the derivative itself is a function of y.

Here again you need the chain rule to differentiate $\cos(xy)$, and then the product rule to differentiate xy.

$$\cos(xy) = \sin(x) \tag{7.91}$$

$$\big[\cos(xy)\big]' = \big[\sin(x)\big]' \tag{7.92}$$

$$-\sin(xy)(y + xy') = \cos(x) \tag{7.93}$$

$$-y\sin(xy) - xy'\sin(xy) = \cos(x) \tag{7.94}$$

$$-xy'\sin(xy) = \cos(x) + y\sin(xy) \tag{7.95}$$

$$y' = \frac{\cos(x) + y\sin(xy)}{-x\sin(xy)} \tag{7.96}$$

What do Equations 7.91 and 7.96 mean and what do they look like on a graph? I have absolutely no idea what they mean, although I'm reasonably confident that they have no connection to the physical universe in which you are currently existing.

As for what they look like: visualizing implicit functions is different from visualizing explicit functions, and requires a different approach. I will show you how to visualize implicit functions later, but I want you to work through two more examples first.

Example 4 This problem will help you gain more experience with the chain rule and differentiating natural exponentials. As before, I encourage you to work through the problem before seeing the full solution.

$$e^{x^2+y^2} = x + y$$

$$y' = \frac{1 - 2x\exp(x^2 + y^2)}{-1 + 2y\exp(x^2 + y^2)}$$

You don't need any special tricks to solve this problem; just some diligence and patience. Most of the difficulty with this problem, in my opinion, is keeping track of all the terms.

Here is the fleshed-out sequence of expressions that lead to the derivative.

$$e^{x^2+y^2} = x + y \tag{7.97}$$

$$\left[e^{x^2+y^2}\right]' = (x + y)' \tag{7.98}$$

$$\exp(x^2 + y^2)(2x + 2yy') = 1 + y' \tag{7.99}$$

$$2x\exp(x^2+y^2)+2yy'\exp(x^2+y^2)=1+y' \tag{7.100}$$

$$2yy'\exp(x^2+y^2)-y'=1-2x\exp(x^2+y^2) \tag{7.101}$$

$$y'\left[2y\exp(x^2+y^2)-1\right]=1-2x\exp(x^2+y^2) \tag{7.102}$$

$$y'=\frac{1-2x\exp(x^2+y^2)}{-1+2y\exp(x^2+y^2)} \tag{7.103}$$

Remember that $\exp(u)$ can make e^u more readable when u is a long expression.

Example 5 This final example problem is shorter ($\cos(x^2)=e^y$), and can be solved both implicitly and explicitly.

I encourage you to solve this problem twice: following the implicit differentiation strategy and then using "regular" differentiation after transforming the expression into an explicit function form. Getting the same result twice will give you more practice differentiating, and will confirm your math.

Below is the sequence for implicit differentiation.

$$\cos(x^2)=e^y \tag{7.104}$$

$$\left[\cos(x^2)\right]'=(e^y)' \tag{7.105}$$

$$-2x\sin(x^2)=e^y y' \tag{7.106}$$

$$y'=\frac{-2x\sin(x^2)}{e^y} \tag{7.107}$$

And now for the explicit approach. Remember that the natural log and natural exponent are inverses of each other, so $\ln(e^x)=x$. You need to apply the chain rule twice to differentiate $\ln(\cos(x^2))$.

$$\ln(\cos(x^2))=\ln(e^y) \tag{7.108}$$

$$\ln(\cos(x^2)) = y \tag{7.109}$$

$$y' = \frac{-2x\sin(x^2)}{\cos(x^2)} \tag{7.110}$$

Because this function can be written explicitly with y on one side of the equation and x on the other, the function and its derivative can be visualized using standard `numpy` and `matplotlib` code. In fact, visualizing this function (Figure 7.4) reveals an interesting quirk about computer-implemented calculus.

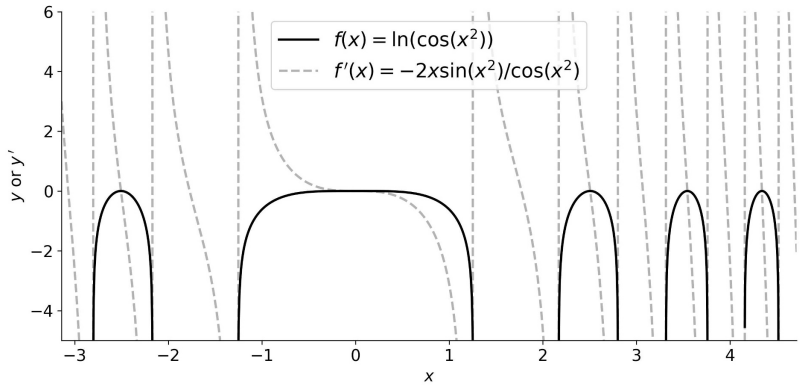


Figure 7.4: Visualization of a function (solid black line) and its derivative (dashed grey line) from implicit differentiation of Example 5.

Is there anything unsettling about this figure? There should be; feel free to take another close look at the figure before reading further.

The issue with that figure is that the derivative is defined where the function is not. In particular, the function is defined only where $\cos(x^2) > 0$. However, the formula of the derivative is defined for all x except where $\cos(x^2) = 0$. If these were two distinct functions, there would be no problem. But the derivative encodes the slope of the tangent at each point in the function; how can the slope of the tangent exist where the function itself does not exist?!?

The answer is that the derivative is undefined where the function

is undefined[2]. In other words, the domain of the derivative must be constrained by the domain of its parent function. But `numpy` doesn't "know" to inherit domain restrictions. That is expert calculus knowledge that you must bring into the problem.

The lesson here is that Python is an amazing *tool* that helps you learn and implement math, but Python doesn't "do the math" for you, at least not in any meaningful conceptual sense. *You* (the human) still need the conceptual knowledge to understand how to use that tool. Python is great for arithmetic, visualization, and some symbolic calculations.

7.4.3 Implicit differentiation in `sympy`

`sympy` has several methods that are designed to work with implicit functions. To start, we can define two symbolic variables in `sympy`. Notice below how I specify that y is a function of x.

```
x = sym.symbols('x')
y = sym.Function('y')(x)
expr = x**2 + 3*x*y + y**2
```

The variable `expr` outputs: $x^2+3xy(x)+y^2(x)$. Thus, `sympy` now knows that $y = y(x)$.

From here, you can differentiate using `sym.diff(expr,x)` as you've done before.

But it's a better idea, especially for expressions that are more challenging to differentiate, to use `sym.idiff(expr,y,x)`. Figure 7.5 shows output from `sympy` for these two options.

Needless to say, the `i` in `sym.idiff` stands for *implicit*.

The two different versions of the derivative seem, at first glance, quite different from each other. Are they really different? Spend

[2] To understand why this must be the case, consider trying to solve the limits definition of the derivative where $f(x)$ and $f(x+h)$ are undefined.

some time pondering how they may be related; my answer is below.

$$f(x) = x^2 + 3xy(x) + y^2(x)$$

f' from `sym.diff`: $3x\frac{d}{dx}y(x) + 2x + 2y(x)\frac{d}{dx}y(x) + 3y(x)$

f' from `sym.idiff`: $-\frac{2x+3y}{3x+2y}$

Figure 7.5: Screenshot of `sympy` output for implicit differentiation options.

The answer is that the two outputs are the same, but expressed in different forms. The output of `sym.diff` is the expression, including notating y' as $\frac{d}{dx}y(x)$, as an equation set to zero (though the "$0 =$ " is not written out). `sym.idiff`, on the other hand, has the derivative set to y' (not written out). If you set the output of `sym.diff` to zero and solve for $\frac{d}{dx}y(x)$, you'll see the same result as the output of `sym.idiff`.

Seemingly different outputs that actually are the same after some algebraic rearranging is fairly common in `sympy`.

By the way, it is not always necessary to define variable `y` as a function of x. Many implicit differentiation problems can be solved by creating two symbolic variables `x` and `y`; in this case, `sympy` "thinks" that `y` is just another variable and the implicit derivative is solved as a partial derivative.

`sympy` also has a dedicated function for visualizing implicit functions: `sym.plot_implicit()`. It's easy to use and I will explain how it works later in the chapter. First, I want to tell you about numerical solutions to implicit differentiation.

7.5 Numerically solving implicit differentiation

There are exciting and amazing things you can do with implicit differentiation, including proving some differentiation rules that I introduced in Chapter 5. I am looking forward to teaching you about that, and I hope you are looking forward to learning about it.

But I want to take a slight tangent to discuss calculating and visualizing implicit equations in `sympy`, `numpy`, and `matplotlib`. It's a different procedure from the calculations and plotting you've done so far. And I believe that learning how to implement and solve implicit functions in Python will help you understand why implicit functions are unusual and powerful.

Why is it different? The way to implement explicit functions in `numpy` is to define an x-axis grid and then evaluate the function definition at each x grid point.

That approach doesn't work for implicit functions, because the function definition includes the variable y. It feels a bit circular: you need to know the value of y in order to determine the value of the function, but y is itself part of the value of the function.

7.5.1 Searching for solutions in `numpy`

Therefore, the way to solve implicit functions is by picking one (x, y) pair and asking: Does this pair solve the equation? If the answer to that question is *yes*, then keep that (x, y) pair in a list; and if the answer is *no*, then disregard that pair.

Here's an example expression: $e^{xy} = x$. Let's consider some (x, y) pairs. In the table below, the left column is a pair of (x, y) values to plug into the expression, and the right column indicates whether that pair provides a valid solution.

$(0,0)$	No
$(1,0)$	Yes
$(2,2)$	No
$(2,\ln(2)/2)$	Yes

The pairs associated with a *Yes* lead to a consistent expression, e.g., $e^{1\times 0}=1$, whereas the pairs associated with a *No* lead to an inconsistent expression, e.g., $e^{2\times 2}=e^4\neq 2$.

After compiling such a table, the next step is to create a 2D plot with dots at each (x,y) pair that has a *Yes* in the "is a valid solution" column.

As you might imagine, this approach is rather tedious, and can be computationally inefficient if implemented as a double for-loop over myriad (x,y) pairs with high resolution.

But this really is the general idea: Test lots of possible solutions, find the ones that make the equation true, and make a scatter plot of the valid solutions.

There is one problem with implementing this computational method: As you've seen several times so far in this book, empirical methods can miss true results if the discretization does not fall *exactly* on the analytic solution (e.g., Exercise 6.1). Therefore, we need to identify solutions that are "close enough" to making the equation true. We lose some perfection in the plot, but we gain an understanding of the function that would otherwise be out of reach to all but the most gifted and visually inclined mathematicians.

With that in mind, here's how to find solutions in `numpy`. Start by creating a 2D grid of (x,y) pairs:

```
plotbnd = 2*np.pi
x_vals = np.linspace(-plotbnd,plotbnd,1000)
y_vals = np.linspace(-plotbnd,plotbnd,1000)
X,Y    = np.meshgrid(x_vals,y_vals)
```

Variables `X` and `Y` are 1000×1000 matrices of x and y coordinates. These are the grids (the xy-pairs) that we will plug into the equation.

The next step is to create a Python function that can evaluate the math function at hand. To illustrate the procedure, I'll use the implicitly defined function $\cos(xy) = \sin(x)$. We need to re-express this equation as

$$z = \cos(xy) - \sin(x) \tag{7.111}$$

Here's what the `sympy` code for that looks like:

```
x,y = sym.symbols('x,y',real=True)
expr = sym.cos(x*y)-sym.sin(x)
expr_lam = sym.lambdify((x,y),expr)
```

And now the goal is to find the (x, y) pairs that make $Z = 0$. Well, that's the theory. In practice, we find the (x, y) pairs that make $Z \approx 0$, which is translated as $|Z| < \eta$ for some small η.

```
Z = expr_lam(X,Y)
eta = .01
approxZero = np.where(np.abs(Z) < eta)
```

Where variable `eta` corresponds to η.

Finally, we extract the coordinates and make a scatter plot.

```
x_coords = X[approxZero]
y_coords = Y[approxZero]
plt.plot(x_coords,y_coords,'k.')
```

And that's the idea! In Figure 7.6 the black dots correspond to solutions to the implicit expression and the gray plus-signs correspond to solutions to its derivative (I omitted the code for

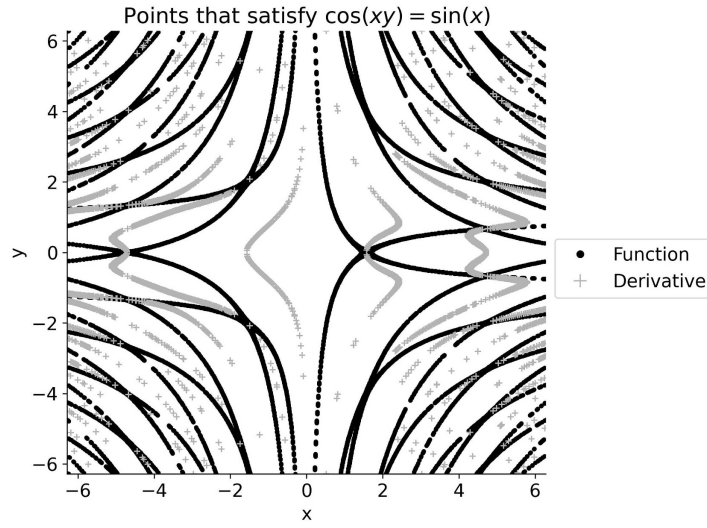


Figure 7.6: Visualization of solutions to the expression and its derivative from Example 3.

the derivative, but the grid search procedure is identical to that used for the original function).

The visualization reveals that the expression is not a *function* according to the formal definition of a function that I discussed in Chapter 2. Also note that the lined-up dots are actually just discrete samples from a line. Using a higher grid resolution will create more dots that are closer to each other. And, of course, as the resolution increases, $\Delta x \to dx$ and we'd have the analytic solution.

The higher the resolution, the more computationally difficult the problem becomes. For example, a 1000×1000 matrix requires evaluating the equation 1,000,000 times. Of course, with vectorization the computational burden isn't so bad, but you can see how scaling becomes a problem, especially for complicated or multivariable functions that take longer for a computer to calculate.

On the other hand, re-running the code using a 100×100 matrix produces a result that largely fails to reveal the patterns (not shown here but it's easy for you to explore in the online code). Thus, if the resolution is too coarse you'll miss the pattern of

solutions, and if the resolution is too fine you'll waste compute power and time.

7.5.2 Implicit plots in `sympy`

You can avoid a lot of the `numpy` and `matplotlib` code by plotting implicit functions directly in `sympy` (Figure 7.7).

```
expr = sym.cos(x*y)-sym.sin(x)
sym.plot_implicit(expr)
```

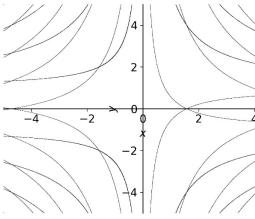


Figure 7.7: Implicit plotting in `sympy`.

I hope you agree that while the `numpy` approach reveals the mechanism of evaluating and visualizing implicit functions, the `sympy` built-in method is easier and faster to evaluate.

That said, using the `numpy` grid-search approach gives you more control over the resolution of the grid, the threshold for considering a solution to be "good enough," and detailed control over the aesthetics of the plots. You'll see examples of the benefits of `numpy` and `matplotlib` for implicit plotting in the exercises.

7.6 Implicit differentiation proofs (log, exp, power)

Implicit differentiation opens new doors for understanding and solving calculus problems — including several proofs of differentiation rules. I think you will be happily surprised at how easy it is to prove some of the claims in Chapter 5 when using implicit differentiation.

7.6.1 Natural log

I claimed that $[\ln(x)]' = 1/x$. That claim can be proven using implicit differentiation. In fact, I'd like *you* to prove this claim.

There is one trick that you'll need to get started, which is to take the natural exponential of both sides of the function definition:

$$y = \ln(x)$$

$$e^y = e^{\ln(x)}$$

$$e^y = x$$

From here, you can apply the implicit differentiation procedure and solve for y'. When you get the answer, or if you get stuck, you can consult my solution below.

$$(e^y)' = (x)' \tag{7.112}$$

$$e^y y' = 1 \tag{7.113}$$

$$y' = \frac{1}{e^y} \tag{7.114}$$

$$y' = \frac{1}{e^{\ln(x)}} \tag{7.115}$$

$$y' = \frac{1}{x} \tag{7.116}$$

The "trick" at the end was seeing that $e^y = x$, by definition.

7.6.2 Natural exponential

Please try to prove that $(e^x)' = e^x$ before looking at my solution below.

The trick to getting started here is similar to the trick in the previous proof. In this case, you take the natural log of both sides of the equation.

$$y = e^x \tag{7.117}$$

$$\ln(y) = \ln(e^x) \tag{7.118}$$

$$\ln(y) = x \tag{7.119}$$

$$[\ln(y)]' = (x)' \tag{7.120}$$

$$\frac{y'}{y} = 1 \tag{7.121}$$

$$y' = y \tag{7.122}$$

$$= e^x \tag{7.123}$$

As with the natural log, the y in the penultimate equation simply equals e^x by definition.

7.6.3 Power rule

Finally, the power rule. Remember that the power rule is

$$[x^n]' = nx^{(n-1)}$$

In Chapter 5 I demonstrated this rule using some examples of positive integers; now you are going to prove that it works for any exponent n. To make progress with this proof, you will need to use one of the properties of logs: $\ln(x^n) = n\ln(x)$.

As always, I encourage you to see how far you can get on your own, and then compare against my solution below when you finish or if you get stuck. When you're finished, consider what happens when $n = 0$.

$$y = x^n \tag{7.124}$$

$$\ln(y) = \ln(x^n) \tag{7.125}$$

$$\ln(y) = n\ln(x) \tag{7.126}$$

$$[\ln(y)]' = [n\ln(x)]' \tag{7.127}$$

$$\frac{1}{y}\, y' = n\,\frac{1}{x} \tag{7.128}$$

$$y' = n\,\frac{y}{x} \tag{7.129}$$

$$= n\,\frac{x^n}{x} = nx^{n-1} \tag{7.130}$$

This proof reveals why the power rule does not apply for $n = 0$: Equation 7.126 becomes $\ln(y) = 0$, which means $y = 1$ for all positive values of x. $y = x^0 = 1$ is a constant function, and its derivative is zero.

Re-proving the proofs I encourage you to take out a fresh piece of paper and re-derive the three proofs in this section from scratch. Seeing a proof and then working through it yourself is a great way to boost your calculus thinking skills.

7.7 L'Hôpital's rule

Variously spelled L'Hôpital's rule, L'Hopital's rule, L'Hospital's rule, or Bernoulli's rule.

L'Hôpital's rule is a trick for finding the limit when the other limit-finding strategies lead to an indeterminate form $0/0$ or $\pm\infty/\infty$.

In words: If the limit as $x \to c$ of the ratio of two functions is indeterminate, then the limit might be found by taking the ratio of the derivatives of those two functions. In math:

$$\textbf{if } \lim_{x\to c}\frac{f(x)}{g(x)} = \frac{0}{0} \text{ or } \frac{\pm\infty}{\pm\infty}$$

$$\textbf{then } \lim_{x\to c}\frac{f(x)}{g(x)} = \lim_{x\to c}\frac{f'(x)}{g'(x)}$$

"L'Hôpital" is pronounced "loh-pee-tahl"

To preemptively avoid confusion: L'Hôpital's rule tells us to take the ratio of derivatives, which is different from the derivative of a ratio. That is,

$$\frac{f'(x)}{g'(x)} \neq \left[\frac{f(x)}{g(x)}\right]'$$

The right-hand-side is solved by the quotient rule.

L'Hôpital's rule doesn't always work. I'll start with a few examples where it does work, and then I'll explain the conditions that make it valid.

Example 1 We're going to use L'Hôpital's rule to solve two trig limits problems that I introduced in Section 4.8.

$$\lim_{\theta \to 0} \frac{\cos(\theta) - 1}{\theta} = \frac{0}{0} \tag{7.131}$$

$$\lim_{\theta \to 0} \frac{[\cos(\theta) - 1]'}{[\theta]'} = \lim_{\theta \to 0} \frac{-\sin(\theta)}{1} \tag{7.132}$$

$$\lim_{\theta \to 0} \frac{-\sin(\theta)}{1} = \frac{0}{1} = 0 \tag{7.133}$$

Equation 7.131 is our limit problem. Recall that in Chapter 4 I used some intricate algebra to solve this problem. Armed with L'Hôpital's rule, I simply take the derivative of the numerator and denominator, and find that their ratio is $0/1$. That was really easy!

Figure 7.8 shows the situation.

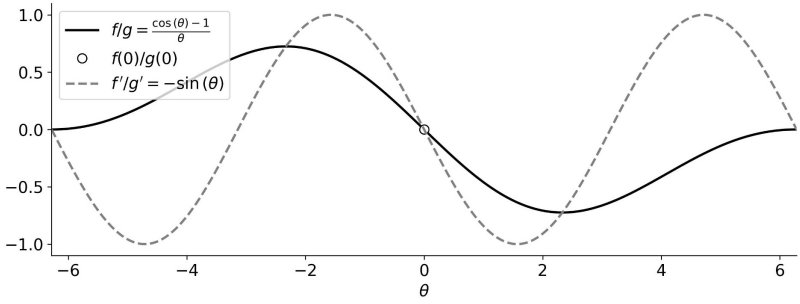


Figure 7.8: L'Hôpital's rule in action. The ratio of functions is undefined at $\theta = 0$ (white circle) but the ratio of their derivatives is defined and its limit can be easily determined.

Now for the other trig limit. That's the one for which I provided

only a geometric intuition. Using L'Hôpital's rule, we can prove this limit quickly and easily:

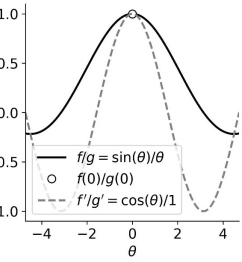


Figure 7.9: The other trig limit from Example 1.

$$\lim_{\theta\to 0}\frac{\sin(\theta)}{\theta} = \frac{0}{0} \tag{7.134}$$

$$\lim_{\theta\to 0}\frac{[\sin(\theta)]'}{[\theta]'} = \lim_{\theta\to 0}\frac{\cos(\theta)}{1} \tag{7.135}$$

$$\lim_{\theta\to 0}\frac{\cos(\theta)}{1} = \frac{1}{1} = 1 \tag{7.136}$$

Example 2 Here's the problem; see if you can solve it on your own before reading my solution below.

$$\lim_{x\to\infty}\frac{2x^2+4x-7}{7x^2-13x+2}$$

The plug-in method fails because you end up with ∞/∞. I guess you tried L'Hôpital's rule but again got an answer of ∞/∞.

The "trick" to solving this limit problem is to apply L'Hôpital's rule a second time when the ratio of the first derivatives still produces an indeterminate result.

$$\lim_{x\to\infty}\frac{2x^2+4x-7}{7x^2-13x+2} = \frac{\infty}{\infty} \tag{7.137}$$

$$\lim_{x\to\infty}\frac{4x+4}{14x-13} = \frac{\infty}{\infty} \tag{7.138}$$

$$\lim_{x\to\infty}\frac{4}{14} = \frac{4}{14} = \frac{2}{7} \tag{7.139}$$

It is no coincidence that the limit ended up being the same as the coefficients of the highest-power x terms. When the limit goes to ∞, the squared terms grow so much faster than the linear terms, that the latter become negligible. Of course, when x is "close" to zero ("close" relative to ∞), all three terms in $2x^2+4x-7$

meaningfully contribute to the output of the function. But when x is some unfathomably gigantic number, then $2x^2+4x-7 \approx 2x^2$.

The function and its first two derivatives are shown in Figure 7.10. Don't be distracted by the explosions between $x = 0$ and $x = 2$; the example problem here asked about the limit as $x \to \infty$.

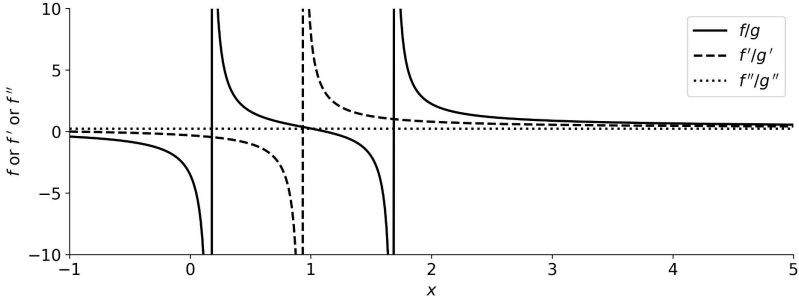


Figure 7.10: The function and two derivatives from Example 2.

The plot certain does appear consistent with the limit asymptotic to a finite value close to zero, but the exact answer can be calculated only from the math, not from visual inspection.

Example 3 Here's the problem:

$$\lim_{x \to 0^+} \left[x^2 \ln(x) \right] \tag{7.140}$$

Plugging in 0 gives the bizarre-looking result of $0 \times (-\infty)$. Thus, this limit is a competition between $x^2 \to 0$ and $\ln(x) \to -\infty$, and the result of the limit is the answer of who gets there first. What's your guess? Does your intuition tell you that the result is 0 or $-\infty$?

More importantly, this problem initially looks like it wouldn't be appropriate for L'Hôpital's rule: The expression is *not* a ratio of two functions (you might be tempted to consider the function $g(x) = 1$ as the denominator, but that would give $g' = 0$, so that's not a viable approach).

Therefore, you need to figure out a way to re-express that expression as the ratio of two functions. The solution is written below,

so if you'd like to try to figure this one out on your own, don't look down ;)

The solution is to rewrite x^2 as $1/x^{-2}$. That changes the problem to:

$$\lim_{x\to 0^+} \frac{\ln(x)}{x^{-2}} = \frac{-\infty}{\infty}$$

From here, the problem is relatively straightforward to solve.

$$\lim_{x\to 0^+} \frac{\ln(x)}{x^{-2}} = \frac{-\infty}{\infty} \tag{7.141}$$

$$\lim_{x\to 0^+} \left[\frac{x^{-1}}{-2x^{-3}} = \frac{-1}{2}x^3x^{-1} = \frac{-1}{2}x^2 \right] = 0 \tag{7.142}$$

Are you surprised at the result? I remember being surprised when I first saw this problem. I had (naively, and for no good reason) imagined that $\ln(x)$ would plummet down to $-\infty$ faster than x^2 would taper down to zero. The graph of this function is interesting to explore, and you'll get to do that in Exercise 8.

7.7.1 Conditions for L'Hôpital's rule

L'Hôpital's rule is applicable if the following conditions are met:

1. The expression is a ratio of functions.
2. The $\lim(f/g)$ is indeterminate.
3. f' and g' both exist.
4. $\lim(f'/g')$ exists and $g' \neq 0$.

With that in mind, I'm going to show a few examples where L'Hôpital's rule fails.

Example 4 While looking through these examples, please look back at the list of conditions to determine *why* each example fails.

$$\lim_{x\to 3} \frac{x+1}{3x+1} = \frac{4}{10} = \frac{2}{5} \tag{7.143}$$

$$\lim_{x\to 3} \frac{(x+1)'}{(3x+1)'} = \frac{1}{3} \neq \frac{2}{5} \tag{7.144}$$

Ignoring for the moment that the plug-in method works just fine, the question is: Why did L'Hôpital's rule give the incorrect answer?

The reason is that condition #2 was violated: The limit of the function is a finite number, not indeterminate.

Example 5 For this problem, first determine why L'Hôpital's rule fails, and then determine the true limit.

$$\lim_{\theta\to\infty} \frac{\sin(\theta)+\theta}{\theta} \tag{7.145}$$

$$\lim_{\theta\to\infty} \frac{(\sin(\theta)+\theta)'}{(\theta)'} \tag{7.146}$$

$$\lim_{\theta\to\infty} \frac{\cos(\theta)+1}{1} = \frac{\text{d.n.e.}}{1} \tag{7.147}$$

Why does L'Hôpital's rule fail here? The problem was with condition #3: The limit of f' as $\theta \to \infty$ does not exist: It's an oscillating discontinuity.

So what is the answer to Equation 7.145? The answer is 1, and there are three ways to explain this. First, the intuitive explanation: The limit of $\sin(\theta)$ as $\theta \to \infty$ is indeed undefined, but its range is bound by ± 1. On the other hand, ± 1 is insignificantly tiny as θ gets huge, and therefore $\sin(\theta)$ can be ignored. That leaves $\theta/\theta = 1$, so the limit is 1.

Second, the geometric explanation: Figure 7.11 shows that Equation 7.145 tapers to $y = 1$ while Equation 7.147 oscillates indefinitely.

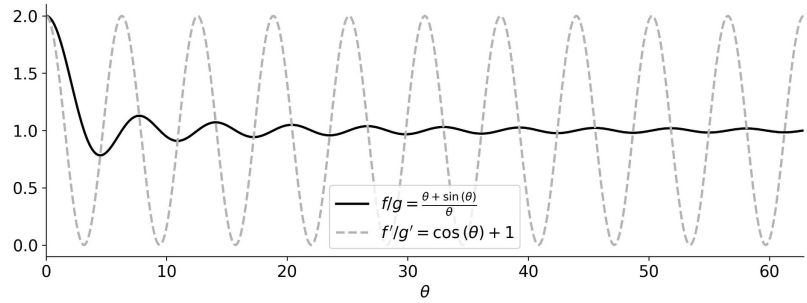


Figure 7.11: Example failure case of L'Hôpital's rule. The ratio of functions converges to $y = 1$ as x increases, but the ratio of their derivatives offers only an oscillating discontinuity.

Third, a more rigorous mathematical explanation can be obtained by manipulating the equation to make the problem clearer:

$$\lim_{\theta \to \infty} \frac{\sin(\theta) + \theta}{\theta} \tag{7.148}$$

$$\lim_{\theta \to \infty} \frac{\sin(\theta) + \theta}{\theta} \cdot \frac{1/\theta}{1/\theta} \tag{7.149}$$

$$\lim_{\theta \to \infty} \frac{\sin(\theta)/\theta + \theta/\theta}{\theta/\theta} \tag{7.150}$$

$$\lim_{\theta \to \infty} \frac{\sin(\theta)/\theta}{\theta/\theta} + \frac{\theta/\theta}{\theta/\theta} \tag{7.151}$$

$$\lim_{\theta \to \infty} \frac{\sin(\theta)}{\theta} + 1 \;=\; 0 + 1 \tag{7.152}$$

7.8 Exercises

7.1. This and the following several exercises are focused on using `sympy` to confirm the rules introduced in this chapter. This will boost both your `sympy` and your differentiation skills.

The goal here is to demonstrate that $(fg)' \neq f'g'$. Your task is simple: Write `sympy` code to reproduce Figure 7.12. (Don't stress about the column formatting; my online code solution creates only one column at a time.)

A) $h = f \times g$	**B)** $h = f + g$
$f(x) = (x-1)^3$	$f(x) = (x-1)^3$
$g(x) = x^2$	$g(x) = x^2$
$h(x) = x^2(x-1)^3$	$h(x) = x^2 + (x-1)^3$
$f'(x) = 3(x-1)^2$	$f'(x) = 3(x-1)^2$
$g'(x) = 2x$	$g'(x) = 2x$
$h'(x) = 3x^2(x-1)^2 + 2x(x-1)^3$	$h'(x) = 2x + 3(x-1)^2$
$f'g' = 6x(x-1)^2$	$f' + g' = 2x + 3(x-1)^2$

Figure 7.12: Screenshot for Exercise 1.

7.2. Now to confirm the product rule by comparing `sympy`'s differentiation of a composite function to the "manual" implementation of the product rule. The goal is to produce output like in Figure 7.13. To produce the final equation, you need to differentiate $f(x)$ and $g(x)$ separately, then multiply (that is, implement Equation 7.1 directly).

$$f(x) = x^2$$

$$g(x) = \cos(x)$$

$$f \times g = x^2 \cos(x)$$

$$[f \times g]' = -x^2 \sin(x) + 2x \cos(x)$$

$$f'g + fg' = -x^2 \sin(x) + 2x \cos(x)$$

Figure 7.13: Screenshot for Exercise 2. If the product rule is true, then the last two lines should be equal.

7.3. Same as the previous exercise, but now for the quotient rule.

$$\frac{f}{g} = \frac{x^2}{\cos(x)}$$

$$\left[\frac{f}{g}\right]' = \frac{x^2 \sin(x)}{\cos^2(x)} + \frac{2x}{\cos(x)}$$

$$\frac{f'g - fg'}{g^2} = \frac{x^2 \sin(x) + 2x \cos(x)}{\cos^2(x)}$$

Figure 7.14: Screenshot for Exercise 3. If the quotient rule is true, then the last two lines should be equal.

Hmm... the last two lines appear to differ. Was there a mistake somewhere? Or perhaps the quotient rule doesn't work?!? Sometimes when results appear to differ in `sympy`, you can use `sym.simplify` to see equivalence. But that just complicates the output in this case. For example, `sym.simplify` on the final line produces the following output:

$$\frac{x(x\tan(x) + 2)}{\cos(x)}$$

So now we have three seemingly different expressions for the derivative of f/g, which *should* all be identical.

The last part of this exercise is for you to write these expressions on paper, and do whatever algebraic manipulations, expansions, and simplifications are necessary to show that they really are the same, despite their typographical

distinctions.

7.4. Last exercise in this series is for the chain rule. Your goal is to reproduce Figure 7.15 using `sympy`. You'll need to use the `.subs()` method to solve this problem. Similar to the previous exercise, the two pairs of "identical" solutions may initially look different, but some algebraic rearrangements will show that they are equal.

$$f(x) = 3x^2$$

$$g(x) = x^3 + \log(x)$$

$$h(x) = \cos(x)$$

$$f(g(x)) = 3\left(x^3 + \log(x)\right)^2$$

$$[f(g(x))]' = 3 \cdot \left(6x^2 + \frac{2}{x}\right)\left(x^3 + \log(x)\right)$$

$$f(g)'\,g' = \left(3x^2 + \frac{1}{x}\right)\left(6x^3 + 6\log(x)\right)$$

$$f(g(h(x))) = 3\left(\log(\cos(x)) + \cos^3(x)\right)^2$$

$$[f(g(h(x)))]' = 3\left(-6\sin(x)\cos^2(x) - \frac{2\sin(x)}{\cos(x)}\right)\left(\log(\cos(x)) + \cos^3(x)\right)$$

$$f(g(h))'\,g(h)'\,h' = -\left(6\log(\cos(x)) + 6\cos^3(x)\right)\left(3\cos^2(x) + \frac{1}{\cos(x)}\right)\sin(x)$$

Figure 7.15: Screenshot for Exercise 4.

7.5. Now for a few exercises on implicit differentiation. We start by exploring Example 2 from earlier in this chapter (Equation 7.79, reprinted below). To begin, use `sympy` to convert the implicit form of the function to an explicit form:

$$x^3y^2 - 5x^4 \tag{7.153}$$

$$y = \pm\sqrt{5x} \tag{7.154}$$

To be clear, the goal is to figure out how to get `sympy` to arrive at Equation 7.154 from 7.153, not that you manually type in the explicit form.

Equation 7.154 is formally not a function: It fails the "vertical line test" because of the $\pm$ operator. Therefore, split the solution for y into two functions.

Now use `sympy` to differentiate three functions: Implicit differentiation on Equation 7.153 (using `sym.idiff`) and explicit differentiation on Equation 7.154 split into two. Your results should match my screenshot in Figure 7.16

mplicit form: $y' = \frac{10}{y} - \frac{3y}{2x}$

xplicit form (1 of 2): $f'(x) = -\frac{\sqrt{5}}{2\sqrt{x}}$

xplicit form (2 of 2): $f'(x) = \frac{\sqrt{5}}{2\sqrt{x}}$

Figure 7.16: `sympy` output for Exercise 5.

Finally, plot both signed functions, and their corresponding derivatives, using `matplotlib`, as in Figure 7.17.

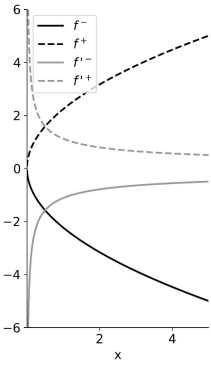


Figure 7.17: Second part of Exercise 5.

7.6. This exercise provides another demonstration that math formulas and algorithms can sometimes produce valid numerical solutions even if the results cannot be meaningfully interpreted.

Let's return to Example 4 of implicit differentiation (Equation 7.97, reprinted below). Implement this equation in `sympy`, setting y to be a function of x.

$$e^{x^2+y^2} = x + y$$

Try to get `sympy` to get transform this equation to an explicit function. Then find the derivative.

Next, use `numpy` to find solutions to the equation and its derivative using the grid search method shown in this chapter, and use `matplotlib` to plot them as in Figure 7.18.

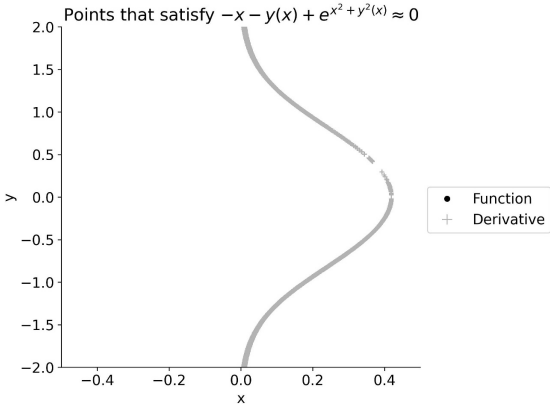


Figure 7.18: Visualization for Exercise 6. Why are there no black dots?

There are no black dots because there are no (x, y) pairs that satisfy the equation. Intuitively, the exponential term $e^{x^2+y^2}$ grows so much faster than the linear combination $x + y$, that they never intersect. On the other hand, there are many solutions that make the derivative valid! What does it mean for there to be a derivative of an expression that has no solutions?

7.7. It is perfectly valid to rewrite the expression from the previous exercise as $Z = e^{x^2+y^2} - x - y$. This produces a Z value for each (x, y) pair. The absence of solutions from the previous exercise simply indicates that there are no (x, y) pairs for which $Z = 0$.

The implicit plots in this chapter were created by finding (x, y) pairs where $Z \approx 0$ (operationalized as $|Z| < \eta$). You can also inspect the full map of Z values as a heatmap. The heatmaps for the equation and its derivative from the previous exercise are shown in Figure 7.19.

Your goal in this exercise is to create those heatmaps. I transformed the results to $-\ln(|Z|)$ for visibility; $-log$ transformations are common in machine learning and optimization because it expands the range of very small numbers such as probabilities. The upshot is that in the images below, larger (whiter) pixel grayscale values correspond to

smaller Z values.

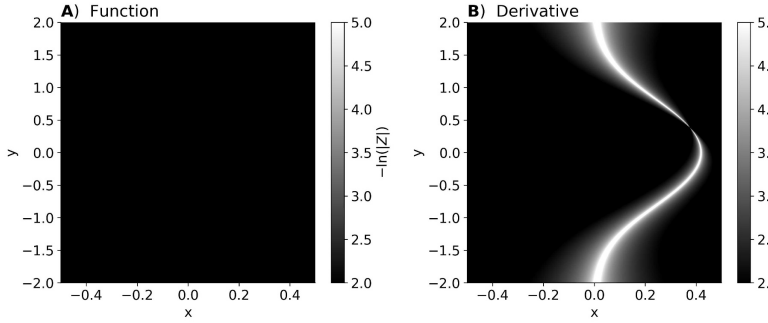


Figure 7.19: Visualization for Exercise 7.

7.8. Let's return to Example 3 from L'Hôpital's rule ($\lim_{x\to 0^+}\left[x^2 \ln$ Implement that expression in `sympy` and show that `sympy` can accurately determine the limit. Next, create separate variables for $f(x)$ and $g(x)$, and show that $(f/g)' \neq f'/g'$. I'm not showing the results here, but it's in the online code.

Finally, create a plot of the function to build intuition for why the limit is zero. Solve for $f' = 0$ in `sympy` to find the function minimum, and plot it on the graph as in Figure 7.20.

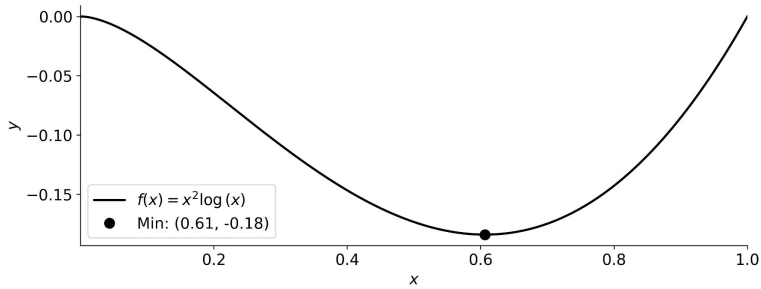


Figure 7.20: Visualization for Exercise 8.

Before even looking at the graph, you should know that the non-zero critical point of this function corresponds to a minimum. How can you intuit that? My answer is in the next paragraph.

The answer is that the natural log is negative for $x < 1$ and

increases slowly as x gets further from zero; on the other hand, x^2 has a minimum at $x = 0$ and increases quickly thereafter. For $x > 1$, the function is the product of two positive and increasingly large numbers. (Indeed, if you redraw the plot up to $x = 10$, the initial negative dip isn't even visible at that y-axis scaling.) So the function $x^2 \ln(x)$ will necessarily have a minimum between 0 and 1, and we know that the limit as $x \to 0 = 0$. Therefore, any non-zero critical point must be a minimum.

7.9. Here's a question: Both x^2 and e^x grow to ∞ as $x \to \infty$, but which one grows *faster*? Imagine a race between those functions; which one do you put your money on?

Perhaps you have an intuition about which function grows faster, but how can you provide evidence from which to draw a firm conclusion? You could take an empirical approach: calculate x^2 and e^x for some really big numbers and see which function is bigger.

A more rigorous solution is to use L'Hôpital's rule. If you'd like to come up with a solution on your own, then please stop reading now! The rest of this exercise will guide you through the solution.

The "trick" here is to take the ratio of these expressions, and then consider that the function that increases faster to ∞ will dominate that ratio. More precisely:

$$f(x) = e^x \tag{7.155}$$

$$g(x) = x^2 \tag{7.156}$$

$$\frac{f(x)}{g(x)} = \frac{e^x}{x^2} \tag{7.157}$$

$$\lim_{x \to \infty} \left[\frac{e^x}{x^2} \right] = ? \tag{7.158}$$

The solution to Equation 7.158 is the answer: If the limit is ∞, then e^x climbs faster; and if the limit is 0, then x^2 climbs faster.

Obviously, the limit is ∞/∞, but you can use L'Hôpital's rule. After solving this problem analytically, implement these two functions in a plot to visually confirm the analytic result. Also, what would change if you swapped $f(x)$ and $g(x)$?

To build suspense, I won't reveal the answer here, but I have some discussions about this problem in the online code.

Chapter 8

Differentiation theorems

The theorems discussed in this chapter are cornerstones of differential calculus. The lay foundations for advanced topics in calculus and number theory, and they provide practical tools to analyze and solve real-world problems in optimization, machine-learning, engineering, financial modeling, and computational physics.

Fortunately, the theorems presented in this chapter are fairly sensible and intuitive — and they provide excellent opportunities for exercises focused on developing calculus-inspired algorithms and function visualizations.

8.1 Differentiability implies continuity

Here's an important theorem: **If a function is differentiable at a point, then it is also continuous at that point**. Be mindful that this theorem is about a particular *point*; a function can be differentiable and therefore continuous at some points while having discontinuities at other points.

The concept is illustrated in Figure 8.1.

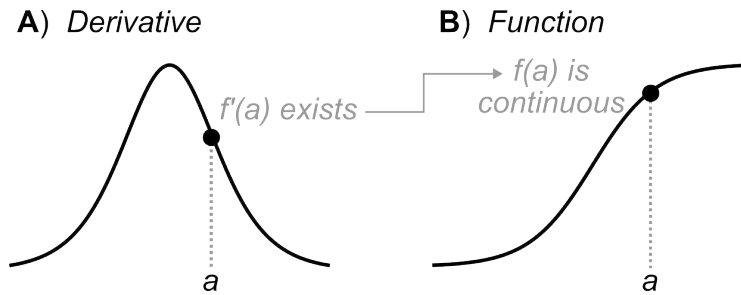


Figure 8.1: Illustration of the theorem that if a function's derivative exists at $x = a$ (left), then that function is continuous at $x = a$ (right).

8.1.1 Proof

The idea of the proof shown below is to begin by stating the continuity assumption (Equation 8.1 below), rearranging that equation

using properties of limits, and finding no contradictions.

See if you can narrate the proof by following the equations below, and then you can confirm or correct your interpretation in the text below that. The proof relies on two assumptions: That the derivative exists at $f(a)$ and that it is a finite number; we do not consider continuity or differentiability at $x = \pm\infty$.

$$\lim_{x\to a} f(x) = f(a) \tag{8.1}$$

$$\lim_{x\to a} [f(x) - f(a)] = 0 \tag{8.2}$$

$$\lim_{x\to a} \left[\frac{f(x) - f(a)}{x - a} \cdot (x - a)\right] = 0 \tag{8.3}$$

$$\lim_{x\to a} \frac{f(x) - f(a)}{x - a} \cdot \lim_{x\to a}(x - a) = 0 \tag{8.4}$$

$$f'(x) \cdot 0 = 0 \tag{8.5}$$

Equation 8.1 states that function $f(x)$ is continuous at $x = a$. In Equation 8.2, $f(a)$ is subtracted from both sides of the equation (note that $\lim_{x\to a} f(a) = f(a)$). Equation 8.3 contains the one trick to this proof: Multiply and divide by $(x - a)$. That allows us to include the definition of the derivative in the first term of Equation 8.3 (the format looks different from the h expression that I've used previously, but a moment's inspection will reveal that this is an equivalent definition of the derivative). Equation 8.4 breaks the limit of a product into the product of two limits, the second of which is zero using the plug-in method to obtain $a - a$.

It might not be immediately obvious that Equation 8.5 completes the proof. The idea is that we started from two assumptions — the derivative exists at $x = a$ and the function is continuous at $f(a)$ — and ended with a valid expression (0=0). Had we started from the same two assumptions and ended with an invalid expression like 1=0, then the assumptions would have been wrong.

8.1.2 Does continuity imply differentiability?

Sometimes in mathematics, you can turn statements around: For example, if $3 + 4 = 7$ then $7 = 3 + 4$. In logic, this would be called "affirming the consequent" (if A then B; B, therefore A). It's actually often a logical fallacy because B could exist for reasons other than A. The question here is whether continuity and differentiability have bidirectional causality.

In other words, if you know that a function is continuous at a certain point, does that mean it's also differentiable at that point?

The answer is *no*, and I discussed several examples in earlier chapters of discontinuities where the function is continuous but the derivative does not exist. A formal proof with equations is not necessary; we need only one counter-example to disprove the statement. The function $f(x) = |x|$ is a common counter-example: At $x = 0$ the function is defined and continuous, but has no derivative.

Conclusions: If a function is differentiable at point $x = a$, then it is continuous at $x = a$. But if a function is continuous at $x = a$, then we cannot trivially know whether it is differentiable at $x = a$.

Why is this rule important? Differentiability implying continuity is foundational to several other calculus theorems and proofs, including some you'll learn in this chapter (e.g., intermediate value and mean value theorems), as well as Taylor series and solving some classes of differential equations.

This theorem also has real-world applications in physics, engineering, finance, optimization, and data science. One example: the gradient descent algorithm that underlies modern deep learning is based on optimizing *differentiable* loss functions, and the *implication* that the underlying function is continuous guarantees that learning will converge towards better solutions.

8.2 Intermediate value theorem

The intermediate value theorem states that a function that is continuous in the interval $[a, b]$ has all values between $f(a)$ and $f(b)$. This is illustrated in Figure 8.2 using the function $f(x) = x^2$. In this example, the intermediate value theorem states that all numbers between $f(1) = 1$ and $f(3) = 9$ can be produced by the function for at least one value of x in the closed interval $[1, 3]$.

Continuity refers to one particular point, so this condition can be interpreted as the function being continuous at all points in the interval.

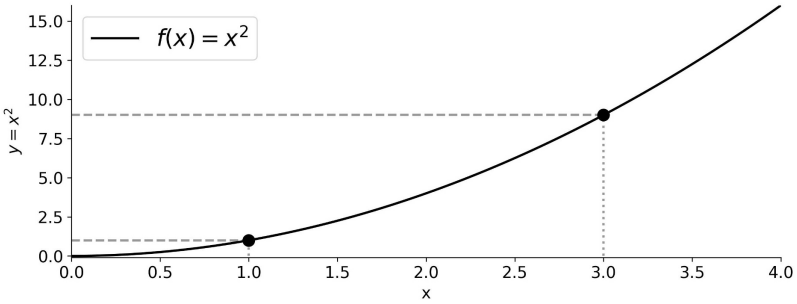


Figure 8.2: Illustration of the intermediate value theorem.

I believe this theorem is intuitive: If you want to travel along a function from one point to another point, then you must pass through all the intermediate points along the way.

The clause *a function that is continuous in the interval* is key to the theorem: Discontinuities will break the rule. That is clearly the case for piecewise functions: imagine a function that has one piece defined for $x < 1$ and another piece defined for $x > 2$ with no pieces defined in between; there could be numbers in between $f(a)$ and $f(b)$ that are missing from the function output (Figure 8.3).

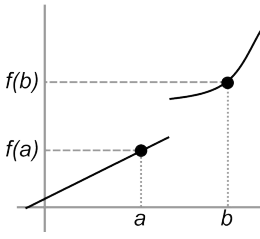


Figure 8.3: An example of where the intermediate value theorem does not apply.

Applications of the intermediate value theorem The intermediate value theorem is used to prove other rules and theorems, so it provides a foundation for other topics.

A direct application is determining whether a polynomial equation has solutions in a certain interval. The "roots" of a polynomial

are the values of x that satisfy $f(x) = 0$. How do you know if there are real-valued roots of a particular polynomial?

Consider Figure 8.4. Without solving any equations, I know with 100% certainty that $f_1(x)$ has at least one real-valued root while $f_2(x)$ has no real-valued roots[1]. How do I know that?

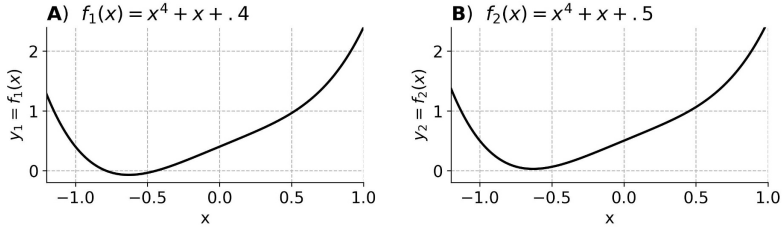


Figure 8.4: Application of the intermediate value theorem.

From visual inspection of the graphs, $y_1 = f_1$ dips below zero while $y_2 = f_2$ does not. If f_1 has at least one negative value and at least one positive value — and if it is continuous in that interval — then the intermediate value theorem tells us that there is at least one x such that $f_1(x) = 0$ (in fact, there are two such values of x in this example). On the other hand, y_2 has no non-positive values, which means that it never passes through zero; ergo, it has no real-valued roots.

8.3 Rolle's theorem

Rolle's theorem is geometrically intuitive — perhaps almost trivially obvious. Have a look at Figure 8.5 and see if you can guess the theorem before reading the text.

Here's the theorem: For some function $f(x)$, if $f(a) = f(c)$, then there is at least one point b such that $f(b)' = 0$. In other words, there is at least one point in between $x = a$ and $x = c$ where the function has a flat tangent line.

[1] The Fundamental Theorem of Algebra tells us that a 4^{th} order polynomial has 4 roots, but they may be complex-valued.

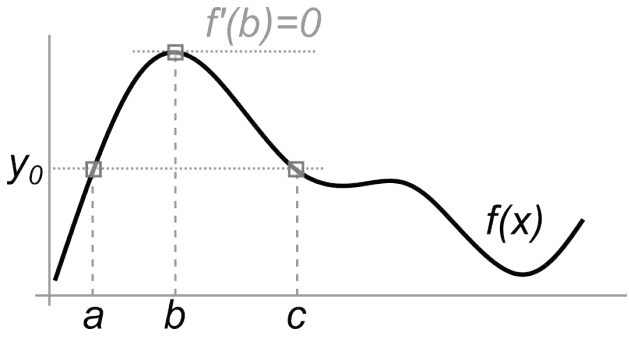


Figure 8.5: Illustration of Rolle's theorem.

The intuition behind Rolle's theorem is that a function can have the same value at $x = a$ and at $x = c$ only if one of three things happen: (**1**) the function goes up and then comes back down; (**2**) the function goes down and then comes back up; or (**3**) the function remains at a constant value. All of those possibilities involve the derivative being zero for at least one point.

The following statement formalizes the theorem.

$$\text{if } f(a) = f(c), \text{ then } \exists\, a < b < c \;\; s.t. \;\; f'(b) = 0 \tag{8.6}$$

The $\exists$ symbol means "there exists," and *s.t.* means "such that." It turns out that there is an assumption for Rolle's theorem to be valid. Can you figure out that assumption? I'll write it explicitly after walking you through an example problem.

8.3.1 Example problem

A typical Rolle's theorem problem is to find the point b such that $f'(b) = 0$ given a function and two points a and c. Here's an example problem:

$$f(x) = -\frac{x^2}{3} + 2x - 3, \quad x \in \left\{ 3 \pm \sqrt{3} \right\} \tag{8.7}$$

$x = 3 \pm \sqrt{3}$ are our two points a and c. The first step in a problem like this is to confirm that $f(a) = f(c)$:

$$f(a) = -\frac{\left(3 - \sqrt{3}\right)^2}{3} + 2\left(3 - \sqrt{3}\right) - 3 \tag{8.8}$$

$$= -\frac{9 + 3 - 6\sqrt{3}}{3} + 6 - 2\sqrt{3} - 3 \tag{8.9}$$

$$= -3 - 1 + 2\sqrt{3} + 6 - 2\sqrt{3} - 3 \tag{8.10}$$

$$= -1 \tag{8.11}$$

And for now for $f(c)$:

$$f(c) = -\frac{\left(3 + \sqrt{3}\right)^2}{3} + 2\left(3 + \sqrt{3}\right) - 3 \tag{8.12}$$

$$= -\frac{9 + 3 + 6\sqrt{3}}{3} + 6 + 2\sqrt{3} - 3 \tag{8.13}$$

$$= -3 - 1 - 2\sqrt{3} + 6 + 2\sqrt{3} - 3 \tag{8.14}$$

$$= -1 \tag{8.15}$$

Now we determine $f'(x)$, set it to zero, and solve for x.

$$f'(x) = -\frac{2}{3}x + 2 \tag{8.16}$$

$$0 = -\frac{2}{3}x + 2 \tag{8.17}$$

$$\frac{2}{3}x = 2 \tag{8.18}$$

$$x = 3 \tag{8.19}$$

Why didn't I just solve for $f'(x) = 0$ right away? It's useful to confirm that $f(a) = f(c)$, just in case your math teacher includes some trick questions. But it's also possible that there are several critical points in the function; you only want the critical points that are between a and c (refer back to Figure 8.5).

Visual confirmation is, of course, always a great idea (Figure 8.6).

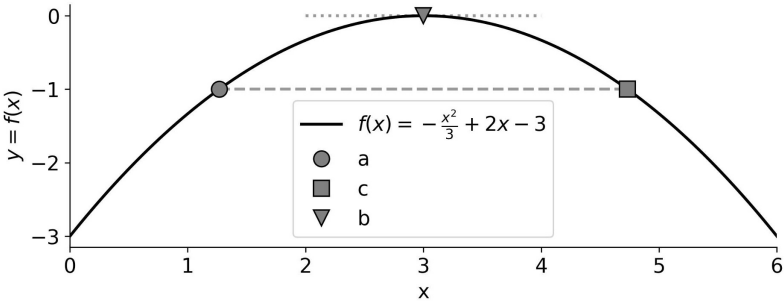


Figure 8.6: Example of Rolle's theorem.

8.3.2 Conditions for Rolle's theorem

Did you take my suggestion to think of the conditions that make Rolle's theorem valid?

Here are the three conditions; I hope these are consistent with your ideas:

1. $f(x)$ must be continuous in $[a, c]$
2. $f(x)$ must be differentiable in (a, c)
3. $f(a) = f(c)$

I believe these conditions are self-evident. If the function is not continuous between a and c, then there need not be any points in between that have $f' = 0$. For example, imagine a piecewise function that is undefined between a and c; or the function $1/x$ where $a = -1$ and $c = +1$.

That third "condition" is literally part of the definition of the theorem, so it is, admittedly, a little strange to list it as a condition. But it does help link to the *mean value theorem*, which is the topic of the next section.

Why does the condition on continuity have *in*clusive bounds while the condition of differentiability have *ex*clusive bounds?

The reason is that we need to know the values of the function

at $x = a$ and $x = c$, but we're not interested in the derivative at *exactly* points a and c; instead, the goal is to find some point in between but not including a and c. Remember that for the derivative to exist, it must have a two-sided limit; allowing for open intervals provides the freedom of a and c being boundaries of the function domain.

Applications of Rolle's theorem Rolle's theorem has many applications in calculus and in the real world. It's used in the mean value theorem (next section) and to find or estimate roots of equations. It's used in optimization problems to find minima within certain bounds. It's used in physics to help determine the maximum height of a projectile, in economics to find the date of maximum GDP after a boom-bust cycle, in climate science to identify transitions between seasons, and to predict when physical stresses on structural support beams will return to points of stability.

In other words, if you study any system that changes over time, you will likely encounter Rolle's theorem.

8.4 Mean value theorem

Figure 8.7 illustrates the mean value theorem. It looks fairly similar to the illustration of Rolle's theorem. As with that theorem, I would like you to guess the mean value theorem purely based on visual inspection of the figure, and then you can confirm or correct your hypothesis after the text below.

The mean value theorem states that there is at least one point in between $x = a$ and $x = c$ where the derivative equals the slope of the secant line connecting the two points $(a, f(a))$ and $(c, f(c))$. Here's how it looks in equation form:

$$f'(b) = \frac{f(c) - f(a)}{c - a}, \quad a < b < c \tag{8.20}$$

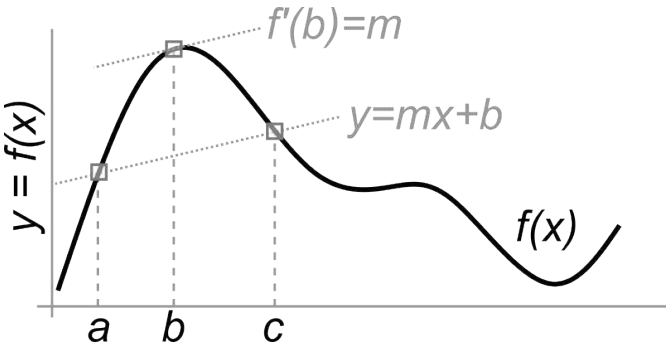


Figure 8.7: Illustration of the mean value theorem.

As you might have guessed, Rolle's theorem is just a special case of the mean value theorem, in which a and c are chosen such that their slope is zero.

Example Mean value theorem problems are similar to Rolle's theorem problems: Given a function and an interval, find a point in the interval in which the derivative equals the slope of the secant line.

Here's the example problem; your goal is to find the point b that satisfies the mean value theorem, before reading my solution below.

$$f(x) = 2x^2 - 3x + 1, \quad x \in \{-1, 2\} \tag{8.21}$$

The sequence of equations below provides the answer, and the text below that explains the logic.

$$f'(x) = 4x - 3 \tag{8.22}$$

$$f'(b) = \frac{f(2) - f(-1)}{2 - (-1)} \tag{8.23}$$

The right-hand-side of Equation 8.23 is $\Delta y / \Delta x$.

$$4b - 3 = \frac{(2[2]^2 - 3[2] + 1) - (2[-1]^2 - 3[-1] + 1)}{3} \tag{8.24}$$

$$4b - 3 = \frac{(8 - 6 + 1) - (2 + 3 + 1)}{3} \tag{8.25}$$

$$4b - 3 = -1 \tag{8.26}$$

$$4b = 2 \tag{8.27}$$

$$b = 2/4 = 1/2 \tag{8.28}$$

The first step is to find the derivative of the function (Equation 8.22). Then we set that equal to the slope of the secant line connecting $f(-1)$ and $f(2)$ (Equation 8.23), expand the terms (Equation 8.24), simplify (Equations 8.25-8.28), and arrive at the solution of $b = 1/2$.

The main source of confusion and mistakes in mean value theorem problems is confusing the derivative (left-hand side) with the function (right-hand side).

Figure 8.8 visualizes this example. Writing an algorithm to solve mean-value theorem problems and create this plot is part of the exercises.

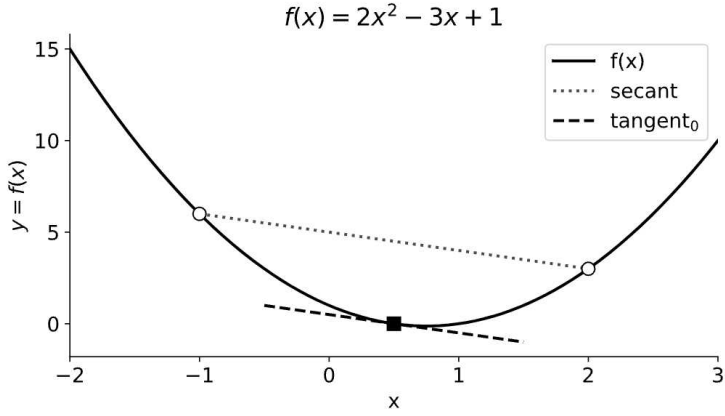


Figure 8.8: Example of the mean value theorem. Code to produce this Figure is part of Exercise 8.2.

Conditions for the mean value theorem The mean value theorem is valid under the same first two conditions as for Rolle's theorem: continuity and differentiability between $x = a$ and $x = c$.

8.4.1 What is the "mean value"?

Why is it called the *mean value theorem*? You might be familiar with the concept of the "mean" (that is, the arithmetic average) from statistics, which involves adding data values and dividing by the number of values; that algorithm does not appear in any of the equations in this section.

Instead, the term "mean" here comes from one of the implications of the Fundamental Theorem of Calculus, which is that the average rate of change over an interval — here quantified as the slope of the secant line from $(a, f(a))$ to $(c, f(c))$ — equals the average of the instantaneous rates of change along that same interval. In other words, the *global* slope between two bounds equals the average of all *local* slopes between those same bounds. And the average of all local slopes is the average of the derivatives.

Thus, the mean value theorem tells us that that *average derivative* value appears at least once in between $f(a)$ and $f(c)$.

8.5 The Fundamental Theorem of Calculus

You'd think that something called the "Fundamental Theorem of Calculus" is important enough, and relevant enough, to be in a chapter on calculus theorems.

However, the Fundamental Theorem is largely about integration, which I haven't yet introduced in this book. I'm going to explain the theorem conceptually here, because it is (**1**) important to know and (**2**) conceptually straightforward. If you don't understand all of the notation and concepts below, then don't worry about it; you will understand these in detail later in the book. For now, just focus on the idea.

8.5.1 Fundamental Theorem of Calculus, part 1 (FTC-1)

Remember from Chapter 5 that the derivative is, conceptually, like subtraction. In this way, integration (indicated using the symbol $\int dx$) is, also conceptually, like summation.

FTC-1 states that differentiation is the opposite of integration, and is formalized as the following equivalent expressions:

$$\frac{d}{dx}\int f(x)\,dx = f(x) \tag{8.29}$$

$$\int f(x)'\,dx = f(x) \tag{8.30}$$

Equation 8.29 states that the derivative of the integral of a function is that function itself, and Equation 8.30 states that the integral of a derivative is the function[2]. In other words, integration and differentiation are inverse operators of each other[3], analogous to how adding and subtracting are inverse operators of each other.

8.5.2 Fundamental theorem of calculus, part 2 (FTC-2)

FTC-2 is unrelated to differentiation, so I will mention it here briefly, and you'll gain an in-depth understanding of it in Chapter 12.

FTC-2 states that the area under a curve equals the difference of definite integrals of the bounds of the curve. In other words, to calculate the area under $f(x)$ in Figure 8.9, evaluate and subtract the integral of the function at $x = b$ from that at $x = a$.

Formally, it's expressed like this:

$$\int_a^b f(x)\,dx = F(b) - F(a) \tag{8.31}$$

[2]There is a "constant of integration" that I'm ignoring here but that adds important nuance; you'll learn about this in later chapters.

[3]Again, there are important nuances to this claim, but this is the basic idea.

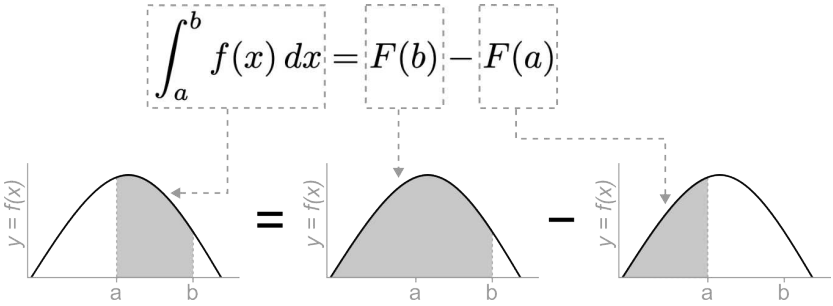


Figure 8.9: The FTC-2 states that the area under a curve between two points (shaded region in the left-most plot) is the difference of the integral of the function at those two points. You'll learn a lot more about this theorem in Chapters 12 and 13.

where $\int_a^b$ indicates the shaded area shown in the left-most plot in Figure 8.9, and $F(x)$ is the indefinite integral of $f(x)$. I realize this is a lot of information that might not make sense at this point in the book. If you're confused or unsure, then that's excellent, because it means you can look forward to understanding exactly what the FTC-2 means, where it comes from, and how to use it, in later chapters.

8.6 Exercises

8.1. The goal of this exercise is to write a Python function that implements an algorithm to solve mean value theorem problems. Several subsequent exercises depend on the accuracy of this exercise, so please make sure you solve this one correctly, and check against my online solution if you're unsure of yours.

Create a Python function that takes three inputs: a `sympy` expression corresponding to $f(x)$, and interval bounds $x = a$ and $x = c$. I called my function `solveMVT`. Inside that function, implement the following steps:

Step 1: Determine the derivative using `sympy`.

Step 2: Find $f(a)$ and $f(c)$.

Step 3: Create a symbolic expression for the right-hand-side of Equation 8.20.

Step 4: Solve for $x = b$ using `sym.solve`. Remember that solving equations in `sympy` can be implemented by putting all terms on the left-hand-side, and then solving for 0.

Step 5: Return the elements of b that are between a and c (this excludes solutions outside the interval).

Test your Python function on $f(x) = 2x^2 - 3x + 1$ in the interval $x \in \{-1, 2\}$. This was the equation I used earlier in the chapter, and your Python function should output $b = .5$.

8.2. The code you will write in this exercise is also used for the next several exercises. The goal here is to write a Python function that takes the same three inputs as your `solveMVT` function, call the `solveMVT` function to obtain `b`, and then generate a plot that visualizes the answer. In fact, Figure

8.8 is the output of this function.

The end goal of the previous and this exercise is to be able to evaluate the following code in order to solve and visualize mean value theorem problems:
`MVTandPlot(expr,a,c)`
where `expr` is a `sympy` expression object corresponding to a math function, and `a` and `c` are the interval bounds.

8.3. Now to explore the function. Use the expressions and intervals shown in Figure 8.10. The interval for $\sin(x)$ is $[0, 1.8\pi]$.

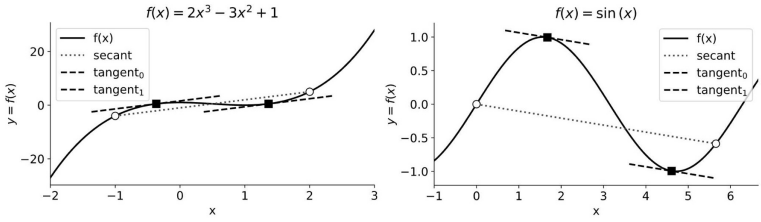


Figure 8.10: Results for Exercise 3.

Now for another function in the interval $x \in [-2, 1]$:

$$f(x) = \frac{2x}{(2x^2 + 1)^2} \tag{8.32}$$

This math function will cause the Python function to crash (assuming you haven't explicitly put any restrictions on the field of symbolic variable `x`). Your goal is to figure out why. There's a hint in the footnote[4]. My answer to this error is in the online code.

8.4. Same as above but for the function $f(x) = \cos(x^2)$ in the interval $x \in [-\pi/2, 6\pi/7]$.

Spoiler alert! The Python function will crash again, but for a different reason than the problem in the previous exercise. Keep investigating the source of the problem until you get the following error message:

[4]Isolate **Step 4** from Exercise 1.

```
No algorithms are implemented to solve equation
    -2*x*sin(x**2) - 316212178523559/10000000000000
```

Translated into math, the problem is that `sympy` cannot analytically solve the key equation $-2x\sin(x^2) = \frac{22}{243}$.

Lucky for you, you know how to find estimate solutions using empirical methods. Segue to the next exercise!

8.5. This exercise integrates what you learned in Chapter 6 about empirically finding critical points through grid-search, with what you learned in this chapter about the mean value theorem.

The goal is simple: Write code to solve a mean-value theorem problem using the equation and interval in Exercise 4, and to produce Figures 8.11 and 8.12.

Be careful with this exercise! The goal is simple but it involves some careful thinking. In particular, be mindful of the equation you need to find solutions for.

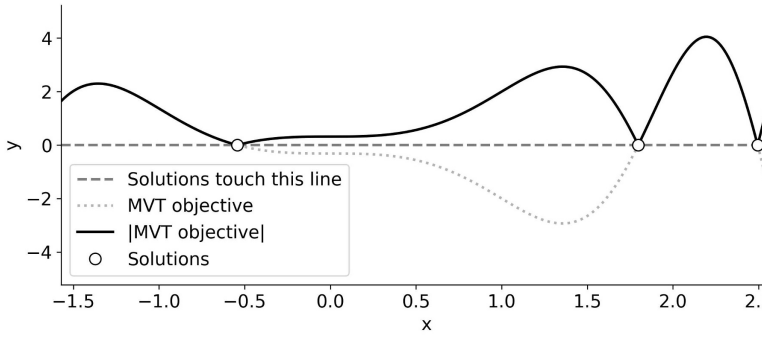


Figure 8.11: Intermediate results for Exercise 5, showing the MVT objective function and its approximate roots identified using the grid-search method.

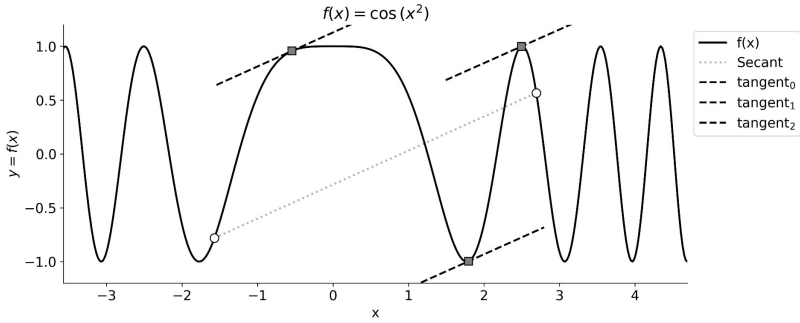


Figure 8.12: Final results for Exercise 5.

8.6. This exercise is about Rolle's theorem. Follow the procedure for solving Rolle's theorem problems in Section 8.3.1 (page 307) using the function and interval:

$$f(x) = |\sin(x)|, \quad x \in [\pi/2, 3\pi/2]$$

You'll get stuck with `sympy` again. So you'll need to proceed the "old-fashioned way:" using paper and pencil. My answers are in the sequence below.

$$f(a) = |\sin(\pi/2)| = 1 \tag{8.33}$$

$$f(c) = |\sin(3\pi/2)| = 1 \tag{8.34}$$

$$f'(x) = \text{sgn}\big(\sin(x)\big)\cos(x) \tag{8.35}$$

$$0 = \text{sgn}\big(\sin(x)\big)\cos(x) \tag{8.36}$$

$$x = 0, \pi/2, \pi, 3\pi/2, 2\pi, ... \tag{8.37}$$

The first two equations confirms that $f(a) = f(c)$. Then we use the chain rule to find the derivative, set it to zero, and solve for x.

There is an infinite number of solutions, but they're all outside the interval, except for $x = \pi$. But is that the solution? Have a look at the graph of the function (Figure 8.13).

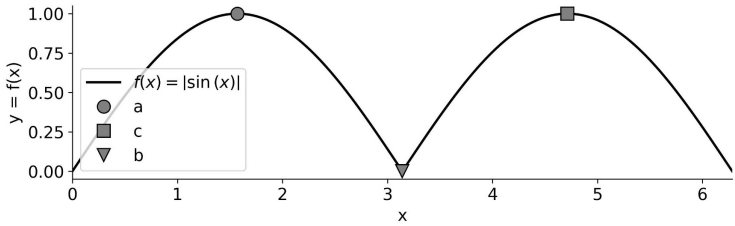


Figure 8.13: Visualization for Exercise 6.

The "solution" is a cusp! Which means the derivative does not exist at that point. Thus, there is no solution to Rolle's theorem in this example, because the derivative is not defined over the entire interval, thus violating Condition 2.

If you've read this far without any coding, then please write code to confirm, visualize, and explore what I've written here.

8.7. What happens when you take iterative cosines or sines? That is, what happens when you calculate $\sin(\sin(\sin(\theta)))$? Let's find out ;)

The end result of this exercise will be code that produces Figure 8.14. In a for-loop over ten iterations, take the cosine of repeated cosines, and the sine of repeated sines. That is, the first iteration is $y = \cos(\theta)$, the second iteration is $y = \cos(\cos(\theta))$, the third iteration is $y = \cos(\cos(\cos(\theta)$ and so on.

Plot each iterated cosine and sine using different line colors, as in panels A and B. Panel C shows the cosines on the x-axis by the sines on the y-axis.

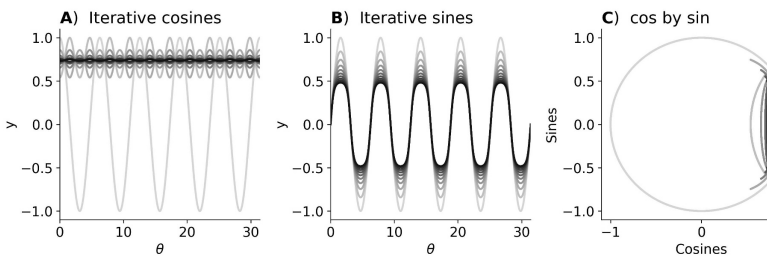


Figure 8.14: Visualization for Exercise 7. Darker lines correspond to later iterations in the for-loop (more embedded trig functions).

There are no deep mathematical theorems to learn here; just an opportunity to review and practice your math-coding and visualization skills.

Chapter 9

Applications of differentiation

It would be an impossible task to list all the applications of differential calculus that humans have hitherto discovered: By the time that list would be written, new items would need to be added.

The purpose of this chapter is to introduce a few of the important applications that are not specific to individual disciplines. If you study climate science, for example, you will learn myriad applications of calculus for modeling weather patterns.

Even if your planned application of calculus is not represented in this chapter, I still recommend going through it. The problems here will help you develop your general thinking skills of calculating and using derivatives.

9.1 Linear approximations

Nonlinear functions can be difficult and time-consuming to calculate, and may be prone to numerical inaccuracies. Linear functions, on the other hand, tend to be computationally easy, fast, and accurate. Therefore, the idea of linear approximation is to use a linear function to "guess" the value of a nonlinear function at some point. The mechanism is depicted in Figure 9.1.

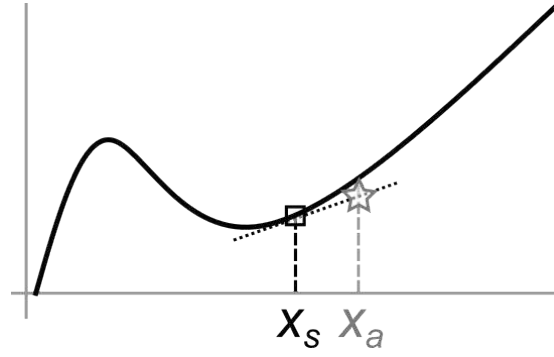


Figure 9.1: Graphical depiction of linear approximation: Given a known starting point (x_s, y_s) and the derivative of the function at that point, approximate the value of the function at (x_a, y_a).

To motivate the formula, let's imagine you want to solve $f(x) =$

$\sqrt{x}+2x$ for $x = 4.37$. Now, for $x = 4$ this is a simple problem you can do in your head: $f(4) = 10$. But I'm going to venture a guess that you cannot calculate $\sqrt{4.37}$ in your head. So instead, we'll use linear approximation to *guess* a reasonable value of $f(4.37)$.

Equation 9.1 is the key formula for linear approximation. I will expand on the meaning and interpretation of this formula later, but first I want to show an example so you can see how easy and accurate it can be.

$$f(x_a) \approx f'(x_s)(x_a - x_s) + f(x_s) \tag{9.1}$$

x_a is the point for which we want an approximate value (in this case, 4.37) and x_s is a starting value that is close to x_a and is easy to calculate. Let's pick $x_s = 4$. To implement Equation 9.1, we need the function and its derivative.

$$f(x) = \sqrt{x} + 2x \tag{9.2}$$

$$f'(x) = \frac{1}{2}x^{-1/2} + 2 \;=\; \frac{1}{2\sqrt{x}} + 2 \tag{9.3}$$

And now we can proceed:

$$f(x_a) \approx f'(x_s)(x_a - x_s) + f(x_s) \tag{9.4}$$

$$\approx \left[\frac{1}{2\sqrt{4}} + 2\right](4.37 - 4) + \left(\sqrt{4} + 2 \cdot 4\right) \tag{9.5}$$

$$\approx \frac{9}{4}(.37) + 2 + 8 \tag{9.6}$$

$$\approx 10.8325 \tag{9.7}$$

$$f(x_a) = 10.8304544960 \tag{9.8}$$

Equation 9.7 is the approximation and Equation 9.8 is the function value calculated in Python. I'd say that the approximation was really good! The remarkable thing is that this approximation can be done without a calculator; the multiplication in Equation 9.6 could have been further approximated using $.37 \approx \frac{1}{3}$.

9.1.1 Intuition of linear approximations

What's the logic of Equation 9.1? It is literally the equation for a straight line ($y = mx + b$) but using different terms. Consider the annotated version of the same formula:

$$\underbrace{f(x_a)}_{\substack{\text{approx.}\\(y)}} \approx \underbrace{f'(x_s)}_{\substack{\text{slope}\\(m)}} \underbrace{(x_a - x_s)}_{\substack{\Delta x\\(x)}} + \underbrace{f(x_s)}_{\substack{\text{intercept}\\(b)}} \tag{9.9}$$

You can write either "$f(x_a) \approx$" to indicate that the value is approximate, or you can write "$L(x_a) =$" to indicate that the linear approximation (L) is equal to a particular value.

Same formula, different order The linear approximation formula is also written using terms in different orders:

$$L(x_a) = f(x_s) + f'(x_s)(x_a - x_s) \tag{9.10}$$

$$L(x_a) = f(x_s) + (x_a - x_s)f'(x_s) \tag{9.11}$$

$$f(x + \Delta x) \approx f(x) + \Delta x \frac{df}{dx}, \quad x_a = x + \Delta x \tag{9.12}$$

I prefer writing the approximation formula in a way that clarifies the link between linear approximations and straight lines. But Equation 9.12 is also insightful: Watch what happens when I move $f(x)$ to the left-hand side and divide by Δx:

$$f(x + \Delta x) \approx f(x) + \Delta x \frac{df}{dx} \tag{9.12}$$

$$f(x + \Delta x) - f(x) \approx \Delta x \frac{df}{dx} \tag{9.13}$$

$$\frac{f(x + \Delta x) - f(x)}{\Delta x} \approx \frac{df}{dx} \tag{9.14}$$

$$\frac{\Delta f}{\Delta x} \approx \frac{df}{dx} \tag{9.15}$$

Equation 9.15 says that the formula for the slope of the secant line is approximately equal to the function derivative (as long as Δx is small relative to the changes in the function around x). Adding a limit as $\Delta x \to 0$ gives us the definition of the derivative.

9.1.2 Approximation accuracy

The linear approximation in the example problem above was very close to the value calculated by Python. Let's see what it looks like in a graph (Figure 9.2).

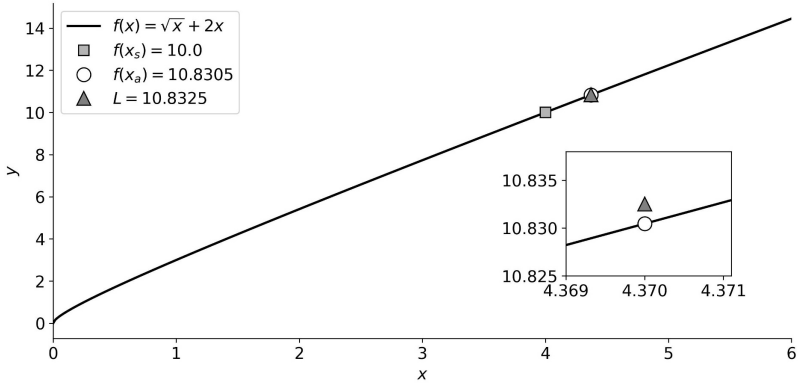


Figure 9.2: Visualization of the linear approximation example. Gray square is the starting point, dark gray triangle is the linear approximation, and white circle is the true function value. Their discrepancy can be seen in the inset plot (note the axis limits).

The graph makes the accuracy of the linear approximation less impressive: the function itself appears approximately linear in this interval, so it's not so surprising that the linear approximation was accurate.

In general, the accuracy of the linear approximation is qualitatively related to two features:

1. The Δx term, in other words, how close x_s is to x_a. The closer the approximating value, the more accurate the approximation. As an extreme example, you would not expect $x_s = 10^9$ to provide an accurate approximation of $f(4.37)$.

2. The nature of the function's nonlinearities around x_a. If the function is truly linear, then the approximation will be perfect. If the function is highly nonlinear with drastic changes around x_a, then a linear approximation will likely be less accurate.

Of course, these two features interact with each other: the closer x_a and x_s, the more robust the approximation to twists and turns.

How much error is acceptable? "Error" here can be defined as $\epsilon = L(x_a) - f(x_a)$. The better the linear approximation, the smaller the error.

The acceptable magnitude of error depends entirely on the application. If you are linearly approximating a function for a math exam, then the error probably doesn't matter as long as you demonstrate knowledge of the procedure. But if the approximation is to ensure that heat dispersion in an engine is kept within safety limits, then you would hope that the engineers minimized the approximation errors.

The sign of the error (that is, whether the linear approximation under- or over-estimates the true value) depends both on the sign of Δx and on whether the function bends upwards or downwards at x_a.

9.1.3 A second example

Let's do another example (and there will be more in the exercises). Try this on your own before reading my solution — and try solving it without a calculator. Then use Python to calculate the answer and compare it to the accuracy of your approximation.

$$f(\zeta) = |\sin(\zeta)| \tag{9.16}$$

$$x_a = 5\pi/8 \tag{9.17}$$

$$x_s = \pi/2 \tag{9.18}$$

Below is my solution, and I'll provide some explanation below that.

$$f'(\zeta) = \operatorname{sgn}[\sin(\zeta)]\cos(\zeta) \tag{9.19}$$

$$\begin{aligned} L(x_a) &= f'(x_s)(x_a - x_s) + f(x_s) \\ &= \operatorname{sgn}[\sin(\pi/2)]\cos(\pi/2)(5\pi/8 - \pi/2) + |\sin(\pi/2)| \end{aligned} \tag{9.20}$$

$$= 1 \cdot 0 \cdot () + 1 \tag{9.21}$$

$$= 1$$

$$f(5\pi/8) = 0.923879532511287$$

Notice that in Equation 9.21, I didn't even bother calculating the third term; as soon as you see that there is a zero in the multiplication, the rest of the terms don't matter.

What do you think of the approximation? Is $\epsilon \approx .08$ a "big" or "small" error? Let's have a look at the graph of this function (Figure 9.3).

On the one hand, a qualitative evaluation of the error term being "big" or "small" depends on the goal and application. But in this case, the linear approximation of *any* x_a using $x_s = \pi/2$ will *always* be $L(x_a) = 1$, because the derivative of the function — and therefore the slope of the approximating line — is zero at $x_s = \pi/2$.

From these examples and discussions, I think you can appreciate that linear approximations can be powerful but must be done carefully. Linear approximations, like many applications of math-

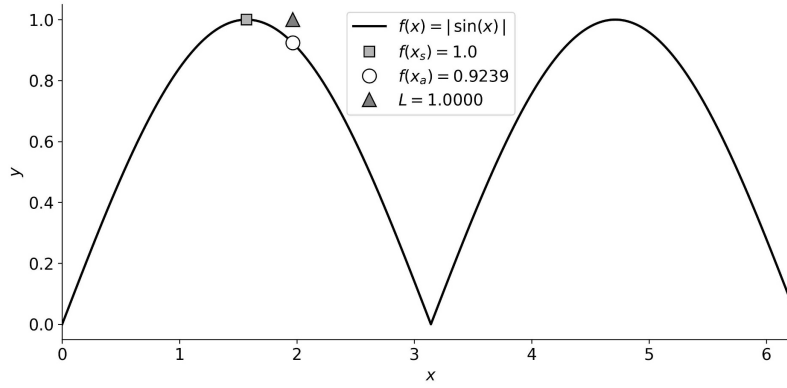


Figure 9.3: Visualization of the second linear approximation example. Gray square is the starting point, dark gray triangle is the linear approximation, and white circle is the true function value.

ematics, requires some expertise and checking to be done appropriately and accurately.

Applications Linear approximations were more important in the past than they are in the present: until a few decades ago, a lot of mathematical calculations were done by hand, and so being able to hand-calculate reasonably accurate approximations using simple formulae used to be extremely important.

But linear approximations are still used even with modern computing power. Linear models simplify the analysis and interpretation, increase numerical stability, and decrease computation time. Pharmacokinetics, for example (how the concentration of medicine in the body changes over time), are determined by highly complex nonlinear functions that depend on the medication chemical structure, patient weight and age, and individual metabolism and genetics, and yet linear approximations are used to simplify the dynamics and prescribe appropriate doses.

9.2 Newton's method for finding roots

As I've mentioned earlier in the book, the roots of an equation are the values of x that solve $f(x) = 0$. In some equations, the roots are easy to find, e.g., the roots of $x^2 = 4$ are $x \pm 2$. But many other equations have roots that are difficult to solve analytically, and instead are better estimated empirically.

In Chapter 6, you learned how to estimate roots of an equation using the grid-search method. Grid-search is great in theory — and excellent for visualizing — but can become prohibitively computationally expensive for functions that are complicated or multivariable.

Newton's method is an alternative (and pre-computer) empirical technique to approximate roots of equations. It's an iterative method, meaning you start from an estimate and then keep improving that estimate. And it requires only a small number of calculations compared to the hundreds, millions, or trillions of calculations that could be required for grid-search.

Consider the equation $3x^3 + 2x = 7\pi$. The goal is to find the value(s) of x that satisfy that equality. This can equivalently be expressed by setting the equation to zero:

$$3x^3 + 2x - 7\pi = 0 \tag{9.22}$$

The Fundamental Theorem of Algebra dictates that there are three solutions (because this is a third-order polynomial), but some of them may be complex-valued. In fact, the graph of the function (Figure 9.4) indicates that there is one real-valued solution (that is, the x-axis value where $f(x) = 0$) at around $x \approx 1.8$.

Newton's method involves starting from an initial guess of the root (x_0 in the figure), and then iteratively improving that guess until it converges to a solution (I used three iterations in this case to arrive at x_3).

The initial guess was $x_0 = 1$. Interestingly, the first iteration to update the guess from x_0 to x_1 actually went too far past the solution, but the next few iterations got closer to the true solution.

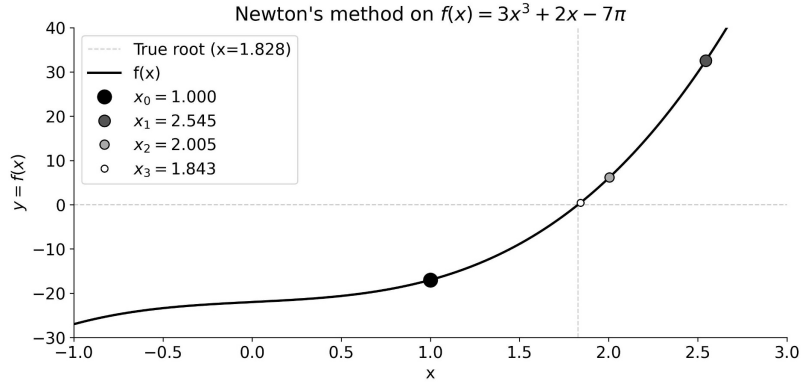


Figure 9.4: Example of using Newton's method to find the root of a polynomial. Each x_n is the estimate of the root after the n^{th} iteration through the algorithm.

I hope you are impressed by the figure, and curious to learn how it works! Keep reading...

9.2.1 Newton's method: algorithm

In broad strokes, the idea of Newton's method is to pick some reasonable starting guess (e.g., $x_0 = 1$), calculate the tangent line of the function at $f(x_0)$, find where the tangent line crosses $y = 0$, and then call that x_1. Then repeat: find where the tangent line of the function at $f(x_1)$ crosses $y = 0$ to obtain x_2. Hopefully, each iteration gets closer to the true root, and convergence can be very fast (although there are sources of difficulties that I'll discuss later).

Try to understand the method conceptually by studying Figure 9.5. I'll provide a written description below.

Here's the algorithm:

1. Start from an initial estimate on the function. In Figure 9.5 this is indicated as $p_0 = (x_0, f(x_0))$.
2. Define the tangent line, which is the line that touches $f(x_0)$ and has slope $f'(x_0)$.

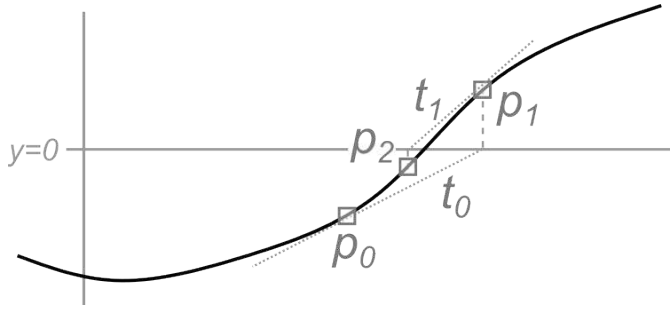


Figure 9.5: Illustration of Newton's Method for finding roots. The root is the point where the function (black curve) intersects $y = 0$. p_n refers to points and t_n refers to tangent lines.

3. Identify the value of x where the tangent line intersects $y = 0$. This is the new estimate of the root. In Figure 9.5 this is indicated as the transition from the dotted line t_0 to the vertical dashed line underneath p_1.

4. Keep repeating Steps 2-3 until some stopping criteria is reached. Criteria could be either a set number of iterations or minimal change from x_n to x_{n+1}.

The crux of the method can be described using one equation:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{9.23}$$

You will have the opportunity to implement this algorithm in Exercises 4-7.

9.2.2 Challenges and failures

You've probably already been thinking about important considerations and implementational details, including (**1**) how to choose a starting value x_0, (**2**) how to deal with functions or derivatives that are undefined or have discontinuities at or near x_n, (**3**) how many iterations to implement, and (**4**) how to identify convergence failures and whether they can be remedied (e.g., restarting with a better initial estimate) or whether the function has no real-valued roots.

These issues concern numerical implementations that are unrelated to the calculus concepts, so I won't go into detail about them here. But there has been considerable research and development on these issues that you can read about online.

I do want to highlight a few failure scenarios.

The method can fail if $f'(x_n) = 0$. But even a derivative that is not exactly zero but very small can be problematic if it causes the update to be huge.

The method can also fail if the initial guess was a poor choice. In the examples shown here, you can eyeball a good initial guess through visual inspection or a quick calculation with some numbers, but good initial guesses can be difficult in complicated or multivariable functions.

A third source of difficulty happens when the derivative is unknown. Remember that not all functions have an analytically solvable derivative. There is an adjustment to Newton's method called the "secant method" that replaces the tangent line with a secant line, and that allows for local empirical estimates of a derivative when the true derivative cannot be calculated.

Finally, it is possible that the function has no real-valued roots. In this case, it might be acceptable to find a minimum (that is, where the function gets as close as possible to zero).

9.3 Optimization

What does "optimization" mean and what does it have to do with calculus?

In common-use terms, "optimizing" means to improve something to reach its maximum potential. In the context of mathematics, optimization means finding the minimum or maximum of a func-

tion. You'd find the maximum if the function corresponds to a desired variable, like profit of a company or usable floor area in a house; and you'd find the minimum if the function corresponds to an undesirable variable, like materials costs or the discrepancy between a high-resolution digital image and its low-res compressed version.

Optimization is a major topic in applied mathematics. There are many textbooks dedicated entirely to this topic, there are many ways to optimize a problem, and there is an uncountable number of applications.

I already introduced three optimization methods in this book: gradient descent, grid-search, and Newton's method of finding roots. All of those methods are empirical and iterative, meaning that you continue searching and improving until you find a good solution, with the understanding that that solution might not be the *exact* best solution.

In this section, you will learn a derivative-based analytic method for optimization. This is where calculus comes into play: To find the minimum or maximum of a function, you take the derivative of the function, identify critical points, and find the critical point that optimizes your goal.

There are myriad methods for optimization that vary in their complexity and implementation. On the other hand, all optimization problems involve essentially the same steps: Understand the problem, translate the problem into a differentiable mathematical function, find the minimum or maximum of that function, and check whether the mathematical answer is an appropriate solution to the original problem. Thus, even if your applications of optimization don't use the methods described below, *any* experience with optimization problems will help you understand and solve *any* optimization problems.

9.3.1 How to solve derivative-based optimization problems

The kinds of optimization problems I will highlight in the rest of this chapter are typical for calculus course homework exercises and exams. But even if you're a self-learner, it's good to work through problems like this, because they will help you solve more complex optimization problems.

These word problems usually involve some kind of weird back story about people trying to maximize something or minimize something. In most cases, the solution — and the calculus — is straightforward; the hard part is setting up the problem and translating the words into equations.

I'm going to walk through a simple example. Read the paragraph below, but don't worry about solving the problem yet. Just figure out what information is (1) the optimization objective, (2) the constraint, and (3) irrelevant.

> Maximize the area of a rectangular garden using 50 meters of fencing, given that one side of the garden is an outside brick wall. What are the lengths of the sides of the garden?

What is the optimization objective? What do we want to minimize or maximize? The answer is *area*; we want to maximize the area of a rectangle.

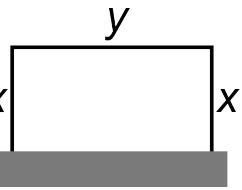


Figure 9.6: A rough sketch of the problem. Gray block depicts brick wall.

And what are the constraints? We have 50 meters of fencing, and the final shape should be a rectangle. Those two pieces of information become terms in the equations.

Finally, "garden" is useless information. Who cares if it's a garden or a parking lot or a dog house? And if the wall is made of brick or wood? That doesn't affect the math; it's just part of the back story. That information is literally just included to distract you and make the problem more difficult.

Next, make a sketch of the problem, and create variable names in the sketch (Figure 9.6). Don't worry about drawing to scale or being accurate; you just need a quick visualization to help you think about the equations.

Next, start writing down equations. If you don't know what equation to write, then write down any equations that might be relevant. Here are a few:

$$A = xy \tag{9.24}$$

$$F = 2x + y = 50 \tag{9.25}$$

$$y = 50 - 2x \tag{9.26}$$

$$A = x(50 - 2x) \tag{9.27}$$

$$= 50x - 2x^2 \tag{9.28}$$

Again, if you don't know what equations are relevant, then just start writing down whatever you can and hopefully things will start to make sense. I started by writing down the formula for area (length times height; Equation 9.24), then writing a function for the total length of fencing (F), which is one of the constraints. From here, I simply started moving terms around to define y in terms of x.

Here's the thing: This problem initially looks like it's a function of two variables (x and y), which is confusing because you haven't yet learned about multivariable calculus. But the constraint allows you to define one variable (y) in terms of the other (x), which means this is really a one-variable problem in disguise.

We're done with the algebra and can move on to calculus. But before differentiating, it's helpful to think about the equation. $A = 50x - 2x^2$ is a second-order polynomial with a negative coefficient, so it will open downwards. Because we're trying to maximize area, we can predict that the length of x that maximizes A

will be the peak of this curve. And how do we find the peak of that curve?

With differentiation, of course! We just need to find the critical point of this function:

$$A'(x) = 50 - 4x \tag{9.29}$$

$$0 = 50 - 4x \tag{9.30}$$

$$4x = 50 \tag{9.31}$$

$$x = 12.5 \tag{9.32}$$

Now the final (and sometimes tricky) part of the problem: Answer the original question. You might be tempted to say that the final answer is the area of the garden: $A = 12.5 \times 25 = 312.5$ m^2. But refer back to the original question: We're supposed to find the *dimensions of the garden*, not the maximum possible area. So the final answer to this question is $x = 12.5$ and $y = 50 - 2 \times 12.5 = 25$.

In other words, the math part of the problem involved maximizing *area*, but the final answer required reporting the *lengths* of the sides.

As a final check, it's a good idea to draw the objective function and its derivative (Figure 9.7); this will help you determine whether the numerical result makes sense given the nature of the problem.

What does it mean for a result to "make sense given the problem"? It's possible that an optimization function has multiple critical points but that some of them are not useful answers. For example, imagine that there were an additional critical point at $x = 0$, which would mean that $A = 0$. That could solve the problem numerically but wouldn't make sense in terms of the problem. You'll see another example of this later in this section.

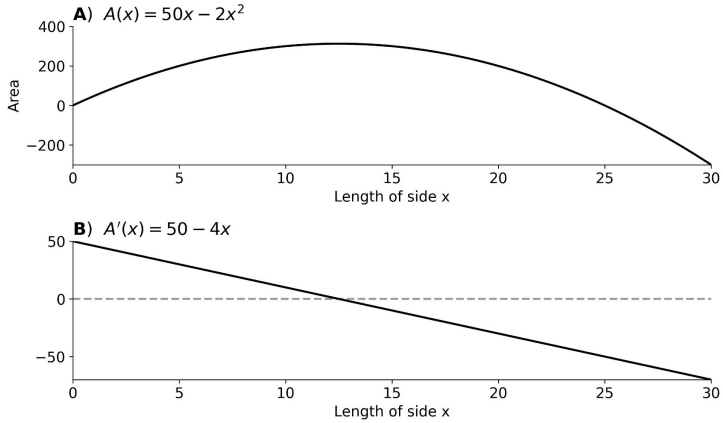


Figure 9.7: Panel A shows the optimization function (area as a function of side x), and panel B shows the derivative. The critical point identifies the function maximum, which is the solution to this optimization problem.

9.3.2 Example 1: Optimize for surface area

In the rest of this section, I will provide two examples of derivatives-based optimization problems; one about surface area and one about volume. Try to solve each problem on your own before reading my explanation below the problem statements. Then in the exercises, I'll give three more problems with full solutions in the online code.

Here's the first one:

> A gardener needs 100 square meters to grow pomegranates using the least amount of fencing to surround the area, which must be some rectangular shape. Find the lengths of the sides of the garden area.

Before writing down any math, figure out the optimization objective, constraints, and useless information.

See Figure 9.8 for a diagram.

The objective is to *minimize* perimeter (the lengths of the sides), which means we'll be looking for a critical point corresponding to a minimum. The constraints are that the area must equal 100 and be rectangular-shaped. And there is a lot of useless

distracting information that you can ignore, like "gardener" and "pomegranate."

Next we write down some equations. There are two equations we can derive from this story: Those for area and perimeter. I'll use x and y to indicate the pairs of sides.

$$A(x, y) = xy = 100 \tag{9.33}$$

$$P(x, y) = 2x + 2y \tag{9.34}$$

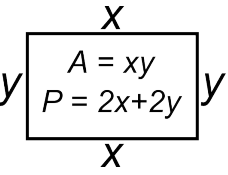


Figure 9.8: Rough sketch that helps with Example 1.

As in the previous example, this initially seems like a problem of two variables (x and y), but you can define one variable in terms of the other to create a single-variable equation. Let's set $y = 100/x$. The objective is to minimize P, so we re-define P in terms of x without y, then find its critical points:

$$P(x) = 2x + \frac{200}{x} \tag{9.35}$$

$$P'(x) = 2 - \frac{200}{x^2} \tag{9.36}$$

$$0 = 2 - \frac{200}{x^2} \tag{9.37}$$

$$1 = \frac{100}{x^2} \tag{9.38}$$

$$x^2 = 100 \tag{9.39}$$

$$x = \pm 10 \tag{9.40}$$

There are two critical points ($x = -10$ and $x = 10$). Does that mean this problem has two correct answers?

It takes a moment's reflection to realize that $x = -10$, while mathematically correct, cannot be physically correct. So the answer to the optimization is that the perimeter is minimized when two sides have length $x = 10$, which means $y = 100/x = 10$. In other

words, the area that minimizes the perimeter is a perfect square. This means that my sketch in Figure 9.8 was inaccurate — but that's fine; the goal of that sketch was to help me write down the variables and equations *before* knowing the answer.

A graph of the function and its derivative (Figure 9.9) clearly shows that $x = 10$ corresponds to a minimum. But it's also a good idea to use the second-derivative test to confirm.

$$P''(x) = \frac{400}{x^3} \tag{9.41}$$

$$P''(10) > 0 \tag{9.42}$$

You don't actually need to calculate $P''(10)$; you just need to know its sign. The second derivative is positive, meaning the first derive flips from negative to positive at the critical point, meaning the function is a minimum.

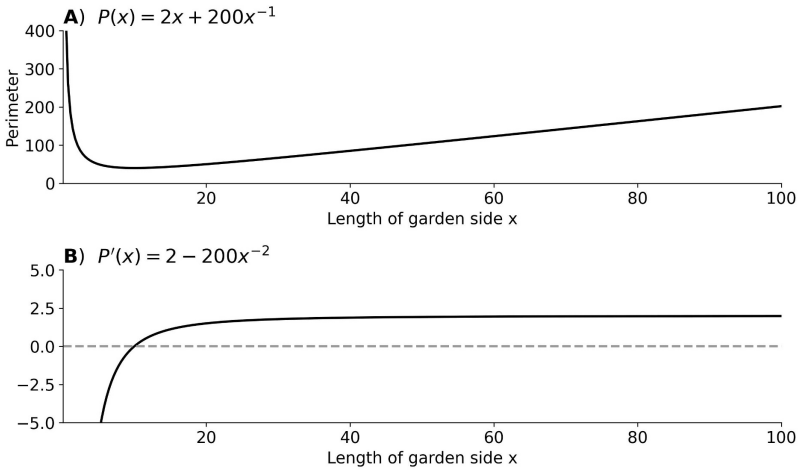


Figure 9.9: The function and its derivative. The length of side x corresponding to $P' = 0$ is visually consistent with the result of our calculations.

9.3.3 Example 2: Optimize for volume

Word problems involving optimizing for volume are conceptually identical to those optimizing for surface area or perimeter, although the math can get slightly trickier. See how much progress you can make on the problem below on your own, before reading my solution.

> The crazy neighborhood cat lady wants to build a box for her cats to sit in. She starts with a flat sheet of cardboard (45×80 cm) and will cut out square corners to fold up the sides (the box has no top to facilitate feline entry and egress). What size corners will make the biggest box?

The opimization goal is to maximize area (remember that the area of a box equals length times width times height). The constraints are the size of the flat cardboard sheet, and that the corner cutouts are square.

This one takes a little more imagination to sketch, but hopefully you arrived at something in Figure 9.10. Remember that the sketch doesn't need to be to scale; it simply needs to help you write out the equations. In this case, the goal is to find the lengths of the sides x that maximize the volume of the box.

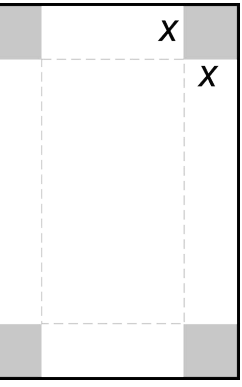


Figure 9.10: Sketch of the cardboard sheet used to create Crazy Cat Lady's cat box.

The box is made by folding the cardboard sheet along the dashed lines in Figure 9.10. The four small gray squares at each corner are removed, thus creating an open box.

Let's start with some equations. Having the diagram is really crucial in this problem, for example, to see that the length of the box is 80 cm *minus* whatever length the two corner cutouts take up.

$$A = LWH \tag{9.43}$$

$$A(x) = (80 - 2x)(45 - 2x)x \tag{9.44}$$

$$= \big[\ (80)(45) - (80)(2x) - (2x)(45) + (-2x)(-2x)\ \big]x \tag{9.45}$$

$$= \Big(4x^2 - 250x + 3600\Big)\, x \tag{9.46}$$

$$= 4x^3 - 250x^2 + 3600x \tag{9.47}$$

As usually happens in applied math, translating a real-world problem into an equation is the hardest part. From here, we simply differentiate and find critical points — and then use critical thinking.

$$A'(x) = 12x^2 - 500x + 3600 \tag{9.48}$$

$$0 = 12x^2 - 500x + 3600 \tag{9.49}$$

$$x = \frac{125 \pm 5\sqrt{193}}{6} \tag{9.50}$$

$$\approx \{9.3, 32.4\} \tag{9.51}$$

Solving Equation 9.50 by hand is not easy; I used `sympy`.

There are two solutions. Let's see what they look like (Figure 9.11).

Here we have two mathematical solutions (two critical points), just like in the previous example. In that example, one solution suggested a *negative* length, so we could easily rule it out. Here, however, we have two positive values. Can we rule out one?

Yes we can. Notice that if $x \approx 32$, then two corner cutouts would be around 64 cm, which is wider than the cardboard sheet. So that answer doesn't make sense. And visual inspection of the Area function (Figure 9.11A) confirms that $x \approx 9$ corresponds to a maximum.

The final answer to the question is that corners of roughly 9.3×9.3 cm will maximize the area of the box.

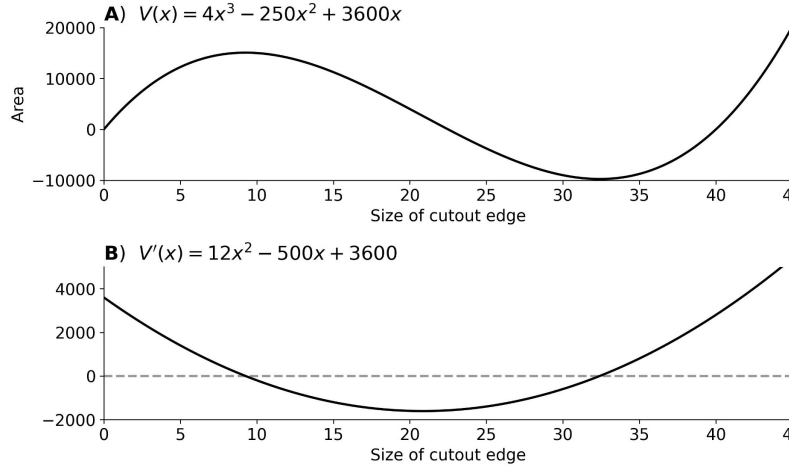


Figure 9.11: The function and its derivative. The length of side x corresponding to $P' = 0$ is visually consistent with the result of my calculations.

The problem specification didn't ask for the volume, but I calculated that to be 12,695 cm^3. I'm not a cat person, but ChatGPT told me that a typical cat has a volume of 7500 cm^3, which means 1.69 cats could fit inside Crazy Cat Lady's box. ChatGPT clarified that "since we cannot fit a fraction of a cat, the maximum number of whole cats that can fit inside the box is 1." I'd imagine that even if the box were slightly larger, not all cats would be willing to share their box, but that's a discussion for a textbook on cat psychology.

9.4 Exercises

9.1. Write a Python function that can implement the linear approximation method described in Equation 9.1, using symbolic expressions. Your Python function should take four inputs: the function, its derivative, the desired point x_a, and the starting point x_s.

Confirm the accuracy of your code using the first example problem from Section 9.1 by printing out the following:

```
Linear approximation: 10.8325000000000
Sympy calculation   : 10.8304544960367
```

9.2. Call the linear approximation function from the previous exercise in a for-loop over a range of starting values and $x_a = 10$. Calculate the error term and visualize as a function of the x_s value (vertical dotted line in Figure 9.12 is x_a). Not surprisingly, the error decreases as Δx increases.

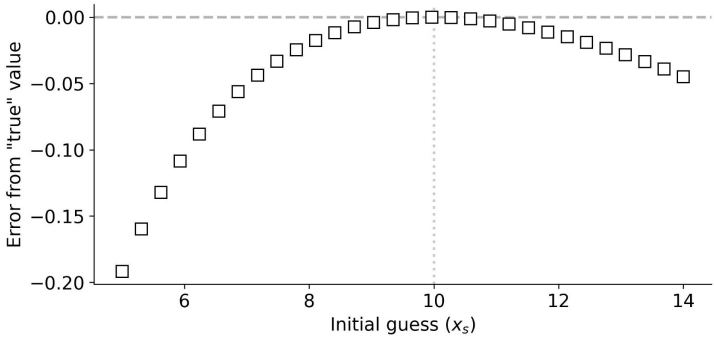


Figure 9.12: Results for Exercise 2.

9.3. Repeat the previous exercise but using the function from the second example in Section 9.1 ($f(\zeta) = |\sin(\zeta)|$). Try the approximation function once, using $\zeta_a = 5\pi/8$ and $\zeta_s = \pi/2$. The first hint that something is amiss is that the approximation isn't 1 as we calculated manually earlier in

the chapter, but instead is

$$f(\zeta_a) = 2.40458836715224 \cdot 10^{-17} \left. \frac{d}{dx} \operatorname{re}(x) \right|_{x=1.5707963267949} + 1.$$

When you figure out how to resolve this issue, run a systematic evaluation of the relationship between the starting values and the errors, using starting values in the interval $\zeta_s \in [-\pi, 2\pi]$. Define ζ_a as the average of all starting values. You should able to reproduce Figure 9.13.

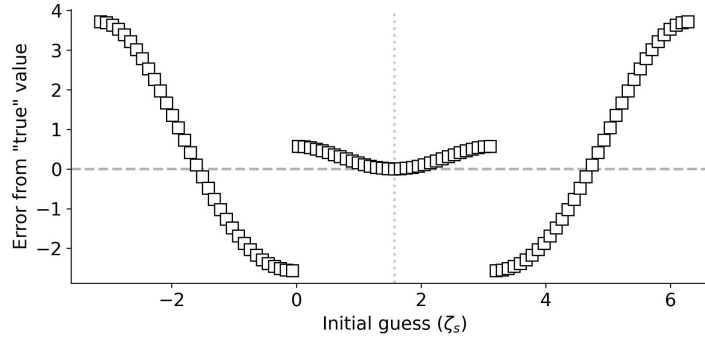


Figure 9.13: Results for Exercise 3.

In this example, it is not trivially the case that the error decreases as $\Delta\zeta$ increases. In fact, there are very small errors for relatively larger values of $\Delta\zeta$. Why is this the case?

As I guess you've guessed, it's due to the cyclic nature of sine. In fact, for this function, ζ_s will produce the same estimate as $k\pi\zeta_s$ for any integer k. That's not necessarily the case for all functions that have trig terms; it just happens to be the case in this example. But the general point here is that any time you work with trig functions, you should think about the possibility of cyclic solutions.

9.4. Now for a few exercises on Newton's method for finding roots. Write a Python function that implements the calculus aspect of Newton's method, which is Equation 9.23. Your function should take three inputs — a `sympy` function, its derivative, and the estimate x_n — and return the

updated estimate x_{n+1}.

Let's start with the equation $2x^3 = 3$. Use `sympy` to find the real-valued solution to this equation. Then use Newton's algorithm to estimate that solution. Perform two iterations by calling the function twice (don't worry about writing a for-loop for this exercise). Print your results like below.

```
Initial guess:    x = 0.800
First iteration:  x = 1.315
Second iteration: x = 1.166
Analytic root:    x = 1.145
```

Before moving on to the next exercise, spend a few minutes playing around with the algorithm: try adding a few more iterations, and try a different initial guess x_0 (e.g., what happens when you start with $x_0 = -1$?). Just don't start with $x_0 = 0$ because that's the goal of the next exercise!

9.5. There are three goals of this exercise: First, confirm that something bad happens in the previous exercise when the initial guess is $x_0 = 0$. Second, figure out what the problem is and why it happens. Third, think of a few ways to fix this problem. My answers are in the online code.

9.6. The goal of this exercise is to write code that produces Figure 9.4 (page 332).[1]

Getting `sympy` to return only the real-valued root of $3x^3 + 2x = 7\pi$ may require some trial-and-error finagling. Put the code into a for-loop that implements one root-estimate update, and one additional dot in the graph, in each iteration.

Once you have the code working, try different starting values, e.g., $x_0 = -1$ or $x_0 = 10$.

[1] As I've stated before, don't worry about aesthetic issues like dot size and color; just focus on the important aspects of the figure.

9.7. Using the same equation as in the previous exercise, run five iterations, store the estimate at each iteration, and make a scatter plot with iteration number on the x-axis as in Figure 9.14. I wrote a new for-loop for this exercise to avoid dealing with the plotting code from the previous exercise.

Note 1: This pattern of rapid convergence is not guaranteed for all problems. Although Newton's method can converge quadratically (meaning that the number of accurately estimated digits doubles with each iteration), it can also fail for several reasons, take an excessive number of iterations, or be insufficiently accurate for a sensitive application.

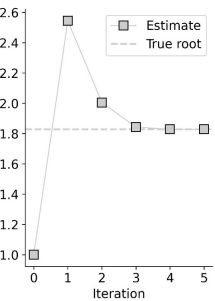


Figure 9.14: Visualization of Exercise 7.

Note 2: Lines connecting the squares is technically incorrect, because it implies an approximated value at, e.g., iteration number 1.5 (which makes no sense). But I think the line facilitates visualization and understanding. What do you think? Try redrawing the plot without the line, and perhaps try other ways of visualizing this result, e.g., using a bar plot.

9.8. This and the next two problems focus on derivatives-based optimizations. Your goal is to solve the word problems listed here and produce the accompanying figures. Full write-ups of my solutions are in the online exercises.

Here's the description of the first problem, and the associated Figure 9.15.

> Farmers Fred and Fran are neighbors in Fabregues, France. Fred wants to fence in a rectangle of 400 m^2 that shares one side with Fran. Fran agrees to pay 50% of the cost of that side. Fencing is 100 Euros/meter. Find the lengths of the sides that minimize Fred's fencing cost.

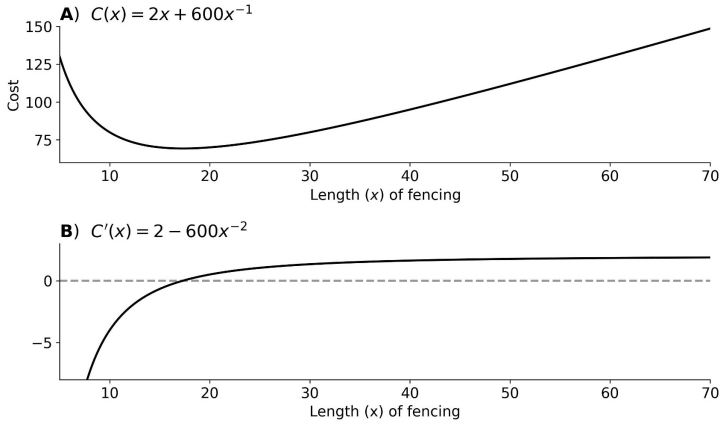


Figure 9.15: Results for Exercise 8.

9.9. Here's the next exercise. Again, solve the problem and produce Figure 9.16, and then check your answer against mine in the online code.

> QueBurt the curious space alien makes enclosed boxes of 200 cm^3, with the top and bottom being squares. But material is expensive and QueBurt is an honest hard-working alien. What size edges will minimize materials costs?

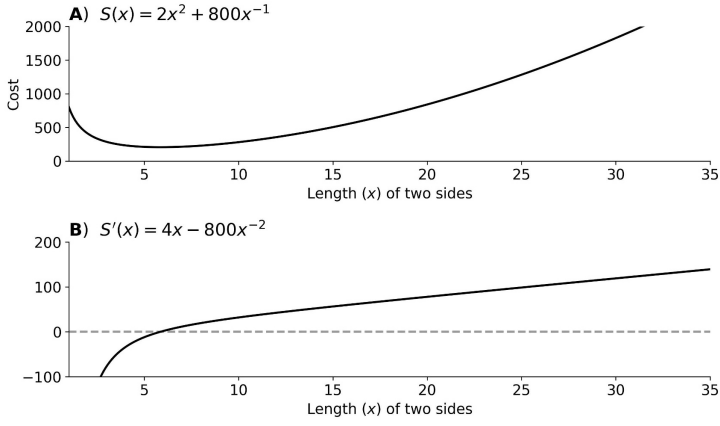


Figure 9.16: Results for Exercise 9.

9.10. Final optimization problem. This one is an adjustment of the previous exercise.

QueBurt realizes that his boxes don't need to be covered, so he makes *open boxes* of 200 cm^3, with the bottom being a square and no top. What size faces will minimize materials costs?

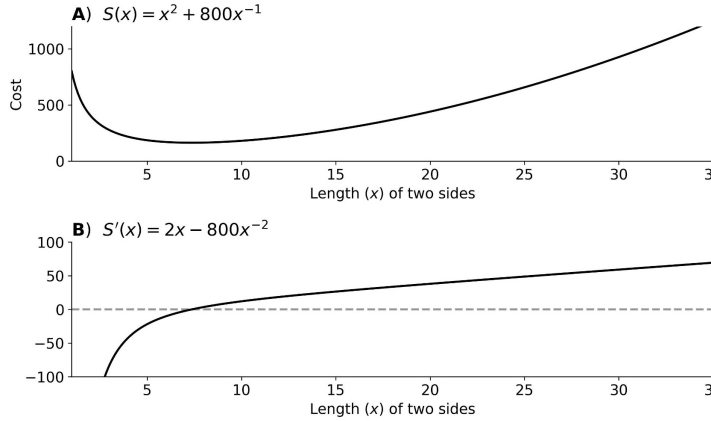


Figure 9.17: Results for Exercise 10.

Chapter 10

Multivariable differentiation

Multivariable calculus is a beautiful topic — both because it opens the door to advanced mathematics such as differential geometry, multidimensional optimization, and myriad real-world applications, and because the visualizations arising from multivariable calculus are fascinating and aesthetically satisfying to look at.

Multivariate calculus is also a big topic. There are books and semester-long courses entirely dedicated to multivariable calculus; this chapter will provide an introduction to the topic that will give you a solid foundation for your future deep-dives.

On the other hand, multivariable calculus is challenging mainly because there are more variables to keep track of; conceptually, it's not much different from single-variable calculus.

Terminology You might hear or read the terms *multivariate*, *multivariable*, or *multidimensional* calculus.

These terms mean more-or-less the same thing and can be used interchangeably. There are some subtleties, for example, in statistics, *multivariable* means multiple variables whereas *multivariate* refers to a category of regression models. Furthermore, the term *multidimensional* is often used in context of geometry or vector spaces in linear algebra.

The point is that there are field-specific connotational differences amongst the terms, but the underlying mathematical concepts and code-based implementations are the same.

Single- vs. multivariable: All the calculus you learned so far in this book is called "single-variable calculus" (even the optimization problems in the previous chapter that were defined using multiple variables were solved using single-variable equations).

Technically, "*multi*variable" calculus includes any number of variables greater than one. But in this chapter, multivariable always means two variables. Two-variable functions are convenient because they are easy to visualize and the calculus doesn't get too tedious. But all of the concepts and methods you'll learn here

apply to any number of variables.

10.1 Two-variable functions

A two-variable function is also referred to as a two-dimensional function (because of plotting), so a "2D function" has two input variables.

Remember that a function is a set of rules for mapping an input variable onto an output variable. A 2D function is the same concept, but there are two input variables. There is still one output variable[1]. Here is a simple example:

$$f(x, y) = x + y \tag{10.1}$$

Well, I wrote that it would be a *simple* example, not necessarily an *interesting* example. But you can see that the function takes two inputs and produces one output for each pair of x, y values.

Here is the general expression of a 2D function:

$$Z = f(x, y) \tag{10.2}$$

In single-variable functions, y is often used to denote the output, whereas in two-variable functions, y is often used to indicate the second input while Z indicates the output. The reason for this convention will become apparent when you learn about visualizing two-variable functions in a 3D axis. The inputs are also sometimes written as x_1, x_2, and so on, up to x_k for k variables.

Also notice that the output variable is capitalized (Z not z). That's because 2D functions generally produce a 2D output *matrix*, and matrices are often indicated using capital letters.

[1] Multivariable functions can produce multiple outputs (in which case they are called "vector-valued functions"), but in this chapter we will consider only single-output functions.

The reason why 2D functions output a matrix is that there is an output value for each pair of (x, y) values[2].

Here is another two-variable function (still simple but more interesting than the first example above):

$$f(x, y) = \frac{1}{x + y - 1} \tag{10.3}$$

If you use only positive integers for x and y, then Equation 10.3 is called a *Hilbert matrix* and is used in statistics, for example in modeling polynomial functions. The matrix below shows an example Hilbert matrix with five rows and columns. Notice that each element in the resulting matrix $Z = f(x, y)$ is given by evaluating Equation 10.3 for a given pair of (x, y) values.

$$Z = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} \\ \frac{1}{5} & \frac{1}{6} & \frac{1}{7} & \frac{1}{8} & \frac{1}{9} \end{bmatrix}$$

Also notice that the top-left element results from the function evaluation $f(1, 1)$, not $f(0, 0)$. Math functions start indexing at 1. This is something to watch out for when translating math functions into Python code, where indexing begins at 0.

10.1.1 Creating 2D functions in Python

Remember that single-variable functions are visualized by defining a grid on the x-axis, evaluating the function $y = f(x)$ for each value of x, and then plotting dots (possibly with connecting lines) on a graph. A single-variable function requires two axes to visualize: One axis to define the grid (the function input x), and one axis to show the function output (y).

[2]Not all two-variable functions have 2D matrix outputs, but that will be the case for the 2D functions used in this book.

Two-variable functions require three axes to visualize: Two axes to define the grid (x and y) and one axis (z) to show the function output (Figure 10.1).

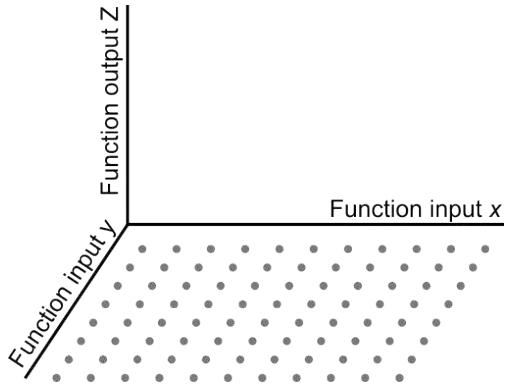


Figure 10.1: Discretizing a function with two inputs and one output. Gray dots indicate discretization coordinates at which the function is evaluated. The height on the Z axis at each coordinate is given by the output of the function.

How do you create that grid of coordinates shown in Figure 10.1? You can use the function `meshgrid` to specify the x- and y-axis coordinate values, and the function will return two matrices comprising x- and y-axis coordinates. For example, the following code:
`X,Y = np.meshgrid([-1,0,3,4],[1,2,3])`
will return the following matrices:

```
Matrix of x-indices (variable 'X'):
[[-1 0 3 4]
 [-1 0 3 4]
 [-1 0 3 4]]

Matrix of y-indices (variable 'Y'):
[[1 1 1 1]
 [2 2 2 2]
 [3 3 3 3]]
```

Notice that (**1**) the two matrices are the same size (3×4) although the number of elements in the two inputs to `meshgrid` differ; (**2**) x-axis values are identical in each column whereas the y-axis values are identical in each row; and (**3**) each corresponding pair

of numbers is a coordinate that can be used as input to a 2D function.

For example, the function $Z = X^2 + Y$ using the above matrices produces the following result:

```
array([[ 2, 1, 10, 17],
       [ 3, 2, 11, 18],
       [ 4, 3, 12, 19]])
```

Make sure you understand how each element in that matrix is the result of applying the function rule to each corresponding pair of x, y values that were created by the `meshgrid` function.

10.1.2 Visualizing 2D functions

Creating a two-variable function in Python involves three matrices: the two matrices of x- and y-axis coordinates on which to evaluate the function, and the matrix Z that is the result of the function evaluated at those values.

For simple functions with only a few input values, it might be sufficient simply to print out all the numbers for inspection. But that is obviously not a scalable method for understanding how functions behave over a hundreds, thousands, or millions of x, y value pairs. So instead, the matrices can be visualized as *images*.

To visualize a 2D function as an image, the value of the function at each x, y coordinate is mapped onto a color value and shown as a picture. The small example I showed above is not suitable for creating such an image, because the x- and y- axis coordinates should be evenly spaced (that is the case for the y-axis but not the x-axis). So let's try again with a more interesting-looking function. The code below evaluates the function $Z = \sin(x) + \cos(y)$.

```
x = np.linspace(0,2*np.pi,31)
```

```
X,Y = np.meshgrid(x,x)
Z = np.sin(X) + np.cos(Y)
```

Figure 10.2 shows two ways to visualize this function. The "surface" plot in panel A depicts the function as a membrane floating on top of the input plane, whereas the "flat" image in panel B depicts the function as a flat image in which the color value at each (x, y) coordinate maps onto the function output value.

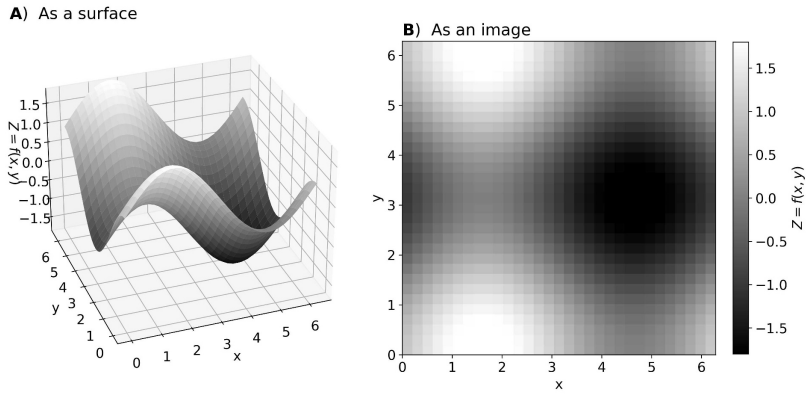


Figure 10.2: Two ways of visualizing a 2D function output: as a surface floating on top of the XY plane with height and color showing the function value (panel A), and as a flat image with color representing the function value (panel B).

Surface plots look neat, don't you think? They can be useful for qualitative interpretation but tend to be difficult for quantitative inspection (for example, what is the value of the function at $f(2, 3)$? I guess you had to look at the flat map in panel B to answer). Indeed, the difficulty of visually mapping x, y coordinates to a z-axis height is the reason why the color and height on the z-axis are redundant — notice that if you would rotate the surface so that you're on top of it, looking directly down through the z-axis, you'd get the flat image in panel B.

The conclusion is that if you want to make neat-looking plots to show off on a blog, t-shirt, or a broche that your grandmother wears at her bingo parties, then make the surface plots; but if you want to maximize effective and efficient interpretability, then make a flat image.

On the other hand, if you could rotate that 3D axis with your mouse, you'd be able to appreciate how the function looks from different view angles. That is possible in Python notebooks, for example using the `plotly` library. I will guide you through multiple ways of visualizing 2D functions in `matplotlib`, `sympy`, and `plotly` in the exercises.

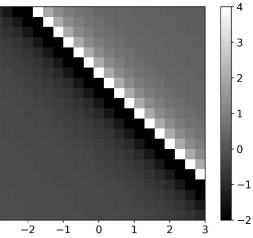


Figure 10.3: A "Hilbertish" matrix.

Out of curiosity, I also implemented and visualized Equation 10.3 (Figure 10.3). I used negative and non-integer values for x and y, so it's no longer formally a Hilbert matrix. I didn't try making a 3D surface plot of it, though; perhaps you can try that ;)

10.2 Limits of multivariable functions

In a conceptual sense, the limit of a multivariable function is exactly the same as the limit of a single variable function: It is the value of the function as it approaches, but does not equal, a particular input value.

However, limits of multivariable functions are more complicated than their single-variable counterparts, for two reasons that I will state here and elucidate below. (**1**) The limit as *one* variable approaches some value (e.g., $x \to x_0$) may depend on the value of *another* variable (e.g., whether $y = a$ or when $y = b$). (**2**) The limit value could change depending on the direction in which the function approaches the limit coordinate.

Let's consider the first point. What is the limit of the following function?

$$\lim_{x\to 0} \frac{2x + 3y + 4xy^2 + 7}{\sqrt{x^2 + y^2}} \tag{10.4}$$

We don't need any fancy limit-solving tricks here, because the plug-in method works. But the answer is that the limit is another

function: In this example, it's a function of variable y.

$$\lim_{x\to 0} f(x,y) = \frac{3y+7}{\sqrt{y^2}} \tag{10.5}$$

The interpretation of this result is that we choose one particular *line* in the function — the line corresponding to $x = 0$ — and find that the limit depends on the value of y when the function approaches that line. There is no single numerical value for this limit, because there is an infinite number of y points that approach the function at the line corresponding to $x = 0$.

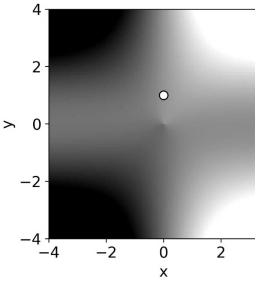


Figure 10.4: Visualization of Equation 10.7 with the dot indicating the limit point.

Therefore, the limit as the function approaches $x = 0$ produces a single-variable function, and we can calculate the limit of *that* function for some specific value of y:

$$\lim_{y\to 1} \frac{3y+7}{\sqrt{y^2}} \tag{10.6}$$

Of course, the answer is that the limit is 10. Combining these two equations into one, we get:

$$\lim_{(x,y)\to(0,1)} \frac{2x+3y+4xy^2+7}{\sqrt{x^2+y^2}} = 10 \tag{10.7}$$

Note the new notation!

In this particular example, the order in which you apply the limit operator doesn't matter. That is,

$$\lim_{x\to 0}\lim_{y\to 1} f(x,y) = \lim_{y\to 1}\lim_{x\to 0} f(x,y) = \lim_{(x,y)\to(0,1)} f(x,y)$$

But that symmetry is not guaranteed; symmetry of limits is not a general property of multivariable functions. In fact, I specifically created a function that would have the same limit at the point (0,1) regardless of the order in which the variables are considered.

This brings me to the second point of difficulty of limits of multivariable functions: The limit value can depend on the order in

which you calculate the limits of the variables, or on how you define the line along which the limit point is approached.

Let's look at an example.

Visualized in Figure 10.5.

$$f(x,y) = \frac{xy}{x^2+y^2} \tag{10.8}$$

The function is undefined for $x = y = 0$, but we can ask what is the limit of this function as it *approaches* (0,0)? It turns out that the answer to that question depends on the starting point from which we approach that point.

Let's consider the limit when we fix $y = 0$ (the horizontal dashed line in Figure 10.5).

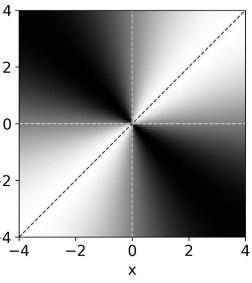


Figure 10.5: The limit to Equation 10.8 at the origin depends on whether you approach from the light gray lines or dark gray line.

$$\lim_{x\to 0} \frac{x\cdot 0}{x^2+0^2} \;=\; \lim_{x\to 0} \frac{0}{x^2} \;=\; 0 \tag{10.9}$$

Remember that the limit here is not 0/0: The numerator really is zero, but the denominator is some tiny tiny number that is arbitrarily close to but not exactly equal to zero.

Now for the limit as the function approaches the origin along the line $x = 0$ (vertical dashed line in Figure 10.5).

$$\lim_{y\to 0} \frac{0\cdot y}{0^2+y^2} \;=\; \lim_{y\to 0} \frac{0}{y^2} \;=\; 0 \tag{10.10}$$

So far, we seem to have symmetry: the limit is the same regardless of how we approach the point (0,0).

But wait a minute: There's another dashed line drawn in the figure, so obviously we should check for the limit there. That diagonal line is the line of unity, where $x = y$. Let's see what happens here:

$$\lim_{x\to 0} \frac{x \cdot x}{x^2 + x^2} = \lim_{x\to 0} \frac{x^2}{2x^2} = \frac{1}{2} \tag{10.11}$$

Now you can see that the limit depends on how we approach the coordinate in question.

In single-variable functions, we speak of the two one-sided limits (approaching x_0 from the left or the right); in multivariable functions, there is an infinite number of "one-sided" limits because a given (x, y) point can be approached from an infinite number of directions (a few of which I showed above). When *any* of these directions gives a different answer, the limit of the function at that point does not exist.

Limits in sympy You might expect to be able to use a syntax like

```
sym.limit(expr,(x,y),(x0,y0))
```

But that doesn't work. Instead, you need to calculate limits separately for each variable, in the order you want to calculate them. Let's work through an example.

$$f(x, y) = 2x + \frac{4}{y} \tag{10.12}$$

$$\lim_{(x,y)\to(0,2)} f(x, y) = ? \tag{10.13}$$

Before going any further, you should be able to calculate that the limit of this function exists at (0,2), and the value of the limit is $f(x, y) = 2$. Let's see how to obtain this result in `sympy`.

We start by confirming that the single-variable limit of a multivariable function is itself a function of one variable[3]:

[3]The result of a single-variable limit of a multivariable function is not necessarily a single-variable function; in general, the single-variable limit of a k-variable function is a function of at most $(k - 1)$ variables.

```
x,y = sym.symbols('x,y')
f = 2*x + 4/y

limX = sym.limit(f,x,0)
limY = sym.limit(f,y,2)
```

This code produces the following results:

$$\lim_{x\to 0}\left(2x+\frac{4}{y}\right) = \frac{4}{y} \tag{10.14}$$

$$\lim_{y\to 2}\left(2x+\frac{4}{y}\right) = 2x+2 \tag{10.15}$$

The output variables `limX` and `limY` are `sympy` objects, just like other symbolic functions. That means that you can calculate the limit of the output of the `limit` function. Thus, the next step would be to compute `sym.limit(limX,y,2)`.

It is also possible to use a more compact option by combining the *method* `limit` on the output of the *function* `limit`:

```
limXY = sym.limit(f,x,0).limit(y,2)
limYX = sym.limit(f,y,2).limit(x,0)
```

Both of these results give the expected value that the limit is 2. Be mindful that when you call `limit` as a method on a `sympy` object, you input only the limit variable (x or y) and the limit value, not the function expression (in this example, variable `f`).

10.3 Partial derivatives

Multivariable differentiation is actually straightforward: you just differentiate one variable at a time, treating the other variable as a constant. That is, you differentiate a function $f(x,y)$ *with respect*

to x while holding y constant, and then differentiate it again *with respect to* y while holding x constant.

The result of differentiating one variable while holding others constant is called a *partial* derivative. A two-variable function has two partial derivatives, a three-variable function has three partial derivatives; more generally, a k-variable function has k partial derivatives.

As a simple example, consider the function $f(x, y) = 2x + 3y$. Differentiating with respect to x gives 2 ($3y$ drops out because it is treated as if it were a constant), while differentiating with respect to y gives 3.

Notation There are several ways to indicate the partial derivative of a function. The two most common are using the "del" operator and using subscripts.

$$\frac{\partial f}{\partial x} = f_x \tag{10.16}$$

$$\frac{\partial f}{\partial y} = f_y \tag{10.17}$$

The ∂ symbol is called "del" or "partial."

Read out loud, you would say "the partial derivative of f with respect to x." It's also common to shorten this to "partial f, partial x," or, even more simply, "partial x."

An example Consider the three expressions below.

$$f = x^2 + xy + y \tag{10.18}$$

$$f_x = 2x + y \tag{10.19}$$

$$f_y = x + 1 \tag{10.20}$$

I have two questions that I would like you to answer, and then consult the footnotes to confirm or correct your answer. (**1**) In Equation 10.19, why did xy disappear yet y remains?[4] (**2**) In Equation 10.20, where does the "1" come from?[5]

A second example Please calculate the two partial derivatives of the following function. I'll write the answers below; I trust you not to look at them until after doing the work yourself :)

$$f(x, y) = 2x^3 + 5y^3 + \cos(x)\sin(y) \tag{10.21}$$

Remember that when differentiating with respect to x, any term with y is treated as a constant, just like 2 or π are constants.

The answers are below.

The ∂ fraction notation is also sometimes called "operator form."

$$\frac{\partial f}{\partial x} = 6x^2 - \sin(x)\sin(y) \tag{10.22}$$

$$\frac{\partial f}{\partial y} = 15y^2 + \cos(x)\cos(y) \tag{10.23}$$

10.3.1 Visualizing partial derivatives

Although y is treated as a constant when calculating f_x, the resulting partial derivative is still a function of two variables (x and y). Therefore, we need three images to show the function and its two partial derivatives.

Let's start with the first example above (Equation 10.18). Figure 10.6 shows the function and its two partial derivatives.

[4] This is intentionally confusingly worded: When calculating the derivative with respect to x, we treat y as a constant; therefore, the derivative of the xy term is y while the derivative of the y term is zero: it does not vary as a function of x.

[5] When calculating the partial derivative with respect to y, x^2 is treated as a constant, and the derivative of y is 1.

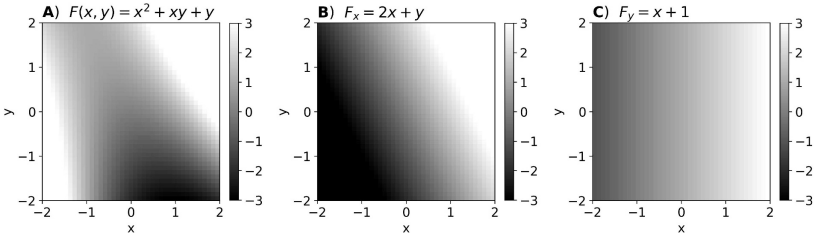


Figure 10.6: Visualizing a two-variable function (panel A) and its partial derivatives (panels B and C).

And Figure 10.7 shows the second example from above. I would like you to take a few moments to inspect these two figures and think about what the partial derivatives mean and how to interpret them. I'll write a longer discussion in the next subsection.

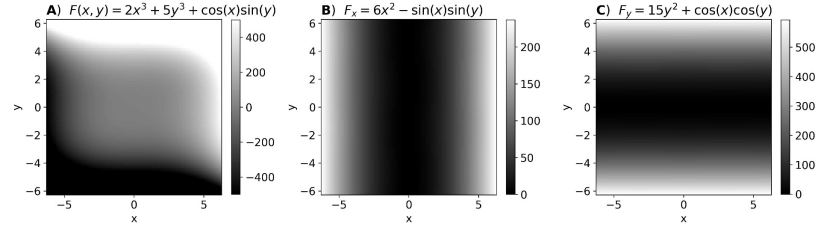


Figure 10.7: Another example of partial derivatives. Same organization as the previous figure but a different function.

10.3.2 Interpreting partial derivatives

First, I'll make some observations that are specific to the functions shown in Figures 10.6 and 10.7, then I'll make some more general remarks about interpreting partial derivatives.

1. In Figure 10.6C, the variable y does not appear in F_y. This means that the partial derivative with respect to y is simply a function of x; the image shows a matrix but each row in that matrix is identical, because the way that the function changes with y depends only on the value of x. The single-variable analog of this is when a derivative is a constant value.

2. On the other hand, F_x (panel B) is a function of both x and

y. This means that the way the function changes along x depends both on the value of x and on the value of y.

3. In Figure 10.7, the two partial derivatives appear — from visual inspection — to be constant across the rows (panel B) and columns (panel C), but this is not technically the case. Notice that the partial derivatives each comprise the sum of two terms, a quadratic and a trig. The trig term is bound by $[-1, +1]$ whereas the quadratic term can become arbitrarily large and therefore dominates the derivative except for values close to zero.

But how do you interpret a plot of partial derivatives more generally? First of all, keep in mind that, as with line plots of single-variable functions, the plot of the derivative is not the same as the plot of the function. The plot of the function is the value of the function for each input value. In contrast, the value of the derivative describes how the function output changes in response to a tiny change in the input.

Same for multivariable derivatives: The image of a partial derivative tells you how the output of a function changes *along one direction* when the input changes a tiny bit *along that same direction.*

This concept is illustrated in Figure 10.8. Panel A shows a 2D function. The white arrow indicates how the partial derivative F_x is determined at each point: Calculate how the function changes with Δx *for one fixed value of* y. The corresponding white dot in panel B provides information only about how $F(x, y)$ changes for that one value of y, and not for any other value of y. Same story for F_y in panel C but for changes in y calculated separately for each value of x.

10.3.3 Partial derivatives: formal definition

It is worth your time to inspect the formal limits-based definitions of partial derivatives at least once, to make sure the idea is clear.

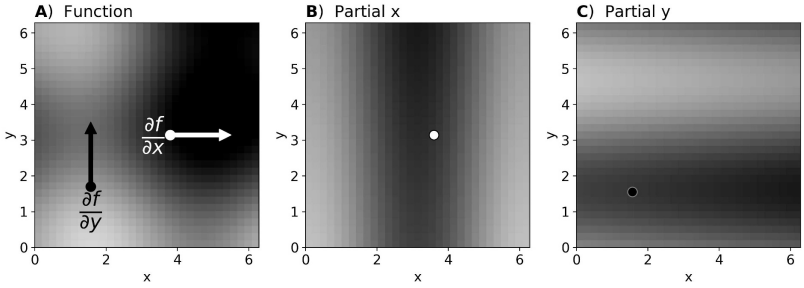


Figure 10.8: A 2D function ($F(x,y) = \sin(x)+\cos(y) - xy/10$), its two partial derivatives maps, and arrows indicating the directions in the function along which the partial derivatives are calculated at each point.

I'll also use these formulas to prove the symmetry of higher-order partial derivatives in the next section.

$$f_x = \frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x+h, y) - f(x, y)}{h} \tag{10.24}$$

$$f_y = \frac{\partial f}{\partial y} = \lim_{h \to 0} \frac{f(x, y+h) - f(x, y)}{h} \tag{10.25}$$

Briefly, the idea is to add a vanishingly small number (h as $h \to 0$) to one variable but not to the other. This shows how, for example, variable y is treated as a constant when differentiating with respect to x.

10.4 Higher-order partial derivatives

Higher-order partial derivatives reveal how calculus can become tedious even when it is not difficult. For example, one function of two variables has two partial derivatives, each of which can be differentiated again with respect to x and y, which means that one function has four second partial derivatives. In terms of calculus, there is nothing special about calculating second-order partial

derivatives (or any higher order): You simply differentiate as you already know how to do. The tricky part comes with interpretation.

First, some notation. The following sets of expressions show different ways of indicating second-order partial derivatives of a two-variable function. The first and third columns indicate the mechanism and order by which the derivatives are calculated.

$$\frac{\partial}{\partial x}\left(\frac{\partial f}{\partial x}\right) = \frac{\partial^2 f}{\partial x^2} = (f_x)_x = f_{xx} \tag{10.26}$$

$$\frac{\partial}{\partial y}\left(\frac{\partial f}{\partial y}\right) = \frac{\partial^2 f}{\partial y^2} = (f_y)_y = f_{yy} \tag{10.27}$$

$$\frac{\partial}{\partial y}\left(\frac{\partial f}{\partial x}\right) = \frac{\partial^2 f}{\partial y \partial x} = (f_x)_y = f_{xy} \tag{10.28}$$

$$\frac{\partial}{\partial x}\left(\frac{\partial f}{\partial y}\right) = \frac{\partial^2 f}{\partial x \partial y} = (f_y)_x = f_{yx} \tag{10.29}$$

I believe that Equations 10.26 and 10.27 are self-explanatory: differentiate the function twice for the same variable, both times treating the other variable as a constant.

Equations 10.28 and 10.29 require some more explanation. These are called the "mixed" partial derivatives and involve differentiating with respect to one variable (e.g., the derivative of f with respect to x), and then differentiating that partial derivative with respect to the other variable (e.g., the derivative of f_x with respect to y).

But how do you know which order to differentiate? The two notational forms use different orders: Note the $\partial y \partial x$ in the denominator of the Leibniz notation, but f_{xy} in the subscript notation.

In the Leibniz notation, the first derivative is to the right while the second derivative is to the left. That follows from thinking of differentiation as an operator acting on a function: First apply the

differentiation operator for x, then apply differentiation operator for y.

But the subscript notation has the opposite pattern, because it indicates differentiating with respect to y, of the function f_x. Perhaps it's helpful if you think of the ∂ notation as verbs acting on a function, and of the subscript notation as nouns that result from the actions of the verbs.

10.4.1 Symmetry of higher-order partial derivatives

Is there a risk of apocalyptic disaster if you confuse the order to differentiate to obtain the mixed partial derivatives?

Fortunately, the answer is *no*, and the reason is that the mixed derivatives are equal. That is, $f_{xy} = f_{yx}$. It doesn't matter whether you differentiate with respect to x then with respect to y, or the other way around; you will get the same answer. This is called Clairaut's Theorem, named after Alexis-Claude Clairaut[6].

The mixed partial derivatives are equal only if the function is continuous and differentiable; more on this later.

There are several ways to prove Clairaut's theorem, some of which rely on integration and the Fundamental Theorem of Calculus. But we can prove the symmetry by applying the limits definition to partial derivatives and showing that both orders give the same results. The idea is to write out the limits definition of f_{xy} and f_{yx}, and find that they are equal.

Equation 10.31 shows the limits definition of the derivative with respect to y (using variable k) with the numerator showing the partial derivative with respect to x. Then Equation 10.32 expands all the terms.

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x+h, y) - f(x, y)}{h} \tag{10.30}$$

Equation 10.30 was presented earlier in the chapter, and is repeated here for convenience.

[6] As far as I know, Clairaut was not a musician, but I think it would be fitting if he had played the clarinet.

$$\frac{\partial}{\partial y}\left(\frac{\partial f}{\partial x}\right) = \lim_{k\to 0} \frac{\frac{\partial f}{\partial x}(x, y+k) - \frac{\partial f}{\partial x}(x, y)}{k} \tag{10.31}$$

$$= \lim_{h\to 0} \lim_{k\to 0} \frac{\left[\frac{f(x+h,y+k)-f(x,y+k)}{k}\right] - \left[\frac{f(x+h,y)-f(x,y)}{k}\right]}{h} \tag{10.32}$$

Now consider the equations for the other mixed derivative:

$$\frac{\partial}{\partial x}\left(\frac{\partial f}{\partial y}\right) = \lim_{h\to 0} \frac{\frac{\partial f}{\partial y}(x+h, y) - \frac{\partial f}{\partial y}(x, y)}{h} \tag{10.33}$$

$$= \lim_{h\to 0} \lim_{k\to 0} \frac{\left[\frac{f(x+h,y+k)-f(x+h,y)}{k}\right] - \left[\frac{f(x,y+k)-f(x,y)}{k}\right]}{h} \tag{10.34}$$

It may not be immediately obvious that Equations 10.32 and 10.34 are identical, but if you write out these equations and simplify a bit more (hint: remove k from the numerator to have $1/kh$ as a separate term), you will see that they are equal.

After working through the algebra, the conclusion of Clairaut's theorem is that the order of the limits is interchangeable:

$$\lim_{h\to 0} \lim_{k\to 0} f(h,k) \;=\; \lim_{k\to 0} \lim_{h\to 0} f(h,k) \;=\; \lim_{(h,k)\to(0,0)} f(h,k) \tag{10.35}$$

Is symmetry guaranteed? The two orders of mixed partial derivatives will be equal if both first-order partial derivatives are continuous and differentiable.

That is, if f_x and f_y have defined derivatives at (x_0, y_0), then $f_{xy} = f_{yx}$ at $f(x_0, y_0)$.

If the first-order derivatives have discontinuities or are undefined for some restricted domains, then it is possible that f_{xy} exists while f_{yx} does not.

10.4.2 Example

After you understand the concept of higher-order partial derivatives, it's often a good idea to let `sympy` do the hard work. But for now, I'd like you to solve this problem by hand.

$$f(x,y) = \cos(2xy^2) + x^2 - ye^x \tag{10.36}$$

Your goal is to calculate the two partial derivatives and four second-order derivatives. Thus, you should get six functions out of Equation 10.36. My answers are below.

$$f_x = -2y^2 \sin(2xy^2) + 2x - ye^x \tag{10.37}$$

$$f_y = -4xy \sin(2xy^2) - e^x \tag{10.38}$$

$$f_{xx} = -4y^4 \cos(2xy^2) + 2 - ye^x \tag{10.39}$$

$$f_{yy} = -4x \sin(2xy^2) - 16x^2y^2 \cos(2xy^2) \tag{10.40}$$

$$f_{xy} = -8xy^3 \cos(2xy^2) - 4y \sin(2xy^2) - e^x \tag{10.41}$$

$$f_{yx} = -8xy^3 \cos(2xy^2) - 4y \sin(2xy^2) - e^x \tag{10.42}$$

This example also demonstrates the equality of f_{xy} and f_{yx}.

What on Earth do these equations mean?! Probably nothing in terms of physical reality, considering it's an arbitrary function that doesn't come from any theory of nature or experimental result. But the amazing part of using computers to help learn math is that we can still visualize these mysterious equations (Figure 10.9).

I'm sure you agree that these images are tantalizingly intriguing. The baroque doily swirly patterns result from the xy^2 terms in

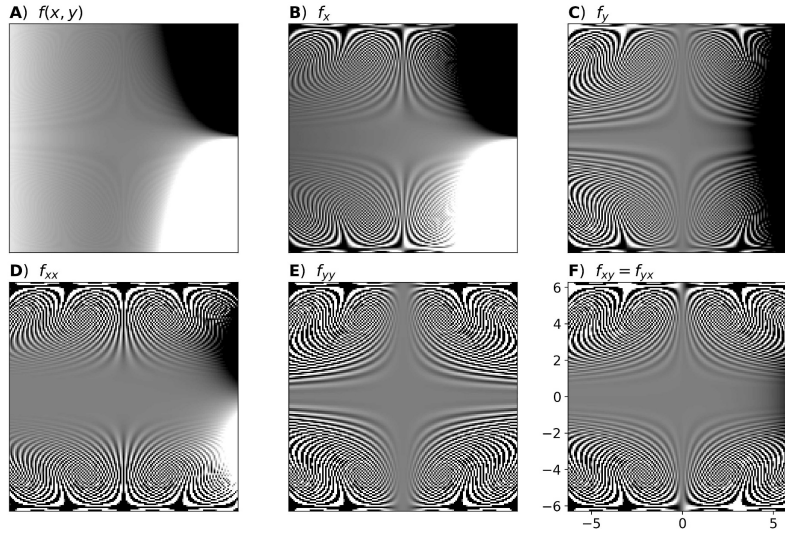


Figure 10.9: Equations 10.36-10.42 visualized.

the trig functions: those values get really large really quickly, and so the frequency of the sine waves increases rapidly. And the extreme dark/light patches to the right (positive x) result from the addition of the exponential term, which, when multiplied by y for some of the functions, flips the sign.

10.5 Gradients and gradient fields

What do you think of when you read the word "gradient"? Perhaps you think of a color gradient (color palettes that change smoothly from darker to lighter colors) or a nutrient gradient (plant roots grow towards richer soil). Maybe you think of "gradient descent," which is one of the most important algorithms in modern machine-learning and nonlinear optimization.

In the context of multivariable calculus, a *gradient* is a way to organize the partial derivatives of a function in a compact form. Consider that so far, I've been treating f_x and f_y as if they are separate mathematical objects. On the one hand, they are distinct from each other, so it does make sense to speak of them as

separate functions. But on the other hand, the partial derivatives are linked because they are derived from the same "parent" function $f(x, y)$.

A gradient is simply the partial derivatives organized into a vector. A *vector* is an organized list of numbers or functions. The gradient vector of a two-variable function looks like this:

$$\nabla f = \begin{bmatrix} f_x \\ f_y \end{bmatrix} \tag{10.43}$$

The triangle ∇ is called the "nabla" symbol, and is often pronounced as "del." It is simply a vector with one element for each partial derivative. If the function has five variables, then ∇f would be a 5-element vector.

Why do we need ∇f? Storing the partial derivatives in one mathematical object instead of separate equations has several advantages, including (**1**) simplifying the notation and equations, (**2**) allowing for linear algebra operations to be performed on the collection of partial derivatives (e.g., matrix-vector multiplication), and (**3**) identifying directions in the function that point towards maximal increases in the local landscape.

10.5.1 Gradient fields

I'm now going to expand on that third advantage of ∇f (finding directions that point towards local increases).

Recall that the derivative encodes the slope of the tangent line at each point in a function. The partial derivative is the slope of the tangent line along one variable for each value of the other variables.

Earlier in this chapter, I visualized the partial derivatives as a pair of images, in which the color value at each pixel encoded the

tangent line slope along direction x while holding y constant, or along direction y while holding x constant. An alternative way to visualize the partial derivatives is by drawing arrows on top of the original function, with the direction of the arrow indicating the differentiated variable (that is, left/right for negative/positive f_x and down/up for negative/positive f_y) and the length of the arrow encoding the magnitude (absolute value) of the derivative at each point.

Let's see an example. Figure 10.10 shows a function and its two partial derivatives as images. This is not an example of gradients or of visualizing partial derivatives as overlying arrows, but I want you to see what the images look like for comparison.

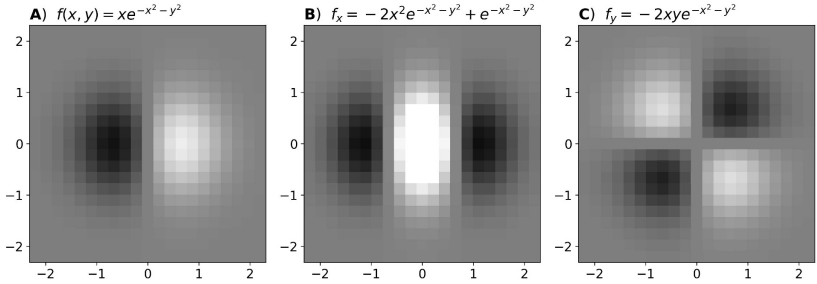


Figure 10.10: A function and its two partial derivatives.

Now consider Figure 10.11, which shows the same information but in a different format. Here are some observations to ponder:

Background image

The background images are the same ($f(x, y)$) in all three panels of Figure 10.11. Thus, all images show the function itself.

Panel A

The arrows point only left or right. The left-pointing arrows correspond to black pixels in Figure 10.10B (negative f_x) while the right-pointing arrows correspond to white pixels in 10.10B (positive f_x). The length of the arrow corresponds to the magnitude of the partial derivative, and is close to zero for the gray pixels in Figure 10.10B. (The arrow lengths are normalized for visualization, so you can interpret relative lengths but not absolute lengths).

Notice that the arrows point away from darker regions in the

image (corresponding to negative values in the function) and towards lighter regions in the image (corresponding to positive values in $f(x, y)$).

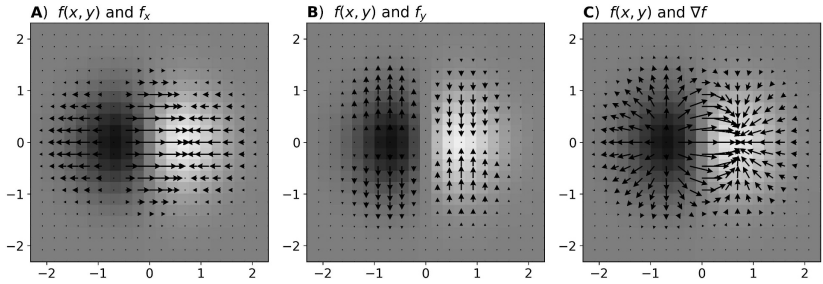


Figure 10.11: A function and its two partial derivatives.

Panel B

Here you see the partial derivatives along y, so the arrows point only up or down. Interpreting the sign and length of the arrows is the same as in panel A, but this corresponds to f_y, which is panel C in Figure 10.10. Again, the arrows point in the direction of lighter image pixels, which correspond to larger function values.

Panel C

No offense to panels A and B, but panel C shows the real gem of ∇f, and the way you will nearly always see gradient fields. Each (x, y) coordinate has two partial derivatives (f_x, f_y), and their combination determines the angle and the length of the gradient at each point.

Again: The arrows point away from negative values of the function, and towards larger positive values of the function, and the length of each arrow is related to the steepness of the function. The difference from the other panels is that the the arrows here show the direction of maximal function value ascent, and are not constrained to point only along x or y.

∇f is quite profound. It gives us a way to climb to the nearest peak of the function simply by following the gradient. This is the basis for many optimization algorithms used in engineering and machine-learning. All you need to do is represent your goal (e.g., maximize profit, or minimize energy expenditure) as a differentiable function, and then you can find the maximum or minimum

points of the function by following the gradient. In different contexts, the objective might be to maximize profit or usable volume of a spaceship, or the objective might be to minimize the error in music compression or the electricity consumed in a factory.

It is important that you understand what those arrows mean and where they come from, so I will explain it again (apologies for the redundancy if you already understand it): There are two partial derivatives matrices (one for f_x and one for f_y), and each element in each matrix is a number that encodes the slope of the tangent line along one variable for a fixed value of the other variable. When a two-variable function is evaluated at specific coordinates (that is, $Z = f(x, y)$ for a grid of X and Y coordinate values), then each matrix element $_{i,j}$ has two numbers corresponding to partial derivatives $f_{x_{i,j}}$ and $f_{y_{i,j}}$. Imagine a mini-coordinate system at each (x, y) location with the axes corresponding to f_x and f_y. Then imagine drawing an arrow from the origin of that mini-coordinate system to the coordinates given by the values of the partial derivatives at $_{i,j}$. That's the gradient ∇f at that point (Figure 10.12).

This paragraph refers to Figure 10.12.

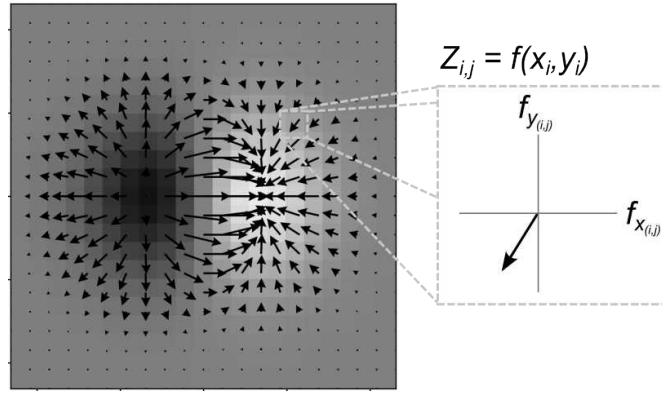


Figure 10.12: A function $Z = f(x,y)$ has, at each indexed location (i,j) (indicated with the gray dashed box), two partial derivatives. The gradient at the i,j coordinate is represented as an arrow that points from the origin of that space to the coordinates given by the partial derivatives f_x and f_y at that point.

Each arrow therefore contains two pieces of information that are used in applications: the direction (points towards locally larger values in the function) and the length of the arrow (the steepness

with which the function changes along that direction; it is also called the "magnitude" of the gradient).

In theory, there is an infinite number of arrows because the functions are (in theory) infinitely discretizable (as Δx and $\Delta y \to 0$). But for visualization, a manageable number of arrows is shown; for example, Figure 10.12 contains 441 arrows coming from a 21×21 grid. I hope that's clear. You'll have several opportunities to gain hands-on experience with gradients in the exercises.

10.5.2 Gradients and slopes

If the derivative of a single-variable function is the slope of the tangent line at each point, then what is the geometric interpretation of the gradient of a two-variable function?

The gradient of a function at each point is the "slope" of the tangent plane at each point, which is described as the tilt of the plane at the point where it touches the function. You can see an example of this for the function $f(x, y) = x^2 + y^2$ in Figure 10.13. The direction and steepness of the tilt of that plane is encoded in the angle and length of the arrows in the previous figures.

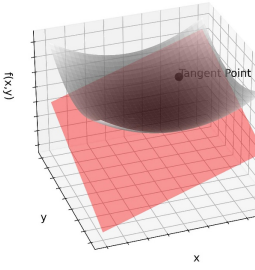


Figure 10.13: The tangent plane that touches a function at a point.

In theory, this concept extends to any number of dimensions. For example, a 200-variable function has a 200-element gradient vector, which describes a 200-dimensional hyperplane touching the 201-dimensional surface floating on top of a 200-dimensional coordinate system... oh my, I'm getting a headache. The point is that geometry is great for visualizing 1D and 2D functions, and above that, it's best to follow the algebra.

10.5.3 Jacobian matrix for first-order partial derivatives

You now know that the partial derivatives of a multivariable equation can be stored in a vector. If you have more than one equation using the same variables (for example, $f(x, y)$ and $g(x, y)$), then

the collection of all partial derivatives for all functions in that set can be organized into a matrix with one row for each equation.

This matrix is called a Jacobian matrix. To motivate the Jacobian, consider the following two equations, each with three variables:

$$f(x_1, x_2, x_3) = 4x_1 + x_1x_2 - \sqrt{x_3}e^{x_2} \tag{10.44}$$

$$g(x_1, x_2, x_3) = x_2^2x_3 + \cos(x_1x_2 + x_3) \tag{10.45}$$

The expressions below show all the partial derivatives.

$$\frac{\partial f}{\partial_{x_1}} = 4 + x_2 \tag{10.46}$$

$$\frac{\partial f}{\partial_{x_2}} = x_1 - \sqrt{x_3}e^{x_2} \tag{10.47}$$

$$\frac{\partial f}{\partial_{x_3}} = -\frac{e^{x_2}}{2\sqrt{x_3}} \tag{10.48}$$

$$\frac{\partial g}{\partial_{x_1}} = -x_2 \sin(x_1x_2 + x_3) \tag{10.49}$$

$$\frac{\partial g}{\partial_{x_2}} = 2x_2x_3 - x_1 \sin(x_1x_2 + x_3) \tag{10.50}$$

$$\frac{\partial g}{\partial_{x_3}} = x_2^2 - \sin(x_1x_2 + x_3) \tag{10.51}$$

That's a lot of lines to write out. The idea of the Jacobian is to organize those expressions into one matrix. In this case, it will be a 2×3 matrix with two rows corresponding to the two functions, and three columns corresponding to the three variables. The Jacobian matrix is indicated using the capital bold-faced **J**. For the example above, it looks like this:

$$\begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} & \frac{\partial f}{\partial x_3} \\ \frac{\partial g}{\partial x_1} & \frac{\partial g}{\partial x_2} & \frac{\partial g}{\partial x_3} \end{bmatrix} = \begin{bmatrix} 4 + x_2 & x_1 - \sqrt{x_3}e^{x_2} & -\frac{e^{x_2}}{2\sqrt{x_3}} \\ -x_2 \sin(x_1x_2 + x_3) & 2x_2x_3 - x_1 \sin(x_1x_2 + x_3) & x_2^2 - \sin(x_1x_2 + x_3) \end{bmatrix}$$

There's a lot going on in that matrix, which is why it's convenient to refer to it as **J** instead of writing out all six partial derivatives. The value of any particular set of numbers is expressed as $\mathbf{J}(x_1, x_2, x_3)$. For example,

$$\mathbf{J}(1,2,3) = \begin{bmatrix} 6 & -\sqrt{3}e^2 + 1 & -\sqrt{3}e^2/6 \\ -2\sin(5) & 12 - \sin(5) & 4 - \sin(5) \end{bmatrix} \tag{10.53}$$

More generally, a Jacobian matrix has size $m \times n$ for m equations and n variables.

10.5.4 Hessian matrix for second-order partial derivatives

Let's return to considering one multivariable equation, but now considering its higher-order partial derivatives: The two first-order partial derivatives can be organized into a two-element vector. Is it possible to organize the second-order partial derivatives into a four-element vector?

The answer is *yes*, you could put the four second-order partial derivatives into a vector. But for higher-order derivatives, the appropriate organization is a *matrix*, in which each element in the matrix is the second partial derivative.

This matrix is called a *Hessian matrix*:

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} f_{xx} & f_{yx} \\ f_{xy} & f_{yy} \end{bmatrix} \tag{10.54}$$

Let's do an example problem. Consider the equation below.

$$f(x,y) = 4x\sqrt{xy} + \ln(y^4)e^{\cos(\pi x)} \tag{10.55}$$

Please take a moment to calculate the four partial derivatives (and check that $f_{xy} = f_{yx}$), and organize those into the Hessian **H**. Remember to use the chain rule! My answers are below.

I'll start by showing the four second-order partial derivatives.

$$f_{xx} = \pi^2 e^{\cos(\pi x)} \log\left(y^4\right) \sin^2(\pi x) - \pi^2 e^{\cos(\pi x)} \log\left(y^4\right) \cos(\pi x) + \frac{3\sqrt{xy}}{x} \tag{10.56}$$

$$f_{yy} = \frac{-x\sqrt{xy} - 4e^{\cos(\pi x)}}{y^2} \tag{10.57}$$

$$f_{xy} = f_{yx} = \frac{3\sqrt{xy} - 4\pi e^{\cos(\pi x)} \sin(\pi x)}{y} \tag{10.58}$$

Quite a lot of terms in the second derivatives... math isn't always so compact. That's why it's convenient to organize functions into vectors and matrices. Here's how it looks in a matrix:

$$\mathbf{H} = \begin{bmatrix} \pi^2 e^{\cos(\pi x)} \log\left(y^4\right) \sin^2(\pi x) - \pi^2 e^{\cos(\pi x)} \log\left(y^4\right) \cos(\pi x) + \frac{3\sqrt{xy}}{x} & \frac{3\sqrt{xy} - 4\pi e^{\cos(\pi x)} \sin(\pi x)}{y} \\ \frac{3\sqrt{xy} - 4\pi e^{\cos(\pi x)} \sin(\pi x)}{y} & -\frac{x\sqrt{xy} + 4e^{\cos(\pi x)}}{y^2} \end{bmatrix} \tag{10.59}$$

Now that we have the matrix, we can simply refer to it as $\mathbf{H}$, and we can obtain a particular value of $\mathbf{H}$ for particular values of x and y. For example,

$$\mathbf{H}(1,2) = \begin{bmatrix} 3\sqrt{2} + \frac{\pi^2 \log(16)}{e} & \frac{3\sqrt{2}}{2} \\ \frac{3\sqrt{2}}{2} & -\frac{1}{e} - \frac{\sqrt{2}}{4} \end{bmatrix} \approx \begin{bmatrix} 14.3 & 2.1 \\ 2.1 & -0.7 \end{bmatrix} \tag{10.60}$$

Hessian matrices are used, for example, in geometry to calculate the curvature of 3D objects. You may encounter Hessian matrices if your future studies or work applications include image processing or multivariable optimization.

The Hessian matrix of a two-variable function is of size 2×2. A three-variable function has a 3×3 Hessian, and so on. This is just

for second-order derivatives. The third-order derivatives would be organized into a 3-way tensor (a cube).

10.6 Gradient descent in 2D

In Chapter 6 I introduced you to gradient descent in one variable. If you only vaguely remember 1D gradient descent, you might want to re-read or at least skim Section 6.3.3 (page 231) and Exercise 6.6 before proceeding here.

As a brief reminder, the idea of gradient descent is to find the minimum of a function using an iterative optimization algorithm. The algorithm starts by guessing a minimum (typically as some random point in the function). Then, the derivative of the function at that point is evaluated, and the guess of the minimum is updated to be itself minus the derivative, scaled by the learning rate. This update procedure repeats until a criterion is reached, which is either a pre-specified number of iterations or when the magnitude of the gradient goes below some threshold. This algorithm is guaranteed to move towards decreasing function values, although it is not guaranteed to find the global minimum if there are local minima, zero-derivative regions, or discontinuities.

How do you extend gradient descent to two variables? It's really easy! You simply repeat gradient descent for each variable in the function. For a two-variable function, the key update equations would look like this:

$$x_0 \leftarrow x_0 - \eta f_x(x_0, y_0) \tag{10.61}$$

$$y_0 \leftarrow y_0 - \eta f_y(x_0, y_0) \tag{10.62}$$

But now that you know about storing partial derivatives in vectors, you might have guessed that these two "separate" equations

can be bundled together into a vector equation. That makes the math and code more compact.

Remember that η is the learning rate, which slows and stabilizes learning.

$$\mathbf{z}_0 \leftarrow \mathbf{z}_0 - \eta \nabla f(\mathbf{z}_0) \tag{10.63}$$

$$\mathbf{z}_0 = (x_0, y_0) \tag{10.64}$$

You'll get to implement gradient descent in 2D in Exercises 7 through 9.

10.7 Exercises

10.1. This first exercise is not about calculus per se, but will help you gain familiarity with creating, visualizing, and interpreting matrices in Python. Start by creating a 5×10 matrix in which the element of matrix $\mathbf{A}$ in the i^{th} row and j^{th} column is defined by the following equation (i and j are zero-based indices):

$$\mathbf{A}_{i,j} = 3i - \frac{4(j-5)^2}{17} \tag{10.65}$$

The resulting matrix comprises floating-point numbers, but print the matrix as integers. Your matrix should look like the following:

```
[[-5 -3 -2  0  0  0  0  0 -2 -3]
 [-2  0  0  2  2  3  2  2  0  0]
 [ 0  2  3  5  5  6  5  5  3  2]
 [ 3  5  6  8  8  9  8  8  6  5]
 [ 6  8  9 11 11 12 11 11  9  8]]
```

Now for the visualization. Use `plt.imshow` to create Figure 10.14. Notice that the matrix *columns* correspond to x-axis coordinates while matrix *rows* correspond to y-axis coordinates.

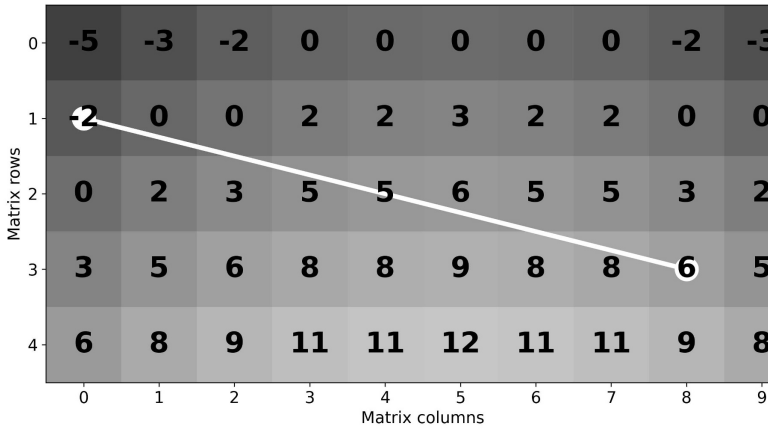


Figure 10.14: Results for Exercise 1.

10.2. The purpose of this exercise is to gain familiarity with creating matrices representing the inputs to, and outputs from, a two-variable function. But before getting to Python, write down the following equation and calculate the two partial derivatives — by hand (don't use `sympy`!).

$$f(x_1, x_2) = \sin\left(x_1^2\right) + \cos\left(\sqrt{x_2}\right) x_1 x_2 \tag{10.66}$$

Next, write three Python functions that take x_1 and x_2 as input, and output the function and the two partial derivatives (one Python function for each math function). Write the functions using `numpy` (that is, don't differentiate in `sympy`). Check that your differentiation and implementation are correct by producing the output shown in Figure 10.15 (I rounded the numbers for readaibility).

$(\pi, \pi/2) = 1.1102$

$_{x_1}(\pi, \pi/2) = -5.1814$

$_{x_2}(\pi, \pi/2) = -0.8896$

Figure 10.15: Confirmation of accuracy in Exercise 2.

Now to create the results matrices. Start by defining a domain and resolution to use for x_1 and x_2, and be mindful of domain restrictions imposed by the function definition. Then, initialize the three results matrices (Z, Z_{x_1}, and Z_{x_2}) using `np.zeros`. Write a double for-loop to define the matrices element-wise.

Next, repeat the matrix creation but using `np.meshgrid` without for-loops. At this point, you will have six matrices in six Python variables. But there should only be three unique matrices, because the element-wise and matrix-wise calculations should be identical. Confirm that they are identical by visualizing and by calculating the absolute value of the differences between the corresponding matrices.

Your resulting visualization should look like Figure 10.16.

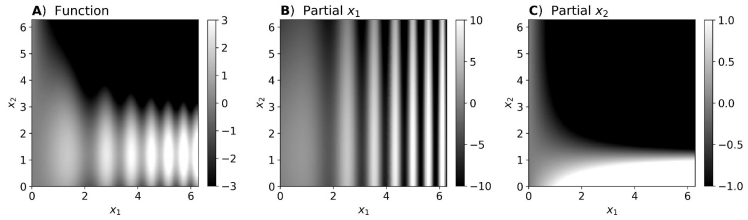


Figure 10.16: Results for Exercise 2.

10.3. Although flat 2D images are useful for conveying accurate information about a matrix, 3D surface renderings can be quite beautiful. The goal of this exercise, therefore, is to generate interactive 3D renderings of functions using the `plotly` library. Import the library using the following code:

```
import plotly.graph_objects as go
```

I haven't introduced using this library so you might need to search online or ask ChatGPT for some help. If you want a hint, check the footnote[7].

Your goal for this exercise is to implement and visualize the three functions shown in Figure 10.17. Below are the equations for each function; note that the function in panel C has additional parameters to define.

$$\mathtt{A} = \sin(x + y^2) \tag{10.67}$$

[7] `fig = go.Figure(data=[go.Surface(x=xx,y=xx,z=Z)])`

$$\mathtt{B} = x^2 + \sqrt{|y|} \tag{10.68}$$

$$\mathtt{C} = \sin(2\pi f U) e^{-(x^2+y^2)/(2\sigma^2)} \tag{10.69}$$
$$U = \cos(\phi)x + \sin(\phi)y$$
$$f = .05$$
$$\phi = \pi/4$$
$$\sigma = 3\pi$$

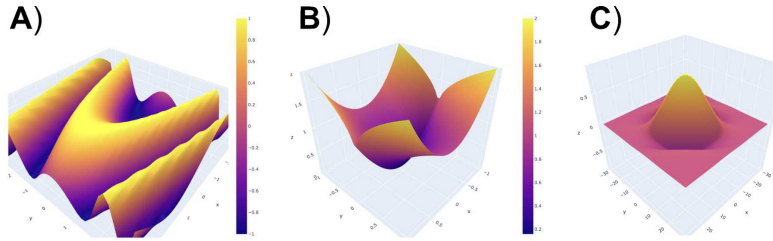


Figure 10.17: Results for Exercise 3. ***Note*****: These were created as three separate figures in Python, and I grouped them afterwards in Inkscape.**

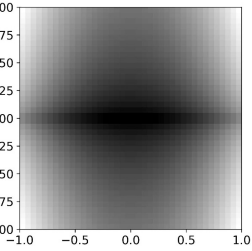


Figure 10.18: The function in 10.17B as a flat gray image. Looks kinda boring compared to the 3D surface...

$$z, y) = xe^{-x^2-y^2}$$
$$x, y) = -2x^2e^{-x^2-y^2} + e^{-x^2-y^2}$$
$$x, y) = -2xye^{-x^2-y^2}$$
$$(x, y) = 4x^3e^{-x^2-y^2} - 6xe^{-x^2-y^2}$$
$$(x, y) = 4xy^2e^{-x^2-y^2} - 2xe^{-x^2-y^2}$$
$$(x, y) = 4x^2ye^{-x^2-y^2} - 2ye^{-x^2-y^2}$$
$$(x, y) = 4x^2ye^{-x^2-y^2} - 2ye^{-x^2-y^2}$$

Figure 10.19: `sympy` output for Exercise 4.

Once you've created the plot, you can use the mouse to left-click-drag the surface around to different views, scroll to zoom, right-click-drag to translate, and so on. The `plotly` library has many options for creating and exporting fantastic interactive plots. If you like this basic functionality, then feel free to continue exploring the options!

By the way, Figure 10.18 shows panel B as a flat image. It really doesn't look nearly as captivating as the surface rendering.

10.4. `sympy` can calculate partial derivatives of a multivariable function. Write code to calculate and print the function and its derivatives shown in Figure 10.19.

Next, use `sym.plotting.plot3d` to generate 3D surfaces of the function and its derivatives, as in Figure 10.20.

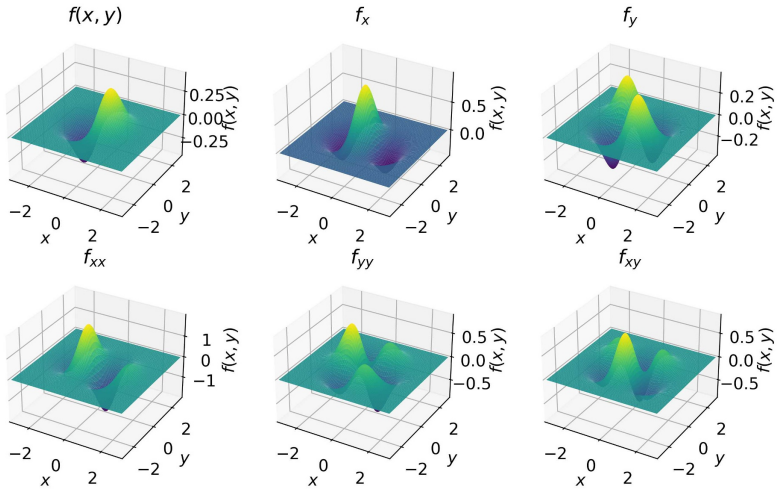


Figure 10.20: `sympy` graphing for Exercise 4.

As I've mentioned multiple times before in this book, `sympy` plotting is great to have a quick look at a symbolic expression, but for customizations it's best to evaluate the function into `numpy` format and use `matplotlib`.

10.5. So far in this chapter, you've calculated multivariable derivatives by hand and using `sympy`. In previous chapters, you also learned how to calculate the empirical derivative (that is, the numerical difference) in `numpy`. That is useful for applications in which the true derivative cannot be analytically determined.

Your goal here is to create an image like Figure 10.16 from Exercise 2, but based on empirical numerical differentiation. Start by creating the matrix corresponding to Equation 10.66 (reprinted below for convenience). Next, calculate the numerical partial difference by implementing Equations 10.24-10.25 in `numpy`. Set $h = .01$. To check the accuracy of your plot, use the same colorscaling as used in Figure 10.14.

$$f(x_1, x_2) = \sin(x_1^2) + \cos(\sqrt{x_2})x_1x_2 \tag{10.66}$$

I don't show the figure here because it looks indistinguishable from Figure 10.16, but it is created in the online Python code solution.

Finally, explore other values of h. Do the empirical results still match the analytic results for $h = .1$? How about $h = 1$? What are some reasons to prefer larger vs. smaller values of h?

10.6. In this exercise you will explore limits of two-variable functions. Begin by implementing the following function in `sympy`, then use `sympy` to obtain the two partial derivatives, use `sym.lambdify` to evaluate those functions in `numpy`, and visualize all three using `matplotlib` as shown in Figure 10.21.

$$f(x,y) = \frac{\sin\left(\sqrt{x^2+y^2}\right)}{\sqrt{x^2+y^2}} \tag{10.70}$$

The third and fourth equations in Figure 10.21 appear identical, but they are calculated differently in `sympy`: First the limit for x then for y (third equation), vs. first the limit for y then for x (fourth equation).

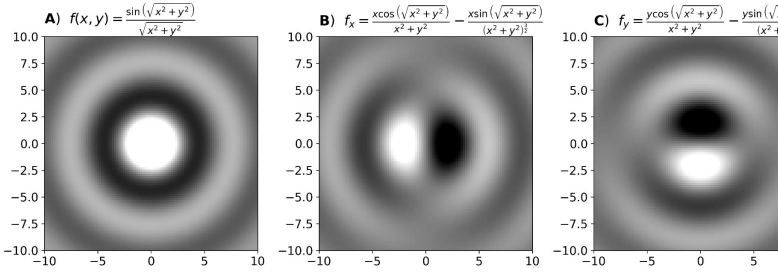


Figure 10.21: Graph for Exercise 6.

A quick inspection of Equation 10.70 shows that the function is undefined for $x = y = 0$. (Indeed, if you create the images using an odd number of points, you may get a warning message in Python about invalid results.) However, we can ask about the limit of the function as it approaches the

origin. Use `sym.limit` to calculate and print out the limit at two (x, y) points, as shown in Figure 10.22.

$$\lim_{x \to 1} f(x, y) = \frac{\sin\left(\sqrt{y^2 + 1}\right)}{\sqrt{y^2 + 1}}$$

$$\lim_{y \to \pi} f(x, y) = \frac{\sin\left(\sqrt{x^2 + \pi^2}\right)}{\sqrt{x^2 + \pi^2}}$$

$$\lim_{(x,y) \to (1,\pi)} f(x, y) = \frac{\sin\left(\sqrt{1 + \pi^2}\right)}{\sqrt{1 + \pi^2}} \approx -0.0469$$

$$\lim_{(x,y) \to (1,\pi)} f(x, y) = \frac{\sin\left(\sqrt{1 + \pi^2}\right)}{\sqrt{1 + \pi^2}} \approx -0.0469$$

Figure 10.22: Screenshot of `sympy` output for Exercise 6.

10.7. In this chapter, I showed you how to calculate and visualize gradient fields "manually" by separately calculating the two partial derivatives matrices, either analytically via `sympy` or empirically via `numpy`. The goals of this exercise are to learn how to obtain the gradient field via the function `np.gradient()`, and to learn how to calculate and work with the magnitude of the gradient field.

Using only `numpy` (with `np.gradient`) and `matplotlib`, produce Figure 10.23. Panel A shows the same result as shown in Figure 10.11C ($f(x, y) = xe^{-(x^2+y^2)}$). Panel B shows the same function, but the gradients have been normalized so that the gradient field shows only directions with every arrow having the same length. You will need to figure out how to normalize the gradients such that the magnitudes all equal 1. If you want a hint on normalizing, check the footnote[8].

[8] The magnitude of a vector is the Euclidean distance $\sqrt{f_x^2 + f_y^2}$. Calculating and dividing each element in the gradient matrix by its magnitude will normalize the magnitude to 1.

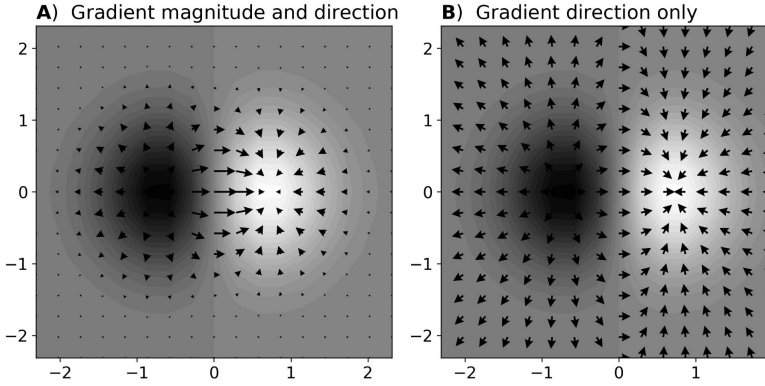


Figure 10.23: Results for the first part of Exercise 7.

What is the meaning of the gradient magnitudes? They encode how steeply the function moves in the direction of maximal ascent. If the function is a mountainous region and you are trying to get to the nearest peak to get cell phone reception, then the angle of the gradient tells you which direction to climb, and the magnitude tells you how much altitude you'll gain with each step.

The next part of this exercise is to visualize the magnitudes of the gradient before and after normalization. Write code to produce Figure 10.24. Notice that the grayscale intensity in panel A maps onto the length of the arrows in Figure 10.23A (and also that it looks like a pig snout; I promise that's just serendipity and had no influence over me choosing this function). Panel B, in contrast, is all white — you might be concerned that there's a coding bug, but it is accurate: this image shows that the *normalized* gradient magnitudes are all one.

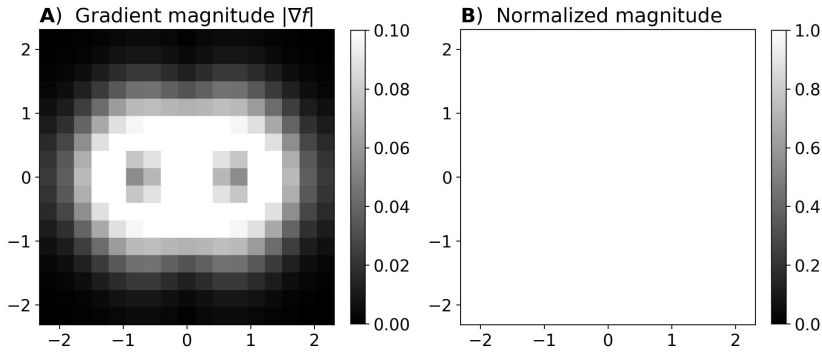


Figure 10.24: Results for the second part of Exercise 7.

The final work for this exercise is to inspect the source code to see how the `np.gradient` function works (hint: run `np.gradient??` in a Jupyter notebook). You will discover that it is simply implementing Equations 10.24-10.25, just like what you did in Exercise 5! (There is more code in `np.gradient`, but most of it is housekeeping and error-checking. Being able to find the key lines in source code is an important skill worth practicing.)

Two points of discussion before closing this exercise: (**1**) As I stressed several times before in this book (and in many of my other books and online courses): it is important for you to take the time to directly implement math in code (e.g., as in Exercise 5), but once you understand the concepts, it's usually better to use established functions like `np.gradient`. (**2**) Separating the magnitude of the gradient from its direction is important for optimization methods in machine-learning. In fact, the current best deep-learning optimizers (Adam and RMSprop) are basically minor modifications to gradient descent in which the learning rate is scaled by the recent history of the gradient magnitudes.

10.8. In this exercise, you will implement 2D gradient descent. You can consider starting this exercise by copying the code from Exercise 6.7, but be mindful of what needs to be changed to get it to work here.

This function is part of the MATLAB logo, and is called the "peaks" function.

Below is the equation for the function. The first step in this exercise is to translate this from a formula into `sympy` code, and then use `sym.lambdify` to get three python functions for Equation 10.71 and its two partial derivatives.

$$f(x,y) = 3\,(1-x)^2\,e^{-x^2-(y+1)^2} - \left(-10x^3 + 2x - 10y^5\right)e^{-x^2-y^2} - \frac{e^{-y^2-(x+1)^2}}{3} \tag{10.71}$$

Next, evaluate the function and its partial derivatives on a grid to visualize as in Figure 10.25. In the function (Figure 10.25A), white is positive and black is negative, so the goal of gradient descent will be to find the darkest pixel in the image.

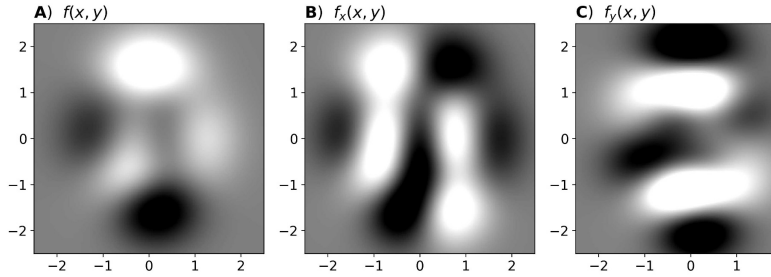


Figure 10.25: Results for the first part of Exercise 8.

In applications, gradient descent starts at a random location, but starting at a fixed point here will allow you to confirm your code by replicating my results.

Now for gradient descent. Have your code start from $(x_0, y_0) = (-.4, -1)$ using $\eta = .01$ and 1000 updates (that is, 1000 iterations inside the for-loop). If you use these parameters and you've correctly implemented the equations, you should replicate my results below:

```
Start location: (-0.40,-1.00)
Final location: ( 0.23,-1.63)
```

The final part of this exercise is to modify the code so that the estimates of $\mathbf{z}_0$ (from Equation 10.63) at each iteration in the for-loop are stored in a 1000×2 `numpy` array. This is a matrix of (x, y) coordinates that you can plot to see how

gradient descent guided the particle down from its starting point to its final position. My results are shown in Figure 10.26.

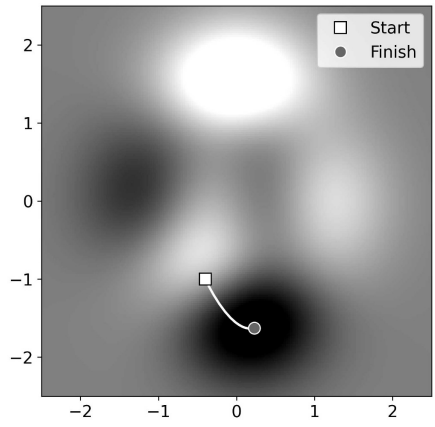


Figure 10.26: Results for the second part of Exercise 8.

10.9. This exercise is a bit open-ended. I want you to spend some time exploring successes and failures of gradient descent using the code for the previous exercise.

Begin by exploring the following questions: What happens when you set the start location to be the "north" peak at around (0,1.8)? What happens if you set the start location to be the origin of the graph? How about when you start towards the side (e.g., (2,0))?

Here's a question: *Does gradient descent always work*? There are a few ways you can interpret this question, and I provide two answers in the online code.

10.10. This exercise is simple: Modify the code from Exercise 8 to implement gradient *ascent* instead of *descent*.

10.11. The purpose of this exercise is to explore the Hessian matrix of a function. I will introduce some concepts that are taught in linear algebra (e.g., the determinant of a matrix), so if you're unfamiliar with matrix operations, you can fo-

cus on implementing the math without worrying about the theory. There are six parts to this exercise.

First: Implement Equation 10.55 (reprinted below) in `sympy`.

$$f(x, y) = 4x\sqrt{xy} + \ln(y^4)e^{\cos(\pi x)} \tag{10.55}$$

Second: Determine the domain of this function. My answer is in the online code.

Third: Calculate the first- and second-order partial derivatives, and visualize them as in Figure 10.27. I used a visualization range of .01 to π.

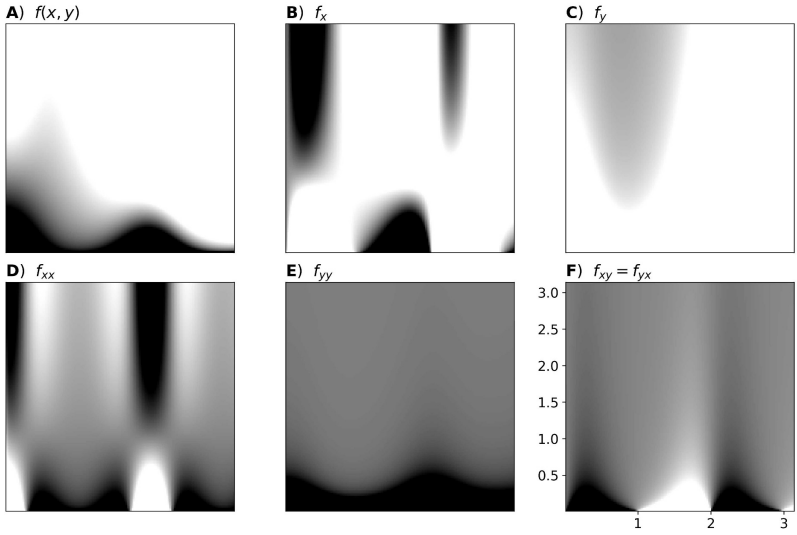


Figure 10.27: Results for part 3 of Exercise 11.

Fourth: Create the Hessian of this matrix as a `Matrix` object in `sympy` (hint: `sym.Matrix`). Then use the substitution method to display the following:

$$\mathbf{H}(1,2) = \begin{bmatrix} 3\sqrt{2} + \frac{\pi^2 \log(16)}{e} & \frac{3\sqrt{2}}{2} \\ \frac{3\sqrt{2}}{2} & -\frac{1}{e} - \frac{\sqrt{2}}{4} \end{bmatrix} \tag{10.72}$$

Fifth: Figure out how to calculate the determinant of a matrix in `numpy`. The "determinant" is a number, calculated from all matrix elements, that provides information into the characteristics of the matrix. For a 2×2 matrix with elements $\left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right]$, a short-cut to calculate the determinant is $ad - bc$.

Find the `numpy` function that calculates the determinant and confirm the following (create the matrix as a `numpy` array, not a `sympy Matrix`):

$$det\left(\begin{bmatrix} 1 & 2 \\ 5 & -1 \end{bmatrix}\right) = -11 \tag{10.73}$$

Sixth: Create an image of Hessian determinants. Notice that each pair of particular (x, y) values that you used to

construct the bottom row of Figure 10.27 creates a 2×2 Hessian matrix ($\mathbf{H}(x_0, y_0)$). In a double for-loop over all (x, y) value pairs, calculate the determinant and store that in a new matrix[9]. Visualize that matrix as in Figure 10.28.

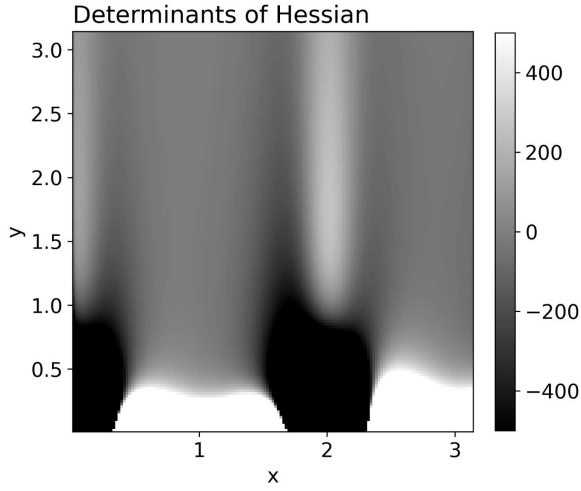


Figure 10.28: Results for the second part of Exercise 11.

The Hessian matrix is the 2D analog of the second derivative of a single-variable function, and therefore can be used to identify inflection points in multivariable functions, which in turn is used to identify local minima. I won't discuss Hessian determinant matrices further in this book, but you might use them if you study optimization or image processing.

[9]Tip: Consider lambdifying the Hessian matrix.

Chapter 11

Make differential art!

Calculus seems like such a serious topic. Many TV shows and cartoons depict nightmares of showing up to a calculus exam without having studied.

To be sure, calculus is an advanced topic in mathematics. It is challenging and requires serious effort to learn. A very small number of extraordinarily mathematically gifted people can simply absorb calculus like a sponge, but most of us (including me) need to struggle and push ourselves to gain proficiency.

But just because a topic is serious doesn't mean that *we* need to be serious all the time. And that leads me to this chapter.

The purpose of this chapter is to have some light-hearted enjoyment with visualizing concepts you've learned so far in this book. I will show images of math-inspired art and explain how you can recreate those images. You can code the images from scratch based on my description, or you can simply look at my code online and modify it as you like. There are no correct or incorrect answers, there are no mistakes, and there are no formulas to memorize or understand; there is only your personal aesthetic style to explore.

Will you learn new things in this chapter? I'm sorry to say that the answer might be *yes*. You might improve your math-coding skills, and you might learn some new `matplotlib` visualization tricks. You might even (accidentally) understand some of the math better. But that's not the goal. The goal is simply to make beautiful and interesting-looking pictures by playing around with math in Python.

Do you need to go through this chapter? Absolutely not. If you're in a hurry or are not interested in math-art, then please feel free to skip to the next chapter without hesitation or remorse. If you do work through this chapter, I hope you find it an enjoyable challenge, and I hope you take inspiration from my feeble attempts and continue making your own calculart.

11.1 Elf hat

Figure 11.1 is made by drawing tangent lines to the function $f(x) = -x^2$. You can see the function as the empty white space towards the bottom, and the purple "hat" is made from 300 tangent lines drawn between $x = -2$ and $x = 2$.

Start by creating a Python function that takes three inputs: a `sympy` expression for the function, the x_0 value at which to evaluate the tangent, and the x-axis bounds for the tangent line. Inside the Python function, differentiate the `sympy` expression, then calculate and return the y-axis values to draw the tangent line (see Chapter 5 for the algorithm to calculate and plot the tangent line).

I set the color and x-axis bounds of each tangent line to be a function of x_0 to produce Figure 11.1. I won't tell you the exact parameter values I used because I want you to flex your creativity muscles to create your own little masterpiece. Of course, you can check my online code for inspiration on implementing the color gradient.

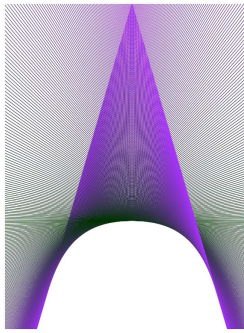


Figure 11.1: A neat-looking image created by drawing 300 tangent lines with different colors.

11.2 Elf hat to infinity (and beyond)

In geometry, a "normal" line is perpendicular to some other line or surface. The normal line to a tangent line meets the tangent at a 90° angle.

You can calculate the normal line simply by taking the negative reciprocal of the slope. That is, if the slope of tangent line is defined by $m = f'(x_0)$, then the slope of the line normal to the tangent is defined by $m = -1/f'(x_0)$. That very small adjustment to the code will transform Figure 11.1 into 11.2.

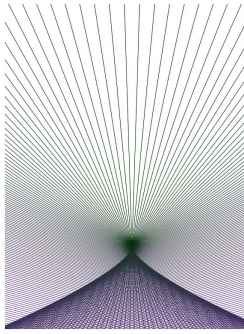


Figure 11.2: The elf walked on a road towards infinity with rays shooting out. Or something.

11.3 Acorn from Ooo

I was initially inspired for Figure 11.3 by the discussion from Chapter 2 about how the formula for a circle is not a *function*, although the top and bottom halves of a circle can be described by expressions that conform to the definition of a function. I thought it could look interesting to have different parameterized equations for the top and bottom halves, and use distinct colors for each of many lines.

Recall that the expression for a full circle is $r^2 = x^2 = y^2$, where r is the radius. The functions for the top (y_t) and bottom (y_b) curves for each of 201 lines in the restricted domain $x \in [\,-1, 1\,]$ is given by the following formulas (x_i is each unique value in x that produces each individual line i).

$$y_{b,i} = -\sqrt{2^2 - (x + x_i)^2} \tag{11.1}$$

$$y_{t,i} = \sqrt{(.1 + x_i^2)^2 - x^2} \tag{11.2}$$

The color of each line is defined by the following RGB code:

$$c_{b,i} = \left[\; .4\,,\, \frac{|x_i|}{2}\;,\; x_i^2 \;\right] \tag{11.3}$$

$$c_{t,i} = \left[\; .2\;,\; x_i^4\;,\; \frac{|x_i|}{2} \;\right] \tag{11.4}$$

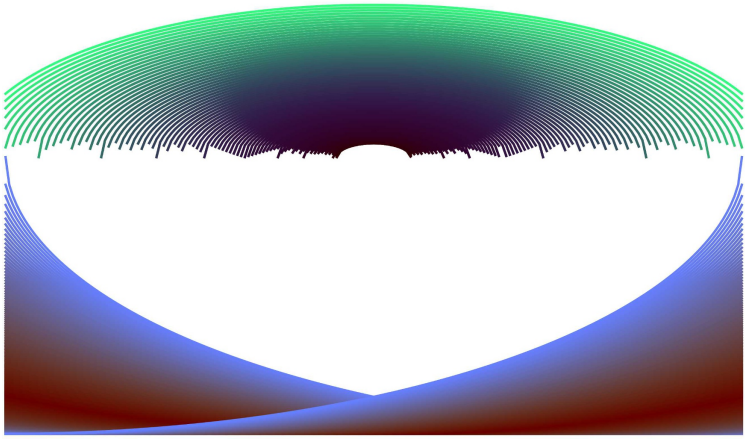


Figure 11.3: I initially had this image flipped on the y-axis and thought it looked like a Turkish-inspired dome over a grassy landscape, but a friend suggested to swap the top and bottom to make an acorn from the negative space. What do you think?

11.4 Smooth gray waves

I thought the function in Exercise 4.11 (Equation 4.62, repeated below) looked neat and wanted to spend more time exploring it.

$$f(\theta) = \theta^2 e^{-\theta^2} \ln\left(\theta^2\right) \sin(\theta) \tag{11.5}$$

I decided to shift one of the θ terms by a small amount, but couldn't decide which term to shift. So I shifted each one in a separate plot. Thus, the four subplots in Figure 11.4 show each of the four following equations, where ν (Greek letter "nu") is a scalar that ranges from $-.1$ to $.1$ in 20 steps, and the grayscale intensity of each line is defined by $.7 \times |\nu/.1|$.

$$f_1(\theta) = (\theta+\nu)^2 e^{-\theta^2} \ln\left(\theta^2\right) \sin(\theta) \tag{11.6}$$

$$f_2(\theta) = \theta^2 e^{-(\theta+\nu)^2} \ln\left(\theta^2\right) \sin(\theta) \tag{11.7}$$

$$f_3(\theta) = \theta^2 e^{-\theta^2} \ln\left((\theta+\nu)^2\right) \sin(\theta) \tag{11.8}$$

$$f_4(\theta) = \theta^2 e^{-\theta^2} \ln\left(\theta^2\right) \sin\left(\theta + \nu\right) \tag{11.9}$$

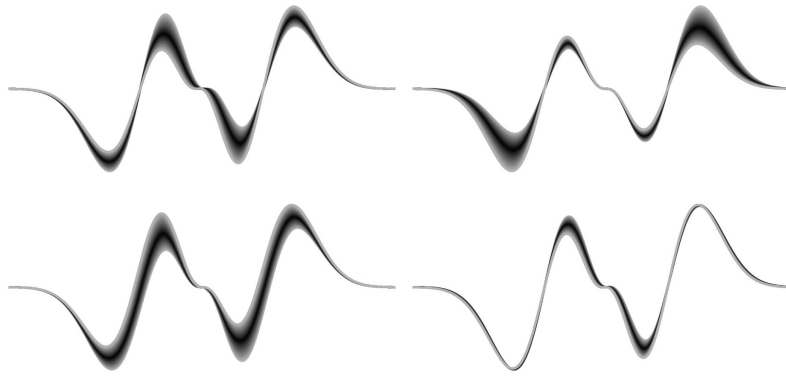


Figure 11.4: Simple but pleasant to look at.

11.5 Fun by diff

Dynamical systems are sometimes explored by creating a "phase diagram," which depicts the function on the x-axis and its derivative on the y-axis. In Chapter 17, I will refer to these kinds of plots as parametric curves, but you don't need to wait that long to appreciate the beauty that these plots can produce.

Start by differentiating $f(\theta)$ (without the shifting term) in Equation 11.5. If you want a challenge, you can differentiate it by hand, but you can also let `sympy` do the work (after all, this chapter is more about fun than work!). You should get the following:

$$f(\theta) = \theta^2 e^{-\theta^2} \log\left(\theta^2\right) \sin\left(\theta\right) \tag{11.10}$$

$$f'(\theta) = -2\theta^3 e^{-\theta^2} \log\left(\theta^2\right) \sin\left(\theta\right) + \theta^2 e^{-\theta^2} \log\left(\theta^2\right) \cos\left(\theta\right) + 2\theta e^{-\theta^2} \log\left(\theta^2\right) \sin\left(\theta\right) + 2\theta e^{-\theta^2} \sin\left(\theta\right) \tag{11.11}$$

Next, pick one of the function variants from the previous section for the x-axis coordinates, and the derivative of one of the function variants for the y-axis coordinates, and then plot the function (x-axis) by the derivative (y-axis). After some explorations, I chose to use f_2 and f_3'. In other words, each line in Figure 11.5 is given by the points $(x, y) = (f_2(x), f_3'(x))$ for each value of θ_i.

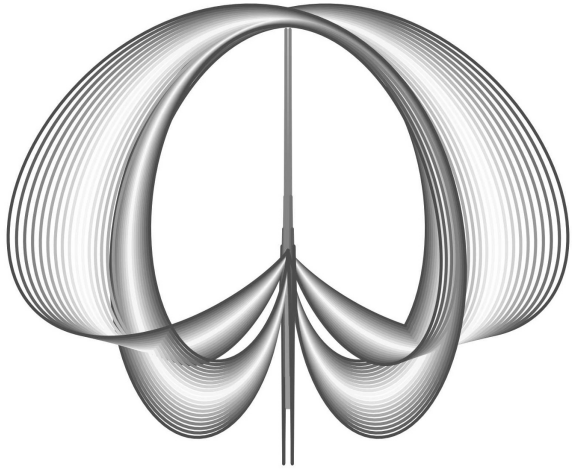


Figure 11.5: I wonder what this would look like if it were animated?

I find this result breathtakingly beautiful, and I hope you agree. It looks like a 3D object, although there is no "depth" dimension. I also hope you feel inspired by this approach, and explore other combinations of function variants and derivatives, or create your own function/derivative pairs.

11.6 Dancing petals

I will now guide you through creating Figure 11.6.

This figure shows two equations that vary according to a parameter. The two equations are:

$$f(x) = \frac{x}{x^2 + \gamma} \tag{11.12}$$

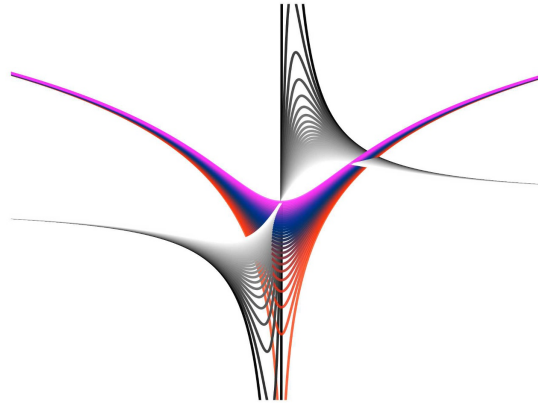


Figure 11.6: Two gracefully balanced functions.

$$g(x) = \frac{\ln(x^2 + \gamma)}{2} \tag{11.13}$$

γ is a scalar that ranges between .001 and 1 in 30 steps. Each line in the plot is a different value of γ. The RGB color codes follow these rules:

$$c_f = [\, \sqrt{\gamma},\ \sqrt{\gamma},\ \sqrt{\gamma}\,] \tag{11.14}$$

$$c_g = \left[\, (1 + \cos(2\pi\gamma))/2,\ .2,\ \gamma,\ 1 - (\gamma - .5)^2 \,\right] \tag{11.15}$$

The fourth element in c_g is the "alpha" parameter, which encodes transparency.

11.7 Radial curves

A "radial curve" is a general term in geometry to describe a curve whose relationship to the origin follows some particular function or has a particular constraint.

More on radial curves in Chapter 17!

Implement the following two expressions in `numpy` to create the coordinates for a radial curve for some positive α, using $\theta \in [\,-6\pi, 6\pi\,]$.

$$x = \theta^{\alpha} \cos^3(\theta) \tag{11.16}$$

$$y = \theta^{\alpha} \sin^3(\theta) \tag{11.17}$$

Each of the nine subplots in Figure 11.7 is created by setting α to be the integer index of the subplot (that is, the fifth subplot has $\alpha = 4$).

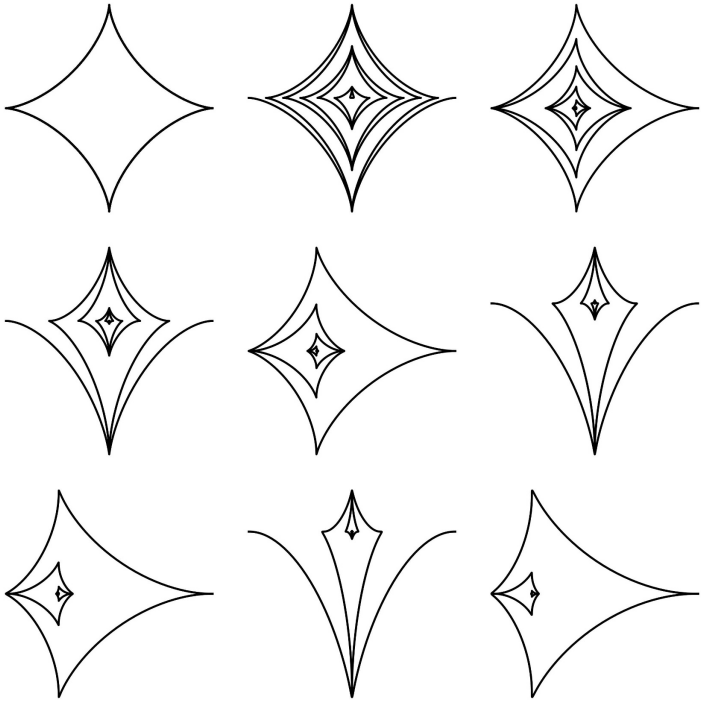


Figure 11.7: Some radial curves.

Typically, radial curves do not raise the trig terms to higher powers, but I think the curves look more interesting when cubed. You can try exploring these parametric equations — for example, try keeping the cosine term cubed and squaring the sine term.

11.8 Rose curves

Rose curves are created by having the radius of a circle defined by some combination of cosine and sine functions. I created Figure 11.8 using the following functions.

$$x = \cos(k\theta)\cos(\theta) \tag{11.18}$$

$$y = \cos(k\theta)\sin(\theta) \tag{11.19}$$

The values of k are printed in the titles of each subplot, and were created as nine linearly spaced numbers between 0 and 1.5, and for $\theta \in [\,0, 4\pi\,]$.

To create a color gradient, I plotted these curves not as lines but as individual dots using `plt.scatter`. The colors were defined using the following RGB vectors (Python variable `t` corresponds to θ in the equations above).

```
R = np.linspace(-.6,1,len(t))**2
G = np.ones(len(t))*.2
B = np.sin(np.linspace(0,2*np.pi,len(t)))/2+.5
```

11.9 Riemann's complex nondifferentiable ice cream

Riemann was a mathematician in the 19^{th} century who made important contributions to myriad areas of mathematics and physics — including integral calculus as you'll learn in the next several chapters — and whose work helped Einstein develop his theories of relativity. (Sadly, Riemann died of tuberculosis at age 39; we can never know what incredible gifts to humanity he could have

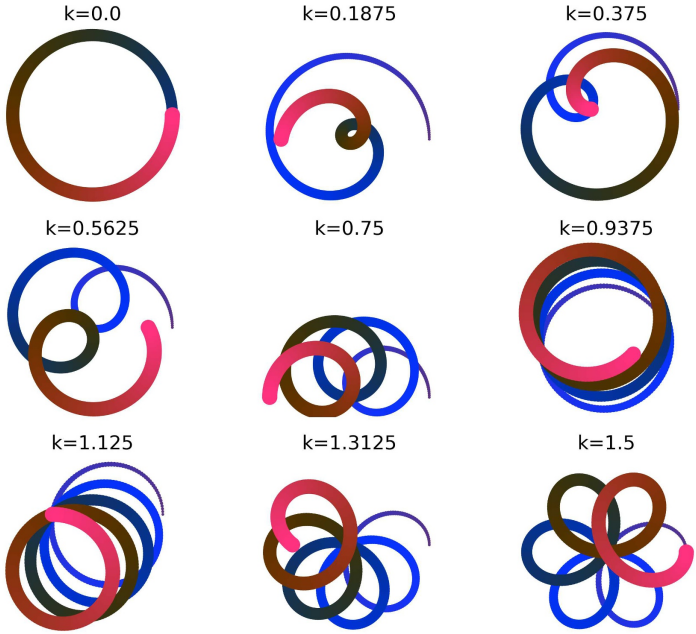


Figure 11.8: Some rose curves.

provided had he lived another 40 years — or if his maid hadn't thrown out his unpublished work after his death.)

Riemann developed several functions that are continuous but not differentiable; such functions are studied in advanced and more abstract calculus courses. Here we will simply admire the rugged beauty provided by this function.

Implement the following equation in Python to approximate a continuous-but-nondifferentiable function[1].

$$f(t) = \sum_{n=1}^{k} \frac{e^{i\pi n^2 t}}{i\pi n^2} \tag{11.20}$$

$i = \sqrt{-1}$

The formal definition sets $k = \infty$, but in the interest of computation time, you can use $k = 300$. Plotting the real part of the

[1] Riemann initially used a real-valued function and the complex-valued generalization was developed later, but we don't need to worry about the historical details here.

function by the imaginary part produces the thick black line in Figure 11.9. The light gray lines to create the shading effect are created by implementing the same function but with smaller values of k. I used 55 lines with k ranging from 5 to 300 in logarithmic steps.

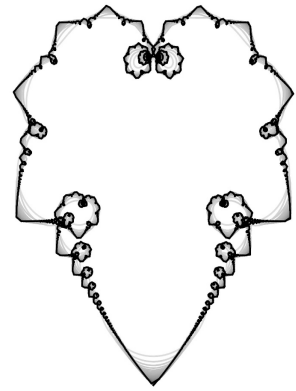


Figure 11.9: Ice cream cone? Maybe some kind of fractal cauliflower?

If you would like to explore other continuous nondifferentiable functions, you can search the Internet and translate the equations you find interesting into Python code. You can also look for the "Weierstrass function."

11.10 Mandelbrot set

Even if you're not familiar with the name, I'm sure you've seen pictures of the Mandelbrot set on t-shirts or mugs (Figure 11.10A). But what is the Mandelbrot set and how do you create it?

The Mandelbrot set is defined by iteratively updating a two-variable function and counting how many iterations it takes for the function at each (x, y) coordinate to exceed a particular threshold. Start by writing Python code to implement the following expression update in a for-loop over 40 iterations.

$$Z(x,y) \;=\; Z^2 + C \tag{11.21}$$

I've omitted the (x, y) from Z and C on the right-hand-side for simplicity. Z starts off as a matrix of zeros, and C is a complex matrix whose elements are defined by the `np.meshgrid` function for the real part going from -1.5 to .5 in 1000 steps, and the imaginary part going from -1 to 1 in 1000 steps. In other words, create $C = X + Y\sqrt{-1}$ where X and Y are the outputs of `np.meshgrid` that define the coordinates for the images.

In each iteration of the for-loop, find the indices where $|Z| > 2$ and set those indices in another matrix M to equal the iteration count. This matrix M is the Mandelbrot set, and can be visualized as the typical Mandelbrot picture in Figure 11.10A.

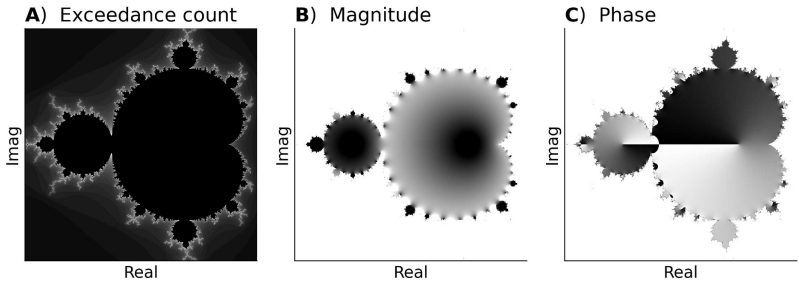


Figure 11.10: Sets of Mandelbrot sets.

So, what does the Mandelbrot set mean? It's effectively a count of how long it takes an iteratively growing function to exceed some value. Lighter colors in Figure 11.10A correspond to higher counts, meaning the function takes longer to increase, whereas black pixels indicate that the function converges to a value less than two.

Because Z is a complex-valued function, we can also visualize its magnitude and angle. That's what I've shown in panels B and C. Many function values grow to such gigantic values (10^{307}!!) that they cause numerical overflow issues and become NaN — not a number. Those are the white pixels in the periphery in panels B and C. The color limits in panel B are .1 to .5.

11.11 The beginning

If you enjoyed this chapter, then don't stop exploring! Let your imagination run wild, make beautiful calculart, put it on t-shirts and mugs and postcards that your grandmother can tape to her refrigerator.

But I do think it's time to get back to the serious business of learning calculus. Please keep reading to learn about integration.

Chapter 12

Intuition about integration

Welcome to "part two" of this book. So far, you've learned about functions, limits, and differentiation. Now it is time to learn about integration.

The good news is that integration is conceptually fairly straightforward. In fact, by the end of this chapter, you will have a solid idea of the goals, interpretations, and definitions of integration — although you'll have to wait until the next chapter to learn the details and techniques for integrating particular types of functions.

The bad news is that integration is a lot harder than differentiation. Like, really a lot harder. I will discuss why this is the case at the end of this chapter, but the upshot is that differential calculus takes one semester to learn while integral calculus takes a lifetime to master.

However, there is good news from the bad news: Python can implement a lot of integration details for you, so my approach for the rest of this book will be to do my absolute best to help you understand the important concepts, and show you how to use Python accurately and effectively to solve the more intractable integration problems.

12.1 Integration as "inverse differentiation"

Consider the equation $5 - 2 = 3$. The minus sign indicates the operation of subtraction. We can "invert" this operation using addition: $3 + 2 = 5$. In other words, you can think of addition and subtraction as inverse operations of each other: once cancels the other.

That may seem trivial and simplistic, but it is one way to think about the relationship between differentiation and integration: differentiation is like subtraction while integration is like addition, and each operation is like the inverse of the other. It's not

quite that simple, but the analogy is useful.

12.1.1 Cumulative summation

More specifically, integration is like a cumulative sum. "Cumulative" means that each value is the sum of all previous values. Consider the following vector of numbers:

$$\mathbf{v} = [\ 1, 3, 0, -2, 4\ ]$$

The sum of all those numbers is 6. That's one number. The *cumulative* sum, however, is another vector of the same size:

$$\mathbf{c} = [\ 1, 4, 4, 2, 6\ ]$$

Each element in $\mathbf{c}$ is the sum of all previous elements in $\mathbf{v}$. For example, the fourth element of $\mathbf{c}$ is the sum of the first four elements in $\mathbf{v}$.

Figure 12.1 shows vectors $\mathbf{v}$ and $\mathbf{c}$, as well as an additional example of the cumulative sum of random numbers.

Cumulative summation is indicated using the summation notation:

$$c_k = \sum_{i=1}^{k} v_i \tag{12.1}$$

Thus, the k^{th} element in $\mathbf{c}$ is the sum of all elements in $\mathbf{v}$ from the beginning to element k.

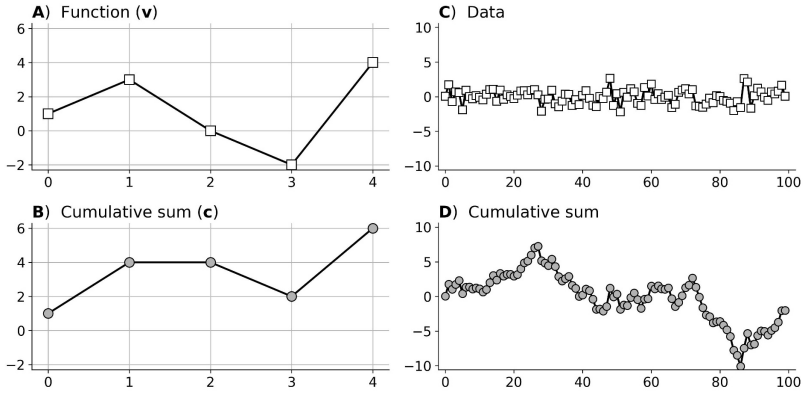


Figure 12.1: Two examples of the cumulative sum of a function. Panels A and B illustrate the example vectors in the text. Panels C and D show random numbers and their cumulative sum.

12.1.2 Approximate integration

There are several ways to approximate an integral that are based on different ways of conceptualizing integration. An algebra-based approximation simply involves cumulative summation. For a function $f(x)$, the integral $F(x)$ is approximated by:

Note the notation: lower-case letters for functions; upper-case letters for their integrals.

$$F(x_k) = \sum_{i=1}^{k} f(x_i)\,\Delta x \tag{12.2}$$

Δx is the x-axis resolution, same as the Δx term you learned about when calculating slopes and numerical differences. You can think of Δx as a scaling factor to account for the number of data points (k) when the x-axis resolution changes.

What happens when Δx gets smaller and smaller? Remember that with differentiation, as $\Delta x \to 0$, *differences* become *derivatives*. Analogously, as $\Delta x \to 0$, *cumulative sums* become *integrals*.

$$F(x) = \int f(x)\,dx \tag{12.3}$$

The summation sign $\sum$ turns into an integral sign $\int$ when Δx turns into dx. But the concept is the same: The value of the integral at each value of x is the sum of all function values left of that x value.

Some terminology: $F(x)$ is called the *indefinite integral* or the *antiderivative* of function $f(x)$.

The two equations below are the approximations of differentiation and integration. You've seen both equations before, but I'd like you to see them together to appreciate that differentiation involves differences between successive points in the function, while integration involves summations of successive points in the function.

$$\frac{\Delta y}{\Delta x} = \frac{f(x + \Delta x) - f(x)}{\Delta x} \tag{12.4}$$

$$F(x_k) = \sum_{i=1}^{k} f(x_i)\,\Delta x \tag{12.2}$$

An example Here's a function and its indefinite integral:

$$f(x) = x^2 \tag{12.5}$$

$$F(x) = \frac{1}{3}x^3 \tag{12.6}$$

In the next chapter, you'll learn why that integral is correct; for now, just trust me. I implemented this integral, and its approximation by applying Equation 12.2 on Equation 12.5, in Python to create Figure 12.2.

If the approximation were perfect, the circles would perfectly overlap with the black line. The approximation in this example is decent but imperfect. These approximation errors are due to the discretization (I set $\Delta x = .1$) and the simplistic algorithm for estimating the integral.

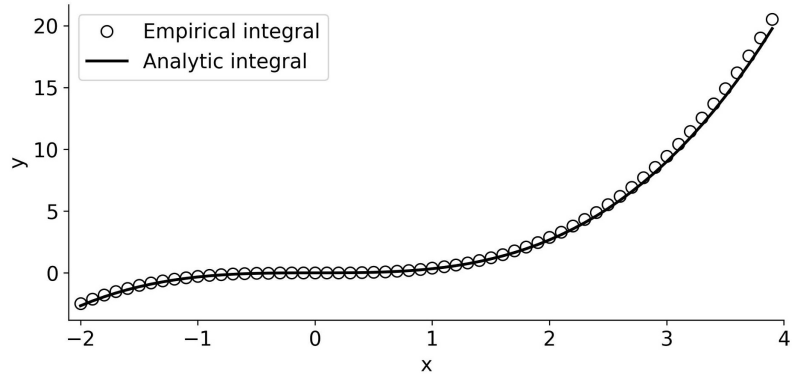


Figure 12.2: Comparison of analytic and numerical approximation of the integral of x^2.

In the next chapter, you'll learn more about the different algorithms for estimating integrals and how to improve the accuracy of the approximations; and in the exercises in this chapter you'll learn how adjusting Δx can improve (or degrade!) the approximation accuracy.

12.2 Integration as geometric area

In this section, I will present a different conceptualization of integration: as geometric area. It may initially seem quite different from the algebraic conceptualization as cumulative summation, but you will learn how these approaches are actually just two different perspectives of the same operation.

Consider Figure 12.3, which shows a function on a graph. The function is the constant $f(x) = 2$. The goal here is to calculate the area of the shaded region, which is defined by the line described by the function and the x-axis, between $x = 1$ and $x = 4$.

You don't need calculus to calculate this area: The shaded region is a rectangle, and I'm sure you know that the area of a rectangle is its height times its width. For this function, the height is $f(x) = 2$ and the width is $4 - 1 = 3$. So the area is 6 squared units.

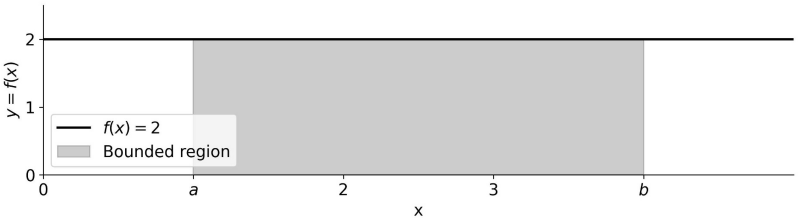


Figure 12.3: The geometric interpretation of integration is a formula to calculate the area (gray patch) between the line representing a function (thick black line) and the x-axis, between two x-axis bounds.

I'm going to recalculate that area using the approximate integration formula. Let's consider the function at $k = 2$ points: $f(1) = 2$ and $f(4) = 2$ (this means that $\Delta x = 4 - 1$).

$$F(x_k) = \sum_{i=1}^{k} f(x_i)\,\Delta x \tag{12.2}$$

Equation 12.2 is repeated from a few pages ago.

$$A = (2 + 2)\,3 = 12 \neq 6 \tag{12.7}$$

Uh oh. The "area" as calculated by the approximate integral formula is clearly incorrect. What's going on here??

The "problem" is that I've implemented the equation incorrectly. To understand what I did wrong and how to calculate area using the approximate integration formula, you need to learn about the distinction between definite and indefinite integrals, and about the Fundamental Theorem of Calculus. So, I will return to this example after the next section; for now, please embrace that feeling of confusion and get excited to replace that confusion with the satisfying feeling of comprehension.

Another example Now consider Figure 12.4, which shows a curve instead of a constant function. Imagine we want to calculate the area between the curve of the function and the $y = 0$ line (that is, the area of the shaded patch in panel A).

Can we still calculate the area using one rectangle? From visual inspection of panel B, it seems like using one rectangle is... well, it's not completely horrible, but it's obviously imprecise.[1]

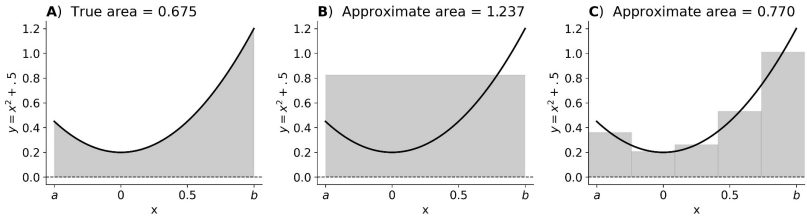


Figure 12.4: If the goal is to estimate area under a curve (panel A), can rectangles be useful (panels B and C)?

I will define the terms *definite* and *indefinite* integral in the next subsection.

Perhaps instead of using *one* rectangle, we can use *several* rectangles (panel C). Then we say that the area under the curve, which is called the *definite integral* of the function, is approximately equal to the sum of the areas of all of the rectangles.

But why five rectangles? Wouldn't the estimate be more accurate if I had used 10 rectangles? 100 rectangles?

I'm sure you see where this is going: As the number of rectangles increases to infinity, the width of each rectangle on the x-axis decreases towards zero. And that's how Δx becomes dx. And then our approximation becomes perfect. That intuition (the sum of rectangle areas as $\Delta x \to 0$) is actually one of the definitions of the integral. I'll formalize that in the next chapter, but for now I want you to appreciate how intuitive the integral is from a geometric perspective.

In fact, if we define the area of each rectangle as its width (Δx) times its height ($f(x_i)$), and we define the area under the function as the sum over all those rectangles, then we can express the approximate area of the function as the following:

$$A = \sum_{i=1}^{k} f(x_i)\,\Delta x \tag{12.8}$$

[1] OK, maybe it is completely horrible. I guess that's a subjective evaluation.

which is the same as the function for the approximate antiderivative I showed earlier.

In the exercises in this and the following chapters, you will implement this geometric approximation of the integral and discover how it converges to the analytic integral.

12.3 Integral as a function or number?

You may have noticed a disparity in the algebraic vs. geometric interpretations of the integral: The algebraic definition produced a new function $F(x)$ that had the same number of points as $f(x)$ and was defined for each value of x; in contrast, the geometric definition produced one number A corresponding to the area between the function's curve and the x-axis.

So what's the deal: Is the integral of a function another function or just a number?

It's both. This is the distinction between a *definite integral* and an *indefinite integral*.

A definite integral is the area under the curve between two x-axis points, and the result is one number. In contrast, an indefinite integral is the result of applying integration techniques to a function, and the result is another function of x.

This distinction is indicated below.

$$\text{Definite integral}: \left\{ \quad A = \int_a^b f(x)\,dx \quad \right\} \quad \text{One number}$$

$$\text{Indefinite integral}: \left\{ \begin{array}{c} F(x) = \int f(x)\,dx \\ F(x_i) = \int_{-\infty}^{x_i} f(u)\,du \end{array} \right\} \quad \begin{array}{l} \text{A function value for} \\ \text{each value of } x \end{array}$$

"Indefinite integral" is also called "antiderivative."

Notice that the definite integral has *bounds of integration* (a and

b next to the integral sign) to indicate the area between $x = a$ and $x = b$, while the indefinite integral is "naked" because it describes a function and not a numerical result within an interval. Indicating each element x_i in the antiderivative as the definite integral from $-\infty$ to x_i is not the typical way to express an antiderivative, but links to the algebraic approximation formula I showed earlier. The variable u is called a "dummy variable" and avoids confusion between a particular value x_i and all values of $x < x_i$ (similar to how I labeled the vectors $\mathbf{v}$ and $\mathbf{c}$ when introducing cumulative sums).

Integration bounds are also called intervals or limits.

To summarize: a definite integral has integration bounds and produces one number that corresponds to area, whereas the indefinite integral has no bounds on the integration sign and returns a function value for each value of x, which is the total area of the function from minus infinity to each value of x.

Caveats In the interest of simplicity, I've omitted two important subtleties that you'll learn about later: (**1**) the constant of integration and (**2**) not all functions have an analytically defined integral.

12.3.1 Two perspectives of integration

So far, I've introduced two different ways of thinking about integration: as the cumulative sum of the function, which is the algebraic perspective; and as the area under a curve, which is the geometric perspective. The table below provides a comparison of these two perspectives.

Algebraic	**Geometric**
As inverse differentiation.	As area between $f(x)$ and $y = 0$.
Used in formalizations and proofs	Visualization and intuition
Used for analytic integration (pure math)	Used for empirical integration (empirical science)

To be honest, these distinctions are somewhat exaggerated: the geometric perspective is also used in pure-math proofs, and the algebraic perspective is used in empirical sciences. Furthermore, there are yet other ways of conceptualizing integration for applications in physics, engineering, and time series analysis.

On the other hand, these two perspectives will help you understand and learn integral calculus, and will link directly to the two parts of the Fundamental Theorem of Calculus.

12.4 Terms and notations

Please take a moment to memorize the labels in Figure 12.5. Not explicitly indicated in the figure is that, in most cases, $b > a$ for reasons that will be clear when I explain the second part of the Fundamental Theorem of Calculus.

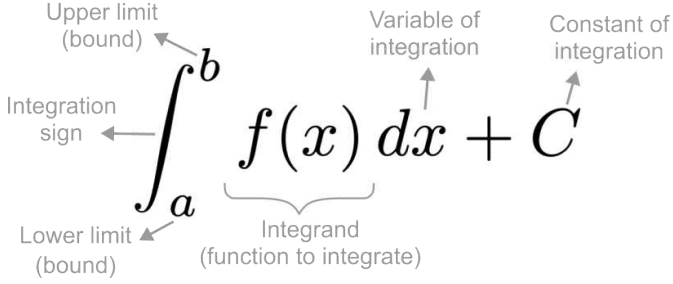


Figure 12.5: Terms of integration.

Both "limit" and "bound" are used for a and b; I prefer "bound" because the term "limit" already indicates a specific concept in calculus (i.e., the topic of Chapter 4), which is different from the two values of x (a and b) in definite integrals.

The only difference between definite and indefinite integrals in terms of notation is the absence of the two bounds for indefinite integrals.

When reading an integral out loud, you need to use the word "integral" and state the bounds if it is a definite integral. For

example, the following expression is read out loud as "the integral of the function from minus two to four."

$$\int_{-2}^{4} 4x + 3\sin(3x) + \frac{\sqrt{|x|}}{\ln(x^2)}\,dx$$

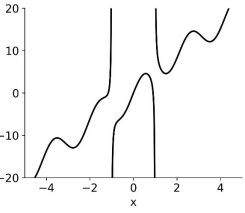


Figure 12.6: A curve defined by a function.

I don't know what that function means, but I think it looks pretty neat (Figure 12.6).

Regarding lower-case and upper-case letters: As I wrote in a margin note a few pages ago, it is common to use lower-case letters for functions (e.g., $f(x)$ and $g(\theta)$) and upper-case letters for their antiderivatives (e.g., $F(x)$ and $G(\theta)$). I will use that terminology consistently in this book (with an exception in Chapter 18), but be mindful that in other contexts, it is not necessarily the case that any function with a capital letter indicates an integral.

Final note about notation: Although it is common to put the dx after the function definition, it is sometimes incorporated into the expression. That might look a little unusual at first, but as long as the dx multiplies the entire integrand, it's fine.

$$\int \frac{1}{x+1}\,dx \quad = \quad \int \frac{dx}{x+1} \tag{12.9}$$

12.5 Fundamental Theorem of Calculus, part 1

There are two parts to the FTC. Part 1 is the formalization of the algebraic perspective of integration: integration and differentiation are like inverse operations.[2] Consider the following expressions:

[2] The qualifying term "like" is important here: integration and differentiation are not trivially always inverses of each other, due to the constant of integration and non-integrable functions.

$$\frac{d}{dx}\int f(x)\,dx \;=\; f(x) \tag{12.10}$$

$$\int f'(x)\,dx \;=\; f(x) \tag{12.11}$$

Equation 12.10 states that the derivative of the integral of a function is the original function, and Equation 12.11 states that the integral of the derivative of a function is the original function.

Some informal intuition: Look at Equation 12.10 and imagine that the dx in the denominator cancels the dx in the integral, and that the d in the numerator (difference) cancels the $\int$ (summation); the equation then simply reads $f(x) = f(x)$.

Figure 12.7 illustrates the FTC-1 in an example. I started with the function $f(x) = x^2$ and visualized it in panel A. Then I calculated its empirical derivative (panel B) using `np.diff` as you learned in previous chapters. Finally, I calculated the empirical integral of the derivative (not of the original function) using `np.cumsum` and visualized that, along with the original function, in panel C.

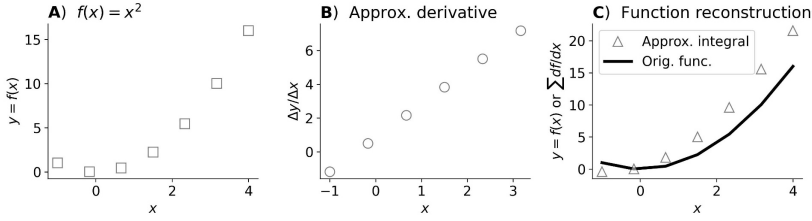


Figure 12.7: Illustration of how the integral of the derivative of a function is that function. Producing this figure is part of Exercise 3.

According to the FTC-1 (Equation 12.11), the black line and the gray triangles in panel C should be identical. But they are not. The issue here is that numerical errors arise from the discretization, and those errors compound as the function gets further from $x = 0$. This is an example of the distinction between pure math and digitized computer-based math. Fortunately, however, these are not fatal errors, and you'll learn how to improve the accuracy of numerically approximated integrals in the next chapter.

12.6 Fundamental Theorem of Calculus, part 2

FTC-2 reveals how to calculate the definite integral of a function:

$$\int_a^b f(x)\,dx \;=\; F(b) - F(a) \tag{12.12}$$

Remember that the indefinite integral of a function ($F(x)$) is itself a function, which means that we can evaluate the integral function at a specific value of x. FTC-2 says that to calculate the definite integral of a function between $x = a$ and $x = b$, first calculate the antiderivative of the function, evaluate that antiderivative at $x = b$ and at $x = a$, and then subtract those two values.

FTC-2 is illustrated in Figure 12.8. The goal is to calculate the area under the curve described by $f(x)$ between $x = a$ and $x = b$, and FTC-2 tells us that that area can be calculated as the indefinite integral at $x = b$ (which is the sum of the area from $x = -\infty$ to $x = b$) minus the indefinite integral at $x = a$.

This graph explains why integration bounds are usually sorted so that $b > a$.

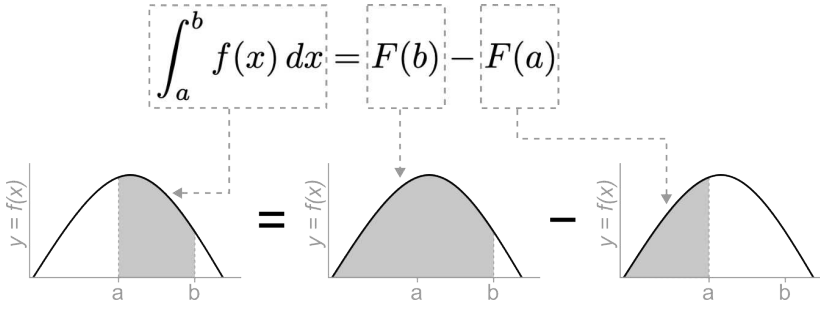


Figure 12.8: The geometric interpretation of FTC-2. The definite integral can be interpreted as the area under the curve between two bounds. This area is calculated as the area to the left of $x = b$ minus the area to the left of $x = a$, where "area to the left" is given by the antiderivative of the function evaluated at those x values.

Example 1 I'm going to show two examples to help you build intuition for FTC-2. The first example is the area of the rectangle from earlier in this chapter. Remember that I tried to apply the approximate integration formula but ended up with an obviously incorrect answer.

The FTC-2 tells us how to use integrals to calculate area: First calculate the indefinite integral of the function, then evaluate that integral function at the two boundary points, and finally subtract the two values.

The antiderivative of a constant function is that constant times x. Thus:

$$f(x) = 2 \tag{12.13}$$

$$F(x) = 2x \tag{12.14}$$

I will explain how to integrate functions in the next few chapters, but you can confirm that that the answer is correct by applying FTC-1: The derivative of the integral returns the original function. That is,
$F'(x) = 2 = f(x)$.

Now we apply FTC-2:

$$A = \int_1^4 2x\,dx = F(4) - F(1) \tag{12.15}$$

$$= 2(4) - 2(1) = 6 \tag{12.16}$$

This confirms that the area is 6. Armed with this knowledge, we can also recalculate the area using the approximate integral formula that I showed earlier using only two discrete values of $x = [\,1, 4\,]$:

$$F(x_k) = \sum_{i=1}^{k} f(x_i)\,\Delta x \tag{12.17}$$

$$F([1,4]) = [\,2 \times 3, 4 \times 3\,] = [\,6, 12\,] \tag{12.18}$$

$$A = 12 - 6 = 6 \tag{12.19}$$

The function has a constant value of 2 for all values of x (thus, $f([1,4]) = [2,2]$), but the cumulative sum function $\sum f([1,4]) = [2, 2+2] = [2,4]$. And then scale each value by $\Delta x = 3$ to obtain $F([1,4]) = [6,12]$. Finally, apply FTC-2 and subtract those values to get the definite integral.

Important subtleties Some important details that you will learn about in the next few chapters that I will mention here so that you are aware of what you'll learn in the next few chapters: not all functions can be analytically integrated (though many of these can be empirically estimated); the constant of integration is necessary for antidifferentiation but not for definite integrals; the constant of integration has a somewhat awkward geometric interpretation; there is a difference between *net area* and *total area*; conceptualizing the definite integral as geometric area relies on some assumptions; the definite integral can have other non-geometric interpretations in certain applications.

12.7 Reversing the limits of integration

Earlier in this chapter, I wrote that the bounds of the integration should be ordered such that the smaller number (that is, leftwards on the number line) should be on the bottom while the larger number (that is, rightwards on the number line) should be on the top of the integration sign. That is common practice, and it's a convention you should follow.

But that's not a *rule* per se. In fact, when you swap the integration bounds, the sign changes.

$$\int_a^b f(x)\,dx \;=\; -\int_b^a f(x)\,dx \tag{12.20}$$

From the perspective of the FTC-2, you can think of it this way:

$$\int_a^b f(x)\,dx = F(b) - F(a) \tag{12.21}$$

$$\int_b^a f(x)\,dx = F(a) - F(b) \tag{12.22}$$

$$= -\big(F(b) - F(a)\big) \tag{12.23}$$

"Forwards area" and "backwards area" are not formal calculus terms, but I hope they help with understanding.

Geometrically, the idea is to measure the area "backwards" such that the sign swaps (Figure 12.9).

Regardless, swapping the bounds does not affect the magnitude of the definite integral; only the sign.

What is the purpose of swapping the sign? There are situations where the calculations can be easier, or more numerically stable, when the integration is done in reverse order. Swapping bounds is also used in some mathematical proofs. It's used to reverse or invert "work" in some physics problems, and to try to predict the present based on past data in financial modeling. Another application is in geometry when dealing with shapes that have a particular orientation.

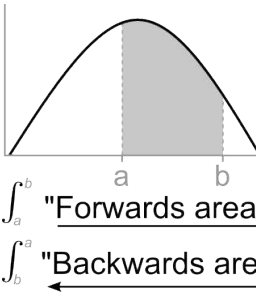


Figure 12.9: Reversing the bounds of integration swaps the definite integral sign.

12.8 Integration is harder than differentiation

Differentiation is always taught before integration. That is not just an arbitrary byproduct of precedent; integration is more chal-

lenging than differentiation, and also relies on knowing differentiation. I dare not imagine the torture of trying to learn how to integrate functions before learning about limits and derivatives.

Why is integration so much harder than differentiation?

In differentiation, elementary functions have a clear set of rules that you can follow; differentiation can be tedious but the difficulty comes from keeping track of which rules and tricks to use in which order.

In contrast, there is no systematic procedure or universal method to integrate all functions, making it more of an art than a science. Different functions require different integration techniques, such as substitution, integration by parts, partial fractions, or trigonometric substitutions. While differentiation can usually be solved mechanistically, integration often involves recognizing patterns or making educated guesses based on experience with similar problems.

Indeed, a lot of proofs in integration involve being told the answer and then confirming it via FTC-1. I showed an example of this with integrating a constant function, and you'll see more examples in the next several chapters.

12.9 Exercises

12.1. Cumulative summation is the foundational building-block underlying numerical approximations of integrals. The goal of this exercise is to ensure that you understand cumulative summation by writing code that implements this operation. Calculate the cumulative sum of the integers 1 through 10 — without using `np.cumsum`. After writing your code, confirm your answer using `np.cumsum`.

There are several ways to implement the cumulative sum. In the online code, I show three different options using for-loops and list comprehension, with and without `numpy`.

12.2. The goal of this and the next few exercises is to write code that will allow you to confirm FTC-1 using numerical approximations. In this exercise, you will write a Python function that implements the approximation of the derivative and integral, and in the next exercise, you will visualize the outputs to produce Figure 12.7.

Your Python function should take a vector of x values and corresponding $y = f(x)$ values, and output a numerical estimate of the derivative and the integral of the derivative. Calculate the numerical derivative using `np.diff` as you learned in previous chapter, but repeat the final value so that the derivative has the same length as the original function.

To calculate the numerical integral, apply Equation 12.2 on the numerical derivative calculated above. But you will need to normalize that integral by defining the constant of integration. I haven't yet introduced the concept of normalizing an integral, although I have mentioned the constant of integration a few times. Follow the instructions in the next paragraph; in the next chapter you will learn why these steps are sensible.

There are three steps to this normalization. First, find the array index at which x is closest to zero (hint: `np.argmin`). Second, subtract the value of the integral at $x = 0$ from the entire integral. This step effectively shifts the integral by an offset that forces $F(0) = 0$. Third, add back to the integral the value of the original function at $x = 0$. The result of this three-step procedure guarantees that $\int f'(0)\,dx = f(0)$. Again, the reason for this will be clear to you in the next chapter.

When you've written your function, confirm its accuracy by reproducing the table below (`fx` is the function, `df` is the numerical approximation of the derivative, and `idf` is the numerical approximation of the integral of the derivative of the function). I won't tell you the function; you need to guess it by inspecting the x, y pairs in the table below.

```
  x    |   fx    |   df    |   idf
-----------------------------------
-1.00  |   1.00  |  -1.00  |  -1.00
 0.00  |   0.00  |   1.00  |   0.00
 1.00  |   1.00  |   3.00  |   3.00
 2.00  |   4.00  |   5.00  |   8.00
 3.00  |   9.00  |   7.00  |  15.00
 4.00  |  16.00  |   7.00  |  22.00
```

In theory, the second and fourth columns should be identical. But they're not — not even close! That's due to accumulating errors from the discretization. However, reproducing this table means that your Python function is accurate.

Finally, qualitatively explore how the accuracy of the integration changes when you change `dx`. For example, increasing the number of points between $x = -1$ and $x = 4$ from 6 to 60 will reveal a considerably closer match between the original function and the integral of the derivative.

12.3. The goal of this exercise is write `matplotlib` code that will

visualize the outputs of the Python function you wrote in the previous exercise. If you use the function $f(x) = x^2$ with 27 x-axis values linearly spaced between -1 and 4, you should be able to reproduce Figure 12.7.

Now that you have the code, you can explore some other functions. In the online code, I've implemented the following two equations as additional explorations.

$$f(x) = x^3 + 4$$

$$g(x) = \frac{x^3}{10} - \pi e^{-x^2} + \sin(4x)$$

12.4. The purpose of this exercise is to write code to implement the geometric approximation of the definite integral as the area between a function curve and the x-axis. This and the following exercise will prepare you for the more rigorous and detailed discussions of geometry and area in Chapters 13 and 17.

The goal is to write code to produce Figure 12.10. The function is $f(x) = x^2 - .5$, the function is evaluated between $x = -.5$ and $x = 1.1$, and there are nine bars when using a resolution of $\Delta x = .2$. I have some coding hints in the footnote[3].

I normally encourage you to explore different parameters in coding exercises, but please only use the exact parameters I use here, so that you have a fresh perspective for the following exercise.

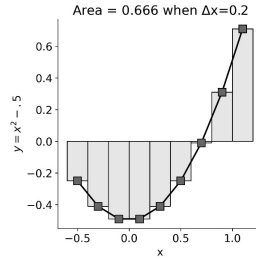


Figure 12.10: Visualization for Exercise 4.

Notice that the approximate area is positive, although many function values are negative. In the next chapter, I will in-

[3]The width of each rectangle is Δx, so you can draw each rectangle using centered x-axis coordinates as $[x_i - \Delta x/2, x_i + \Delta x/2]$. You can inspect the code that generates Figure 12.4 if you need help getting started.

troduce the distinction between *net area* and *total area*, the former of which can be negative. For now, use the absolute value of the area formula to obtain the total area.

12.5. The area in the previous exercise was approximated to be .666 squared units (that's just how it happened to work out... nothing demonic going on here). Visual inspection of the figure reveals that some pieces of the rectangles over-estimate the area (where the rectangles protrude beyond the curve) whereas other pieces of the rectangles under-estimate area. Is your intuition that .666 is an over-estimate or under-estimate of the true area? Make a guess before reading the next paragraph.

Have you guessed yet or did you just keep reading?? Spend a moment to change Δx in the code for the previous exercise, e.g., try $\Delta x = .1$ or $\Delta x = .05$. Do those new approximations confirm or conflict with your intuition?

You will learn more about approximation methods and estimation errors in the next chapter. In this case, the nine bars over-estimate the true area, which is closer to .573. I calculated that value using the following `sympy` code:
`sym.integrate(sym.Abs(sx**2-.5),(sx,-.5,1.1))`
`sx` is a symbolic variable. I will explain the meaning of this code in the next chapter, but the idea is to integrate the function (first input) using the specified variable and bounds (second input).

Regarding the numerical approximations: As Δx decreases, the number of bars increases, and the accuracy of the estimate increases. The goal for the rest of this exercise is to systematically explore how the approximation of area approaches the true value.

Create Figure 12.11 by pasting the code from Exercise 4 (but not the plotting code) into a for-loop over a range of Δx values between .001 and .5. I used logarithmically spaced values and then log-spaced the x-axis. You will need

to invert the x-axis in `matplotlib` to have large values on the left of the plot.

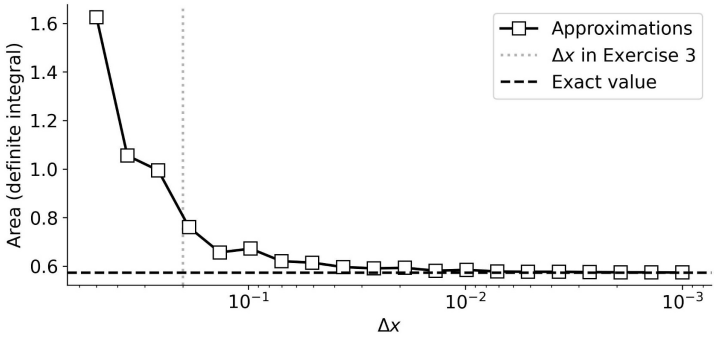


Figure 12.11: Results for Exercise 5.

The approximations converge to the analytic result at around $\Delta x = .01$ (that's my eyeball evaluation of this particular example, and is not necessarily true for all functions).

Chapter 13

Geometry of integration

This chapter provides a detailed overview of the geometric conceptualization of the definite integral as the area between a curve defined by a function and the x-axis. You'll see how these approximations lead to formal definitions, and you'll learn how to implement these concepts in Python so you can explore and visualize integration techniques like Riemann and Lebesgue sums.

13.1 Riemann sums

The purpose of this section is to show you how to conceptualize and define the definite integral using Riemann sums and limits. In fact, I've already introduced you to the concept and Python implementation of Riemann sums in the previous chapter, so this section will add more detail to what you're already familiar with.

"Riemann sum" might sound like a singular noun — one specific algorithm. But in fact, the Riemann sum refers to a general concept in geometry for calculating areas and approximating functions. The idea is to represent a function using a collection of small shapes, each of which has an area that is easy to calculate. Rectangles, triangles, and trapezoids are examples of shapes with easily calculable areas that can be combined to represent a function.

"Partitioning" a function is also called "discretizing" a function.

So, the idea of a Riemann sum is to partition an area into simple shapes like rectangles, calculate the areas of those partitions, and sum them to approximate the total area. When the area is defined by the region between a function curve and the x-axis, the Riemann sum is an approximation of the definite integral of that function. You saw several examples of this in the previous chapter, and Figure 13.1 shows another example for your viewing pleasure.

As you also know from the previous chapter, the idea is then to use an increasing number of smaller partitions, and at the limit as the number of partitions goes to infinity, the Riemann sum ($\sum$)

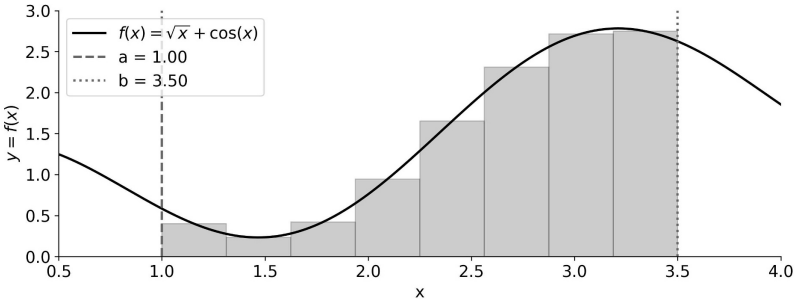


Figure 13.1: Graphical overview of the Riemann approximation to the area under a curve between two bounds: Tile the area with rectangles, calculate and sum the areas of those rectangles. That's the approximation of the area under the curve.

becomes the definite integral ($\int$).

When learning about the Riemann sum, you'll see examples using single-variable functions. The Riemann sum is valid for any number of variables (that is, any number of geometric dimensions), although there are computational limitations in higher dimensions that lead to alternative approximations of the integral that I'll discuss later. But the Riemann sum is the standard for approximating integrals of one- or two-variable functions, especially for functions that do not have an analytic representation, for example, in empirical data.

13.1.1 Definition of Riemann sum

Equation 13.1 shows the definition of the Riemann sum. The area under a function curve is approximated as the sum (S) of n rectangles between the two bounds $x = a$ and $x = b$ for $b > a$.

$$S = \sum_{i=1}^{n} f(x_i)\,\Delta x \tag{13.1}$$

$$\Delta x = \frac{b-a}{n} \tag{13.2}$$

I'm using S for sum instead of A for area to make the equation a bit more general.

The set of rectangles "tiles" the function area. How do we know that the tiles are rectangular? Because the area of a rectangle is defined as the width times the height; in this case, the width of each rectangle is Δx and the height of each rectangle is the y-axis coordinate, which is $f(x_i)$.

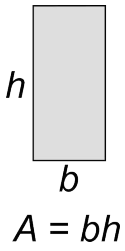


Figure 13.2: The area of a rectangle is the base (Δx) times the height $f(x)$.

This setup is slightly different from how I coded the Riemann sum in the previous chapter: There, I defined Δx as a variable and n was determined by a, b, and Δx. That doesn't fundamentally change the equations, though, because you can see in Equation 13.2 that these four variables are all inter-related, and any one can be defined as a function of the other three.

I haven't yet explained how to get the x_i values. Clearly they will be linearly spaced between $x = a$ and $x = b$ with Δx defining the step size. The following expression shows how to obtain the partition boundaries for the n rectangles.

$$x_p = \{a,\ a + \Delta x,\ a + 2\Delta x,\ ...,\ a + (n-1)\Delta x,\ b\} \tag{13.3}$$

To make the x tiling more concrete, let's work through a simple example. For now, it doesn't matter what $f(x)$ is; let's just focus on partitioning any function into five tiles between $a = 0$ and $b = 5$. According to Equation 13.3, the set of x_p (partitioning) values is:

$$\Delta x = \frac{b-a}{n} = \frac{5-0}{5} = 1 \tag{13.4}$$

$$x_p = \{0,\ 0{+}1,\ 0{+}2(1),\ 0{+}3(1),\ 0{+}4(1),\ 5\} \tag{13.5}$$

$$= \{0,\ 1,\ 2,\ 3,\ 4,\ 5\} \tag{13.6}$$

Huh, so we set $n = 5$ but ended up with 6 values of x_p. How can we make sense of that discrepancy? See if you can figure it out before reading the next paragraph.

The answer is that Equation 13.3 defines the *boundaries* of the tiles. n tiles requires $n+1$ boundaries.

Thus, we have $x = a$, and then a plus integer multiples of Δx, which is the discretization of the x-axis, up to $n-1$ points, and then we have $x = b$ at the end.

But we still don't yet have the x_i values to use in Equation 13.1: How do we define n x_i values from $n+1$ boundary points? Segue to the next subsection...

Riemann summation: partition an interval, calculate the function value within each partition, and sum those values scaled by the partition width.

13.1.2 Riemann summation rules

It turns out that there are different ways to calculate the values for x_i, and the method you choose has implications for the values of $f(x_i)$ — and for the approximation of the integral. Figure 13.3 illustrates this concept.

Notice that the function (black line) is the same in all three panels, and that all three panels have 14 rectangles with the same bounds and the same Δx. But the rectangles are aligned slightly differently, and therefore the summed areas (printed in the title of each panel) are slightly different.

Before reading further, take a moment to inspect the figure to determine the differences across the three panels.

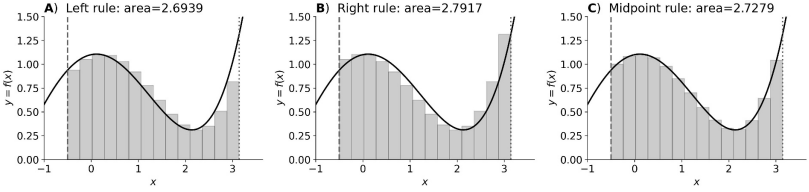


Figure 13.3: Illustration of the left, right, and midpoint rules for Riemann sums. You will create this figure in Exercise 4.

The list below describes each of the rules. After introducing these rules, I'll discuss how they compare with each other, and you'll get to implement each of these rules in the exercises.

Left rule (Figure 13.3A)

When summing using the left rule, the value of $f(x_i)$ is calculated using the "left" boundary points, which means that the function values for the n partitions are defined using the values $x_i = a$ through $x_i = a + (n-1)\Delta x$ (thus skipping b at the end). Notice in Figure 13.3A that the upper-left edge of the rectangles touch the function line.

Right rule (Figure 13.3B)

When summing using the right rule, x_i is defined using the "right" boundary points, which means starting at $x_i = a + \Delta x$ and ending at $x_i = b$. You can see in Figure 13.3B that the upper-right corner of the rectangles touches the function line.

Midpoint rule (Figure 13.3C)

I'm pretty sure you can guess this rule: Define x_i to be the average of the left and right bounds, which means the upper-center of each rectangle touches the function line. Here, each x_i is defined as the average of two adjacent partition points: $x_i = a + (i\Delta x + (i+1)\Delta x)/2$.

Infimum rule (Figure 13.5A)

This is also called the "lower rule." Here the idea is to take the minimum value of the function inside the x_i to x_{i+1} window. That is, evaluate the function between $f(x_i)$ and $f(x_{i+1})$, find the value of x that has the smallest $f(x)$, and use that point as the height of that rectangle.

Supremum rule (Figure 13.5B)

This is also called the "upper rule." It's the same idea as the infimum rule, but you use the value of x that maximizes the function between x_i and x_{i+1}.

A trapezoid is a quadrilateral with one pair of parallel sides. In the context of Riemann sums, it's like a rectangle but the two function-touching corners can have unique y-axis values.

Trapezoid rule (Figure 13.5C)

The trapezoid rule doesn't use rectangles to estimate the sum; it uses (drum-roll...) trapezoids. The idea of the trapezoid rule is to combine both the left and the right rule: Instead of a rectangle, create a trapezoid in which both the upper-left corner and the upper-right corner of the trapezoid touches $f(x)$. The area of a trapezoid with two right angles is the base times the average of the two heights (Figure 13.4).

Simpson's rule

Simpson's rule (named after the 18^{th} century mathematician, not the 20^{th} century TV show) is based on taking a weighted average of the function values within each partition. The idea is to fit a parabola to each partition, and then calculate the areas between those parabolas and the x-axis. It is quite accurate and used for numerical calculations, as you'll see in the exercises in this and following chapters.

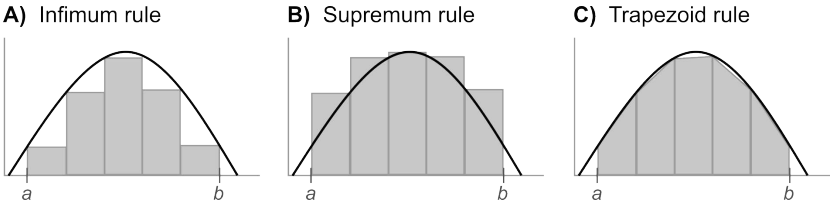


Figure 13.5: Illustration of the infimum, supremum, and trapezoid rules for Riemann sums.

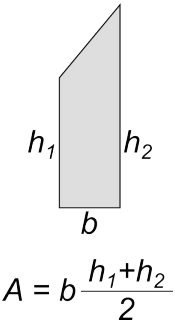


Figure 13.4: The area of a trapezoid is the base (Δx) times the average of the two heights (function values at each of two partition bounds).

Under- or over-estimating the sum From inspecting the figures, you can see that all of these rules are only approximations, because each rectangle can underestimate or overestimate the area under the function curve. But you can also see that there is no consistent under- or overestimation. That is, whether the rectangles are too low or too high depends on the nature of the function at each value of x, and not on the particular rule. Therefore, all we can say is that these rules can have some estimation errors.

The exceptions, of course, are the infimum and supremum rules: The infimum rule will always underestimate the area while the supremum rule will always overestimate the area.

Which rule is best? OK, this is a trick question: None of these rules is *best*. In fact, the sums calculated from all of these rules converge to the same value as Δx decreases. I hope that feels intuitive: As the number of rectangles increases, the approximation errors get smaller and smaller, and the estimated sum from all summation rules converge to the same value.

That said, the trapezoid and Simpson's rules tend to have the

highest accuracy for smooth functions like polynomials, though with increased computation time.

It's interesting to compare the convergence of the infimum and supremum rules as Δx shrinks: the true area of the function is necessarily between these two approximations, and as Δx decreases, the two areas converge to the same value with the true value being sandwiched between them. In fact, this is an example of using the squeeze theorem to determine the area under a curve.

Python implementations In the exercises of this chapter, you'll translate Equation 13.1 into Python code to approximate areas. That's great for building lasting intuition about definite integrals, but it's not the best way to calculate numerical integrals in practice: A simple `numpy`-based approach can lead to numerical issues and compounding precision errors. In the next chapter, I will show you how to implement the trapezoid and Simpson's rules using the `scipy` library, which are preferred methods in real-world applications.

13.1.3 Riemann integral

One definition of the definite integral is to keep shrinking down Δx until it becomes dx:

As $\Delta x \to 0$, $n \to \infty$.

$$\int_a^b f(x)\,dx = \lim_{\Delta x \to 0} \sum_{i=1}^{n} f(x_i)\Delta x \tag{13.7}$$

This is *one* definition of the integral; it's not *the* definition of the integral. There are several definitions of the integral, and there are functions that can be integrated but have no Riemann integral.

But the Riemann definition is popular because it is intuitive, links closely to the geometric interpretation of a definite integral, and is applicable to many applications in abstract and applied math.

13.1.4 Why always the *x*-axis?

Why do we always use the x-axis (which is the same as saying the line corresponding to $y = 0$) as the reference to calculate area under the curve? Couldn't we also calculate the area using $y = 1$ or $y = -\pi^{e^2}$ — or even another curve that isn't just a horizontal line?

The answer is *yes we can*. You can use a different line, or a different function, as the reference. In fact, if you think of the function $g(x) = 0$, then the integral as I've introduced it in this chapter can be written as $\int f(x) - g(x)\,dx$. And then you can imagine replacing $g(x) = 0$ with some other function.

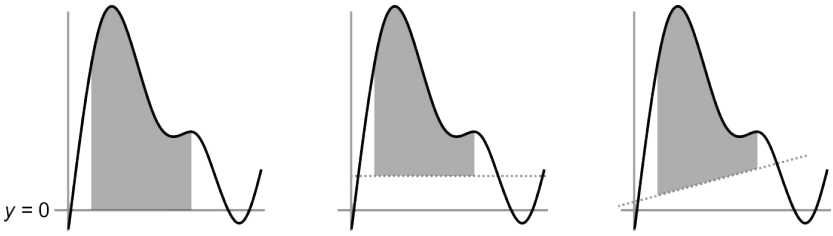


Figure 13.6: The Riemann sum is usually calculated between the function curve and the x-axis (left panel), but other reference lines are possible.

I'll discuss when and why to integrate function differences in Chapter 17. Until then, you can keep in mind that using the $y = 0$ line as a reference for calculating area under a function curve is the "default" for definite integrals, but it's not the only reference.

13.2 Net vs. total area

So far, you have learned that the integral is the area between a curve representing a function and the x-axis. You may have been wondering about functions that take on negative values, or functions that are sometimes negative and sometimes positive: How do you compute the area of a function that takes on both

negative and positive values?

The answer to this question is the distinction between *net area* and *total area*.

13.2.1 Net area

Net area is also called *signed area*, and involves considering any negative function values to have "negative area" (Figure 13.7). From a geometric perspective, it may seem awkward to consider area to be "negative," but in the context of definite integrals, negative area simply means the area of a function that is below the x-axis. In other words, any function with $f(x) < 0$ in some interval has negative area within that interval.

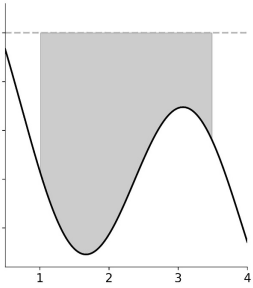


Figure 13.7: "Negative area" between a function curve and the x-axis.

Because a function can be both below and above the x-axis, the net area can be close to zero, even equaling exact zero if the positive and negative values of a function balance. This is the case, for example, with the cosine function if the bounds are selected to be $[a, a + k2\pi]$ for any integer k (Figure 13.8).

13.2.2 Total area

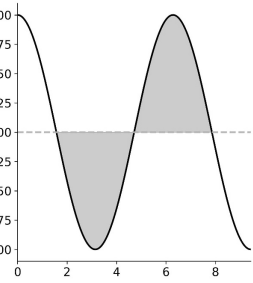


Figure 13.8: Functions can have a net area of zero if the positive and negative signed areas cancel.

Can you guess what "total area" means? If you haven't already, please form a guess before reading the next paragraph.

Total area is also called "unsigned area." It is the area between the curve and the x-axis, regardless of the sign of the function. You can think of it like counting all the gray pixels in Figure 13.8 regardless of whether they are above or below the x-axis.

Figure 13.9 shows some examples of the distinction between net and total area. Because the function in the left panel has both negative ($f(x) < 0$) and positive ($f(x) > 0$) regions, the net area is smaller than the total area. In the middle panel, the function is negative across the entire integral interval, and so the net and total areas have the same magnitude but different sign. And the

function in the right panel has only positive values, and thus the net and total areas are the same.

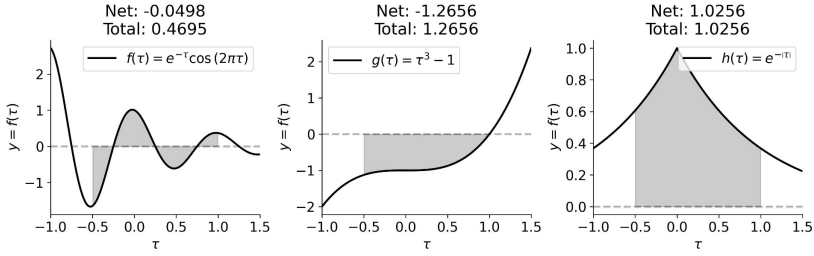


Figure 13.9: Examples of functions and subdomains for which the net and total area are the same or different. Creating this figure and calculating the areas is part of Exercise 7.

13.2.3 Calculating net and total area

The way to calculate net and total area depends on how you calculate the definite integral: via approximations, via analytic calculations, or via code.

Via approximations The Riemann sum that you learned about earlier in this chapter (Equation 13.1, reprinted below) is the formula for approximating net area, because negative function values will sum with positive function values. Total area is calculated as the sum the absolute values of the areas of the individual rectangles.

$$A_{\text{net}} = \sum_{i=1}^{n} f(x_i)\,\Delta x \tag{13.1}$$

$$A_{\text{total}} = \sum_{i=1}^{n} |f(x_i)|\,\Delta x \tag{13.8}$$

Via analytic calculations Net area is calculated simply by applying the FTC-2 to the function and bounds. The resulting definite integral will be the net area.

In theory, the way to calculate total area is to integrate the absolute value of the function. That is:

$$A_{total} = \int_a^c \left| f(x) \right| dx \tag{13.9}$$

That approach works for some simple functions that are easy to integrate, but can be intractable or computationally difficult because it requires integrating the composition of a function embedded inside the absolute value function.

Thus, instead, the way to calculate total area is to break up the interval into the sum of smaller intervals that have the same sign, calculate the net definite integral of each segment separately, and then sum the absolute values of the individual areas:

$$A_{total} = \left| \int_a^b f(x)\, dx \right| + \left| \int_b^c f(x)\, dx \right| \tag{13.10}$$

Notice that the absolute values are now *outside* the integration, meaning we're taking the absolute value of the definite integral, which is just a number. Also notice the integration bounds: Assuming that $a < b < c$, the two separate integral pieces span the same interval as the original integral. If the function flips sign more than once, you'd need to use additional segments. Figure 13.10 illustrates the idea.

Splitting the interval is based on the roots of the function (that is, the values of x for which $f(x) = 0$). Thus, the procedure to calculate the total area of a function that has both negative and positive values in the integration window is (**1**) identify the roots of the equation, (**2**) split the integral into intervals based on those roots, (**3**) calculate the areas of each piece, and (**4**) sum the absolute values of those individual areas.

Why would you break up the integral into multiple pieces instead of integrating the absolute value of the function? It's the latter easier?

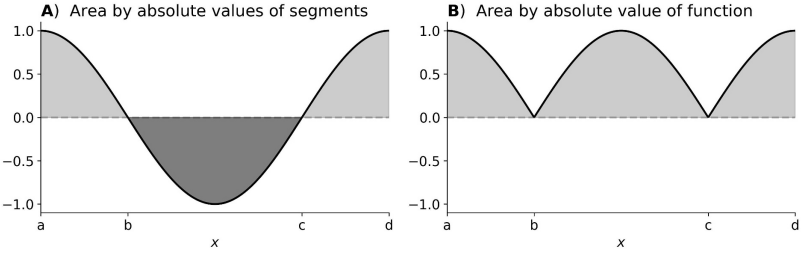


Figure 13.10: Two ways to calculate total area of a function that has both negative and positive values. Panel A depicts segmenting the function into regions in which the signs are the same (in this example, three segments); the definite integral is then calculated separately on each interval and the absolute values of those areas are summed. Panel B depicts calculating the area of the absolute value of the function.

The answer, as I mentioned earlier in this section, is that composing a function into the absolute value function (that is, $|f(x)|$) can be somewhere between tricky and nightmarish to integrate analytically, and so breaking up the integration interval into smaller windows is often easier than integrating a composition. Indeed, there are cases where `sympy` won't be able to find an exact solution to a composed integral of the form $\int |f(x)|\, dx$. You will see examples of this in the exercises of this chapter, and you'll learn best-usage approximation methods using `scipy` in the next chapter.

Via code `sympy` is able to analytically integrate a wide variety of functions, and I will explain how to obtain indefinite and definite integrals in `sympy` in the next section and in Exercises 7-9.

However, there are many functions that cannot be analytically integrated, or that `sympy` struggles to integrate. In these cases, you can use numerical approximations in `numpy` or in `scipy`. Coding integration in `numpy` involves translating the formulas presented in this chapter into code, and is an excellent way to learn the mechanics of integration. You'll get to practice integrating in `numpy` in Exercises 1-6, and in exercises in the next few chapters. However, integration methods provided in the `scipy` library can be more accurate while requiring less coding (although the link to

the math is less clear). You'll learn about those methods in the next chapter.

13.3 Definite integrals in `sympy`

To calculate the area under a curve between two x-axis bounds, use the following code:

```
sym.integrate(expr,(x,a,b))
```

`expr` is a `sympy` expression representing a function, `x` is the symbolic variable, and `a` and `b` are the lower and upper bounds to integrate across.

The output of `sym.integrate` is the analytically calculated integral, and is based on integration techniques that you will learn in the next few chapters. If `sympy` cannot determine the analytic integral, the output will be the same expression as the input.

Here's an example.

```
x = sym.symbols('x')
expr = x**2
a,b = 0,2
area = sym.integrate(expr,(x,a,b))
display(Math('\int_{%s}^{%s} %s \,dx = %s'
        %(a,b,sym.latex(expr),sym.latex(area))))
```

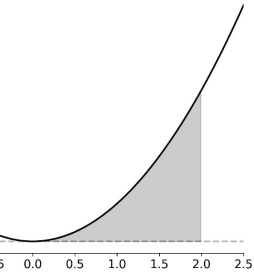


Figure 13.11: `sympy` calculated the area of the shaded region to be 8/3.

The final line prints the following:

$$\int_0^2 x^2\,dx \;=\; \frac{8}{3}$$

For now, you can just trust that `sympy` calculates the answer correctly. In the next chapter, you will learn how to calculate definite integrals by hand.

The code shown above will calculate the net area. To calculate the total area, integrate the absolute value of the function:

```
sym.integrate(sym.Abs(expr),(x,a,b))
```

Unfortunately, although this approach is theoretically sound and the syntax is correct, it is easy to find situations where `sympy` is unable to integrate a function analytically, or gives an incorrect answer. Splitting the interval into segments with the same sign (Equation 13.10) can provide accurate solutions. You will see examples of this in Exercises 8 and 9.

13.4 Lebesgue integrals

The Lebesgue integral is a different approach to conceptualizing and calculating integrals. It was developed by the French mathematician Henri Lebesgue. Before getting into the math, let me save you some embarrassment at calculus parties by making sure you can pronounce the name correctly. You might be tempted to pronounce it "hen-ree leh-bess-cue." In my humble opinion, that is an understandable mistake, but if those sounds come out of your mouth at a French-themed math soirée, party-goers may turn to you with offended gasps and ban you from their future parties. The proper pronunciation is closer to "uhn-ree leh-bek," although you'll need to ask a native French speaker to hear the true pronunciation.

The ~~leh-bek~~ Lebesgue integral is designed to address some issues that arise with the Riemann integral. I will list those potential issues, as well as a comparison between the two types of integrals, at the end of this section; for now you should focus on the concept and calculation of the Lebesgue integral.

13.4.1 Partition the domain vs. the range

As you now know, the Riemann sum involves partitioning the x-axis and calculating the function height within each partition. The Lebesgue sum, in contrast, involves partitioning the y-axis and calculating the spread of the function across the x-axis for each y-axis partition.

In other words, whereas the Riemann sum involves partitioning the *domain*, the Lebesgue sum involves partitioning the *range* (Figure 13.12). That is a subtle but important distinction, and leads to several advantages, and difficulties, of the Lebesgue integral.

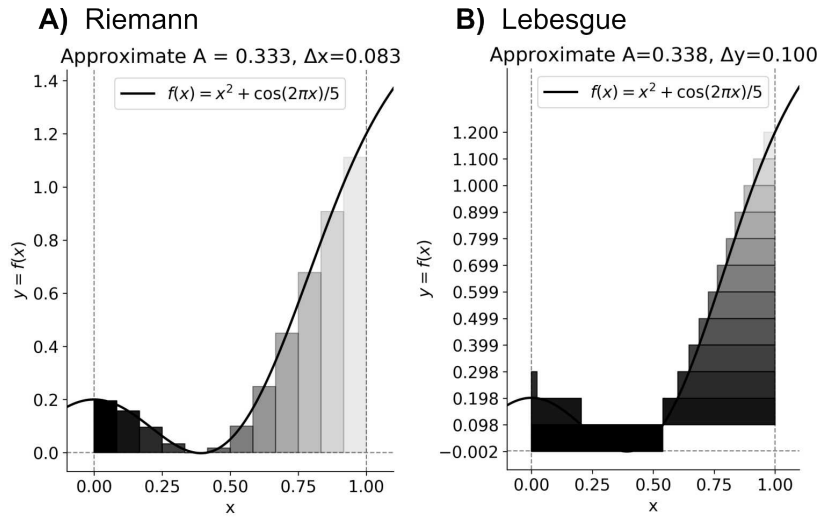


Figure 13.12: Comparison between Riemann's (panel A) and Lebesgue's (panel B) approaches to calculating definite integrals.

Notice that in Figure 13.12A (Riemann approximation) the bars are vertically oriented, whereas in Figure 13.12B the bars are horizontally oriented. Again, the idea is to partition the y-axis (the range) of the function within the bounds, and then calculate the area of the x-axis that corresponds to each y-axis partition.

13.4.2 Riemann vs. Lebesgue comparisons

First, the good news: The Riemann and Lebesgue sums converge to the same value (the true analytic definite integral) as the number of partitions increases, which you'll demonstrate in Exercise 12. The Lebesgue sum can take longer to converge, but as $\Delta y \to 0$, the approximating sum turns into the integral, just like with Riemann's approximation method.

Limitations of the Riemann sum The Riemann sum, and the Riemann definition of the integral, is excellent for conceptual understanding and for practical implementation in many situations, but it has some limitations.

In particular, the Riemann integral requires function continuity (in some cases it is possible to split the function into continuous partitions and calculate the integral of each partition, but this is not guaranteed to work for all discontinuous functions), is not suitable for unbounded integrals (that is, if one bound is $\pm\infty$; although there are ways to adapt the Riemann sum that you'll learn about in Chapter 15), and can be practically and computationally intractable for multivariable functions. Furthermore, not all functions can be integrated using Riemann integration.

These limitations are not fatal, and the Riemann integral is still the most popular and widely used method of approximating and defining integrals. But the Lebesgue integral is designed to handle failure scenarios of the Riemann integral.

Advantages of the Lebesgue integral There are functions whose integrals cannot be calculated using the Riemann integral but can be calculated using the Lebesgue integral.

The Lebesgue integral also generalizes to multivariable functions better than the Riemann integral does. This is advantageous for certain problems in higher-dimensional geometry and physics. The Lebesgue integral is also used to solve certain proofs in pure

mathematics.

To be honest, you probably won't encounter the Lebesgue integral unless you study advanced mathematics. My primary motivation for including it in this book is to introduce you to the idea of partitioning the range instead of the domain. And my secondary motivation is that the Lebesgue approximation provides several opportunities for challenging coding exercises that will boost your math-coding skills (Exercises 11-12).

13.5 Exercises

13.1. The goals of Exercises 1-4 are to implement and explore the Riemann integral approximation. This exercise involves creating the partition points according to Equation 13.2.

Calculate Δx based on $a = -.5$, $b = \pi$, and $n = 14$. Define partition breakpoints twice, once implementing Equation 13.3 using list-comprehension, and once using `np.linspace` to obtain linearly spaced numbers between a and b. Calculate the distance between each x_i value. Finally, print out your results as follows:

```
delta-x: 0.260
x_p dist.: 0.260

Breakpoints from list comprehension:
['-0.500', '-0.240', ..., '2.621', '2.881', '3.142']

Breakpoints from np.linspace:
['-0.500', '-0.240', ..., '2.621', '2.881', '3.142']
```

I've truncated the list of breakpoints so they would fit in the page width of the printed book; you should print all 14 values. This exercise shows that you can discretize the x-axis in two ways — by specifying Δx or by specifying n — and get an identical partitioning.

13.2. The purpose of this exercise is to graph the function and its bounds, and a shaded region indicating the area we will approximate. This exercise is only for visualization; approximating the area is the goal of the next exercise.

Translate $f(x) = e^x/10 + \cos(x)$ into a Python function that takes x values (e.g., as a `numpy` array) as input and provides corresponding values of y as output.

This is the function that creates Figure 13.3.

Note that the bounds and x-axis resolution for plotting the function line is different from those for approximating the integral that you created in the previous exercise. Your results should look like Figure 13.13.

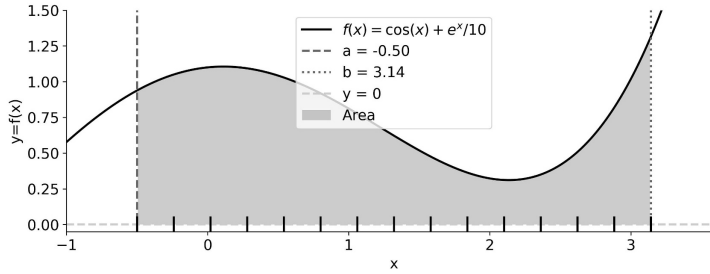


Figure 13.13: Results for Exercise 2. The x-axis ticks indicate the boundary points for the geometric approximation.

13.3. Now to approximate that shaded area. Do this four times: Using the Riemann approximations using the left, right, and midpoint rules, and once using `sympy` to determine the area analytically. The `sympy` result can be considered the ground truth, and the approximations should be reasonably close. Print your results as follows:

```
    Using left rule: 2.69394686
   Using right rule: 2.79170636
Using midpoint rule: 2.72785581
     Sympy integral: 2.73284174
```

In this case, the approximations are all fairly close to `sympy`'s result.

I suppose you might be wondering how those estimates change when you change the number of partitions. That exploration is fairly easy to implement in code, and is the main purpose of Exercise 5.

13.4. The goal here is to create Figure 13.3 (page 439). You might find it useful to start by copying the code from Exercise 2.

There are several ways to draw the boxes; my recommendation is to draw each rectangle separately using the `fill_between` function in a for-loop over partitions.

13.5. In this exercise, you will observe how the approximations converge to the exact definite integral as the number of partitions increases.

Estimate the integral using the left, right, and midpoint rules, in a for-loop over a range of partitions from $n = 5$ to $n = 190$. Visualize the results as in Figure 13.14, with a horizontal dashed line representing the analytic result obtained from `sympy`.

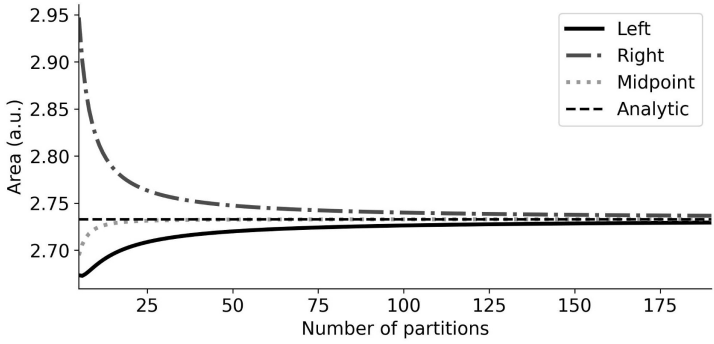


Figure 13.14: Results for Exercise 5.

From visual inspection, the approximations converge within around 100 partitions, with the right-rule providing the only overestimate.

But be careful about generalizing from this example: On the one hand, the various approximation rules will always converge to the analytic result as the number of partitions increases. On the other hand, the rate at which the accuracy increases, and which rules under- vs. overestimate the true integral, depend on the characteristics of the function in the specified bounds.

13.6. Write code to visualize and implement the trapezoid rule to approximate the area of the function $f(u) = u^4 - .5$ and

I called the variable u instead of x

produce Figure 13.15. Implement the trapezoid rule manually by calculating the area of each trapezoid (as shown in Figure 13.4); don't use the `spi.trapezoid` function[1].

Use $n = 6$ partitions between $u = -.5$ and $u = 1$. Color each bar according to the sign of its area (red for negative, green for positive).

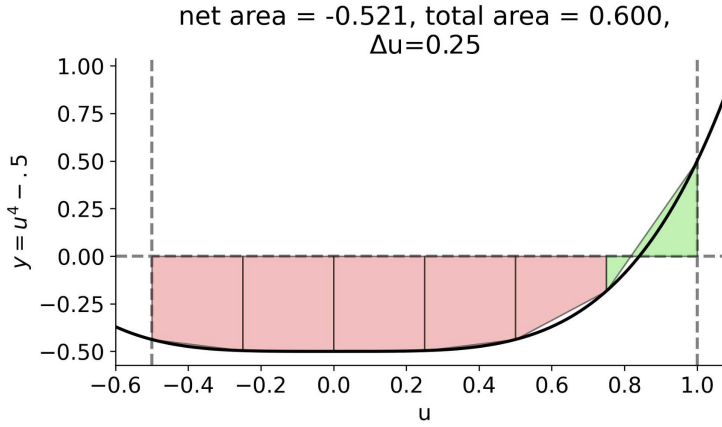


Figure 13.15: Results for Exercise 6.

Notice that the right-most bar in Figure 13.15 includes both positive and negative regions of the function, but most of the area is positive, so the entire area counts as positive. This introduces estimation error, although such errors decrease as Δu decreases (which means more, thinner, trapezoids).

13.7. In this and the next exercises, you will explore using `sympy` to calculate net and total areas. Write code to reproduce Figure 13.9 (page 445). Calculate the net and total areas using `sym.integrate`. The function definitions are printed below, and the integration interval is $[-.5, 1]$ for all functions.

$$f(\tau) = e^{-\tau}\cos(2\pi\tau) \tag{13.11}$$

$$g(\tau) = \tau^3 - 1 \tag{13.12}$$

[1]To be clear, `spi.trapezoid` is a professional-quality function that you should use in applications; I want you to avoid using it for this exercise so you can implement the trapezoid rule manually.

$$h(\tau) = e^{-|\tau|} \tag{13.13}$$

13.8. The goal of this exercise is to calculate the area of the cosine function in the interval shown in Figure 13.16. In particular, the goal is to use `sympy` to determine the following definite integrals:

$$A_N = \int_{\pi/2}^{5\pi/2} \cos(x)\, dx \tag{13.14}$$

$$A_{Ta} = \int_{\pi/2}^{3\pi/2} |\cos(x)|\, dx \tag{13.15}$$

$$A_{Ts} = \left| \int_{\pi/2}^{s} \cos(x)\, dx \right| + \left| \int_{s}^{5\pi/2} \cos(x)\, dx \right| \tag{13.16}$$

Note that the upper bound of the first integral in A_{Ts} must match the lower bound of the second integral.

The net area (A_N) is simply the definite integral using the lower and upper bounds. The total area A_{Ta} is calculated by integrating the absolute value of the function. The total area A_{Ts} is calculated by subdividing the integration interval into sections where the sign of the function is the same, then summing the absolute values of those definite integrals.

The first steps of this exercise is to implement the integrand as a `sympy` expression and visualize the integration areas as in Figure 13.16. Calculating A_{Ts} requires additional code to find the splitting value s. Use `sym.solveset` and specify that the domain for the solution should be in the open interval $(\pi/2, 5\pi/2)$.[2]

[2]Hint: Use the `open` method on the `Interval` function to specify an open interval.

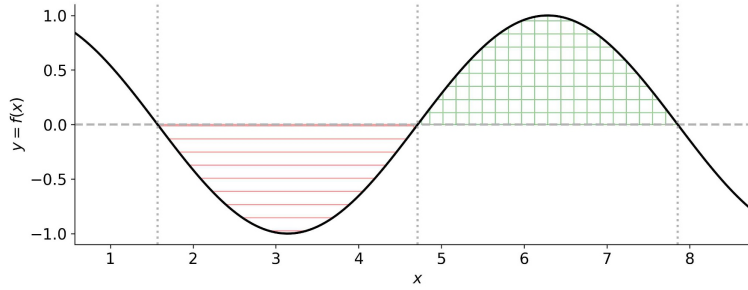


Figure 13.16: Visualization for Exercise 8. The negative area is hatched with minus signs, and the positive area is hatched with plus signs.

Next, calculate the areas. The net and total areas using absolute value (A_{Ta}) can be obtained using one line of code; A_{Ts} requires at least one line of code per subinterval. Finally, print out your results as follows:

$$A_n = \int_{\frac{\pi}{2}}^{\frac{5\pi}{2}} \cos(x)\, dx = 0 \tag{13.17}$$

Note: `sympy`'s answer to A_{Ta} is wrong!

$$A_{Ta} = \int_{\frac{\pi}{2}}^{\frac{5\pi}{2}} |\cos(x)|\, dx = 2 \tag{13.18}$$

$$A_{Ts} = \left| \int_{\frac{\pi}{2}}^{\frac{3\pi}{2}} \cos(x)\, dx \right| + \left| \int_{\frac{3\pi}{2}}^{\frac{5\pi}{2}} \cos(x)\, dx \right| = 2 + 2 = 4 \tag{13.19}$$

Even though you don't yet know how to hand-calculate integrals (topic of the next chapter!), you should recognize that the `sympy` result for A_{Ta} is incorrect (the other two definite integrals are accurate). `sympy` struggles with integrating the absolute value of trig functions, because the antiderivative is an infinitely alternating piecewise function with boundaries of $k2\pi$ for integers k.

13.9. This exercise will continue to explore definite integrals, and net and total areas, using `sympy`. There are two parts to this exercise: an investigation and an exploration.

For the first part, begin by implementing the function $f(x) = x^2 - 2x$ as a symbolic expression. In a for-loop, use `sym.integrate` to calculate the net and total areas using different intervals. The lower bound is always fixed at $a = -1/2$, and the upper bound varies in 30 linearly spaced steps between $b = 0$ and $b = 3.5$.

Figure 13.17 shows the results. The black line in panel A is the function, and the two lines in panel B show the areas for different values of the upper integration bound b.

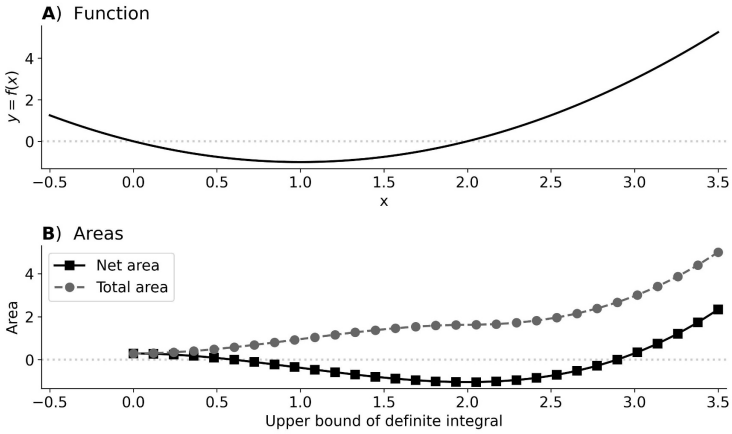


Figure 13.17: Results for Exercise 9.

Notice that the total area is a monotonically increasing function[3], which makes sense because the total area over an increasing subdomain can increase or remain constant, but could not decrease.

Now for the second part of this exercise: Use `sympy` and latex to print the antiderivatives of the net and total areas of the function. Your output should look like the following:

$$\int \left(x^2 - 2x\right)\, dx \;=\; \frac{x^3}{3} - x^2 \tag{13.20}$$

$$\int \left|x^2 - 2x\right|\, dx \;=\; \int \left|x^2 - 2x\right|\, dx \tag{13.21}$$

[3] A "monotonic" function is one that only increases or only decreases, though the changes could be linear or nonlinear.

Notice that `sympy` did not display an antiderivative function, but instead returned the same integration expression that was provided as an input. In other words, the integration remains unevaluated. It is possible to write out an explicit form of this antiderivative, which would require a piecewise function because the antiderivative depends on the value of x. For this reason, `sympy` calculates exact solutions if provided integration bounds. For example, the following is the output of `sym.integrate(sym.Abs(f),(x,0,1))`

$$\int_0^1 \begin{cases} x^2 - 2x & \text{for } x^2 - 2x \geq 0 \\ -x^2 + 2x & \text{otherwise} \end{cases} dx$$

Here's a final bonus part of this exercise: a `sympy` failure scenario. Keep the code the same but use the function $f(x) = \cos(2\pi x)$. I don't show the results here, but it's in my online code solution, along with a brief discussion.

I will return to this issue in the next chapter, but this and the previous exercise clearly illustrate why it's best to calculate total area by approximating definite integrals with high resolution using `scipy`.

13.10. The goal of this and the next two exercises is to implement the Lebesgue approximation and compare it against the Riemann approximation.

Start this exercise by coding a Riemann approximation and associated graph. There's actually nothing new in this exercise, but it will set you up for the comparison in Exercise 11. If you want to challenge yourself, you can code this exercise from scratch; otherwise, feel free to copy and modify code from earlier in this chapter.

Create Figure 13.18 (using $n = 12$). I used the midpoint rule, but you can also use the left or right rules if you prefer. I also coded the bars to be lighter with increasing x, although that's just an aesthetic choice that is not relevant for this exercise.

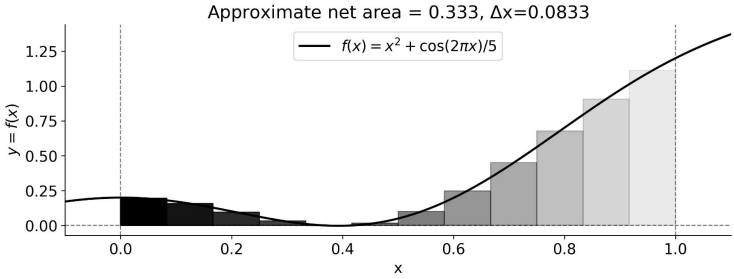


Figure 13.18: Results for Exercise 10.

13.11. Now to implement the Lebesgue approximation of the function and bounds described in the previous exercise, according to the procedure described in the next few paragraphs. *Warning:* Implementing the Lebesgue approximation in Python is fairly complicated. Feel free to consult my online solution.

The first subgoal is to partition the range of the function between two x-axis bounds. This is not as simple as partitioning the domain, because the range depends on the characteristics of the function. Thus, start by defining a grid of 1000 x-axis values between $x = a$ and $x = b$, evaluate $f(x)$ at each of those values, and find the smallest and largest values of y. Then, define the range partition boundaries as $n + 1$ linearly spaced numbers between y_{min} and y_{max}. You can also define Δy from these partition boundaries.

The next subgoal is to measure the size of each y partition. In a for-loop over the y partitions, find and count the x-axis values (from the discretized grid) for which the corresponding y values are within that partition, scaled by Δx. Multiply this quantity by the average y value in the y-partition to obtain the area for this partition. Then sum over all partitions.

The final subgoal is to visualize the horizontal bars. This part can be tricky to code because there might be multiple separate bars for the same y partition. You will need to write code to identify and separate contiguous x-axis

points. For example, a set of indices [1,2,3,7,8,9,10] would need to be split into two sets [1,2,3] and [7,8,9,10] so that each can be drawn as a rectangle with the same y-axis coordinates.

Your results should look like Figure 13.19 (but don't worry about the grayscale gradient). In the online code, I've provided two additional options for visualizing the bars.

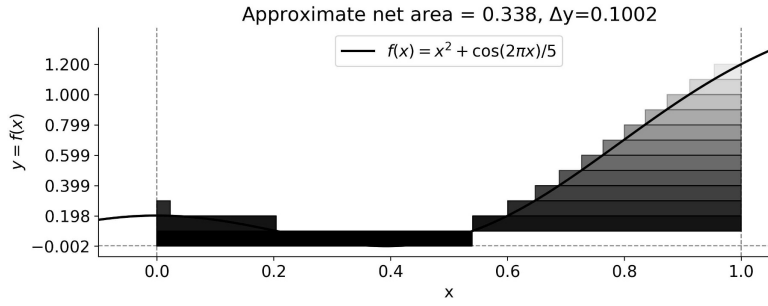


Figure 13.19: Results for Exercise 11.

13.12. Now that you have code for both the Riemann and Lebesgue approximation methods, it's time to compare them directly. Copy the code from Exercises 10 and 11 — just the area calculations, not the visualizations — into two Python functions (one function for each method). Using $n = 12$ for both methods, confirm the approximate areas, as well as the result of `sym.integrate`, for the same function and boundaries that you used in the previous two exercises. You should get the following results:

```
Riemann:  0.332755
Lebesgue: 0.336895
Sympy:    0.333333
```

Now for the comparison of convergence. Approximate the area using $n = 4$ to $n = 40$ in a for-loop, and plot the resulting approximations as in Figure 13.20.

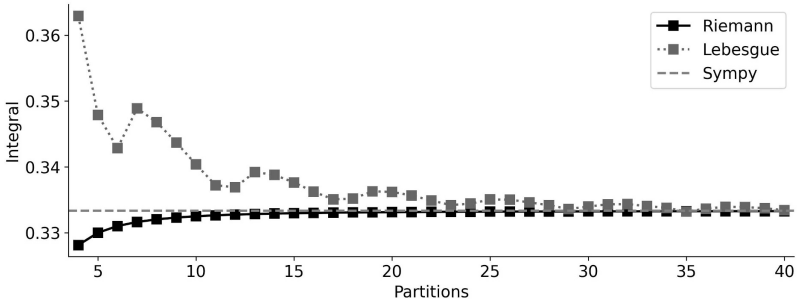


Figure 13.20: Results for Exercise 12.

You can see that both methods converge to the analytically determined value, although the Riemann approach converged faster and smoother. It is not trivially always the case that the Riemann method will converge faster, although, as I wrote earlier in the chapter, Riemann's method is simpler and should be preferred except in certain situations such as high-dimensional integration.

Important note: The code for approximating integrals in this chapter is for educational purposes only. Please don't use this code for real-world applications (except for educational applications ;)). Instead, use the more accurate and numerically stable algorithms provided by `scipy` that you will learn about in the next chapter.

Chapter 14

Integrating functions

I hope you now feel comfortable with the main concepts of integration. The purpose of this chapter is to introduce you to the analytic methods for integrating functions. This is a major topic in calculus, and so it's spread across three chapters.

Before starting, let me remind you of the distinction between a definite integral (A, one number reflecting the area under the function curve between two x-axis bounds), and an indefinite integral ($F(x)$, also called an antiderivative; has no bounds and produces a function).

$$A = \int_a^b f(x)\,dx \qquad \text{One number}$$

$$F(x) = \int f(x)\,dx \qquad \text{A function value for each value of } x$$

14.1 The power rule for integration

The power rule for integration is like the inverse of the power rule for differentiation. I hope that the differentiation power rule is permanently etched into your brain, but here's a quick reminder:

$$[\,x^n\,]' = nx^{(n-1)}$$

Fortunately, the power rule for integration is easy to learn, easy to remember, and easy to apply. In fact, the power rule is so simple and powerful that it's one of the reasons why polynomials are so important in both pure and applied mathematics. Indeed, when you get to advanced topics in integration, you will breathe a sigh of relief when you see polynomial expressions in the integrands.

14.1.1 Indefinite integrals

Here is the rule for integrating exponents on variable x:

$$\int x^n \, dx \;=\; \frac{1}{n+1} x^{n+1} + C \tag{14.1}$$

I introduced the constant of integration (C) in Chapter 12. The next section is solely dedicated to understanding what that term means and why it's necessary. For now, try to build the habit of including that term when integrating.

Anyway, to integrate an expression of the form x^n, increment the exponent by one, and then bring down that incremented exponent as a divisive coefficient.

Why is this the correct way to integrate x^n? The proof comes from assuming that Equation 14.1 is correct and then taking its derivative. By the FTC-1, we know that $[\int f(x) \, dx]' = f(x)$. Thus, if the derivative of the antiderivative is the original function, then we have the correct antiderivative.

$$\frac{d}{dx}\left[\frac{1}{n+1} x^{n+1} + C\right] \;=\; \frac{n+1}{n+1} x^{n+1-1} + 0 \tag{14.2}$$

$$\;=\; x^n \tag{14.3}$$

Equation 14.1 is so important that you really need to memorize it. Again, the idea is to add 1 to the exponent and then divide by the new exponent (and remember to add C).

The exception to this rule is when $n = -1$, because that would lead to $\frac{x^0}{0}$. I'll get back to this exception in a few pages.

A meta-comment: You learn how to differentiate functions by plugging those functions into the formal limits definition, doing some algebraic simplifications, and then getting an answer. That's

working *forwards* from a problem to the solution. On the other hand, you learn how to integrate by being told the solution and then confirming it via the FTC-1. That's working *backwards* from the solution to the problem. It took real math geniuses — true intellectual giants of humanity — to develop calculus, but you can now benefit from centuries of their hard work and learn calculus in a much shorter period of time and with only modest intellectual prowess and dedication. I find that thought humbling and encouraging.

Anyway, let's solve some practice problems. Determine the antiderivatives of the following expressions. Then differentiate the antiderivatives to check that you recover the original function. The answers are printed below, so don't look too far down the page until you've worked through these examples ;)

$$\begin{array}{lll} (\mathbf{1}) \int x^2\,dx & (\mathbf{2}) \int 4x\,dx & (\mathbf{3}) \int x^2 + 4x\,dx \\ (\mathbf{4}) \int \sqrt{x}\,dx & (\mathbf{5}) \int x^2 \cdot x^3\,dx & (\mathbf{6}) \int \frac{1}{x^7}\,dx \end{array}$$

I hope you managed to solve those with minimal confusion. Remember that $\sqrt{x} = x^{1/2}$ and that $1/x^7 = x^{-7}$.

Here are the answers:

$$\begin{array}{lll} (\mathbf{1}) \frac{1}{3}x^3 + C & (\mathbf{2}) \; 2x^2 + C & (\mathbf{3}) \frac{1}{3}x^3 + 2x^2 \\ (\mathbf{4}) \frac{2}{3}x^{3/2} + C & (\mathbf{5}) \frac{x^6}{6} + C & (\mathbf{6}) \frac{-1}{6x^6} + C \end{array}$$

Did you get the same results as I did? Perhaps there was a minor discrepancy somewhere?[1]

The exception for x^{-1} The antiderivative of $1/x$ is $\ln|x|$. That follows from the derivative of $\ln(x)$ (see also Equation 5.79, page

[1] I realized when checking my own work that I forgot to include the C in (**3**), and I decided to leave that mistake in there to make sure you're being careful ;)

201).

More generally, for any scalars a and b:

$$\int \frac{a}{x+b}\,dx \;=\; a\ln|x+b|+C \tag{14.4}$$

14.1.2 Definite integrals

There are four steps to calculating definite integrals. These four steps apply to integrating any type of function, not only polynomials.

Step 1:	Write down equation and bounds.	$A=\int_a^b f(x)\,dx$
Step 2:	Simplify and apply tricks.	oOo *magic* oOo
Step 3:	Calculate the antiderivative.	$F(x)=\int f(x)\,dx$
Step 4:	Plug in bounds and subtract.	$A=F(b)-F(a)$

Step 2 is hard (Step 3 can be hard if Step 2 wasn't successful enough). This is the step that takes years to master. The goal of Step 2 is to simplify, expand, re-express, or apply any other tricks to make Step 3 as manageable as possible. Some of the integration tricks you'll learn over the next few chapters may feel like magic, but they're not: they're the result of centuries of tinkering by brilliant mathematicians.

Let's work through a few examples. There is some new notation in the first example below; see if you can guess what it means, and I'll provide a written explanation below.

$$\int_0^3 x^2\,dx \;=\; \frac{x^3}{3}+C\,\Big|_0^3 \tag{14.5}$$

$$=\left(\frac{3^3}{3}+C\right)-\left(\frac{0^3}{3}+C\right) \tag{14.6}$$

$$= 9 + C - 0 - C = 9 \tag{14.7}$$

The new notation is the vertical line with the integration bounds. That symbol indicates to evaluate the antiderivative by replacing x with the integration bounds, and then subtract the result with the lower integration bound from the result with the upper integration bound. That is, evaluate the antiderivative (in this example: $\frac{x^3}{3} + C$) for $x = 3$ and $x = 0$, and then subtract those two results.

Why do you evaluate and subtract? That comes from the FTC-2:

$$\int_a^b f(x)\,dx = F(b) - F(a) \tag{14.8}$$

If you didn't fully understand the transition from Equation 14.5 to 14.6, then please re-inspect those equations now that you know what the notation means and how this example follows the FTC-2.

Notice that the constant of integration canceled. That is no quirk of this specific example; the constant of integration is not a function of x, and therefore will *always* cancel. For this reason, you don't need to include the constant of integration when calculating a definite integral. Observe:

$$\int_0^3 x^2\,dx = \frac{x^3}{3}\Bigg|_0^3 \tag{14.9}$$

$$= \frac{3^3}{3} - \frac{0^3}{3} = 9 \tag{14.10}$$

Congratulations! You now know how to calculate definite integrals of polynomial expressions. The four-step procedure I described at the outset of this subsection is the way to calculate any definite integral for any function, when the goal is to calculate net area. Calculating total area involves a slight modification to Steps 3 and 4, which I'll discuss later in the chapter.

Let's do another example. Determine the following definite integral. My solution is below, so don't look down the page until after you solve this.

$$\int_{-2}^{1} x^2 + 4x \, dx \tag{14.11}$$

The answer is -3, and Figure 14.1 shows the function and the requested area shaded. You can see that the area includes both positive and negative regions, but the negative area is larger, thus giving a net negative area. The equations below show the full solution.

$$\begin{aligned}
\int_{-2}^{1} x^2 + 4x \, dx &= \frac{x^3}{3} + 2x^2 \Big|_{-2}^{1} && (14.12)\\
&= \frac{1}{3} + 2 - \left(\frac{-8}{3} + 8\right) && (14.13)\\
&= \frac{1}{3} + 2 + \frac{8}{3} - 8 && (14.14)\\
&= \frac{9}{3} + 2 - 8 \;=\; -3 && (14.15)
\end{aligned}$$

Be mindful of all the minus signs floating around!

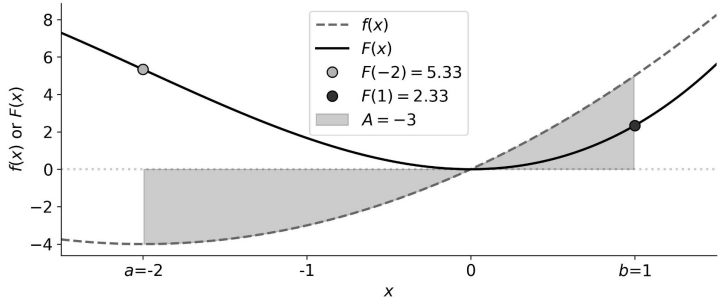


Figure 14.1: Visualization of the example problem.

Before moving to the next example, let me explain what Figure 14.1 shows. The dashed gray line and the gray shaded region are the function and the area between its curve and the $y = 0$ axis, just like what you've seen in the previous two chapters.

As with the derivative of a function, the indefinite integral does not look like the function from which it was calculated.

The new feature in this plot is the black line corresponding to the antiderivative $F(x)$. The two circles plotted on top of the integral are the values of the antiderivative at the boundaries used to calculate the definite integral, which in this example is $F(-2)$ and $F(1)$. The FTC-2 tells us that the definite integral (the net area of $f(x)$ between $x = a$ and $x = b$) equals $F(b) - F(a)$. In this example, that is $2.33 - 5.33 = -3$.

One more example:

$$\int_0^3 \pi \, dx \tag{14.16}$$

The answer is 3π. If you got stuck on this one, keep in mind that $f(x) = \pi = \pi x^0$. In other words, a constant function is a polynomial with $n = 0$. Here's the solution:

$$\int_0^3 \pi \, dx = \frac{\pi}{1} x^1 \Big|_0^3 \tag{14.17}$$

$$= \pi(3) - \pi(0) = 3\pi \tag{14.18}$$

Thus, the definite integral of a constant function is simply the difference of that constant times the bounds. This should make sense when considering that the graph of a constant function is a horizontal line; thus, the area under the function is the area of the rectangle with height C and width $b - a$ (Figure 14.2).

14.2 The constant of integration

Here's a motivating question: In the equation below, which of the three expressions on the right-hand-side is the correct antiderivative?

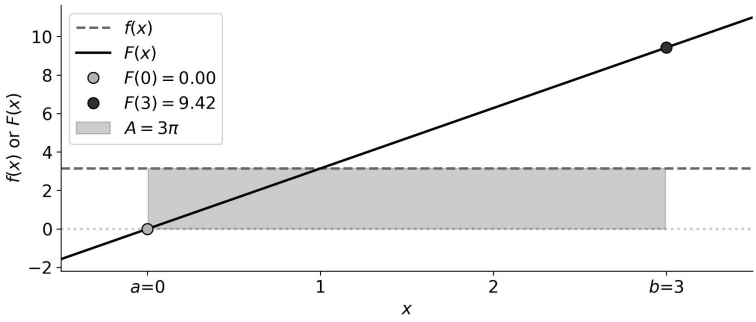


Figure 14.2: Visualization of the second problem.

$$\int x^2\,dx \stackrel{?}{=} \begin{cases} \dfrac{1}{3}x^3 \\ x^3\dfrac{1}{3} + 10 \\ \dfrac{x^3}{3} - 14.7 + \pi^e \end{cases}$$

The answer, as I guess you've guessed, is that they're all correct. That may seem strange, because $f(x) = x^2$ is *one* function, so how can its indefinite integral equal three *distinct* functions? And of course, those are just three specific examples; there is an infinite number of unique functions that could go into the list on the right-hand-side — all of which have the $x^3/3$ term and some constant added.

The key insight here, which you'll remember from earlier chapters, is that the derivative of a constant is zero. That is, $[\,C\,]' = 0$. The derivative of all the equations in the right-hand-side is x^2, meaning we have no idea which *particular* function actually corresponds to $f(x)$.

14.2.1 Fundamental Theorem of Calculus, with *C*

Including the constant of integration in the FTC-1 is more precise than how I introduced it in Chapter 12 (Equations 12.10 and 12.11):

$$\frac{d}{dx}\left[\int f(x)\,dx + C\right] = f(x) \tag{14.19}$$

$$\int f'(x)\,dx = f(x) + C \tag{14.20}$$

The constant does not need to be included when writing out FTC-2, because of the cancellation. I provided a demonstration and algebraic explanation for the cancellation above, and I'll provide a geometric explanation in the next subsection.

14.2.2 Geometry of *C*

The constant of integration reflects a shift on the y-axis. That is, the entire curve can be shifted up or down without changing the shape of the curve.

This is illustrated in Figure 14.3: The three curves in panel A correspond to three distinct functions, because they have distinct constants (but the same two x terms). Their derivatives, however, are identical (panel B) because the constant terms all differentiate to zero.

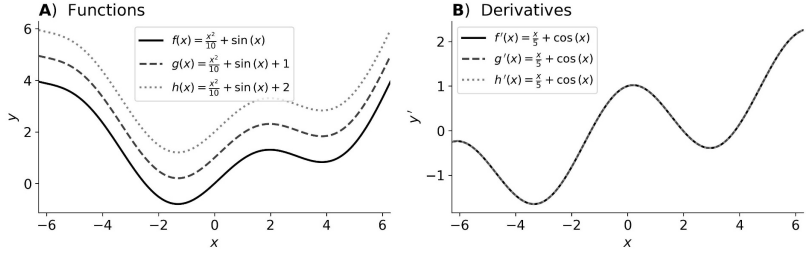


Figure 14.3: Distinct constants create distinct functions (panel A), yet those functions have identical derivatives (panel B).

It is also revealing to observe what happens when you integrate these functions. You'll explore that in the exercises.

Constant of integration in definite integrals You learned in the previous section that the constant of integration is not needed in

definite integrals. I showed why that's the case algebraically (it cancels in the subtraction), and now I will provide a geometric explanation.

The FTC-2 tells us that calculating a definite integral involves subtracting the values of the antiderivative at the two integration bounds. Figure 14.4 shows three curves corresponding to the indefinite integral of the same function, with different values for C (0, 1, or 2). Those distinct values of C shift the integral on the y-axis. But because that is simply a linear translation, the difference between $F(b)$ and $F(a)$ remains the same (note that the slope of the line connecting the pairs of circles is the same).

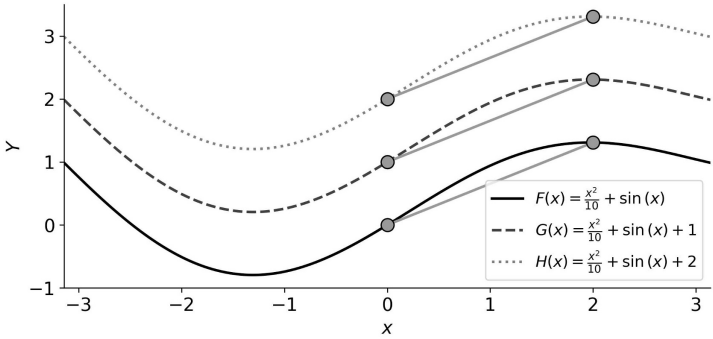


Figure 14.4: Constants have no impact on the discrete integral, because that is calculated as the y-axis difference between two x-axis points on the indefinite integral.

14.2.3 Indeterminacy of indefinite integrals

The phenomenon that a unique function has an infinite number of equally correct antiderivatives is called the "indeterminacy of indefinite integrals." This indeterminacy is not really a *problem* per se, but can cause confusion and ambiguity, and therefore requires a solution.

There are two solutions to this indeterminacy: include a generic constant of integration (this is the approach taken so far), or determine a particular value of C to resolve the indeterminacy.

The second solution is possible if you are given an additional con-

straint about the function, which is given as the value of the function at a particular value of x. For example, you might be asked to integrate a function and then will be given the information that, e.g., $f(0) = 4$. This is called an "initial value problem."

14.2.4 Initial value problems

Initial value problem is sometimes abbreviated to IVP, although I will refrain from this compressive indulgence.

You now understand that an infinite number of functions have the same derivative, because constants simply shift the function on the y-axis without changing any of the local slopes.

On the other hand, there is exactly one unique function that passes through a certain value. Therefore, the idea of an "initial value" is to provide a constraint — one y value of the function for a given x value — that excludes all-but-one of the infinite number of functions.

A typical initial value problem statement looks like this:

"Given $f'(x) =$ _____ and given $f(x_0) =$ ___, find $f(x)$"

In other words, you are given the derivative and one unique (x, y) pair, and are asked to find the unique function. The strategy for solving a problem like this is to integrate to find $f(x)$ including the constant of integration, then use the constraint to solve for C.

Here's an example:

$$\text{Given } f'(x) = 2x + 3 \text{ and } f(1) = 2, \text{ find } f(x). \tag{14.21}$$

To find a function given its derivative, integrate the derivative (following FTC-1).

$$f(x) = \int 2x + 3\,dx \tag{14.22}$$

$$= x^2 + 3x + C \tag{14.23}$$

That's the beginning of the solution. We now have a general form of $f(x)$, although we don't have a particular solution. This is where the constraint enters the solution:

$$f(1) = 2 \tag{14.24}$$

$$2 = (1)^2 + 3(1) + C \tag{14.25}$$

$$2 = 4 + C \tag{14.26}$$

$$-2 = C \tag{14.27}$$

And this brings us to our final answer: $f(x) = x^2 + 3x - 2$. That is a unique solution with no indeterminate constant of integration.

Figure 14.5 illustrates the initial value problem. The derivative of the function is drawn as a thick dashed gray line. The antiderivative is $f(x)$, but *which* $f(x)$ is it? A few of the infinite possibilities are shown in the thin gray lines. The constraint (white circle) provides an intersection point that the function (with the indeterminate constant of integration) must satisfy, which reduces the solution space to one particular solution (thick black line).

So in fact, solving the indeterminacy issue boils down to middle-school algebra of solving an equation for C.

Writing an algorithm in `sympy` to solve the initial value problem is a great exercise in thinking and coding, and I hope you're looking forward to that experience in the exercises.

14.3 Linearity of integration

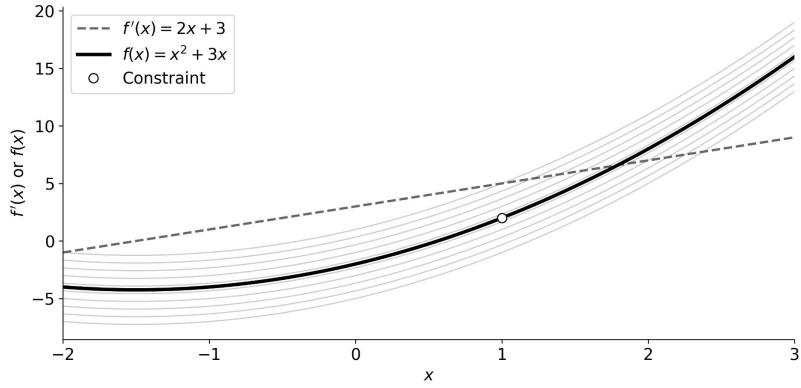


Figure 14.5: Initial value problem: One unique solution (thick black line) can be selected from the infinity of possible antiderivatives (think gray lines) if an additional constraint (white dot) is provided.

The goal of this section is to describe and prove the following mathematical statement, which describes two properties of integration. I will explain this in detail, but I'd like you to challenge yourself by trying to understand what this equality means before reading further.

α and β are real-valued scalars.

$$\int \alpha f(x) + \beta g(x)\, dx \;=\; \alpha \int f(x)\, dx + \beta \int g(x)\, dx \qquad (14.28)$$

This equation combines two features of linearity: scalar multiplication and function addition. Those can be expressed separately.

$$\int \alpha f(x)\, dx \;=\; \alpha \int f(x)\, dx \qquad (14.29)$$

$$\int \left[\, f(x) + g(x) \,\right] dx \;=\; \int f(x)\, dx \;+\; \int g(x)\, dx \qquad (14.30)$$

Equation 14.29 states that scalars can be moved outside the integral, for example:

$$\int 4x\, dx = 4 \int x\, dx \qquad (14.31)$$

Equation 14.30 states that the integral of the sum of functions equals the sum of the integrals of those functions. For example:

$$\int x^2 + \sqrt{x}\,dx \;=\; \int x^2\,dx + \int \sqrt{x}\,dx \tag{14.32}$$

These two properties are useful when solving integration problems, because they can simplify the expressions and allow you to split a difficult expression into a collection of simpler expressions.

Do these two properties seem familiar? Do they bring back memories from earlier in this book? I sure hope so! Limits and differentiation follow these same two properties — and in fact, the linearity of limits is key to proving these two properties for integrals.

In the subsections below, I will use the Riemann definition of integrals to prove these limits, and thereafter show examples of using these properties, and finally provide geometric intuition for what they mean.

Even before seeing the mathematical proofs, I hope you have the intuition that these two properties hold, partly because integration is like inverse differentiation, and partly because integration can be defined from limits.

14.3.1 Proving the scalar multiplication property

Try to follow this sequence of equations before reading my explanation below.

$$\int_a^b f(x)\,dx \;=\; \lim_{\Delta x \to 0} \sum_{i=1}^{n} f(x_i)\,\Delta x \tag{14.33}$$

$$\int_a^b \alpha f(x)\,dx \;=\; \lim_{\Delta x \to 0} \sum_{i=1}^{n} \alpha f(x_i)\,\Delta x \tag{14.34}$$

$$= \lim_{\Delta x \to 0} \alpha \sum_{i=1}^{n} f(x_i)\,\Delta x \tag{14.35}$$

$$= \alpha \lim_{\Delta x \to 0} \sum_{i=1}^{n} f(x_i)\, \Delta x \tag{14.36}$$

$$= \alpha \int_a^b f(x)\, dx \tag{14.37}$$

Equation 14.33 is the Riemann definition of the definite integral. Equation 14.34 includes a scalar multiple α. Equations 14.35-14.36 move the scalar from inside the summation to outside the limits — this is possible because of the linearity of summation and of limits. Finally we arrive at 14.37, which equals the left-hand-side of Equation 14.34 and therefore completes the proof.

14.3.2 Proving the function addition property

Now for the addition property: The integral of a sum equals the sum of the integrals. Try to explain the proof using the following equations, and then I will provide my explanation below.

$$\int_a^b [\, f(x) + g(x) \,]\, dx = \lim_{\Delta x \to 0} \sum_{i=1}^{n} [\, f(x) + g(x) \,] \Delta x \tag{14.38}$$

$$= \lim_{\Delta x \to 0} \left[\sum_{i=1}^{n} f(x) \Delta x + \sum_{i=1}^{n} g(x) \Delta x \right] \tag{14.39}$$

$$= \lim_{\Delta x \to 0} \sum_{i=1}^{n} f(x) \Delta x \; + \lim_{\Delta x \to 0} \sum_{i=1}^{n} g(x) \Delta x \tag{14.40}$$

$$= \int_a^b f(x) dx \; + \int_a^b g(x) dx \tag{14.41}$$

We begin again with the Riemann definition of the integral in Equation 14.38. Equation 14.39 leverages the linearity of summation to rewrite the limit as the sum of two summations for each function. Then Equation 14.40 uses the linearity of limits to separate the limit of two sums into the sum of two limits. And finally we evaluate those limits in Equation 14.41 to arrive at our goal.

14.3.3 Examples

Simplify and then solve the following integrations. I'll write and describe the solutions below.

$$\int 2x^2\,dx \tag{14.42}$$

$$\int \pi\sqrt{x} + \frac{3e^2}{x^3}\,dx \tag{14.43}$$

The first one is fairly straightforward; pulling out the constant "2" is probably not necessary but is good practice for this section and will lead to understanding an important subtlety of integration constants.

$$\int 2x^2\,dx \;=\; 2\int x^2\,dx \;=\; 2\left(\frac{1}{3}x^3 + C\right) \;=\; \frac{2}{3}x^3 + C \tag{14.44}$$

Are you confused about the final expression? Perhaps you thought that the constant should be $2C$ instead of just C. It does indeed look like I've accidentally neglected to multiply through.

But it's correct to write C instead of $2C$. Without having a constraint to resolve the indeterminate C, that constant is just a placeholder for *any* constant. In other words, because C is an unknown value, $C = 2C = C + \sqrt{\pi}e^{e^e} = ...$

The second example problem is a good illustration of how the linearity principles will help you simplify an integration.

$$\begin{aligned}\int \pi\sqrt{x} + \frac{3e^2}{x^3}\,dx \;&=\; \pi\int x^{1/2}\,dx \;+\; 3e^2\int x^{-3}\,dx \qquad (14.45)\\ &=\; \left(\frac{2\pi}{3}x^{3/2} + C_1\right) + \left(\frac{-3e^2}{2}x^{-2} + C_2\right)\end{aligned}$$

$$= \frac{2\pi}{3}x^{3/2} + \frac{-3e^2}{2}x^{-2} + C \tag{14.47}$$

I must say, for such complicated-looking functions, their graphs are not particularly interesting to look at (Figure 14.6).

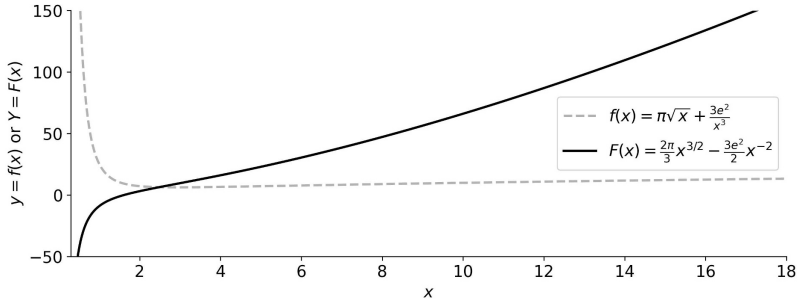


Figure 14.6: Visual depiction of the function and its integral from Equations 14.43 and 14.47. I've set $C = 0$ in $F(x)$.

14.3.4 Mistakes to watch out for

The following two equations are *incorrect*, and I will explain why they are incorrect. But please figure out on your own what the problems are, and think of alternatives that would be correct.

$$\int x^2 + x\,dx \;= \int x(x+1)\,dx \;= x\int x+1\,dx \tag{14.48}$$

$$\int 4x+1\,dx \;= 4\int x+1\,dx \tag{14.49}$$

Have you figured out the problems? Let's start with Equation 14.48. The problem here is that the scalar-multiplication property is only for *scalars*. x is not a scalar, but instead is a variable. You cannot pull it out of the integrand. In fact, although factoring the integrand into $x(x+1)$ is technically correct, it is not the best way to simplify this expression. Better would be the following:

$$\int x^2 + x\,dx = \int x^2\,dx + \int x\,dx$$

Now for Equation 14.49. This is just a silly little arithmetic error. The scalar needs to be factored out of the *entire* integrand, not just one term in the integrand. The following two options would be correct:

$$\int 4x + 1\,dx = 4\int x + \frac{1}{4}\,dx \tag{14.50}$$

$$= 4\int x\,dx + \int 1\,dx \tag{14.51}$$

As I'm sure you know, lots of mistakes in mathematics come from careless errors that happen when you are tired or are rushing through a problem. That means that lots of mistakes can be avoided by approaching the problem with a refreshed (possibly caffeinated) mind and a positive attitude[2].

14.3.5 Geometry of scalars in integration

Figure 14.7 illustrates using definite integrals to calculate the area under two functions, one of which is a scalar multiple of the other: $g(x) = \pi f(x)$. Notice that the area in the same bound increases by a multiple of that scalar.

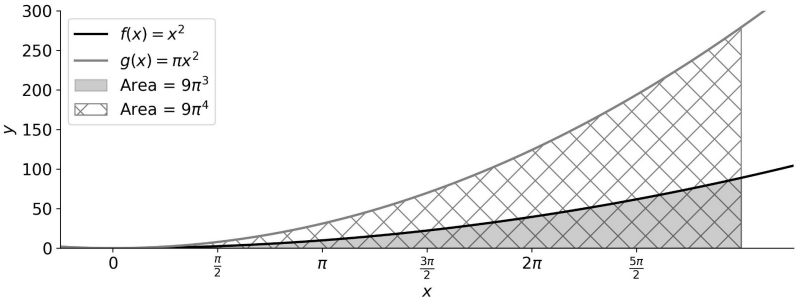


Figure 14.7: Scaling area.

To be clear, this figure does not illustrate the *linearity* of scalar multiplication per se; it illustrates how scalar-multiplying a function multiplicatively increases the definite integral by that scalar.

[2] True for lots of problems in life, not only integration ;)

The next geometric illustration does show the linearity property of addition. Figure 14.8A-B show two functions and the area under their curves between $x = 0$ and $x = 3\pi$. Panel C shows the two functions summed, and the area of $h(x)$ exactly equals the sum of the areas of $f(x)$ and $g(x)$. In Exercise 6, you will code this example and continue exploring it.

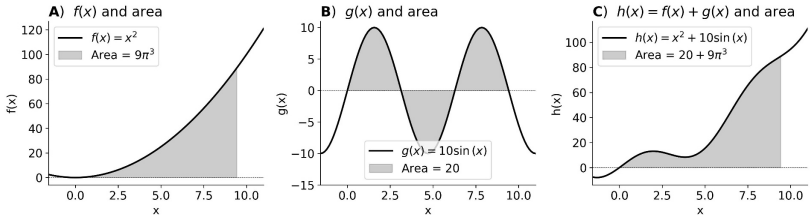


Figure 14.8: Illustration of the additive property of definite integrals. You will create this figure in Exercise 6.

14.4 Integrating transcendental functions

The rules for integrating functions like cosine, sine, natural exponential and natural logarithm are learned in a way that is typical for integral calculus: I will tell you the rule and then you'll confirm it using FTC-1.

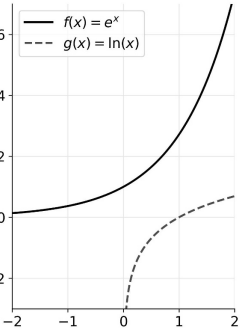


Figure 14.9: Visual reminder of the "natural" functions.

14.4.1 Integrating the natural exponential

The functions e^x and $\ln(x)$ are nearly ubiquitous in pure math and in applications in data science, engineering, physics, biology, finance, navigation, and lots of other fancy-sounding arenas of human intellectual pursuit.

As a brief reminder (see Figure 14.9): e^x is defined for all x, is always positive, and races up to infinity as x increases; $\ln(x)$ is defined for positive x and crawls up to infinity. For $x > 0$, these two functions are inverses of each other, such that $\ln(e^x) = e^{\ln(x)} = x$.

The natural exponential function is really easy to integrate:

$$\int e^x \, dx \;=\; e^x + C \tag{14.52}$$

Recall that the derivative of e^x is e^x, so therefore its integral is also the same function.

If the exponent includes a coefficient other than 1, the integral looks a little bit like the power rule — but it's different, so please don't confuse them.

$$\int e^{\alpha x} \, dx \;=\; \frac{1}{\alpha} e^{\alpha x} + C \tag{14.53}$$

α is any real-valued number except zero.

The power rule involves incrementing the power by one, which isn't the case here.

How do you know that Equation 14.53 is correct? You can confirm it by differentiating the right-hand-side of that equation. I show that result below, and I'd like you to work through it on your own.

$$\frac{d}{dx}\left[\frac{e^{\alpha x}}{\alpha} + C\right] \;=\; \frac{\alpha e^{\alpha x}}{\alpha} + 0 \;=\; e^{\alpha x} \tag{14.54}$$

Figure 14.10 shows the function $f(x) = e^{\alpha x}$ and its integral $F(x)$ for a collection of α values between -1 (darker lines) and 2 (lighter lines) (assuming $C = 0$ for all integrals). The integral lines have a larger spread over the y-axis because of the division by α.

14.4.2 Integrating *ln*(*x*)

The integral of the natural log is not quite as intuitive as the one for the natural exponential, but you'll see why it is correct. Here's the key result:

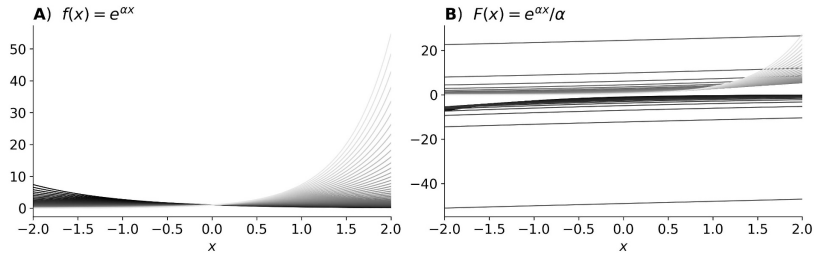


Figure 14.10: Visualizing the natural exponent (panel A) and its integral (panel B) for various power coefficients.

$$\int \ln(x)\,dx \;=\; x\ln(x) - x + C \tag{14.55}$$

Again, the way to prove that this is correct is to start from the assumption that it is correct, and then apply the FTC-1 by differentiating the right-hand-side of Equation 14.55. I show that result below, and I encourage you to confirm it on your own.

$$\frac{d}{dx}\Big[\, x\ln(x) - x + C \,\Big] \;=\; \ln(x) + \frac{x}{x} - 1 \;=\; \ln(x) \tag{14.56}$$

You need to use the product rule, and to remember that the derivative of $\ln(x)$ is $1/x$. Also remember that the domain of the natural log function is $x > 0$.

Coefficients inside the natural log remain inside the log function:

$$\int \ln(\alpha x)\,dx \;=\; x\ln(\alpha x) - x + C \tag{14.57}$$

I encourage you to confirm this statement by differentiating the right-hand-side of Equation 14.57.

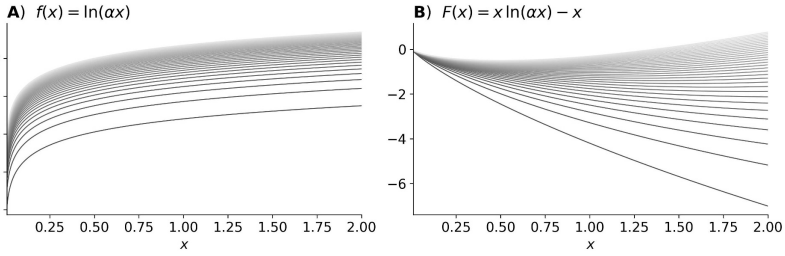


Figure 14.11: Visualizing the natural log function and its integral for several values of α. In my humble opinion, the integrals here are much more visually pleasing compared to those in Figure 14.10B.

14.4.3 Integrating sin and cos

Let me start by reminding you of the derivatives of sine and cosine (Figure 14.12), in particular, the cyclic pattern of derivatives with alternating signs.

Therefore, the antiderivatives of sine and cosine are obtained simply by reversing the directional arrows: The indefinite integral of cosine is sine, the integral of sine is minus cosine, and so on.

This is good news, because it means that if you've memorized this cycle for differentiation, you don't need to learn anything new for integration. (And if you haven't memorized the differentiation cycle, then this is a great opportunity for a refresher!)

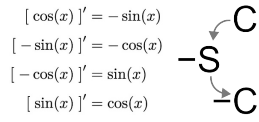


Figure 14.12: The cyclic nature of sine and cosine derivatives. Repeat of Figure 5.13.

With coefficients Integrating sine and cosine with a coefficient inside the function involves repeating the coefficient as a divisor, similar to integrating the natural exponential function. Observe:

$$\int \sin(\alpha x)\,dx \;=\; \frac{-\cos(\alpha x)}{\alpha} + C \qquad (14.58)$$

I encourage you to take a moment to prove this claim by differentiating the right-hand-side. You will get an α/α term multiplying

the sin, and of course the constant of integration differentiates to zero.

I implemented the function and its integral in Equation 14.58 using a variety of α values (Figure 14.13). Notice that because the α is inside the sine function, this coefficient changes only the frequency of the sine wave (that is, how fast it oscillates) without affecting the range (panel A). On the other hand, the integral has the α coefficient inside and outside the cosine function, and therefore affects both the frequency of the cosine and the range of the function (panel B).

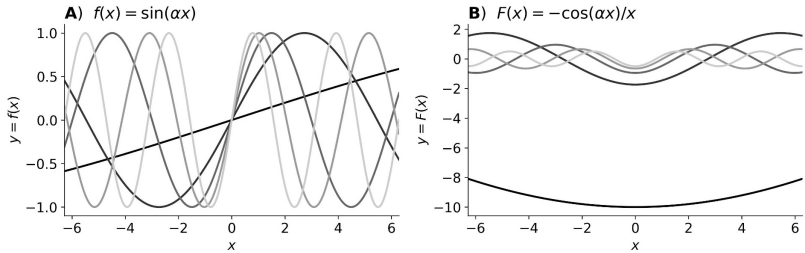


Figure 14.13: The sine function and its integral for various values of α.

I used five values of α between $\alpha = .1$ and $\alpha = 2$ to create Figure 14.13; it looks even cooler when using $N = 50$ lines (Figure 14.14). If you are so inclined, I encourage you to modify my code to make the graphs look even better. Let you artistic and creative side shine through :)

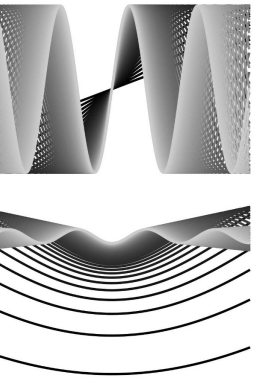


Figure 14.14: Using 50 values for α.

14.5 Product, quotient, and chain rules?

I opened this chapter with with power rule for integration. But then I moved onto other topics without introducing other rules, like the product rule, the quotient rule, and the chain rule. Those are three of the most important rules in differential calculus, so where are they in integral calculus?

I have mentioned multiple times that integration is like the inverse

of differentiation. And in fact, we used that argument to guess the antiderivatives of several functions already, including polynomials and some transcendental functions. So, you might be thinking that to integrate expressions like the following, all we need to do is apply, e.g., the product rule in reverse.

$$\int x\cos(x)\,dx$$
Inverse product rule?

$$\int \frac{3x^2}{\tan(2\pi x)}\,dx$$
Inverse quotient rule?

$$\int \cos(\sin(x))\,dx$$
Inverse chain rule?

Unfortunately, it's not that simple. It turns out that these differentiation rules cannot be inverted in a way that would work for all functions. Instead, you will need integration techniques like u-substitution, integration by parts, partial fractions, and trig substitutions.

Even more unfortunately, those techniques are complicated and take *a lot* of practice to master. In fact, many calculus course spend most of their time drilling endless numbers of these kinds of integration techniques. I will introduce these techniques in the next few chapters.

14.6 Calculating net and total area

I introduced this distinction in Section 13.2. As a brief reminder: "net area" is the signed area, which means that negative values of the function contribute negatively to the area; whereas "total area" is the unsigned area, and negative values contribute positively to the total area.

Now that you know how to integrate common functions, we can return to this discussion in more depth. There are three types of net-vs-total area problems that you are likely to encounter, and in this section I will explain these types, give tips for solving them, and go through a few examples to make it more concrete.

14.6.1 Problem type 1: given integrals

This is the easiest type of net-vs-total problem you'll encounter, and involves a bit of reasoning and basic arithmetic.

In this type of problem, you are given some definite integrals and perhaps also a graph of the function, and are asked to report the area of some other interval. Figure 14.15 shows an example. Try to solve for the question mark before reading the next paragraph.

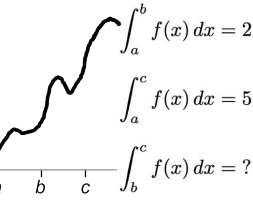


Figure 14.15: Setting up an area calculation problem.

It doesn't actually matter what $f(x)$ is — you know the areas and the bounds, so you can calculate that $\int_b^c$ is $5 - 2 = 3$.

Let's do two more examples of this problem type. Given the results in the first two rows, calculate the definite integrals in the third row:

$$\int_{-2}^{0} f(x)\,dx = 4 \qquad \int_{-2}^{0} g(x)\,dx = -1 \tag{14.59}$$

$$\int_{-2}^{2} f(x)\,dx = 7 \qquad \int_{-2}^{2} g(x)\,dx = 0 \tag{14.60}$$

$$\int_{0}^{2} f(x)\,dx = ? \qquad \int_{0}^{2} [\, f(x) - g(x) \,]\,dx = ? \tag{14.61}$$

Below are the answers; the second problem gets a bit confusing with the minus signs, but I'm confident that you successfully solved it.

$$\int_0^2 f(x)\,dx = \int_{-2}^{2} f(x)\,dx \; - \; \int_{-2}^{0} f(x)\,dx \tag{14.62}$$

$$= 7 - 4 = 3$$

$$\int_0^2 [\, f(x) - g(x) \,]\,dx = \int_0^2 f(x)\,dx - \int_0^2 g(x)\,dx \tag{14.63}$$

$$= (7 - 4) - (0 - (-1)) = 3 - 1 = 2$$

14.6.2 Problem type 2: Given graph with roots

This type of problem is more difficult than the previous because you need to integrate and calculate, but you are given the roots of the equation, which saves you some time in solving equations.

Here's an example (see Figure 14.16):

$$\text{Given } f(x) = x^2 - 2x, \text{ find } \int_0^3 f(x)\,dx \text{ and } \int_0^3 |f(x)|\,dx$$

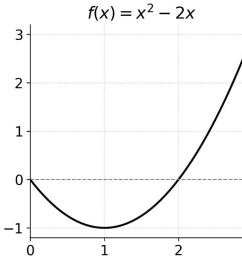


Figure 14.16: Setting up an area calculation problem.

Inspection of the graph reveals that you'll need to segment the function into two intervals. You can also calculate in your head that the two roots of this function are $x = \{0, 2\}$. Thus, calculating the total area will involve splitting the integral into two intervals, using bounds of [0,2] and [2,3].

But before doing that, we need to know the antiderivative to apply the FTC-2.

$$F(x) = \int x^2 - 2x\,dx \;=\; \frac{x^3}{3} - x^2(+C) \tag{14.64}$$

I wrote the $+C$ in parentheses as a reminder: It is formally part of the antiderivative, but we don't need it here because the problem asks us to calculate a definite integral.

From here, you apply the FTC-2 to obtain numerical results for the areas, and then you can sum the individual parts:

$$F(0) = \frac{0^3}{3} - 0^2 = 0 \tag{14.65}$$

$$F(2) = \frac{2^3}{3} - 2^2 = \frac{8}{3} - 4 = \frac{8}{3} - \frac{12}{3} = \frac{-4}{3} \tag{14.66}$$

$$F(3) = \frac{3^3}{3} - 3^2 = \frac{27}{3} - 9 = 0 \tag{14.67}$$

$$\int_0^3 f(x)\,dx = F(3) - F(0) = 0 - 0 \tag{14.68}$$

$$\int_0^3 |f(x)|\,dx = |F(2) - F(0)| + |F(3) - F(2)| = \frac{4}{3} + \frac{4}{3} = \frac{8}{3} \tag{14.69}$$

So the final answers are that the net area is zero (the positive bits cancel the negative bits) and the total area is 8/3.

Let's do one more example of this type. Based on the information below and Figure 14.17, please try to solve this on your own before checking your answer against my solution below.

$$\text{Given } f(x) = \sqrt{x} - x^2, \text{ find } \int_0^2 f(x)\,dx \text{ and } \int_0^2 |f(x)|\,dx$$

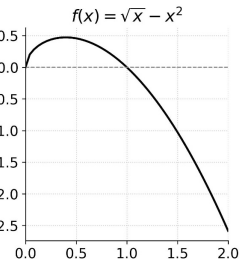


Figure 14.17: Setting up an area calculation problem.

I'll start by finding the antiderivative of the function.

$$F(x) = \int \sqrt{x} - x^2\,dx \;=\; \frac{2}{3}x^{3/2} - \frac{1}{3}x^3 \tag{14.70}$$

And then do some calculations to get the definite integral for each segment before doing some arithmetic to arrive at the final answers.

$$F(0) = 0 \tag{14.71}$$

$$F(1) = \frac{2}{3}1^{3/2} - \frac{1}{3}1^3 = \frac{1}{3} \tag{14.72}$$

$$F(2) = \frac{2}{3}2^{3/2} - \frac{1}{3}2^3 = \frac{2 \cdot 2\sqrt{2}}{3} - \frac{8}{3} = \frac{4(\sqrt{2} - 2)}{3} \tag{14.73}$$

$$\int_0^2 f(x)\,dx = F(2) - F(0) = \frac{4(\sqrt{2} - 2)}{3} - 0 \tag{14.74}$$

$$\int_0^2 |f(x)|\,dx = |F(1) - F(0)| + |F(2) - F(1)|$$

$$= \left|\frac{1}{3} - 0\right| + \left|\frac{4(\sqrt{2}-2)}{3} - \frac{1}{3}\right| \approx 1.448 \qquad (14.75)$$

You should have been able to solve everything by hand except for the final approximation.

14.6.3 Problem type 3

This type of problem is no different from the previous, except that you are not given the roots of the equation, which means you don't initially know how to split the integral into smaller intervals. Therefore, you need to start by finding the roots of the equation so you know how to split up the definite integral.

Try to solve this problem on your own; my solution is below.

$$\text{Given } f(x) = (x-1)^3, \text{ find } \int_0^3 f(x)\,dx \text{ and } \int_0^3 \left|f(x)\right| dx$$

Begin with some algebraic expansion. A hint here is that we know the bounds of integration are 0 and 3, and so you don't need to worry about any roots that are negative or greater than 3.

From inspection of the equation and from your knowledge of polynomials, it should take only a moment's reflection to see that $x = 1$ is the only root we care about for this problem[3].

It turns out that the expression $(x-1)^3$ cannot be integrated using any methods you've learned in this chapter. It can be solved using a technique called "u-substitution," which I will introduce in the next chapter. But you can solve this integration by expanding the expression and then using the additive linearity property.

$$(x-1)^3 = (x-1)(x-1)(x-1) \qquad (14.76)$$

[3]This is a third-order polynomial, so it actually has three roots. But the other two are complex-valued and we are only considering real numbers here.

$$= \left(x^2 - 2x + 1\right)(x - 1) \tag{14.77}$$

$$= x^3 - x^2 - 2x^2 + 2x + x - 1 \tag{14.78}$$

$$= x^3 - 3x^2 + 3x - 1 \tag{14.79}$$

Now that we know how to split up the integration bounds and we have a function that we know how to integrate, the solution proceeds as with the previous example problems.

The antiderivative is as follows.

$$F(x) = \int x^3 - 3x^2 + 3x - 1\,dx \;=\; \frac{x^4}{4} - x^3 + \frac{3x^2}{2} - x \tag{14.80}$$

And here is the solution to the problem using FTC-2.

$$F(0) = 0 \tag{14.81}$$

$$\begin{aligned} F(1) &= \frac{1^4}{4} - 1^3 + \frac{3(1^2)}{2} - 1 = \frac{1}{4} - 1 + \frac{3}{2} - 1 \\ &= \frac{1}{4} + \frac{6}{4} - 2 = \frac{7}{4} - \frac{8}{4} = \frac{-1}{4} \end{aligned} \tag{14.82}$$

$$F(3) = \frac{3^4}{4} - 3^3 + \frac{3(3^2)}{2} - 3 = \frac{81}{4} - 27 - \frac{27}{2} - 3 = \frac{15}{4} \tag{14.83}$$

$$\int_0^3 f(x)\,dx = F(3) - F(0) = \frac{15}{4} - 0 \tag{14.84}$$

$$\begin{aligned} \int_0^3 |f(x)|\,dx &= |F(1) - F(0)| + |F(3) - F(1)| \\ &= \left|\frac{-1}{4} - 0\right| + \left|\frac{15}{4} - \frac{-1}{4}\right| = \frac{17}{4} \end{aligned} \tag{14.85}$$

I hope you enjoyed working through these problems. If you are taking a university-level course on calculus, you will almost certainly see problems like this in homework assignments and exams.

To be honest, these kinds of problems are... unlikely to show up in an applied machine-learning job, but solving these problems does help you understand and think about functions and calculus, so they're worth putting in the effort to go through.

14.7 Integrating even and odd functions

The is an interesting property of definite integrals for symmetric (even and odd) functions.

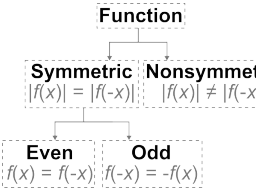


Figure 14.18: Reminder of taxonomy of function parity.

First, a quick reminder about function parity (introduced in Section 3.11): Functions can be symmetric or nonsymmetric. If they are symmetric, then they are even or odd. An even function is mirrored across the y-axis and is defined as $f(-x) = f(x)$. Two common examples of even functions are x^2 and $\cos(x)$. An odd function is mirrored across the y-axis and then across the x-axis and is defined as $f(-x) = -f(x)$. Two common examples are x^3 and $\sin(x)$.

Here are the shortcuts for integrating symmetric functions:

$$\text{if } f(x) = f(-x), \text{ then } \int_{-a}^{a} f(x)\,dx = 2\int_{0}^{a} f(x)\,dx \tag{14.86}$$

$$\text{if } f(-x) = -f(x), \text{ then } \int_{-a}^{a} f(x)\,dx = 0 \tag{14.87}$$

In words, Equation 14.86 states that if a function is even, and if the definite integral bounds are symmetric around zero, then the area between $x = -a$ and $x = 0$ is the same as the area between $x = 0$ and $x = a$, which means that you can simply integrate only one side and then double the area.

In contrast, Equation 14.87 states that if the function is odd, and if the integration bounds are symmetric around zero, then the area must be zero and you don't need to do any calculations.

Figure 14.19 illustrates this idea: panel A shows an odd function ($f(x) = x^3 + \sin(2\pi x)$) with the shaded region depicting the desired definite integral. The area between $x = -a$ and $x = 0$ is equal to the area between $x = 0$ and $x = a$ but with opposite sign. As long as the bounds are symmetric ($\pm a$), those two areas will cancel.

On the other hand, the symmetry of even functions (panel B; $f(x) = x^2 + \cos(2\pi x)$) guarantees that the areas will be equal with the same sign. You need only calculate the area of one side and then double that result.

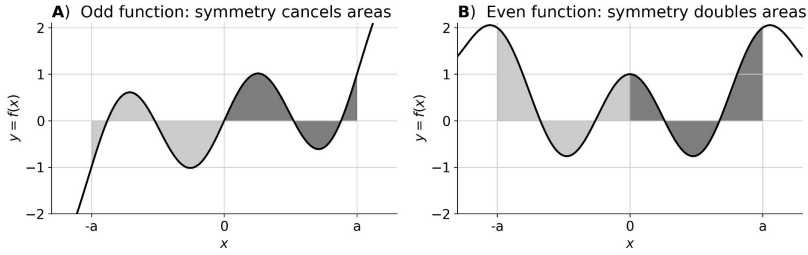


Figure 14.19: Visual illustration of the impact of function symmetry on the net area with symmetric bounds (notice the bounds are $\pm a$).

Why are these shortcuts useful? I think the usefulness of Equation 14.87 is self-evident. Equation 14.86 can be useful because many functions are easy to evaluate at $x = 0$.

14.8 Numerical integration in `scipy`

`scipy` is a Python library for scientific calculations. You are likely to use this library if you work in scientific or engineering professions, and need to perform tasks such as optimization, statistics, simulations, filtering, etc. `scipy` also comes with several methods for numerically approximating definite integrals with high accuracy and numerical stability.

In this section, I will introduce two methods in `scipy` to calculate

definite integrals.

Using a function definition The first method is to start from a symbolic expression of an equation, and the bounds, in `sympy`:

```
x = sym.symbols('x')
fx_sym = x**2 + 10*sym.sin(x)
a,b = 0,sym.pi
```

$f(x) = x^2 + 10\sin(x)$

This integral can be calculated analytically using `sympy` methods you already know:

```
int_sympy = sym.integrate(fx_sym,(x,a,b))
```

That gives the exact integral of $\frac{\pi^3}{3} + 20$. You can empirically estimate this in `scipy`:

```
fx_lam = sym.lambdify(x,fx_sym,'scipy')

import scipy.integrate as spi
int_scipy, error = spi.quad(fx_lam, a,b)
```

Note that you don't need to specify an x-axis grid, as you did when using `numpy`. `spi.quad` will determine an appropriate resolution based on the bounds and on the characteristics of the function.

How do these two libraries compare? When the `sympy` analytic result is evaluated to a floating-point result, the accuracy seems perfect:

```
Numerical integral from sympy: 30.33542556
Numerical integral from scipy: 30.33542556
```

If `scipy` is so accurate, why bother with the extra code of porting from one library to another? The answer is you shouldn't bother in situations like this. `sympy` can analytically integrate many "well-behaved" functions. But there are functions that are

sufficiently difficult to integrate that `sympy` will fail to produce a result, and in these cases, you'd use `scipy`. You'll see examples of such cases in the exercises and in subsequent chapters.

Using numerical function values A second approach to estimating the integral in `scipy` is to input a numerical array of numbers corresponding to y for $y = f(x)$, and optionally a vector of x values or Δx. For a discretized function, you can use the `trapezoid` or `simpson` method, which implements the approximation methods you learned about in the previous chapter. The code would look like this:

```
int_trap = spi.trapezoid(y,x=x,dx=dx)
```

where `y` is the array of function values. The `x=x` and `dx=dx` inputs look redundant, but the `x=` and `dx=` tells Python what input to expect, and I named the relevant variables `x` and `dx`.

Of course, Δx can be calculated from `x`, and the exact x values are not necessary for numerical integration if Δx is known. So in practice, you need to specify only `x` or `dx`; I included both above to show usage.

Other approximation algorithms There are several algorithms for approximating integrals, and `scipy` has additional functions for calculating indefinite and definite integrals. The details of the estimations differ somewhat, but the syntax is similar to that used for the approaches I showed in this section.

14.9 Exercises

14.1. This exercise is designed to give you more practice working with powers, indefinite integrals, and `sympy`. In a for-loop, create the symbolic function $f(\zeta) = \zeta^{\kappa/2}$ for $\kappa \in \{1, ..., 6\}$. Calculate the antiderivative in `sympy`. List the functions and their antiderivatives as in Figure 14.20. Notice that the exponents are symbolic numbers, e.g., $\zeta^{3/2}$ and not $\zeta^{1.5}$.

$$f(\zeta) = \sqrt{\zeta}, \quad F(\zeta) = \frac{2\zeta^{\frac{3}{2}}}{3}$$

$$f(\zeta) = \zeta, \quad F(\zeta) = \frac{\zeta^2}{2}$$

$$f(\zeta) = \zeta^{\frac{3}{2}}, \quad F(\zeta) = \frac{2\zeta^{\frac{5}{2}}}{5}$$

$$f(\zeta) = \zeta^2, \quad F(\zeta) = \frac{\zeta^3}{3}$$

$$f(\zeta) = \zeta^{\frac{5}{2}}, \quad F(\zeta) = \frac{2\zeta^{\frac{7}{2}}}{7}$$

$$f(\zeta) = \zeta^3, \quad F(\zeta) = \frac{\zeta^4}{4}$$

Figure 14.20: `sympy` output in Exercise 1.

Visualize all functions and integrals as in Figure 14.21. The dashed lines are unlabeled, but correspond to the antiderivatives of the functions with matching grayscale intensity.

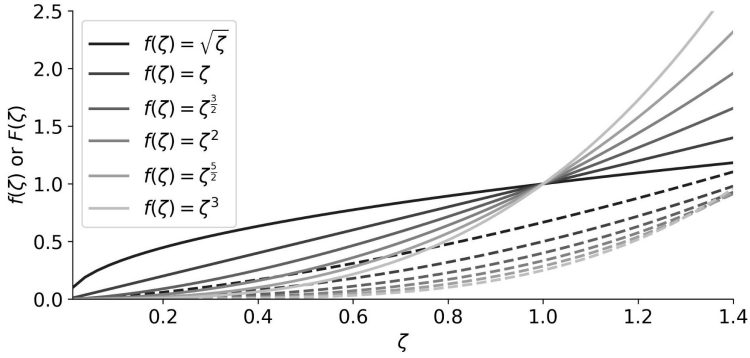


Figure 14.21: Results for Exercise 1.

You can also try plotting the derivatives of the functions. In my humble opinion, that's a lot less aesthetically pleasing, but it's a matter of personal taste.

14.2. The goal of this exercise is to write code to produce Figure 14.22. Panels A and B are the same as Figure 14.3; panel C shows the indefinite integrals of the functions shown in panel A, and panel D shows the indefinite integrals of the derivatives of those functions shown in panel B.

Best practice here is to copy/ paste my code from earlier in the chapter into a new code cell, rather than modify the figure code.

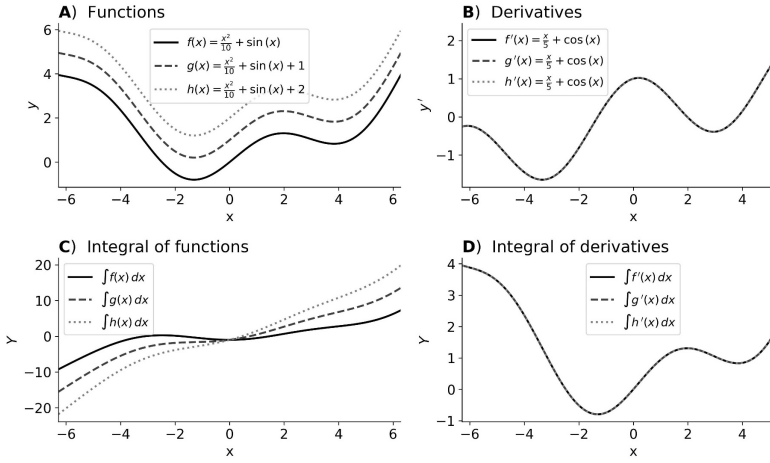


Figure 14.22: Results for Exercise 2.

The key learning theme here is to appreciate that the integral of a constant is a first order polynomial (that is, $\int C\,dx = Cx + D$, where D is another constant). Constants in the integrand will change the resulting integral. Be mindful to add constants *after* the integration is done.

14.3. In this exercise, you will see an example of how `scipy` can successfully approximate definite integrals that `sympy` cannot analytically calculate.

Implement the following function in `sympy`, and visualize it in `matplotlib` as in Figure 14.23.

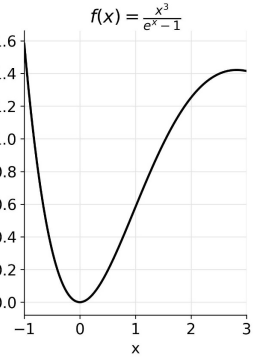


Figure 14.23: Visualizing the function in Exercise 3.

$$f(x) = \frac{x^3}{e^x - 1} \tag{14.88}$$

Try to calculate the following definite integral in `sympy`:

$$\int_{-1}^{3} \frac{x^3}{e^x - 1}\,dx \tag{14.89}$$

If you use `sym.integrate` as I showed in the previous chapter, the output will be a repeat of Equation 14.89. In other words, `sympy` cannot calculate this integral exactly, so it just prints out the question instead of giving an answer. You might suspect this is due to the removable discontinuity (note what happens at $x = 0$). But you'll get the same output if you try it using bounds of [1,3].

The problem here is that `sympy` is unable to calculate an exact integral. Now estimate the integral using `scipy` methods you learned in this chapter. You should get a value of 3.02702364.

14.4. In this exercise, you will write an algorithm to solve the initial value problem using `sympy`. The result of this exercise will be `sympy`-printed output like in Figure 14.24 (this is the same equation I used to introduce the initial value problem on page 476).

PROBLEM:
Given $f'(x) = 2x + 3$, and $f(1)$

SOLUTION:
$f(x) = x^2 + 3x - 2$

Figure 14.24: `sympy` output for Exercise 4.

Here's how to do it: Create the derivative as a symbolic expression, then integrate it — including a symbolic variable C — to get an expression for the function with the indeterminate constant of integration. Next, use the constraint to solve for C using `sympy` methods to solve algebraic equations. Finally, combine Python and latex code to get output that looks like Figure 14.24.

Now try your code using the equation in Figure 14.25 (also printed below).

$$\text{Given } f'(x) = x^{5/2} + \cos(x), \text{ and } f(1) = \pi$$

PROBLEM:
Given $f'(x) = x^{\frac{5}{2}} + \cos(x)$, and $f(1)$

SOLUTION:
$f(x) = \frac{2x^{\frac{7}{2}}}{7} + \sin(x) - \sin(1) - \frac{2}{7} +$

Figure 14.25: Additional `sympy` output for Exercise 4.

If you get really long numbers instead of fractions and sin(1), then try using `sym.Rational` to define the exponent.

Finally, try modifying the previous expression to $x^{5/2}\cos(x)$ (that is, multiplying the terms instead of adding them). The result is a valid expression, but quite complicated...

14.5. In this and the next exercise, you will explore the linearity of integration in `sympy`. This exercise will focus on the scalar multiplication property of linearity.

Use `sympy` to show that $\int \pi x^2\,dx = \pi \int x^2\,dx$, and print your results as in Figure 14.26A. Because this part of the exercise deals only with antiderivatives, you will need to create a symbolic constant of integration to ensure an accurate result.

Notice that `sympy` multiplied through by π to get πC, although you know from earlier in this chapter that $C = \pi C$.

A) First part of Exercise 5

$$\int \pi x^2 dx = C + \frac{\pi x^3}{3}$$

$$\pi \int x^2 dx = \pi C + \frac{\pi x^3}{3}$$

B) Second part

$$\int_0^{3\pi} \pi x^2\,dx = 9\pi^4$$

$$\pi \int_0^{3\pi} x^2\,dx = 9\pi^4$$

Figure 14.26: Results for Exercise 5.

Next, extend this demonstration by showing that the definite integrals are also equal, regardless of whether the scalar (in this case, π) is inside or outside the integrand. Use latex code to print your results as in Figure 14.26B.[4]

14.6. Now to demonstrate the additive property of integration. There are three parts to this exercise (indefinite integral, definite integral, visualization). Create the following three functions in `sympy`:

$$f(x) = x^2 \tag{14.90}$$

$$g(x) = 10\sin(x) \tag{14.91}$$

[4] Friendly reminder that demonstrations in `sympy` are great for building intuition but do not constitute rigorous proofs.

$$h(x) = f(x) + g(x) \tag{14.92}$$

For the first part of this exercise, calculate and print the indefinite integrals in `sympy` as in Figure 14.27. The third and fourth rows look similar, but they are calculated differently: The third row is $\int f(x)\,dx + \int g(x)\,dx$ while the fourth row is $\int h(x)\,dx$. `sympy` simplifies the output, although you can see the distinction in the $2C$ vs. C.

$$f: \quad \int x^2\,dx = C + \frac{x^3}{3}$$
$$g: \quad \int 10\sin(x)\,dx = C - 10\cos(x)$$
$$f+g: \quad \int x^2\,dx + \int 10\sin(x)\,dx = 2C + \frac{x^3}{3} - 10\cos(x)$$
$$h: \quad \int \left(x^2 + 10\sin(x)\right)\,dx = C + \frac{x^3}{3} - 10\cos(x)$$

Figure 14.27: Results for Exercise 6.

The second part of this exercise involves confirming that the definite integral of $h(x)$ equals the sum of the definite integrals of $f(x)+g(x)$. Use integration bounds of 0 to 3π, and have `sympy` print results like Figure 14.28.

$$\int_0^{3\pi} x^2\,dx = 9\pi^3$$
$$\int_0^{3\pi} 10\sin(x)\,dx = 20$$
$$\int_0^{3\pi} x^2 + 10\sin(x)\,dx = 20 +$$

Figure 14.28: `sympy` output for the second part of Exercise 6.

The third part of this exercise is to write code to produce Figure 14.8 (page 484).

14.7. In Section 14.8 on estimating definite integrals with `scipy`, I wrote that the function `spi.trapezoid` is used to calculate a numerical definite integral, given a numerical array. That is, you input a `numpy` array and `spi.trapezoid` returns a number corresponding to the definite integral over that array.

The purpose of this exercise is for you to figure out how you can use the *definite integral* approximation to produce an *indefinite integral* using `spi.trapezoid`.

Start with the following function, and obtain its antideriva-

tive in `sympy`; the analytic antiderivative will allow you to evaluate the accuracy of the numerical approximation.

$$f(x) = \frac{2x^2 - 3x}{\sqrt{x^2 - x + 1}} \tag{14.93}$$

Plot the function and its `sympy`-calculated antiderivative as in Figure 14.29. I evaluated the functions using 901 x-axis points between -2 and 5.

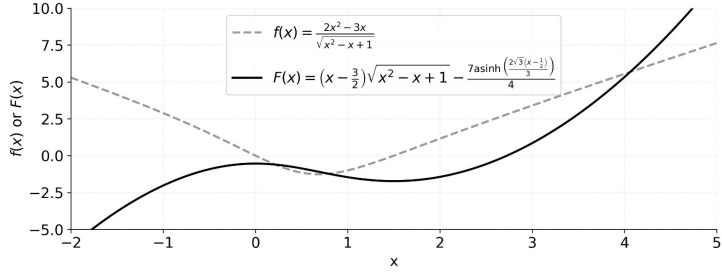


Figure 14.29: Results for Exercise 7.

Now for the numerical approximation. Given what you know about `spi.trapezoid` and what you know about the indefinite integral (hint: refer back to Section 12.3), how can you use definite integrals to approximate the indefinite integral?

When you figure it out, draw the figure again with the `scipy`-estimated indefinite integral plotted on top of the `sympy`-calculated analytic indefinite integral (Figure 14.30).

Note: The `spi.trapezoid` function uses numerical values for integration; it doesn't have access to an equation that can be evaluated for any value of x. This makes the constant of integration even more ambiguous, because the numerical values of the antiderivative depend on the x-axis values that were evaluated from the function (or measured in an experiment). This means that the y-axis shift of the antiderivative depends on whether you start your simulation at $x = -2$ or $x = 0$ or $x = -10$. In Figure 14.30, I

eyeball-estimated a constant of integration that made the analytic and empirical antiderivatives overlap.

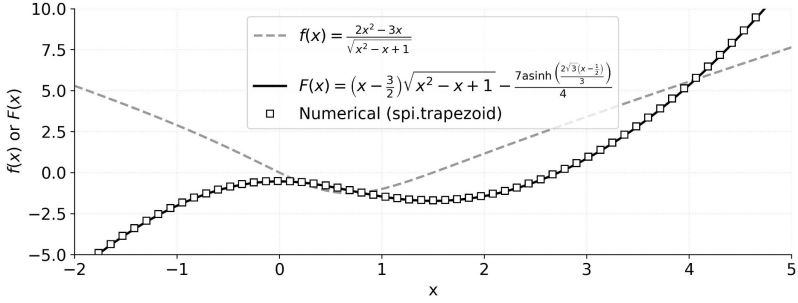


Figure 14.30: Final results for Exercise 7. The numerical approximation was downsampled to show every 15 points for visibility.

Final task for this exercise: Figure out how to use the function `spi.cumulative_trapezoid` and make sure you get the same results. In applications, `spi.cumulative_trapezoid` is easier to use because it requires less coding, but solving this exercise using `spi.trapezoid` is an excellent way to understand the relationship between definite and indefinite integrals.

14.8. The function in the previous exercise could be analytically integrated by `sympy`, which means we didn't technically need to approximate the integral. The goal of this exercise is to repeat the previous exercise but for a different function. Implement this function:

$$f(x) = \frac{e^{-x^2}}{e^{x+1}} \tag{14.94}$$

See what happens when you integrate this function in `sympy`. It won't give errors but it also won't give a result that you might find satisfactory.

Therefore, use `sympy`'s substitution method to numerically evaluate the function, and then input that array into

`spi.cumulative_trapezoid` to approximate the indefinite integral. Specify the Δx parameter as `dx=xx[1]-xx[0]` (`xx` is the vector of x-axis grid values). Show both as in Figure 14.31.

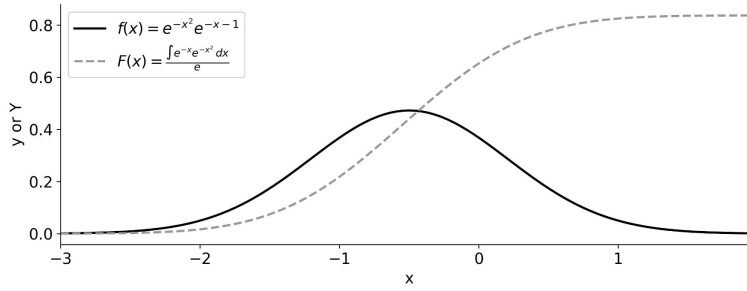


Figure 14.31: Results for Exercise 8.

Final task for this exercise: What would happen if you use the input `dx=xx[2]-xx[0]` instead? Make a prediction about the impact on the graph, then change the code to test your prediction.

14.9. This exercise will help you understand the concepts and challenges in selecting constants of integration. You will use this function:

$$f(x) = e^{p/q} \tag{14.95}$$

$$p(x) = 2x^3 - x^2 + 4x \tag{14.96}$$

$$q(x) = 7x^3 + 4x^2 - 1 \tag{14.97}$$

Implement $f(x)$ in `numpy`, and approximate its antiderivative using the `cumulative_simpson` function in the `scipy` integration module. Visualize the rational function inside the exponential as in Figure 14.32A, and the function and its integral as in Figure 14.32B. I evaluated the function using 901 points between $x = -3$ and $x = 2$.

The rational function has an infinite discontinuity at $x \approx .386$, which means the function has a one-sided infinite discontinuity there as well. This creates *NaN* values in the antiderivative after that discontinuity.

One solution to this issue is to calculate the antiderivative as a two-part piecewise function: One piece for before the discontinuity, and one piece for after the discontinuity (I started the second piece at .1 units after the discontinuity for numerical stability).

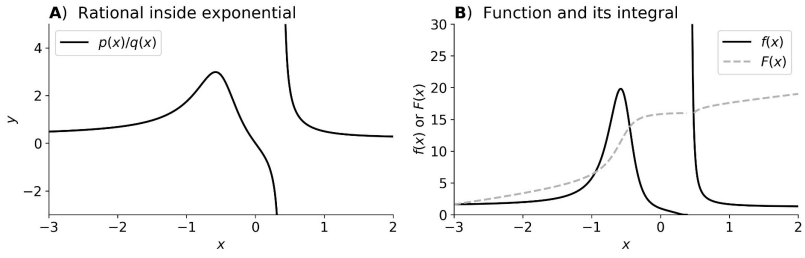


Figure 14.32: Results for Exercise 9.

Because the antiderivative is created as a two-piece piecewise function, there are two constants of integration (one for each piece). The problem description did not include constraints (initial values) that would allow us to resolve the indeterminate constants. Therefore, we can choose how to visualize $F(x)$. I made the arbitrary choice to shift the two pieces of the antiderivative such that (**1**) the start of the second piece matches the end of the first piece, and (**2**) $F(-3) = f(-3)$. In other words, the antiderivative looks smooth across the discontinuity, and the function and its integral meet at the first x-axis value that I simulated.

Thus, I set both constants to maximize visual interpretability of the antiderivative vis-à-vis the original function. What happens when you don't constrain the constants? Does the visual appearance of the antiderivative change if the x values of the simulation change (e.g., start at $x = -10$ instead of $x = -3$)?

14.10. One interesting feature of numerical approximations to integrals (and also derivatives, although that's not the focus here) is that the spacing between successive x-axis grid points need not be constant. That is, dx can vary from point to point. The goal of this exercise is to explore this

phenomenon.

To start, implement the function $f(x) = x^2 + \cos(x) + x$ as a `sympy` expression and lambdify it. Then, create two sets of x-axis grid points: One comprising 555 regularly spaced points between -2 and 4, and one comprising 555 *randomly sampled* points between these same bounds[5]. Evaluate the function using those two x-axis grid vectors and plot the intermediate results as in Figure 14.33.

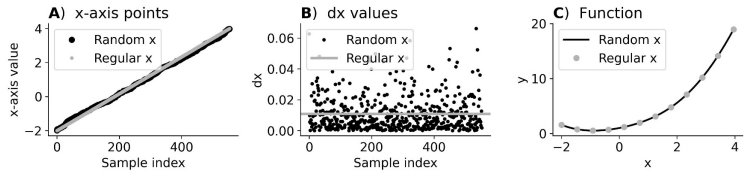


Figure 14.33: Visualization for Exercise 10.

Panel A shows the two sets of x-axis grid points; notice that the gray dots lie in a perfect line, whereas the black dots monotonically increase but at an irregular pace. This happens because Δx is not constant from point to point, which is illustrated in panel B. On the other hand, the function itself (panel C) appears smooth and regular — although this is due to the zoom factor and the width of the line.

The key question here is whether irregular spacing is detrimental to the estimate of the definite integral. To assess this, calculate four definite integrals: (**1**) using `sympy` on the symbolic expression, which we will consider the ground truth; (**2**) using `spi.simpson` on the regularly spaced x grid with the second function input specifying the x grid (that is, input `x=`); (**3**) using `spi.simpson` on the regularly spaced x grid with the second function input specifying Δx (that is, input `dx=`); (**4**) using `spi.simpson` on the irregularly spaced x grid.

My results are below. Yours should match for the `sympy`

[5]There are several ways to randomly sample points between two values; you can consider using the function `np.random.uniform`.

and regularly spaced results, but of course will differ somewhat for the random x result.

```
sympy      : 30.15249493
x-reg (x)  : 30.15249493
x-reg (dx): 30.15249493
x-rand     : 29.69840983
```

The conclusion here is that random, irregular sampling of a function risks reduced accuracy of the numerical approximation of the integral. On the other hand, as the number of sampled points increases, the accuracy will generally increase. I don't show that in the code, but you can explore this by using, e.g., 100 or 50000 sample points.

But why would you randomly sample points? When generating data from mathematical functions, you wouldn't. But real data measured from nature may contain irregularly sampled points, due to sensor malfunction, signal drop-out, interference, or other issues. Therefore, this exercise reveals a potential issue for applied calculus that does not impact pure calculus.

Chapter 15

Improper integrals

15.1 What are improper integrals?

Improper integrals are certain types of definite integrals. There are two ways that an integral can be "improper."

One or both integration bounds is $\pm\infty$

Any definite integral with bounds that look like the following is considered improper:

a and b are finite numbers.

$$\int_a^{\infty} \quad \text{or} \quad \int_{-\infty}^{b} \quad \text{or} \quad \int_{-\infty}^{\infty}$$

Discontinuities within the bounds

Even if both integration bounds are finite numbers, a definite integral can be improper if the function has a discontinuity between a and b. Here is an example:

$$\int_0^2 \frac{1}{x^2 - 1}\, dx$$

This function has finite bounds, but there is a discontinuity at $x = 1$. The impropriety comes from the discontinuity and the interval, and so the definite integral of the same function would be proper if the bounds excluded 1, e.g., $\int_2^3$.

If neither of the above is true, then the integral is called a proper integral.

Who cares? Why does it matter if an integral is improper? On the one hand, there's nothing "wrong" with improper integrals, and many improper integrals can be successfully integrated and produce a finite result.

The issue with improper integrals is that you cannot simply plug ∞ into the antiderivative and expect a reasonable result. And some discontinuities can produce senseless results.

So how do you solve improper integral problems? You start by understanding the source of the impropriety, characterizing the

function (visualization often helps), and then determining the best integration strategy.

There are two common strategies that are likely to be successful: Using limits instead of infinities, and breaking up the integral into subintervals where the function is continuous.

The rest of this chapter is basically just unpacking that previous paragraph, with explanations, examples, visualizations, and code. Along the way, you will discover some interesting and surprising properties of functions, and you will learn about the concepts of convergence and divergence of power functions.

15.2 One infinite integration bound

Example 1 I'm going to start by showing an example problem that illustrates the solution to a one-infinite-bound definite integral. I will explain each step of the example below the equations, but I encourage you to inspect the equations below and see if you can understand the solution and why it works.

$$\int_1^\infty \frac{1}{x^2}\,dx = \lim_{t\to\infty}\int_1^t \frac{1}{x^2}\,dx = \lim_{t\to\infty} \frac{-1}{x}\bigg|_1^t \tag{15.1}$$

$$= \lim_{t\to\infty}\left[\frac{-1}{t} - \left(\frac{-1}{1}\right)\right] \tag{15.2}$$

$$= \lim_{t\to\infty}\left(\frac{-1}{t}\right) + \lim_{t\to\infty}(1) \tag{15.3}$$

$$= 0 + 1 = 1 \tag{15.4}$$

The idea is to replace the ∞ with a new variable (here I'm using variable t) and calculate the limit as $t \to \infty$. It may seem like we're still just plugging in ∞ with more steps, but calculus is designed to work with limits, not with infinities.

And that's the key to solving improper integrals with one infinite integration bound. The math might get trickier, but the concept is the same. Solving definite integrals with two infinite bounds is a similar procedure but requires one additional step; I'll discuss this later in the chapter.

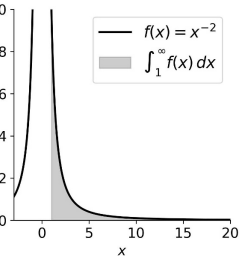


Figure 15.1: Visualizing Example 1.

What do you think of the result, that the area under the curve of x^{-2} is a finite number — and a "small" integer like 1? I find this result surprising. After all, we're taking the limit out to INFINITY, which is a really really endlessly big number. It seems to me like if we have an infinite number of values of x — meaning an infinite expanse of an axis under the curve — that the area of any non-zero function should also approach ∞. The thing to appreciate is that as x gets really really huge, x^{-2} gets really really tiny. So, although x keeps growing, the contribution of $f(x)$ to the area under the curve becomes infinitesimal (Figure 15.1). I must admit that although I understand the mathematical principle behind that previous sentence, it still "feels" like the area should keep growing.

Example 2 Anyway, let's do another example. I'd like you to work through this solution on your own before checking your work against my solution and explanation below.

$$\int_1^\infty \frac{\pi}{x^3}\,dx \;= ? \tag{15.5}$$

You solve this problem using exactly the same strategy as the previous example. The solution is below, and I'll have another brief discussion below that.

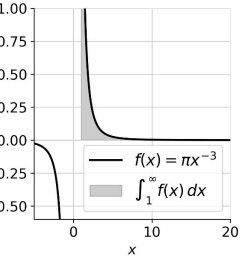


Figure 15.2: Visualizing Example 2.

$$\int_1^\infty \frac{\pi}{x^3}\,dx \;=\; \lim_{t\to\infty}\int_1^t \frac{\pi}{x^3}\,dx \;=\; \lim_{t\to\infty} \frac{-\pi}{2x^2}\Bigg|_1^t \tag{15.6}$$

$$= \lim_{t\to\infty} \frac{-\pi}{2t^2} - \left(\frac{-\pi}{2}\right) \tag{15.7}$$

$$= 0 + \frac{\pi}{2} \tag{15.8}$$

There are no special tricks to getting this integral to work; just be mindful of the minus signs when evaluating the integral, and that terms without a t variable can be ignored when calculating limits.

And again, the result is (in my humble opinion) quite surprising: The area under an infinite number x-axis values that grows to infinity is the beautiful number $\pi/2$.

Example 3 You know the drill: Work through this problem on your own, then check my solution and discussion below.

$$\int_{-\infty}^{-1} x^4\,dx \;= ? \tag{15.9}$$

This problem isn't so difficult to solve in the sense that the antiderivative of x^4 is easy to calculate based on the power rule. But the result seems different from those of the previous two examples — perhaps more what you might have expected at the outset of this chapter? Here's my solution:

$$\int_{-\infty}^{-1} x^4\,dx \;=\; \lim_{t\to-\infty}\int_{t}^{-1} x^4\,dx \;=\; \lim_{t\to-\infty}\left.\frac{x^5}{5}\right|_{t}^{-1} \tag{15.10}$$

$$= \lim_{t\to-\infty}\left(\frac{-1}{5} - \frac{t^5}{5}\right) \tag{15.11}$$

$$= \infty \tag{15.12}$$

Huh, so now we actually do get an infinite area coming from an infinite integration bound[1]. What is different about this example compared to the previous two? Why do some improper integrals produce a finite number while others produce infinity?

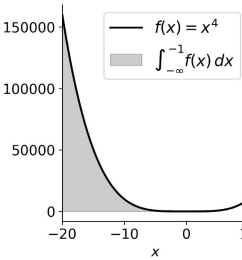


Figure 15.3: Visualizing Example 3.

[1] Is the answer to this integral ∞ or ∞^5? In many situations in mathematics, there is no meaningful distinction between infinities. It's not a finite number, and so we can consider $\infty = \infty^5$. There are situations in philosophy of mathematics and set theory in which distinct infinities are considered and ordered (thus, $\infty < \infty^5$). However, in the context of calculus and limits, you can simply interpret the symbol ∞ to indicate unbounded growth.

It turns out that there is a rule for knowing whether an expression of the form x^p will have a finite or an infinite improper integral. Perhaps you can already guess that rule based on the three examples above. That rule is related to *convergence*, and is the main topic of the next section.

15.3 Convergence and divergence

I will begin to by defining two terms, and then state the rule for when an improper integral with an x^p term will converge or diverge. Try to figure out the rule based on the examples above and the definitions below, before seeing the rule written out.

Convergent

A definite integral is convergent if the limit exists and the integral is a finite number. In other words, as x grows unbounded, the definite integral converges to a particular finite number.

Of course, the numerical value to which the integral converges depends on the function and the bounds. Examples 1 and 2 in the previous section were convergent integrals; here's another example:

Remember that the antiderivative of $1/x$ is $\ln|x|$.

$$\int_{-\infty}^{-2} \frac{1}{x^2+x}\,dx = \lim_{t\to-\infty} \ln|x| - \ln|x+1| \Big|_t^{-2} \tag{15.13}$$

$$= \lim_{t\to-\infty} \Big[(\ln|-2| - \ln|-2+1|) - (\ln|t| - \ln|t+ \tag{15.14}$$

$$= \ln(2) - \ln(1) - (\infty - \infty) \tag{15.15}$$

$$= \ln(2) \tag{15.16}$$

Integrating $(x^2+x)^{-1}$ relies on a technique called "partial fractions," which I'll explain in the next chapter.

The properties of logarithms include that subtraction of logs equals division inside a log. Thus,

$$\ln|-2| - \ln|-2+1| \;=\; \ln\left|\frac{-2}{-2+1}\right| \;=\; \ln(2) \tag{15.17}$$

$$\lim_{t\to-\infty}\left(\ln|t|-\ln|t+1|\right) \;=\; \lim_{t\to-\infty}\ln\left|\frac{t}{t+1}\right| = \ln\left|\frac{-\infty}{-\infty}\right| = \ln(1) \tag{15.18}$$

Divergent

A definite integral is divergent if the limit does not exist, or if the definite integral evaluates to $\pm\infty$. Thus, as x grows towards ∞, the definite integral continues to grow, thus diverging from any particular finite number. Example 3 above showed a divergent integral; here's another example:

$$\int_1^\infty \frac{1}{\sqrt{x}}\,dx \;=\; \lim_{t\to\infty} 2\sqrt{x}\,\Big|_1^t \tag{15.19}$$

$$= \lim_{t\to\infty}\left(2\sqrt{t}-2\right) \tag{15.20}$$

$$= 2\sqrt{\infty}-2 \;=\; \infty \tag{15.21}$$

A definite integral of $\pm\infty$ is technically called *unbounded* divergence. There are other types of divergences, for example oscillatory divergence, which you'll see later in this chapter in the section on trig functions. More generally, any definite integral that doesn't converge is termed divergent.

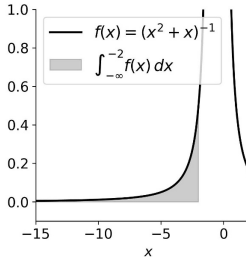


Figure 15.4: Visualizing the convergence example.

15.3.1 The rule

The rule for determining whether a power function converges or diverges is based on the value of the exponent. In particular:

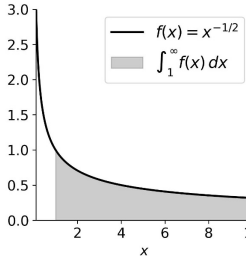


Figure 15.5: Visualizing the divergence example.

$$\int_a^\infty x^{-p}\,dx \begin{cases}\text{convergent if } p>1\\ \text{divergent if } p\le 1\end{cases} \tag{15.22}$$

Some intuition for this rule: When $p > 1$, x^{-p} decays *really fast* as $x \to \infty$, and therefore makes negligible contributions to the area for large x (for example, think about $1/10^{100}$). On the other hand, when $p < 1$, x^{-p} either grows or decays very slowly, and thus its contribution to the area gets larger as $x \to \infty$ (think about x^2).

Reminder: $x^{-p} = 1/x^p$

15.3.2 Important caveat to this rule

This rule about convergence of powers applies only to functions that are continuous in the integration bounds. The rule does not apply to functions with discontinuities that are within the bounds. For example:

$$\int_{-\infty}^{-1} x^{-2}\,dx = 1 \tag{15.23}$$

$$\int_{-\infty}^{0} x^{-2}\,dx = \infty? \tag{15.24}$$

The second integral is not convergent because it includes $x = 0$ for the function $1/x^2$, which is a discontinuity. As long as we exclude that discontinuity, we'll get a finite area — and the closer to that explosive discontinuity, the larger that finite area.

More on discontinuities later in this chapter!

$$\int_{-\infty}^{-1/10000} x^{-2}\,dx = 10000 \tag{15.25}$$

15.3.3 Check the highest-power term

Recall the discussion in Chapter 4 about limits and polynomials: When considering very large values of x, only the highest-term power matters. That is:

$$\lim_{x\to\infty}\left(x^2 + 1000000x + 9345739\sqrt{x}\right) = \lim_{x\to\infty} x^2 \tag{15.26}$$

Of course, for values of x close to zero, the smaller terms impact the behavior of the function; but as x gets galactically gigantic, only the largest-power term needs to be considered.

Same story for convergence and divergence: If there is an infinite bound on the definite integral, then the divergence is driven by the term with the highest power. If the term with the highest power is divergent, then the entire function is divergent.

On the other hand, if the highest-power term indicates convergence, then the smaller-power terms will influence the definite area.

The conclusion is that you can use the rule to determine whether the integral is convergent or divergent. If it's divergent, then the definite integral is $\pm\infty$ or does not exist; on the other hand, if it's convergent then you will need to work through the integration problem to find the finite value to which the integral converges.

15.4 Two infinite bounds

Solving a definite integral with two infinite bounds is the same as solving a definite integral with one infinite bound, except that you break it up into two pieces. That is,

$$\int_{-\infty}^{\infty} f(x)\,dx \;=\; \int_{-\infty}^{a} f(x)\,dx \;+ \int_{a}^{\infty} f(x)\,dx \qquad (15.27)$$

Once you've split the integral, you solve each piece the same way you solved the single-infinite-bound problems earlier in this chapter: replace ∞ with the limit as some other variable tends towards ∞. If either piece has an integral of $\pm\infty$, then the whole expression is divergent. If both pieces have finite integrals, then you sum those two pieces to get the final answer.

In Equation 15.27 I used a placeholder a as the splitting value. You can split based on any finite number, and you should pick the number based on what's easy to calculate given the definition of the function. For example, if the function includes a term like $(x-3)$, then $a=3$ might be a convenient value to split the interval. If you don't see any obvious choice based on the function definition, then split at $a=0$.

Example 1 I will give you the function and its antiderivative, and I would like you to solve the definite integral on your own before reading my solution.

$$f(x) = \frac{e^{x-1}}{1+e^{x-1}} \tag{15.28}$$

$$F(x) = \ln(e^{x-1}+1) + C \tag{15.29}$$

$$\int_{-\infty}^{\infty} f(x)\,dx = ? \tag{15.30}$$

I hope you took the time to work through this problem. What did you choose for the splitting value? Based on the equation, $x = 1$ is a convenient choice, because $e^{1-1} = 1$.

Here is my solution. First I calculated the integral up to 1.

$$\int_{-\infty}^{1} \frac{e^{x-1}}{1+e^{x-1}}\,dx = \lim_{t\to-\infty} \ln(e^{x-1}+1)\,\Big|_t^1 \tag{15.31}$$

$$= \lim_{t\to-\infty} \left[\ln(2) - \ln(e^{t-1}+1)\right] \tag{15.32}$$

$$= \ln(2) - \ln(0+1) = \ln(2) \tag{15.33}$$

So far we have discovered that the area under the curve left of $x = 1$ is convergent, with a definite integral of $\ln(2)$. Let's see what happens on the other side:

$$\int_{1}^{\infty} \frac{e^{x-1}}{1+e^{x-1}}\,dx = \lim_{t\to\infty} \ln(e^{x-1}+1)\,\Big|_1^t \tag{15.34}$$

$$= \lim_{t\to\infty} \left[\ln(e^{\infty}+1) - \ln(e^{1}+1)\right] \tag{15.35}$$

$$= \infty \tag{15.36}$$

This result is interesting: The area left of $x = 1$ is finite while the area right of $x = 1$ is infinite. This means that the solution to the original double-infinite-bound problem is ∞.

This result makes more sense when you see the graph of the function (Figure 15.6): It gets squashed down to near-zero for $x < 0$ while being asymptotic to $y = 1$ as x grows.

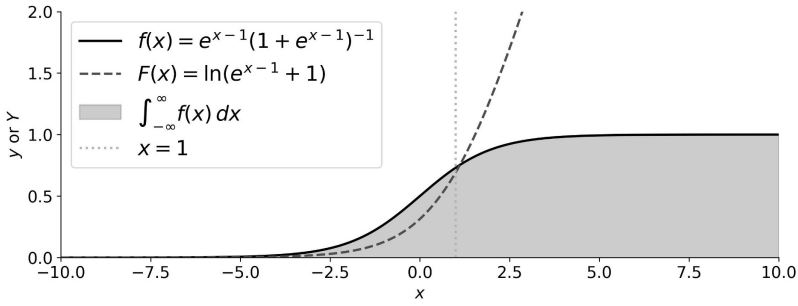


Figure 15.6: Equation 15.28 visualized.

You should also be able to have an intuition for this result just by looking at the function definition: As $x \to -\infty$, the function starts to look like 0/1; and as $x \to \infty$, the function starts to look like 1/1. Visualizing the antiderivative (dashed line in Figure 15.6) also shows that the area will continue to grow as x increases.

Example 2 As with the previous example, I'll start you off with the function and its antiderivative, and then you can work through the rest of the problem on your own. *Note:* You should work through the calculus by hand, but feel free to use a calculator or Python to evaluate the arctangent function.

$$f(x) = \frac{4x}{x^4 + 2} \tag{15.37}$$

$$F(x) = \sqrt{2}\tan^{-1}\left(\frac{\sqrt{2}x^2}{2}\right) + C \tag{15.38}$$

$$\int_{-\infty}^{\infty} f(x)\,dx = ? \tag{15.39}$$

Now for my solution. $x = 0$ seems like a good place to split the integral. First, the left side of the interval:

$$\int_{-\infty}^{0} \frac{4x}{x^4+2}\,dx = \lim_{t\to-\infty} \sqrt{2}\tan^{-1}\left(\frac{\sqrt{2}t^2}{2}\right)\Bigg|_t^0 \tag{15.40}$$

$$= \sqrt{2}\tan^{-1}(0) - \sqrt{2}\tan^{-1}\left((-\infty)^2\right) \tag{15.41}$$

$$= 0 - \frac{\sqrt{2}\pi}{2} \tag{15.42}$$

What is $(-\infty)^2$? Is this a bigger negative infinity, or does it become positive infinity? ∞ is not a finite number, and so the square operator doesn't work for ∞ the way it works for finite numbers.

There's no single correct answer for what to do here; you have to infer from context. It turns out that in this case we need to have $+\infty$ to make the area negative, because the function dips below the x-axis for $x < 0$. I'll show that in a graph in a bit, but first let's see what happens when evaluating the right side of the integral.

$$\int_{0}^{\infty} \frac{4x}{x^4+2}\,dx = \lim_{t\to\infty} \sqrt{2}\tan^{-1}\left(\frac{\sqrt{2}t^2}{2}\right)\Bigg|_0^t \tag{15.43}$$

$$= \sqrt{2}\tan^{-1}(\infty) - \sqrt{2}\tan^{-1}\left(0^2\right) \tag{15.44}$$

$$= \frac{\sqrt{2}\pi}{2} - 0 \tag{15.45}$$

This is an interesting result: Each single-infinite-bound integral converges to the same finite number (with opposite sign), and therefore the total area from $-\infty$ to $+\infty$ is exactly zero. Again, inspecting the graph of the function (Figure 15.7) allows us to understand why we get that result.

In fact, this function is odd-symmetric ($f(-x) = -f(x)$), which means we can take advantage of the shortcut you learned about in Section 14.7. The antiderivative (dashed line in Figure 15.7)

looks a bit like an upside-down Gaussian, and appears to be an even function, which would mean that $F(a) - F(-a) = 0$. In fact, the arctangent function is odd-symmetric, but becomes an even function when the x term is squared.

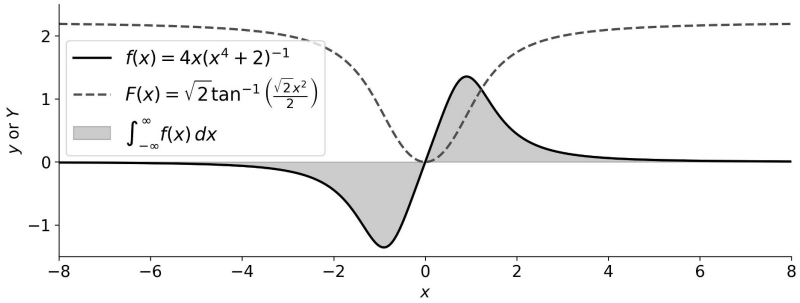


Figure 15.7: Equation 15.37 visualized.

15.5 Improper trig integrals

The interpretation and mechanics of improper integrals with trig functions is the same as with any other functions, but the cyclicity of sine and cosine have some interesting properties.

15.5.1 Definite integrals of sine and cosine

Let's start with a simple example:

$$\int_0^\infty \sin(\theta)\,d\theta \;= ? \tag{15.46}$$

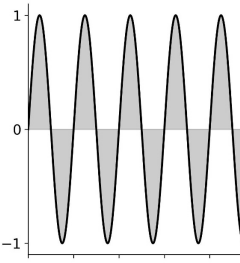


Figure 15.8: $f(\theta) = \sin(\theta)$ (not plotted to $\theta = \infty$). Each x-axis tick mark is $2\pi\theta$.

Before investigating this rigorously, what is your intuition? Do you think the net area of the sine function for $\theta > 0$ (Figure 15.8) converges to a finite value, or explodes up to $A = \infty$? Or perhaps something else happens?

Let's first work through the math, and then I'll discuss the answer.

$$\int_0^\infty \sin(\theta)\,d\theta \;=\; \lim_{t\to\infty} -\cos(\theta)\,\Big|_0^\infty \tag{15.47}$$

$$= \lim_{t\to\infty} \Big[-\cos(t) \,+\, \cos(0)\Big] \tag{15.48}$$

$$= \lim_{t\to\infty} \Big(-\cos(t)\Big) \,+\, 1 \tag{15.49}$$

So what's the answer? Well, the limit of $\cos(t)$ as $t \to \infty$ does not exist — it just keeps oscillating until the end of the universe. What we can say is that the definite integral as θ increases is bounded by (0,2), meaning it is never negative or greater than 2. It does not converge to a finite value, nor does it diverge to ∞. This is an example of an oscillatory divergence.

How about the integral of sine for negative θ? You are now smarter than you were before reading the above discussion, so please guess the result of this equation before working through the answer rigorously:

$$\int_{-\infty}^0 \sin(\theta)\,d\theta \;= ? \tag{15.50}$$

Same function, but now we're considering the area left of zero. Please look away from the text, work through this problem, and see what you can discover using pencil and paper. Then keep reading.

My math is below, and a discussion below that.

$$\int_{-\infty}^0 \sin(\theta)\,d\theta \;=\; \lim_{t\to-\infty} -\cos(\theta)\,\Big|_t^0 \tag{15.51}$$

$$= \lim_{t\to\infty} \Big[-\cos(0) + \cos(t)\Big] \tag{15.52}$$

$$= \lim_{t\to\infty} \Big(-1 + \cos(t)\Big) \tag{15.53}$$

The conclusion is similar to that above, but now the oscillatory divergence is bounded by $(-2, 0)$.

That's interesting: The definite integral is divergent on both sides of zero, and is bounded by $(-2, 0)$ on the left and (0,2) on the right. Don't give in to the temptation to conclude that those bounds cancel to produce an indefinite integral of zero; the correct conclusion is that $\int_{-\infty}^{\infty} \sin(\theta)\, d\theta$ does not exist in a finite sense; it is an oscillatory divergence that is bounded by $(-2, 2)$.

Again, with cosine What do you think will happen with the following integrals?

$$\int_{-\infty}^{0} \cos(\theta)\, d\theta \; = ? \tag{15.54}$$

$$\int_{0}^{\infty} \cos(\theta)\, d\theta \; = ? \tag{15.55}$$

$$\int_{-\infty}^{\infty} \cos(\theta)\, d\theta \; = ? \tag{15.56}$$

I won't walk through all the math — it's very similar to the sine example but the integral is slightly different. But I will tell you the final answers: All three definite integrals have oscillatory divergence. The two one-sided integrals are bound by $(-1, 1)$ while the two-sided integral is bound by $(-2, 2)$.

15.5.2 Gausian-tapered cosine

Consider the integral in Equation 15.57 (this function is called a "Morlet wavelet," and is used in time series and geospatial analyses).

$$\int_{-\infty}^{\infty} \cos(2\pi\theta) e^{-\theta^2}\, d\theta \; = ? \tag{15.57}$$

From inspecting the graph of the function (Figure 15.9), you can see how the Gaussian tapers the cosine wave.

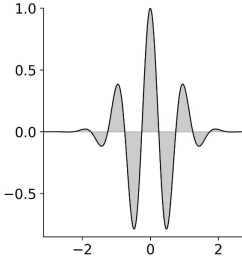


Figure 15.9: A "Morlet wavelet," defined as the product of a cosine and a Gaussian.

The definite integral, shaded in light gray, technically goes out to $\pm\infty$, but because $e^{-\theta^2}$ becomes vanishingly tiny, you don't need to see a wide domain to understand the characteristics of this function.

What is your intuition about the net area of that shaded region — do you think it converges or diverges, and if the former, what might its area be?

You might guess that the definite integral converges, considering that the function values become infinitesimal as $|\theta|$ increases. Furthermore, you might guess that the net area is zero, because it sorta-kinda looks like the positive bits and negative bits cancel.

That first intuition is correct: The integral in Equation 15.9 does indeed converge to a finite value. The second guess (if that was your guess), however, is not quite correct. The positive and negative lobes do not fully cancel. In fact, the net area is very slightly above zero.

$$\int_{-\infty}^{\infty} \cos(2\pi\theta)e^{-\theta^2}\, d\theta \;=\; \frac{\sqrt{\pi}}{e^{\pi^2}} \approx 0.0000916770 \qquad (15.58)$$

Why is the integral close-to-but-not-exactly zero? It's because the Gaussian tapering favors the central bump of the cosine, so it has slightly larger area than the neighboring bumps. You can explore this in the online code by changing the function to minus cosine (which will make the net area slightly negative), or by decreasing the frequency of the cosine term to $\cos(\pi\theta)$ or even $\cos(\theta)$. In Exercises 4 and 5 you will see what happens when using a sine wave instead of a cosine wave.

The other side of this coin is that increasing the frequency of the cosine wave (Figure 15.10 and Equation 15.59) increases the number of positive and negative deflections, which increases the cancellations.

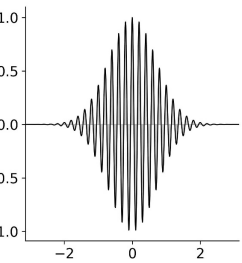


Figure 15.10: A faster Morlet wavelet.

Now the net area is quite a bit smaller. According to `sympy`:

$$\int_{-\infty}^{\infty} \cos(10\pi\theta)e^{-\theta^2}\, d\theta \approx 1.23227230324944 \times 10^{-107} \qquad (15.59)$$

By the way, the function $f(\theta) = \exp(-\theta^2)$ is non-integrable over finite bounds. There are advanced techniques for estimating the area with finite bounds — in fact, it's used very often in statistics

and machine-learning. I'll return to this discussion in Chapter 18.

15.6 Functions with discontinuities

Discontinuities can create improper integrals, even if the integration bounds are finite. It is possible to calculate definite integrals in functions that have discontinuities, although there is no single one-size-fits-all strategy that works for all discontinuities. Instead, you will need to diagnose each problem and determine the best course of action. That said, there are a few common strategies for calculating definite integrals with discontinuities, and in this section I will introduce those approaches.

Figure 15.11 shows a reminder of three common types of discontinuities, with their label on the top, and a recommended strategy for calculating discrete integrals beneath.

Discontinuities were discussed in detail in Section 3.8

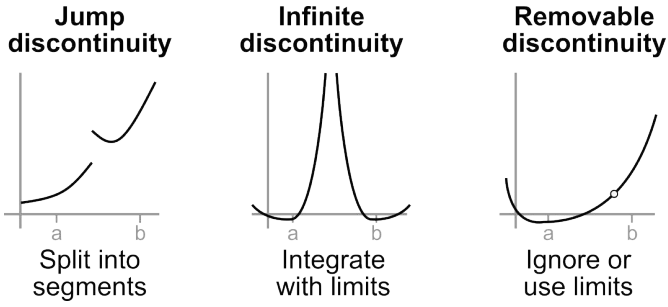


Figure 15.11: Reminder of three types of discontinuities, and suggested strategies for integration.

15.6.1 Jump discontinuity

Recall that a jump discontinuity occurs when there is a sudden jump on the y-axis from one x-axis value to the next. Here's an example of a definite integral of a function with a discontinuity.

$$\text{Find } \int_{-2}^{1} f(x)\,dx \text{ for } f(x) = \begin{cases} x/2 & \text{if } x < 0 \\ x^2 + 1 & \text{if } 0 \le x < 1 \end{cases} \tag{15.60}$$

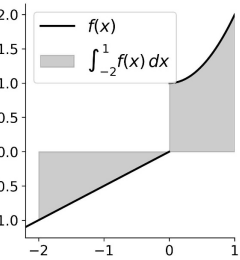


Figure 15.12: Visualization of problem 15.60.

Solving this problem is straightforward: Break up the integral into two integrals (each of which is proper), solve them separately, then sum the areas.

$$\int_{-2}^{1} f(x)\,dx \;=\; \int_{-2}^{0} f(x)\,dx + \int_{0}^{1} f(x)\,dx \tag{15.61}$$

The two intervals are split at the breakpoint. That split created two proper integrals instead of one improper integral.

I'll show the solution below, but I encourage you to solve it on your own to get some more practice with integration.

$$\int_{-2}^{0} \frac{x}{2}\,dx \;=\; \frac{x^2}{4}\Big|_{-2}^{0} \;=\; 0 - 1 \tag{15.62}$$

$$\int_{0}^{1} x^2 + 1\,dx \;=\; \frac{x^3}{3} + x\Big|_{0}^{1} \;=\; \frac{4}{3} \tag{15.63}$$

$$\int_{-2}^{1} f(x)\,dx \;=\; -1 + \frac{4}{3} \;=\; \frac{1}{3} \tag{15.64}$$

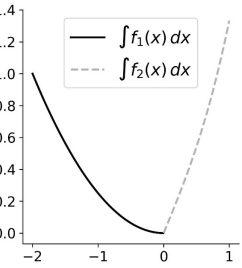


Figure 15.13: Solution to problem 15.60.

Figure 15.13 shows the indefinite integral of the function. Visual appearance suggests that the integral is continuous, but don't jump[2] to that conclusion! I set the constants of integration such that the two antiderivatives meet at the origin of the graph. This means that although you can visually confirm the FTC-2 within each piece of the function (e.g., $F(0) - F(-2) = -1$), you cannot know whether those two antiderivatives truly meet at the origin without additional constraints.

[2] Apologies for the bad pun.

15.6.2 Infinite discontinuity

Consider the following function and its antiderivative, which are shown in Figure 15.14.

$$\int \frac{dx}{\sqrt{x-1}} = 2\sqrt{x-1} + C \tag{15.65}$$

This integral is solved using u-substitution, which you'll learn about in the next chapter.

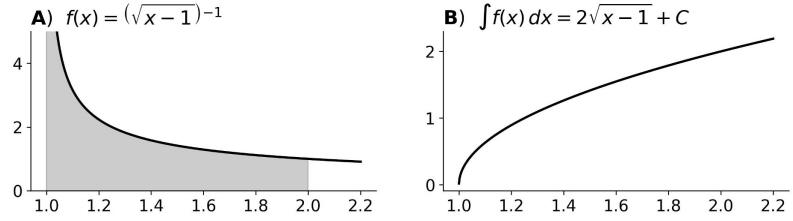


Figure 15.14: A function with an infinite discontinuity at $x = 1$, and its antiderivative (plotted using $C = 0$).

How would you calculate the following definite integral?

$$\int_1^2 \frac{dx}{\sqrt{x-1}} = ? \tag{15.66}$$

The bounds are finite, but the function is undefined for $x \leq 1$, and it blows up to ∞ as $x \to \infty$ from the right.

I will explain how to solve this type of integration problem, but please spend some time thinking about it before reading the next paragraph.

I'm sure you guessed that the solution involves taking the limit as x approaches the discontinuity point. But we need to be mindful of taking the one-sided limit as $x \to 1$ from the right — we'd run into trouble if we try to approach $x = 1$ from the left.

I'm going to write out this problem is a way that is slightly different from how you've seen limits before in this book: Rather than approach the number itself, add a small constant and then take the limit as that constant approaches zero.

$$\lim_{\epsilon \to 0^+} \int_{1+\epsilon}^2 \frac{dx}{\sqrt{x-1}} = \lim_{\epsilon \to 0^+} 2\sqrt{x-1} \Big|_{1+\epsilon}^2 \tag{15.67}$$

$$= \lim_{\epsilon \to 0^+} \left(2\sqrt{2-1} \; - \; 2\sqrt{1+\epsilon-1} \right) \tag{15.68}$$

$$= 2 - \left(\lim_{\epsilon \to 0^+} 2\sqrt{\epsilon} \right) \tag{15.69}$$

$$= 2 - 0 = 2 \tag{15.70}$$

Again, we have an example of a function that extends to infinity (in this example: as $f(x)$ approaches $x = 1$ from the right) and yet has a finite area. You can also infer that from the antiderivative: The $\sqrt{x-1}$ term becomes vanishingly close to zero as $x \to 1^+$, thus making a negligible contribution to the definite integral.

15.6.3 Removable discontinuity

Here's the thing about removable discontinuities: If there's really only exactly one function value missing, then its impact on a definite integral is negligible and can be ignored.

Consider this function and its antiderivative, and its graph in Figure 15.15.

$$\int \frac{x^2 - 2x}{x^2 - 4} \, dx \; = \; x - 2\ln(x+2) + C \tag{15.71}$$

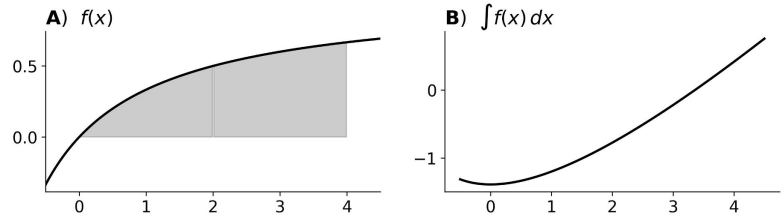


Figure 15.15: A function with a removable discontinuity at $x = 2$, and its antiderivative. I created the x-axis grid in Python to include exactly $x = 2$, which produced a warning message and a small discoloration in the gray patch in the figure.

The discontinuity happens at $f(2) = 0/0$. However, this function has a two-sided limit at $x = 2$ (it's $1/2$, which you can see in the graph). So the function values get arbitrarily close to $1/2$ as $x \to$

2. Because integration works by computing sums over arbitrarily thin slices of x, this little removable discontinuity doesn't actually impact the total integral.

Therefore, the solution to this type of discontinuity is simple: Pretend it doesn't exist.

$$\int_0^4 \frac{x^2 - 2x}{x^2 - 4}\, dx = x - 2\ln(x+2)|_0^4 \tag{15.72}$$

$$= 4 - 2\ln(4+2) - (0 - 2\ln(0+2)) \tag{15.73}$$

$$= 4 - 2\ln(12) \approx 1.8 \tag{15.74}$$

Ignoring a discontinuity might not always be a successful strategy. Depending on the nature of the function and the integration bounds, you might need to use limits. You'll see an example of such a problem in Exercise 8.

15.7 Exercises

15.1. Implement and display the following functions and their definite integrals using `sympy`.

a) $\int_1^\infty \frac{1}{x^2}\,dx$ **b)** $\int_1^\infty \frac{\pi}{x^3}\,dx$ **c)** $\int_1^\infty \frac{1}{\sqrt{x}}\,dx$

d) $\int_0^\infty |\cos(x)|\,dx$ **e)** $\int_0^\infty \cos(x)\,dx$ **f)** $\int_0^\infty \cos\left(x^2\right)\,dx$

Notice in particular what happens with the cosine integrals.

15.2. The goal of this exercise is to explore the rule about convergence for power functions.

In each iteration of a for-loop over 21 values of p between -3 and 1: (**1**) implement the function $f(x) = x^p$ as a `sympy` expression, (**2**) obtain the indefinite integral using `sym.integrate`, (**3**) calculate $\int_1^\infty x^p\,dx$ via directly implementing FTC-2 (that is, write code to find the output of $F(\infty) - F(1)$), (**4**) determine whether that result is finite or infinite, (**5**) visualize the indefinite integral and result of the convergence test as in Figure 15.16.

The results in Figure 15.16 show convergent integrals using black lines and squares, and divergent integrals using gray lines and squares. Note that you don't need to know the value of the definite integrals; you only need to figure out how to check whether a `sympy` variable is finite.

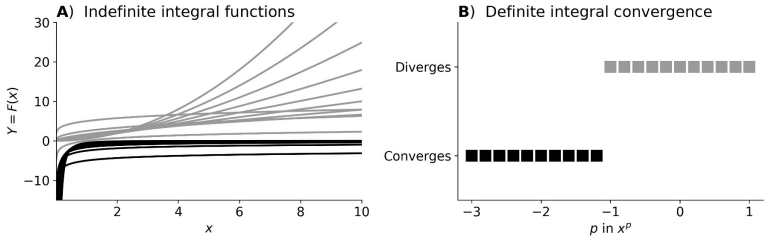


Figure 15.16: Results for Exercise 2.

15.3. In this exercise, you will explore the how the definite integral depends on the value of the upper limit in numerical approximations, and on the exponent p in x^p.

Here's the setup: We want to calculate Equation 15.75, but in practice we might need to approximate it numerically using summations as in Equation 15.76. The question is how to choose a value for γ (Greek letter "gamma").

$$A_p = \int_1^{\infty} x^p \, dx \tag{15.75}$$

$$A_{p,\gamma} = \sum_1^{\gamma} x^p \Delta x \tag{15.76}$$

Your goal is to understand and then recreate Figure 15.17. Panel A shows 21 lines corresponding to $f(x) = x^p$ for 21 values of p between -3 and -1.5.

In panel B, the x-axis is the γ parameter in Equation 15.76, which ranged from 2 to 10^5 in 43 steps and scaled logarithmically. Each line shows the value of the definite integral for each parameter pair p and γ. You can think of the lines as comprising dots, with each dot defined as $A_{p,\gamma}$ for one parameter-pair $_{p,\gamma}$.

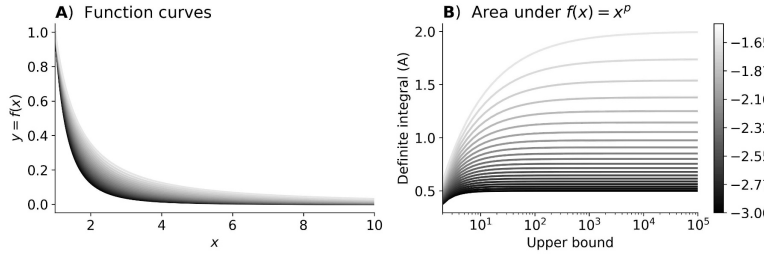


Figure 15.17: Results for Exercise 3.

The upshot of this figure is that the definite integrals of power functions investigated here converge to their values-at-infinity with a "relatively small" x, and the convergence is faster when p is more negative[3].

On the one hand, that conclusion is limited to the exponents and the bounds simulated here — it would be inappropriate to conclude that *all* functions converge to their infinite limit value by $x \approx 100$ — but this exercise does highlight that the asymptotic behavior of many functions can be understood by approximations to a "reasonably large" value without having to risk computational problems from numerical overflow.

By the way, the colorbar in panel B is tricky to implement. If you've solved the rest of this exercise but are struggling with that colorbar mapping, then feel free to inspect my online code solution (hint: look up `cm.ScalarMappable` in the `matplotlib` library).

15.4. The goal of this exercise is to explore the improper definite integral of a Morlet wavelet. There are two parts to this exercise. First, modify the `sympy` code that calculates the integral of the cosine-based wavelet (Equation 15.57) to use a sine wave instead of a cosine. What happens?

...what happens is something bad. `sympy` is unable to solve this integral. This brings us to the second part of this exercise.

[3] "Relatively small" compared to ∞.

Implement the function in `numpy` in order to approximate the net area using `scipy`. Define θ to be a grid of 5001 points between $-\pi$ and π. Visualize your results like Figure 4.

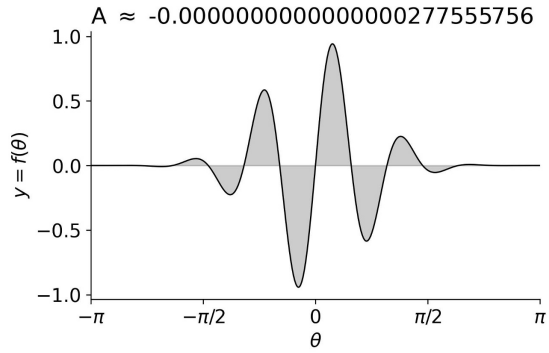


Figure 15.18: Results for Exercises 4.

Although the area is not *exactly* zero, I think we can all agree that it is not appreciably different from zero. This happens because the Gaussian taper is no longer centered on the cosine peak at $\theta = 0$, but instead is balanced between the negative and positive peaks in the sine function.

15.5. Earlier in the text, I showed that increasing the frequency of the cosine term brings the net area closer to zero. The purpose of this exercise is to explore this phenomenon in more detail.

Implement the following two indefinite integrals in a for-loop over a range of values of f (frequency) from $1/2$ to 4π.

$$\int_{-\infty}^{\infty} \sin(f\theta)e^{-\theta^2}\, d\theta \tag{15.77}$$

$$\int_{-\infty}^{\infty} \cos(f\theta)e^{-\theta^2}\, d\theta \tag{15.78}$$

Plot the net areas of the sine and cosine wavelets across the

specified frequencies as in Figure 15.19. What conclusions do you draw from this investigation?

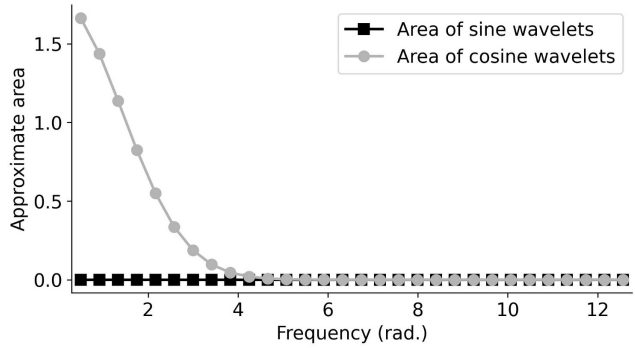


Figure 15.19: Results for Exercise 5.

15.6. This exercise continues to explore the definite integral of wavelets as a function of their frequency.

Generate 30 Morlet wavelets with frequencies ranging from $\pi/10$ to 6π (Equation 15.78), using 5001 values of θ between -20π and 20π. To improve visual interpretation, I plotted every 9^{th} wavelet in Figure 15.20A, and restricted the x-axis limits.

Next, use `spi.trapezoid` to approximate the net and total areas, and show the areas in a plot like Figure 15.20B.

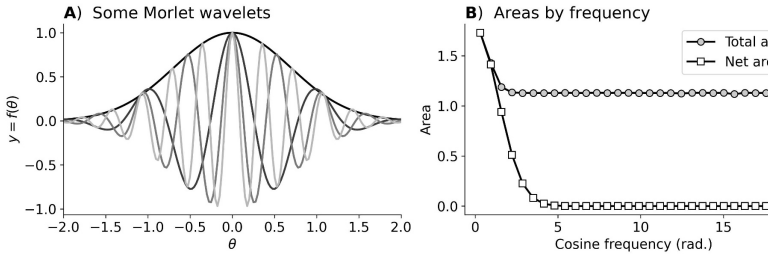


Figure 15.20: Results for Exercise 6.

Finally, explore the impact of the θ resolution on the results. Try using 501 points instead of 5001 and recreate the figure. How do the results look? How about if you use 50001 points? What conclusions can you draw about the

impact of resolution on area calculations?

15.7. Implement and calculate the antiderivatives of the two equations in Figure 15.21 in `sympy`. Notice that `sympy` is unable to derive an analytic formula for the integral, and so represents the integral of the function using the integration symbol instead of as an explicit equation.

$$f_1(x) = \frac{1}{\sqrt{x}\,(\sin(x)+2)}, \quad F_1(x) = \int \frac{1}{\sqrt{x}\,(\sin(x)+2)}\,dx$$

$$f_2(x) = \frac{1}{x^2\,(\sin(x)+2)}, \quad F_2(x) = \int \frac{1}{x^2\,(\sin(x)+2)}\,dx$$

Figure 15.21: Screenshot of `sympy` output for Exercise 7.

Numerically evaluate the function and its antiderivative, and visualize as in Figure 15.22.

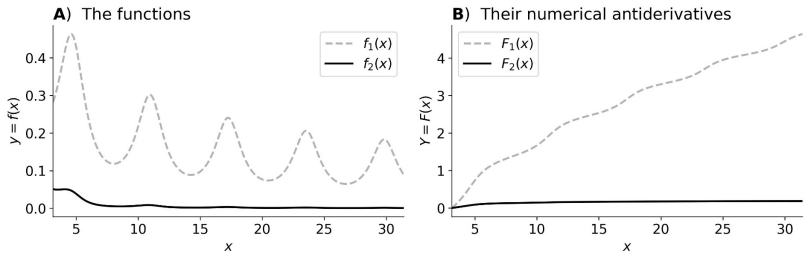


Figure 15.22: Results for Exercise 7.

The main calculus question of this exercise is whether the one-side unbounded definite integral converges or diverges. In particular, whether neither, one, or both of the following two expressions will be ∞ or some finite value.

$$\int_{\pi}^{\infty} \frac{1}{\sqrt{x}\,(\sin(x)+2)}\,dx \tag{15.79}$$

$$\int_{\pi}^{\infty} \frac{1}{x^2\,(\sin(x)+2)}\,dx \tag{15.80}$$

What's your intuition? Spend some time looking at the equations and make an educated guess. Then evaluate

these two expressions in `sympy` and see if your intuition was correct.

After that you can keep reading below.

To gain some intuition, consider that the denominators are the product of an x term and a sine term. As $x \to \infty$, the x terms both grow arbitrarily large whereas the sine term is bounded by [1,3]. This means that you can ignore the sine term and think about these functions simply as $x^{-1/2}$ and x^{-2}. To be clear: the sine term does impact the functions for "reasonably small" values of x, but the problem asks us to consider the behavior of the functions for "unreasonably large" values of x, and the sine term (shifted so that it is always positive) is irrelevant at that galactic scale.

The point is that we know from the exponent rule I described earlier for x^{-p} that $f_1(x)$ will diverge while $f_2(x)$ will converge to some value. `sympy` has some difficulties with calculating an analytic value, but the results do confirm our intuition (Figure 15.23). (The first expression in each row is a statement of the integral, the second expression is supposed to be the result of the integration, and the third term is the numerical evaluation.)

$$\int_{\pi}^{\infty} \frac{1}{\sqrt{x}\,(\sin(x)+2)}\,dx \;=\; \int_{\pi}^{\infty} \frac{1}{\sqrt{x}\,(\sin(x)+2)}\,dx \;\approx\; 2.0 \cdot 10^{52}$$

$$\int_{\pi}^{\infty} \frac{1}{x^2\,(\sin(x)+2)}\,dx \;=\; \int_{\pi}^{\infty} \frac{1}{x^2\,(\sin(x)+2)}\,dx \;\approx\; 0.20483128$$

Figure 15.23: Screenshot of `sympy` output for Exercise 7.

Comment: The lines in Figure 15.22 do look like they confirm our answers, but you always need to exert caution when interpreting graphs of functions when the question asks about behavior outside the plotted area. For example, the line corresponding to $F_2(x)$ appears flat, but perhaps this is just a visual artifact of the y-axis scaling; it could be possible for that line to continue increasing to ∞ but more slowly than the gray dashed line.

And this brings me to the final task for this exercise: Recreate Figure 15.22 but normalize each of the four lines to their own maximum (resulting graph not shown here). Thus, the largest value of each line will be at $y = 1$. This is sometimes referred to as max-val (maximum-value) normalization, and is used in data visualization to compare the behavior of different functions or data streams.

Note that after this normalization, the y-axis values are no longer directly comparable across different functions; instead, the advantage of this normalization is to compare the overall behaviors of the different functions.

15.8. Let's return to the sinc function. I happen to be a fan of this function — and so are many scientists, engineers, and mathematicians, because the sinc function has many interesting properties that make it useful in signal processing applications. For purposes of this exercise, it is interesting because it has a removable discontinuity at $x = 0$.

Implement the sinc function in `sympy`, calculate and print its indefinite integral as in Figure 15.24, and then evaluate those two numerically to visualize as in Figure 15.25 for x between -14π and 14π using $\Delta x = .01$.

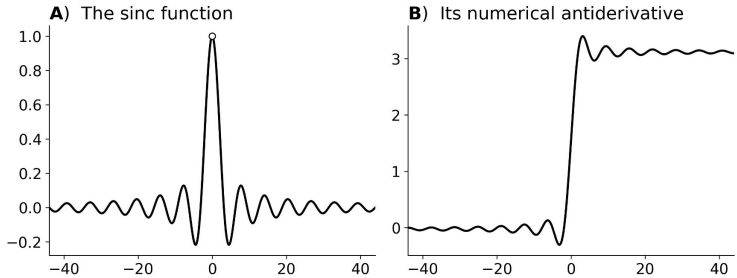


Figure 15.25: Results for Exercise 8.

$$f(x) = \frac{\sin(x)}{x}$$

$$F(x) = \mathrm{Si}(x)$$

Figure 15.24: Screenshot of `sympy` output for Exercise 8.

"Si(x)" stands for "sine integral." The sinc function does not have an antiderivative that is expressible using elementary functions.

Now that you've setup the problem, the goal is to determine the following integral:

$$\int_0^1 \frac{\sin(x)}{x}\, dx \tag{15.81}$$

Figure 15.25 shows the function evaluated over a much larger domain than the interval specified in Equation 15.81; Figure 15.26 zooms into the relevant subdomain. Re-calculate this function using $\Delta x = .001$.

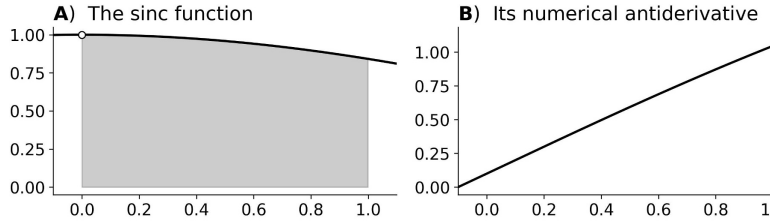


Figure 15.26: Results for Exercise 8.

Finally, calculate the definite integral twice: once using `sympy`, and once by applying the FTC-2 on the numerical antiderivative you obtained from `scipy`. I got the following results:

```
Sympy result: 0.946083070367183
Scipy result: 0.9460830452697918
```

15.9. This exercise is open-ended, and there is no correct answer for you to find in the online code.

Go to the Wikipedia page for the list of definite integrals[4], scroll down about one-third of the page until you find a list of improper trig integrals. Pick a few that look interesting, implement them in `numpy`, visualize them using `matplotlib`, and approximate their improper definite integrals using `scipy`.

Translating math equations into code that you can visualize and explore is a great skill to have, especially if you

[4] https://en.wikipedia.org/wiki/List_of_definite_integrals

want to implement, adapt, or develop novel data-related techniques in machine-learning.

Chapter 16

Integration techniques and tricks

Before getting into the content, I want to tell you about my struggles and eventual decision with this chapter.

Here's the thing: learning integral calculus mostly boils down to (**1**) concepts and (**2**) integration tricks. There are thousands of websites, YouTube videos, online courses, and books with countless worked examples of various integration tricks to memorize and rote-practice.

On the other hand, there are no modern applications of calculus in which you will need to perform, for example, u-substitution by hand. But there are myriad applications in which you need to understand, visualize, calculate, and interpret integrals using `sympy` or `scipy`, e.g., to measure energy, revenue, volume, probability densities, or population dynamics.

So my decision was to have one chapter to introduce common integration techniques, and to focus the book on concepts and code implementations that are under-emphasized in plug-and-chug courses and books.

16.1 U-substitution

Recall that the chain rule is used to differentiate composite functions: $\big[f(g(x))\big]' = f'(g(x))g'(x)$. There is no universal way to reverse the chain rule for integration. But u-substitution is the closest thing to an "inverse chain rule," and it's often used to integrate functions that can be expressed as compositions of simpler functions.

U-substitution is often abbreviated to "u-sub."

The idea of u-sub is to create a new variable u that substitutes for an expression of x, then rewrite the integration problem as a function of u instead of x, solve that function, then substitute back x.

Example 1 The first example is to integrate the following:

$$\int \frac{x}{x^2+2}\,dx \tag{16.1}$$

The function in the integrand is not terribly complicated (in fact, if you ignore the shift in the denominator, the integrand is basically just $1/x$). But it cannot be integrated using methods you learned in previous chapters.

Below I will define a new expression u, take its derivative to find du, and then solve for dx. These expressions do not yet solve Equation 16.1; I'm just setting up the problem so that it can be solved.

$$u = x^2+2 \tag{16.2}$$

Note the chain rule when differentiating x^2.

$$du = 2x\,dx \tag{16.3}$$

$$dx = du/2x \tag{16.4}$$

Now in the integrand, we can replace x^2+2 with u, and we can replace dx with $du/2x$. The x terms cancel and we've gained a factor of $1/2$. This means that we've removed variable x from the equation, and we now have a function of u (where u is a function of x).

$$\int \frac{1}{2u}\,du = \frac{1}{2}\int u^{-1}\,du \tag{16.5}$$

This is fantastic, because Equation 16.5 can easily be integrated:

$$\frac{1}{2}\int u^{-1}\,du = \frac{1}{2}\ln|u| + C \tag{16.6}$$

That concludes the calculus part of u-sub. But it's not the end of the problem: We started with a function of x, and so we need to finish with a function of x. Therefore, the final step is to back-substitute x^2+2 whenever you see u:

$$\int \frac{x}{x^2+2}\,dx \;=\; \frac{1}{2}\ln|x^2+2| + C \tag{16.7}$$

And that's the end. For your viewing pleasure, I've drawn the function and its antiderivative in Figure 16.1.

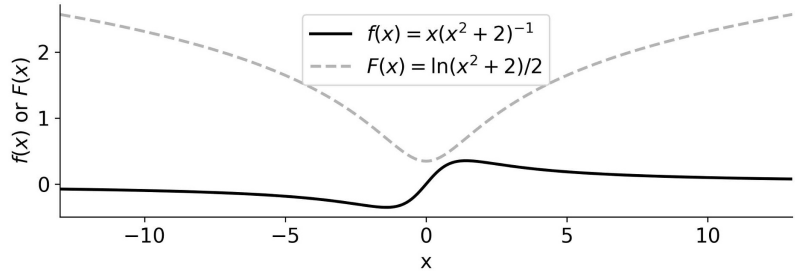


Figure 16.1: Visualizing the problem and its solution (with $C = 0$).

When does u-sub work, and how do you know what to use for the substitution? You need to look at the equation and see if you can identify one term that could form part of the derivative of another term.

I encourage you to put the book aside and re-work Equation 16.1 from scratch.

Example 2 See how far you can get with this example before reading my explanation below. If you don't know where to start, then read the paragraph after this equation and try again.

$$\int \left(3x^4+5\right)^6 x^3\,dx \tag{16.8}$$

Notice that x^3 is part of the derivative of $3x^4 + 5$. It's not the full derivative, of course; that would be $12x^3$. But the factor of 12 is just a constant that we could introduce and then factor out of the integrand.

If you got stuck with the substitution, then you can peek at the following expressions:

$$u = 3x^4 + 5 \tag{16.9}$$

$$du = 12x^3\,dx \tag{16.10}$$

$$dx = du/(12x^3) \tag{16.11}$$

And now for the rest of the solution:

$$\frac{1}{12}\int u^6\,du = \frac{1}{12}\frac{1}{7}u^7 + C \tag{16.12}$$

$$= \frac{1}{84}\left(3x^4 + 5\right)^7 + C \tag{16.13}$$

By the way, you don't actually *need* u-sub to solve this problem. You could have expanded all the terms to get a polynomial function that you can integrate using the power rule, then factor. But it would be tedious, time-consuming, and error-prone. Here's what it would look like:

$$f(x) = 729x^{27} + 7290x^{23} + 30375x^{19} + 67500x^{15} + 84375x^{11} + 56250x^7 + 15625x^3 \tag{16.14}$$

No, I didn't do this by hand; I used `sym.expand` :P

$$F(x) = \frac{729x^{28}}{28} + \frac{1215x^{24}}{4} + \frac{6075x^{20}}{4} + \frac{16875x^{16}}{4} + \frac{28125x^{12}}{4} + \frac{28125x^8}{4} + \frac{15625x^4}{4} \tag{16.15}$$

You can see that u-sub is conceptually straightforward. It can get tricky to know what expression to pick for u; here are a few tips:

1. Expressions inside parentheses (or inside functions like log, cosine, or square root) often make good substitutions.

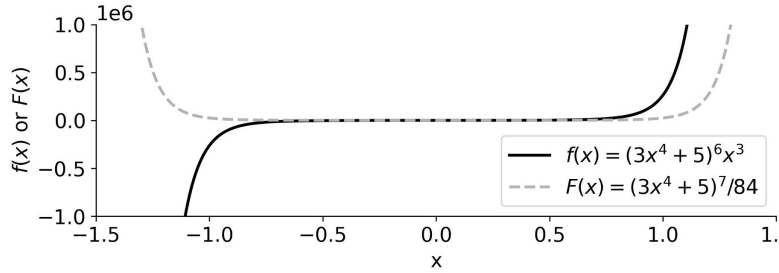


Figure 16.2: The second example. Check out the y-axis; these large polynomials really shoot off to the moon.

2. Make sure the integrand includes the derivative of u (ignoring constants that can be included and then factored out of the integral).

3. Choose a u that reduces the number of terms in the integrand. If you find that your choice of substitution either preserves or increases the number of terms in the integrand, then you're probably not choosing the right substitution.

4. Some integrands are not immediately amenable to u-sub, but can be algebraically transformed, for example by expanding, simplifying, or completing the square.

5. To be honest, if you really want to master u-sub as opposed to just being familiar with the concept, you need to solve *lots* of problems over a long period of time.

16.2 Integration by parts

Analogous to how u-sub is kind of like an inverse chain rule, *integration by parts* is kind of like an inverse product rule: You can use integration by parts to integrate the product of terms. However, it's not useful for integrating *any* given product of functions; it's useful only when one of the functions is closely related to the derivative of the other. You'll see what I mean as I explain the rule and show examples.

The rule for integration by parts is expressed like this:

$$\int u\,dv \;=\; uv - \int v\,du \tag{16.16}$$

where u and v are functions of x. Notice that v appears in the right-hand-side while dv (the derivative of v) appears in the left-hand-side. That's already a hint that integration by parts works only when the functions follow a specific pattern.

I will first explain how to arrive at this rule, and then I'll show a few examples.

The proof of integration by parts starts from the product rule and then integrates. In the equations below, $f = f(x)$ and $g = g(x)$; I'm omitting the (x) for simplicity.

$$[\,f\,g\,]' = f'\,g + f\,g' \tag{16.17}$$

$$f g' = [\,f g\,]' - f' g \tag{16.18}$$

$$\int f g'\,dx = \int \Big([\,f g\,]' - f' g\Big)\,dx \tag{16.19}$$

$$= \int [\,f g\,]'\,dx - \int f' g\,dx \tag{16.20}$$

$$= f g - \int f' g\,dx \tag{16.21}$$

Now for some explanations. Equation 16.17 is the definition of the product rule. Equation 16.18 moves the terms around. Equation 16.19 is the "trick": integrate both sides of the equation. Equation 16.20 simply rewrites the right-hand-side leveraging the linearity of integration. Notice that the first term is the integral of a derivative, and so Equation 16.21 applies the FTC-1 to finish the proof. (I've omitted the constant of integration in Equation 16.21, but that would be absorbed into the constant from the second integral.)

It is common to write the integration by parts rule using u and v instead of f and g. Thus, to link Equations 16.16 and 16.21,

$u = f(x)$ and $v = g(x)$. In fact, I recommend writing the proof out again, starting from Equation 16.17 but using u and v instead of f and g.

The way to apply this integration technique is to pick one function to be u and the other to be dv, then rewrite the expression according to Equation 16.16, and then back-substitute at the end.

Example 1 Here's the problem:

$$\int x \cos(x)\, dx \tag{16.22}$$

We need to pick one of those terms to be u and the other term (including dx) to be dv. The best option is $u = x$ and $dv = \cos(x)dx$. That's a good start, but we're not yet ready to substitute, because we need du and v. So we need to differentiate u and integrate dv:

$$\begin{aligned} u &= x & v &= \sin(x) \\ du &= dx & dv &= \cos(x)\, dx \end{aligned}$$

Now we're ready to substitute and solve:

$$\int x \cos(x)\, dx = x \sin(x) - \int \sin(x)\, dx \tag{16.23}$$

$$= x \sin(x) + \cos(x) + C \tag{16.24}$$

Confirm Equation 16.24 by differentiating to reveal the original integrand!

Integration by parts can be tricky. I have two recommendations before proceeding to the next example: First, solve this example problem on your own using the same choices of u and dv that I showed. Second, try to solve it again using $u = \cos(x)$ and $dv = x\, dx$. You will discover that selecting the right function pair is really important — and not trivial.

Example 2 Before walking through the solution step-by-step, I want to show you the final result (Equation 16.25 and Figure 16.4) so you can marvel at the feat you are about to accomplish.

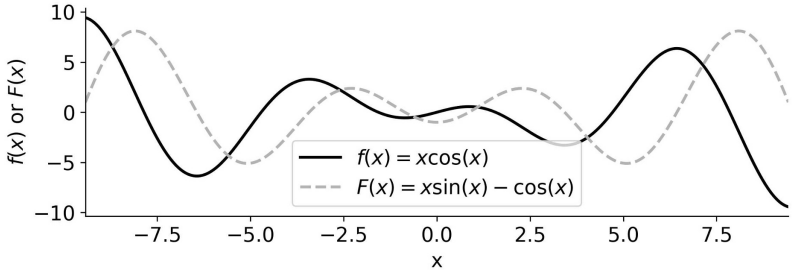


Figure 16.3: The integrand and its antiderivative from Example 1.

$$\int e^{-\pi x}\sin(7x)\,dx \;=\; \frac{-\pi\sin(7x)e^{-\pi x} - 7\cos(7x)e^{-\pi x}}{\pi^2 + 49} + C \tag{16.25}$$

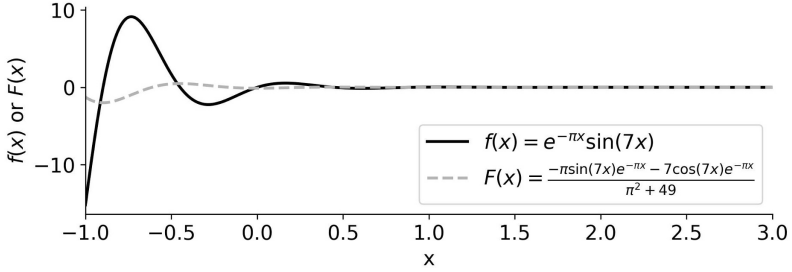


Figure 16.4: The function gets really wild left of the origin.

The best choice here is to set u as the sine function and dv as the natural exponential function. Both are easy to integrate, but the exponential is its own integral, which will help simplify the subsequent steps.

$$u = \sin(7x) \qquad v = \frac{-e^{-\pi x}}{\pi}$$

$$du = 7\cos(7x)\,dx \quad dv = e^{-\pi x}\,dx$$

Now we substitute and see what progress we've made.

$$\int e^{-\pi x}\sin(7x)\,dx \;=\; \frac{-\sin(7x)e^{-\pi x}}{\pi} + \frac{7}{\pi}\int \cos(7x)e^{-\pi x}\,dx \tag{16.26}$$

Hmm... it looks like we haven't made meaningful progress. We still have an indefinite integral with two terms — in fact, it's basically the same integral that we started with except with a cosine instead of a sine.

But we actually are making progress. We need to apply integration by parts *again* on the second integral. Looking ahead, the exponential term will stay the same while the cosine will cycle back to a sine. That may seem like we're just spinning wheels and making an even longer expression, but the cyclicity of sine will allow for cancellations.

We need another set of substitutions, this time using u as the cosine term.

$$u = \cos(7x) \qquad v = \frac{-e^{-\pi x}}{\pi}$$

$$du = -7\sin(7x)\,dx \quad dv = e^{-\pi x}\,dx$$

Now we can substitute again. Be mindful that the substitution is for the integral, and the $7/\pi$ scales the entire integral.

Equation 16.26 is repeated from earlier so you can see the progression.

$$\int e^{-\pi x}\sin(7x)\,dx \;=\; \frac{-\sin(7x)e^{-\pi x}}{\pi} + \frac{7}{\pi}\int \cos(7x)e^{-\pi x}\,dx \tag{16.26}$$

$$= \frac{-\sin(7x)e^{-\pi x}}{\pi} +$$

$$\frac{7}{\pi}\left[\frac{-\cos(7x)e^{-\pi x}}{\pi} - \frac{7}{\pi}\int e^{-\pi x}\sin(7x)\,dx\right.$$

$$= \frac{-\sin(7x)e^{-\pi x}}{\pi} - \frac{7\cos(7x)e^{-\pi x}}{\pi^2} - \frac{49}{\pi^2}\int e^{-\pi x}\sin(7x)\,dx \tag{16.28}$$

At this point, you might be thinking that we still haven't made any progress, but notice that the integrals on the left- and right-hand sides of the equation are identical except for a scalar. So if we add $49/\pi^2 \int$ to both sides of the equation, we'll end up with an integral on the left and no integral on the right. From there, it's just a bit of algebra to isolate the integral on the left-hand-side.

$$\frac{\pi^2+49}{\pi^2}\int e^{-\pi x}\sin(7x)\,dx = \frac{-\sin(7x)e^{-\pi x}}{\pi} - \frac{7\cos(7x)e^{-\pi x}}{\pi^2} \tag{16.29}$$

$$(\pi^2+49)\int e^{-\pi x}\sin(7x)\,dx = -\pi\sin(7x)e^{-\pi x} - 7\cos(7x)e^{-\pi x} \tag{16.30}$$

$$\int e^{-\pi x}\sin(7x)\,dx = \frac{-\pi\sin(7x)e^{-\pi x} - 7\cos(7x)e^{-\pi x}}{\pi^2+49} + C \tag{16.31}$$

And that's the final solution. The key insight that led to this solution was seeing that e^x integrates to itself while sine/cosine cycle around, so applying integration by parts twice would allow for cancellations. That is not the kind of insight that you have when you first learn about integration techniques, but instead is a skill and intuition that you develop after watching dozens of solutions, and then solving hundreds of problems on your own.

16.3 Partial fractions

Partial fractions isn't really a *calculus* technique per se; it's an algebra technique that allows you to simplify some types of expressions so that they are easier to integrate.

$$\int \frac{5x+3}{(2x+2)(x-3)}\,dx \quad \Rightarrow \quad \int \frac{1/2}{2x+2} + \frac{9/4}{x-3}\,dx \tag{16.32}$$

The integrand on the left is not easily integrated, whereas the integral on the right is straightforward to solve. The method of partial fractions is how you transform the left side to the right side of Equation 16.32. Recall from Chapter 14 (also Equation 5.79, page 201) that

$$\int \frac{a}{x+b}\,dx \;=\; a\ln|x+b| + C \tag{16.33}$$

Partial fractions can be used when the integrand has the product of x terms in the denominator. You may need to factor the expression to get the denominator expressed as a product. For example, if Equation 16.32 had the denominator $2x^2 - 4x - 6$, then partial fractions wouldn't immediately work until you factor that quadratic into the form you see in the equation.

Then, the goal is to apply algebra to solve for A and B in the following.

$$\frac{5x+3}{(2x+2)(x-3)} \;=\; \frac{A}{2x+2} + \frac{B}{x-3}$$

Example 1 The first example will be the equation above. I'll start by showing the series of equations to remove the fractions. The goal is to have x terms of the same power on each side of the equation. That is, group all the x^0 terms together and group all the x^1 terms together.

$$\frac{5x+3}{(2x+2)(x-3)} = \frac{A}{2x+2} + \frac{B}{x-3} \tag{16.34}$$

$$(2x+2)(x-3)\frac{5x+3}{(2x+2)(x-3)} = \left[\frac{A}{2x+2} + \frac{B}{x-3}\right](2x+2)(x-3) \tag{16.35}$$

$$5x+3 = A(x-3) + B(2x+2) \tag{16.36}$$

$$5x + 3 = Ax - 3A + 2Bx + 2B \quad (16.37)$$

$$5x + 3 = x(A + 2B) + (-3A + 2B) \quad (16.38)$$

Notice that both sides of Equation 16.38 are the sum of two terms, one with x^0 and one with x^1 (the second set of parentheses facilitates visual grouping). That's useful because we now know the following:

$$5 = A + 2B \quad (16.39)$$

$$3 = -3A + 2B \quad (16.40)$$

This provides a system with two unknowns and two equations. I won't write out the algebra (you can solve this by, for example, solving for A in the first equation to substitute into the second), but the conclusion is that $A = 1/2$ and $B = 9/4$.

Finally, we go back to the original problem and integrate:

$$\int \frac{1/2}{2x+2} + \frac{9/4}{x-3}\,dx \;=\; \frac{1}{2}\ln|2x+2| + \frac{9}{4}\ln|x-3| + C \quad (16.41)$$

Figure 16.5 shows the function and its derivative. As you can clearly see from the function definition, there are infinite discontinuities at $x = -1$ and $x = 3$.

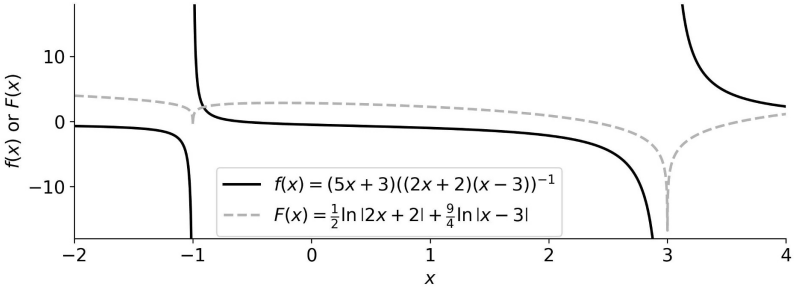


Figure 16.5: The function and its derivative.

Example 2 In this example, I will show you a slightly different way of solving the partial fractions problem. First, the problem

and its solution:

$$\int \frac{2x^2 - 3x - 5}{(x^2 - 1)(x - 2)}\, dx \;=\; 3\ln|x-1| - \ln|x-2| + C \qquad (16.42)$$

You need to start this problem by expanding the denominator terms to get the product of three terms:

$$\frac{2x^2 - 3x - 5}{(x^2 - 1)(x - 2)} \;=\; \frac{2x^2 - 3x - 5}{(x-1)(x+1)(x-2)}$$

From here, partial fractions begins as before, although we now have three unknowns (A, B, and C). The equations below multiply through by the denominator to remove the fractions.

$$\frac{2x^2 - 3x - 5}{(x-1)(x+1)(x-2)} \;=\; \frac{A}{x-1} + \frac{B}{x+1} + \frac{C}{x-2} \qquad (16.43)$$

$$2x^2 - 3x - 5 \;=\; A(x+1)(x-2) + B(x-1)(x-2) + C(x-1)(x+1) \qquad (16.44)$$

From here, you can expand all the multiplications, organize the right-hand-side of the equation into three terms (one with x^2, one with x^1 and one with x^0), rewrite into a system of three equations with three unknowns, and then solve...

But there is a better way. Notice that we have the sum of three terms (one for each unknown variable), and that two of each of the three terms can be set to zero by selecting a particular value of x. For example, when $x = -1$, the A and C terms drop out.

Therefore, we can solve for the unknowns by assuming particular values of x.

$$x = 2 \left\{ \begin{array}{rcl} 2(2)^2 - 3(2) - 5 & = & 0 + 0 + C(2-1)(2+1) \\ 8 - 6 - 5 & = & C(4-1) \end{array} \right\}$$

$$x = 1 \left\{ \begin{array}{rcl} 2 - 3 - 5 & = & A(2)(-1) + 0 + 0 \\ & & \\ -6 & = & -2A \quad \Rightarrow \quad A = 3 \end{array} \right\} \tag{16.46}$$

$$x = -1 \left\{ 2 + 3 - 5 = B(-2)(-3) \quad \Rightarrow \quad B = 0 \right\} \tag{16.47}$$

And this brings us to the final answer, which is visualized in Figure 16.6:

$$\int \frac{3}{x-1} + \frac{0}{x+1} + \frac{-1}{x-2}\, dx \;=\; 3\ln|x-1| - \ln|x-2| + C \tag{16.48}$$

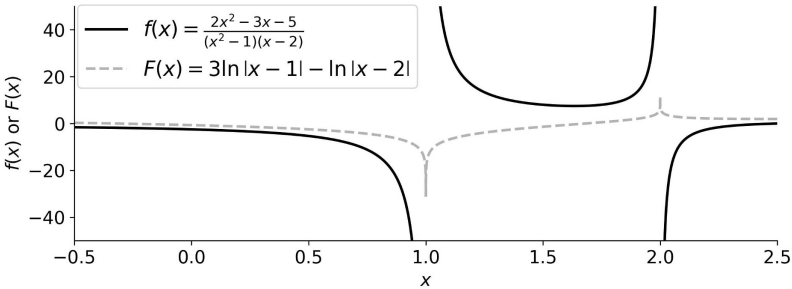


Figure 16.6: The function and its derivative.

16.4 Exercises

16.1. Implement Equation 16.22 in `sympy`, and confirm the antiderivative that we calculated using u-sub.

Then make a plot of the function on the x-axis by its antiderivative on the y-axis, as in Figure 16.7. I think it looks really neat, like a minimalist logo for an egg company or something. I prefer the black over the color gradient, but perhaps you prefer the colors.

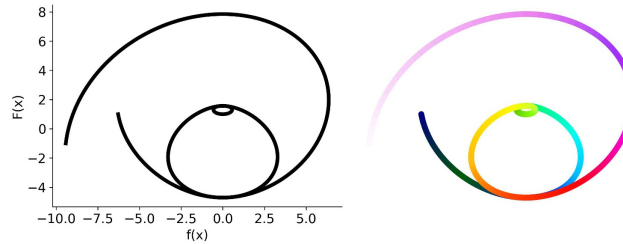


Figure 16.7: Results for Exercise 1.

16.2. Now implement Equation 16.25 (reprinted below) in `sympy` and compare it to the solution we obtained using integration by parts. Are the answers the same?

$$\int e^{-\pi x} \sin(7x)\, dx \;=\; \frac{-\pi \sin(7x)e^{-\pi x} - 7\cos(7x)e^{-\pi x}}{\pi^2 + 49} + C \tag{16.25}$$

The visual appearances of the equations are different. As you might imagine, the result in the book and in `sympy` are actually the same. Your goal is to do some algebra to transform one version into the other.

This exercise is not (only) algebraic busy work; there is an important message here: If you use `sympy` to check your work, you need to be aware that `sympy` may simplify, expand, or rewrite equations according to guidelines the developers programmed. If your solutions look different from

`sympy`'s, check whether they are really distinct or simply rewritten.

(I don't think the visualization like Figure 16.7 for this function is so interesting, but feel free to disagree with me!)

16.3. The goal of this exercise is to write an algorithm in `sympy` that can implement the partial fractions decomposition. *Note:* This is not meant to be a general algorithm that works for any possible rational function. Instead, the algorithm is tailored to one of the example problems in this chapter, and the purposes are (**1**) to ensure that you really understand the procedure, and (**2**) to learn how to work with, separate, and put together expressions and fractions in `sympy`.

Start from Equation 16.32 (repeated below).

$$f(x) = \frac{5x+3}{(2x+2)(x-3)}$$

Implement the following steps in code:

Step 1: Separate the numerator and denominator into separate variables, using a `sympy` method on the expression variable. Use the `display` and `Math` functions to print the results as follows:

The numerator is $5x + 3$
The denominator is $2x^2 - 4x - 6$

Step 2: Factor the denominator and print out the results like this:

Denominator factor 1: 2
Denominator factor 2: $x + 1$
Denominator factor 3: $x - 3$

Step 3: Create simple fractions in separate variables, and print as follows (note that A and B are `sympy` symbolic variables):

Simple fraction 1: $\frac{A}{2x+2}$

Simple fraction 2: $\dfrac{B}{x-3}$

Step 4: Solve for A and B. You should get $A = 1/2$ and $B = 9/4$.

Step 5: Substitute A and B in each separate expression, and integrate each of the two pieces separately. You should get the following results:

$$\int \frac{1}{2\,(2x+2)}\,dx \;=\; \frac{\ln\,(4x+4)}{4} + C$$
$$\int \frac{9}{4\,(x-3)}\,dx \;=\; \frac{9\ln\,(4x-12)}{4} + C$$

Step 6: Sum the two parts together.

The result of Step 6 should match the results on page 555.

16.4. I call this exercise "the lemon and the infinite." It's unrelated to integration tricks, and is another attempt to give you an opportunity to improve your math-coding skills in an enjoyable way. It's also a great review of material from earlier in the book.

Start by implementing the following functions in `sympy`.

$$f(x) = 10\sin^5(\sin^3(\sin(x))) \tag{16.49}$$

$$g(x) = \cos^6(\cos^4(\cos^2(x))) \tag{16.50}$$

Obtain their derivatives using `sympy`. You should try to get the analytic indefinite integrals via `sympy`, but I think you will need to use numerical approximations using methods you learned about in the previous chapter.

Finally, show the results as in Figure 16.8.

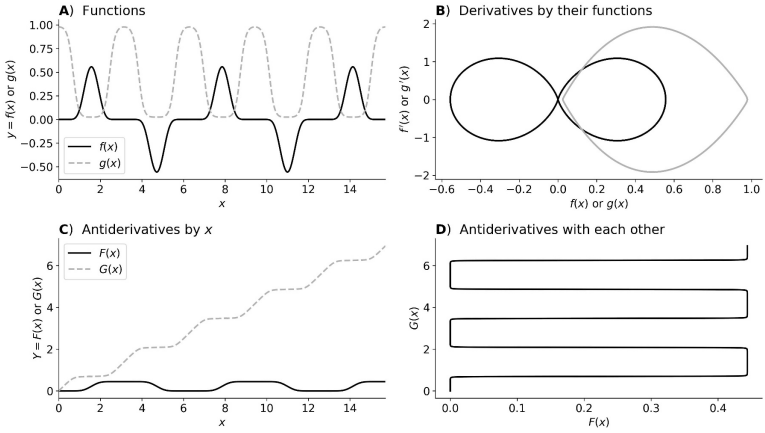


Figure 16.8: Results for Exercise 4.

Chapter 17

Integration applications in geometry

Solely from the chapter titles, you might think that this chapter is redundant with Chapter 13; indeed, both are about integration and geometry.

However, Chapter 13 was about using geometry to understand and define integrals, whereas this chapter is about using integrals to quantify geometric shapes. But there is some overlap between the chapters, and concepts like Riemann sums will be central to the topics in this chapter.

17.1 Area between two curves

In the previous several chapters, we've been quantifying the area between the curve defined by a function and the x-axis, which is the same thing as the line corresponding to $y = 0$.

But what if we want to quantify the area between some function and some other line — or another curve?

The idea is depicted in Figure 17.1: the area to calculate is defined between two functions on the y-axis for some interval on the x-axis.

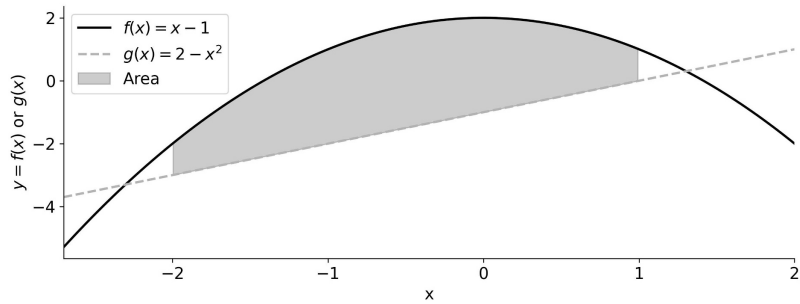


Figure 17.1: The area between two functions.

Conceptually, the area between $f(x)$ and $g(x)$ is straightforward: Calculate the area under each curve within the same interval, then subtract.

$$A_\Delta = \int_a^b f(x)\,dx \; - \; \int_a^b g(x)\,dx \tag{17.1}$$

Because of the linearity of integration, we can simplify Equation 17.1:

$$A_\Delta = \int_a^b f(x) - g(x)\,dx \tag{17.2}$$

And that's all there is to it! In fact, if you consider defining a new function as the difference of two functions, then there is really nothing new to learn here.

$$h(x) = f(x) - g(x) \tag{17.3}$$

$$A = \int_a^b h(x)\,dx \tag{17.4}$$

This means that you can think about calculating the area between two functions as equivalent to calculating the area between the difference of those functions and the x-axis. In other words, the areas indicated by the gray patches in panels A and B of Figure 17.2 are the same.

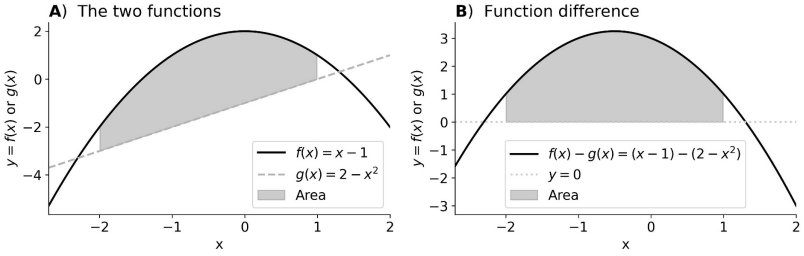


Figure 17.2: The area between two functions can also be expressed as the area between their difference and the x-axis.

Order of subtraction What would happen if we swapped the order of the functions? That is, what is the relationship between

$$\int f(x) - g(x)\,dx \quad \text{and} \quad \int g(x) - f(x)\,dx \; ?$$

The answer is that the resulting indefinite integral is the same except for a sign flip. And for a definite integral, the magnitude of the result is the same but the sign changes.

Whether to integrate $(f - g)$ or $(g - f)$ depends on the context of the application. That said, it is typical to order the subtraction such that the resulting definite integral is positive, which geometrically means that the function with smaller values is the subtracted function in the integrand. For example, the integral in Figure 17.2A expressed as $\int g(x) - f(x)\, dx$ would have a negative-signed area.

What intervals to use for integration? There are two ways to choose the bounds of integration. One way is to select the bounds based on predefined values of x, as in the graphical example in Figure 17.2. This is likely to be the case for real-world applications in which the interval bounds are chosen based on theoretical or experimental considerations.

Alternatively, the interval can be selected according to the intersection points where the two functions cross paths.

The next two subsections show examples of each of these approaches.

17.1.1 Area with predefined intervals

Example 1 Here are the two functions to compute the area between (shown in Figure 17.3).

$$f(x) = e^{-x^2} \tag{17.5}$$

$$g(x) = x^2 \tag{17.6}$$

$$\int_0^1 f(x) - g(x)\, dx = \int_0^1 e^{-x^2} - x^2\, dx \tag{17.7}$$

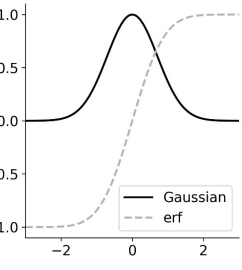


Figure 17.4: The Gaussian bell-curve function and its antiderivative.

The function e^{-x^2} is the famous Gaussian "bell-curve" that you might be familiar with (Figure 17.4). It plays a central role in

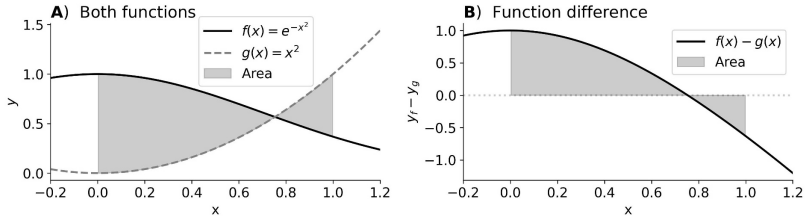


Figure 17.3: Example 1.

myriad areas of math, including statistics (next chapter). Curiously, this function does not have an antiderivative that can be expressed using elementary functions. Instead, the antiderivative is termed erf, which stands for "error function" and looks similar to a sigmoid function.[1]

Here is the antiderivative of Equation 17.7:

$$\int e^{-x^2} - x^2 \, dx = \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) - \frac{x^3}{3} \tag{17.8}$$

And here is the solution:

$$\int_0^1 e^{-x^2} - x^2 \, dx = \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) - \frac{x^3}{3}\Big|_0^1 \tag{17.9}$$

$$= \left(\frac{\sqrt{\pi}}{2} \operatorname{erf}(1) - \frac{1}{3}\right) - \left(\frac{\sqrt{\pi}}{2} \operatorname{erf}(0) - \frac{0}{3}\right) \tag{17.10}$$

$$\approx .4135 - 0 \tag{17.11}$$

The erf function can be numerically approximated in `sympy` using the function `sym.erf` (there are also erf-estimating functions in the `math` and `scipy` libraries).

Notice that the functions cross at around $x = .75$, which means their difference has both negative and positive values. The net definite integral is positive because there is more positive area

[1] It was termed "error function" for its applications in calculating the likelihood of errors, which was a topic of concern in the early days of probability theory development.

than negative area. In Exercise 1, you'll calculate the total area to be around .562, which means that the negative area (to the right in Figure 17.3B) is around −.1485.

Example 2 This example has more terms, but it's the same as the previous example in concept and implementation. Below are the functions to calculate area between, and the whole setup is visualized in Figure 17.5.

$$f_1(x) = \sin(10x) + \ln(x) - (x-1)^2 + 3 \tag{17.12}$$

$$f_2(x) = x^2 + \cos(8x) \tag{17.13}$$

$$\int_{.5}^{1.4} f_1(x) - f_2(x)\, dx = ? \tag{17.14}$$

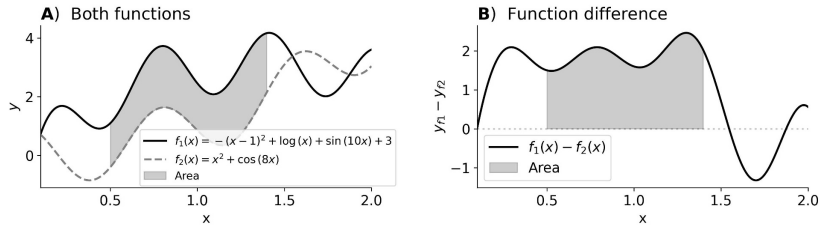


Figure 17.5: Example 2.

The indefinite integrals are

$$\int f_1(x)\, dx = x\log(x) + 2x - \frac{(x-1)^3}{3} - \frac{\cos(10x)}{10} \tag{17.15}$$

$$\int f_2(x)\, dx = \frac{x^3}{3} + \frac{\sin(8x)}{8} \tag{17.16}$$

The functions don't cross and the integral is setup such that $f_1(x) > f_2(x)$ in the integration interval, meaning that the net area equals the total area, and will be positive.

The calculus aspects of this example (setting up the problem and solving the definite integral) are not difficult, but it's best to let the computer do the arithmetic. The definite integral of f_1 is around 2.57, the definite integral of f_2 is around .84, and their difference — the shaded area in Figure 17.5 — is around 1.72.

17.1.2 Area defined by function intersections

Consider Figure 17.6, which shows the area between $x = 0$ and $x = 1$ for the following two functions:

$$f_1(x) = \sqrt{x} \tag{17.17}$$

$$f_2(x) = x \tag{17.18}$$

Why did I choose the interval $\{0, 1\}$? In the previous subsection, I chose the interval somewhat arbitrarily. However, in this subsection, the x-axis bounds are chosen based on intersections between the two functions.

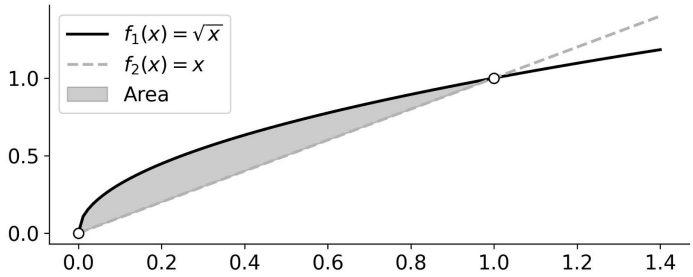


Figure 17.6: Calculating the area between two functions, in an interval defined by the functions' intersections.

The procedure to calculate the area is identical to what you learned previously, but we need an additional step prior to the integration: We need to solve for the intersection points to find the integration bounds. This is achieved by identifying the roots of the difference equation, which can be expressed using either of the following equations:

$$f_1(x) = f_2(x) \tag{17.19}$$

$$f_1(x) - f_2(x) = 0 \tag{17.20}$$

For our example functions, that's $\sqrt{x} = x$, and the only two solutions are $x = 0$ and $x = 1$. These are the integration bounds for defining the area between the two curves. Now we can write out the problem.

$$\int_0^1 \sqrt{x} - x \, dx \tag{17.21}$$

I encourage you to work through the integration on your own. You should get an area of 1/6.

17.2 Parametric curves

The functions you've worked with so far in this book involve defining a function as some transformation of variable x. That is, x is the independent variable and there is some rule (e.g., $y = \sqrt{|x - e^{\cos(x)}|}$) that transforms x into y.

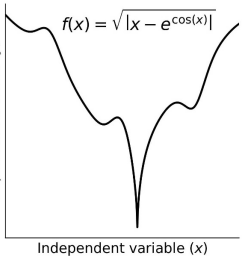


Figure 17.7: An interesting-looking function.

Parametric curves are an alternative way to define certain functions. Instead of setting $y = f(x)$, the two variables x and y can be defined independently of each other, based on transformations of a third variable t. Consider this example:

$$x = \cos(t^2) \tag{17.22}$$

$$y = \sin(t) \tag{17.23}$$

This is different from the "regular-looking" functions you've seen before: With parametric functions, both x and y are defined as a function of t. You can think of this as two separate functions, $x(t)$ and $y(t)$, that are linked together.

In fact, you've already worked with and plotted parametric curves several times in this book, although I haven't previously used this term. While reading the explanation below, see if you can think of where in the book you've come across parametric curves.

Figure 17.8 shows two example parametric curves in panels A and B. Notice that these two panels do not show t on any of the axes. Instead, t is implicitly represented as the trajectory of the curves.

Panel C, in contrast, shows two functions with t explicitly on the horizontal axis. In fact, panels B and C show the same functions. Please spend a moment seeing how the two plots show the same functions represented in different ways.

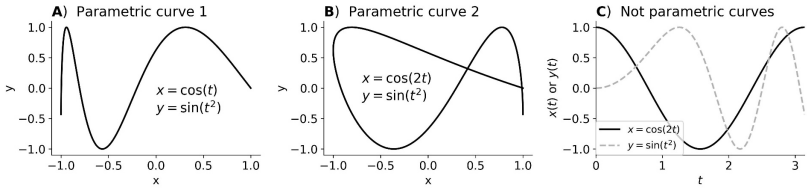


Figure 17.8: Two parametric curves with $t \in [0, \pi]$.

Here's a way to think about parametric curves: Imagine that the axis is a football field, the black line is a path, and there is a little person walking along that path. Variable t represents time, and the x and y values are the spatial locations that our little friend traverses. For a discretized t (that is, Δt instead of dt), you can think of Δt as the duration of each step. Perhaps $\Delta t = 1$ and we consider that this corresponds to one step each second.

The secret parametric curves Have you figured out where you've seen parametric curves before? I used them in several exercises to visualize a function by its derivative or integral. In those cases, I defined, for example,

$$x = f(t) \tag{17.24}$$

$$y = f'(t) \tag{17.25}$$

You also saw parametric curves in Chapter 11 (radial curves and rose curves).

17.2.1 Examples of parametric curves

Parametric curves can be elegant and geometric. Figure 17.9 shows some examples of curves defined by parametric functions.

Parametric curves are often defined in terms of trig functions. That's not a requirement, but does help ensure that the curves stay close to the origin, and produce nice cyclic patterns.

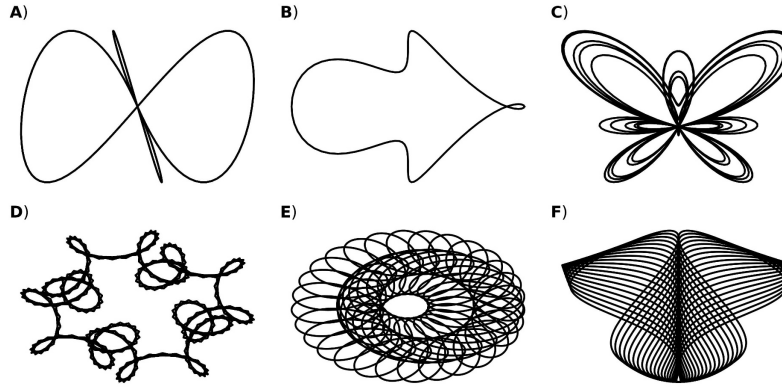


Figure 17.9: Some nice-looking parametric curves. You will create this figure in Exercise 6.

Where's the calculus? I haven't mentioned anything about calculus (differentiation or integration) yet. Parametric functions lead directly to calculating arc length, which requires calculus and which is the main topic of the next section.

17.2.2 Applications of parametric curves

There are many applications of parametric curves — some that you might use regularly but aren't aware that they are implemented as parametric functions.

If you use a vector graphics program like Adobe Illustrator or Inkscape, then you might be familiar with the Bézier curve tool, which allows you to draw smooth curves between points. Bézier parametric functions are also used in computer graphics and animations to produce smooth, realistic motion.

Computational sciences often use parametric curves to visualize and characterize dynamics of complex systems. For example, you might have come across "state-space" or "phase-plane" representations of data, which are based on the idea of visualizing multivariable dynamics in a small number of axes. Parametric curves are particularly useful for nonlinear dynamical systems like wind, traffic patterns, fluid dynamics, ocean currents, movement in electromagnetic fields, etc.

In data science and machine-learning, parametric curves can be used to quantify dimensionality-reduced data, low-dimensional embeddings, curve-fitting time series data, and manifold projections.

Path-planning in robotics is another application, where an algorithm will plan a movement trajectory in 3D space using parametric functions. Relatedly, some computer vision algorithms incorporate parametric curves to model contours and edges.

Some aspects of time series analysis including financial modeling are also based on parametric curves to predict future values, understand cyclical patterns, and interpolate missing data.

17.3 Arc (curve) length

Arc length is a way of quantifying the length of a parametric curve.

Here's the equation to calculate the length (L) of a curve.

$$L = \int_a^b \sqrt{1 + f'(x)^2}\, dx \tag{17.26}$$

Where does this equation come from, what does it mean, and why is there a "1" in there? That's what you're going to learn in the next subsection.

17.3.1 Deriving and understanding the formula

Let's take a step back and think about how we could approximate the length of a curve.

Consider the two curves in Figure 17.10A: How long are they?

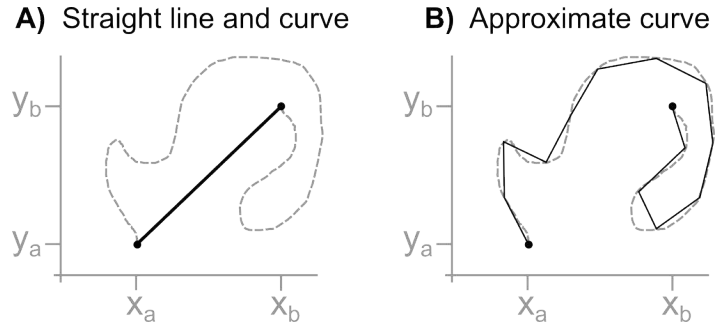


Figure 17.10: A straight line and a curve that share start and end points (panel A). We can use more straight lines to approximate the length of the curve (panel B).

The length of the straight line is calculated using Euclidean distance. In particular, the length of a line is defined as:

$$L = \sqrt{(x_b - x_a)^2 + (y_b - y_a)^2} \tag{17.27}$$

$$= \sqrt{(\Delta x)^2 + (\Delta y)^2} \tag{17.28}$$

How about the curve; how can we measure the length of that curve? Obviously, that curve is longer than the straight line, and so measuring the distance between the two end points will be an underestimate — potentially a horrible underestimate if the curve is very curvy.

Let's draw some inspiration from Riemann sums. Imagine that we approximate the curve by a series of straight lines (Figure 17.10B). We can measure each of those lines using Euclidean distance, and then sum the lengths.

$$L \approx \sum_{i=2}^{n} \sqrt{(\Delta x_i)^2 + (\Delta y_i)^2} \tag{17.29}$$

Each line segment is defined between two points along the curve, and n is the number of points. Thus, having n points along the curve will produce $n - 1$ line segments.

Δx_i refers to the difference in x-axis values between x_i and x_{i-1},

and Δy_i refers to $(y_i - y_{i-1})$. Hence, i starts at 2 instead of 1 in the summation.

You can alternatively write n as the number of line segments (and thus starting the counter at $i = 1$), although defining n as the number of points provides a more direct link to the Python code implementation.

How many line segments should we use and how good will the approximation be? I'm sure you have guessed where this is going: we increase the number of straight line segments to ∞, which means that $\Delta x \to dx$ and $\Delta y \to dy$.

But before we get to integrals and limits, I want to rewrite Equation 17.29.

$$L \approx \sum_{i=2}^{n} \sqrt{(\Delta x_i)^2 + (\Delta y_i)^2} \tag{17.29}$$

$$\approx \sum_{i=2}^{n} \sqrt{(\Delta x_i)^2 + \frac{(\Delta x_i)^2}{(\Delta x_i)^2}(\Delta y_i)^2} \tag{17.30}$$

$$\approx \sum_{i=2}^{n} \sqrt{\left[1 + \frac{1}{(\Delta x_i)^2}(\Delta y_i)^2\right](\Delta x_i)^2} \tag{17.31}$$

$$\approx \sum_{i=2}^{n} \sqrt{1 + \left(\frac{\Delta y_i}{\Delta x_i}\right)^2}\,\Delta x_i \tag{17.32}$$

Notice how multiplying the Δy term by 1 (expressed as $\Delta x/\Delta x$) in Equation 17.30 allowed us to factor out a Δx term to outside the square root. That's where the "1" comes from.

And how about the $\Delta y/\Delta x$; does that look familiar? Of course it does: it's the definition of the slope of a line, and equals the derivative as the Δ's get tiny.

Note: In principle, square roots have sign indeterminacies, but length is a non-negative quantity so we only consider the positive square roots.

Now you're ready to see the formula for the exact length of a

curve.

$$L = \lim_{n\to\infty} \sum_{i=2}^{n} \sqrt{1 + \left(\frac{\Delta y_i}{\Delta x_i}\right)^2}\, \Delta x_i \tag{17.33}$$

$$= \int_a^b \sqrt{1 + \left(\frac{dy}{dx}\right)^2}\, dx \tag{17.34}$$

$$= \int_a^b \sqrt{1 + \left(f'(x)\right)^2}\, dx \tag{17.35}$$

A curve defined by two functions is called a *planar curve*. Functions in three dimensions are called *space curves*.

Where a and b are the start and end points of the curve segment.

I hope Equation 17.35 now makes sense: Basically, to measure the length of a curve, you cut up the curve into tiny straight line segments, measure the lengths of each of those segments, and sum all the lengths.

Because arc lengths are calculated between two values, the constant of integration is unnecessary.

17.4 Numerical arc length approximations

Deriving analytic solutions to Equation 17.35 is notoriously difficult for all but the simplest, hand-crafted functions. The integrand will always have a square root containing squared terms, and there are often divisions and other terms that are tricky to integrate. Even for simple curve definitions, the integration can get really hairy, really quickly.

Therefore, in practice — certainly in modern applications — curve lengths are numerically approximated. In this section, I will introduce you to several approximation techniques using `numpy` and `scipy`.

An attempt in `sympy` First I want to show you what happened when I tried to calculate a curve length analytically using `sympy`.

I'll use an example from earlier, with $x = \cos(2t)$, $y = \sin(t^2)$, and $t \in [0, 4\pi/5]$ (redrawn in Figure 17.11).

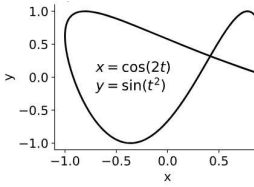


Figure 17.11: How long is this curve?

`sympy` has a module called `Curve` that you can use to specify, manipulate, and characterize curves.

```
t = sym.symbols('t')
C = sym.Curve((sym.cos(2*t), sym.sin(t**2)), (t,0,4*sym.pi/5))
C.length
```

The method `.length` calculates the length of the specified curve, although it was unable to produce a result, and instead just continued to occupy the processor.

Next I tried building the individual components of Equation 17.35, which gives the following antiderivative.

$$\int_0^{\frac{4\pi}{5}} \left(-2\sqrt{1 + \frac{\sin^2(2t)}{t^2 \cos^2(t^2)}} \sin(2t) \right) dt \tag{17.36}$$

`sym.integrate` could not find an analytic solution to the problem, and after around 10 minutes of calculations, Google-colab produced an error message: "Your session crashed after using all available RAM."

In summary, even a curve defined by elementary trig functions was sufficiently difficult to integrate analytically that it overwhelmed the Python session.

In fairness, `sympy` is an actively and professionally developed library with continual improvements. It is possible that future versions will be able to handle this integration problem.

And to be clear: `sympy`'s failure to solve this problem is in no way a critique of `sympy`. Instead, it highlights how intractable curve length calculations can be. Calculating arc lengths by hand

can be useful for practicing advanced integration methods, but in the age of modern computing, understanding how to approximate curve lengths with high accuracy has more practical benefit.

17.4.1 Direct implementation in `numpy`

The goal of this subsection is to implement Equation 17.29 (reprinted below) in `numpy`, which will allow you to see understand the mechanisms of curve length approximations and visualize the intermediate results.

$$L \approx \sum_{i=2}^{n} \sqrt{(\Delta x_i)^2 + (\Delta y_i)^2} \tag{17.29}$$

First we define x, y, and t.

```
t = np.linspace(0,4*np.pi/5,123)
x = np.cos(2*t)
y = np.sin(t**2)
```

The code below is a direct implementation of the approximation formula. Notice that I store each individual segment length in an array, which allows visual inspection.

```
segLengths = np.zeros(len(t))

for i in range(1,len(t)):
  # calculate deltas
  dx = x[i] - x[i-1]
  dy = y[i] - y[i-1]

  # this little curve segment
  segLengths[i] = np.sqrt( dx**2 + dy**2 )
```

I used a for-loop to help you understand that the Δ and segment-length calculations are based on each successive pair of sample

points along the curve. But the loop is not necessary for coding. The lines below produce the same result, although the array `segLengths` will be one element smaller compared to the for-loop implementation due to the initialization. That's why I prepended a zero.

```
dx = x[1:] - x[:-1]
dy = y[1:] - y[:-1]
segLengths = np.append(0,np.sqrt( dx**2 + dy**2 ))
```

Several results are shown in Figure 17.12. Panel A shows the curve, with the color mapping onto values of t such that lighter circles correspond to larger values of t (which you can think of as being "later in time") (see also the four text annotations). As I wrote earlier, a parametric curve has no axis for the independent variable; it is represented implicitly: Imagine a little person hiking from the darker to the lighter part of the path, and each dot represents a step.

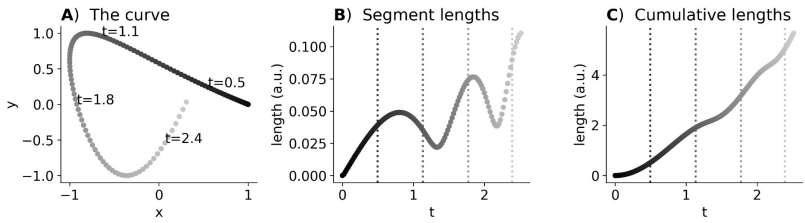


Figure 17.12: Calculating curve length in `numpy`. ("a.u." stands for "arbitrary units" and is often used in scientific graphs when the axis values do not directly correspond to physical observables.)

Panel B shows the individual segment lengths, corresponding to the square root term for each individual segment. The segment lengths are all positive, which makes sense because lengths cannot be negative. Some of the lengths are longer while others are shorter. Of course, Δt is, by definition, the same for each step, but the interaction between $x(t)$ and $y(t)$ cause some steps to be further apart while others are closer together. Notice that the segment lengths are longer when the curve is relatively straight, and shorter when the curve is curving. The reason for the overall

increase in path length is the t^2 in the sine function. The vertical dashes in panel B indicate the locations on the t-axis corresponding to the marked values in panel A.

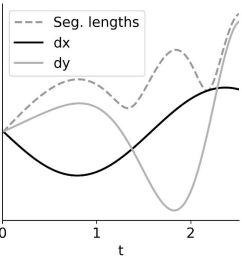


Figure 17.13: Segment lengths are defined from Δx and Δy; all are visualized here.

Finally, panel C is the cumulative sum of the segment lengths. The cumulative sum of lengths monotonically increases (it can go up or stay the same, but can never go down), and represents the total distance traversed from the start of the path until each value of t.

The total curve length is the sum over all segment lengths, which is the same as the final value of the cumulative lengths. That's around 5.65; I'll show more exact numbers below.

You've seen previously in this book that errors in numerical approximations can arise if the function is too coarsely discretized; in other words, if n in the approximation formula is too small. At the extreme, that's obviously true: If $n = 2$, then the curve length approximation is the distance between the end points. I checked four values for n and got the following results:

```
Total curve length for n =    10: 5.51108755297420
Total curve length for n =   100: 5.65598002379908
Total curve length for n =   500: 5.65722746701867
Total curve length for n = 10000: 5.65727846070746
```

To be clear, these results do not mean that $n = 100$ is sufficient for any curve length you want to approximate, but they do show that the approximation algorithm can be quite stable even for a small number of segments.

In real-world applications, you may be able to explore the impact of Δt on curve length — but if you're calculating the lengths of empirically obtained data curves, then Δt is given by the experimental data and might be outside your control.

17.4.2 Approximations using `scipy`

Implementing calculus concepts directly in `numpy` is a fantastic way to learn and to visualize intermediate steps in multi-step algorithms. But in practice it's better to use more sophisticated numerical techniques that can be accessed in the `scipy` library.

There are two functions in the `integrate` library suitable for curve integration: `spi.quad` (used when you have the mathematical equations and their derivatives) and `spi.simpson` (used for numerical arrays such as empirical data where you don't have the generating mathematical equations). I'll introduce both methods in Examples 4a and 4b (`spi.quad`) and Example 4c (`spi.simpson`).

Using `spi.quad` with known derivatives Use the `spi.quad` function when you have an analytic expression for the functions and their derivatives. If you already have mathematical expressions for the derivatives, write them as Python functions, as shown below.

```
def dx_dt(t): return -2*np.sin(2*t)
def dy_dt(t): return 2*t * np.cos(t**2)
def integrand(t): return np.sqrt( dx_dt(t)**2 + dy_dt(t)**2 )
```

Next, use `spi.quad` to evaluate the integrand.

```
a,b = 0, 4*np.pi/5
L,_ = spi.quad(integrand,a,b)
```

Notice that I haven't actually defined a vector `t`. As you might remember from Chapter 14, `spi.quad` takes *Python functions* as inputs, not numerical arrays. This is advantageous because `spi.quad` will determine a suitable t-axis sampling based on the properties of the function.

The final estimate of length is:

```
Arc length using spi.quad:  5.657278588025542
```

That result is extremely similar to our `numpy` calculations in the previous example. That shouldn't be surprising: The functions that define this curve are continuous, differentiable, and smooth. You'll see some more "difficult" examples in the exercises.

Using `spi.quad` with `sympy` Perhaps you know the functions that define the curve ($x(t)$ and $y(t)$), but you don't have their derivatives already calculated. In this case, you would start by defining the functions, having `sympy` calculate their derivatives, and then defining the integrand in `sympy` (I'm using the variable `p` for "path" to avoid overwriting `t`):

```
p  = sym.symbols('p')

# the functions
xp = sym.cos(2*p)
yp = sym.sin(p**2)

# their derivatives
dxp = sym.diff(xp)
dyp = sym.diff(yp)

# the integrand
integrand = sym.lambdify(p, sym.sqrt(dxp**2 + dyp**2) )
```

Notice that the functions and their derivatives are symbolic expressions, while the `integrand` is lambdified into a Python function. `spi.quad` takes a function as input, but we only input the integrand, not the symbolic expressions that formed the integrand. From here, the call the `spi.quad` is the same as I showed previously.

```
a,b = 0, 4*np.pi/5
L,_ = spi.quad(integrand,a,b)
```

The output is

```
Arc length using spi.quad:  5.657278588025542
```

Using `spi.simpson` with numerical arrays A third way to use `scipy` to integrate curves is to evaluate them numerically without any symbolic expressions or known derivative functions. You need only vectors of numbers with which to calculate an empirical derivative.

In the code below, I use `np.gradient` to calculate the derivative of the functions.

```
a,b = 0, 4*np.pi/5
t = np.linspace(a,b,500)

# empirical derivatives
dx_dt = np.gradient( np.cos(2*t) , t)
dy_dt = np.gradient( np.sin(t**2), t)
```

Then specify the integrand based on the derivatives, and call `spi.simpson` to calculate the arc length.

```
integrand = np.sqrt(dx_dt**2 + dy_dt**2)
L = spi.simpson(integrand,x=t)
```

This code produced the result

```
Arc length using spi.simpson:  5.657073423218569
```

In case you're curious, here are the results for the four approaches above.

```
numpy (n=500)  : 5.65722746701867
quad from diffs: 5.657278588025542
quad with sympy: 5.657278588025542
simpson (n=500): 5.657073423218569
```

`quad` vs. `simpson` Use `spi.quad` when possible, and `spi.simpson` when necessary.

`spi.quad` integrates from analytically expressed derivatives, and — most importantly — it dynamically adjusts the numerical evaluations according to the function characteristics (for example, more curvy parts of the curve require more sample points than straigher parts of the curve), which means it can achieve higher accuracy with fewer numerical calculations. Indeed, we didn't specify any resolution or grid for t when calling `spi.quad`.

`spi.simpson`, on the other hand, is based on numerical values (not analytic functions). It is likely to be more accurate than the `numpy` implementation (and requires less code), and is what you'd use for empirical data or functions that are already numerically approximated.

17.5 Volumes of solids of revolution

"Volume" is a measure of how much space an object occupies. In day-to-day life, we often think about volume in terms of the size of drinks (e.g., single or double espresso) or whether we can take our tube of toothpaste into the airplane.

You don't need calculus to calculate the volume of regular shapes like cubes and pyramids. For example, the volume of a cube is its length times its height times its width. Calculus would just be overkill.

But calculus is necessary for shapes with *curvature* that is defined by, or can be characterized using, mathematical functions.

The volume of real-world objects is measured empirically, for example by submerging that object into a tank to measure the increase in water level. But the volume of solids defined by equations can be calculated analytically using integration, and that's what you will learn about in this section.

Many manufactured products are created from mathematical equations, so the methods

Examples of solids The solids you will work with in this section are created by taking a curve defined by a function, and revolving it around the x-axis. For example, the curve in Figure 17.14A is the function $f(x) = \sqrt{x}$. Imagine taking that curve and revolving it (spinning it) around the x-axis so it creates the volume shown in Figure 17.14B.[2]

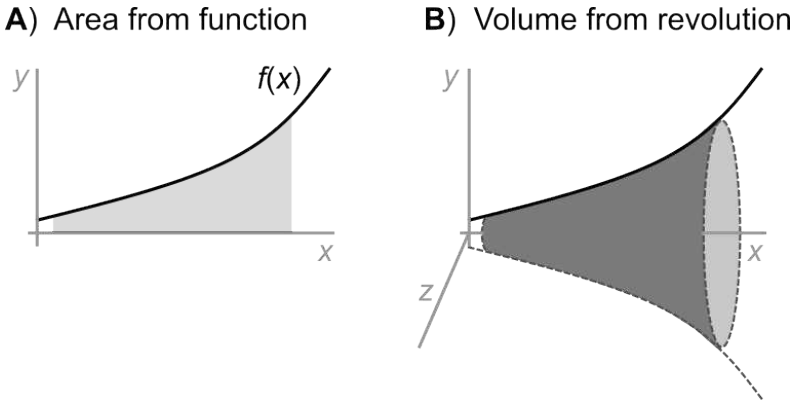


Figure 17.14: A function of one variable (panel A) is revolved around the x-axis to create a volume (panel B).

These types of objects are called "solids of revolution" because they're created by revolving a 2D shape around the x-axis.[3] Notice that the 2D area preserves its shape as it revolves around the axis, and therefore the distance of each point in the 3D surface from the x-axis is defined by $f(x)$, and is fixed as it goes around. In other words, each x-axis "slice" of the solid is a perfect circle with radius $f(x_0)$. That's key to calculating the volume of the solid, as I will explain later.

Figure 17.15 shows three examples of single-variable functions (left column) that are revolved around the x-axis to produce solids (right column). The first two rows show solids created by rotating one function, whereas the third row shows a solid created by revolving the area between two functions.

[2] Does the 2D area *rotate* or *revolve* around the axis? Both terms are correct and are used. But *revolution* connotes a full 360° turn around the axis whereas *rotation* could indicate only a partial turn. Therefore, *revolve* is often preferred.

[3] When creating a 3D solid of revolution, the revolving function is considered "2D" because it exists in a 2D plane. Don't confuse this with a two-variable function $Z = f(x, y)$. Revolving a two-variable function would create a "hyper-solid" in 4D.

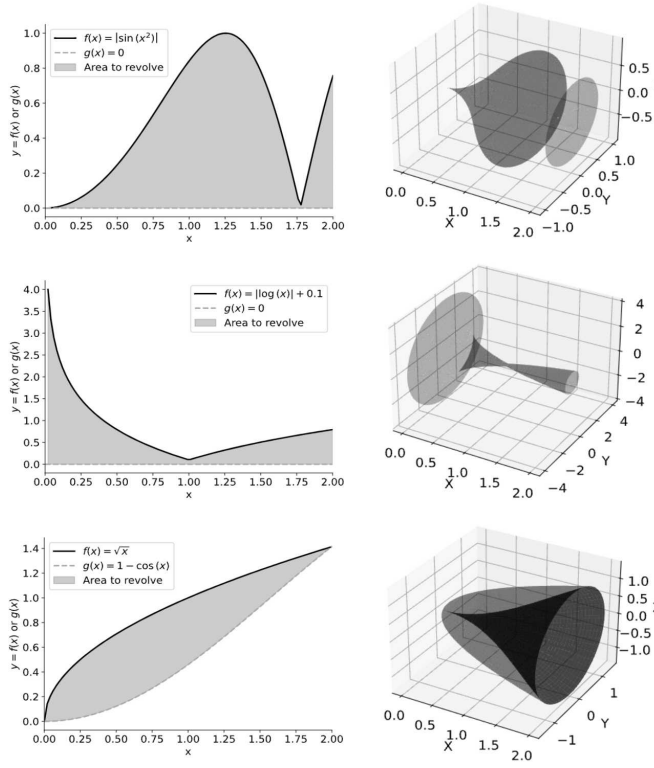


Figure 17.15: Three examples of 2D areas (left column) that form solids when revolved around the x-axis (right column).

Positive-only functions Volume is a non-negative quantity, so it's awkward to create a volume by rotating a function with negative values. There are ways around this, for example by taking the absolute value of the function, clipping negative values to zero, or adding a constant to the function to shift its values to be non-negative in the revolving domain.

If you're going to calculate the volume by hand or with `sympy`, it's best to choose simple functions that have all positive values in the range in which the volume is created. But if you will numerically approximate the volume, then you won't be constrained by easily integrable functions, and you can let your imagination run wild.

17.5.1 Visualizing volumes

There are four steps to creating 3D surfaces in `matplotlib` based on revolving a 2D surface around the x-axis.

The first step is to define the math functions as Python functions. That can be done by lambdifying `sympy` objects or as regular Python functions using `def`. Which format to use depends on the subsequent analyses, e.g., whether the functions can be analytically solved by `sympy` or estimated numerically using `scipy`. Here's an example of setting up code to calculate the volume using `sympy`.

```
x  = sym.symbols('x')
fx = sym.sqrt(x)
f  = sym.lambdify(x,fx)
```

The second step is to define 2D grids on which to evaluate the function surface. One grid corresponds to the x-axis between the bounds of integration (hard-coded below to 0 to 2), and the other grid will always range between 0 and 2π corresponding to one full revolution around the x-axis.

```
xx = np.linspace(0,2,100)
X,Theta = np.meshgrid(xx,np.linspace(0,2*np.pi,100))
```

The third step is to map the 2D function values onto surfaces. The general formula for this is to scale the function values by cosine and sine to map the function onto a circle around the x-axis.

$$Y(x) = f(x)\cos(\theta) \tag{17.37}$$

$$Z(x) = f(x)\sin(\theta) \tag{17.38}$$

Where Y and Z form the y- and z-axis coordinates. In Python code, it looks like this:

```
Y_f = f(X) * np.cos(Theta)
Z_f = f(X) * np.sin(Theta)
```

That's the end of the math. The fourth step is to visualize the surface[4]. The code below creates a surface using `matplotlib`.

```
fig = plt.figure(figsize=(6,5))
ax  = fig.add_subplot(111,projection='3d')
ax.plot_surface(X, Y_f, Z_f, color='k', alpha=.6)
```

The code above produces Figure 17.16.

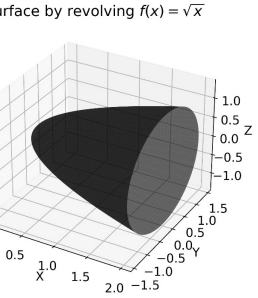


Figure 17.16: A surface that indicates a solid.

Solid volumes need only one function (in this case, $f(x) = \sqrt{x}$) while volumes with a cut-out need two functions. The code for generating a surface defined by two functions can be implemented simply by doubling the code for a second function $g(x)$ and drawing that function into the same axis with a different color or transparency.

Using `plotly` In Chapter 10, you learned how to use the `plotly` library to make interactive visualizations of 3D surfaces. That's also a good way to visualize volumes of revolution. However, it requires more code and offers less advantage for the printed page, so I decided to use the simpler `matplotlib` approach for the 3D surfaces in this chapter. The online code, however, has a code cell with sample `plotly` code that you can adapt to other functions.

17.5.2 The disk method

There are a few ways to set up the equation to calculate volume. In this subsection, I'll introduce the "disk method," and in the next subsection, I'll introduce the "washer method." (Spoiler alert: They're actually the same method with minor differences.)

[4]Note that the variables `Y_f` and `Z_f` repeat the start and end points of θ, because $\sin(0) = \sin(2\pi)$. The redundant point helps make the visualization look complete.

The idea of calculating the volume of a solid made by revolution is to slice that solid into disk-shaped cross-sections (Figure 17.17). As is often the case in integration, you can think about Riemann sums, although here we're not using rectangles, but "slabs" cut through the solid.

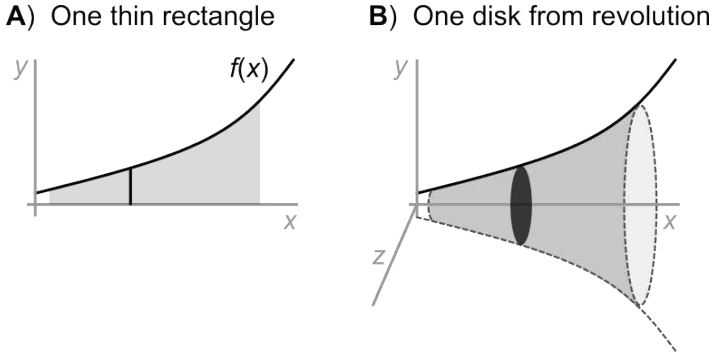


Figure 17.17: Illustration of the disk method. One thin rectangle in the function graph (panel A) gets revolved around the x-axis to create a disk (panel B) with thickness Δx and radius $f(x)$. The sum of all disks between the integration intervals is the approximation of the volume of the solid, which equals the exact integral as $\Delta x \to 0$.

An analogy: Imagine you want to know the volume of an irregularly-shaped loaf of bread. You cut the bread into slices, measure the volume of each slice, then sum all the volumes. Now cut thinner and thinner slices of bread until the thickness of each slice approaches zero[5].

Anyway, if the function is revolved around the x-axis, then it has the same radius for each x value — the value of the function $f(x)$. The curvature of the solid varies as a function of x, but the slice at any one value x_i is a perfect circle with $x = 0$ at the origin. The area of a circle is πr^2, and therefore, the area of each slice is $\pi f(x)^2$.

But areas are for 2D shapes; we are dealing with 3D slices. The volume of one 3D slice is the area of the 2D shape multiplied by the width of the slice, which is the distance between two successive

[5] I apologize to my German readers, who are probably deeply offended at the idea of cutting a loaf of bread into infinitesimal slices. Don't worry; it's just a thought experiment. Do not try this at home.

slice bounds (x_i and x_{i-1}). Let's call this Δx.

If the width is large relative to the changes in the function, we would need to decide whether to measure the distance from the x-axis as $f(x_i)$ or $f(x_{i-1})$. This is the volumetric version of the left/right/midpoint rules in Riemann sums. For convenience, I will write out the equations below using the right rule, but all rules will converge to the analytic result in the transition from sums to integrals.

$$V_i = \pi f(x_i)^2 \Delta x \tag{17.39}$$

And therefore the volume of the entire surface can be approximated by summing the volumes of all the slices.

$$V \approx \sum_{i=2}^{n} \pi f(x_i)^2 \Delta x \tag{17.40}$$

And you know what happens next: We take the limit as the number of slices goes to ∞, which means Δx becomes dx.

The π is a constant and can be pulled out of the integrand.

$$V = \pi \int_a^b f(x)^2 \, dx \tag{17.41}$$

Example 1 The first example is the curve $f(x) = \sqrt{x}$, shown in Figure 17.18A. When revolved around the x-axis between $x = 0$ and $x = 2$, it creates a bullhorn-like shape. `sympy` calculated the volume of that bullhorn to be $2\pi \ au^3$ (the units are meaningless in this example, so I labeled the volume au for arbitrary units). The code below implements Equation 17.41.

```
x  = sym.symbols('x')
fx = sym.sqrt(x)
a,b = 0,2
volume = sym.pi * sym.integrate(fx**2,(x,a,b))
```

Let's confirm `sympy`'s calculation:

$$V = \pi \int_0^2 \left(\sqrt{x}\right)^2 \, dx \tag{17.42}$$

$\sqrt{x} = x^{1/2}$

$$= \pi \int_0^2 x \, dx \tag{17.43}$$

$$= \pi \left. \frac{x^2}{2} \right|_0^2 \tag{17.44}$$

$$= \pi \left(\frac{2^2}{2} - \frac{0}{2} \right) \tag{17.45}$$

$$= 2\pi \tag{17.46}$$

It turns out that in this example, the volume is proportional to the square of the upper bound. For example, when $b = 1$, $V = \pi/2$. In fact, if the lower bound is zero, then the volume can be directly calculated from the upper bound: $V = \pi b^2/2$. That's just how this particular function works out; it's not a general principle of calculating volumes from revolutions.

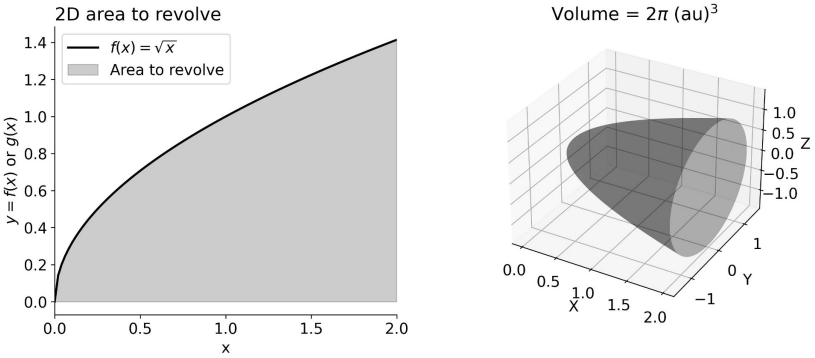


Figure 17.18: Dimensionality expansion: A 3D solid from a 2D function.

Example 2 The next example is created by revolving $f(x) = |\cos(x)|$ between $x = 0$ and $x = \pi$ (Figure 17.19). The volume of this solid is obtained by solving the following equation:

$$V = \pi \int_0^\pi |\cos(x)|^2 \, dx$$

This initially looks like a difficult integration problem because of the three-level composite function (x is inside cosine, which is inside the absolute value, which is inside the square).

However, squaring leads to a positive-valued number, so in fact we can ignore the absolute value (*note:* you can ignore the absolute value only during integration, not in the original function). That leads to an integration problem that can be solved using u-substitution:

$$V = \pi \int_0^{\pi} \cos^2(x)\, dx \tag{17.47}$$

$$= \pi \left[\frac{x}{2} + \frac{\sin(x)\cos(x)}{2} \Big|_0^{\pi} \right] \tag{17.48}$$

$$= \frac{\pi}{2} \Big[(\pi + \sin(\pi)\cos(\pi)) - (0 + \sin(0)\cos(0)) \Big] \tag{17.49}$$

$$= \frac{\pi}{2} \Big[(\pi + 0) - (0 + 0) \Big] \tag{17.50}$$

$$= \frac{\pi^2}{2} \tag{17.51}$$

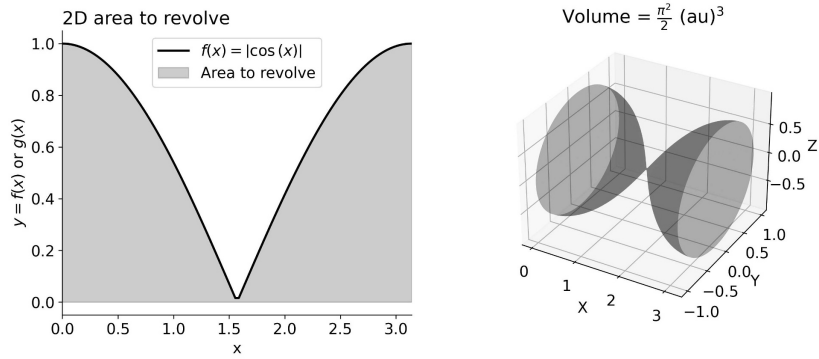


Figure 17.19: An hourglass-shaped solid created by revolving a trig function around the x-axis.

Now that you have the code to make the figure and calculate the volume, you can explore the impact of modifications to the function. For example, try revolving the following functions:

$$f_1(x) = |\cos(x)| + .1 \tag{17.52}$$

$$f_2(x) = |\cos(x)| \left(1 - \frac{x}{\pi}\right) \tag{17.53}$$

17.5.3 The washer method

The disk method works when the revolved area is between the function and the x-axis. But solids of revolution can also be defined by revolving the area between two functions around the x-axis. That difference is like a cut-out that creates a solid with some volume removed.

To measure the shape of a solid with a cut-out, use the washer method. Where does that name come from? You might be familiar with a "washer" as the small metal ring that helps keep screws tightly fastened (Figure 17.20). Perhaps the term "doughnut method" never caught on because no one wanted to think about doughnuts of near-zero thickness.

Figure 17.20: Two examples of washers (the two thin circular rings to the right) on a screw. Picture cropped from https://en.wikipedi wiki/Washer_(hard

The formula for the washer method is a combination of the disk method for revolving one function, and the area function for the difference between two functions. Indeed, think about the radius of the disk as the value of one function ($g(x)$) being "cut out" (literally subtracted) from another function ($f(x)$).

$$V = \pi \int_a^b f(x)^2 - g(x)^2 \, dx \tag{17.54}$$

$f(x)$ is the "outer" function (with larger values) and $g(x)$ is the "inner" function (smaller values). In other words, the order of functions in the integrand is chosen such that $f(x) > g(x)$ in the interval between a and b. If $f(x) < g(x)$ then the volume would have a negative sign, which is mathematically possible but physically awkward to interpret.

Another way to think about Equation 17.54 is to imagine two distinct solids created by two distinct functions, and then subtracting their volumes. The linearity of integration allows us to combine those two separate integrals into one (this is valid if the integration bounds are the same).

$$V_f = \pi \int_a^b f(x)^2 \, dx \tag{17.55}$$

$$V_g = \pi \int_a^b g(x)^2 \, dx \tag{17.56}$$

$$V_{f-g} = \pi \int_a^b f(x)^2 \, dx - \pi \int_a^b g(x)^2 \, dx \tag{17.57}$$

$$V = \pi \int_a^b f(x)^2 - g(x)^2 \, dx \tag{17.58}$$

Example The functions and volume is visualized in Figure 17.21, along with the two equations in the figure legend. The title shows the volume $(3\pi/10)$; I would like you to confirm that result on your own before looking at my solution in the equations below.

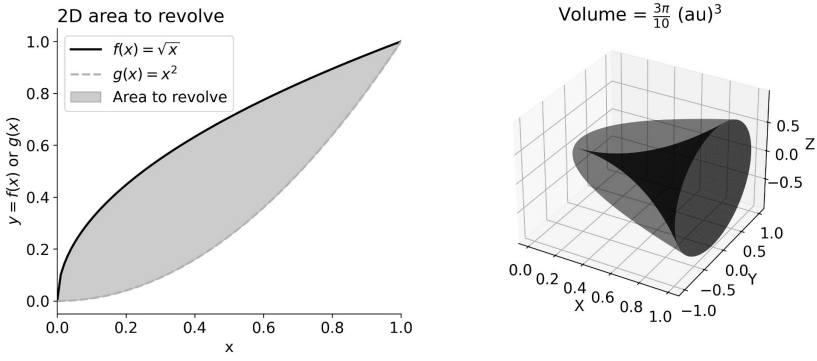


Figure 17.21: A solid created from revolving the area between two functions.

$$V = \pi \int_0^1 (x^2)^2 - \left(\sqrt{x}\right)^2 \, dx \tag{17.59}$$

$$V = \pi \int_0^1 x^4 - x \, dx \tag{17.60}$$

$$= \pi \left[\frac{x^5}{5} - \frac{x}{2} \Big|_0^1 \right] \tag{17.61}$$

$$= \pi \left[\left(\frac{1}{5} - \frac{1}{2} \right) - 0 \right] \tag{17.62}$$

$$= \pi \left(\frac{2}{10} - \frac{5}{10} \right) \tag{17.63}$$

$$= \frac{-3\pi}{10} \tag{17.64}$$

What do you think of my answer? It's a negative volume... you can see that my "mistake" was to subtract the larger equation from the smaller. As I wrote earlier, that is a perfectly acceptable *mathematical* answer, but it doesn't make sense given that we are calculating *volumes*, which are non-negative.

Fortunately, other than the negative sign, the calculations are accurate, so this mistake is easy to catch and fix: integrate $x - x^4$.

Example 2 The next example is a solid created by revolving the difference between the following two equations between $x = 0$ and $x = \pi/4$. Again, please take a moment to work through this problem and confirm `sympy`'s answer in the figure.

$$f(x) = e^{-x} \tag{17.65}$$

$$g(x) = \sin(x)/2 \tag{17.66}$$

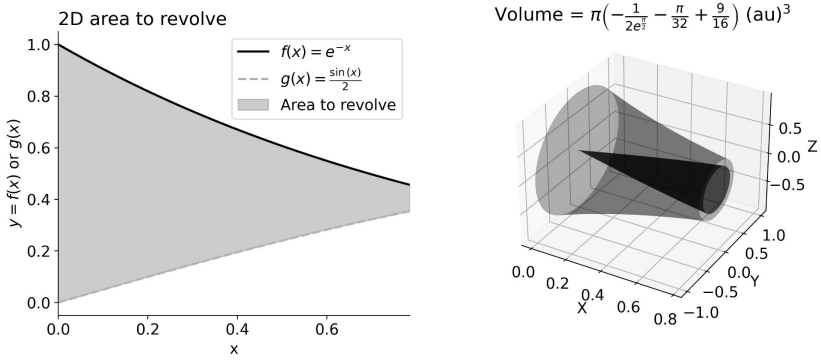


Figure 17.22: A solid created from revolving the area between two functions.

The integration is a bit more involved than the previous examples, but I'm sure you'll get the correct answer if you stay focused.

$$V = \pi \int_0^{\pi/4} \left(e^{-x}\right)^2 - \left(\frac{\sin(x)}{2}\right)^2 dx \tag{17.67}$$

$$= \pi \int_0^{\pi/4} e^{-2x} - \frac{\sin^2(x)}{4} dx \tag{17.68}$$

$$= \pi \left[\frac{e^{-2x}}{-2} - \left(\frac{x}{8} - \frac{\sin(x)\cos(x)}{8} \right) \Bigg|_0^{\pi/4} \right] \tag{17.69}$$

$$= \pi \left[\frac{e^{-2x}}{-2} - \frac{x}{8} + \frac{\sin(x)\cos(x)}{8} \Bigg|_0^{\pi/4} \right] \tag{17.70}$$

$$= \pi \left[\left(\frac{e^{-\pi/2}}{-2} - \frac{\frac{\pi}{4}}{8} + \frac{\sin(\frac{\pi}{4})\cos(\frac{\pi}{4})}{8} \right) - \left(\frac{e^0}{-2} - \frac{0}{8} + \frac{\sin(0)\cos(0)}{8} \right) \right] \tag{17.71}$$

$$= \pi \left[\frac{1}{-2e^{\pi/2}} - \frac{\pi}{32} + \frac{1}{16} - \frac{8}{-16} \right] \tag{17.72}$$

$$= \pi \left[\frac{1}{-2e^{\pi/2}} - \frac{\pi}{32} + \frac{9}{16} \right] \tag{17.73}$$

Disk vs. washer methods As you may have surmised, the washer and the disk method are the same, if you consider that the disk method *is* the washer method but setting $g(x) = 0$.

In fact, volume-calculation formulas can be generalized to Equation 17.74, where A is the area of a slice.

$$V = \int_a^b A \, dx \tag{17.74}$$

17.5.4 Numerical approximations

Example 1 In this first example, I will show that `scipy` can reproduce the analytic calculation that we and `sympy` obtained in Example 2 above (Figure 17.22).

Rather than defining the functions as symbolic expressions, I will use lambda expressions with `numpy` functions. A lambda expression is not necessary for the numerical approximation, but it is useful for visualization.

```
a,b = 0,sym.pi/4
xx = np.linspace(a,b,500)
f = lambda u: np.exp(-u)
g = lambda u: np.sin(u)/2
```

Next, approximate the area by implementing the volume formula. You can use `spi.trapezoid` or `spi.simpson`.

```
dx = xx[1]-xx[0]
volume = np.pi * spi.simpson(f(xx)-g(xx),dx=dx)
```

Here is the comparison of the volume from `sympy` and `scipy`:

```
Volume from sympy: 1.13218425516276
Volume from scipy: 1.24914518865331
```

Hmm... divergences can arise due to computer precision errors, but that's too big a discrepancy to be a rounding error. Can you spot the bug in my code?

The bug is that I forgot to square the functions. That is, the code above integrates $[f(x) - g(x)]$ instead of $[f(x)^2 - g(x)^2]$.[6] The corrected code is

```
volume = np.pi * spi.simpson(f(xx)**2-g(xx)**2,dx=dx)
```

And with this correction, the volumes from the two libraries, and the different algorithms in `scipy`, match nearly perfectly:

[6] This is another situation where it was a mistake in the code that I discovered when checking my results, and then decided to include in the book. Yet another illustration of the importance of rigorously checking your code!

```
Volume from sympy:          1.13218425516276
Volume from spi.simpson:    1.1321842551630092
Volume from spi.trapezoid: 1.1321851204921054
```

In the above numerical approximations, I used $n = 500$ points between $x = 0$ and $x = \pi/4$. Is that a large enough sampling? I tried various values, and found that even with $n = 3$ the resulting volume approximation was quite accurate. That's not surprising in this case because the functions, though technically nonlinear, are nearly linear in the integration interval (as you see in Figure 17.22A). That's just a demonstration here; using so few sample points along a function is not recommended for applications.

Example 2 Now that we've confirmed the accuracy of the numerical approximation code, we can use it to explore other functions. The functions and resulting solid for this example are shown in Figure 17.23.

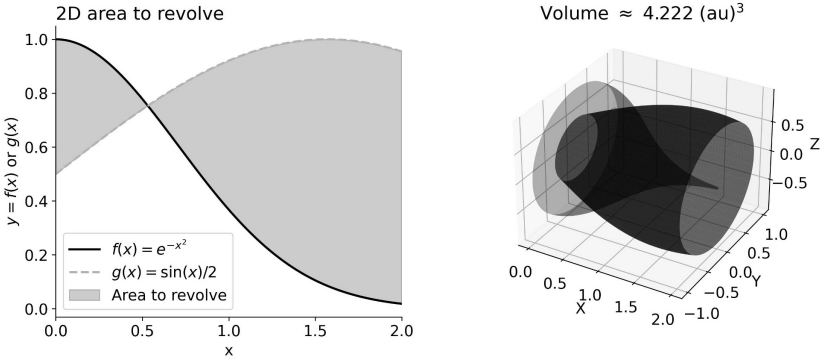


Figure 17.23: Numerically approximating a volume that is difficult to solve analytically.

This is an interesting example for two reasons. First, the functions cross, which means that the net area will have a negative component, although the function values remain positive in this interval. The strategy I'll use here is to integrate the absolute value of the function difference. In other words, the volume of the solid is given by $\int |f^2 - g^2| \, dx$. Second, the Gaussian function e^{-x^2} is not analytically integrable.

`sympy` was able to express the integral, but was unable to calculate

a numerical value for it. Below is the integral.

$$\pi \int_0^2 \begin{cases} \frac{\sin^2(x)}{8} - \frac{\cos^2(x)}{8} + \frac{1}{8} - e^{-2x^2} & \text{for } \frac{\sin^2(x)}{4} - e^{-2x^2} \geq 0 \\ -\frac{\sin^2(x)}{8} + \frac{\cos^2(x)}{8} - \frac{1}{8} + e^{-2x^2} & \text{otherwise} \end{cases} dx \tag{17.75}$$

`scipy` calculated the volume to be around 4.2 au^3. Because this solid does not correspond to a real-world object (although I think it would make an interesting wine glass), the numerical value itself is not very insightful. The key point here is that the "pure" calculus (the analytic definite integral) is too difficult to solve, but the numerical approximation is trivial with the help of Python.

Volumes of solids from measured coordinates Final note for this section: The techniques described above are for finding volumes that are created from mathematical functions that are revolved around an axis to create a 3D volume.

If you already have 3D coordinate points (e.g., measured xyz coordinates from a physical object) and want to calculate the volume contained inside, you would need methods developed in computational geometry such as convex hull estimation or Delaunay tetrahedralization. That's beyond the scope of integral calculus, and I mention it here so you are aware of the distinction.

17.6 Exercises

17.1. In the example problems for calculating the area between two functions (Figures 17.3, 17.5, and 17.6) I used `sympy` to calculate the definite integrals analytically. Your goal here is to re-run those area calculations using `scipy`. Calculate the net and total area, and make sure your net area approximation equals the analytic results (within some reasonable tolerance for numerical precision).

17.2. As you know, the FTC-2 states that the definite integral within some interval between a function and the x-axis is given by $F(b) - F(a)$. In this chapter, you learned how to extend definite integrals to the area between two functions, and the goal of this exercise is to explore how to express that area using FTC-2. On the one hand, if you define $h(x) = f(x) - g(x)$, then the FTC-2 is simply $H(b) - H(a)$. But what would the FTC-2 look like when expressed using the two functions?

It turns out that the answer is $A_\Delta = F(b) + G(a) - G(b) - F(a)$. Your first goal is to derive that expression on paper starting from the FTC-2 applied to $h(x)$.

Next, implement the FTC-2 equations ($H(b) - H(a)$ and $F(b)+G(a)-G(b)-F(a)$) to demonstrate their equivalence in an example using the equations below. Use $a = 0$ and $b = 1$.

$$F(x) = \int e^{-x^2}\, dx \tag{17.76}$$

$$G(x) = \int x^2\, dx \tag{17.77}$$

Use the `.subs()` method on the `sympy` antiderivative variables instead of using `sym.integrate` to calculate the definite integral. In the code, I did this using the two equations from Example 1 in the first section (Figure 17.3).

Finally, print the results as in Figure 17.24.

$$H(1) - H(0) = -\frac{1}{3} + \frac{\sqrt{\pi}\,\mathrm{erf}(1)}{2} - 0 = -\frac{1}{3} + \frac{\sqrt{\pi}\,\mathrm{erf}(1)}{2}$$

$$F(1) + G(0) - G(1) - F(0) = \frac{\sqrt{\pi}\,\mathrm{erf}(1)}{2} + 0 - \frac{1}{3} - 0 = -\frac{1}{3} + \frac{\sqrt{\pi}\,\mathrm{erf}(1)}{2}$$

Figure 17.24: `sympy` output for Exercise 2.

17.3. I really enjoyed making this exercise[7], and I hope you enjoy working through it. The goal is to reproduce the "smile" in Figure 17.25, and determine whether the "upper lip" or "lower lip" has more area.

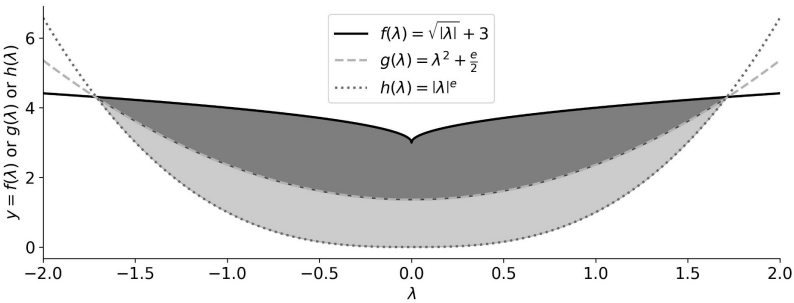


Figure 17.25: Visualizing Exercise 3.

Start by implementing the following equations in `sympy`:

$$f(\lambda) = \sqrt{|\lambda|} + 3 \tag{17.78}$$

$$g(\lambda) = \lambda^2 + \frac{e}{2} \tag{17.79}$$

$$h(\lambda) = |\lambda|^e \tag{17.80}$$

Notice from the figure that the functions all pinch together at approximately the same x-axis coordinates ($x \approx \pm 1.7$). The two gray patches show the two areas that form the lower and upper lips. To calculate these two areas, you need to find the intersection points between $f(\lambda)$ and $g(\lambda)$,

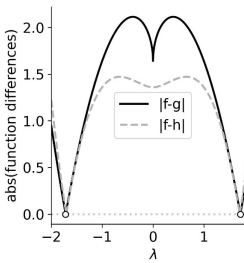


Figure 17.26: Finding roots empirically in Exercise 3.

[7] I enjoyed writing this entire book, but this exercise stood out as particularly enjoyable.

and between $g(\lambda)$ and $h(\lambda)$. It turns out that these equations (e.g., $f(\lambda) - g(\lambda) = 0$) cannot be solved analytically in `sympy`, although you should try it anyway to make sure you can write the code and to see what the error message is.

Therefore, you will need to estimate the intersection points numerically using methods you learned about in Chapter 6 on finding critical points. I recommend finding these intersection points without using the `scipy` function `find_peaks` as an additional math-coding challenge. To visually confirm your procedure for numerical approximation of real-valued roots, create a graph like Figure 17.26.

Finally, calculate the areas as the definite integrals. Although `sympy` struggled to find the roots of the difference equations, it can evaluate the definite integrals given those interval bounds, so you should do these calculations in `sympy` and print your results as in Figure 17.27.

$$\int_{2}^{2} \left(-\lambda^2 + \sqrt{|\lambda|} - \frac{e}{2} + 3\right) d\lambda \approx 5.26$$

$$\int_{-.04}^{.04} \left(\lambda^2 - |\lambda|^e + \frac{e}{2}\right) d\lambda \approx 4.03$$

Figure 17.27: Latex-formatted `sympy` output. The upper lip has more area than the lower lip.

One last (very optional) part of this exercise: Modify the colors and tweak the function parameters to get a pair of lips that you like, and send it to that special someone in your life with a note that you are doing *very serious mathematics*.

17.4. The natural log function $f(x) = \ln(x)$ intersects its indefinite integral twice, which makes it amenable to calculating the area between intersection points.

The goal of this exercise is to produce Figure 17.28. Implement the function $f(x) = \ln(x)$ in `sympy` and calculate its antiderivative. Try to solve for the intersection points analytically in `sympy`, although you'll find that it doesn't work[8]. Therefore, you'll need to approximate them empirically, this time using the `scipy` function `find_peaks`.

[8]Because libraries like `sympy` are continuously improving, it's possible that this part of the exercise will work in the future.

Use the intersection points to calculate the definite integral in `sympy`.

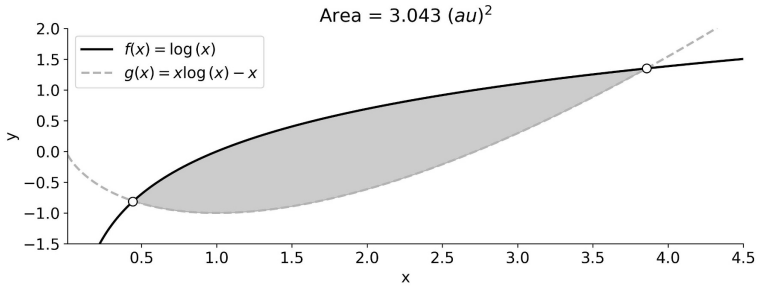


Figure 17.28: Visualization for Exercise 4.

17.5. This exercise follows from the previous, so it would behoove you to solve that one first.

Recall that the integral of $\ln(\alpha x)$ (for some real-valued scalar α) is $x\ln(\alpha x) - x$. This means that different values of α will impact the shape of both the function and its integral, which in turn will impact the intersection points and the area between them. The goal of this exercise is to explore this parameter in more detail.

In a for-loop over 33 values of α between .4 and 5, create Python functions for $f(x) = \ln(\alpha x)$ and its integral by creating the expressions in `sympy` and then using `sym.lambdify`. Plot the functions as in Figure 17.29A, with solid lines corresponding to $f(x)$ and dashed lines corresponding to $F(x)$, and with lighter lines corresponding to larger values of α.

Panel B shows two of the functions corresponding to the smallest and largest α values, with patches drawn to indicate the area between the functions. Finally, panel C shows the area as a function of the α parameter.

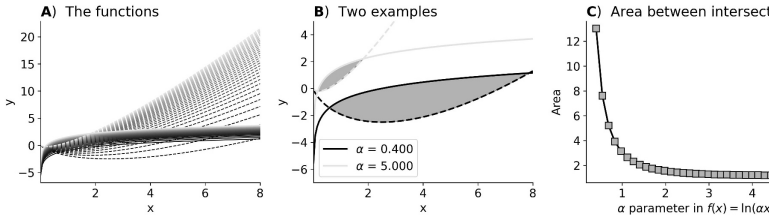


Figure 17.29: Visualization for Exercise 5.

Final thought for this exercise: If you're up for an additional challenge, try repeating this exercise for the function $f(x) = \ln(\alpha x^2)$, using a subdomain of $x \in \{-4, 4\}$. Some values of α produce three intersection points in this interval, so you'll need to make a decision about how to deal with two enclosed areas (e.g., just measure the area of one, or perhaps sum the net or total of both areas). The modifications are sufficiently significant that it's best to modify a fresh copy of the code, rather than directly modifying the code that solves this exercise.

17.6. The goal here is to write code to make Figure 17.9 (page 572) by implementing the following sets of expressions.

$$\textbf{A)} \begin{cases} t \in \{-2\pi, 2\pi\} \\ x = \sin(2t) + \sin(t) \\ y = 2\sin(3t) \end{cases}$$

$$\textbf{B)} \begin{cases} t \in \{0, 2\pi\} \\ x = 3\cos(t) + \cos(3t) \\ y = 3\sin(t) - \sin(4t) \end{cases}$$

$$\textbf{C)} \begin{cases} t \in \{0, 12\pi\} \\ x = \sin(t)\left[e^{\cos(t)} - 2\cos(4t) - \sin^5(t/12)\right] \\ y = \cos(t)\left[e^{\cos(t)} - 2\cos(4t) - \sin^5(t/12)\right] \end{cases}$$

$$\textbf{D)} \begin{cases} t \in \{0, 12\pi\} \\ x = \sqrt{3}\cos(2t) - \cos(10t)\sin(20t) \\ y = \sqrt{2}\sin(2t) - \sin(10t)\sin(20t) \end{cases}$$

$$\textbf{E)}\ \begin{cases} t \in \{0, 2\pi\} \\ x = \cos(t) + \cos(52t)/2 + \sin(25t)/3 \\ y = \sin(t) + \sin(52t)/2 + \cos(25t)/3 \end{cases}$$

$$\textbf{F)}\ \begin{cases} t \in \{-2\pi, 2\pi\} \\ x = 2.5\sin(\sin(-5t))\cos^2(9.844t) \\ y = 2.5\sin^2(-5t)2^{\cos(\cos(9.844t))} \end{cases}$$

I used lines to draw that figure; you can try using `plt.scatter` to associate a color with each dot; Figure 17.30 shows one example.

Parametric curves defined by complicated trig functions can be extremely sensitive to minor changes in parameters. For example, try changing 9.844 to 10 in the x expression in **F**. I encourage you to spend some time exploring and enjoying these functions. You can just randomly change numbers, include more or remove existing terms, and see what happens! Maybe you'll discover some beautiful patterns that you can print on a t-shirt.

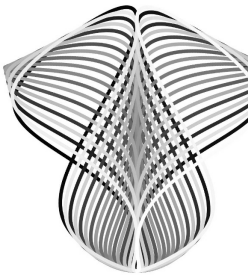


Figure 17.30: Adding grayscale intensity to the parametric curves.

17.7. The main goal of this and the next exercise is to investigate the impact of Δt on the numerically approximated length of a curve. This exercise involves writing code to measure the length of the "butterfly curve" (Figure 17.9C), and the next exercise will involve systematically varying the Δt and measuring the length of the curve.

Write code to produce Figure 17.31. Make sure your code allows you to specify a desired Δt, as opposed to specifying n, which is how most of the code in this chapter was written. When calculating curve length, implement the integration directly in `numpy`, by writing out $\sum \sqrt{(\Delta x)^2 + (\Delta y)^2}$.

I haven't yet introduced histograms in this book, although they will be featured in the next chapter on statistics. In case you are unfamiliar with histograms (this is the bar plot in Figure 17.31B): A histogram shows the distribution of data values, in this case, the lengths of the line segments.

The idea is to create a set of non-overlapping bins (I used 30 bins here) spanning the smallest to the largest segment lengths, and then count the number of line segments in each bin. You can use the function `plt.hist` to create and display the histogram[9].

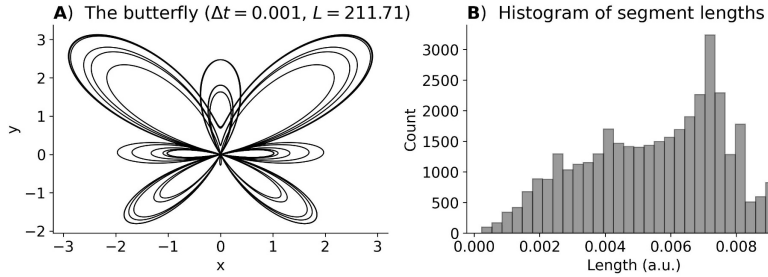


Figure 17.31: Visualization for Exercise 7.

Now that you have code to produce the figure, try some other values for the Δt parameter, and observe how the histogram changes. In particular, look for changes in the x- and y-axis values, and the overall shape of the distribution. But don't spend *too much* time exploring this; the following exercise involves systematically varying Δt.

Note: My online code solution for this exercise uses variables created in Exercises 6 and 7.

17.8. Before writing any code, inspect Figure 17.32 and make sure you understand what it shows and what it means. I'll write a description below the figure.

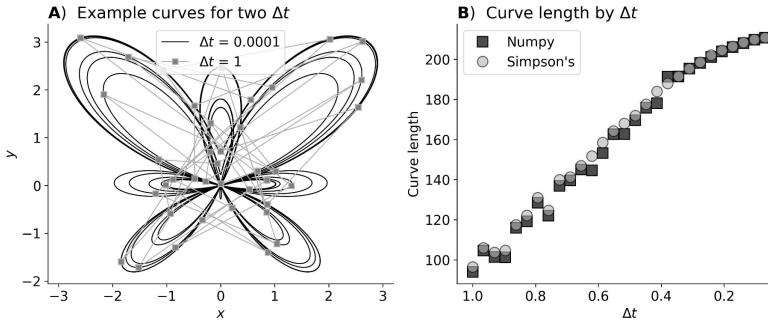


Figure 17.32: Visualization for Exercise 8.

[9] Alternatively, you can use `np.histogram` to generate the x and y values and then draw the histogram using `plt.bar`.

Panel A shows two curves created by evaluating the same function but with different resolutions. The light gray line using $\Delta t = 1$ is barely recognizable as a butterfly, because the resolution is too coarse to be meaningfully visually interpreted.

Calculate the lengths of the curves twice: Once using a direct implementation in `numpy` as in the previous exercise, and once using `spi.simpson` on the integrand $\sqrt{(\Delta x/\Delta t)^2 + (\Delta y/\Delta t)^2}$.

Panel B shows the length of the curve on the y-axis as a function of Δt on the x-axis (ranging from .0001 to 1 in 30 steps). The key result is that the curve appears to get longer when measured with a finer scale. The effect is rather extreme here (more than doubling the apparent length!) because there are so many tight twists and turns in this curve. If the path would curve more gently, then Δt would be less impactful. To be clear: The curve as a pure mathematical object has exactly one unique length; the issue is that the lengths we calculate are numerical approximations that depend on our measurements.

The phenomenon of differences in measured size as a function of the measuring unit is well-known in mathematics, and is studied in the context of fractals and self-similarity. If you're curious to learn more about this, you can search the Internet for "Mandelbrot how long is the coast of Britain."

17.9. There is an interesting curve called the "Folium of Descartes"[10]. It has a number of interesting properties that illustrate some challenges in computational mathematics.

The goal of this exercise is to implement the curve in polar and parameterized formats, and then in the next exercise you will calculate its length. The result of this exercise will be to produce Figure 17.33.

[10] Allegedly, Descartes developed this curve as an "impossible" challenge to Fermat, who easily solved it. Even the most brilliant mathematicians are still just humans.

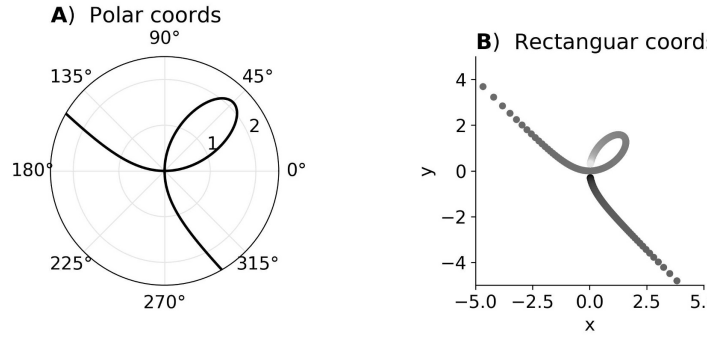


Figure 17.33: Visualization for Exercise 9.

The Folium is described by the following polar coordinates, where r is the radius and θ is radian values between 0 and 2π.

FYI: Increasing the "3" makes the loop bigger.

$$r = \frac{3\sin\theta\cos\theta}{\sin^3\theta + \cos^3\theta} \tag{17.81}$$

To visualize the polar coordinates in panel A, you will need to use a polar projection in `matplotlib`. I haven't introduced plotting on a polar axis, but the basic syntax is to create a polar axis and then plot θ by r instead of x by y. `matplotlib` code looks like this:

```
ax = plt.subplot(1,projection='polar')
ax.plot(theta,radius)
```

The polar coordinates can be represented as a parametric curve using the following expressions (I used 900 points for t in the plot).

Notice that bad things happen as $t \rightarrow -1$...

$$x = \frac{3t}{1+t^3} \tag{17.82}$$

$$y = \frac{3t^2}{1+t^3} \tag{17.83}$$

$$t \in \{-10, 10\} \tag{17.84}$$

When you first plot these functions, your graph probably won't look like mine. There are extreme numerical values

in these functions. Check the axis limits of your graphs and the limits in Figure 17.33, and adjust accordingly.

17.10. The goal of this exercise is to measure the curve length of the Folium of Descartes. It's not as straightforward as with previous curves you've worked through, because of the extreme values you discovered while creating the plot in the previous exercise. While those data points are valid parts of the curve, they are extreme outliers. "Outlier" is a term in statistics to describe data points that are so extreme that they bias data characterizations and statistical results.

Start this exercise by removing any segments that come from data points greater than five units away from the origin of the graph (measured as Euclidean distance). Then make a figure like Figure 17.34.

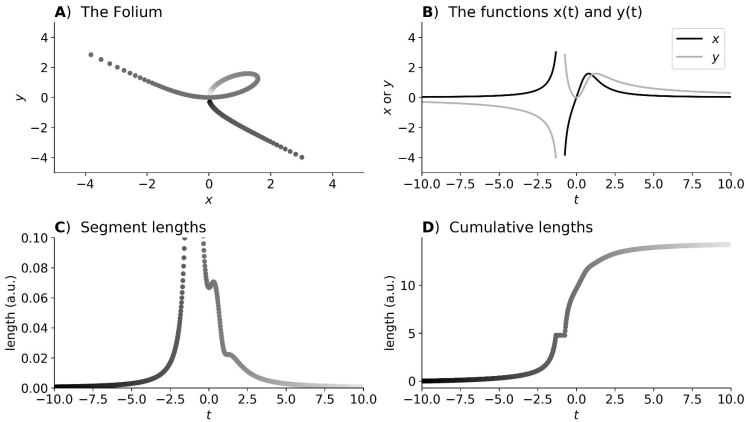


Figure 17.34: Visualization for Exercise 10.

Notice that the x and y function values are clipped in panel B, and that the cumulative lengths have a brief "flat" area around $t = -1$. You can explore different thresholds for removing the extreme data values and different y-axis scaling.

Calculate the curve length using both the direct calculation in `numpy`, and using `spi.simpson`, as you coded in the previous exercise. Be mindful of how to deal with the outliers that were removed (or labeled as NaN).

With the outlier thresholding, I got a curve length of around 14.25, compared to an estimated length of over 500 without removing any values. The curve lengths you will get depend on parameters including the threshold, the bounds of t, and the number of points to estimate. The curve lengths will also differ between the two calculations, and their discrepancy depends on how you deal with outlier values.

More generally, dealing with unusual values is a crucial yet ambiguous part of applied calculus, especially when dealing with empirical data. You'll need to use domain knowledge and experience to determine appropriate thresholds for excluding unusual data values.

17.11. This exercise is an extension of Exercise 3. The goals is to revolve the difference between $f(x)$ and $h(x)$ around the x-axis, and calculate the volume of that solid, as shown in Figure 17.35.

To add a bit of a challenge, figure out how to shift the functions so that the left function intersection point is at $\lambda = 0$ (note the values of the λ axis in the figure).

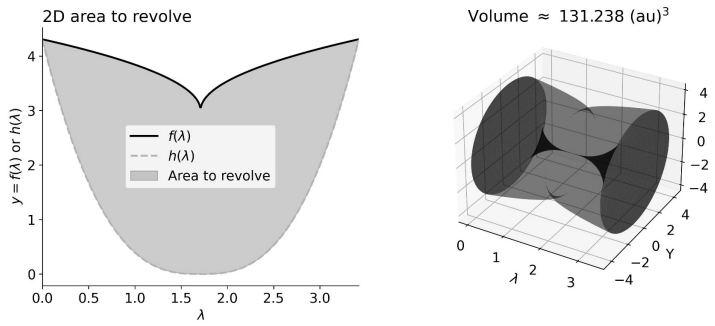


Figure 17.35: Visualization for Exercise 11.

17.12. Are you familiar with the "Ruben's face illusion"? It's an ambiguous drawing that is either a vase or two faces. Figure 17.36 shows an example. Before reading further, make sure you can see the vase in the middle (white) and the two faces looking at each other (gray).

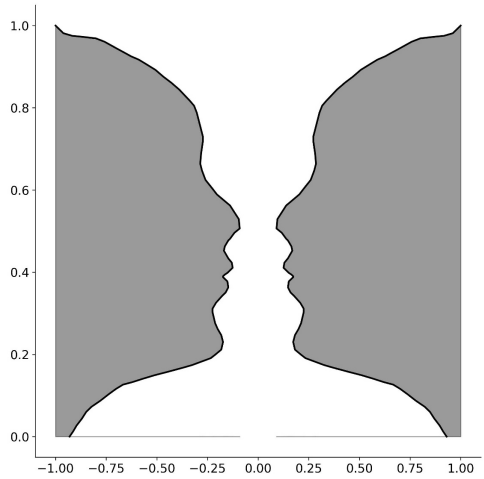


Figure 17.36: Visualization for Exercise 12.

The online code for this exercise includes variables that provide the x- and y-axis coordinates to draw this illusion.

The purpose of this exercise is to calculate the areas of the three shapes, using Riemann sums with the trapezoid rule. Report each area and confirm that the sum of all areas is two (notice that the entire graph area spans one unit on the y-axis and two units on the x-axis).

Reminder: the area of a trapezoid is $\Delta x(f(a) + f(b))/2$.

Print your results as follows:

```
Total area of the vase       : 0.743
Total area of the left face  : 0.629
Total area of the right face : 0.629
Sum of three areas           : 2.000
```

If you need a hint for solving this exercise, check the footnote[11].

[11] The black lines defining the contours cannot be considered "functions" (they fail the vertical line test). Therefore, swap the axes such that the vase is on its side. This will help you conceptualize each object as a collection of trapezoids. I show this in the online code before showing the area calculation.

Chapter 18

Integration and statistics

Probability theory and statistics intersect nearly every aspect of modern life, because probability and statistics have allowed for developments and discoveries in nearly every area of scientific inquiry and engineering.

Probability theory and statistics are, therefore, major topics of human intellectual pursuit. Fortunately for purposes of this book, many of the mathematical foundations of probability theory involve calculus concepts that you have already encountered in this book. In fact, you'll see in this chapter that the use of calculus in probability theory and statistics is basically just applying the two Fundamental Theorems of Calculus.

18.1 Introduction to probability and statistics

Few things in life are certain, other than the infamous death and taxes. Instead, most events occur with some *probability*, or chance. Probability theory is the mathematical framework with which uncertainty is quantified, characterized, and predicted.

Probability theory was developed over many centuries, coinciding with developments in physics, chemistry, biology, and economics, that required mathematically precise characterizations of uncertainty in order to make meaningful progress[1].

Statistics is a closely related discipline. In fact, statistics can be described as applied probability theory, with additional considerations for collecting, presenting, and interpreting data.

Introductions to probability theory often begin with an example of flipping a coin: What is the probability of a flipped coin showing "heads" vs. "tails?" Is this case, it's 50/50: there is a 50% chance of seeing "heads" and a 50% chance of seeing tails.

Here's a more interesting question: What is the probability that

[1] Gambling was also a major factor in the development of probability theory.

a randomly selected worker in the EU has an annual salary exceeding €100,000? To answer this question, we would need to know the salaries of all workers in the EU, then calculate how many make more than €100k. But you might already be thinking that it's more complicated than this, because the range of salaries depends on many factors, including the country: There is a higher probability that a randomly selected worker in Luxembourg has a high salary, compared to a randomly selected worker in Bulgaria.

In other words, the *distribution* of salaries differs by country, and calculating an accurate probability requires knowing information about the relevant distribution.

18.1.1 What are distributions?

A probability distribution is a function that describes the possible values a variable can take, and the probabilities of observing each of those values. Distributions provide a structured way to understand data and are fundamental tools in statistics and computational sciences. They allow both quantitative analysis (e.g., finding probabilities or percentiles) and qualitative insights (e.g., visual patterns of variability).

Let's look at an example of adult human height. Each possible height (within a realistic range, plotted on the x-axis) is assigned a probability (Figure 18.1). This distribution often resembles a "bell curve" or normal distribution, with most values clustering around an average height and fewer individuals at the extremes. The distribution tells us that a randomly selected adult is more likely to be around 170 cm tall, compared to 165 or 190 cm tall.

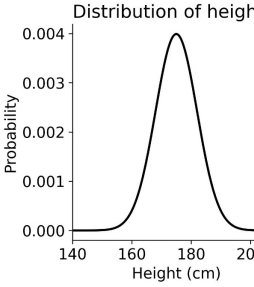


Figure 18.1: Probability distribution of adult male heights in cm.

There are many types of distributions, each with unique mathematical properties and visual characteristics. These properties make certain distributions better suited for particular types of statistical analyses or real-world applications.

Where do distributions come from? Broadly speaking, distribu-

tions come from one of two sources:

Analytic Distributions

Analytic distributions are defined by mathematical formulas. Examples include the normal, exponential, and logarithmic distributions, and you'll see more examples later in this chapter.

Analytic distributions provide exact, continuous functions for probabilities, which allow for efficient and precise calculations of probabilities and other statistical characteristics. They are also easy to work with computationally, partly because they do not require any data to construct, and partly because their properties can be defined using calculus methods such as integration.

Empirical distributions

Empirical distributions are based on data obtained from observations or experiments. Unlike analytic distributions, empirical distributions don't follow a predefined formula; instead, they represent the counts of measurements such as height, income, web-link clicks, or rainfall.

A common way to represent empirical distributions is through histograms. A histogram divides the data range into equally spaced bins, then counts the number of values falling within each bin. The resulting plot shows the amount of data at different values, which in turn provides insights into the probabilities of each value being observed. Empirical distributions can look similar to analytic distributions, and indeed, are sometimes used as approximations of analytic distributions.

Figure 18.2 highlights the difference between an analytic distribution that is obtained by evaluating a mathematical function (panel A), vs. an empirical distribution represented as a histogram (panel B).

Although there are important distinctions between analytic and empirical distributions, empirical distributions are sometimes considered approximations of analytic distributions, and analytic distributions are sometimes used to model empirical distributions.

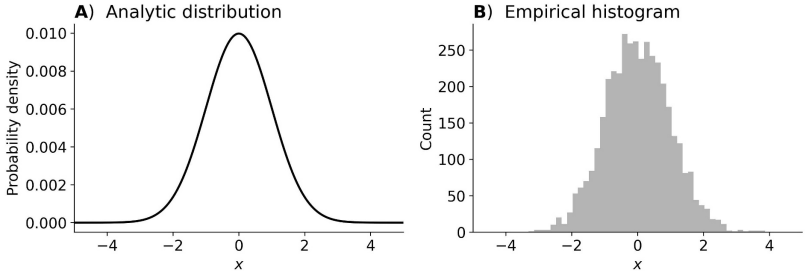


Figure 18.2: Example of an analytic distribution from an equation (panel A) and a histogram from simulated data (panel B).

18.2 Probability density functions

Probability density functions, often abbreviated to pdf, are continuous functions in which the function value at each point x_i indicates the chance that a value of x_i will be observed. Figure 18.2A showed a pdf while Figure 18.2B showed a histogram that, while not formally a pdf, can be used as an empirical approximation to a pdf.

Probability functions are mathematical functions, just like all the other functions you've encountered in this book. On the other hand, probability functions have additional requirements and are used in special ways in statistics. Therefore, for the rest of this chapter, I will use $p(x)$ to indicate a probability function (that's not ubiquitous notation in the statistics literature).

$p(x)$ is a function that transforms events into probabilities of those events occurring or being observed. For example, imagine that X is the set of two elements: {rain, no rain} on November 4^{th} in Barcelona, and the function $p(x)$ transforms those two elements into the numbers {.7, .3}. In statistics, the set of events can be discrete (examples include rain, gender, movie genre, pass/fail an example, alive/dead after a disease treatment) or it can be continuous (examples include amount of rain, height, income).

Sets of discrete events are called *probability mass functions*. They are used in statistics but I won't discuss them here. The principles

are the same but the math involves sums instead of integrals, so they are less germane to this book[2].

Formally, a pdf must satisfy the following requirements.

$$p(x) \geq 0, \quad \forall\, x \in X \tag{18.1}$$

$$\int_{-\infty}^{\infty} p(x)\, dx = 1 \tag{18.2}$$

Equation 18.1 states that all function values must be non-negative, and Equation 18.2 states that the integral over the entire domain must equal one. You can summarize these requirements as *non-negativity* (Equation 18.1) and *normalization* (Equation 18.2).

Why does a pdf need these properties? Probability values reflect the chance, or likelihood, of an event occurring, and thus cannot be negative. Of course, we can speak of the probability of an event *not* happening, but that's still a positive probability. For example, the probability of *not* being eaten by a shark increases dramatically if you stay out of the ocean.

Probability functions are normalized to have an area of 1, but probability results are sometimes multiplied by 100 to obtain units of percent.

The reasoning of Equation 18.2 is that if $p(x)$ encodes all possible events in set X, then the probability of *something* happening must be 1. Even if *nothing* happens, then *nothing* must an element in X.

18.2.1 Examples of pdfs

Example 1: Normal distribution The normal distribution, a.k.a. the Gaussian distribution, a.k.a. the bell curve, is so important and foundational to so many applications of probability and statistics, that it's usually the first pdf people learn about.

The normal distribution is defined by the following equation.

[2]Sometimes, sets are not formally continuous but are treated as such for convenience, such as income (e.g., currencies are not infinitely discretizable) or pages in a book.

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\frac{-(x-\mu)^2}{2\sigma^2}\right) \tag{18.3}$$

μ is called a "shift parameter" and produces an x-axis offset that slides the entire distribution left or right. For simplicity, μ is often omitted (thus implicitly set to 0), although of course you can explore the impact of this parameter on the graphs and numerical calculations using the online code.

σ is variously called the "stretch," "shape," or "scale" parameter, and makes the pdf wider or narrower.

Notice that the key "skeleton" of Equation 18.3 is e^{-x^2}; all the other terms and parameters are merely scalars that modify the shape and ensure that the function conforms to the constraint that the pdf integrates to 1.

To illustrate the normal distribution, I implemented the function in `sympy` and lambdified the symbolic expression to obtain a numerical array.

```
x,s = sym.symbols('x,s')
px = 1/sym.sqrt(2*sym.pi*s**2) * sym.exp( -x**2/(2*s**2) )
px_l = sym.lambdify((x,s),px)
```

Notice that `px_l` is a function of two variables (x and σ); there is no μ term in the expression; hence that parameter is fixed to zero.

I then evaluated the function in `numpy` and summed over all the values — according to the definition of a pdf, that sum should be exactly 1.

```
xx = np.linspace(-4,4,1001)
pdf = px_l(xx,.2) # \sigma=.2
sumPdf = np.sum(pdf)
```

Figure 18.3 shows pdfs and their numerical sums for three values of σ. What are some of your observations about the graph?

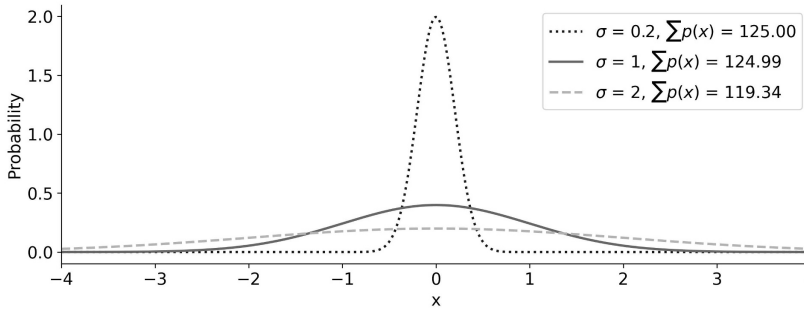


Figure 18.3: Three normal pdfs using three values of σ.

Here are a few of my observations: (**1**) The σ parameter affects both the width and the height of the pdf: Curves with larger σ values appear "squashed down." (**2**) The functions appear symmetric around $x = 0$. Indeed, the normal function is even-symmetric around $x = \mu$, as the only appearance of x in Equation 18.3 is squared. (**3**) All three pdfs have their peak at $x = 0$. That's because $\mu = 0$. If you set $\mu = 1$, then the shapes of the pdfs will be the same, but they will be shifted to peak at $x = 1$.

But perhaps the most striking observation is (**4**) the sums are *much* larger than 1. Look again at my code above, and see if you can spot the error before reading the next paragraph.

Remember that the constraint for a pdf is that the function *integrates* to one. The sum is the approximation of the integral, but only when scaling by dx. The code above does not scale by dx, which means I've actually calculated $\int_{-\infty}^{\infty} p(x)$ — without the dx. That's the error. The solution is to include `dx` in the sum, although it's usually better to scale the `pdf` variable by `dx`. Figure 18.4 shows the same functions but now scaled by `dx`.

Scaling by `dx` does seem to improve the situation, although the curve for $\sigma = 2$ still doesn't sum to 1. The issue in this case is that the integral is defined from $-\infty$ to $+\infty$, but the numerical evaluation is only done between -4 and $+4$. This means that some of the density of the pdf is excluded by focusing on this restricted domain. In fact, this also happens for the other two

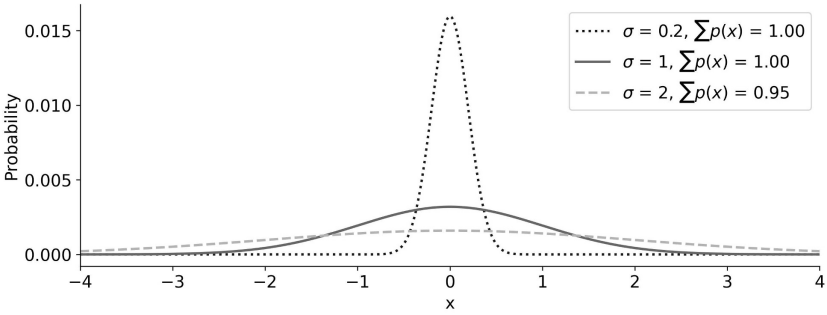


Figure 18.4: The three pdfs again, scaled by `dx`. Compare the legend and y-axis values here against those in Figure 18.3.

curves, but the contribution of the excluded mass is so tiny that the sum rounds up to 1.00 in the visual formatting. For most applications, this isn't a problem, but is something to be aware of.

Example 2: F-distribution The equations underlying pdfs can get very complicated. Equation 18.4 is for the F-distribution, which is used to evaluate the statistical significance of regressions and ANOVAs — two of the most important families of statistical models. The F pdf takes two inputs, d_1 and d_2, which are called "degrees of freedom" and together determine the shape of the distribution.

Degrees of freedom are positive-valued numbers that describe the sample size and model complexity.

$$p(x; d_1, d_2) = \frac{\sqrt{\frac{(d_1 x)^{d_1}\ d_2^{d_2}}{(d_1 x + d_2)^{d_1 + d_2}}}}{x\,\mathrm{B}\left(\frac{d_1}{2}, \frac{d_2}{2}\right)} \tag{18.4}$$

The B() term is a separate function, called the Beta function, that is used in statistics and physics to calculate certain features of probability functions.

Rather than implement this function directly as a `sympy` expression, I will use the `FDistribution` function that `sympy` provides in its `stats` module. The syntax is slightly different from other functions in `sympy`: You first create a probability function object and then request the pdf of that object. That pdf expression can

then be lambdified for evaluation in `numpy` and visualization in `matplotlib`.

```
import sympy.stats
d1, d2 = 5, 20
F = sym.stats.FDistribution('F',d1,d2)
Fpdf_expr = sym.stats.density(F)(x)
Fpdf_l = sym.lambdify(x,Fpdf_expr)
```

The F pdfs for three pairs of d parameters are shown in Figure 18.5. Please make some observations and then compare your notes against mine below.

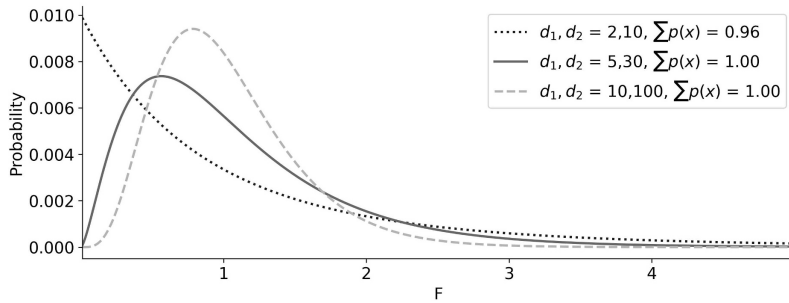


Figure 18.5: Three *F*-pdfs.

(**1**) The x-axis here is positive. That's not just a choice of visualization; the F-distribution is undefined for $x \leq 0$. (**2**) These distributions are non-symmetric and have a long tail stretching off to the right. (**3**) Two of the functions shown have peaks while the third (with the smallest d values) is a smooth decay. Thus, the shape of the pdf can qualitatively change depending on the degrees of freedom. (**4**) The sum over all extracted (and correctly dx-scaled) values is close to 1; again, small discrepancies are expected in numerical calculations with restricted domains.

Example 3: Pareto The Pareto pdf is often used in economics, for example to model distributions of wealth and productivity.

The pdf follows a fairly simple function for some positive parameter α. The x_m is a minimum value of the function.

$$p(x) = \begin{cases} \dfrac{\alpha x_m^\alpha}{x^{\alpha+1}} & \text{for } x \geq x_m, \\ 0 & \text{for } x < x_m. \end{cases} \tag{18.5}$$

Three curves for three values of α are shown in Figure 18.6. As usual, I will make some observations below.

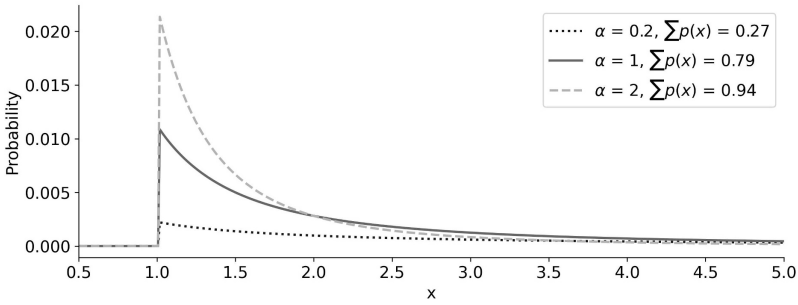


Figure 18.6: Three Pareto-pdfs.

(**1**) This pdf is non-symmetric. (**2**) It doesn't have a "peak" in the same sense that the normal and F pdfs have peaks. Instead, it has a sudden onset at x_m (in the figure, $x_m = 1$) and a smooth decay afterwards. (**3**) None of the numerical evaluations of the Pareto function shown here sum to 1. The Pareto distribution has a very long tail to the right. This is part of the reason why it's used for modeling wealth distributions: Lots of people have little wealth, there is a floor below which wealth cannot go (unless you consider debt to be "negative wealth"), and a very small number of people have a very large amount of wealth.

18.3 Cumulative distribution functions

Simply put, a cumulative distribution function (abbreviated cdf) is the antiderivative of the pdf[3]. We could notate cdf as $P(x)$,

[3]Why is it called probability *density* but cumulative *distribution*? The *density* refers to a particular value, whereas the *distribution* highlights that the cdf at each value is the sum over the distribution left of that value.

however, I will use $c(x)$ in this chapter. Again, that is not a universal statistical standard terminology, but helps highlight the unique features of cdfs compared to indefinite integrals of non-pdf functions.

$$c(x) = \int p(x)\, dx \tag{18.6}$$

$$c(x_i) = \int_{-\infty}^{x_i} p(x)\, dx \tag{18.7}$$

Equation 18.7 clarifies that each value $c(x_i)$ in the cdf is the definite integral of the pdf from $-\infty$ to x_i.

The interpretation of the cdf is different from that of the pdf: Whereas a pdf is the probability of each value x_i being observed, the cdf is the total probability of observing *all* values of x between $-\infty$ and x_i. This is illustrated in Figure 18.7. On the one hand, this is literally just the interpretation of constructing an antiderivative from a function, but it does have implications for interpreting cdfs in context of probabilties.

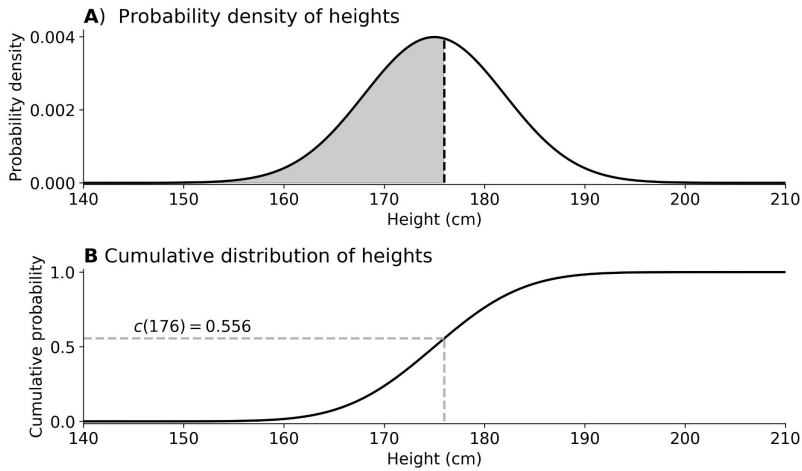


Figure 18.7: The cdf value for 176 cm (dashed lines in panel B) is the integral of all pdf values left of $x = 176$ (shaded area in panel A).

For example, imagine a function $p(x)$ that is a pdf of the height of adult male humans in cm; $c(176)$ is the probability of finding

a male who is *at most* 176 cm. That value is around .55, that is, $c(176) = .55$. This number means that there is a 55% chance that a randomly selected adult male is 176 cm or shorter.

Notice that the cdf increases with increasing x. Indeed, all cdfs are monotonically increasing functions, because they are the integral of non-negative probability values (due to the nonnegativity constraint of pdfs). All cdfs end at $y = 1$ (that is, $c(\infty) = 1$) due to the normalization constraint of pdfs. Many cdfs that are used in applications start at zero (that is, $c(-\infty) = 0$), although that is not a necessary feature of all cdfs; some probability functions have non-zero values at the smallest x values.

To clarify the interpretation of a cdf, statistics texts will often express the cdf as a definite integral instead of an indefinite integral. That is, Equation 18.6 can be written as:

$$c(x) = \int_{-\infty}^{x} p(u)\, du \tag{18.8}$$

Variable u is a dummy-variable used to indicate that the cdf at any particular value of x is the integral of the pdf from $-\infty$ to that particular value of x (I introduced notation like this on page 419).

Constant of integration? You may have noticed that there is no constant of integration term in the antiderivative functions in this chapter.

Technically, those constants should be included in any indefinite integral. However, statistics applications of the cdf involve calculating definite integrals, where the constant is unnecessary.

Moreover, cdfs are constrained such that $c(\infty) = 1$ (and $c(-\infty) = 0$ in many distributions). These constraints can be used to resolve the indeterminacy of the constant of integration.

18.3.1 Examples of cdfs

Below I will highlight three examples of cdfs. You'll notice that they all look quite similar to each other. Indeed, unlike pdfs, cdfs from different distributions are difficult to differentiate based on visual appearances alone, because they are all monotonically increasing functions.

Example 1: Triangular The triangular distribution has four segments and is expressed as a piecewise function. The triangular distribution takes three parameters: a, b, and d, sorted such that $a < b < d$. The resulting pdf has a triangular shape with a and d as the two bases, and b in between. You'll see what the pdf looks like in Exercise 3; for now, let's focus on creating the cdf. In fact, the cdf formula is simple enough to be implemented in `numpy`.

I'm using a, b, and d as parameters to avoid the clash between *parameter* c and *cdf* $c(x)$.

$$c(x) = \begin{cases} 0 & \text{for } x \leq a \\ \dfrac{(x-a)^2}{(d-a)(b-a)} & \text{for } a < x \leq b \\ 1 - \dfrac{(d-x)^2}{(d-a)(d-b)} & \text{for } b < x < d \\ 1 & \text{for } x \geq d \end{cases} \tag{18.9}$$

A cdf generated from this equation is shown Figure 18.8. It's not obvious from the plot that its pdf forms a triangle. Indeed, cdfs rarely reveal the characteristics of their pdf — just like how you cannot necessarily understand the characteristics of a function by looking at its antiderivative. That's why many statistics applications use cdfs for key analyses but visualize the pdfs (I'll get back to this point in a later section).

Triangular distributions are useful because the location of the peak and slope down to $p = 0$ can be arbitrarily controlled.

Example 2: Laplace Figure 18.9 shows the cdfs and corresponding pdfs of the Laplace distribution with three different parameter

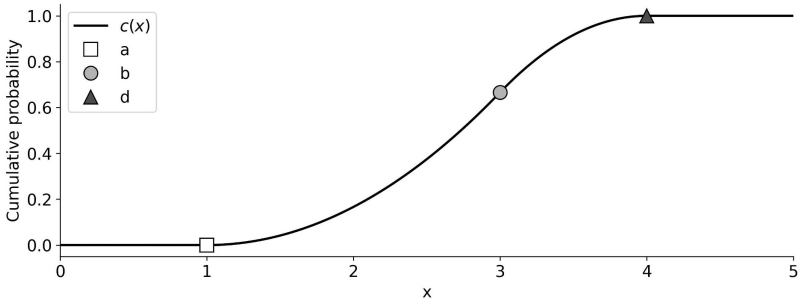


Figure 18.8: A cdf of a triangular distribution.

settings. The key parameters are μ, which is the "shift" or "location" parameter that translates the function along the x-axis; and b, which is the "stretch" parameter that defines the steepness of the slope on either side of the peak.

$$p(x, \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right) \tag{18.10}$$

$$c(x, \mu, b) = \begin{cases} \frac{1}{2} \exp\left(\frac{x-\mu}{b}\right) & \text{for } x < \mu \\ 1 - \frac{1}{2} \exp\left(-\frac{x-\mu}{b}\right) & \text{for } x \geq \mu \end{cases} \tag{18.11}$$

Note that the "skeleton" of the pdf is $e^{-|x|}$ — similar to the Gaussian but without squaring the x. The absolute value ensures that the function is even-symmetric around μ.

To continue illustrating the myriad ways to calculate pdfs and cdfs in Python, I implemented this function using `sympy`'s substitution method with list comprehension.

```
x = sym.symbols('x')
xx = np.linspace(-4,4,501)
laplacian = sym.stats.Laplace('L',mu,b)

cdf_expr = sym.stats.cdf(laplacian)(x) # cdf object
cdf = [ cdf_expr.subs(x,xi) for xi in xx ] # cdf values

pdf_expr = sym.stats.density(laplacian)(x) # pdf object
```

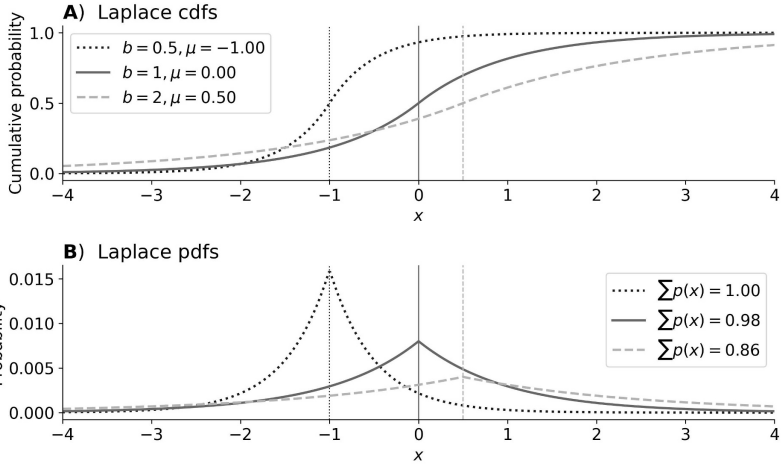


Figure 18.9: Three examples of Laplace cdfs and pdfs. The thin vertical lines demarcate the peak of the pdf (μ parameter).

```
pdf = [ pdf_expr.subs(x,xi)*dx for xi in xx ] # pdf value
```

Example 3: Logistic The third example cdf is the logistic distribution. The formula that generates a logistic cdf is shown in Equation 18.12, and the cdfs and pdfs for a few parameter sets are shown in Figure 18.10.

$$c(x, \mu, \sigma) = \frac{1}{1 + e^{-(x-\mu)/\sigma}} \tag{18.12}$$

Notice the similarities between the logistic cdf and the sigmoid function[4]:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{18.13}$$

To generate these cdfs, I used the `scipy.stats` library.

```
import scipy.stats as stats
```

[4]Here you see the overloading of *parameter* σ and *function* $\sigma(x)$. Those represent different objects yet use the same Greek letter. Math is full of confusing terminology; it's just something to get used to.

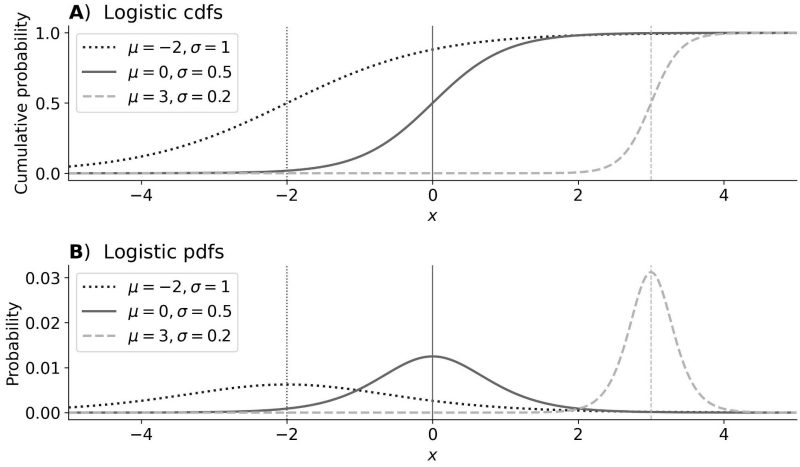


Figure 18.10: Three examples of logistic cdfs and pdfs. The thin vertical lines indicate the μ parameter, which defines the pdf peak location.

```
xx = np.linspace(-5,5,401)
mu, sigma = 0, .5
cdf = stats.logistic.cdf(xx,loc=mu,scale=sigma)
```

Note that `scipy` uses the input term "`loc`" for the location parameter μ, and "`scale`" for the scaling parameter σ.

18.4 Application: pdf and cdf in the *z*-test

A z-test is a statistical evaluation of whether the average value of a sample (in this context, a "sample" is a group of members of a population, e.g., data from a sample of 500 university students) differs from an expected population value. The z-test provides a great introduction to calculus in statistics because it allows us to explore key concepts without requiring a lot of background information.

Here's the question that we will use the z-test to answer: *Do readers of this book have higher IQ scores than those who don't read*

this book? IQ scores are normally distributed with a population average of 100 and a standard deviation of 15. Therefore, we can create a pdf of IQ scores in the population using $\mu = 100$ and $\sigma = 15$ (Figure 18.11).

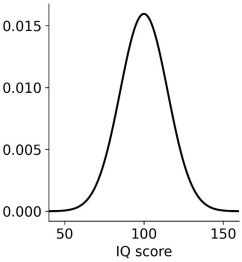


Figure 18.11: Distribution of expected IQ scores in the population.

Imagine that I randomly sample 15 readers of this book and measure their IQ scores. The question at hand is whether the average IQ in this sample is significantly higher than 100. Here are the data[5]:
$\text{IQ} = [106, 96, 110, 125, 119, 103, 112, 114, 99, 108, 106, 94, 101, 115, 111$

The average is 107.9. Of course, that average is *numerically* greater than 100, but is it *significantly* greater than 100? The issue is that because there is individual variability in IQ scores, even a purely random sample from the population could have an average IQ score greater than 100.

The z-test allows us to calculate probabilities that help us determine whether the characteristics of our sample are statistically significantly different from those of the population (indicating something special about our sample), or whether the difference between our sample and the population can be attributed to random sampling. For example, we could ask the question: *What is the probability that a randomly selected group of 15 people (not only readers of this book) have an average IQ score of at least 107.9?*

Deriving and explaining this formula would be covered in a statistics book, but the basic idea is to calculate the normalized distance between the sample mean and the population mean.

The first step in a z-test is to calculate the z value:

$$z = \frac{\overline{x} - \mu}{\sigma/\sqrt{n}} = \frac{107.9 - 100}{15/\sqrt{15}} = 2.048 \qquad (18.14)$$

$\overline{x}$ is the average of our sample, and n is the sample size. The z-score is a standardized measure of how our sample mean compares to the population pdf. In the context of our research question, the z-score is zero if the book-readers' average IQ score is identical to the population mean, positive if our sample has a higher average

[5]These are fake data that I made up for this demo.

IQ score than the population average, and negative if the sample average is below the population mean.

What is the probability that a randomly selected group of 15 people will have an average IQ score of 107.9?

Well, it doesn't make sense to ask what is the probability of *exactly* 107.9333333333333... Indeed, $p(107.933...) = 0$, because that question involves calculating the definite integral at a single point (where $dx \approx 0$).

But it is meaningful to ask questions about an *interval* of IQ scores that might be observed in a sample. In the next two subsections, I will show how two such questions are answered using calculus.

18.4.1 Probability of IQ scores in a certain range

Although we cannot calculate the probability of an exact IQ score, we can calculate the probability of a narrow range of IQ scores. So let's ask this question: *What is probability that a randomly selected sample of N=15 have an average IQ score between 107 and 109?*

The equation corresponding to this question is as follows (I'm using p_v for the probability value, to avoid confusion with the pdf function $p(x)$).

$$p_v = \int_{z(107)}^{z(109)} p(x)\,dx \tag{18.15}$$

$$= c(z(109)) - c(z(107)) \tag{18.16}$$

Where $z(107)$ is the z-score corresponding to $\overline{x} = 107$ (Equation 18.14). Be mindful when setting up this problem: The integration is of the pdf $p(x)$ and the bounds are the IQ scores. But the probability is calculated from the cdf — and the cdf is based on z-scores, not raw IQ scores. If you would mistakenly calculate $p(z(109)) - p(z(107))$, then that is actually implementing the

Remember that the cdf is the antiderivative of the pdf: $c(x) = \int p(x)\,dx$.

equation $f(b) - f(a)$, which is not the FTC-2. The FTC-2 is $F(b) - F(a)$.

To obtain this probability value numerically, we need the z-values corresponding to IQ scores of 107 and 109, and then we implement the FTC-2 by subtracting the corresponding cdf values for those two z-values.

```
z107 = (107-mu) / (sigma/np.sqrt(n))
z109 = (109-mu) / (sigma/np.sqrt(n))
prob = stats.norm.cdf(z109) - stats.norm.cdf(z107)
```

Where `mu` is 100, `sigma` is 15, and `n` is 15.[6]

The probability (variable `prob`) is .025, which means that there is a 2.5% chance that a randomly selected group of 15 people will have an average IQ between 107 and 109. It seems rather unlikely — though not impossible — that a randomly selected group of people would have an average IQ score of around 107.9, which means it is likely that our sample of textbook readers has a higher average IQ score.

On the other hand, this interpretation depends heavily on the interval we used. For example, the probability of a random sample from the population having an average IQ score between 107.4 and 108.4 is .013 (1.3%), while the probability of IQ scores between 100 and 118 is nearly 50%.

The conclusion is that although calculating probabilities within intervals around an average value can be insightful if the interval bounds are based on meaningful considerations, it can be difficult to make firm conclusions from this approach because the results depend strongly on potentially arbitrary interval bounds.

[6] It's also possible to shift the pdf and cdf to have $\mu = 100$ and $\sigma = 15/\sqrt{N}$, and then input the raw IQ scores. I show this equivalence in the online code.

18.4.2 Statistical inference

A more common application of the z-test is to obtain a "p-value" to evaluate the statistical significance of the group's average score. A p-value is the probability that the characteristics of our sample were drawn at random from the population, i.e., there is nothing special about our sample of textbook readers.

In the context of our example research question: If there is nothing special about readers of this book (in terms of IQ scores), then we would expect an average IQ score of 107.9 to be likely to be observed when randomly sampling from the population. On the other hand, if readers of this book are, on average, more intelligent than the general population, then we would expect that the probability of observing a group-average IQ score of 107.9 to be very unlikely.

As you already know, we cannot evaluate the p-value corresponding to one specific IQ score. So instead, we ask *what is the probability of a randomly selected group having an average IQ of at least 107.9?* Now we're asking about an average IQ score between 107.9 and ∞.

Mathematically, answering this question corresponds to evaluating the improper integral:

$$p_v = \int_{z_0}^{\infty} p(x)\,dx \tag{18.17}$$

Where z_0 is the z-score measured from our sample (2.048 in this dataset). Graphically, that corresponds to the area of the shaded region in Figure 18.12.

In theory, the area to the right of the dashed vertical line can be calculated from the pdf. However, for numerical accuracy reasons that you will observe in Exercise 4, it's better to calculate the p-value from the cdf. But because the cdf reflects the area of the probability density to the *left* whereas we want the area to the *right*, we actually calculate

```
1 - stats.norm.cdf(zval)
```

This is conceptually equivalent to $c(\infty) - c(z_0)$, because we know

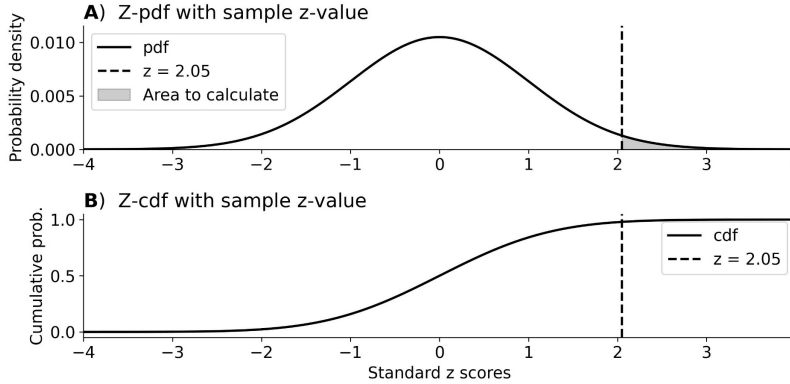


Figure 18.12: Using calculus to calculate a *p*-value.

that the maximum value of the cdf is 1 (that is, $c(\infty) = 1$ by definition).

The p-value here is around .02. This means that there is a 2% chance that a randomly selected group of 15 people have an average IQ score equal to or greater than 107.9. That's very unlikely! If these were real data, we would conclude that our sample of calculus-book readers is unlikely to be a randomly selected group, i.e., they have a statistically significantly higher IQ score than the population average.

By the way, notice that the sample contained three individuals with IQ scores below 100. The question we explored here is whether the group *on average* has an IQ higher than 100, not whether each individual's IQ score is greater than 100.

Statistical inference is a huge topic and there are many important subtleties and myriad other statistical tests. But I hope you see from this section that important questions in statistics are answered using calculus methods that you learned about in this book.

18.5 Data sample distributions

The distributions you've seen so far in this chapter have been smooth and continuous. That's not surprising: they are derived from mathematical functions.

But the empirical sciences involve measuring data from the world, and datasets can be noisy and have limited sample sizes. Therefore, in the context of applied statistics, the distributions introduced here are thought of as expectations about how data might be distributed. Indeed, an important quest in many statistics applications is to find the distribution that best characterizes a particular dataset.

You can generate random numbers drawn from a particular distribution using the `scipy.stats` library. For example, the following code will generate 300 random numbers drawn from a Laplace distribution using the function's default parameters (`rvs` stands for "random variates").

```
data = stats.laplace.rvs(size=300)
```

Figure 18.13 shows an example of random data drawn from a Laplace distribution. Panel A shows a scatter plot of the 5000 data samples. Panel B might look like a pdf, but it's not a true pdf — it is an empirical distribution of the data, generated by creating a histogram: bin the data domain into 40 intervals, count the number of data values in each interval, and draw the y-axis value corresponding to the number of data values per interval. This histogram can be considered an empirical estimate of a pdf. Indeed, I normalized the data count values such that the sum of the histogram values is 1, just like a pdf. The cumulative sum in panel C is therefore the estimate of the cdf.

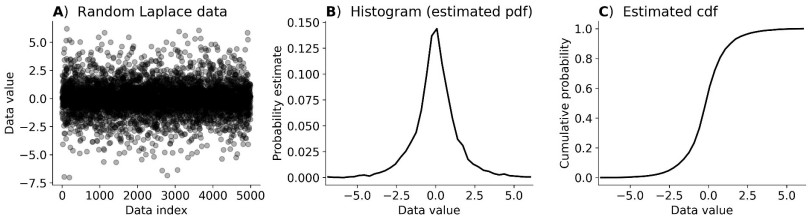


Figure 18.13: Random numbers drawn from a Laplace distribution.

Next, I re-ran the code but generated random numbers from a Pareto distribution (Figure 18.14). The estimated pdf does indeed have a shape consistent with the theoretical Pareto pdf shown in Figure 18.6.

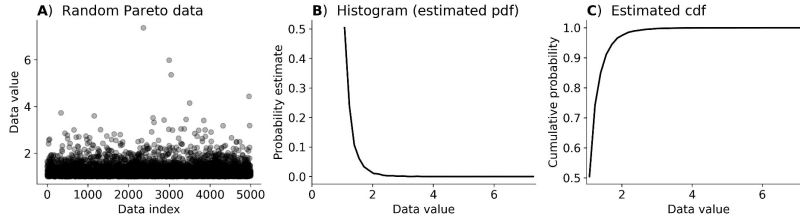


Figure 18.14: Random numbers drawn from a Pareto distribution.

18.6 Statistical moments

18.6.1 What are moments?

A statistical "moment" is a numerical value that characterizes a pdf or a data distribution. There are multiple moments of a distribution that describe different characteristics. For example, you've probably heard of the terms *mean* and *variance*; these are the first and second moments of a distribution. The third and fourth moments are called *skew* and *kurtosis*; perhaps you've also come across those terms.

Statistical moments are important because they provide quantitative information about a distribution or dataset.

Indeed, it is no understatement to write that most real-world statistical analyses have the goal of determining whether the moments of two samples are significantly different from each other, or are different from a reference value (as in the example of using the z-test to evaluate average IQ scores relative to a population value).

Figure 18.15 (taken from Chapter 4 of my book *Modern Statistics*) lists the names and interpretations of the first four moments, and visualizes what they indicate about a distribution.

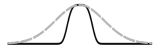
Moment	Stats name	Interpretation	Picture
First	Mean	Average	
Second	Variance	Dispersion	
Third	Skew	Asymmetry	
Fourth	Kurtosis	Tail fatness	

Figure 18.15: Table of the first four moments.

18.6.2 Calculating analytic and empirical moments

Moments are calculated by integrating the pdf multiplied by the data value raised to a positive integer value[7].

There are two equations below; the first is the analytic definition of a moment, and the second is how moments are calculated in empirical sample datasets. The difference is that Equation 18.18 is applied to a mathematical formula that defines a pdf, whereas Equation 18.19 is applied to a dataset of numerical values in which the generating pdf might be unknown.

$$\mu_k = \int_{-\infty}^{\infty} x^k \, p(x) \, dx \tag{18.18}$$

$$M_k = \frac{1}{N} \sum_{i=1}^{N} x_i^k \tag{18.19}$$

In words: raise a variable to the k^{th} power, scale by its probability density, and integrate over all possible values.

[7]Moments can also defined for non-integer powers, but positive integers are more commonly used and interpreted.

Where is the $p(x)$ in Equation 18.19? In real-world data, we don't necessarily know — or don't want to make assumptions about — the true pdf of the system that produced the data, and so the $p(x)\,dx$ is replaced with the normalization constant $1/N$.

These equations are called the "raw" or "unstandardized" moments. The scale of the data is retained in the moment, which means, for example, that the numerical value of a statistical moment of height depends on whether height is measured in mm or inches, even though the shape of the distribution doesn't change.

For this reason, there are "standardized" moments that are calculated by including two additional normalization factors: mean-centering the data, and dividing by the standard deviation raised to the k^{th} power.

$$\mu_k = \frac{1}{\sigma^k} \int_{-\infty}^{\infty} (x - \mu)^k \, p(x) \, dx \tag{18.20}$$

$$M_k = \frac{1}{N s^k} \sum_{i=1}^{N} (x_i - \overline{x})^k \tag{18.21}$$

For analytic moments (Equation 18.20), μ and σ are the known population mean and standard deviations. In empirical data, the population characteristics are unknown (or at least, not assumed), and therefore these parameters are replaced by $\overline{x}$ for the mean of the dataset, and s for the standard deviation of the dataset (Equation 18.21).

18.6.3 Moments of pdfs

Statistical moments of analytic distributions can be obtained by integrating the pdf according to Equation 18.18. Many distributions have a "moment-generating function," which is like a shortcut that transforms the pdf into a form that is easier to integrate. Not all distribution functions have an analytically solvable

moment-generating function, and there are various techniques to solve for analytic moments that I won't go into here.

You can obtain the moments of analytic distributions in `sympy` or `scipy`. For example, the code `stats.norm.stats(moments='mvsk')` will return a 4-element tuple containing the first four moments of a normal distribution (`m` for mean, `v` for variance, `s` for skew, and `k` for kurtosis).

Figure 18.16 shows three examples of analytic distributions and their first four moments.

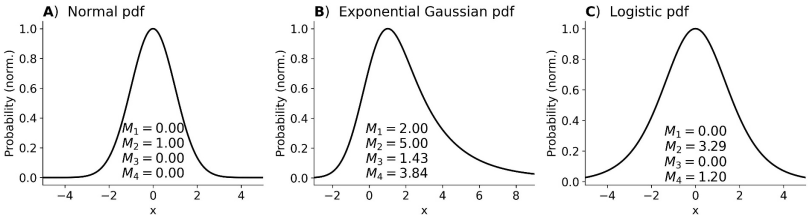


Figure 18.16: Three examples of pdfs and their first four moments.

For an even-symmetric pdf, M_1 (the mean) is the center of the distribution, and M_3 (skew) is zero. Non-symmetric pdfs will have non-zero skew values. For example, the positive M_3 for the exponential Gaussian distribution in panel B is the quantification that the right tail is "fatter" than the left tail. Kurtosis reflects the symmetric "fatness" of the tails, and is zero by definition for a normal distribution. The positive kurtosis for the logistic pdf reflects that it is wider (falls to zero more gradually) compared to the standard normal distribution.

18.6.4 Moments of numerical data

Notice that we didn't need any data to calculate the moments in the previous subsection. In the empirical sciences, moments are usually not known in advance, and instead are calculated numerically based on data.

The interpretations of the moments, however, are the same for

analytic and empirical distributions.

Let's work with a dataset comprising the integers 1-10:
$x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$

The first moment is the average of the sample, and is obtained using the unstandardized first moment: Sum the data values (formally, each data value is raised to the 1^{th} power, but of course we can ignore that) and divide by the number of data points. The first moment is 5.5. The standardized moments subtract the mean, and thus the standardized first moment is trivially zero. That's why the first moment is left unstandardized for interpretation.

Higher moments are typically normalized. The second moment is the variance, is indicated as σ^2, and is normalized only by mean-centering.

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \overline{x})^2 \tag{18.22}$$

The variance of dataset x is 8.25.

After M_2, the standardized equation is used (Equation 18.21). The skew (M_3) is zero because this dataset has no asymmetries around its mean. The kurtosis (M_4) reflects the "fatness" of the tails relative to a normal distribution. This dataset is flatter than a Gaussian and its kurtosis is -1.2.

18.7 Exercises

18.1. The goal of this exercise is to explore creating a Gaussian cdf from the pdf. Start by creating a pdf numerically by implementing Equation 18.3 in `numpy`. Then create three cdfs: (**1**) via the cumulative sum in `numpy` applied to the pdf you created; (**2**) via one of `scipy.integrate`'s cumulative sum functions applied to the pdf you created; (**3**) via the `scipy.stats` module using the `cdf` method.

Plot all results as in Figure 18.17. If your results don't match, think carefully about the dx normalization.

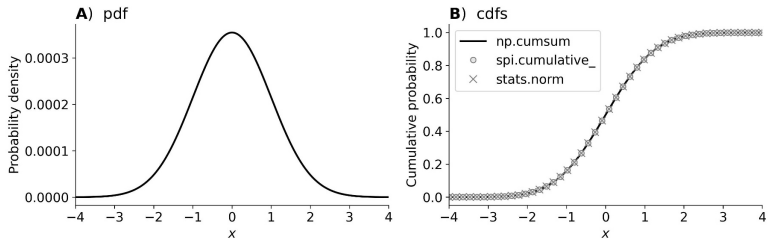


Figure 18.17: Results for Exercise 1. The dots and x's were downsampled to facilitate visual inspection.

Generating the cdf from `stats.norm.cdf` requires the least amount of code — and this is not just about the number of lines of code; translating complicated mathematical equations into code risks introducing coding bugs that could cause systematic biases or errors. So why would you ever calculate the empirical cdf when you can generate it from an established library? Segue to the next exercise...

18.2. The answer to the question at the end of the previous exercise is that you should prefer to generate a cdf from a mathematical formula using `scipy` or `sympy` when possible.

But not all cdfs come from mathematical equations. The empirical sciences are full of pdfs and cdfs that are mea-

sured from data. In some cases, the underlying generative distribution is known, but there are situations in which the underlying probability distribution is unknown. In fact, the goal of the scientific research might be to acquire data in order to characterize an unknown distribution.

The goal of this exercise is to create a simulated dataset in which you combine random numbers from two distributions, then calculate the empirical pdf and cdf.

The combined dataset should have $N = 2000$ samples, half drawn from a logistic pdf and half drawn from a Wald pdf. Use the `rvs` methods in the `scipy.stats` library to generate the random numbers.

Calculate the empirical pdf and cdf estimates and make a plot like Figure 18.18. For bonus points, intermingle the data values so it's not obvious from the scatter plot (panel A) that the data come from two distributions.

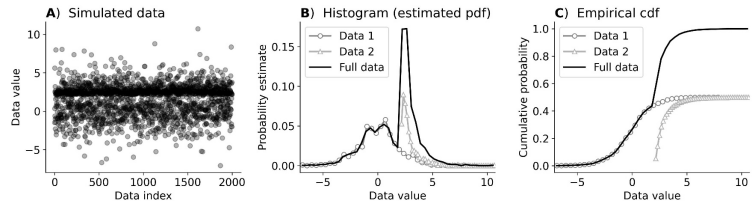


Figure 18.18: Results for Exercise 2.

Of course, this is an artificial dataset that we created based on known distribution functions, but real data can have unusual-looking distributions that don't conform to any particular analytic distribution.

18.3. The goal of this exercise is to produce a triangular pdf in two ways: (**1**) by numerically differentiating the cdf calculated from Equation 18.9 (that is, first create the cdf in `numpy`, then calculate its numerical difference), and (**2**) by using the `triang.pdf` function in `scipy.stats`.

The `triang` function does not simply take the a, b, and d

parameters, so you will need to consult the function docstring and possibly do a bit of algebra to convert between the parameters and the function inputs.

Figure 18.19 shows the resulting pdfs. The pdfs clearly show the triangular shape of the probability function, which was difficult to discern from visual inspection of the cdf.

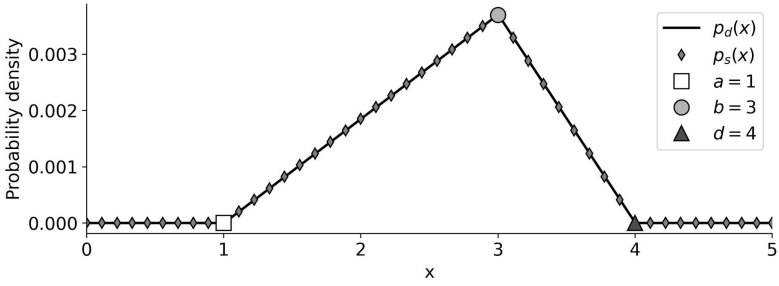


Figure 18.19: Results for Exercise 3. $p_d(x)$ is the pdf obtained via the numerical difference of the cdf, and $p_s(x)$ is the output of the `triang.pdf` function, which I downsampled to facilitate visual comparison.

18.4. In the section on the z-test, I wrote that it's better to calculate the p-value from the cdf instead of from the pdf, although in theory the resulting p-values should be identical.

Calculate the p-value from $z_0 = 2$ using three calculations. First, using `stats.norm.cdf`. Second, from a pdf created from `stats.norm.pdf` and using `numpy` to sum the values right of z_0. Third, from the same pdf but using `spi.trapezoid` to calculate the definite integral. My results are below; your cdf value should match mine exactly, but the other two values may or may not match, depending on parameters you select that I will discuss below.

```
From cdf      : 0.02275013194817921
From np.sum   : 0.024067985483538966
From spi.trap: 0.022770905206945864
```

We can consider the cdf method to be the most precise.

The trapezoid method in `scipy` was more accurate than summing pdf values in `numpy`, which shouldn't be surprising after the discussions about this from earlier in the book (e.g., Chapters 14 and 15).

The difficulty is this: a p-value from the pdf is the definite integral between $z = 2$ and $z = \infty$. But you don't simulate x-axis numbers going up to ∞; instead, you pick some reasonably large number, and a reasonably large number of sample points (that is, a reasonably small dx), and assume that the density of values larger than, say, $z = 50$ make such a tiny contribution that they can be ignored in the calculations. That's why my code to get the p-value from a pdf looks like this:

```
zz = np.linspace(zval,50,1000)
pdf = stats.norm.pdf(zz)
pFromPdf_sp = spi.trapezoid(pdf,x=zz)
```

In other words, the theory tells us to evaluate the *improper* integral from $z = 2$ to $z = \infty$, but in practice I've calculated the *proper* integral from $z = 2$ to $z = 50$. You can see from the results above that the calculations are quite accurate in this example (at least when using `spi.trapezoid`), although it's better to use a more-accurate rather than less-accurate algorithm.

Finally, spend a few minutes exploring the code. For example, what happens to the accuracy of the two pdf-based methods with 100 instead of 1000 sample points for `zz`?

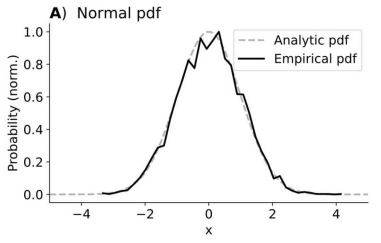
18.5. The purpose of this exercise is to compare the statistical moments calculated analytically from equations, with the statistical moments calculated empirically from random data drawn from those distributions.

Start with the normal (Gaussian) pdf. Generate a pdf and its first four moments using `scipy`'s `stats` library. Then, generate 5000 random data values from a normal distribu-

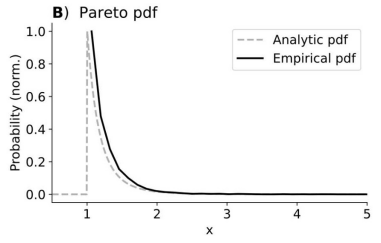
tion, and calculate the first four moments empirically using functions in `numpy` and `scipy`.

Show both the analytic pdf and the empirical approximation of the pdf as in Figure 18.20A, and create a table of the first four statistical moments calculated from the data and the pdf.

Normalize both pdfs so that they have a maximum value of 1. Although this normalization prevents a direct interpretation of the y-axis values, it facilitates qualitative comparisons between different functions.



Source:	Mean	:	Var.	:	Skew	:	Kurtosis
Data:	0.02	:	1.02	:	-0.00	:	-0.07
pdf :	0.00	:	1.00	:	0.00	:	0.00



Source:	Mean	:	Var.	:	Skew	:	Kurtosis
Data:	1.25	:	0.09	:	3.80	:	28.87
pdf :	1.25	:	0.10	:	4.65	:	70.80

Figure 18.20: Results for Exercise 5.

The empirical results match the expected values quite well (still referring to panel A). Minor discrepancies are attributable to a limited sample size, and are non-systematic (that is, each time you generate new random data, the differences might be a bit smaller or a bit larger compared to the analytic result).

On the other hand, the empirical pdf (black line) is noticeably domain-limited compared to the analytic pdf (dashed gray line). That is, the pure mathematics are telling us to expect some data for $|x| > 4$, but there are no random values that large. I'll get back to this point in a moment[8].

Now for panel B. As you can see, it's the same code but using a Pareto distribution. Use $b = 5$ and a sample size of 5000. Your analytic results should match mine, and

[8] Yes, that is a pun. Yes, I know it's terrible. And no, I won't apologize.

the random data results should be comparable though not identical.

What do you notice about the results for the Pareto distribution? The pdfs appear to match, although there is no data for $x < 1$, which is numerically though not visually consistent with the analytic pdf. More importantly, the first two moments match quite well, but the skew and especially the kurtosis are really different.

Why is there such a discrepancy? There's nothing wrong with the code. A hint into the reason comes from Figure 18.6 (page 623) and the discussion about why the sum of the pdf values were less than 1. I will write the answer in the next paragraph, so stop reading here if you want time to think about it first.

Here's the answer: The higher statistical moments like kurtosis reflect extreme values in a distribution (after all, they come from raising the data to the fourth power, so large data values become *really* huge). The Pareto distribution, in theory, contains a very small number of very large values. But in limited samples of random data, very large values are very rare. A sample size of 5000 may seem large, but it's not nearly enough data to sample such unlikely events (for example, what is the chance of having a billionaire in a random sample of 5000 people?).

With that in mind, try re-running the code for the Pareto distribution with a larger sample size, like 50k or 5 million. The empirical kurtosis will increase compared to the $N = 5000$ simulation, though still not as high as the theoretically expected value of 70. (In fact, I tried it with 50,000,000 samples and still only got a kurtosis of 58.)

This systematic underestimation of unlikely events highlights a major distinction between pure math and applied math. For purposes of this book, that's just a curiosity, but if you are doing research using simulated random data (e.g., developing economics models), this discrepancy might be

a crucial factor to account for.

18.6. Consider the following two definite integrals, where $p(x)$ corresponds to the normal distribution. What is your intuition about which area is larger?

$$A_1 = \int_1^2 p(x)\,dx \tag{18.23}$$

$$A_2 = \int_2^\infty p(x)\,dx \tag{18.24}$$

Area A_1 is a proper definite integral with an interval of one unit, whereas A_2 is an improper integral and spans an infinite interval.

After making an hypothesis about which area is bigger and why, calculate the two integrals using `scipy` and report the results as below (obviously filling in the actual numerical percentages in place of `xx.xx`). I will make several remarks about the results and show a figure below, so don't look ahead if you want to maintain some suspense about the rest of the exercise...

```
Area between z = 1 and z = 2:  xx.xx%
Area between z = 2 and z = oo: xx.xx%
```

Are you surprised that the area between 1 and 2 is around six times larger than the area between 2 and ∞? Perhaps by this point in the book you're not so surprised. But anyway, the next part of the exercise will quell any remaining disbelief.

Draw a pdf of the normal distribution, with the two z values and areas to calculate drawn in patches, as in Figure 18.21. This visualization makes it clear that the reason why the second area is so small, despite the interval of integration being incredibly wide, is that the probability values decay to zero incredibly quickly.

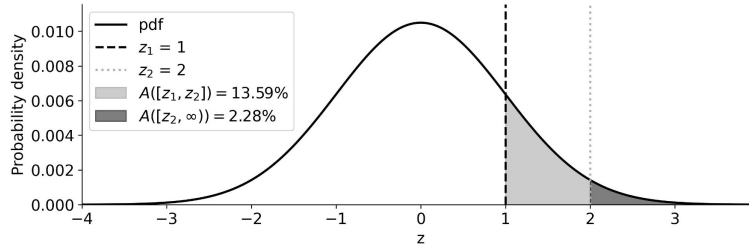


Figure 18.21: Results for Exercise 6.

18.7. This exercise is open-ended. Go to the `scipy` statistics library website. The url is below, but you can also find it by web-searching for "scipy stats."
`https://docs.scipy.org/doc/scipy/reference/stats.ht`

Have a look through the list of "continuous distributions." These are the analytic distributions available in `scipy`. Next, copy the code that creates Figure 18.9, and modify the figure to explore some distributions that you think have interesting-looking names.

There are no right or wrong answers; this is just an opportunity for you to explore some of the various distributions that mathematicians have developed and that `scipy` developers have implemented.

Chapter 19

Multivariable integration

19.1 Terms and concepts

19.1.1 Terminology

Similar to multivariable differentiation, the terms "multivariate integration," "multivariable integration," and "multiple integration" can be used interchangeably. In some disciplines the terms can connote different kinds of problems, but you don't need to worry about that here. Other specific terms can indicate the number of variables under consideration, for example "double integration" refers to integrating two-variable functions while "triple integration" refers to integrating three-variable functions.

Take a moment to study Figure 19.1. I will first describe the left column. This is a two-variable integral: There are two integration signs, two d terms, and the function includes two variables. The "inner integrand" refers to the single-variable integral with the inner-most variable. In this example, variable x is the inner integrand because the dx appears "inside" of the dy.

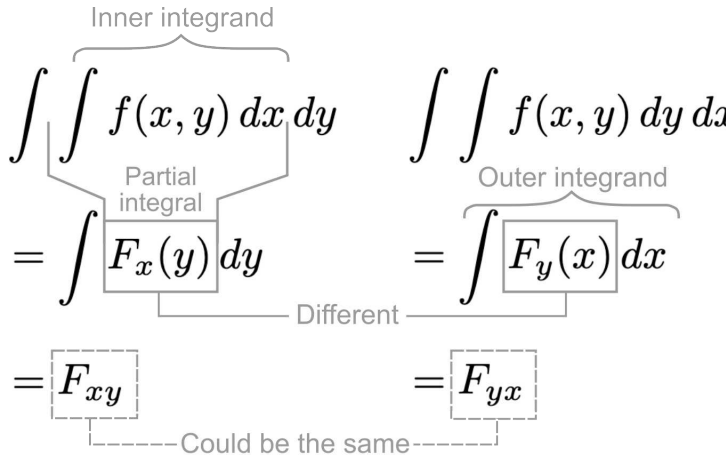


Figure 19.1: Terms and concepts of a multivariable integration.

The integral of one variable is called a "partial integral" (the analog to the partial derivative), and can be indicated as F_x for the partial integral with respect to x.

I will explain in the next subsection how to calculate partial in-

tegrals, but the upshot is that you integrate with respect to one variable while treating the other variables as constants — exactly the same as differentiating multivariable functions.

The right column shows integrating the same function but swapping the order of the variables. This produces a partial integral $F_y(x)$.

Recall that with multivariable differentiation, the double-derivative is the same regardless of the order in which the variables are differentiated (that is, $f'_{xy} = f'_{yx}$). With double integrals, the situation is a little more nuanced.

I'll get into the details in later sections, but briefly: When calculating *definite* integrals, the two results are the same. That is, when both integrals have integration bounds to evaluate, F_{xy} and F_{yx} are identical. When calculating *indefinite* integrals, however, then the two double integrals could be the same if the constants of integration can be resolved in a way that makes them equal. But it is possible that the antiderivatives F_{xy} and F_{yx} differ because of constants of integration.

19.1.2 Multivariable definite integrals and volume

As you know, a single-variable function can describe a curve, and the definite integral of that function reflects the *area* between that curve and the x-axis.

A two-variable function can describe a surface floating on top of the x-y plane, and the definite integral of that two-variable function reflects the *volume* under that surface (Figure 19.2).

19.2 Multivariable indefinite integrals

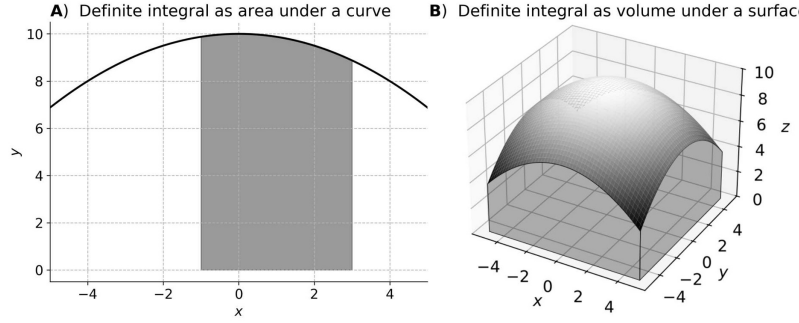


Figure 19.2: The definite integral of a single-variable function is the *area* under the curve representing that function (shaded region in panel A). Analogously, the definite integral of a two-variable function is the *volume* under the surface representing that function (shaded region in panel B).

Integrating multivariable functions is no different from integrating single-variable functions, but you need to keep track of which variable is currently being integrated.

Let's start by considering the following problem. (For now, ignore the constants of integration; I'll get back to them later.)

$$\iint \Big(3x + xy\Big)\, dx\, dy \tag{19.1}$$

Parentheses added for visibility.

To integrate a multivariable integral, start by determining which variable corresponds to the inner integrand. In Equation 19.1, the inner integrand is $\int \ldots dx$, in other words, we first integrate with respect to variable x. I've added brackets and spaces to Equation 19.2 to highlight the inner integrand.

$$\int \quad \Big[\int 3x + xy\, dx\Big] \quad dy \tag{19.2}$$

When calculating the partial integral with respect to x, treat y as if it were a constant. Thus, the partial integral $F_x(y)$ is:

$$F_x(y) = \frac{3x^2}{2} + \frac{x^2 y}{2} \tag{19.3}$$

Notice that y remains in the partial integral, but it isn't integrated. Thus, think of y as just a symbol representing a constant, the same way you'd think of π or $\sqrt{3}$.

That was half of the work. The other half is to finish the integration on variable y, treating the x terms as constants.

$$\int \frac{3}{2}x^2 + x^2\frac{y}{2}\, dy \;=\; \frac{3x^2 y}{2} + \frac{x^2 y^2}{4} \tag{19.4}$$

Notice now that the first term $(3x^2/2)$ got multiplied by y in the integral. Again, when integrating over y, x is treated as a constant. Imagine, for example, setting $\alpha = 3x^2/2$, and so $\int \alpha\, dy = \alpha y$.

Now I'd like you to confirm that $F_{xy} = F_{yx}$ (again, ignoring the constants of integration). In other words, calculate the following double integral and confirm that you reproduce the right-hand side of Equation 19.4.

$$\iint 3x + xy\, dy\, dx \tag{19.5}$$

19.2.1 Constants of integration

The equations below re-integrate the example from above, but this time including the constants of integration. Watch what happens in the final result.

$$\iint 3x + xy\, dx\, dy \;=\; \int \left[\int 3x + xy\, dx\right]\, dy \tag{19.6}$$

$$= \int \left[\frac{3}{2}x^2 + \frac{x^2 y}{2} + C(y)\right]\, dy \tag{19.7}$$

$$= \frac{3x^2y}{2} + \frac{x^2y^2}{4} + C(y) + D \tag{19.8}$$

$$= \frac{6x^2y + x^2y^2}{4} + C(y) + D \tag{19.9}$$

You might be wondering why Equation 19.7 has $C(y)$ instead of just C. Here's the deal: We have to treat the constant of integration as a *function*. To be clear: it's a *constant* with respect to x, but we don't know if it's *really* a constant, or actually a function of y that seems like a constant from the perspective of x because it doesn't vary with x.

In other words, C acts like a "constant" with respect to x (or whatever variable we're integrating over), but it might vary with other variables. Therefore, we must assume that it *could* be a function of y. For example, the function $f(x, y) = 4y + 2$ is a function of x and y, but that function doesn't vary with x, so it seems like a constant from the perspective of x.

On the other hand, D really is a constant, not a function, so we cannot simply combine $C(y) + D = E$ and assume that E is a constant with respect to both x and y.

Now let's integrate again but swapping the order of variables:

$$\iint 3x + xy\,dy\,dx = \int \left[\int 3x + xy\,dy\right] dx \tag{19.10}$$

$$= \int \left[3xy + \frac{xy^2}{2} + E(x)\right] dx \tag{19.11}$$

$$= \frac{6x^2y + x^2y^2}{4} + E(x) + F \tag{19.12}$$

Again, the "constant" E seems like a constant with respect to y, but we don't know whether it's a constant with respect to x. Therefore, we write it down as an unknown function of x. Because it's an unknown function, we don't need to multiply it again by x during integration. That is, until we can resolve $E(x)$, it might be *any* function of x that could include x^2 terms.

The question at hand is whether Equations 19.9 and 19.12 are equal. That is the same as asking whether $C(y) + D = E(x) + F$. On the one hand, D and F are both arbitrary constants that arose after all integrations; therefore, they are constants with respect to all variables, and we can safely combine them into one unknown constant. Thus, the question is whether $C(y) = E(x)$. The answer is that if both are constants, then we can combine all four constants of integration into one — and then $F_{xy} = F_{yx}$. But if one or both functions are not constants but instead are functions of one variable, then the antiderivatives from the two integration orders can differ.

Regardless of the outcome, F_{xy} and F_{yx} are related to each other — they both come from the same "parent" function and share all terms except for the integration constants.

This issue can cause confusion and replication failures in numerical approximations of indefinite integrals. I will return to this discussion, along with options for "normalizing" the antiderivatives, in Section 19.3.

Another example Here's the example equation:

$$f(x, y) \;=\; 3x^2y + 5\sin(x) + 2ye^y + 4 \tag{19.13}$$

Please calculate the double indefinite integral of this function. Integrate twice, once with dx as the inner integral, and once with dy as the inner integral. Make sure you get the same result except for the constants of integration.

My solution is below so you can check your work. I'll start by integrating first with respect to x.

$$\int f(x, y)\, dx = \int 3x^2y + 5\sin(x) + 2ye^y + 4\, dx \tag{19.14}$$

$$= x^3y - 5\cos(x) + 2xye^y + 4x + C(y) \tag{19.15}$$

$$\iint F_x\, dy = \int x^3y - 5\cos(x) + 2xye^y + 4x + C(y)\, dy$$

$$= \frac{x^3y^2}{2} - 5y\cos(x) + 2xye^y - 2xe^y + 4xy + C(y) + D \tag{19.17}$$

And now to integrate using the other order.

$$\int f(x,y)\,dy = \int 3x^2y + 5\sin(x) + 2ye^y + 4\,dy \tag{19.18}$$

$$= \frac{3x^2y^2}{2} + 5y\sin(x) + 2ye^y - 2e^y + 4y + E(x) \tag{19.19}$$

$$\iint f(x,y)\,dy\,dx = \int \frac{3x^2y^2}{2} + 5y\sin(x) + 2ye^y - 2e^y + 4y + E(x)\,dx \tag{19.20}$$

$$= \frac{x^3y^2}{2} - 5y\cos(x) + 2xye^y - 2xe^y + 4xy + E(y) + \tag{19.21}$$

Whew! That's a lot of tedious work, but I hope you agree that it wasn't so difficult if you stayed focused on which variables are to be integrated and which are treated as constants.

Needless to say, once you understand the concepts of multivariable integration, it's handy to have Python deal with the details while you focus on explorations, intuitions, and applications.

19.3 Visualizing double partial antiderivatives

A function of two variables has two partial integrals and one full (double) integral, and therefore can be visualized using four images.

On the other hand, non-zero constants of integration actually create an infinite number of possible indefinite integrals. The con-

stants can be set to zero, or can be determined by a normalization procedure (typically involving finding the constants that set $F(0,0) = 0$).

In this section, I will show you how to obtain and visualize partial indefinite integrals using `sympy` and `scipy`.

19.3.1 Using `sympy`

Let's start with an example where `sympy` can analytically calculate the integrals. I'll use Equation 19.1, reprinted below.

$$\iint 3x + xy \, dx \, dy \tag{19.1}$$

Here's the Python code to get the partial and double integrals in both orders.

```
x,y = sym.symbols('x,y')
f_xy = 3*x + x*y

f_int_x = sym.integrate(f_xy,x)
f_int_y = sym.integrate(f_xy,y)
f_int_xy = sym.integrate(f_int_x,y)
f_int_yx = sym.integrate(f_int_y,x)
```

As you know from previous chapters, `sympy` does not automatically add constants of integration to indefinite integrals, so we would expect variables `f_int_xy` and `f_int_xy` to equal each other. Figure 19.3 shows latex-formatted output of the variables calculated in the `sympy` code above. From a quick glance, it appears that F_{xy} and F_{yx} differ, but they are indeed the same, just with the x^2 term factored out.

$$f(x,y) = xy + 3x$$

$$F_x = x^2\left(\frac{y}{2} + \frac{3}{2}\right)$$

$$F_y = \frac{xy^2}{2} + 3xy$$

$$F_{xy} = \frac{x^2y^2}{4} + \frac{3x^2y}{2}$$

$$F_{yx} = x^2\left(\frac{y^2}{4} + \frac{3y}{2}\right)$$

Figure 19.3: `sympy` output for this example.

Next, lambdify the expressions so we can evaluate them in `numpy`.

```
f_xy_l = sym.lambdify((x,y),f_xy)
f_int_x_l = sym.lambdify((x,y),f_int_x)
f_int_y_l = sym.lambdify((x,y),f_int_y)
f_int_xy_l = sym.lambdify((x,y),f_int_xy)
```

Armed with the lambdified functions, you can use standard `matplotl` code to visualize the matrices. A minimum-worked example is below, and the online code will reproduce Figure 19.4.

```
xx = np.linspace(-10,3,151)
X,Y = np.meshgrid(xx,xx)
plt.imshow(f_xy_l(X,Y))
```

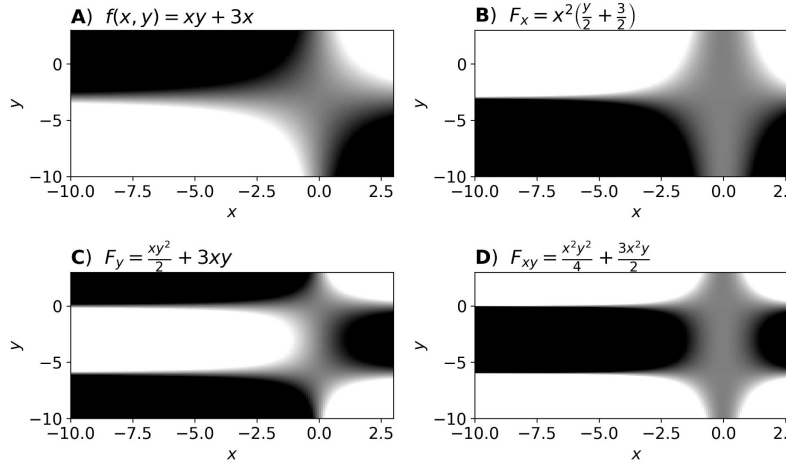


Figure 19.4: Visualizing a two-variable function and its indefinite integrals. Black pixels correspond to negative values, white pixels correspond to positive values, and gray pixels indicate values close to zero.

19.3.2 Using `scipy`

Example 1: Reproducing the `sympy` result If you need to integrate a function that either cannot be analytically integrated or that comes from measured data instead of a mathematical function, you can numerically approximate the indefinite integral using cumulative summation methods in `scipy`. The general

approach is the same as what you learned for single-variable functions, although there are a few additional steps.

I'll begin by reproducing Figure 19.4 using `scipy`. The first step is to setup the equation. I'll use the lambdified `sympy` expression for the original function, but will numerically approximate the integral rather than relying on the analytic solution.

```
xx = np.linspace(-10,3,81)
dx = xx[1] - xx[0]
X,Y = np.meshgrid(xx,xx)
f_xy = f_xy_l(X,Y)

F_x = spi.cumulative_simpson(f_xy,dx=dx,axis=1)
F_y = spi.cumulative_simpson(f_xy,dx=dy,axis=0)
F_xy = spi.cumulative_simpson(F_x,dx=dx,axis=0)
```

Variables `F_x`, `F_y`, and `F_xy` are matrices, and so can be visualized using the same `imshow` code as for the result of the `sympy` calculations. If calculus works the way we think it should, then Figure 19.5 should be a near-perfect match to Figure 19.4.

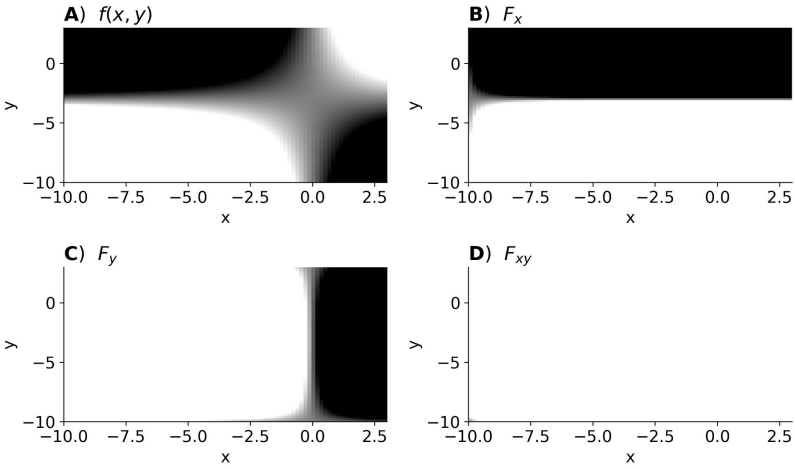


Figure 19.5: First attempt at reproducing `sympy`'s result (in Figure 19.4) using numerical methods in `scipy`.

But they look *really* different. Actually, the function itself looks the same, so the issue is with the partial integrals calculated by

`scipy`.

What's causing these huge discrepancies?

The issue lies with the constants of integration. If we know the actual equations for the partial integrals, we can simply ignore the constants by evaluating the functions for any given values of x and y. For example, in this case, the partial and double integrals all equal zero when $x = y = 0$ (consult Figure 19.3).

But numerical approximations are calculated from numerical values, not equations. Therefore, we can "normalize" the partial integral maps by subtracting the values across the rows or columns to ensure that, for example, $F_x(0, 0) = 0$.

The code below implements this normalization. The code identifies the column (for F_x) or the row (for F_y) that corresponds to $y = 0$ or $x = 0$, and subtracts that vector from all other columns or rows.

```
F_x -= F_x[:,np.argmin(abs(xx))].reshape(-1,1)
F_y -= F_y[np.argmin(abs(xx)),:].reshape(1,-1)
F_xy -= F_xy[np.argmin(abs(xx)),:].reshape(1,-1)
F_xy -= F_xy[:,np.argmin(abs(xx))].reshape(-1,1)
```

That "improves" the results (Figure 19.6). I write "improve" in apology quotes because the results shown in the previous figure were equally valid, though inconsistent with `sympy`'s output. The only difference is that the normalized results are constrained to equal zero at the origin.

The limitation of this approach is that we cannot always know the analytic partial derivatives, perhaps because they are generated from empirical scientific data in which the generating equations are unknown, or perhaps because the equation is too complicated to integrate analytically.

Thus, if you have no a priori constraints on the partial integrals that allow you to solve the initial value problem, you can normal-

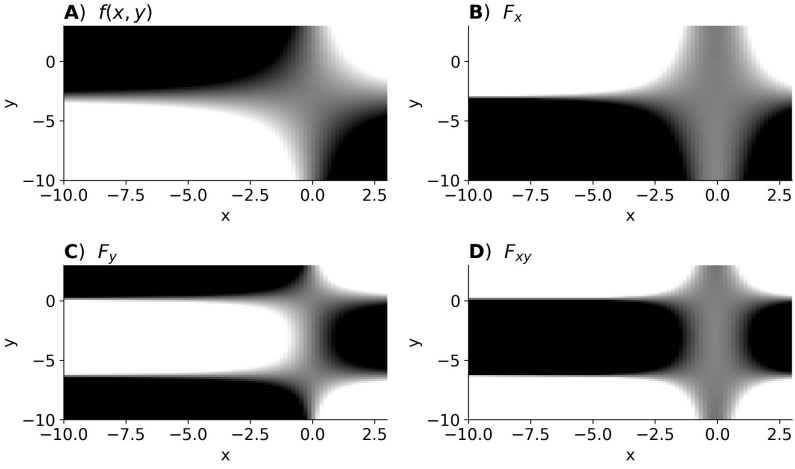


Figure 19.6: Reproducing sympy's result with normalization.

ize the graphs such that $F_x(0,0) = F_y(0,0) = F_{xy}(0,0) = 0$

That's a reasonable strategy, and it does help ensure that the graphs will be visually interpretable. But keep in mind that this is merely one of myriad possible valid solutions to an indefinite integral. This is exactly the same concept as how there is an infinite number of equally valid antiderivatives to a single-variable function, and all solutions are related to each other by a "shift" in the function from different constants of integration.

Example 2 This next example is more difficult to integrate analytically. It's a variant of Equation 19.13.

$$f(x,y) = 3x^2y + 5\sin(xy) + 2ye^{-y^2} + 4 \tag{19.22}$$

The code to implement and visualize this function's integrals is basically just copy/pasted from the previous example, with some minor modifications to the axis limits and color saturation values.

Without having access to the analytic partial integrals, we can

choose to normalize the resulting numerical approximations to have zeros at the origins of the graphs. The normalized partial and full integral maps are shown in Figure 19.7.

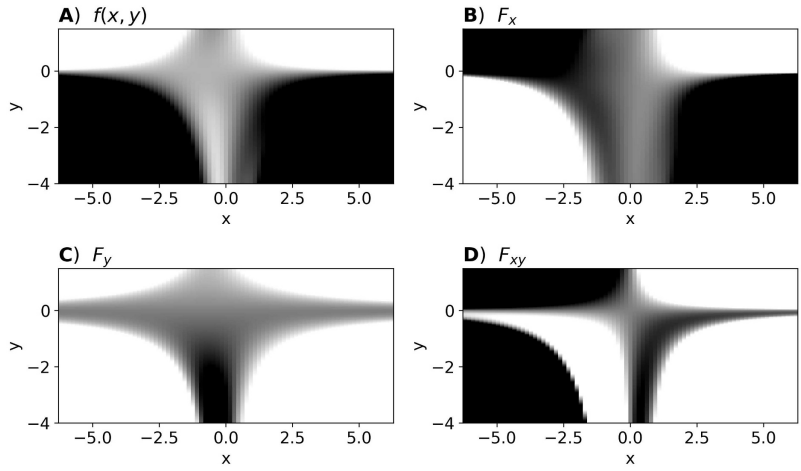


Figure 19.7: A second example, with normalization.

But why even bother with the normalization? Why not just show the "raw" cumulative sums?

Here's the reason: The cumulative sum for each value of x_0 depends on the numerical values evaluated for $x < x_0$. Now, an analytically determined indefinite integral considers all values from $-\infty$ to x_0, but numerical approximations only consider whatever values we happened to evaluate (for example, starting at $x = -10$ in the previous example, and $x = -6$ in this example).

This means that the resulting numerical approximation will depend on whether we numerically evaluate the function starting from $x = -4$, $x = 0$, $x = -20$, and so on. And it gets even more confusing, because the impact of those starting values of x may depend on the starting values of y.

Thus, if we normalize the entire partial integral map to be zero at the origin, then the map will look the same regardless of the x and y bounds that we choose for the numerical function evaluations and integral approximations.

On the other hand, forcing the integrals to be zero at the origin

is a choice that we make for convenience and consistency, not a constraint given to us by some provable mathematical theory.

19.4 Multivariable definite integrals

The main goal of this section is to verify the following:

$$\int_0^2 \int_{-1}^3 \left(2x + y^2\right)\, dx\, dy \;=\; \frac{80}{3} \tag{19.23}$$

How do you calculate a double integral? It's a similar procedure as for multivariable indefinite integrals: Start with the inner integral and treat the non-integrated variable (in this case, that's y) as a constant, and then integrate a second time for the outer integral. The only difference with definite integrals is that the inner integral ends up as a function of y with all x terms resolved.

Let's work through this example piece by piece. Start by integrating the inner integrand:

$$\int_{-1}^3 2x + y^2\, dx = x^2 + xy^2 \Big|_{-1}^3 \tag{19.24}$$

$$= \left(3^2 + 3y^2\right) - \left((-1)^2 + (-1)y^2\right) \tag{19.25}$$

$$= 9 + 3y^2 - 1 + y^2 \tag{19.26}$$

$$= 8 + 4y^2 \tag{19.27}$$

Notice that the result of the definite integral on variable x is a single-variable function of y (F_y). The next step is to integrate Equation 19.27 using the integration bounds defined by the outer integral in Equation 19.23.

$$\int_0^2 8 + 4y^2\, dy = 8y + \frac{4y^3}{3} \Big|_0^2 \tag{19.28}$$

$$= \left(8(2) + \frac{4(2^3)}{3}\right) - 0 \tag{19.29}$$

$$= 16 + \frac{32}{3} \tag{19.30}$$

$$= \frac{48}{3} + \frac{32}{3} = \frac{80}{3} \tag{19.31}$$

As I wrote earlier in this chapter, we'd get the same answer if we swap the order of integration (assuming the function is continuous within the integration bounds, which is the case in this example).

I encourage you to work through this example again to confirm that the definite integral after swapping the order of integration. Don't forget to swap the bounds as well. That is, the bounds of $[-1, 3]$ need to be paired with dx. In fact, you can work through the problem a third time, swapping dx and dy but "forgetting" to swap the bounds, and you should get a final result of 104/3.

19.5 Approximating definite integrals

Numerical approximations of double integrals follow the same procedure as analytic solutions: start by calculating the definite integral over one variable while ignoring the other, then calculate the definite integral over the other variable. In numerical approximations, each variable is encoded as a dimension of the matrix of function values: Variable x corresponds to the second dimension (dimension "1") in a `numpy` array, and variable y corresponds to the first dimension (dimension "0").

19.5.1 Numerical approximations using `numpy`

We know the analytic solution to the definite integral in Equation 19.23, so it's useful to compare the numerical approximations

against the exact result.

The code below creates the function in `sympy` and specifies the integration bounds.

```
x,y = sym.symbols('x,y')
f = 2*x + y**2

# integration bounds
x_a,x_b = -1,3
y_a,y_b = 0,2
```

Next, calculate a `numpy` array (a 2D matrix) representing values of the function inside the bounds.

```
xx = np.linspace(x_a,x_b,1601)
yy = np.linspace(y_a,y_b,1600)
X,Y = np.meshgrid(xx,yy)
fxy = sym.lambdify((x,y),f)(X,Y)
```

Integrating along x involves summing across the rows:

```
intOverX = np.sum(fxy,axis=1) * (xx[1]-xx[0])
```

This is the numerical equivalent of calculating the definite integral of the inner integrand (Equation 19.24). The resulting variable `intOverX` is a vector, and is the numerical equivalent to Equation 19.27 ($8+4y^3$). Then we sum again over this vector, which is the numerical equivalent of evaluating Equation 19.28:

```
doubleInt = np.sum(intOverX) * (yy[1]-yy[0])
```

The result is `26.70336`, which is close to the exact value of $80/3 = 26.666$. The match between the numerical approximation and the exact result will increase if you increase the resolution (that is, decrease Δx and Δy).

When using `numpy`, it is unnecessary to sum separately per dimension; I wrote it that way to help you see the link between the

pure calculus and the numerical implementation. With no input to specify the axis along which to sum, `np.sum` adds all values in the matrix:

Note the scaling by Δx and Δy.

```
doubleInt = np.sum(fxy) * (xx[1]-xx[0]) * (yy[1]-yy[0])
```

19.5.2 Numerical approximations using `scipy`

Numerical approximations in `scipy` follow the same general procedure as I wrote for `numpy`, although the implementation is slightly different. I'll also complexify this example to have the bounds of integration be a subset of the function matrix. That is, Figure 19.8 shows the function evaluated to values beyond the bounds of integration, which are indicated by dashed lines.

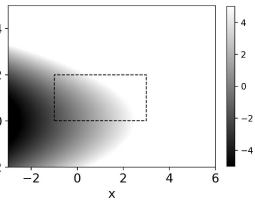


Figure 19.8: Definite integral in a region (dashed box) smaller than the evaluated function.

Start by evaluating the function beyond the bounds of integration.

```
xx = np.linspace(x_a-2,x_b+3,1601)
yy = np.linspace(y_a-2,y_b+3,1600)

dx = xx[1]-xx[0]
dy = yy[1]-yy[0]

X,Y = np.meshgrid(xx,yy)
fxy = sym.lambdify((x,y),f)(X,Y)
```

Then run Simpson's method on the submatrix created using indices for the integration bounds. Note that Simpson's method is applied twice: Once for x, then again for y.

```
# indices for integration bounds
x_idx = (xx >= x_a) & (xx <= x_b)
y_idx = (yy >= y_a) & (yy <= y_b)

F_x    = spi.simpson(fxy[:,x_idx], dx=dx, axis=1)
defint = spi.simpson(F_x[y_idx], dx=dy)
```

Unlike with `np.sum`, it is not possible to perform a double-integration on a numerical matrix in `scipy`.

Below is the comparison of results from `sympy`, `numpy`, and `scipy` (the `numpy` result is slightly different than in the previous subsection because the result below is from the expanded function matrix).

```
Sympy exact result : 26.66667
Numpy approximation: 26.64249
Scipy approximation: 26.53519
```

Normalize the results? Earlier in this chapter I wrote that the numerical approximations won't necessarily match the `sympy` analytic antiderivatives due to the indeterminacy of the constants of integration.

If normalization was necessary in those examples, why did the numerical approximation match `sympy`'s analytic solution in this example without normalization?

The answer is that the constants of integration are unnecessary for definite integrals, as you'll remember from Chapter 12. Notice also that definite integrals involve summing over *function* values, not over *antiderivative* values. Indeed, even if we had the numerical values for the antiderivative F_{xy}, we'd just subtract the antiderivative values at the two integration bounds, according to the FTC-2, in which case normalization would still be unnecessary as any offsets would impact both boundaries.

19.6 Integration with variable limits

Consider Figure 19.9: What is the definite integral of the function in the shaded region between the dashed and solid lines?

The answer to the question is "5/42" but the more im-

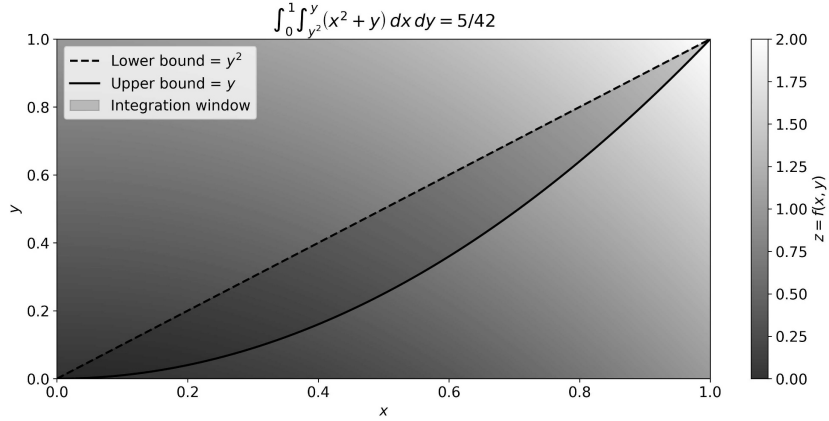


Figure 19.9: Using variables in the integration bounds allows you to integrate within non-square shapes.

I'm not referring to the area of that shaded region per se; that would be calculated as the integral of the single-variable function $\int_0^1 x - x^2\, dx$. Instead, the question here is about the definite integral of the two-variable function $f(x, y) = x^2 + y$ in the curved region bounded by $y = x$ and $y = x^2$.

The tricky part of problems like this is setting up and interpreting the bounds of integration. In all the examples you've seen so far, the bounds of integration were numbers like 0 or $-\pi$ or ∞.

Integrating with variable bounds is no different: Integrate the inner function to obtain a single-variable function, then integrate again to get a final result. I'm going to solve this problem below, but I encourage you to solve it first on your own.

I'll start with the inner integrand.

$$\int_0^1 \int_{y^2}^{y} \left(x^2 + y\right)\, dx\, dy \tag{19.32}$$

$$\int_{y^2}^{y} x^2 + y\, dx = \frac{x^3}{3} + xy \Bigg|_{y^2}^{y} \tag{19.33}$$

$$= \left(\frac{y^3}{3} + y^2\right) - \left(\frac{y^6}{3} + y^3\right) \tag{19.34}$$

$$= \frac{-2y^3}{3} + y^2 - \frac{y^6}{3} \tag{19.35}$$

The result is a single-variable function of y, which we can then integrate.

$$\int_0^1 \frac{-2y^3}{3} + y^2 - \frac{y^6}{3}\, dy = \left.\frac{-2y^4}{12} + \frac{y^3}{3} - \frac{y^7}{21}\right|_0^1 \tag{19.36}$$

$$= \left(\frac{-2}{12} + \frac{1}{3} - \frac{1}{21}\right) - 0 \tag{19.37}$$

$$= \frac{-7}{42} + \frac{14}{42} - \frac{2}{42} = \frac{5}{42} \tag{19.38}$$

The interpretation of this result is that the sum of all two-variable function values inside the curved bounded region is 5/42.

19.6.1 Numerical approximations

Numerical approximations of variable integrals can be done by "slicing" through the shaded region and summing the "sliced" integrals (Figure 19.10).

Imagine, for example, calculating the inner integrand for $y = 0$ (the lower bound), then calculating the inner integrand for $y = 0 + dy$, then again for $y = dy + dy$, etc, all the way up to $y = 1$ (the upper bound).

In a discretized function, this involves choosing a Δy to slice through (in the code below, $\Delta y = (y_b - y_a)/1000$), and initializing a vector to store the partial integrals along x for each value of y.

```
y_vals = np.linspace(y_a,y_b,1000)
```

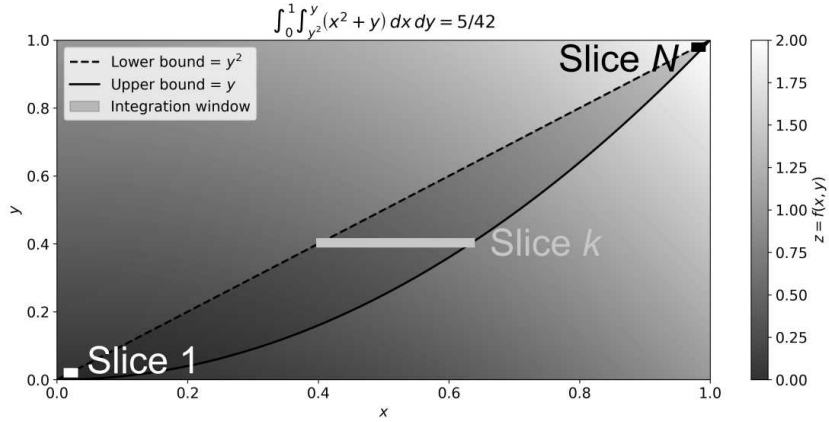


Figure 19.10: Illustrating the concept of calculating the definite integral of a two-variable function by segmenting the y-axis range of the integration window into N slices, and numerically approximating the integral in each slice. The sum of all slice-integrals, scaled by Δy, is the numerical approximation of the double integral.

```
slice_integrals = np.zeros(len(y_vals))
```

Thus, the outer integrand interval has been divided into 1000 segments. Then we loop over each of the y-axis segments, calculate the definite integral for each y-axis slice, and store that integral. The variables `x_a` and `x_b` are `sympy` expressions for the lower and upper bounds (y^2 and y).

y^2 is the lower bound because $y^2<y$ for $0<y<1$.

```
for idx,yi in enumerate(y_vals):

  # x bounds for this y
  lo_bnd = float(x_a.subs(y,yi))
  hi_bnd = float(x_b.subs(y,yi))
  x_vals = np.linspace(lo_bnd,hi_bnd,100)

  # function values for these coords
  f_vals = fxy_lam(x_vals,yi)

  # get the "mini-integral" of this slice
  slice_integrals[idx] = spi.simpson(f_vals,x=x_vals)
```

You can see that the bounds of integration change at each iteration of the for-loop, depending on the value of y. That's the implementation of slicing through the inner integrand for each dy of the outer integrand. For this reason, Δx also changes for each value of y. You will explore the impact of this varying Δx in Exercise 3.

The result of this for-loop is an array of definite integrals, one for each slice of y (Figure 19.11). That's the equivalent of analytically calculating the inner integrand and having a function of y (although now it's a numerical vector not a function). Therefore, the final step is to integrate along y:

```
integral_result = spi.simpson(slice_integrals,x=y_vals)
```

In this example, the approximation is quite accurate:

```
Sympy exact result : 0.1190476190
Scipy approximation: 0.1190476190
```

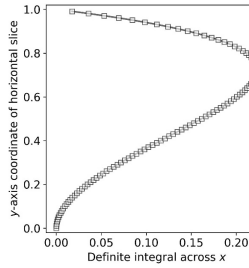


Figure 19.11: The single-variable definite integral for each horizontal slice through the two-variable function. The sum of all slice-integrals, scaled by Δy, is the numerical approximation of the definite double integral.

Approximations in `numpy` An alternative method using `numpy` is to sum all two-variable function elements that are inside the bounded region. This is the same concept as summing all function values inside the rectangle defined by numerical boundaries.

This is implemented by creating a "mask" that indicates which pixels belong in the bounded region.

```
mask = (Y>=X**2) & (Y<=X)
```

The `mask` is a matrix of the same size as the two-variable function, and has a value of `True` for matrix elements inside the integration window and a value of `False` for matrix elements outside the window (Figure 19.12).

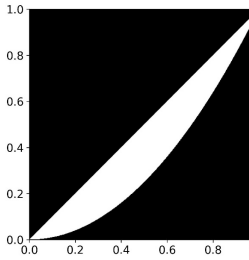


Figure 19.12: White pixels are inside the mask and included in the summation; black pixels are outside the mask.

Then we sum (and scale by $\Delta x \Delta y$) all function values that the mask indicates are inside the bounds.

```
integral_np = np.sum(Z[mask]) * dx*dy
```

In this example, the `numpy` result was less accurate that the `scipy`

result:

```
Sympy exact result : 0.1190476190
Scipy approximation: 0.1190476190
Numpy approximation: 0.1174881049
```

19.7 Exercises

19.1. You know from earlier chapters that constants can be pulled out of the integral. That is, $\int \alpha f(x)\,dx = \alpha \int f(x)\,dx$ for some constant α.

In multivariable integrals, constants can be pulled out of all the integrals. That is:

$$\iint \alpha f(x,y)\,dx\,dy \;=\; \int \alpha \int f(x,y)\,dx\,dy \;=\; \alpha \iint f(x,y)\,dx\,dy$$

The purpose of this exercise is to demonstrate this property using `sympy` and `scipy`. This is also a fantastic way to test and improve your math-coding skills.

There are six parts to this exercise. First, implement Equation 19.39 in `sympy`.

$$f(x,y) = e^{\pi}x^2 + x\cos(y) - ye^{\pi}\ln(x^4) \tag{19.39}$$

Second, factor out the constant e^{π} to have an expression of the form $e^{\pi}f(x,y)$. Print out the first two parts using latex and `sympy` as in Figure 19.13.

$$f(x,y) = x^2e^{\pi} + x\cos(y) - ye^{\pi}\log($$
$$= e^{\pi}\left(x^2 + \frac{x\cos(y)}{e^{\pi}} - y\log($$

Figure 19.13: Screenshot of `sympy` output for parts 1 and 2 of Exercise 1.

Third, calculate three indefinite integrals in `sympy` to show the following equalities, and print the results as in Figure 19.14.

$$\iint e^{\pi}f(x,y)\,dx\,dy \;=\; \int e^{\pi}\int f(x,y)\,dx\,dy \;=\; e^{\pi}\iint f(x,y)\,dx\,dy \tag{19.40}$$

$$\begin{aligned}
\iint \left(x^2e^{\pi} + x\cos(y) - ye^{\pi}\log\left(x^4\right)\right)\,dx\,dy &= \frac{x^3ye^{\pi}}{3} + \frac{x^2\sin(y)}{2} - \frac{xy^2e^{\pi}\log\left(x^4\right)}{2} + 2xy^2e^{\pi} \\
\int e^{\pi}\int \left(x^2 + \frac{x\cos(y)}{e^{\pi}} - y\log\left(x^4\right)\right)\,dx\,dy &= \left(\frac{x^3y}{3} + \frac{x^2\sin(y)}{2e^{\pi}} - \frac{xy^2\log\left(x^4\right)}{2} + 2xy^2\right)e^{\pi} \\
e^{\pi}\iint \left(x^2 + \frac{x\cos(y)}{e^{\pi}} - y\log\left(x^4\right)\right)\,dx\,dy &= \left(\frac{x^3y}{3} + \frac{x^2\sin(y)}{2e^{\pi}} - \frac{xy^2\log\left(x^4\right)}{2} + 2xy^2\right)e^{\pi}
\end{aligned}$$

Figure 19.14: Screenshot of `sympy` output for part 3 of Exercise 1. This `sympy` result confirms that the indefinite integrals are the same for the three configurations of constants.

Fourth, calculate the following three definite integrals in `sympy` and confirm that you get the same answer (Figure 19.15).

$$\int_{-\pi}^{\pi/4} \int_{1}^{3} e^{\pi} f(x,y)\, dx\, dy \tag{19.41}$$

$$\int_{-\pi}^{\pi/4} e^{\pi} \int_{1}^{3} f(x,y)\, dx\, dy \tag{19.42}$$

$$e^{\pi} \int_{-\pi}^{\pi/4} \int_{1}^{3} f(x,y)\, dx\, dy \tag{19.43}$$

$$\int_{-\pi}^{\frac{\pi}{4}} \int_{1}^{3} \left(x^2 e^{\pi} + x\cos\left(y\right) - y e^{\pi} \log\left(x^4\right)\right)\, dx\, dy = 1345.31350049678$$

$$\int_{-\pi}^{\frac{\pi}{4}} e^{\pi} \int_{1}^{3} \left(x^2 + \frac{x\cos\left(y\right)}{e^{\pi}} - y\log\left(x^4\right)\right)\, dx\, dy = 1345.31350049678$$

$$e^{\pi} \int_{-\pi}^{\frac{\pi}{4}} \int_{1}^{3} \left(x^2 + \frac{x\cos\left(y\right)}{e^{\pi}} - y\log\left(x^4\right)\right)\, dx\, dy = 1345.31350049678$$

Figure 19.15: Screenshot of `sympy` output for part 4 of Exercise 1.

The numerical answer is the floating-point evaluation of the analytic answer. The exact answer is

$$-\frac{15\pi^2 e^{\pi}}{4} + 2\sqrt{2} + \frac{65\pi e^{\pi}}{6} + \frac{45\pi^2 e^{\pi}\log\left(81\right)}{32}$$

Fifth, implement a numerical approximation using the `dblquad` function available in the `scipy integrate` library, and print the results as in figure 19.16. I haven't introduced this function, but the usage is nearly identical to using the `quad` function for single-variable functions (e.g., Section 14.8).

$$\int_{-\pi}^{\frac{\pi}{4}} \int_{1}^{3} \left(x^2 e^{\pi} + x\cos(y) - y e^{\pi} \log\left(x^4\right)\right) dx\, dy = 1345.31350049678$$

$$e^{\pi} \int_{-\pi}^{\frac{\pi}{4}} \int_{1}^{3} \left(x^2 + \frac{x\cos(y)}{e^{\pi}} - y\log\left(x^4\right)\right) dx\, dy = 1345.31350049678$$

Figure 19.16: Screenshot of `scipy` output for part 5 of Exercise 1.

Finally, write code to visualize this function and a rectangle to indicate the area of integration as in Figure 19.17.

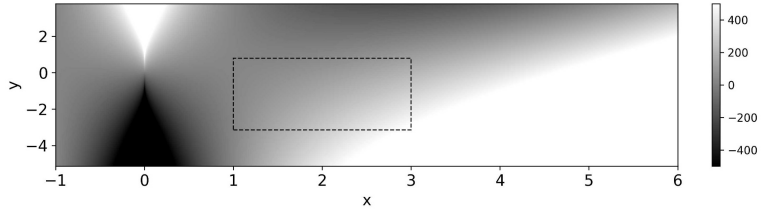


Figure 19.17: Results of part 6 of Exercise 1.

19.2. The goal of this exercise is to reproduce Figure 19.18. You can start by copying the code that created Figure 19.9, although it will require several modifications. `sympy` cannot calculate an analytic definite integral, so you'll need to approximate it using `spi.simpson`.

The approximate definite integral shown in the figure title is the total area, not the net area (notice that the bounding functions cross).

In case the text in the figure title isn't clear enough, the integral to calculate is:

$$\int_{0}^{\pi/4} \int_{y^3}^{\cos(2\pi y)} \left(e^{-x^2-y^2}\right) dx\, dy \tag{19.44}$$

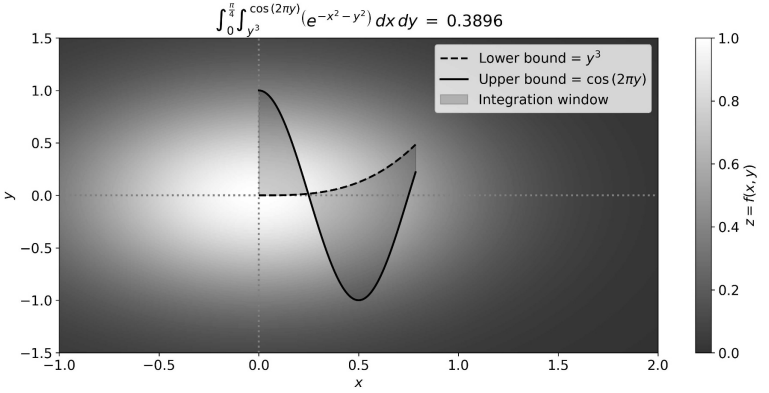


Figure 19.18: Results of Exercise 2.

19.3. In the first example of integration with variable bounds (Figure 19.9), I calculated $f(x, y)$ using 100 x-axis points between the two bounds defined by each value of y. This means that Δx was different for each horizontal slice through the integration window.

The first goal of this exercise is to copy, paste, and modify the code that creates Figure 19.9 to obtain the value of Δx for each value of y. Those results are shown in Figure 19.19A.

Second, modify the code again to fix the value of $\Delta x = .001$ and calculate the number of x-axis points in each y-axis slice. Plot those results as in panel B.

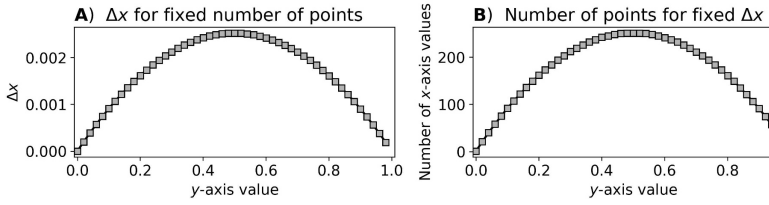


Figure 19.19: First and second sets of results of Exercise 3. Results were downsampled by a factor of 20 to increase visibility of the squares.

Why do the two sets of results show an inverted-U shape? This is due to the nature of the bounding function. Notice in Figure 19.9 that the integration window is narrow at

the two edge bounds of y (close to 0 and close to 1) while being widest around middle values of y. In other words, 100 x-axis values are really close together when y is close to 0 and 1, while being more spread out other when y is closer to .5. Similar argument for the number of spanning x-axis points when y is closer to 0 or 1, vs. when y is closer to .5.

Does this have implications for the accuracy of the numerical approximation? The third goal of this exercise is to evaluate the accuracy, measured as the discrepancy relative to the exact definite integral value from `sympy`, for various values of Δx.

In a for-loop over a range of Δx values between .0001 and .01, use `scipy` to approximate the definite integral with a fixed Δx (thus, the number of x-axis points will vary) in each iteration of the for-loop. Show the results as in Figure 19.20.

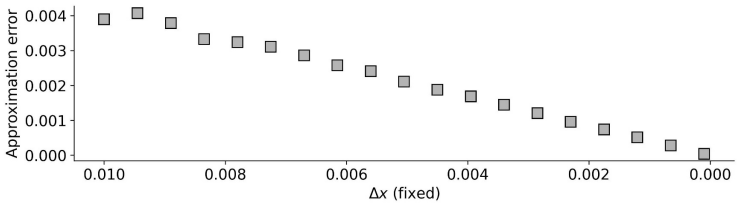


Figure 19.20: Third set of results of Exercise 3.

The key take-home from this final part of the exercise is that Δx does have implications for the accuracy of the approximation of a definite integral, but in a way that is expected for any function and any shape of integration window: Smaller Δx leads to lower approximation errors.

In pure mathematics, these concerns do not arise: dx represents the same infinitesimally small value regardless of the size or shape of the integration window. However, when implementing mathematics on digital computers, choices must be made about how to set up and evaluate functions. These choices impact the accuracy of approximations and,

in scientific or engineering applications, such inaccuracies can have significant implications.

Index

dx, 174, 180, 206, 415
x-axis resolution, 61, 247, 414, 432, 455, 461, 507, 537, 644, 676
`numpy` vs. `sympy`, 57
`sympy` plotting, 117
`sympy` substitution, 58

Absolute value function, 91, 198, 223
AI-assisted writing, 24, 25
Algebra, 16
Antiderivative, 415, 419
  Approximate, 537
  In `scipy`, 503
  Multivariable, 652
Arc length, *see* Curve length
Area between curves, 564
Audience, 15

Babylon, 62
Bijective, 50, 65
  Function inverse, 104
Book code, 19, 61
Bread, 589

Chain rule, 264, 319, 544
  In quotient rule, 269
  Intuition, 295
  Multiple functions, 266
ChatGPT, 25
Circle, 61, 400
Clairaut's theorem, 369
Coding, 16
  Exercise solutions, 18
  IDE, 19
  Other languages, 22
  Prerequisites, 17
  Sharing and reposting, 22
Colab, 19
Commutativity, 97, 126
Composite functions, 96, 124
  Commutativity, 97
  Differentiating, 264
  In coding, 100
Computational sciences, 16, 615
Constant of integration, 470, 472, 499
  As a function, 654
  Geometric intuition, 474
  In statistics, 625
  Multivariable, 653, 660
  Resolving, 476
Continuity, 92, 190, 407
  Definition, 94, 145
  Differentiability, 302
  Rolle's theorem, 309
Convergence (integration), 516, 532
Critical points
  Corner, 220
  Definition, 218
  Endpoints, 224
  Evaluating via derivative test, 234
  Finding, 225
  Gradient descent, 231
  Inflection, 220
  Maximum, 219
  Minimum, 220
  Via grid search, 228
Critical value, 219, 222
Cube roots, 249, 250
Cumulative distribution function, 623
  Relation to pdf, 623, 641
Cumulative sum, 413, 423, 429, 659
Curve length, 573, 609
  In `numpy`, 578
  In `scipy`, 576, 581
  In `sympy`, 576

Data science, 16
Data type, 64
Definite integral, 419, 424
  Area between functions, 564
  Area vs. volume, 651
  As area, 647
  Calculating, 469
  In `numpy`, 664, 671
  In `scipy`, 500, 540, 600, 666
  In `sympy`, 448, 449
  Multivariable, 663
  Net area, 444, 456
  Total area, 444, 456
Derivative
  Constant, 261
  Continuity, 302
  Cosine and sine, 195
  Critical points, 222
  Definition, 173
  Gradient, 372
  Higher-order, 203, 368
  In `sympy`, 210, 316
  Interpreting, 185
  Intuition, 70
  Linear properties, 187, 188
  Notation, 179, 203, 363, 368
  Numerical approximation, 206, 387
  Of a constant, 178
  Of linear functions, 175
  Partial, 363
  Polynomials, 191
  Visualizing, 183
  vs. integral, 314
Derivative vs. integral, 428, 467
Determinant, 395
Differentiation, *see* Derivative
Discontinuities, 527
  Critical points, 223
  Infinite, 529
  Jump, 527
  Removable, 530
Discontinuity, 92, 122, 512
  and intermediate value theorem, 305
  Infinite, 95, 123, 506
  Jump, 95
  Ocillating, 165
  Oscillating, 96, 124
  Removable, 95, 122
Disk method, 588
Distribution, 615
  Characterizing, 636
  Empirical, 635
  F, 621
  From data (empirical), 616
  From equation (analytic), 616
  In `sympy`, 621
  Laplace, 626, 635
  Normal (Gaussian), 618, 645
  Pareto, 622, 636, 646
  Triangular, 626, 642
  Visualizing, 616
Divergence
  Integration, 517, 537
  Oscillatory, 524
Domain, 42, 450
  In `sympy`, 123, 166
  Notation, 46
  Restricted, 45
  Subdomain, 45
Double integral, 655

Exercises
  About, 18
  Solutions, 18
  Video explanations, 19

Factoring in `sympy`, 559
First derivative test, 234
Float vs. int, 64
Folium of Descartes, 607
Fractal, 408, 607
Function
  Definition, 29
  Exact vs. approximate outputs, 33
  Grid resolution, 61
  Inputs and outputs, 30
  Math vs. coding, 37
  Multivariable, 353, 384
  Naming conventions, 28, 40
  Non-function expressions, 34
  Non-numeric, 41
  Relation vs. output, 38
Function inverse
  Calculating, 127
Function symmetry, 109, 127
  And `sympy`, 113

And integration, 495
Determining, 111
Even, 110, 620
Odd, 110
Fundamental Theorem of Calculus, 313
Part 1, 422, 429, 468, 476
Part 2, 424, 470, 600, 632, 667

Gaussian function, 566, 618, 641
Github, 19
Google Colab, 19
Gradient, 372
Direction vs. magnitude, 390
Fields, 389
Vector fields, 373
Gradient descent, 231, 253, 391
In 2D, 381
Grid search, 228, 280, 297, 318
Accuracy, 231
Gym analogy, 56

Heaviside function, 142
Hessian, 394
Hessian matrix, 379
Higher-order derivatives, 203, 379
In `sympy`, 371
Hilbert matrix, 354
Histogram, 605, 616
Horizontal line test, 35

Images in Python, 357, 383
Implicit differentiation, 269, 270
In `numpy`, 279, 296
In `sympy`, 277, 295
Numerical approximations, 279
Implicit functions, 270
Visualing in `sympy`, 283
Visualizing in `matplotlib`, 281
Improper integral, 512
Discontinuities, 527
One ∞ bound, 513, 533
Trig functions, 523
Two ∞ bounds, 519
Indefinite integral, *see* Antiderivative
Infinite integral bound, *see* Improper integral
Infinitesimal, 174
Inflection points, 240, 248, 254
Initial value problem, 476, 501, 660
Injective, 50, 65
Integration
Addition, 480, 502
As area, 416, 431
Bounds, 420, 426, 566, 667
Bounds from intersections, 569
By summing, 414
Definite vs. indefinite, 419, 466
Disk method, 590
Improper, 512
In `scipy`, 460, 496, 659
In `sympy`, 454, 499, 505, 532, 657
Intuition, 412, 414, 418
Linearity, 478, 502, 565
Multivariable, 652
Notation, 421, 470
Other rules, 488
Power rule, 466
Scalar multiplication, 479, 483, 502, 673
Symmetry, 495
Transcendentals, 484
Volume, 584
vs. differentiation, 428
Washer method, 593, 594
Integration by parts, 548, 550, 558
Intercept, 171, 176
Intermediate value theorem, 305
Intuition for functions, 53
Inverse functions, 101
Calculating, 103
Commutativity, 102
Conditions, 104
Domain and range, 104
Notation, 102
IQ distribution, 631
Irregular x-axis spacing, 508

Jacobian matrix, 378

L'Hôpital's rule, 286
Conditions, 290
In `sympy`, 298
To compare functions, 299
Lambda expressions, 65
Lambdify, 59

Lebesgue integral, 449, 461
Limits
Analogy, 132
At infinity, 135
Derivative, 174
Factoring, 139, 151
Finding squeezing functions, 152
Geometric intuition, 133
Improper integral, 513
In `sympy`, 159, 161, 361
Indeterminant, 157
Multi-sided, 361
Multivariable functions, 358
Notation, 136
One-sided, 137, 141
Trig functions, 145
Trigonometry, 153
Two-sided, 136
Limits properties, 161, 163
Additive, 143
Divisive, 144
Factorable, 143
Multiplicative, 144, 166, 262
Qualification, 144
Linear approximation, 324, 345
Accuracy, 327
Linear function, 70, 115, 175, 326
List comprehension, 60
Logistic, 628

Mandelbrot, 408
Math enthusiasts, 16
Matrix, 378, 395
In `sympy`, 395
Mean value theorem, 310
Conditions, 312
Estimating solutions, 318
In `sympy`, 316
Meshgrid, 355, 385, 587
Moments (statistics), 636
Analytic, 637
Empirical, 640, 644
In `scipy`, 639
Standardizing, 638
Monotonic function, 459
Morlet wavelet, 525, 534
Motivation, 16
Multivariable integration, 652
Multivariate, 352

Nabla (del) (∇), 373
Natural exponential, 80, 118
Derivative, 201, 284
Integral, 484
Natural log, 82, 468
Bases, 84
Derivative, 200, 283
Integral, 485, 602
Net area, 444, 470, 526, 600
Newton's method, 331, 346
Algorithm, 332
Non-differentiable, 407
Nonlinear function, 70
Normal line, 399
Normalization, 538, 657, 660
Notation
Cumulative sum, 413
Derivative, 180, 203
Domain and range, 46
Integration, 421
Math functions, 53
Multivariable integration, 650
Number fields, 41
Numerical approximations, 429, 447, 496, 507, 580, 662

Online code, 19, 61
Online courses, 23
Optimization, 334
Critical points, 245
Differentiation, 336, 348
Gradient descent, 231, 381
Grid search, 228
Surface area, 340
Volume, 342
Outliers, 609

P-value, 633, 643
Definite integral, 633
Parametric curve, 402, 406, 558, 570, 604
Parametric function, 570, 608
Partial derivatives, 363
Gradient fields, 375
In `sympy`, 386
Interpreting, 365
Mixed, 368
Symmetry, 369, 370

Visualizing, 364
Partial fractions, 553, 555, 559
Partial integral, 650
In `scipy`, 659
In `sympy`, 657
Normalizing, 660
Visualizing, 656
Piecewise function, 90, 172
Critical points, 239
Visualizing, 92, 119
Planar curve, 576
Polar coordinates, 608
Polynomials, 73, 115, 191
Arithmetic, 77
Critical and inflection points, 240
Roots, 249, 250, 306, 331
To approximate sine, 116
Visual patterns, 75
Power rule, 191, 193, 466
Proof with implicit differentiation, 285
Prerequisites, 16
Probability, 617
Probability density function, 617
Definition, 618
Empirical, 645
In `numpy`, 641
In `scipy`, 641
Product rule, 258, 549
Intuition, 261, 293
Proof, 262
Programming, *see* Coding
Python, *see* Coding

Quotient rule, 266
In `sympy`, 294
Proof, 268

Radial curve, 404
Range, 43, 450
Notation, 46
Rational functions, 78
Relation, 30
ReLU function, 91
Reversing integration bounds, 426, 566
Revolution, 585
Riemann, 407
Riemann sum, 436, 480
Curve length, 574
Definition, 437
Infimum/supremum rule, 440
Integral, 442, 460
Left/right/midpnt rule, 440, 454
Partitions, 453, 455
Resolution (Δx), 438, 605, 607, 676
Rules, 441
Trapezoid and Simpson rule, 440, 456
Riemann vs. Lebesgue integral, 451, 462
Rolle's theorem, 306, 319
Root mean square, 208
Rose curves, 406

Sandwich theorem, *see* Squeeze theorem
Secant line, 182, 183, 212
Second derivative test, 243
Sigmoid function, 242
Signum function, 198, 223
Sinc function, 539
Slope, 71, 170, 205, 575
Multivariable, 377
Smile, 601
Solids of revolution, 585
Space curve, 576
Splitting integrals, 445, 495, 519
Square root algorithm, 62
Squeeze theorem, 149, 154
Sum of squared errors, 247
Surface plots, 357, 385
Surjective, 50, 65
Symbolic numbers, 118
Symbolic variables, 57
Symmetry, *see* Function symmetry

Tangent line, 182, 212, 399
Total area, 444, 446, 449, 675
Transcendental functions, 80
Trigonometric functions, 85, 571
As angles, 86
As ratios, 85
As time series, 87
Cosine, 86
Derivatives, 195, 213, 287
Identities, 88

Integrals, 487
Integration in `sympy`, 539
Inverses, 107
Limits, 146, 153
Sine, 86
Squeeze theorem, 152
Tangent, 88
Two-variable functions, 353
Visualizing, 355, 356

U-substitution, 544, 546

Variable, 31
As x-axis grid, 31
Dependent, 32
Independent, 32
Variable limits of integration, 667
Vertical line test, 35
Volume, 584
In `scipy`, 597, 598
In `sympy`, 591
Visualizing, 587

Washer method, 593

YouTube videos, 19

Z-test, 629, 643
Zeno's paradox, 132, 158